Evolutor

Genomic computation: models, state and systems

Define the mechanism. Derive the result. Test the claim.

From sequence models and biological regulation to native-state and attention engines, an effect-aware compiler and runtime, and the evidence needed to tell a mechanism from a slogan.

PREPUBLICATION REVIEW EDITION

Chapters 1–52 in twelve parts, with Epilogue and Appendices / October 2026

Edition notice. This is a prepublication review edition assembled from the book's authored LaTeX sources. It has had author-side technical, editorial and visual review. It has not had independent specialist review, a reader study or accessibility remediation, and it is not an accepted publication. The PDF is not tagged and makes no PDF/UA claim.

What the book claims. Mathematical statements hold under their stated premises. Executable examples are teaching references, tested on the declared cases. Synthetic traces, numerical emulations and assigned-number examples are labelled as such. Nothing in this edition reports a trained genomic foundation model, a production service, a measured hardware speedup, a biological discovery or a demonstration of general intelligence.

Reproduction. The volume is rebuilt with `python3 scripts/build-convergence.py --through 52`; the chapter references and tests live beside each chapter's source in the companion repository.

Rights. No general reuse licence is granted by this edition, and public availability of the repository does not supply one. Cited works remain the property of their authors and publishers. A rights and source audit of text and artwork is an open publication gate.

Preface

A genome is a stored description that no single cell reads in full. Different cells carrying almost the same DNA express different parts of it, at different times and in response to different histories. Populations change the description itself over generations. Computing has its own versions of these separations: a program and its running state, a model's weights and its activations, a catalogue of modules and the plan selected for one request. This book asks whether making those separations explicit and typed, the way genomes and cells appear to separate them, can give computational organizations that are worth building. The question is *whether*. The book does not start from the assumption that they are.

Three meanings of genomic computation

The phrase "genomic computation" is used for three different subjects, and much confusion comes from sliding between them.

Computation about DNA. Models that read recorded DNA sequence and predict something about it, such as genomic language models, variant scorers and assay predictors. Here DNA is *data*.

Computation with DNA. Molecules as the physical substrate of computation: strand-displacement circuits, molecular neural networks, algorithmic self-assembly and DNA data storage. Here DNA is the *machine*. The companion volume *DNA Computing: Molecular mechanisms, algorithms and evidence* develops this subject from first principles [126].

Computation inspired by DNA. Digital architectures that borrow the *organization* of genomic systems: stored versus expressed structure, regulated selection, strand symmetry, development and inheritance. Here DNA is a *design hypothesis*.

Evolutor works in the first and third senses and keeps both apart from the second. A tokenized DNA string inside a neural network is not a molecule, and a software gate is not a promoter. The Epilogue returns to all three senses and asks where they might productively meet.

Three research directions with fixed names

Throughout the book, **DOGMA** names a proposed non-Transformer, DNA-native research direction together with its state-native engine; **Hermon DNA** names a Transformer-based DNA research direction together with its attention-native engine; and **Evolutor** names the broader theory, compiler, runtime, memory and governance layer above both. The names are commitments about *architecture*, not claims of success. Each direction is reduced to a small executable reference model whose behaviour can be checked exactly. The book then asks what experiment would show that the reference is worth scaling, and what result would show that it is not.

What you will build

Part II builds the machinery of modern sequence learning from first principles: objectives and legal information access, hand-derived gradients, reproducible training, tokenization, recurrence, state-space scans, attention, the Transformer block and conditional routing. Part III maps real regulatory mechanisms to typed software abstractions and stops each mapping where the biology ends. Parts IV and V specify DOGMA and Hermon DNA. Part VI fixes how unlike architectures can be compared without false equivalence. Parts VII and VIII turn both references into serving engines (state pools, checkpoints, batching, paged KV caches, continuous batching and speculative decoding) while preserving exactly what each model computes. Part IX builds Evolutor's runtime, memory and compiler above both engines. Part X covers deployment, isolation, distributed placement and measurement. Part XI turns application and capability claims, including claims about general intelligence, into experimental designs. Part XII collects failures, open problems and the evidence record.

Almost every mechanism is implemented in a small reference program that you can run, and almost every implementation is checked against an independent oracle. Wherever a plausible bug exists, a deliberately broken negative control shows that the test can actually catch it.

An evidence discipline

The book separates kinds of evidence that are easy to blur: a theorem under stated premises; a finite test of an implementation; a synthetic trace; a numerical emulation; a task-trained result on held-out data; a measurement on real hardware; a biological observation; and a hypothesis. Appendix C summarizes what each can and cannot establish. The same discipline applies to the book's own results. Several small trained pilots in Parts IV and V stayed at or near chance under their declared budgets, and they are reported as negative results rather than hidden. A complete manuscript, a passing test suite or an elegant architecture diagram is not evidence of genomic usefulness, production readiness or general intelligence. These distinctions do not dampen ambition. They are what will make any eventual positive result credible.

Who this book is for

The book is written for advanced undergraduates, graduate students, research engineers and scientists who want to build sequence-model systems rather than only use them. It assumes programming in Python, linear algebra, multivariable calculus and basic probability. Prior deep-learning experience helps but is not required: Part II derives what later chapters need. No laboratory biology is assumed. Appendix A gives the molecular biology, coordinate conventions and data formats used in the genomic chapters. The book contains no wet-laboratory procedures and no clinical interpretation of variants.

Acknowledgments

The general implementation-first progression of predicting, deriving, implementing, testing and then examining consequences was informed by the author companion overview and chapter map for Sebastian Raschka's *Build a Large Language Model* (*From Scratch*) [127]. The companion overview was inspected, not the full commercial text. The prose, reference implementations, numerical examples, exercises and illustrations of this book are its own unless explicitly attributed. Primary sources are cited where their ideas are used, and each chapter's source notes record how deeply each source was read.

How to use this book

The shape of the book

The 52 chapters are grouped into twelve parts. The parts form a dependency structure rather than a single line. The foundations in Part II feed both the biological abstractions of Part III and the two model directions, DOGMA (Part IV) and Hermon DNA (Part V). Part VI's experimental contract governs every later comparison. The two engines (Parts VII and VIII) are developed separately and meet only in Evolutor's runtime and compiler (Part IX). Industrial engineering, applications and synthesis then build on that layer. Figure 1 shows the structure and five suggested reading routes.

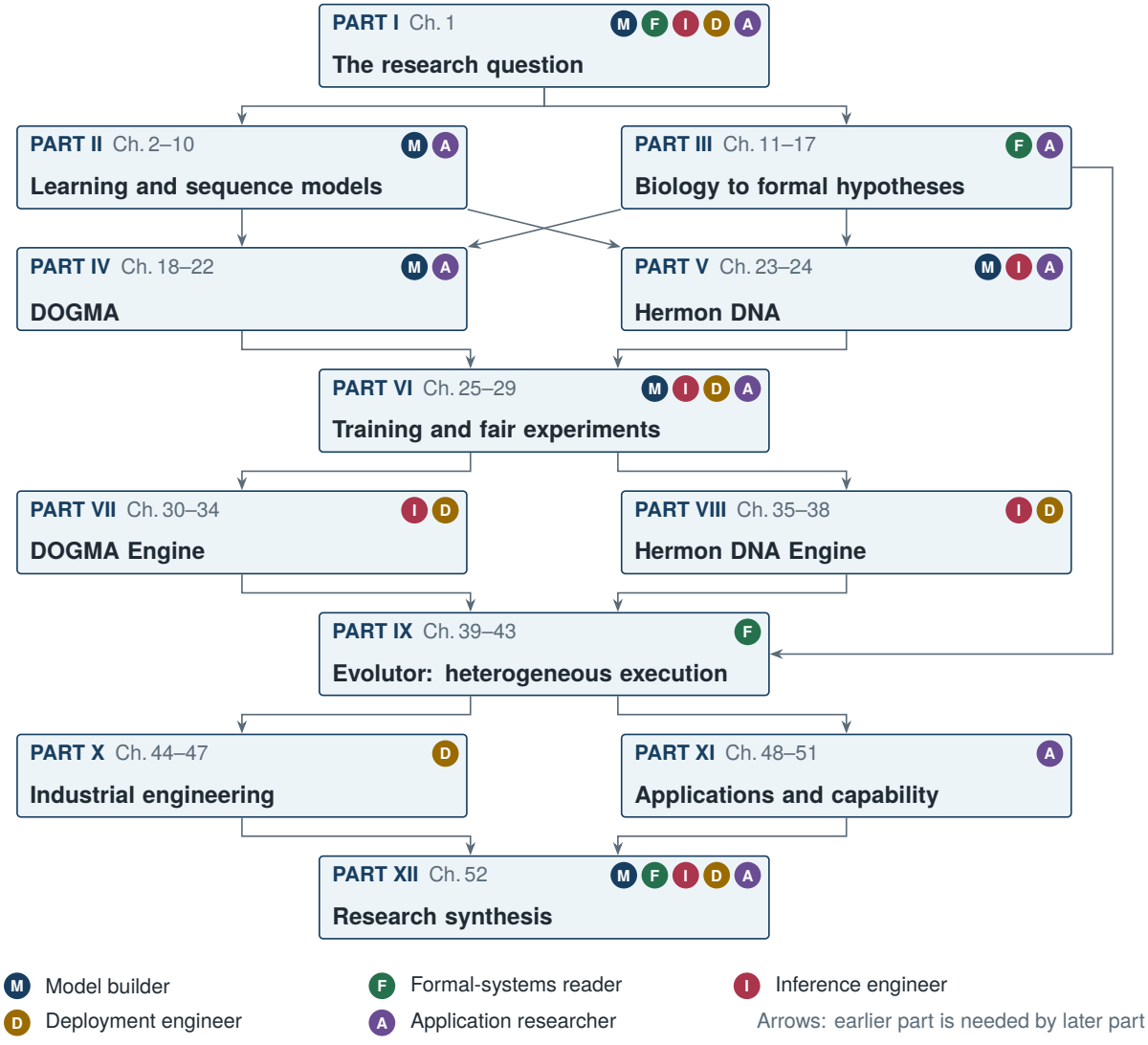


Figure 1: The twelve parts and their main dependencies. Solid arrows mean "is needed by". The coloured markers show five reading routes. Each route lists the parts it reads in order and can skip the others on a first pass. Chapter-level prerequisites are printed on every part page.

Route	Chapters	What it emphasizes
Model builder	2–10, then 18–29	Objectives and state semantics before comparing architectures
Formal-systems reader	11–17, then 39–43	Separating biological analogy from typed semantics and legal lowering
Inference engineer	23, 25–38	Keeping scalar and full-model oracles while storage and scheduling change
Deployment engineer	28–30, 32–38, 44–47	Contracts that a reference test does not establish
Application researcher	2, 5, 17, 21, 24–29, 48–52	Datasets, interventions and negative evidence

Chapter 1 is common to every route, and Chapter 52 together with the Epilogue closes each one.

The anatomy of a chapter

Every chapter follows the same progression, used with judgement rather than as a template:

1. **A concrete problem and a prediction.** Each chapter opens with a small case whose answer you can guess before reading on, such as "must the resumed run take the same next step?" or "which tokens change when one base changes?".
2. **A derivation.** The mechanism is derived with every symbol, shape and unit defined, and its premises stated next to the result.
3. **An implementation.** The printed listings are extracted from the tested reference program at build time, so the code you read is the code that runs.
4. **A test or counterexample.** Each result is checked by an independent oracle, by a deliberately broken negative control, or by a counterexample that shows a missing premise matters.
5. **The consequence.** The chapter says what the result licenses, and what it does not.

A box headed *In this chapter you will* opens each chapter with its goals, and a *Key ideas* box before the exercises states what should stay with you. Exercises come with worked solutions or marking rubrics. A closing *Sources and evidence* note records which primary sources were consulted and how deeply each was read.

Conventions

- Arrays and token positions are zero-based. Genomic intervals are half-open, $[a, b)$, unless a file format states otherwise (Appendix A).
- Tensor shapes are written in square brackets, for example $[B, T, D]$ for batch, time and width. Row vectors are used when the text follows PyTorch storage conventions.
- log and ln denote natural logarithms, and losses are in nats unless bits are stated. MiB and KiB are powers of 1024; MB and GB are powers of 1000.
- DNA strands are written 5′ to 3′. The reverse complement $R(x)$ reverses the order and complements each base; complementing alone is a different operation.

- "Tick" means a logical step of a reference program, not elapsed time. Every timing, throughput or memory figure states whether it was measured, modelled or assigned.

Appendix B collects the notation, and Appendix C collects the evidence classes and the claim-ledger template used throughout.

Running the companion code

Each chapter from 4 onward has a reference program, generated tables and a test file. With the pinned Python environment installed, the commands below rebuild the whole volume and run every test. Rebuild the chapter assets before running the tests on a fresh checkout, because several tests compare printed tables and listings with regenerated ones.

```
python3 scripts/check-chapter-structure.py
python3 scripts/build-convergence.py --through 52 --refresh-assets
python3 -m pytest -q -o addopts=''
```

A single chapter can be rebuilt and checked with `python3 drafts/chNN/build.py --check`. All examples run on a CPU. None needs a GPU, a network service or private data.

Contents at a glance

Part I The research question 1

1 Why Genomic Computation? ... 2

Part II Learning and sequence-model foundations 17

2 Learning objectives, data, tasks, and evaluation ... 18
3 Differentiation, optimization and tensor programs ... 36
4 Reproducible PyTorch training ... 57
5 Tokenization and sequence representation ... 71
6 Recurrent models and gated state ... 82
7 State-space models, selective updates and scans ... 94
8 Attention and content addressing ... 105
9 Assembling a Transformer block ... 116
10 Conditional computation and routing ... 129

Part III From biology to formal computational hypotheses 141

11 Regulation and expression across levels ... 142
12 Development and generated computational structure ... 151
13 Learning, structural adaptation and evolution ... 162
14 Typed genomic computation systems ... 172
15 Operational semantics and expression traces ... 181
16 Expression complexity and resource semantics ... 190
17 Traces, credit and mechanistic evidence ... 199

Part IV DOGMA: non-Transformer DNA-native computation 208

18 DOGMA primitives and state semantics ... 209
19 DOGMA regulation and selective transformations ... 218
20 Structured memory, locality and timescales ... 228
21 Strands, complements and dual-state hypotheses ... 240
22 DOGMA reference model and falsifiable research program ... 251

Part V Hermon DNA: Transformer-based DNA computation 264

23 Hermon DNA reference architecture ... 265
24 DNA-aware Transformer mechanisms ... 278

Part VI Training systems and fair experiments 291

25 Shared experiments without false equivalence ... 292
26 Training/inference parity and parallel recurrence ... 307
27 Optimization and memory-efficient training ... 319
28 Distributed training and reproducible model artifacts ... 330

29 Benchmark design and comparative evidence 342

Part VII DOGMA Engine: state-native inference 353

30 Runtime contracts and request lifecycles 354
31 DOGMA state construction and execution 366
32 DOGMA state pools and memory ownership 377
33 DOGMA checkpoints, branching and prefix state 388
34 DOGMA batching and state-native kernels 401

Part VIII Hermon DNA Engine: attention-native inference 414

35 Hermon DNA prefill, decode and KV state 415
36 Paged KV, prefix sharing and allocation 428
37 Continuous batching and precision 438
38 Speculative decoding and attention kernels 448

Part IX Evolutor: heterogeneous execution 459

39 Evolutor runtime above both engines 460
40 Hybrid compressed and addressable memory 469
41 Compiler, IR and execution planning 477
42 Database engines as a systems comparison 486
43 Structural adaptation and lifecycle governance 494

Part X Industrial engineering 503

44 Packaging, deployment and observable services 504
45 Multi-tenancy, isolation and security 514
46 Distributed serving and state placement 523
47 Profiling, performance and hardware backends 531

Part XI Applications and capability research 540

48 Genomic sequence applications 541
49 Streaming, language, code and persistent agents 549
50 Continual learning and population adaptation 557
51 AGI capability hypotheses and limits 565

Part XII Research synthesis 574

52 Failures, open problems and reproducible synthesis 575

Contents

Preface ii

How to use this book iv

Contents at a glance vii

I The research question 1

1 Why Genomic Computation? 2
1.1 Three ways to organize computation . . . 2
1.2 Stored, expressed, and adapted are different roles . . . 4
1.3 What biological regulation actually contributes . . . 5
1.4 An exact typed example of expression . . . 5
1.5 Development: constructing the working system . . . 8
1.6 Execution, learning, and structural change . . . 9
1.7 Before calling it genomic, check computer science . . . 10
1.8 Evolutor, DOGMA, and Hermon DNA . . . 11
1.9 Two memory strategies, one honest budget . . . 13
1.10 A research program with concrete stopping points . . . 14
1.11 Exercises . . . 15
1.12 Worked solutions and discussion . . . 16

II Learning and sequence-model foundations 17

2 Learning objectives, data, tasks, and evaluation 18
2.1 Five objects that a score cannot identify . . . 18
2.2 A learning problem includes a population and a procedure . . . 19
2.3 Autoregression is an information contract . . . 19
2.4 Tokenization changes the prediction problem . . . 20
2.5 Data families answer different questions . . . 21
2.6 Split relationships before splitting rows . . . 22
2.7 The random-data entropy control . . . 23
2.8 Intervene on future tokens . . . 23
2.9 Why gradients alone are not enough . . . 25
2.10 A CPU reference experiment with a known failure . . . 25
2.11 Capacity is a vector of confounds . . . 26
2.12 Baselines, removal tests and transplants . . . 27
2.13 Seeds reveal outcomes, not just a mean . . . 28
2.14 Loss, perplexity and bits: keep the denominator . . . 28
2.15 Accuracy, calibration and systems measurements . . . 29
2.16 Exact synthetic tasks before learned solutions . . . 30

2.17 Balanced memory tasks and shortcut audits . . . 31
2.18 Budgets and the experiment lifecycle . . . 32
2.19 Exercises . . . 34
2.20 Worked solutions . . . 34

3 Differentiation, optimization and tensor programs 36
3.1 Differentiate one scalar program before a tensor network . . . 36
3.2 From a declared objective to a local change . . . 38
3.3 Read an equation as a tensor program . . . 38
3.4 Start the reverse pass at the loss . . . 39
3.5 Derive each matrix gradient, including shared biases . . . 40
3.6 What automatic differentiation computes . . . 42
3.7 Numerical checking needs a controlled function . . . 43
3.8 The negative control: forward equality is not gradient equality . . . 43
3.9 Why a smaller difference step eventually gets worse . . . 44
3.10 Stable formulas still need finite inputs . . . 45
3.11 An exact gradient does not choose a safe step . . . 47
3.12 Accumulation must implement the declared objective . . . 48
3.13 A biological motivation is not a derivative definition . . . 50
3.14 Optimizer state changes the update, not the derivative . . . 51
3.15 Control the state in a numerical comparison . . . 52
3.16 The handoff to a reproducible training run . . . 53
3.17 Exercises with worked reasoning . . . 54
3.18 Selected glossary . . . 55
3.19 Executable boundary diagnostics . . . 55
3.20 Sources and further reading . . . 56

4 Reproducible PyTorch training 57
4.1 The weights are identical. Why does training diverge? . . . 57
4.2 A dataset small enough to understand . . . 57
4.3 Turn a sequence into an explicit prediction task . . . 59
4.4 One complete training transition . . . 61
4.5 Evaluation has two independent switches . . . 62
4.6 A checkpoint is a serialized continuation state . . . 63
4.7 Measure the restart, including negative controls . . . 64
4.8 Exercises with worked answers . . . 65
4.9 Read the implementation as a contract . . . 67
4.10 Selective glossary and the next question . . . 69

5 Tokenization and sequence representation 71
5.1 Decide what a token means . . . 71
5.2 Build an integer code from first principles . . . 73
5.3 Make the round trip enforce a contract . . . 74
5.4 Count the representation cost . . . 75
5.5 An embedding is a row lookup . . . 75
5.6 Track which bases a prediction can see . . . 77
5.7 Reverse the strand, then reconsider the boundaries . . . 78
5.8 A representation checklist before training . . . 79
5.9 Exercises with worked solutions . . . 79

6 Recurrent models and gated state 82
6.1 Unroll one state transition . . . 82
6.2 Why the gradient can lose the distant past . . . 83

6.3 Separate retention, writing, and reading . . . 84
6.4 Build a GRU cell and name its convention . . . 85
6.5 Verify forward values and backward paths . . . 87
6.6 Measure a controlled retention path . . . 87
6.7 Batch sequences without advancing padded time . . . 88
6.8 Carry, detach, and reset mean different things . . . 89
6.9 What this chapter builds, and what comes next . . . 90
6.10 Exercises with worked solutions . . . 91

7 State-space models, selective updates and scans 94
7.1 Start with a state whose arithmetic is visible . . . 94
7.2 Derive a stable discrete update from continuous time . . . 95
7.3 Expand a fixed recurrence into a convolution . . . 97
7.4 Make updates selective without making them state dependent . . . 98
7.5 Compose intervals, preserving chronological order . . . 99
7.6 Compute every prefix with a tree . . . 99
7.7 Check gradients and finite-precision equivalence . . . 101
7.8 Make padding, reset, and carry algebraic . . . 101
7.9 Place the mechanism in the broader architecture . . . 102
7.10 Exercises with worked solutions . . . 103

8 Attention and content addressing 105
8.1 Separate the address from the retrieved content . . . 105
8.2 A mask is an information-flow contract . . . 107
8.3 Derive the gradient through retrieval . . . 109
8.4 Cache keys and values without changing the computation . . . 110
8.5 Multiple heads: preserve feature ownership . . . 112
8.6 Count storage separately from work . . . 113
8.7 Assemble the next level only after the contracts hold . . . 114
8.8 Exercises with worked reasoning . . . 114
Selective glossary . . . 115
Sources and further reading . . . 115

9 Assembling a Transformer block 116
9.1 Name the residual stream and the two branches . . . 116
9.2 Layer normalization operates along features . . . 117
9.3 Give position an absolute coordinate . . . 119
9.4 Integrate attention and a nonlinear feature branch . . . 120
9.5 Carry one cache per block . . . 122
9.6 Shift targets and count only legal prediction edges . . . 124
9.7 Run a small, inspectable training experiment . . . 125
9.8 What is now assembled, and what remains . . . 127
9.9 Exercises with worked reasoning . . . 127
Sources and further reading . . . 128
Selective glossary . . . 128

10 Conditional computation and routing 129
10.1 Separate scoring, selection, admission, and weighting . . . 129
10.2 Gather only admitted rows, then restore token identity . . . 131
10.3 Differentiate the chosen computation, not a fictional smooth selector . . . 133
10.4 Capacity creates state and grouping semantics . . . 134
10.5 Load balancing is an objective, not a certificate . . . 136
10.6 Replace the feed-forward branch in the existing block . . . 137

10.7 Count arithmetic without promising speed . . . 138
10.8 What the reference establishes . . . 139
10.9 Exercises with worked reasoning . . . 139
Selective glossary . . . 140
Sources and further reading . . . 140

III From biology to formal computational hypotheses 141

11 Regulation and expression across levels 142
11.1 Stored sequence is not executed function . . . 142
11.2 Normalize states before interpreting a gate . . . 143
11.3 Track production, retention and removal . . . 143
11.4 Solve a constant-input interval . . . 144
11.5 A stopped source can leave a rising output . . . 146
11.6 A perfect output trajectory can remain ambiguous . . . 147
11.7 Extract a typed software hypothesis . . . 147
11.8 Exercises with worked reasoning . . . 149
Selective glossary . . . 149
Sources and further reading . . . 150

12 Development and generated computational structure 151
12.1 Three maps, not one biological metaphor . . . 151
12.2 Make a construction step atomic . . . 152
12.3 Define the graph before writing the compiler . . . 153
12.4 Emit a typed, topologically ordered program . . . 153
12.5 Reject illegal structure before running tensors . . . 154
12.6 Run the graph and preserve gradient ownership . . . 156
12.7 Count the expanded structure, not the description . . . 157
12.8 Separate construction correctness from empirical discovery . . . 159
12.9 Exercises with worked reasoning . . . 160
Selective glossary . . . 160
Sources and further reading . . . 161

13 Learning, structural adaptation and evolution 162
13.1 Four objects and four update contracts . . . 162
13.2 A two-feature problem that exposes the difference . . . 163
13.3 Derive a numerical oracle without running the loop . . . 165
13.4 Selection needs a reproducible lineage . . . 165
13.5 Inheritance is a mapping, not a file copy . . . 167
13.6 Search uses up evaluation information . . . 168
13.7 A current frontier: evaluator-driven program evolution . . . 169
13.8 Exercises with worked answers . . . 170
13.9 From adaptation policies to typed candidates . . . 171
Sources and evidence notes . . . 171

14 Typed genomic computation systems 172
14.1 Start with a contract that shapes alone cannot express . . . 172
14.2 Define the alphabet and the computational gene . . . 173
14.3 Type checking as an environment transformation . . . 175
14.4 Expression has a snapshot and a commit boundary . . . 176
14.5 A trace is evidence only if it identifies the experiment . . . 177
14.6 Current compiler practice and the research boundary . . . 178

14.7 Exercises with worked answers . . . 179
14.8 The next step is an operational semantics . . . 180
Sources and evidence notes . . . 180

15 Operational semantics and expression traces 181
15.1 Specify configurations before instructions . . . 181
15.2 Derive the small-step relation . . . 183
15.3 A loop invariant proves the result; a ranking function proves termination . . . 184
15.4 Fuel bounds observation, not the program's meaning . . . 185
15.5 Well-typed transformations can violate effects . . . 186
15.6 Replay checks a claim rather than trusting a log . . . 187
15.7 Exercises with worked answers . . . 188
15.8 Terms and the next contract . . . 189
Sources and evidence . . . 189

16 Expression complexity and resource semantics 190
16.1 A resource is a vector with units, not one adjective . . . 190
16.2 Derive an expert ledger from tensor dimensions . . . 191
16.3 Liveness turns a graph into a memory calculation . . . 193
16.4 Amortized cost explains totals without hiding spikes . . . 194
16.5 A performance envelope is a hypothesis to measure . . . 196
16.6 Exercises with worked answers . . . 197
16.7 Terms and next scope . . . 198
Sources and evidence . . . 198

17 Traces, credit and mechanistic evidence 199
17.1 Observation can fit perfectly and still mislead . . . 199
17.2 Necessity depends on what else remains available . . . 201
17.3 Derive credit as an average over ordering . . . 201
17.4 A trace through time needs timed interventions . . . 203
17.5 Test explanations on interventions, not only observations . . . 204
17.6 An evidence contract for Evolutor . . . 205
17.7 Exercises with worked answers . . . 206
17.8 Vocabulary and the next question . . . 206
Sources and evidence . . . 207

IV DOGMA: non-Transformer DNA-native computation 208

18 DOGMA primitives and state semantics 209
18.1 Give each adjacent pair a stable address . . . 209
18.2 Implement the transition before adding learned parameters . . . 210
18.3 Chunking, padding and reset are different operations . . . 211
18.4 Separate chemical polarity from string orientation . . . 212
18.5 Prove a symmetry, then construct its failure . . . 213
18.6 Memory mass exposes retention and information loss . . . 215
18.7 Resource and numerical claims need their own tests . . . 215
18.8 Connect to research without borrowing its claims . . . 216
18.9 Exercises with worked answers . . . 216
18.10 Vocabulary and the next question . . . 217
Sources and evidence . . . 217

19 DOGMA regulation and selective transformations 218

19.1 A regulator chooses coefficients, not an explanation . . . 218
19.2 Derive a bounded selective transformation . . . 219
19.3 Use the lost-order example as a constructive test . . . 220
19.4 Implement one record with explicit ownership . . . 221
19.5 Prove causality and chunk equivalence . . . 222
19.6 Differentiate the selected operation . . . 223
19.7 A feedback gate changes both the proof and the scheduler . . . 224
19.8 Gate interpretations may be non-identifiable . . . 225
19.9 Place the operator against current research . . . 225
19.10 A falsifiable evaluation contract . . . 226
19.11 Exercises with worked answers . . . 226
19.12 Vocabulary and next question . . . 227
Sources and evidence . . . 227

20 Structured memory, locality and timescales 228
20.1 Begin with the query a memory must answer . . . 228
20.2 Derive a timescale from a declared clock . . . 229
20.3 Unroll the state and inspect what it remembers . . . 230
20.4 Slow updates are not the same as slow retention . . . 231
20.5 Implement the complete carried state . . . 233
20.6 Prove chunk parity, then test the missing-field failure . . . 234
20.7 Coarse memory loses order: an exact counterexample . . . 234
20.8 Timescale gradients and useful information . . . 235
20.9 Research comparisons without architectural substitution . . . 236
20.10 Build an evaluation contract that could reject the design . . . 236
20.11 Exercises with worked answers . . . 237
20.12 Vocabulary and the next boundary . . . 238
Sources and evidence . . . 239

21 Strands, complements and dual-state hypotheses 240
21.1 Choose coordinates before choosing an architecture . . . 240
21.2 Invariance and equivariance answer different questions . . . 241
21.3 Lift a causal operator into paired directional states . . . 242
21.4 A readout must respect the hidden action . . . 244
21.5 Why the reverse pass cannot be ordinary causal decoding . . . 245
21.6 Reverse the record, not the transport chunks . . . 246
21.7 Differentiate the shared computation . . . 247
21.8 A falsifiable strand hypothesis, not a biological shortcut . . . 248
21.9 Exercises with worked answers . . . 248
21.10 Vocabulary and the integration boundary . . . 250
Sources and evidence . . . 250

22 DOGMA reference model and falsifiable research program 251
22.1 The computational contract . . . 251
22.2 Mode one: causal streaming . . . 254
22.3 Mode two: offline strand-symmetric encoding . . . 255
22.4 Why call the memory structured? . . . 256
22.5 Trace as evidence, not explanation . . . 257
22.6 A small executable experiment . . . 257
22.7 Ablations that can kill the hypothesis . . . 259
22.8 Closest established mechanisms . . . 260
22.9 Falsification matrix . . . 260

22.10 What the reference model does not contain . . . 260
22.11 Exercises with worked answers . . . 261
22.12 Vocabulary and next interface . . . 262
Sources and evidence . . . 263

V Hermon DNA: Transformer-based DNA computation 264

23 Hermon DNA reference architecture 265
23.1 Input contract: begin with nucleotides . . . 265
23.2 Self-attention from shapes to equations . . . 266
23.3 Residual block and parameter ownership . . . 268
23.4 Two masks, two scientific questions . . . 268
23.5 Reverse complement is an action, not augmentation folklore . . . 270
23.6 DOGMA and Hermon DNA remain separate . . . 271
23.7 Complexity and the long-context boundary . . . 271
23.8 A controlled executable experiment . . . 271
23.9 What current genomic models teach the design . . . 273
23.10 Evidence ladder . . . 274
23.11 Exercises and worked solutions . . . 274
23.12 Bridge . . . 277
Sources and evidence . . . 277

24 DNA-aware Transformer mechanisms 278
24.1 A token must carry its genomic span . . . 278
24.2 Token causality is weaker than base causality . . . 280
24.3 Build a strand action into a feature space . . . 280
24.4 Position should measure the distance the claim uses . . . 282
24.5 Derive rotary positions as pair rotations . . . 282
24.6 Long context is an attention graph before it is a speed claim . . . 284
24.7 Implement sparse semantics without allocating dense scores . . . 286
24.8 Retrieval adds information, not free context . . . 287
24.9 Design the experiment that could reject a mechanism . . . 287
24.10 Exercises and worked solutions . . . 288
24.11 Bridge . . . 289
Sources and evidence . . . 290

VI Training systems and fair experiments 291

25 Shared experiments without false equivalence 292
25.1 Define the estimand before building the leaderboard . . . 292
25.2 Freeze a common envelope, not a common machine . . . 293
25.3 Leakage prevention is a transitive graph problem . . . 293
25.4 Preprocessing and selection belong to the training side . . . 296
25.5 Accuracy, class balance and missing denominators . . . 296
25.6 Derive paired uncertainty from correctness differences . . . 297
25.7 Cluster resampling and the cost of a false sample size . . . 298
25.8 Seeds, tuning and variance are separate experimental axes . . . 299
25.9 A resource budget is a vector with provenance . . . 300
25.10 Native adapters must pass execution parity first . . . 301
25.11 Diagnose negative results without rewriting history . . . 302
25.12 Ablation, substitution and transplant are different interventions . . . 303

25.13 Twenty worked exercises . . . 304
25.14 Bridge . . . 305

26 Training/inference parity and parallel recurrence 307
26.1 Specify the transition before changing the schedule . . . 307
26.2 Derive chronological affine composition . . . 308
26.3 Prefixes, not just one tree reduction . . . 309
26.4 Resets and padding belong in the algebra . . . 311
26.5 Carry the state and preserve its gradient . . . 312
26.6 An independent closed-form oracle . . . 313
26.7 Derive a reverse-mode gradient oracle . . . 313
26.8 Numerical equivalence has a real counterexample . . . 314
26.9 Depth, work and measured latency are different quantities . . . 315
26.10 Serving parity includes ownership . . . 316
26.11 Failure-oriented verification . . . 316
26.12 Twenty worked exercises . . . 317
26.13 Bridge to memory-efficient training . . . 318

27 Optimization and memory-efficient training 319
27.1 Persistent arrays are not peak memory . . . 319
27.2 Define the reduction before choosing microbatches . . . 320
27.3 Accumulate numerators with one global denominator . . . 320
27.4 An independent derivative oracle . . . 322
27.5 The optimizer clock advances once per window . . . 322
27.6 Clipping and loss scaling have an order . . . 323
27.7 Checkpointing changes retention, not the derivative . . . 324
27.8 Same forward, wrong stochastic backward . . . 325
27.9 Derive a checkpoint-placement toy before measuring a system . . . 325
27.10 Mixed precision needs an operator-level policy . . . 326
27.11 Apply the contracts to both model families . . . 326
27.12 A measurement protocol that can fail . . . 327
27.13 Worked problems and implementation exercises . . . 328
27.14 Bridge to distributed ownership . . . 329

28 Distributed training and reproducible model artifacts 330
28.1 Declare ownership before choosing a parallelism strategy . . . 330
28.2 One global weighted objective . . . 331
28.3 Test the derivative, not just vector addition . . . 332
28.4 Balanced shards require identities and offsets . . . 333
28.5 Derive tensor-parallel operators from matrix algebra . . . 334
28.6 Persistent memory and transient materialization . . . 336
28.7 Model artifacts are byte-level contracts . . . 336
28.8 Integrity is not authenticity or numerical validity . . . 338
28.9 Reload in an independent process . . . 338
28.10 Boundary of the evidence . . . 339
28.11 Worked exercises . . . 340

29 Benchmark design and comparative evidence 342
29.1 Freeze a claim that can fail . . . 342
29.2 Same question, native execution . . . 343
29.3 Make the estimand explicit . . . 344
29.4 An exact bootstrap oracle . . . 344
29.5 Seeds, groups and crossed dependence . . . 346

29.6 The resource budget is a vector . . . 347
29.7 Tuning and task selection change conclusions . . . 348
29.8 Mechanism claims require interventions . . . 349
29.9 Artifacts and claims need separate evidence . . . 350
29.10 Twenty worked exercises . . . 351
29.11 Bridge to runtime contracts . . . 352

VII DOGMA Engine: state-native inference 353

30 Runtime contracts and request lifecycles 354
30.1 Bind a request before allocating state . . . 354
30.2 Admission is a decision, not a scheduler . . . 356
30.3 Terminal outcome and cleanup are separate coordinates . . . 356
30.4 One serialized terminal winner . . . 357
30.5 Ownership survives physical placement changes . . . 357
30.6 Backpressure before model work . . . 358
30.7 Fence a late result, then commit atomically . . . 359
30.8 Deadlines use one explicit clock domain . . . 361
30.9 Draining is not success, and cleanup is not delivery . . . 361
30.10 Reconcile every counter with an invariant . . . 362
30.11 Independent interleavings before load tests . . . 363
30.12 Retries and execution authority are additional contracts . . . 363
30.13 Twenty worked exercises . . . 364
30.14 Bridge to native state . . . 365

31 DOGMA state construction and execution 366
31.1 Make the native state executable . . . 366
31.2 Declare the artifact, not only its name . . . 366
31.3 Construct and validate the state . . . 368
31.4 Derive one native step . . . 368
31.5 Ingest chunks and masks . . . 369
31.6 Verify exact arithmetic before optimizing . . . 370
31.7 A closed form and ordered chunk composition . . . 371
31.8 Check derivatives independently . . . 372
31.9 Define next-token semantics . . . 373
31.10 Stage the native result before shell publication . . . 373
31.11 What the tests establish . . . 375
31.12 Worked exercises . . . 375
31.13 Handoff to state pools and memory ownership . . . 376

32 DOGMA state pools and memory ownership 377
32.1 The address is not the state . . . 377
32.2 Map logical identity to bounded storage . . . 377
32.3 Type the pool before allocating . . . 378
32.4 Allocation has an all-or-nothing publication point . . . 379
32.5 Generation and version solve different stale-state problems . . . 379
32.6 Pin in-flight work before release . . . 380
32.7 Fence completion and settle every pin . . . 381
32.8 Clone values without sharing writable storage . . . 382
32.9 Bridge to the actual Chapter 31 native state . . . 383
32.10 Workspace has a shorter lifetime . . . 383
32.11 Scrubbing: specify the threat model honestly . . . 384

32.12 Account for capacity and physical costs separately . . . 384
32.13 Test against an independent ownership ledger . . . 385
32.14 Twenty worked exercises . . . 386
32.15 Handoff to checkpoints and shared prefixes . . . 387

33 DOGMA checkpoints, branching and prefix state 388
33.1 Cut the computation at a semantic boundary . . . 388
33.2 Derive the continuation obligation . . . 389
33.3 The envelope binds interpretation to bytes . . . 390
33.4 Parse bounded data, not executable objects . . . 391
33.5 Capture only a settled owned state . . . 392
33.6 Restore allocates; it does not resurrect . . . 393
33.7 Work through two actual native branches . . . 394
33.8 A prefix key is an index, not an acceptance decision . . . 394
33.9 Copy-on-write must refine the copy oracle . . . 396
33.10 Derive checkpoint spacing and the metadata bill . . . 397
33.11 State snapshots, generation restart and durability . . . 398
33.12 Exercises with worked reasoning . . . 398
33.13 What the next engine transformation must preserve . . . 400
Selective glossary . . . 400
Sources and further reading . . . 400

34 DOGMA batching and state-native kernels 401
34.1 Begin with one state transition . . . 401
34.2 The row lift and its legality . . . 402
34.3 Inactivity is an identity, not a zero token . . . 402
34.4 Two clocks and changing membership . . . 404
34.5 Gather with identity; scatter through a fence . . . 405
34.6 Cancellation separates rejection from reclamation . . . 407
34.7 Numerical equivalence is not bitwise identity . . . 408
34.8 Layouts, two axes and a state-native kernel boundary . . . 409
34.9 Count bytes before claiming a faster kernel . . . 409
34.10 Reproduce, break and explain . . . 410
34.11 Twenty worked exercises . . . 411
34.12 Handoff: a different representation of serving state . . . 412
Sources and further reading . . . 412

VIII Hermon DNA Engine: attention-native inference 414

35 Hermon DNA prefill, decode and KV state 415
35.1 Specify which Hermon computation is being served . . . 415
35.2 Trace one block and locate its retained state . . . 416
35.3 Define the cache before optimizing it . . . 417
35.4 Prefill constructs a causal history . . . 418
35.5 Decode and causal chunks need the correct offset . . . 419
35.6 Prove parity through tokens and layers . . . 421
35.7 Positions cannot be repaired by tensor shape . . . 422
35.8 Count resident payload separately from copying . . . 422
35.9 Work, traffic and the limits of a serving comparison . . . 423
35.10 Verification and the allocator boundary . . . 424
35.11 Twenty worked exercises . . . 425
35.12 Handoff: retain the oracle while changing storage . . . 426

Sources and further reading . . . 426

36 Paged KV, prefix sharing and allocation 428
36.1 The logical object must survive relocation . . . 428
36.2 References carry identity; counts carry lifetime . . . 429
36.3 Append requires a partial-tail rule . . . 430
36.4 Preflight the entire chunk before publishing it . . . 431
36.5 Prefix caching owns pages too . . . 433
36.6 Count physical capacity and logical sharing separately . . . 434
36.7 What the implementation establishes . . . 435
36.8 Selective glossary . . . 435
36.9 Twenty worked exercises . . . 435
36.10 From storage to service . . . 437
Sources and further reading . . . 437

37 Continuous batching and precision 438
37.1 Define consumed work and emitted output . . . 438
37.2 Admission reserves capacity; scheduling spends work . . . 439
37.3 Construct the next mixed iteration . . . 440
37.4 Latency, throughput and fairness are different questions . . . 441
37.5 Precision belongs to a tensor role . . . 442
37.6 Trace KV error into attention before judging output . . . 443
37.7 Run the real model, then apply a declared gate . . . 444
37.8 Selective glossary . . . 445
37.9 Twenty worked exercises . . . 446
37.10 The next optimization must preserve these contracts . . . 447
Sources and further reading . . . 447

38 Speculative decoding and attention kernels 448
38.1 Construct a target-preserving one-token sampler . . . 448
38.2 A block contains conditional proposals, not independent rows . . . 449
38.3 Verification state is tentative ownership . . . 451
38.4 Acceptance is not a speed measurement . . . 452
38.5 Attention can avoid a global score buffer . . . 452
38.6 Preserve offset causality and the complete model . . . 454
38.7 IO awareness, fusion and evidence . . . 455
38.8 Selective glossary . . . 456
38.9 Twenty worked exercises . . . 457
38.10 Closing the two engine directions . . . 458
Sources and further reading . . . 458

IX Evolutor: heterogeneous execution 459

39 Evolutor runtime above both engines 460
39.1 Make the workflow a typed object . . . 460
39.2 Route semantic requirements before optimizing cost . . . 462
39.3 Keep complete state private and identity explicit . . . 462
39.4 Compose retrieval, symbolic modules and engines honestly . . . 464
39.5 Planning an action does not authorize it . . . 464
39.6 Trace observable execution and close resources on failure . . . 465
39.7 Read the generated result and its negative controls . . . 466
39.8 Twelve worked exercises . . . 466

39.9 Vocabulary and the next boundary . . . 467
Sources and further reading . . . 467

40 Hybrid compressed and addressable memory 469
40.1 Four memory classes answer different questions . . . 469
40.2 Define identity, time and availability before ranking . . . 469
40.3 Compression preserves provenance, not all information . . . 471
40.4 Rank records only after scope and provenance checks . . . 472
40.5 Bound source expansion and working-context promotion . . . 473
40.6 Retention, forgetting and the evidence firewall . . . 474
40.7 Consistency and failure controls . . . 474
40.8 Twelve worked exercises . . . 475
40.9 Vocabulary and the compiler boundary . . . 476
Sources and further reading . . . 476

41 Compiler, IR and execution planning 477
41.1 An IR names semantics that a plan leaves implicit . . . 477
41.2 The verifier owns operation contracts . . . 478
41.3 Define the observation before proving a pass . . . 478
41.4 Fold exact constants and preserve observable effects . . . 479
41.5 Use an independent oracle, then break the premises . . . 480
41.6 Lower legally before scheduling cheaply . . . 481
41.7 Plan resources with allocation-time liveness . . . 482
41.8 Versioning, traceability and integration limits . . . 483
41.9 Twelve worked exercises . . . 484
41.10 Vocabulary and the comparison boundary . . . 485
Sources and further reading . . . 485

42 Database engines as a systems comparison 486
42.1 Logical requirements delimit physical alternatives . . . 486
42.2 Volcano and Cascades transfer structure, not equivalence . . . 487
42.3 Buffer management needs object-specific validity . . . 488
42.4 Transactions belong to particular durable mutations . . . 489
42.5 Derive a small write-ahead recovery contract . . . 489
42.6 Crash cuts need an independent expected state . . . 490
42.7 Recovery, replay and stale statistics are different problems . . . 491
42.8 Twelve worked exercises . . . 492
42.9 Vocabulary and the governance boundary . . . 492
Sources and further reading . . . 492

43 Structural adaptation and lifecycle governance 494
43.1 Name the object that changes . . . 494
43.2 A gate is a conjunction of constraints . . . 495
43.3 Bind evaluation to candidate, protocol and budget . . . 496
43.4 Separate parametric stability from structural legality . . . 496
43.5 Make lifecycle transitions explicit . . . 497
43.6 Promote with authority and a current revision . . . 498
43.7 Rollback is a governed switch, not time reversal . . . 499
43.8 Read the lineage result and monitor after promotion . . . 500
43.9 Twelve worked exercises . . . 500
43.10 Vocabulary and industrial handoff . . . 501
Sources and further reading . . . 501

X Industrial engineering **503**

44 Packaging, deployment and observable services **504**
44.1 Bind the release before starting it . . . 504
44.2 Compatibility is an acceptance relation . . . 506
44.3 Roll out a revision, not a filename . . . 506
44.4 Live, ready and able to reserve are separate predicates . . . 507
44.5 Observe causal operations without copying private content . . . 509
44.6 Turn observations into a falsifiable service objective . . . 509
44.7 Configuration, secrets and a rollback that can actually run . . . 510
44.8 Break the contract deliberately . . . 511
44.9 Twelve worked exercises . . . 511
44.10 Terms and the next boundary . . . 512

45 Multi-tenancy, isolation and security **514**
45.1 Name the adversary and the trusted components . . . 514
45.2 Identity arrives from authentication, not from prose . . . 515
45.3 Make capability scope and lifetime testable . . . 515
45.4 Namespaces extend through every stateful resource . . . 516
45.5 Reserve all resource dimensions before allocation . . . 518
45.6 Untrusted text cannot mint tool authority . . . 519
45.7 Isolation reaches memory, compiler and batch lanes . . . 519
45.8 Audit decisions without retaining secrets . . . 520
45.9 A local adversarial matrix and its limits . . . 520
45.10 Twelve worked exercises . . . 521
45.11 Terms and the distribution boundary . . . 522

46 Distributed serving and state placement **523**
46.1 Separate control decisions from data movement . . . 523
46.2 Represent identity independently of location . . . 524
46.3 Derive migration from a stable snapshot . . . 524
46.4 Failure windows, cancellation and recovery . . . 526
46.5 DOGMA and Hermon move different representations . . . 527
46.6 Place prefill and decode together only when justified . . . 528
46.7 Topology, backpressure and distributed observations . . . 529
46.8 Fault injection is a protocol test, not a slogan . . . 529
46.9 Twelve worked exercises . . . 529
46.10 Terms and the measurement boundary . . . 530

47 Profiling, performance and hardware backends **531**
47.1 Correctness is a precondition for comparison . . . 531
47.2 Freeze a workload and a measurement population . . . 531
47.3 Trace timestamps into user-visible metrics . . . 532
47.4 Percentiles and uncertainty need declared estimators . . . 534
47.5 Measure a critical path before summing durations . . . 534
47.6 Count arithmetic and traffic at a named boundary . . . 535
47.7 Profile top down with tools appropriate to the backend . . . 537
47.8 Time asynchronous work at an intentional boundary . . . 537
47.9 An optimization loop that can reject its own hypothesis . . . 538
47.10 Terms and the application boundary . . . 538
47.11 Twelve worked exercises . . . 538

XI Applications and capability research **540**

48 Genomic sequence applications **541**
48.1 From a biological question to an observation contract . . . 541
48.2 Leakage is a relation, then a transitive closure . . . 542
48.3 Reverse complement acts on data and labels . . . 543
48.4 Resolution, context and the baseline ladder . . . 544
48.5 Derive discrimination and probability checks . . . 545
48.6 Uncertainty follows the independent unit . . . 546
48.7 Variant scores and mechanistic claims . . . 546
48.8 Selective glossary . . . 547
48.9 Exercises and worked solutions . . . 547
48.10 Bridge . . . 548

49 Streaming, language, code and persistent agents **549**
49.1 Events, ordering and bounded pressure . . . 549
49.2 Four states that cannot substitute for one another . . . 550
49.3 Identity binds replay to one contract . . . 551
49.4 An action state machine for ambiguous crashes . . . 551
49.5 Replay is reduction, not re-execution . . . 553
49.6 Language and code workloads need task evidence . . . 553
49.7 Plans, leases, humans and observability . . . 554
49.8 Selective glossary . . . 554
49.9 Exercises and worked solutions . . . 555
49.10 Bridge . . . 556

50 Continual learning and population adaptation **557**
50.1 Five adaptation layers and their owners . . . 557
50.2 A transparent acquisition–retention conflict . . . 557
50.3 Derive an anchored objective and its stability bound . . . 558
50.4 Read the full experience matrix . . . 560
50.5 Replay and modularity change different terms . . . 561
50.6 Population search counts all its opportunities . . . 561
50.7 Feasibility, Pareto comparisons and poisoning . . . 562
50.8 Selective glossary . . . 563
50.9 Exercises and worked solutions . . . 563
50.10 Bridge . . . 564

51 AGI capability hypotheses and limits **565**
51.1 An evidence label is not a performance number . . . 565
51.2 Breadth, depth and autonomy form a vector . . . 567
51.3 Unknown-task evaluation needs an information firewall . . . 568
51.4 Skill acquisition has a budget axis, not a magical ratio . . . 569
51.5 Finite tests cannot prove unrestricted generality . . . 570
51.6 Three research directions, three falsifiable hypotheses . . . 571
51.7 Selective glossary . . . 571
51.8 Exercises and worked solutions . . . 572
51.9 Bridge . . . 573

XII Research synthesis **574**

52 Failures, open problems and reproducible synthesis **575**

52.1 What is actually implemented, and what is still staged? . . . 575
52.2 Evidence matrices should preserve the closest competitor . . . 576
52.3 Turn a symptom into a durable failure artifact . . . 577
52.4 Random-data sanity is an expectation with assumptions . . . 577
52.5 Causality tests need a changing negative control . . . 578
52.6 A ledger distinguishes plans, observations and conclusions . . . 578
52.7 Reproducibility is a dependency closure, not one checksum . . . 579
52.8 Open problems require disconfirming experiments . . . 581
52.9 A staged program and a separate release decision . . . 582
52.10 Selective glossary . . . 582
52.11 Final exercises and worked solutions . . . 583
52.12 Final synthesis . . . 584

Epilogue: genomic computation in three senses **585**

A Molecular biology for computing readers **590**
A.1 The molecule behind the string . . . 590
A.2 Genomes, assemblies and coordinates . . . 591
A.3 From gene to function . . . 591
A.4 Where the labels come from . . . 591
A.5 Variation and relatedness . . . 592
A.6 What this appendix does not cover . . . 592

B Notation and conventions **593**
B.1 Sequences, positions and strands . . . 593
B.2 Tensors, models and state . . . 593
B.3 Probability, evaluation and units . . . 594
B.4 Typographic conventions . . . 594

C Evidence classes and the claim ledger **595**
C.1 Eight classes of evidence . . . 595
C.2 A claim template . . . 595
C.3 Ten questions to ask of any result . . . 596

Glossary **597**

Bibliography **602**

Index **613**

PART I

The research question

Can separating stored structure from expressed computation give a useful computational organization, and what would count as evidence that it does?

The book opens with a deliberately ordinary program. A fixed catalog of modules and a context-dependent plan show what "stored", "expressed" and "adapted" mean as software roles, and why each idea already has a neighbour in computer science. The chapter fixes the vocabulary used everywhere else: DOGMA is the non-Transformer DNA-native direction, Hermon DNA the Transformer-based direction, and Evolutor the theory, compiler, runtime and governance layer above both. It also states the evidence standard: a proposal earns a claim only through a comparison that could have gone against it.

CHAPTERS IN THIS PART

1 **Why Genomic Computation?** Separate biological inspiration, formal computation and empirical capability.

CHAPTER 1

Why Genomic Computation?

In this chapter you will

- separate stored structure, expressed computation and adapted structure, and say which intervention tests each
- trace an exact typed expression example and compare it with its ordinary branching baseline
- place DOGMA, Hermon DNA and Evolutor in the book's fixed taxonomy
- count per-request memory for carried state and a key/value cache without mistaking the ratio for a speedup

The research question

A genome can persist while different parts participate in different activities. Can an engineered separation between stored structure, expressed computation, and structural adaptation yield a useful computational organization beyond ordinary routing and program construction? This chapter makes the question precise without assuming the answer.

1.1 Three ways to organize computation

Imagine a system asked to behave differently in two environments while retaining a common internal library. A program can branch. A neural model can change its activations or route work. A cell can alter gene expression. Each description involves persistent structure and context-dependent activity, but their mechanisms are not interchangeable.

The comparison becomes useful when we ask the same operational questions: what is stored, what executes, what selects the activity, what remains afterward, and what can change the stored organization? Figure 1.1 provides an entry map. The table below sharpens it without treating one column as a replacement for the others.

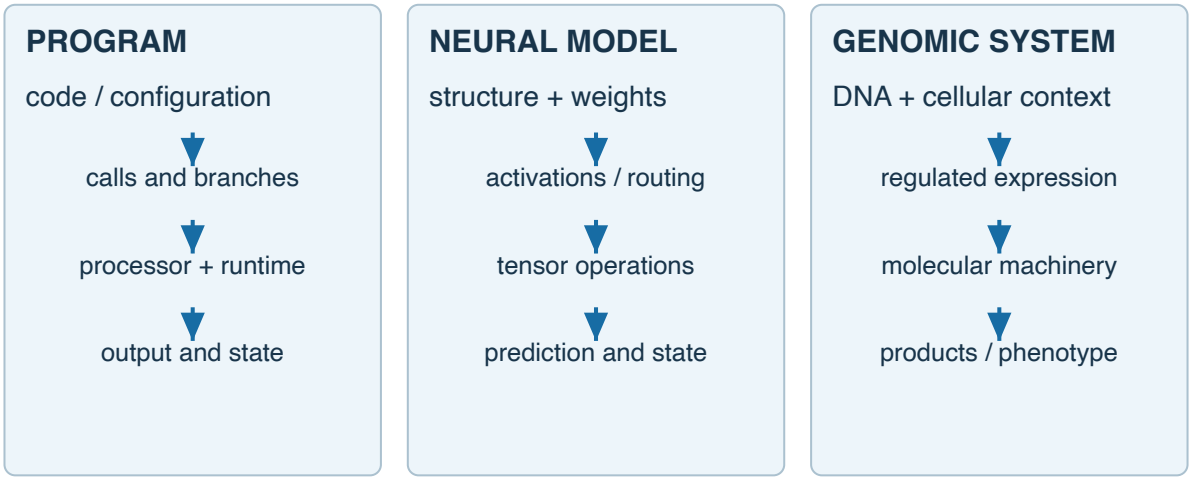


Figure 1.1: Three organizations compared through persistence, control, activity, and change. A program, a neural model, and a biological genomic system can all separate stored information from current activity. Similar roles do not imply identical mechanisms or equivalent capabilities.

Question	Program	Neural model	Biological genomic system
What persists?	Source, binary, configuration, saved data	Architecture, weights, optionally saved state	Genome; inherited and cellular context also matter
What executes?	Instructions and invoked routines	Numerical operations producing activations	Molecular processes involving RNA, proteins, and other components
What selects?	Branches, dispatch, scheduler, inputs	Activations, attention, gates, routing	Regulatory interactions in a cellular environment
What adapts?	Whatever update procedures are implemented	Parameters, state, or structure under specified update rules	Physiological and regulatory changes; population evolution across generations
What changes structure?	Code/configuration changes or generated programs	Architecture search, pruning, growth, redesign	Variation, inheritance, selection, and developmental processes

The biology column is an orientation, not a universal inventory of every organism. Gene expression can yield functional RNA or protein; control occurs at multiple stages, not only through an on/off switch at a DNA site.[1, 2] The software columns are likewise broad: conventional programs can be stateful, reflective, adaptive, or generated by other programs.

This immediately rules out a weak argument for Evolutor. Merely observing that stored components outnumber active components does not establish a new kind of computation. The research must identify a mechanism, a useful property, and a comparison under which that property matters.

1.2 Stored, expressed, and adapted are different roles

Let G denote a persistent computational description, c a context, x an input, and s a runtime state. A proposed abstraction separates selection, expression, and execution:

$$\begin{aligned} r &= \mathrm{Reg}(G, c, s), \\ P &= \mathrm{Expr}(G, r), \\ (y, s') &= \mathrm{Exec}(P, x, s). \end{aligned}$$

Here r is a control decision, P is an executable plan, y an output, and s' the next state. These are candidate software interfaces, not equations of cell biology.

Stored structure describes what the system could use. **Expressed computation** is the selected or constructed activity for this occasion. **Adapted structure** is a later persistent description G' produced by an update process. The shorthand

$$\text{stored structure} \neq \text{expressed computation} \neq \text{adapted structure}$$

distinguishes roles and times, not values that must always be numerically unequal. An update may leave $G' = G$; an expression rule may activate the entire catalog.

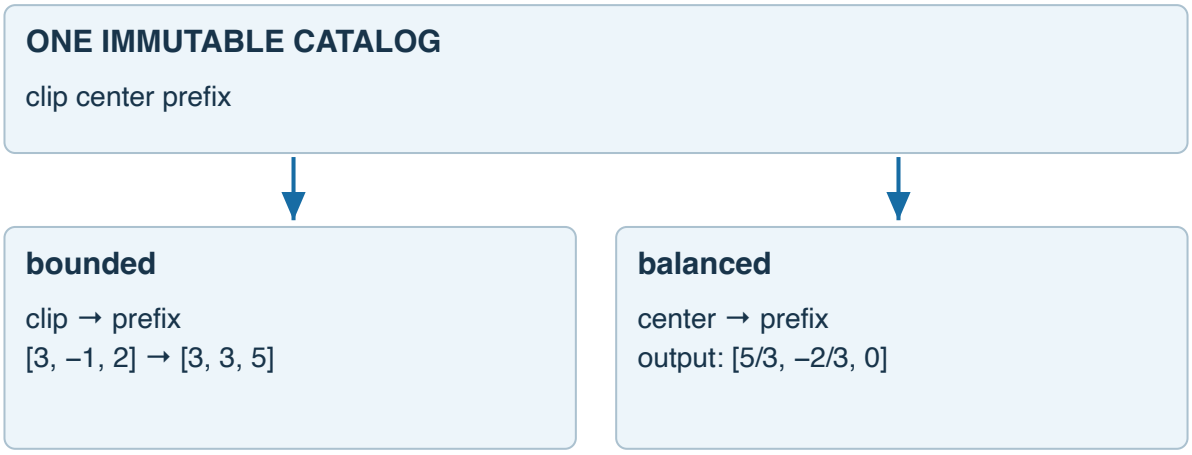


Figure 1.2: One immutable catalog supports two plans for the same input. Only the context changes. This exact routed-program example illustrates stored versus active structure; it does not implement learning, biological expression, or a new architecture.

Separating these roles may help with inspection and controlled change. We can inspect a plan before execution, log which modules ran, and restrict what an update may alter. But those are also familiar benefits of interpreters, workflows, and modular software. The notation creates a place to ask questions; it does not answer the novelty question.

The crucial distinction is counterfactual. If we change the context while holding G fixed, we test regulation. If we change G and replay the same context and input, we test persistent structural change. If neither intervention is possible or observable, describing the system as adaptive may hide rather than clarify its behavior.

1.3 What biological regulation actually contributes

A genome is not a list of Python functions. Its sequence participates in a cell that already contains molecular machinery, spatial organization, and a history. Many differentiated cells can carry substantially the same genome while exhibiting different expression patterns; exceptions such as immune-cell DNA rearrangements prevent an absolute equivalence claim.[2, 3]

Gene expression produces functional products from genomic information. Transcription produces RNA; some RNAs function directly and others serve as templates for protein production.[1] Regulation can affect transcription, RNA processing or stability, translation, and protein activity.[2] Thus a useful computational analogy must not reduce expression to fetching one stored object unchanged.

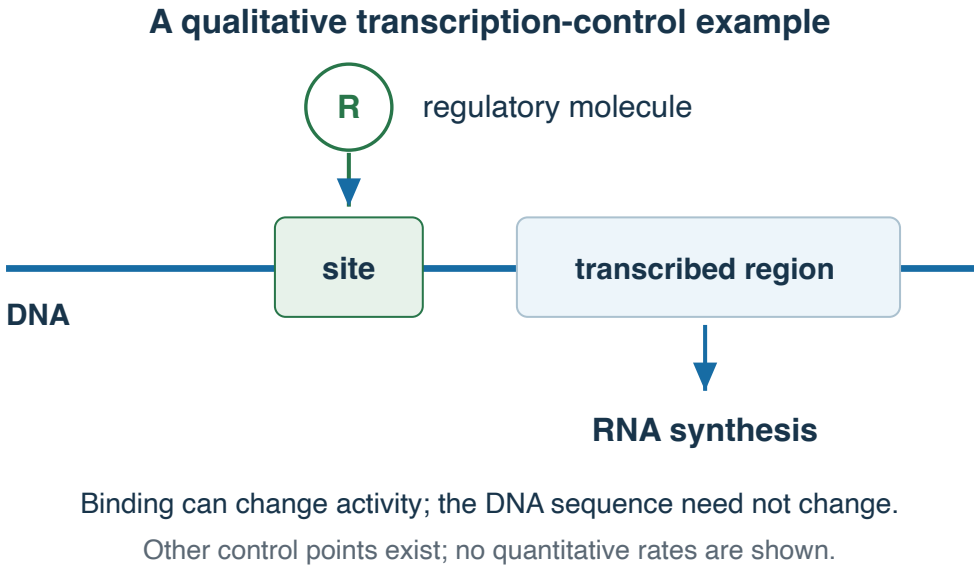


Figure 1.3: A qualitative transcription-control sketch. A regulatory molecule interacts with a DNA-associated site and can alter activity without rewriting the DNA sequence. The figure omits detailed complexes, kinetics, and other control points; the arrow means an interaction, not a software call.

The computational question extracted from this biology is narrower than a claim of imitation: can a persistent description support context-sensitive production of working structure with tractable rules for reuse, interference, and change? In software, we are free to design that interface differently from a cell. What we may not do is use the biological origin of an analogy as evidence of engineering performance.

A switch choosing one of two branches is a valid model of one control decision. It is not a quantitative gene-regulatory network. To justify that stronger description, we would need states, interactions, rates or transition rules, an account of feedback, and evidence that the abstraction preserves the biological property of interest. Chapter 1 introduces the distinction; later biological chapters develop the mechanisms.

1.4 An exact typed example of expression

We now build a deliberately ordinary program. Its simplicity is useful: every change is visible, and it has an exact conventional baseline.

Let

$$\mathcal{X} = \bigcup_{m \geq 1} \mathbb{Q}^m$$

be the type of nonempty finite rational sequences. For $x = (x_1, \ldots, x_m)$ define three length-preserving modules:

$$\begin{aligned} \operatorname{clip}(x)_i &= \max(0, x_i), \\ \operatorname{center}(x)_i &= x_i - \bar{x}, \qquad \bar{x} = \frac{1}{m} \sum_{j=1}^{m} x_j, \\ \operatorname{prefix}(x)_i &= \sum_{j=1}^{i} x_j. \end{aligned}$$

Each module has type $\mathcal{X} \to \mathcal{X}$. A plan is a finite ordered list of module names. Its meaning is their left-to-right composition. The shared type makes every such composition well-typed; it does not make every composition useful.

The context set is $C = \{\text{bounded}, \text{balanced}\}$. Our selection rule is

$$\operatorname{Reg}(c) = \begin{cases} (\text{clip}, \text{prefix}), & c = \text{bounded}, \\ (\text{center}, \text{prefix}), & c = \text{balanced}. \end{cases}$$

The catalog is immutable. There are no learned parameters. Context is an explicit argument, not inferred from hidden state.

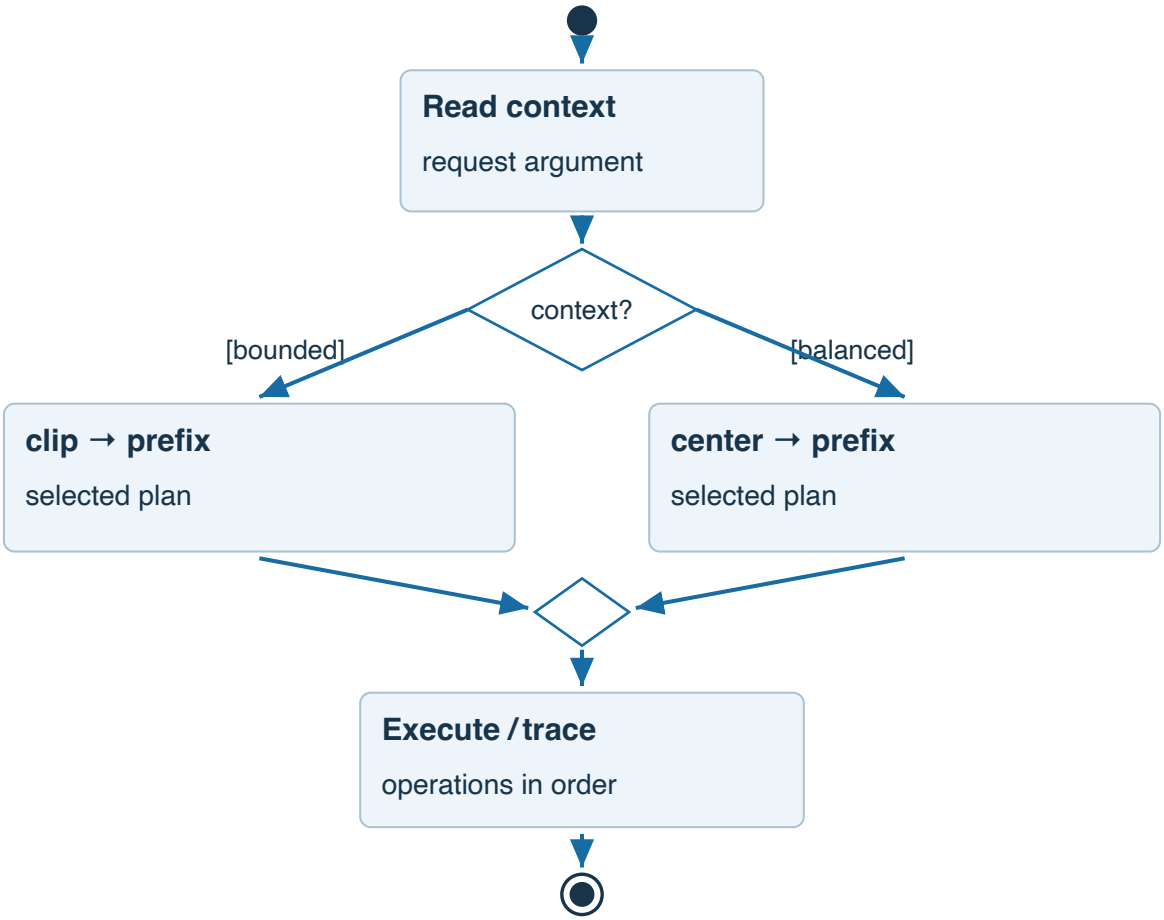


Figure 1.4: UML activity-style control flow for the worked example. Guarded alternatives select one ordered plan; execution records intermediate values. Unknown contexts raise an error outside the two successful branches shown. This is software control flow, not a biochemical pathway.

For $x = (3, -1, 2)$, the bounded route gives

$$(3, -1, 2) \xrightarrow{\text{clip}} (3, 0, 2) \xrightarrow{\text{prefix}} (3, 3, 5).$$

For the balanced route, $\bar{x} = 4/3$, so

$$(3, -1, 2) \xrightarrow{\text{center}} (5/3, -7/3, 2/3) \xrightarrow{\text{prefix}} (5/3, -2/3, 0).$$

Exact rational arithmetic makes these results transparent. The generated table comes directly from the executable source:

Context	Plan	Output
bounded	clip / prefix	3, 3, 5
balanced	center / prefix	5/3, -2/3, 0

The two contexts express different mathematical invariants. The bounded output is nonnegative and nondecreasing because it sums nonnegative terms. The balanced output ends at zero because the centered values sum to zero. Neither invariant says the intermediate balanced prefix must be nonnegative. The negative second entry is correct.

Order matters. Applying prefix before clip to $(3, -1, 2)$ yields $(3, 2, 4)$, not $(3, 3, 5)$. A set of active names cannot represent this distinction; an ordered plan can. This is the first reason to specify expression semantics more precisely than an active/inactive mask.

The source below records both input and output of each operation. The excerpt is generated from the tested file, `examples/deep/ch01.py`, not independently retyped.

```
def select(context):
    if context=="bounded":
        return ("clip","prefix")
    if context=="balanced":
        return ("center","prefix")
    raise ValueError("unknown context")


def execute(plan, values):
    xs=tuple(Fraction(x) for x in values)
    if not xs:
        raise ValueError("nonempty rational sequence required")
    trace=[]
    for name in plan:
        if name not in CATALOG:
            raise ValueError("unknown module")
        before=xs
        xs=CATALOG[name](xs)
        trace.append({"module":name,"before":before,"after":xs})
    return xs,trace
```

A direct branching implementation computes the same specification without a plan interpreter. Tests compare the two implementations over a grid of rational inputs, not just the displayed example. We can therefore say the abstraction exposes control structure; we cannot say that it enables a function ordinary software could not compute.

1.5 Development: constructing the working system

Expression selects or produces activity for a context. **Development** asks a related but distinct question: how is an operational system constructed from a more compact persistent description?

Biological development mediates the relation between genotype and phenotype; it is not a static one-to-one lookup from a DNA string to a finished organism.[3] Environmental influences can affect gene expression during development.[4] An engineering analogy should therefore make the construction process explicit:

$$P = \mathrm{Dev}(G, E).$$

Here G is a software description, E a construction environment, and P a program. Unlike the organism, this particular P is a digital artifact with defined execution semantics. In a richer model, development could depend on time, feedback, and stochastic events; the equation alone supplies none of them.

Our minimal generator takes a positive integer k and constructs a plan containing k copies of prefix. For $k = 2$, the input $(3, -1, 2)$ becomes $(3, 2, 4)$ after the first pass and $(3, 5, 9)$ after the second.



Figure 1.5: A compact description constructs an ordered program before that program processes data. Repeating prefix twice yields the displayed exact trace. This original program generator is an analogy for description-to-structure construction, not a model of embryogenesis.

The construction already exists in ordinary programming. Its research value would have to come from how descriptions compose, how variation changes their products, or what constraints the development process enforces. A small description might generate a large repeated structure, but executing that structure still costs time and memory. Compression of description is not free execution.

This example also separates validity from fitness. Every positive k produces a well-typed plan, but increasing k need not improve a task objective. A type checker can reject malformed structure; it cannot replace an experiment measuring whether the structure solves the intended problem.

1.6 Execution, learning, and structural change

The same system can contain several update processes. A runtime state can change on every input while parameters remain fixed. An optimizer can change parameters while topology remains fixed. A structural procedure can add a module, alter a connection, or replace a description.

A candidate formal separation is

$$\begin{aligned}(s_{t+1}, y_t) &= F_{G,\theta}(s_t, x_t),\\ \theta_{j+1} &= U(\theta_j, G, \mathcal{D}_j),\\ G_{k+1} &= A(G_k, \mathcal{E}_k).\end{aligned}$$

The indices deliberately differ. t counts execution steps, j parameter updates, and k structural proposals. $\mathcal{D}_j$ is learning data and $\mathcal{E}_k$ the evidence used to accept, reject, or revise a proposal. These equations describe possible interfaces; the chapter's exact program implements neither U nor A.

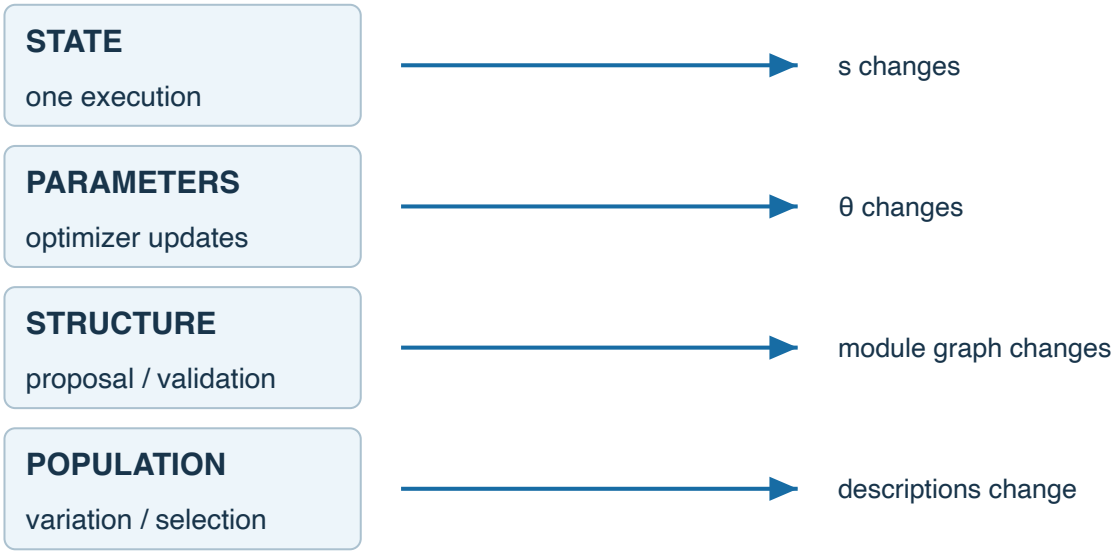


Figure 1.6: Four update roles: state, parameters, structure, and a population of inherited descriptions. Arrow meanings are update relations. No numerical biological or computational duration is asserted, and the rows do not impose a universal speed ordering.

One might run many inference steps between optimizer updates and many training runs between structural changes. That schedule is a design choice, not a theorem. Online learning can update parameters on every sample; program generation can happen in a millisecond; structural search can run offline for days.

An important engineering issue appears as soon as G changes: what happens to s and θ? If the new structure has different state dimensions, blindly restoring an old checkpoint is invalid. A structural transition needs a migration map, a reset policy, or rejection. For example, replacing a state vector in $\mathbb{R}^d$ with one in $\mathbb{R}^{d'}$ requires specifying a map $\mathbb{R}^d \to \mathbb{R}^{d'}$ or explicitly discarding the old state.

This is where the genome analogy may become productive. An architecture that makes variation, construction, and migration explicit could be easier to reason about than ad hoc topology changes. Whether that organization improves learning or robustness is a separate empirical question.

1.7 Before calling it genomic, check computer science

The novelty burden is substantial because the neighboring fields are rich. Dynamic dispatch selects behavior from persistent code. Plugins extend a catalog. Query planning constructs an execution plan. Our exact example is already expressible with these ordinary ideas.

Mixture-of-experts systems route examples through selected subnetworks; Shazeer and colleagues' sparse gate is a concrete established example.[5] Attention computes context-dependent weighted combinations of value representations from query–key scores.[6] Neither activity requires persistent architecture modification.

Neural architecture search can generate and evaluate model descriptions, as in Zoph and Le's reinforcement-learning construction.[7] Program synthesis seeks a program meeting a specification;[8] genetic programming uses evolutionary search over programs.[9] These are

direct neighbors of computational development and structural adaptation, not optional footnotes.

State-space models provide another established way to organize sequence computation. Mamba makes selected state-space parameters depend on the input so the recurrence can retain or discard information conditionally.[10] Agent systems also already compose reasoning with actions that obtain new information; ReAct provides one concrete example.[11]

Dispatch / plugins	select stored routines
MoE / conditional networks	route work among experts
Attention / retrieval	query accessible representations
Recurrence / SSMs	update carried state
Synthesis / architecture search	construct or search programs
Agent / workflow systems	compose tools and actions

A new label must earn a new semantic or empirical contribution.

Figure 1.7: Established alternatives organized by the role they already serve. The same proposed genomic mechanism may need comparison with several rows. This is a conceptual comparison map, not a benchmark or an exhaustive taxonomy of computer science.

A serious proposal should now answer a specific question. Does it offer a new semantics for compositional change? A useful inductive bias? A resource advantage on a defined task family? Better isolation of modifications? Or only a helpful vocabulary? Any of these may be worth studying, but they are different contributions.

For our two-context example, the direct baseline is functionally equivalent. If the plan interpreter is slower, a claim of inference efficiency fails for that implementation. It may still improve inspectability because the plan and trace are explicit artifacts. Such a result would support a tooling argument, not a new learning architecture.

A future comparison should match training data, task distribution, quality criterion, and relevant resource budgets. An ablation removing the proposed regulation mechanism should preserve other capacity as far as possible. Otherwise a larger catalog or extra training may explain the gain. This is the minimum experiment needed to turn a naming proposal into an engineering question.

1.8 Evolutor, DOGMA, and Hermon DNA

This book uses three project levels with fixed meanings.

Evolutor is the broader genomic-computation theory and research framework, including the planned compiler/runtime and orchestration layer above model families. It may eventually coordinate models, retrieval, symbolic modules, and tools. That is a design hypothesis, not a deployed capability established here.

DOGMA names the non-Transformer DNA-native architecture research line and its corresponding non-Transformer inference engine, DOGMA Engine. The opening hypothesis emphasizes carried state, regulation, and expression.

Hermon DNA names the Transformer-based DNA architecture research line and its corresponding Transformer inference engine, Hermon DNA Engine. Its opening hypothesis retains attention and key/value context representations.

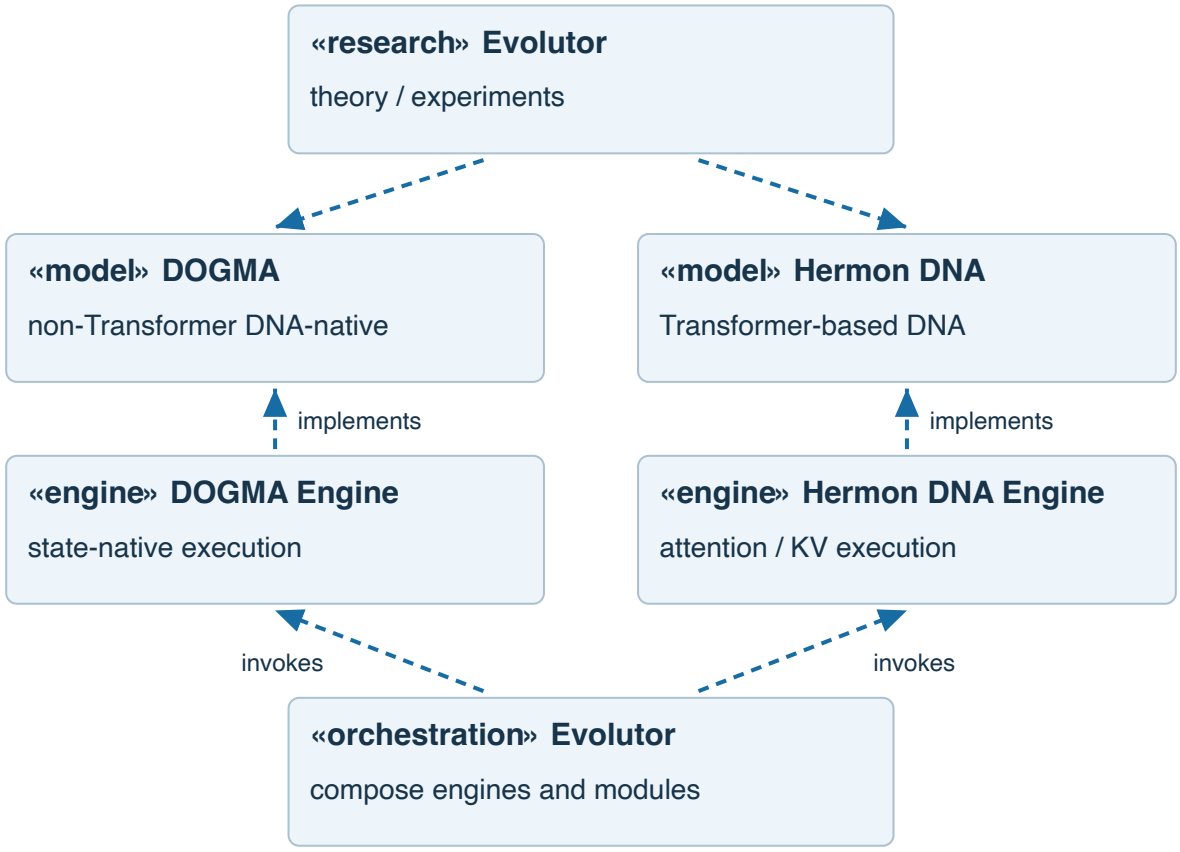


Figure 1.8: UML component-style research taxonomy. Evolutor contains the research and orchestration roles above the two families. DOGMA and Hermon DNA have separate model and engine identities. Dashed arrows express design relationships, not deployment or verified superiority.

In this project vocabulary, DNA-native and DNA-aware are research intentions whose operational content must be supplied by later representations, objectives, and mechanisms. They do not imply that either model runs in a test tube. Physical DNA computing, genomic sequence modeling, and biologically inspired digital organization must remain distinguishable.

An architecture specifies computations and state. An engine implements those computations under a resource and numerical contract. The engine may optimize layouts or kernels without changing the intended model. Conversely, a changed state transition is an architectural change, even if introduced inside an implementation.

This distinction prevents a roadmap from collapsing into a list of products. We first need formal semantics and a reference model, then tests of equivalence, then evidence from

experiments, then justified optimization. Naming an engine before those steps does not make it available.

1.9 Two memory strategies, one honest budget

A recurrent candidate can summarize prior input in carried state:

$$(s_{t+1}, y_t) = F_\theta(s_t, x_t).$$

The state is sufficient only for the behavior the model can actually support. With fixed finite precision, a bounded state cannot preserve arbitrarily many distinguishable histories without collisions.

A causal attention model instead can retain key/value representations indexed by past position. Let C_{t-1} contain the cached pairs for positions $1, \ldots, t-1$. Processing x_t appends (k_t, v_t), forms C_t, and computes an output using the permitted prefix. The indexing matters: using a representation from a future token would violate the causal contract.

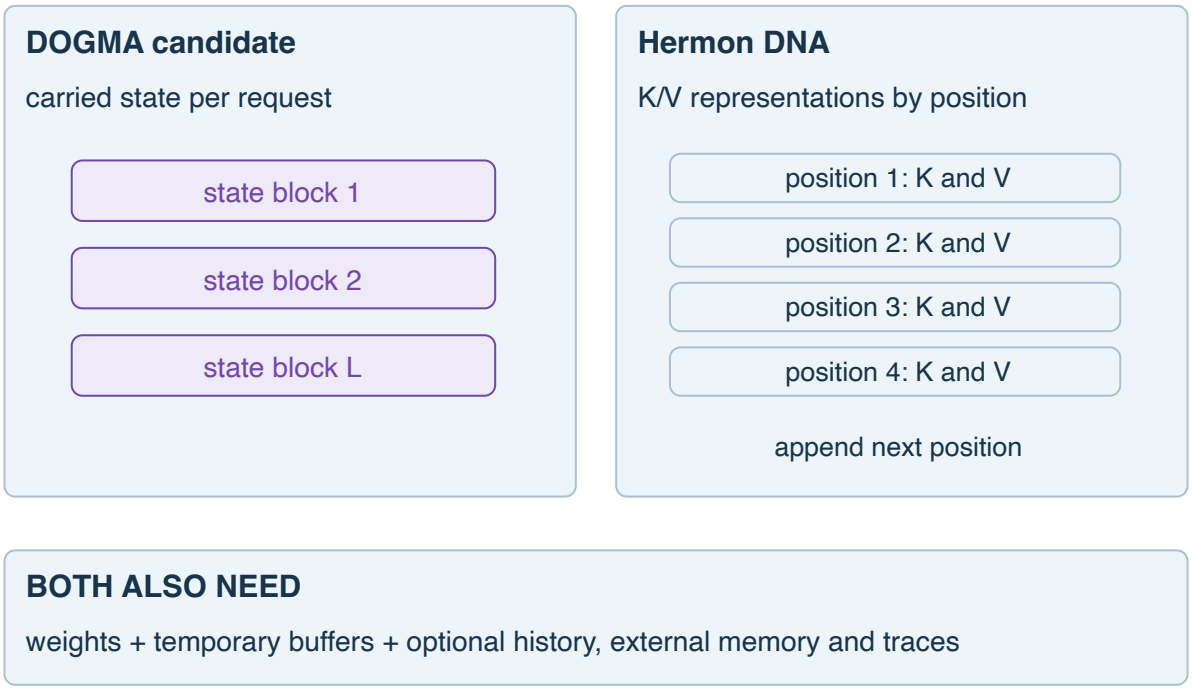


Figure 1.9: Two candidate per-request memory organizations. Carried state transforms a bounded representation; a conventional full-context KV cache retains position-indexed representations. Both also require weights and workspace and may add external memory, traces, or history. The figure does not claim equal capability.

Attention's basic operation is

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V,$$

where $Q \in \mathbb{R}^{q \times d_k}$, $K \in \mathbb{R}^{T \times d_k}$, and $V \in \mathbb{R}^{T \times d_v}$. The mask M excludes forbidden positions, and softmax acts across the T key positions for each query. This is an established mechanism, not a proposed DOGMA operation.[6]

Now derive a deliberately hypothetical storage comparison. Suppose batch size is B, layer count L, retained positions T, key/value head count H_{kv}, head width d_h, and storage per scalar b bytes. A conventional uncompressed KV layout needs

$$M_{\text{KV}} = 2BLTH_{\text{kv}}d_h b.$$

The factor two counts keys and values. A design storing d_s carried-state scalars per layer instead needs

$$M_s = BLd_s b.$$

These formulas count only the declared persistent per-request arrays. They omit weights, temporary activations, allocator overhead, retrieval stores, and any additional recurrent history.

Take $B = 1$, $L = 24$, $T = 4096$, $H_{\text{kv}} = 8$, $d_h = 64$, $d_s = 2048$, and $b = 2$. The exact helper returns 201,326,592 bytes for KV, or 192 MiB, and 98,304 bytes for state, or 96 KiB. Here MiB and KiB mean powers of 1024. The array-size ratio is 2048.

That ratio is *not* a measured speedup and does not compare equal-quality models. A smaller state may discard distinctions a task needs. A cache may be windowed, quantized, shared, compressed, or supplemented by other memory. DOGMA is not proved superior by substituting a chosen state width into an accounting formula.

The productive research question is conditional: for a specified task and error tolerance, how much information must remain accessible, how is it updated or addressed, and what does that cost? Both families exist to explore different answers rather than to predeclare a universal winner.

1.10 A research program with concrete stopping points

The worked program has already taught several useful distinctions. A catalog differs from a plan. A plan differs from its execution trace. Generating a larger plan differs from learning it. Changing runtime values differs from changing persistent structure. These are ordinary, precise concepts on which a more ambitious proposal can be built.

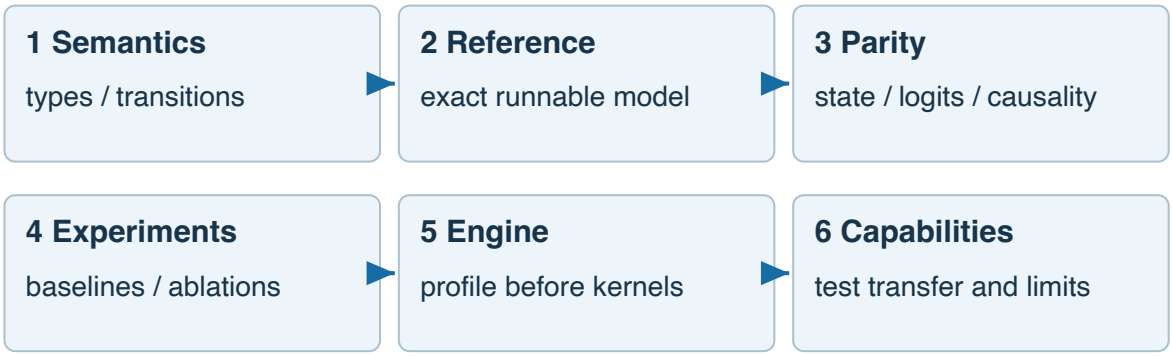


Figure 1.10: A research sequence from semantics to evaluated capabilities. Reference behavior precedes engine optimization; parity checks precede performance claims. Failure at a gate sends a proposal back for revision. These are planned investigations, not completed model or AGI results.

The next steps should preserve that precision. Define a state transition before naming a stateful model. Define what regulation may inspect before claiming causal inference. Define

how development preserves types before allowing generated structure to execute. Define migration and evaluation before treating a structural update as improvement.

Evolutor ultimately studies mechanisms relevant to memory, modularity, adaptation, continual learning, structural learning, tool composition, and generalization. Those motivations connect to broad questions about intelligence, but Chapter 1 establishes no AGI capability. A useful negative result—for example, exact equivalence to a simpler routing baseline—can clarify which mechanisms deserve further work.

Book I supplies the molecular-computation foundation. Its new Chapter 1 teaches a particular graph-search construction and its limits; it does not supply all later biology, thermodynamics, and circuit prerequisites. The shared dependency plan remains explicit about future teaching. Readers may bring equivalent background, but a roadmap reference is not proof that a prerequisite chapter already exists.

Chapter 2 develops learning objectives, data, and evaluation: sequence likelihood, leakage-resistant splits, and held-out generalization. These foundations make later genomic proposals testable. It does not jump directly to an optimized engine. This chapter's standard is the model for the rest of the book: use abstraction freely, make its semantics concrete, and explain the difficult part directly.

Key ideas

- Context changes test regulation; changes to the stored description test structural adaptation. Without an observable intervention, "adaptive" explains nothing.
- An ordered plan carries information that a set of active modules cannot: `clip` then `prefix` differs from the reverse order.
- Every idea in the chapter already has a conventional computer-science neighbour. A genomic proposal must name the property it adds and the comparison that would reveal it.
- Physical DNA computing, genomic sequence modelling and biologically inspired digital organisation are three different subjects.

1.11 Exercises

1. Classify changing a context, changing a weight, adding a module, and recording a trace by the role each changes.
2. Compute both plans on $(-2, 4, 1)$ and verify their stated invariants.
3. Prove that the final entry of prefix after center is zero for every nonempty rational input. Does the proof require all other prefix entries to be nonnegative?
4. Find an input on which clip and prefix do not commute. Explain why an unordered active set is insufficient.
5. Describe a test that distinguishes a changed plan from a changed persistent catalog.
6. Run the development rule for $k = 3$ on $(3, -1, 2)$. Which properties are guaranteed by type compatibility, and which are not?
7. Propose a structural migration from $\mathbb{R}^d$ to $\mathbb{R}^{d+1}$. State what it preserves and what performance claim it does not justify.
8. Double retained context length T in the hypothetical memory example. Then double batch size. Which arrays change, and why is this not a throughput prediction?

9. Compare the exact routed example with its direct baseline. Formulate one legitimate tooling claim and one unsupported architecture claim.

10. Design a future ablation of a regulation mechanism, specifying a task, a matched baseline, an intervention, and a possible negative result.

1.12 Worked solutions and discussion

1. Context changes the selection input; a weight update changes parameters; adding a module changes stored structure; recording a trace changes an observation artifact. None alone establishes evolutionary adaptation.

2. Bounded: $(-2, 4, 1) \to (0, 4, 1) \to (0, 4, 5)$. Balanced: mean 1, then $(-3, 3, 0) \to (-3, 0, 0)$. The first is nondecreasing and nonnegative; the second ends at zero.

3. The final prefix is $\sum_i (x_i - \bar{x}) = \sum_i x_i - m\bar{x} = 0$. No sign constraint on partial sums follows; $(-3, 0, 0)$ supplies a counterexample to nonnegativity.

4. For $(3, -1, 2)$, prefix after clip is $(3, 3, 5)$, while clip after prefix is $(3, 2, 4)$. Both plans use the same two names, so order must be represented.

5. Snapshot the immutable catalog identity and contents, vary only the context, and compare selected plans and traces. A structural-change test instead versions the catalog and replays the same input and context.

6. The third prefix pass turns $(3, 5, 9)$ into $(3, 8, 17)$. Composition preserves nonempty rational sequence type and length. It does not guarantee bounded values, useful predictions, or improved performance.

7. $m(s) = (s, 0)$ preserves the first d coordinates. It does not preserve the full model's behavior unless the new transition and readout are also constrained appropriately.

8. With doubled T, KV becomes 384 MiB while the declared state remains 96 KiB. Doubling B then gives 768 MiB and 192 KiB. Runtime depends on computation, bandwidth, batching, and implementation, none measured by these formulas.

9. The plan and trace expose selected operations for inspection. That supports an inspectability claim for this implementation. It does not show a function inaccessible to ordinary software or a learning advantage.

10. One possible task is context-dependent sequence processing under a fixed total parameter and data budget. Replace learned regulation with a matched fixed routing rule while preserving modules; measure held-out error and resource costs. Equal performance would fail to support a benefit from the learned regulator on that task. It would not establish universal uselessness.

PART II

Learning and sequence-model foundations

What does a sequence model actually compute, learn and remember, and how do we test each part before trusting it?

These nine chapters build the machine-learning foundations from first principles: objectives and legal information, hand-derived gradients, reproducible training, tokenization, recurrence, state-space scans, attention, the Transformer block and conditional routing. Every mechanism is implemented in small, inspectable PyTorch and checked against an independent oracle, often with a deliberately broken negative control beside it. The genomic examples are explicit data contracts, not claims about biology. Readers who already know deep learning should still read Chapters 2, 5 and 7: target alignment, token reach and associative scans are reused throughout the book.

CHAPTERS IN THIS PART

2 **Learning objectives, data, tasks, and evaluation** Define objectives, legal information, controls and uncertainty.

3 **Differentiation, optimization and tensor programs** Derive tensor programs and derivatives before trusting optimization.

4 **Reproducible PyTorch training** Make a training run reproducible, masked and auditable.

5 **Tokenization and sequence representation** Bind DNA tokens to record boundaries, spans and strand conventions.

6 **Recurrent models and gated state** Distinguish carried state from gradient history in gated recurrence.

7 **State-space models, selective updates and scans** Derive selective affine state updates and legal sequence scans.

8 **Attention and content addressing** Construct content-addressed retrieval, masking and attention oracles.

9 **Assembling a Transformer block** Assemble and test a complete causal Transformer block.

10 **Conditional computation and routing** Separate routing decisions, expert capacity and derivative paths.

CHAPTER 2

Learning objectives, data, tasks, and evaluation

In this chapter you will

- distinguish data, task, model, objective and evaluation protocol
- align next-base inputs and targets so that a causal mask actually prevents leakage
- build relationship-aware splits by closing conflict relations before assigning folds
- use the random-data entropy bound and finite future interventions as falsification instruments

The central question

What information may a model use, what must it predict, and what observation would count against the proposed explanation of its success? We answer with exact indexing, deliberately broken predictors, relationship-aware splits, explicit budgets and executable reference tests before introducing new architectures.

2.1 Five objects that a score cannot identify

A model is not a task. A DNA corpus is not an objective. A low loss is not, by itself, evidence for the mechanism named in an architecture diagram. To make a meaningful claim, separate five objects: data, task, model, training objective, and evaluation protocol with its metrics.

Object	Concrete next-base example	Not interchangeable with
Data	sequence records + relationships	task
Task	predict next base from prefix	architecture
Model	prefix counts or trained fθ	objective
Objective	mean negative log probability	success claim
Metric + protocol	held-out nats/base + split + mask	training loss alone

Figure 2.1: Five distinct objects in next-base prediction. A fixed predictor and a trained network can face the same task; one network can face several tasks. The score becomes evidence only after its data, information boundary, target construction and evaluation protocol are specified.

For example, a file of genomic sequences is data. Predicting the next nucleotide from the preceding prefix is a task. A smoothed prefix-frequency rule is a model, even though it has no learned weights. Mean negative log probability is an objective one could optimize. Mean held-out loss on a relationship-disjoint test set is an evaluation measurement. Changing any of these objects changes the question being answered.

DOME organizes reporting around data, optimization, model and evaluation.[12] Our purpose is not to add forms for their own sake. It is to make experimental errors concrete enough to reproduce. A predictor that reads the answer from a future input position can achieve excellent loss without learning the intended causal task; we will construct exactly that failure.

The taxonomy remains fixed throughout: DOGMA is a non-Transformer DNA-native architecture plus its non-Transformer engine; Hermon DNA is a Transformer-based DNA architecture plus its Transformer engine. Evolutor is the broader theory, research, compiler, runtime and orchestration framework above both. This chapter reports no new trained implementation of either family.

2.2 A learning problem includes a population and a procedure

For supervised data $D = \{(x_i, y_i)\}_{i=1}^{N}$, let f_θ predict from x_i. An empirical objective is

$$\widehat{R}_D(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(f_\theta(x_i), y_i), \qquad \theta^\star \in \arg\min_\theta \widehat{R}_D(\theta). \tag{2.1}$$

The parameter vector θ contains trainable quantities; $\mathcal{L}$ penalizes a prediction relative to its target. Optimization is a procedure intended to reduce the empirical objective. Equation 2.1 does not promise that an optimizer finds a global minimum or that such a minimum generalizes.

The population risk is instead $R(\theta) = \mathbb{E}_{(X,Y)\sim P}[\mathcal{L}(f_\theta(X), Y)]$. The distribution P includes how examples are sampled and what deployment population is intended. A held-out measurement estimates something about that population only if its sampling and information boundaries are appropriate. Overlapping windows from the same locus do not become independent biological examples merely because they occupy separate rows.

Regularization, augmentation, early stopping and hyperparameter search are part of the learning procedure. A test score evaluates the selected procedure, not an architecture name in isolation. Repeatedly choosing designs against a finite validation set can overfit that selection criterion;[13] looking at test scores while redesigning a model similarly turns the test into development information.

2.3 Autoregression is an information contract

For a sequence $x_1, \ldots, x_T$,

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \qquad \mathcal{L}_{\text{seq}} = -\sum_{t=1}^{T} \ln p_\theta(x_t \mid x_{<t}).$$

The first prediction needs an initial state or beginning-of-sequence convention. A recurrent state and a causally masked attention mechanism can both implement this factorization, but their code must expose only the allowed information.[6]

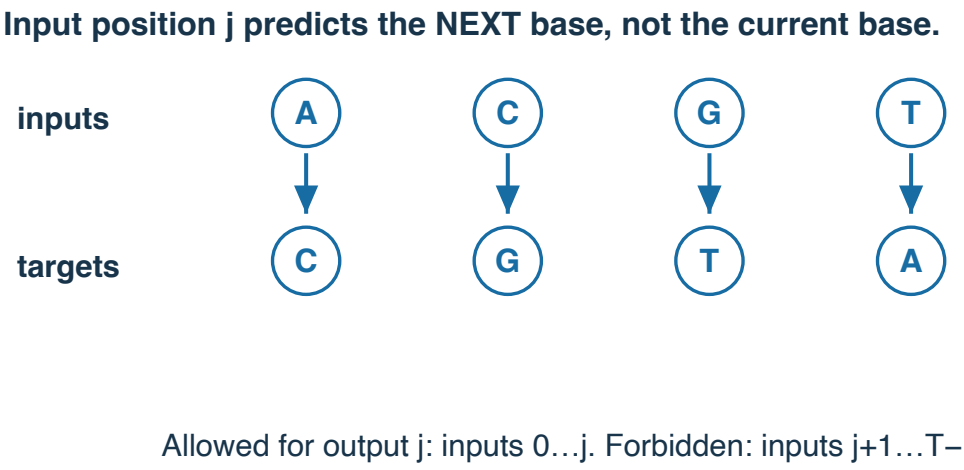


Figure 2.2: The implementation convention used here: input position j predicts the next base. With zero-based arrays, inputs are the full sequence except its last token; targets are the full sequence except its first. Output j may use inputs 0 through j, but not input $j + 1$, which contains its answer.

For a full array $z_0, \ldots, z_T$, set $u_j = z_j$ and $y_j = z_{j+1}$, for $0 \leq j < T$. Then output j may attend to input j without leakage: that input is the previous symbol relative to its target. If targets were instead z_j at the same position, allowing access to u_j would reveal the answer. Thus a lower-triangular mask cannot be judged apart from target indexing.

Teacher forcing supplies true earlier symbols during training. It does not authorize true future symbols. Padding masks, packed-sequence boundaries, state reset, preprocessing, pooling and retrieval can all violate the information contract even when the principal attention mask looks correct. A causal architecture family is not a proof that its surrounding pipeline is causal.

2.4 Tokenization changes the prediction problem

For canonical DNA strings, $\Sigma = \{A, C, G, T\}$. Real records can contain ambiguous bases; an experiment must declare whether they are excluded, masked, or represented with additional symbols. This chapter's executable examples reject anything outside the four-symbol alphabet.

Encoding of A C G T A C	Tokens	What changes
single bases	A / C / G / T / A / C	6 tokens; 4-symbol alphabet
overlapping 3-mers	ACG / CGT / GTA / TAC	next token shares 2 bases
nonoverlapping 3-mers	ACG / TAC	2 tokens; up to 64 types
illustrative segmentation	AC / G / TAC	variable lengths; learned rules

Figure 2.3: Several tokenizations of the same six-base string. The variable-length segmentation is illustrative, not a fitted BPE vocabulary. Overlapping three-mers share two bases, so next-token prediction contains a built-in overlap advantage.

Single-base tokens keep the vocabulary small and retain one position per nucleotide; HyenaDNA is an example of work using single-nucleotide resolution.[15] Fixed k-mers can have up to 4^k types. Stride matters: nonoverlapping k-mers shorten the sequence, whereas overlapping windows retain many positions and repeat information. DNABERT-2 illustrates genomic BPE tokenization,[16] but its masked-language task must not be treated as interchangeable with our autoregressive objective.

Consider iid uniform DNA and overlapping three-mers. Given `ACG`, the next window must begin `CG`; only its last base is new. Its conditional entropy is $\ln 4$, not $\ln 64$. A predictor can look good on this task by exploiting overlap alone, with no biological structure learned. This is not necessarily implementation leakage: it is a property of the declared representation and task. It becomes misleading when interpreted as evidence of long-range genomic understanding.

For variable-length tokens, comparing perplexities across tokenizers is generally invalid. The same raw string produces different numbers of prediction events. Report the summed negative log probability and the number of raw bases represented under a specified reversible coding and boundary convention. Tokenization rules must be fitted using permitted training data, not inferred from the held-out corpus.

2.5 Data families answer different questions

Natural genomic sequences test statistical structure in biological material under a chosen sampling distribution. Synthetic genomic-like sequences test designed regularities whose resemblance to nature must be stated. Independent random sequences are a sanity control. Algorithmic sequences isolate exact computation. Natural language, code and agent trajectories provide different kinds of context, targets and feedback.

A motif task may measure local recognition while saying little about long-term memory. A delay task can stress exact retrieval without biological realism. Agent trajectories introduce action-dependent observations and may require policy evaluation rather than ordinary iid classification. Good evaluation does not force these data types into one score; it states which uncertainty each one reduces.

Tokenizing genomic text in a digital network is not itself molecular computation. The companion DNA Computing book studies physical molecular representations and operations. Here nucleotide strings are data processed by software. Genomic architecture terminology does not supply evidence of biological fidelity, cross-domain transfer or general intelligence.

2.6 Split relationships before splitting rows

Random row assignment is unsafe when rows are related. In genomics, possible relations include overlapping coordinates, exact duplicates, reverse complements, homologous sequences, shared loci, species and phylogenetic proximity. The appropriate exclusion relation depends on the claim: unseen-window interpolation within a known genome is not the same question as transfer to an unseen species.

Similarity-aware splitting methods such as SpanSeq make this issue explicit.[14] No universal identity threshold guarantees independence for every biological task. A chromosome-held-out split can prevent coordinate overlap yet still leave homologous sequences across folds. A species split can still include close evolutionary relatives.

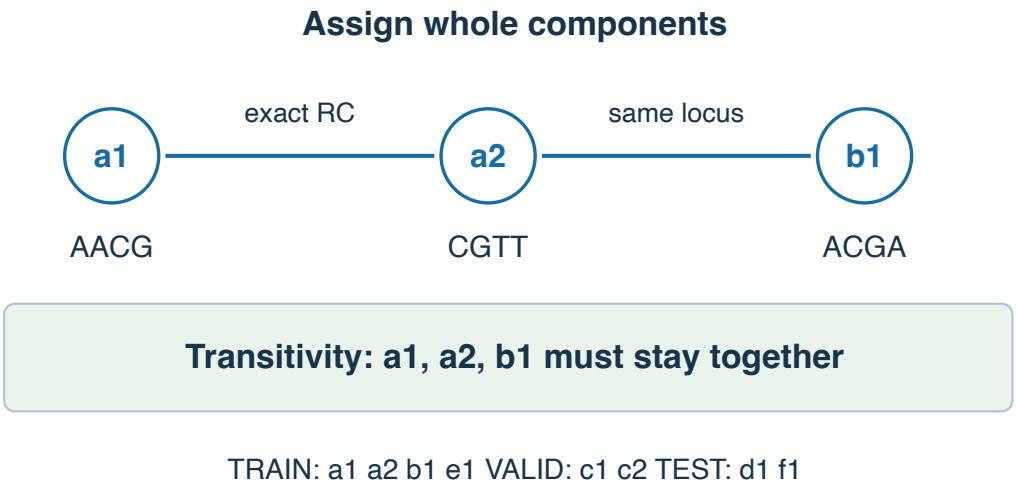


Figure 2.4: Conflict relations must be closed transitively before assignment. Record a1 is the exact reverse complement of a2; a2 and b1 share declared locus metadata. All three must remain together even though a1 and b1 need not match exactly. The displayed fold assignment is actual output of the reference splitter.

Construct a conflict graph $H = (R, C)$ over records. An edge means the two records must not cross folds. If folds have no conflict edge between them, every connected component lies entirely within one fold: along a path, membership must remain equal at each edge. This is why independently grouping duplicates and loci after an initial split is insufficient.

Our splitter unions two relations: exact/reverse-complement sequence identity, using $\min(s, \mathrm{RC}(s))$, and explicitly declared metadata groups. It assigns complete connected components with a deterministic seed-dependent order. Sorting stable IDs makes the result invariant to input row order. Large components can prevent exact target fold ratios; the code rejects fewer than three components rather than pretending to create three independent nonempty folds.

The guarantee is intentionally narrow. The code does not detect homology, coordinate overlap or phylogenetic proximity. Those relations must be computed or supplied before splitting. Sequence-specific preprocessing, augmentation and tokenizer training must also respect the final folds. A perfect conflict-graph algorithm cannot enforce an edge that was never recorded.

2.7 The random-data entropy control

Let the next target Y be independent of the permitted context C and uniform on V symbols. For any predictive distribution $q(\cdot \mid C)$ that depends only on permitted information,

$$\mathbb{E}[-\ln q(Y \mid C)] = \ln V + \mathbb{E}_C[D_{\mathrm{KL}}(U_V \| q(\cdot \mid C))] \tag{2.2}$$

$$\geq \ln V. \tag{2.3}$$

This follows by adding and subtracting $\ln(1/V)$ in the expected log score. If q assigns zero probability to a possible target, its loss is infinite, which does not violate the bound. For iid uniform DNA, the optimum is $\ln 4 \approx 1.386294$ nats/base, or exactly two bits/base.

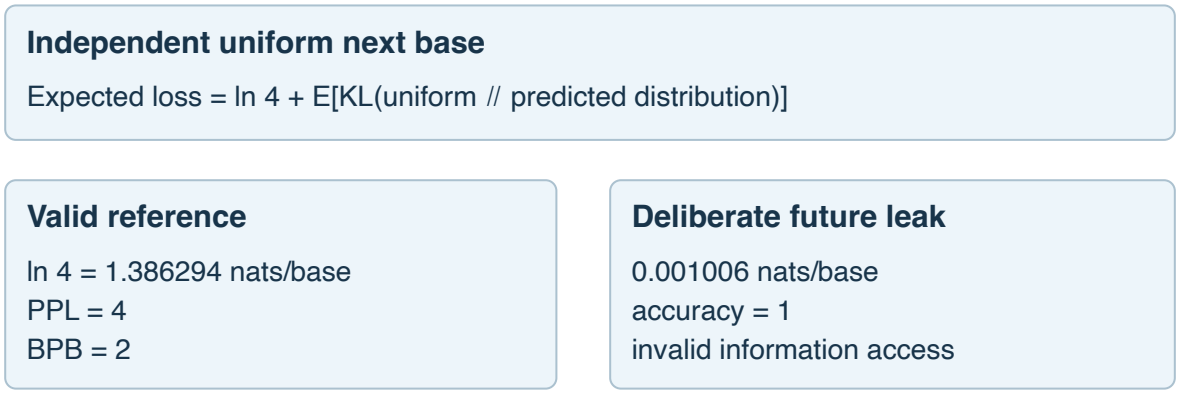


Figure 2.5: The iid entropy control separates expected valid prediction from invalid information access. Uniform logits give exactly $\ln 4$ loss for every supplied canonical target. The deliberately future-reading predictor has much smaller measured loss because the answer is available to it.

The bound is about expectation on fresh independent targets. A finite test sample can favor a nonuniform predictor by chance. Repeated selection on that test sample can exploit its noise. A network can also memorize random training labels,[17] which is compatible with being unable to predict new independent labels. Training loss below $\ln 4$ is therefore not, alone, a causality diagnosis.

A sustained large advantage on genuinely fresh iid data should trigger an audit of target shift, future access, overlapping-token construction, reuse of random sequences, class frequencies, and scoring masks. The entropy control is a falsification instrument under explicit assumptions, not a theorem that natural genomes are incompressible.

2.8 Intervene on future tokens

Suppose two inputs agree through position t. A causal output sequence should satisfy

$$f(u)_{\leq t} = f(u')_{\leq t} \quad \text{whenever} \quad u_{\leq t} = u'_{\leq t},$$

with suitable numerical tolerance in an implementation. Change only the suffix, recompute, and measure

$$\Delta_t = \max \left| f(u)_{\le t} - f(u')_{\le t} \right|.$$

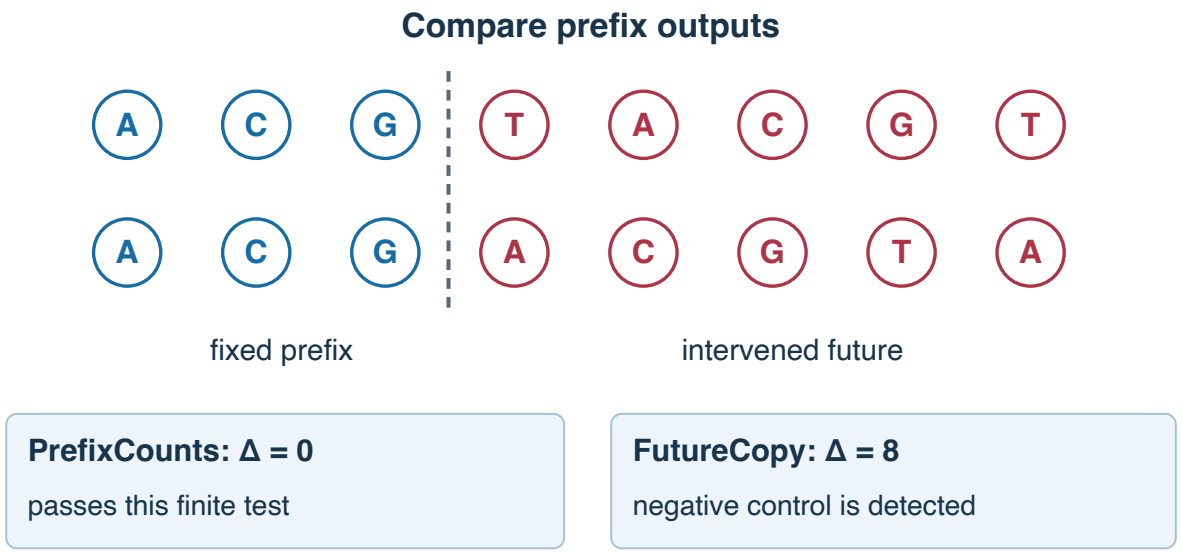


Figure 2.6: A finite behavioral intervention. The prefix remains fixed, the future changes, and prefix logits are compared. The displayed toy results use deterministic float64 CPU references; every tested boundary gives zero for PrefixCounts and eight for FutureCopy.

The repository already provides `prefix_intervention_error`. It adds one modulo vocabulary size to every suffix token, runs the model in evaluation mode, and compares prefix logits. This chapter hardens that helper: nonfinite values cannot pass, malformed outputs are rejected, and original per-module train/eval modes are restored even if evaluation raises.

The helper is intended for deterministic, stateless forward calls of the supplied reference models. A stateful runtime must reset or clone its state for the two calls. Stochastic modules require controlled randomness or a distributional comparison; disabling dropout is not a cure for arbitrary hidden randomness. For large mixed-precision models, tolerance must be justified by the numeric path, scale and a no-intervention control, not chosen after seeing a violation.

The following wrapper tests every nontrivial boundary. It is extracted from `examples/deep/ch02.py`; the full source supplies its imports and the shared helper. The float64 reference tolerance is 10^{-12}, not a universal tolerance for other model implementations.

```
def intervention_suite(model, ids, atol=1e-12):
    if ids.ndim != 2 or ids.shape[0] < 1 or ids.shape[1] < 2:
        raise ValueError("nonempty batch and prefix/future required")
    if ids.dtype != torch.long or ids.min() < 0 or ids.max() >= 4:
        raise ValueError("DNA token indices required")
    if not math.isfinite(atol) or atol < 0:
        raise ValueError("finite nonnegative tolerance required")
    # The existing helper performs +1-mod-V suffix interventions and restores mode.
    errors = [prefix_intervention_error(model, ids, prefix_length=t, vocab_size=4)
              for t in range(1, ids.shape[1])]
    if not all(math.isfinite(e) for e in errors):
        raise ValueError("non-finite intervention output cannot pass")
    return {"errors": errors, "maximum": max(errors), "passes": max(errors) <= atol,
            "atol": atol, "dtype": "float64 reference on CPU"}
```

Passing a finite set of interventions is necessary evidence, not a universal proof. A noncausal function can be insensitive to the particular suffix change; for example, it may depend on a suffix property preserved by that transformation. Test different suffixes, boundaries, sequence lengths, batches and masking modes, and inspect the computational graph. A deliberately leaky negative control establishes that the test can detect at least one relevant failure.

2.9 Why gradients alone are not enough

A derivative such as $\partial f_t / \partial u_{t+1}$ probes local sensitivity where that derivative exists. Tiny weights, saturation, quantization, discrete branching, detached tensors and poorly conditioned numeric paths can make a local gradient zero or uninformative while finite future changes alter behavior.

For a simple counterexample, a function can return one value when a future token equals A and another otherwise. The token ID is discrete; a derivative with respect to that ID is not the behavior we need to test. A detached future-dependent tensor provides another software counterexample: automatic differentiation sees no gradient path even though the forward result still depends on the future.

Finite intervention directly tests the relevant behavior, but it too samples only selected cases. The strongest causal argument combines a structural information-flow proof with executable interventions and negative controls. Neither a small gradient nor one zero difference should be promoted into an unconditional guarantee.

2.10 A CPU reference experiment with a known failure

The registered child record `EVO-EXP01-CH02-SMOKE` extends the existing EVO-EXP01 pipeline experiment. Its question, seeds and falsifiers were written before execution. No optimizer steps, trained DOGMA model, Hermon model, or GPU run are involved.

Run `python examples/deep/ch02.py` from the repository root with PyTorch installed. The recorded reference environment uses torch 2.10.0 on CPU. The JSON output includes individual seeds, split membership, metrics, intervention differences, an exact delay example and the specification hash.

For each seed 11, 29 and 47, generate eight independent sequences of 33 uniform DNA tokens. Inputs and next-base targets each have 32 positions. Uniform logits score all 256 targets at exactly $\ln 4$. `PrefixCounts` uses cumulative observed base counts plus one pseudocount per class and emits their logarithms. Softmax normalizes these into a predictive distribution; the reference has no fitted parameters.

The deliberately invalid `FutureCopy` reads input $j + 1$ to predict target j and places logit 8 on that symbol, zero elsewhere. Its probability on the target is

$$\frac{e^8}{e^8 + 3}, \qquad \mathcal{L} = \ln(1 + 3e^{-8}) \approx 0.001005882.$$

The last output is excluded because no next input token exists there. Consequently this negative control scores 248 targets, while the ordinary baseline summaries score 256. These are diagnostic displays, not a fair model ranking; comparative averages must use the same mask and denominator.

Seed	Uniform nats/base	PrefixCounts nats/base
11	1.386294	1.462666
29	1.386294	1.419432
47	1.386294	1.454510

The finite-intervention suite tests seven boundaries on a fixed eight-token input. PrefixCounts gives $\Delta_t = 0$ at every boundary; FutureCopy gives $\Delta_t = 8$. Excellent leaked-target loss and failed causality are therefore visible in the same executable example.

The result artifact records the specification hash and torch version. Seeds here vary synthetic evaluation data, not training initialization. The parent EVO-EXP01 training study remains NOT RUN. Reproducing a CPU reference diagnostic is a smaller achievement than reproducing a trained-model benchmark, and it is useful precisely because its truth conditions are explicit.

2.11 Capacity is a vector of confounds

Equal parameter count does not imply equal representational width, memory access, train-time work or inference state. Report trainable parameters, embedding width, depth, state dimension, attention heads, KV heads, FFN width, context length and cache/state precision where applicable. Also report shared or tied parameters and any externally retrieved memory.

Matched quantity	Confound controlled	Still uncontrolled
parameters	stored trainable scalars	state, width, data, FLOPs
width	representation dimension	depth, parameters, history
state / cache bytes	runtime memory budget	precision, access semantics
training tokens	data exposure	compute per token
training FLOPs	arithmetic budget	hardware utilization
wall time	time on specified system	optimization / kernel maturity

Figure 2.7: Different matching strategies control different variables. Parameter matching, width matching, state/cache matching, token matching, compute matching and wall-time matching are separate experimental choices, not interchangeable definitions of fairness.

For an intentionally stripped-down example with vocabulary V, width d, untied input/output embeddings and no biases, a single recurrent matrix gives

$$P_{\text{rec}} = 2Vd + d^2.$$

A toy attention block with four d-by-d projections and a two-matrix FFN of width m gives

$$P_{\text{attn}} = 2Vd + 4d^2 + 2dm.$$

At $V = 4, d = 16, m = 32$, the counts are 384 and 2,176. These are declared toy formulas, not the parameter counts of a complete GRU, DOGMA model or Transformer. Their purpose is to show how the same width leaves different parameter budgets.

Conversely, solving for widths under one common parameter budget produces different representation dimensions. Fixed recurrent state and addressable KV history also provide different memory interfaces even when their byte budgets match. A useful study therefore uses more than one controlled comparison and reports the residual mismatches rather than claiming one scalar has removed every confound.

2.12 Baselines, removal tests and transplants

Begin with a trivial baseline: uniform class probabilities or the majority class under the correct split. Add a task-aware heuristic, such as smoothed prefix frequencies. Then include standard architecture baselines with competent tuning. Potential DOGMA comparisons include GRU/LSTM, state-space, Mamba-like and RWKV-like families; potential Hermon DNA comparisons include ordinary Transformers and relevant genomic models. A family's name does not establish that a particular implementation is strong or task-compatible.[10]

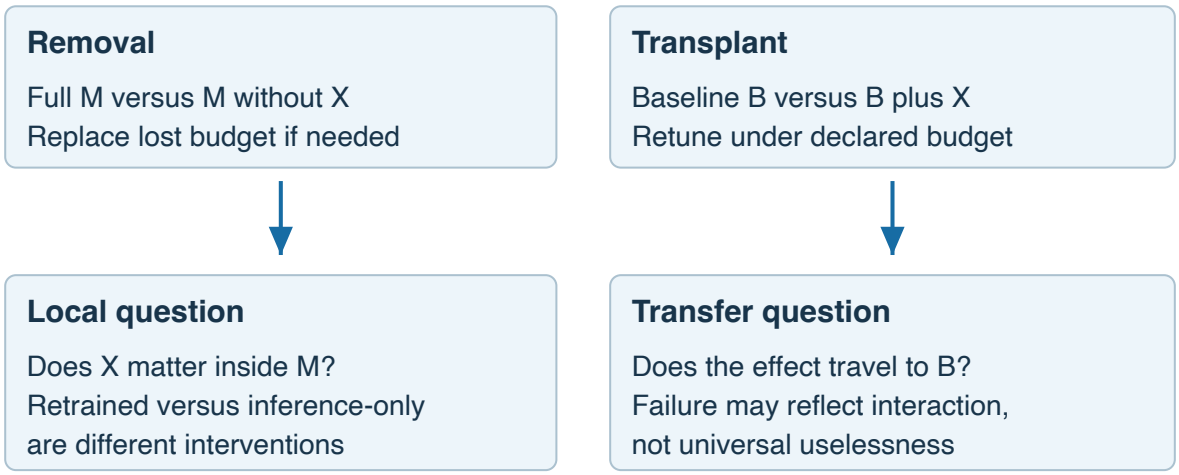


Figure 2.8: Removal and transplant experiments ask different questions. Removing a component tests its contribution inside a particular system. Adding it to another baseline tests whether an effect transfers. Budget replacement, retraining policy and paired uncertainty determine what either contrast can support.

Let Q be a higher-is-better metric and X a component. The difference $Q(M) - Q(M \setminus X)$ is interpretable only after defining the intervention. Removing X at inference from a trained system measures reliance under that disruption. Retraining without X measures performance of a different learning procedure. Replacing X with an equally sized generic block asks whether its structure matters beyond budget.

A transplant compares $B + X$ with B. A transferred improvement supports a portable contribution under those conditions. Failure to transfer can reflect an interaction with the original architecture, optimization or task; it does not prove universal uselessness. Use paired data splits and seeds when appropriate, and compute uncertainty on per-pair differences rather than comparing overlapping marginal error bars by eye.

2.13 Seeds reveal outcomes, not just a mean

For scores $m_1, \ldots, m_k$, preserve individual values and report summaries suited to the question:

$$\bar{m} = \frac{1}{k}\sum_i m_i, \qquad s = \sqrt{\frac{\sum_i (m_i - \bar{m})^2}{k-1}}.$$

The sample standard deviation describes dispersion among these runs. It is not automatically a confidence interval for an architecture's universal performance. Data sampling, initialization, optimizer stochasticity, tuning choices and hardware behavior can contribute different variance sources.[18]

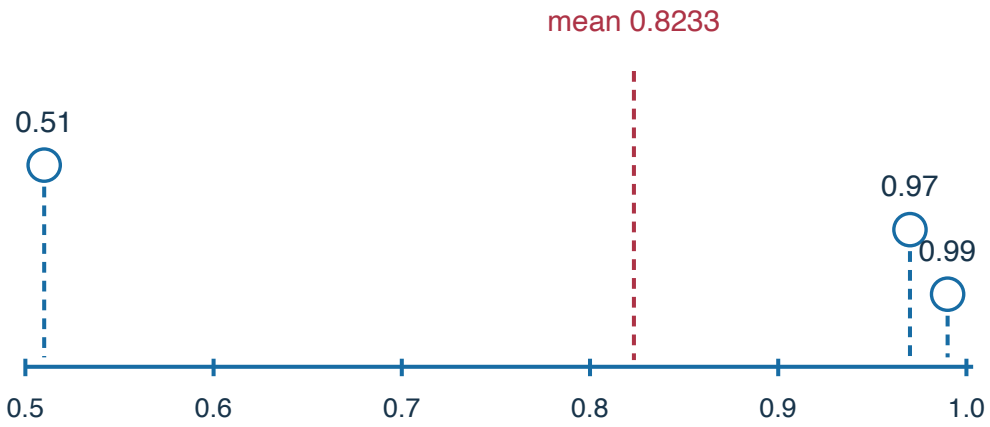


Figure 2.9: The supplied illustrative values 0.51, 0.97 and 0.99 have mean 0.8233, median 0.97 and sample standard deviation about 0.2715. Their individual positions show what the mean hides. These are not measured training runs and are too few to establish bimodality.

One possible explanation is one failed optimization run and two successful runs, but the numbers alone do not prove that explanation. Inspect learning curves, failure traces and additional prespecified runs. If optimization failure is part of the method's behavior, silently discarding that run changes the estimand. Distinguish infrastructure failures from valid but poor outcomes using rules established before results are seen.

Fixed seeds assist rerunning one environment; they do not guarantee identical outcomes across releases, devices or every nondeterministic operation.[21] Record the implementation, environment and input hashes as well as the seed. For genomic test uncertainty, resampling highly related windows independently can understate uncertainty; the resampling unit should reflect the data-generating relationships.

2.14 Loss, perplexity and bits: keep the denominator

For target distribution p and prediction q,

$$H(p, q) = -\sum_x p(x) \ln q(x).$$

For observed target indices, empirical cross entropy is mean negative log probability. With N scored events, $\mathcal{L} = -N^{-1}\sum_i \ln q(y_i \mid c_i)$. Perplexity is $\exp(\mathcal{L})$: the exponential of the average surprise, not an average of per-batch perplexities.

If one event represents one base, bits per base are $\mathcal{L}/\ln 2$. More generally, if scored tokens represent B raw bases,

$$\mathrm{BPB} = \frac{-\sum_i \ln q(y_i \mid c_i)}{B \ln 2}.$$

Boundary symbols, padding, ambiguous bases and overlapped encodings need a declared convention. An overlapping-window token score is not automatically a lossless coding rate per raw base.

Suppose one batch has ten scored positions with mean loss 1 and another has ninety with mean loss 2. The combined loss is $(10 + 180)/100 = 1.9$, not 1.5. Compute total negative log likelihood and total valid-target count, then divide and exponentiate. For distributed evaluation, aggregate both totals across workers before the final conversion.

The reference accepts finite logits of shape $[B, T, V]$ and integer targets of shape $[B, T]$. PyTorch cross entropy applies the log-softmax internally; passing normalized probabilities as though they were logits changes the computation.[20]

```
def token_metrics(logits, targets):
    """Unweighted next-base loss; all supplied positions are scored."""
    if logits.ndim != 3 or targets.shape != logits.shape[:2] or targets.numel() == 0:
        raise ValueError("expected nonempty logits [B,T,V] and targets [B,T]")
    if not torch.isfinite(logits).all():
        raise ValueError("non-finite logits")
    if targets.dtype != torch.long or targets.min() < 0 or targets.max() >= logits.shape[-1]:
        raise ValueError("target indices outside vocabulary")
    loss = F.cross_entropy(logits.reshape(-1, logits.shape[-1]), targets.reshape(-1))
    nats = float(loss)
    return {"nats_per_base": nats, "perplexity": math.exp(nats),
            "bits_per_base": nats / math.log(2),
            "accuracy": float((logits.argmax(-1) == targets).double().mean())}
```

This small function scores every supplied position and has no padding argument. A production evaluator needs an explicit valid-target mask and a declared empty-mask policy. Returning a convenient zero for no scored targets would make a missing evaluation look perfect.

2.15 Accuracy, calibration and systems measurements

Accuracy counts correct argmax decisions; it does not distinguish confidence 0.51 from 0.99 on a correct binary decision. For imbalanced classification, precision and recall separate false-positive and false-negative behavior, and

$$F_1 = \frac{2TP}{2TP + FP + FN}$$

requires a declared policy when the denominator is zero. Macro, micro and per-class reporting answer different aggregation questions. Thresholds and any calibration transform must be chosen using permitted development data, not optimized on the final test labels.

Calibration asks whether stated probabilities correspond to observed frequencies under the evaluated population. It is distinct from accuracy;[19] a high-accuracy classifier can be overconfident. Calibration summaries also depend on binning, sample size and distribution shift, so report their construction rather than treating one number as an invariant property.

Evidence axis	Measure	Required context
predictive quality	NLL, BPB, F1, retrieval accuracy	same task / targets / split
confidence quality	calibration, proper log score	population + fitted calibration
systems performance	latency, throughput, memory	hardware / batch / precision
engine fidelity	reference-output difference	same weights / inputs / dtype

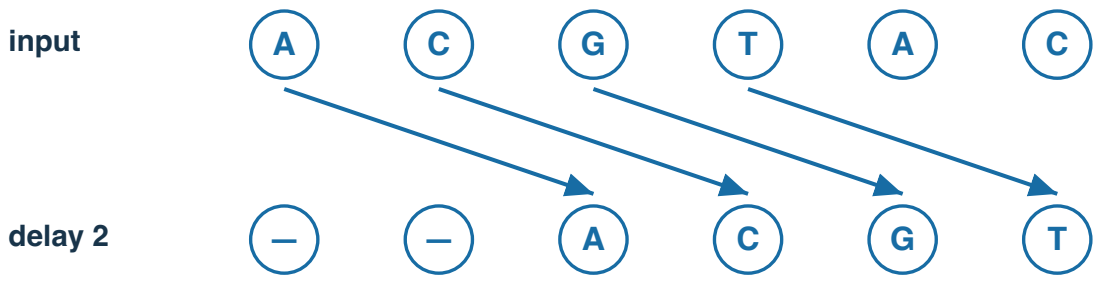
Figure 2.10: Predictive quality, confidence quality, systems performance and engine fidelity are distinct axes. A faithful optimized engine can execute a weak model correctly; a strong model can be executed incorrectly. Neither capability nor parity can be inferred from speed alone.

Retrieval accuracy should specify what is retrieved, top-k versus exact match, and the candidate pool. Latency needs batch size, sequence lengths, prefill versus decode, synchronization and warm-up policy. Throughput needs a unit such as scored bases or generated tokens per second. Runtime memory needs peak versus steady-state and inclusion of weights, cache, workspace and allocator overhead.

Report a vector such as (Q, L, U, R) for quality, latency, throughput and runtime memory. Engine parity is an additional correctness axis: compare the same weights and inputs against a trusted implementation under justified numeric tolerances. Do not infer latency from a parameter formula or an architecture label; this chapter runs no performance benchmark.

2.16 Exact synthetic tasks before learned solutions

An algorithmic task should have an exact oracle before training. For delay ℓ, define $y_t = x_{t-\ell}$ only at positions $t \geq \ell$. The first ℓ positions are unscored, not assigned a convenient default label.



Zero-based scored positions 2…5. First two positions are unscored, not default-A targets.

Figure 2.11: Delay-two oracle on six inputs. Each scored target points to an exact earlier symbol. The first two outputs are excluded because their source positions do not exist. This is a past-retrieval task, not next-base language modeling.

For `ACGTAC` and lag two, scored positions 2,3,4,5 have targets `ACGT`. On iid four-symbol inputs, a context-free guess has expected accuracy 1/4 on these targets, while the exact oracle reaches one. The full-history Bayes predictor can be perfect, so the random next-base entropy lower bound does not apply to this different task.

Test lag one, near-maximum lag, repeated symbols, invalid lag, different lengths and target alignment. An oracle is necessary but not sufficient: its generator can still leak labels through length, frequency, position or output history. A balanced binary parity task, for example, should not accidentally make one class dominate through its sampling scheme.

2.17 Balanced memory tasks and shortcut audits

Task	Required memory	Shortcut to exclude
exact delay / copy	recover specified past symbol	fixed position / output leak
parity / running sum	compressed statistic	class imbalance / length
associative retrieval	key-conditioned lookup	key frequency / duplicates
pointer chasing	sequential dependent lookup	one-hop proxy
motif / conditional routing	pattern and state	future context / motif count

Figure 2.12: A memory suite separates exact retrieval, compressed statistics, associative lookup, dependent lookup and local/state-conditioned recognition. Different tasks expose different memory interfaces. No result for one row establishes superiority on the whole matrix.

Copy and delay test recovery of specified past symbols. Parity and running sum test compact sufficient statistics. A finite-state-machine task tests state transitions. Associative retrieval tests key-conditioned recall and must define duplicate-key semantics, such as first-write, last-write or unique-key-only. Pointer lookup is one addressed retrieval; pointer chasing requires repeated dependent retrievals and should vary chain depth. Motif recognition and conditional routing test pattern-dependent behavior but can admit count or position shortcuts.

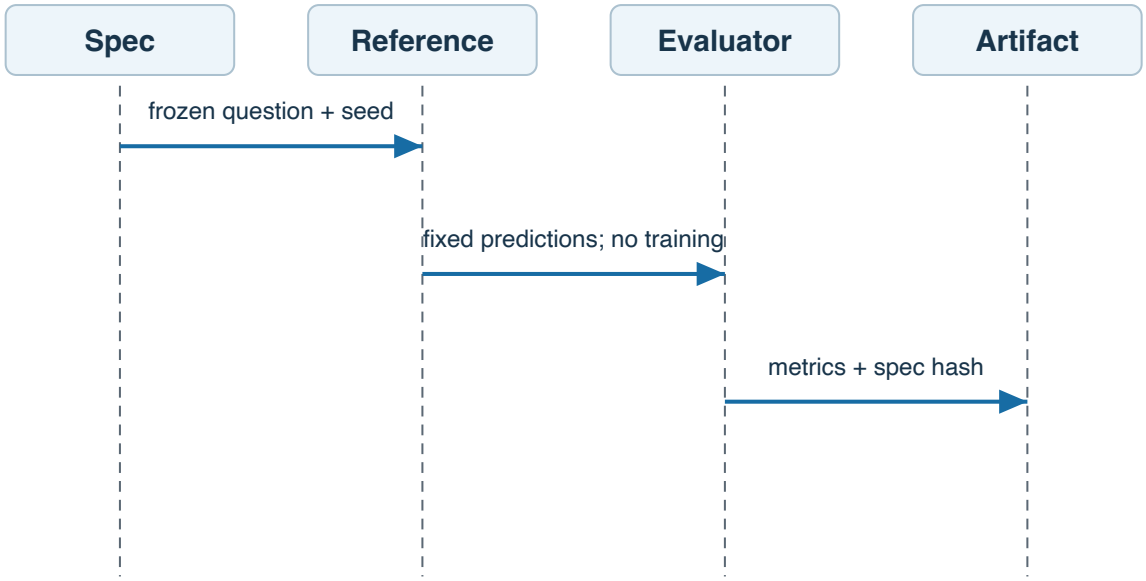
For each generator ask whether position, sequence length, token frequency, reused examples, output history or future context reveals the answer. Permuting nuisance positions while preserving the correct answer can expose position shortcuts. Balancing labels conditional on length can remove a length proxy. Holding out key identities or relationship components tests a different claim from holding out randomly generated rows.

DOGMA-style carried state and Hermon-style addressable history should face both compressed-statistic and exact-retrieval tasks. A fixed-state model may represent a running statistic compactly without retaining every token. A history-addressable model may retrieve an old symbol conveniently while paying cache costs. Testing only the task favored by one interface would answer a narrower question than a general memory comparison.

For length extrapolation, train at lengths up to T_{train} and evaluate beyond it, but specify what changes: absolute delay, relative delay, number of keys, pointer depth, distractors or output length. Keeping delay fixed while increasing irrelevant suffix length is not the same stress as increasing retrieval distance. One successful synthetic extrapolation does not establish general intelligence.

2.18 Budgets and the experiment lifecycle

Before a model comparison, fix or report tokens seen, optimizer steps, batch size, optimizer and learning-rate schedule, tuning budget, hardware, precision and stopping rule. Equal steps with different batch sizes are not equal data exposure. Equal tokens are not equal FLOPs. Equal wall time can confound architecture with implementation maturity.



UML-style sequence for EVO-EXP01-CH02-SMOKE. Parent training remains NOT RUN.

Figure 2.13: UML-style sequence for the actual Chapter 2 child record: a frozen specification drives fixed reference predictions, evaluation and a hashed result artifact. There is no trainer message because this diagnostic performs no training. The parent training study remains unrun.

The existing research program defines EVO-EXP01 through EVO-EXP05. This chapter adds a scoped child to EVO-EXP01, not a competing experiment registry. Its question is whether the reference pipeline distinguishes causal prediction from deliberate future access while keeping declared record conflicts inside folds. Falsifiers include a nonzero causal difference above the declared reference tolerance, an undetected negative control, cross-fold declared conflicts, and accepting nonfinite outputs.

Freeze the specification before running, and record amendments as new versions rather than rewriting the initial hypothesis to fit the observed result. A hash binds a result to exact specification bytes; it does not prove the specification was scientifically wise. Mechanism claims still need valid tasks, controls, measurements and skeptical alternatives.

The next chapter is *Differentiation, optimization and tensor programs*. It derives how parameters are updated and how tensor computations implement those updates. Later chapters can then introduce sequence models without losing the distinction between a new mechanism, a correctly executed program and convincing evidence of improvement.

Key ideas

- For independent uniform targets, no permitted predictor can beat $\ln V$ in expectation. A sustained advantage on fresh data is a reason to audit the pipeline.
- A triangular mask is only correct relative to a declared target shift. Padding, packing, pooling and retrieval can each leak information.
- Report the summed negative log-likelihood and the number of raw bases it covers. Perplexities computed under different tokenizers are not comparable.
- Parameter, width, state, token, compute and wall-time matching are separate choices. No single scalar removes every confound.

2.19 Exercises

1. Derive Equation 2.3. Explain why it does not prohibit fitting a finite random training set.
2. Write inputs and targets for `ACGTA`. Which input positions may output 2 use under this chapter's zero-based convention?
3. Derive the next-token entropy of overlapping three-mers on iid uniform DNA. Compare with nonoverlapping three-mers.
4. Build a three-record transitive conflict example. Explain why splitting exact duplicates and metadata groups independently is insufficient.
5. Implement a future-dependent function with zero automatic gradient to its future input. Which finite intervention would detect it?
6. Why might adding one modulo four to every suffix token miss a future-dependent function? Propose an additional intervention.
7. At $V = 4, d = 16, m = 32$, derive the two toy parameter counts. List three remaining confounds after matching either count.
8. Interpret the supplied values 0.51, 0.97 and 0.99. What additional evidence is needed before calling the distribution bimodal?
9. Combine ten scored targets with mean loss 1 and ninety with mean loss 2. Convert the aggregate to perplexity and BPB for base tokens.
10. Define an associative-retrieval oracle with duplicate keys and audit two shortcuts. Specify an extrapolation test.
11. Design a removal and a transplant experiment for a proposed routing component, including budgets and a falsifier.
12. A fast engine differs from reference logits while the model's task accuracy is high. Which claims remain supported, and which fail?

2.20 Worked solutions

1. Conditional on each permitted context, add and subtract $-\ln(1/V)$ in the uniform target expectation. The residual is $D_{\mathrm{KL}}(U_V \| q)$, which is nonnegative. Memorization uses information about already observed training targets; the bound concerns independent fresh targets under the allowed context.
2. Inputs are `ACGT`, targets `CGTA`. Output 2 predicts T and may use input positions 0,1,2, namely A,C,G. Input position 3 is T and would reveal that target. A diagonal-inclusive causal mask is consistent with this shift.
3. Given the current overlapping three-mer, two bases of the next token are known and one is uniform, so conditional entropy is $\ln 4$. A fresh nonoverlapping three-mer contains three independent bases and has entropy $3 \ln 4 = \ln 64$. The token-level numbers concern different amounts of new information.
4. Let a1 and a2 be reverse complements and a2 and b1 share a locus. The first relation requires a1 and a2 together; the second requires a2 and b1 together; hence all three belong together. Assigning the two group systems independently can satisfy one relation while violating the other.

5. A forward computation using a detached future embedding can affect the output while cutting the automatic-gradient path. Change the actual future token and recompute the prefix output from a reset state. The finite output difference, not the detached gradient, exposes the dependence.

6. A function depending on equality of two future tokens is unchanged when both are incremented modulo four. Change only one suffix position, use independently resampled suffixes, and vary intervention lengths. Even these are finite checks; structural analysis is still needed.

7. Embeddings contribute $2Vd = 128$. The recurrent matrix adds 256, totaling 384. Four attention projections add 1,024 and the FFN adds another 1,024, totaling 2,176. Matching parameters leaves memory access, state/cache bytes, depth, optimization and compute per token uncontrolled.

8. The mean is 0.8233, median 0.97 and sample standard deviation about 0.2715. One poor outcome is visible, but a three-point sample cannot establish the generating distribution's modes or cause. More prespecified runs, learning curves and failure diagnostics are needed.

9. Aggregate loss is 1.9 nats/base. Perplexity is $e^{1.9} \approx 6.686$ and BPB is $1.9/\ln 2 \approx 2.741$. The loss exceeds the uniform-DNA reference because these hypothetical predictions are worse calibrated to their targets. Exponentiating each batch first and averaging gives a different, inappropriate aggregate.

10. One valid rule is last-write-wins: scan key–value writes in order, overwrite that key's stored value, then return the final value for the query key. Generate duplicate keys deliberately and define missing-query behavior. Balance query identity and position; prevent value frequency from predicting the answer. Extrapolate number of stored keys and retrieval distance separately.

11. Compare a full system with a retrained budget-matched generic routing replacement, using paired splits/seeds and equal declared tuning resources. Separately add the proposed component to a standard baseline with a matched added budget. A prespecified lack of improvement on the mechanism-targeted task, or gains explained by a generic replacement, weakens the proposed mechanism claim.

12. High task accuracy supports only the evaluated model/protocol result, subject to its own controls. Engine parity fails if differences exceed justified tolerance on the stated contract. Speed can still be measured, but it does not establish a correct accelerated implementation. Test the engine on identical weights and inputs before attributing model behavior to faithful execution.

CHAPTER 3

Differentiation, optimization and tensor programs

In this chapter you will

- read an equation as a tensor program with explicit shapes and broadcasts
- derive softmax cross-entropy and affine-layer gradients by hand and check them against autograd
- explain why finite-difference error first falls and then rises as the step shrinks
- implement microbatch accumulation that reproduces the declared full-batch objective

3.1 Differentiate one scalar program before a tensor network

Chapter 2 separated a declared objective from evidence that it measures the intended capability. Now freeze the objective and ask a local question: which operation transmits a change in one parameter to the loss?

Begin with two scalars, not an automatic differentiation interface:

$$x = 2, \qquad w = 3, \qquad a = wx, \qquad L = \tfrac{1}{2}a^2. \tag{3.1}$$

The forward values are $a = 6$ and $L = 18$. At the multiply, either input can change the result. At the square, the local rate of loss change is a. The reverse calculation must follow those dependencies, not the order in which variable names happen to appear in a source file.

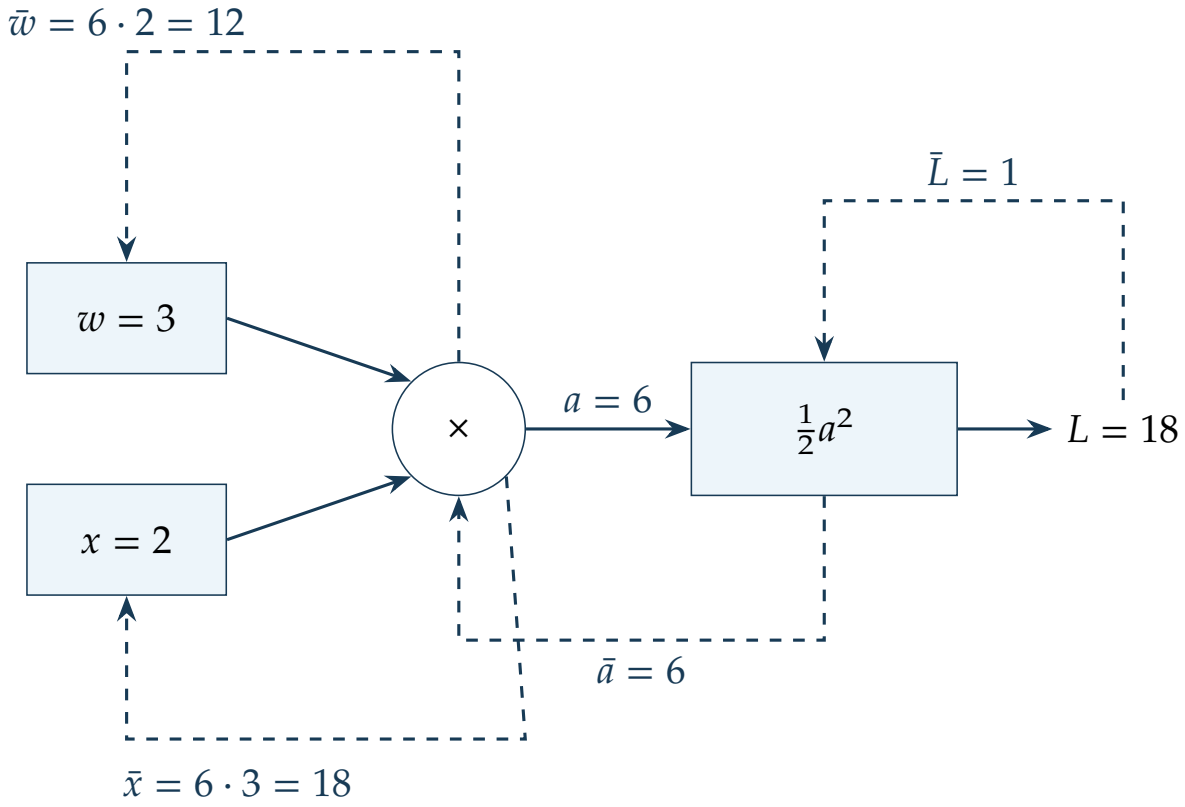


Figure 3.1: Solid arrows carry forward values. Dashed arrows carry reverse sensitivities. Both inputs enter multiplication independently; the reverse pass produces a separate derivative for each input.

Write a bar over a quantity to denote the derivative of the final loss with respect to it. Starting from $\bar{L} = 1$, the chain rule gives

$$\bar{a} = \bar{L}\, a = 6, \tag{3.2}$$

$$\bar{w} = \bar{a}\, x = 12, \tag{3.3}$$

$$\bar{x} = \bar{a}\, w = 18. \tag{3.4}$$

These are not three gradients that should be averaged. They are derivatives with respect to three different variables. The input x can be fixed data while its mathematical derivative is still well defined. Computing $\bar{x}$ does not imply that the optimizer updates the data.

Prediction problem. Update only w with learning rate .01. What are the new weight and loss for the same fixed input?

Worked answer. The update is $w' = 3 - .01(12) = 2.88$. The new loss is $\frac{1}{2}(2 \cdot 2.88)^2 = 16.5888$. The loss decreases in this particular smooth example. A computed gradient does not guarantee improvement for an arbitrary finite learning rate or on a different evaluation dataset.

We can check the derivative without reusing the chain-rule program:

$$\frac{L(w+h) - L(w-h)}{2h} = \frac{2(w+h)^2 - 2(w-h)^2}{2h} = 4w = 12. \tag{3.5}$$

The equality is exact in real arithmetic for this quadratic and nonzero h. Floating-point evaluation introduces subtraction and rounding error. Both evaluations must use the same input and state; changing either means we are subtracting values of different functions.

The matrix derivation that follows is this same calculation with shared parameters, indexed sums and explicit shapes. A broadcast bias receives contributions from several rows, so its

reverse rule sums those contributions. A nonlinear operation multiplies by its local derivative. Tensor notation compresses these relationships; it does not replace their explanation.

3.2 From a declared objective to a local change

Chapter 2 asked what information a model may use and what evidence its score supports. We now take the objective as declared and ask a narrower question: how does the implemented loss change when a parameter changes?

A gradient answers that local question. It does not certify a dataset split, prove that the loss measures the intended capability, or guarantee that the next optimization step improves held-out performance. A perfectly differentiated leaking model is still leaking. A perfectly differentiated wrong objective is still wrong.

We will use a small, smooth network with a fully specified numerical fixture. There is no DOGMA or Hermon architecture hidden behind the example. The point is to derive and test operations that later architectures must also implement correctly.

3.3 Read an equation as a tensor program

Let $X \in \mathbb{R}^{B \times D}$ hold one input per row. Define

$$A = XW + \mathbf{1}_B b^\top, \quad H = \tanh A, \tag{3.6}$$

$$Z = HU + \mathbf{1}_B c^\top, \tag{3.7}$$

where $W \in \mathbb{R}^{D \times H_d}$, $b \in \mathbb{R}^{H_d}$, $U \in \mathbb{R}^{H_d \times V}$, and $c \in \mathbb{R}^V$. Here H_d denotes hidden width; the capital tensor H denotes hidden activations. The explicit outer products reveal the broadcast: the same bias contributes to every row.

The code writes `x @ w + b`, then `torch.tanh`, then `hidden @ u + c`. Broadcasting is convenient syntax, not a new mathematical rule. Each target y_i is an integer in $\{0, \dots, V-1\}$. The row logits Z_i parameterize a categorical distribution.

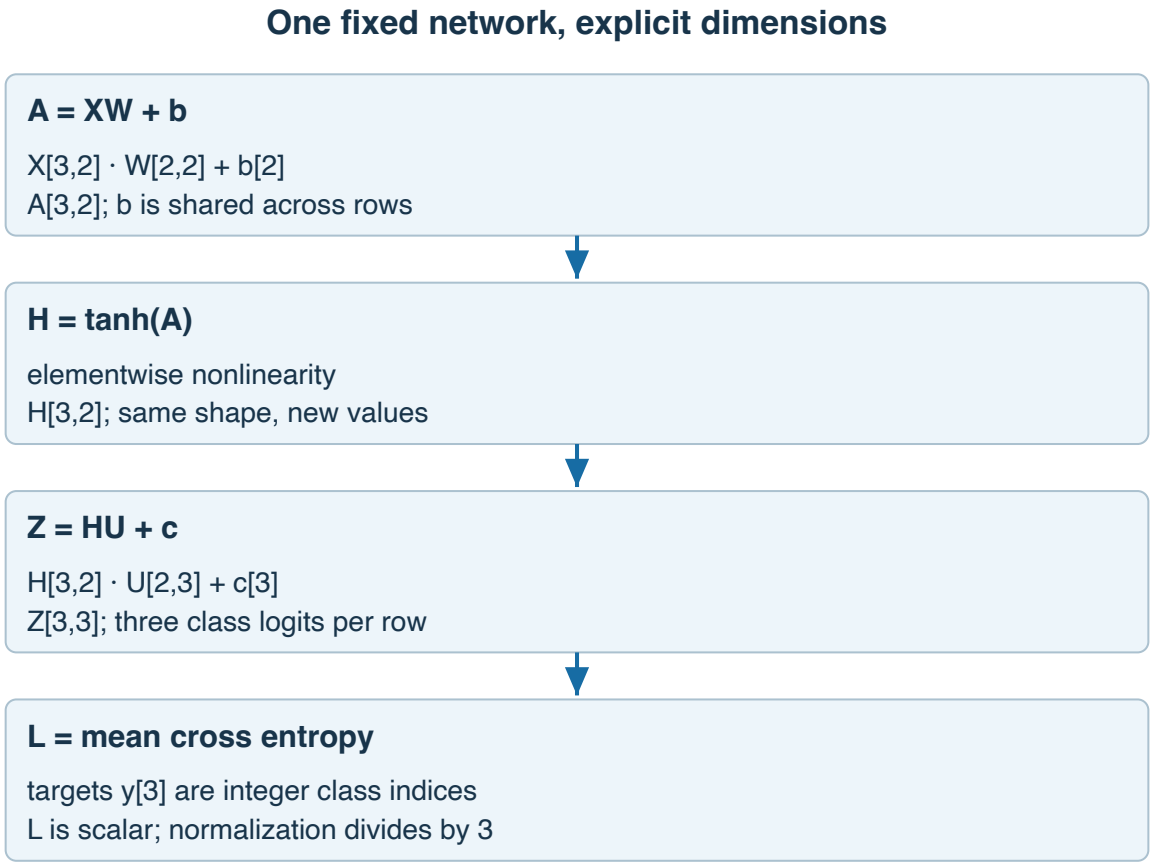


Figure 3.2: The fixed fixture uses $B = 3, D = 2, H_d = 2, V = 3$. This is a generic three-class demonstration, not DNA-base prediction. The classification objective and the tensor shapes are explicit and independently checkable.

Our fixture is

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{pmatrix}, \quad y = (0, 2, 1), \tag{3.8}$$

$$W = \begin{pmatrix} .2 & -.1 \\ .4 & .3 \end{pmatrix}, \quad b = (.05, -.02), \tag{3.9}$$

$$U = \begin{pmatrix} .3 & -.2 & .1 \\ -.4 & .2 & .5 \end{pmatrix}, \quad c = (.01, -.03, .02). \tag{3.10}$$

Every reference tensor is CPU float64. That restriction makes the diagnostic contract narrow and repeatable; it is not a recommendation that all production training use float64.

3.4 Start the reverse pass at the loss

Let $Q_{ij} = \exp Z_{ij} / \sum_k \exp Z_{ik}$. With no class weights or ignored labels, mean cross entropy is

$$L = \frac{1}{B} \sum_i \left(\log \sum_j e^{Z_{ij}} - Z_{i,y_i} \right). \tag{3.11}$$

For one row, differentiating the log-sum-exp gives Q_{ij}; differentiating the selected target logit subtracts one exactly at the target class. Therefore

$$E_{ij} := \frac{\partial L}{\partial Z_{ij}} = \frac{Q_{ij} - \mathbf{1}[j = y_i]}{B}. \tag{3.12}$$

The division by B comes from the chosen mean objective. It does not belong to matrix multiplication, bias broadcasting, or the definition of a derivative. Moving that division carelessly between code components can change the effective update. PyTorch's CrossEntropyLoss documentation specifies that its input is logits and details reductions and target conventions. Our hand derivation uses only the unweighted, fully scored case above.

A useful check follows immediately. Since each row of Q sums to one and each one-hot target does too, $\sum_j E_{ij} = 0$. Adding one common constant to every logit in a row does not change its predicted probabilities; the row gradient has no component in that invariant direction.

3.5 Derive each matrix gradient, including shared biases

For the projection $Z = HU + \mathbf{1}c^\top$, its differential is

$$dZ = (dH)U + H(dU) + \mathbf{1}(dc)^\top. \tag{3.13}$$

Write the scalar loss differential as $dL = \operatorname{tr}(E^\top dZ)$. Rearranging the three terms to pair each parameter differential with its coefficient gives

$$\nabla_U L = H^\top E, \qquad \nabla_c L = \sum_i E_i, \qquad D_H := \nabla_H L = EU^\top. \tag{3.14}$$

Check dimensions: $H^\top E$ has shape $H_d \times V$, exactly the shape of U. The sum over rows has length V, exactly the shape of c. If a proposed derivative has the wrong shape, it cannot be the derivative of that parameter under this convention. Correct shapes are necessary but not sufficient: a mistaken transpose can sometimes preserve dimensions when widths happen to match.

Because tanh acts elementwise,

$$D_A = D_H \odot (1 - H \odot H). \tag{3.15}$$

Applying the same affine-layer argument at the input gives

$$\nabla_W L = X^\top D_A, \qquad \nabla_b L = \sum_i (D_A)_i. \tag{3.16}$$

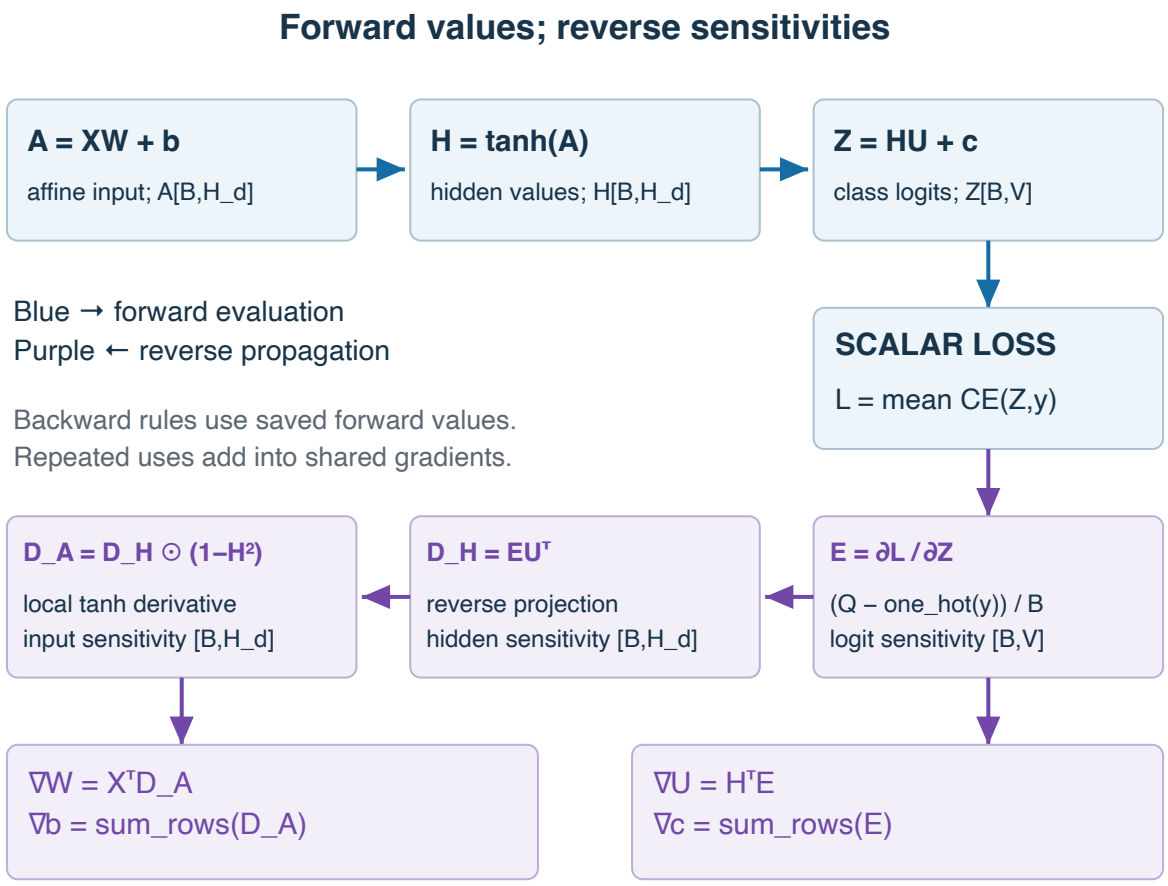


Figure 3.3: Bias gradients collect all uses of the shared parameter. Averaging these rows again would introduce an extra factor $1/B$, because the mean-loss factor is already present in E.

The function `manual_gradients` implements these equations without calling autograd. It reuses the numerical forward operations, so it is independent of the automatic reverse pass, not a completely independent implementation of every arithmetic operation. The zero-parameter fixture supplies an additional hand-computable check: with all targets in class zero, the output-bias gradient is $(-2/3, 1/3, 1/3)$, while the other parameter gradients vanish.

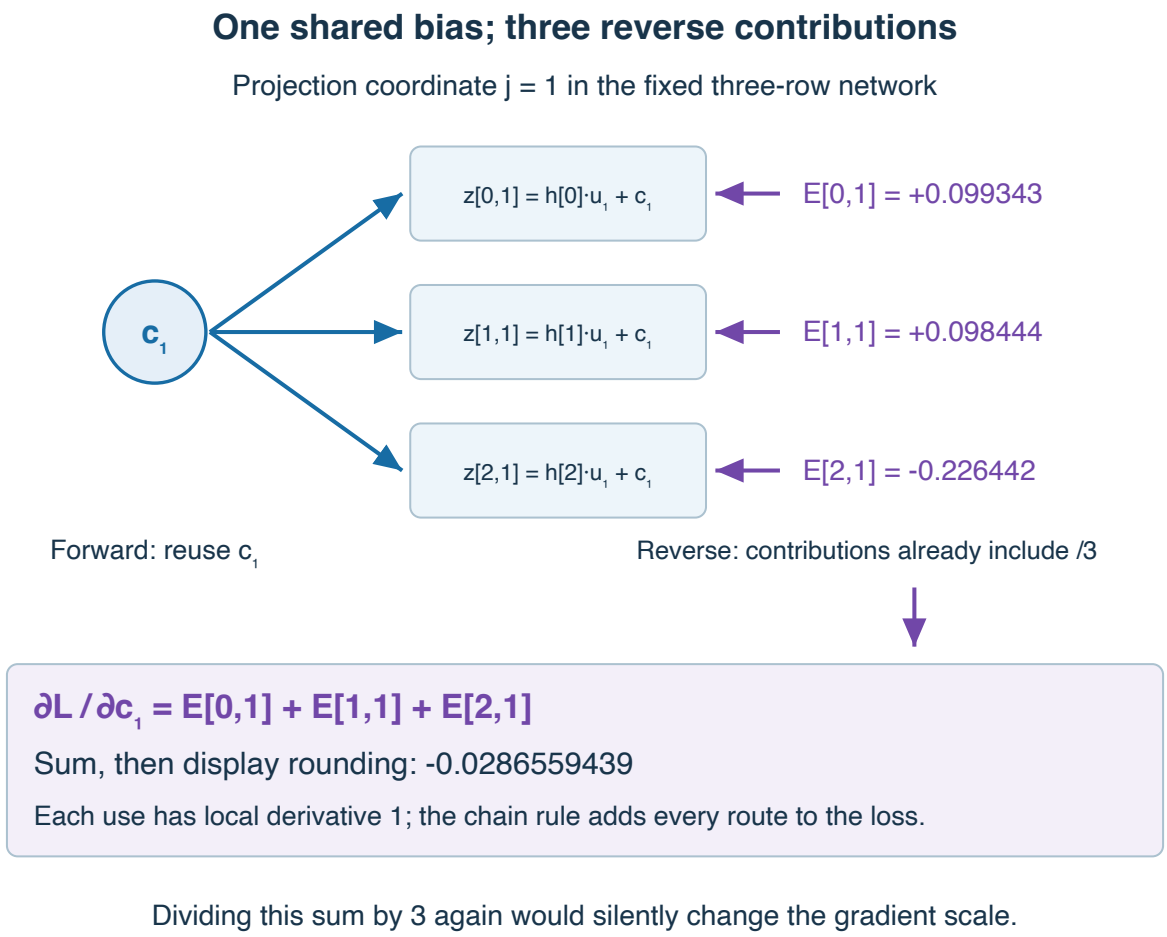


Figure 3.4: For output-bias coordinate c_1, the three contributions are approximately $+0.099343$, $+0.098444$, -0.226442. Summing the unrounded values gives approximately -0.0286559439. The negative contribution outweighs the two positive contributions; the parameter has one gradient, not a separate update for each use.

The component calculation makes broadcasting concrete. Since $z_{ij} = \sum_a H_{ia} U_{aj} + c_j$, we have $\partial z_{ik}/\partial c_j = 1$ when $k = j$ and zero otherwise. Thus

$$\frac{\partial L}{\partial c_j} = \sum_{i,k} \frac{\partial L}{\partial z_{ik}} \frac{\partial z_{ik}}{\partial c_j} = \sum_i E_{ij}. \tag{3.17}$$

The sum is forced by parameter sharing. It is not a normalization choice. For this mean loss, every E_{ij} already contains the factor $1/3$; dividing the bias sum by three again would shrink only that part of the update unless the same mistake were made elsewhere. A shape test would not catch this error because both a row sum and a row mean have the correct output shape.

3.6 What automatic differentiation computes

Automatic differentiation applies local derivative rules to the operations actually executed. Reverse mode propagates an output sensitivity backward; when multiple uses reach one value, their contributions are added. It need not construct the full Jacobian of a large scalar-loss program. Forward mode instead propagates an input direction. These modes compute products with a Jacobian or its transpose; their relative costs depend on the number

and structure of inputs and outputs. Baydin and colleagues' survey, Section 3, explains this distinction.

AD is not finite differencing. It does not estimate a derivative by subtracting two nearby function values. Nevertheless, its arithmetic is performed in finite precision, and its answer depends on the program's defined derivative semantics. An operation such as stop-gradient intentionally alters the derivative graph while leaving forward values unchanged.

PyTorch reconstructs its autograd graph as the forward program executes and may retain intermediate tensors for backward. Its autograd mechanics documentation also distinguishes differentiability, saved values and gradient recording modes. We use `torch.autograd.grad` on freshly cloned leaf parameters so the example returns gradient tensors without accumulating into an existing model's buffers.

This distinction matters for Evolutor's later structural ideas. Differentiating the continuous parameters of the selected branch is not the same as differentiating an arbitrary discrete decision that selected that branch. The existence of an autograd graph does not establish that an architecture-search or genome-editing operation has a useful continuous derivative.

3.7 Numerical checking needs a controlled function

For a scalar smooth function and coordinate direction e_k, central differences estimate

$$g_k(h) = \frac{L(\theta + he_k) - L(\theta - he_k)}{2h}. \tag{3.18}$$

The companion perturbs one parameter entry at a time, clones the parameter collection, and evaluates the same fixed inputs and targets. This is intentionally slow and small. A gradient checker for a stateful or stochastic model would also need to control hidden state and randomness so both evaluations describe the same function. PyTorch's gradcheck mechanics describes numerical and analytical Jacobian comparisons.

At the printed fixture, the maximum absolute difference between hand gradients and autograd is about 2.78×10^{-17}. With $h = 10^{-5}$, the hand-versus-central-difference error is about 1.47×10^{-11}. The exact recorded values and parameter gradients are in results.json (companion file: results.json). PyTorch's own gradcheck passes too.

These are absolute discrepancies at one fixed scale. A production check should state absolute and relative tolerances and examine near-zero components separately. Additional tests vary batch size, input dimension, hidden width and vocabulary so the matrices are not all square; a test performed only with equal widths could conceal a transpose error. Noncontiguous parameter views are tested too. Passing a derivative check also does not validate that an input/target shift matches Chapter 2's information contract.

3.8 The negative control: forward equality is not gradient equality

Consider a program whose forward expression is

$$f(x) = x \cdot \text{stop_gradient}(x). \tag{3.19}$$

When we vary the input and rerun it, its numerical values follow x^2. Central differences near $x = 2$ therefore give approximately 4. But the recorded derivative graph treats the second factor as constant, producing derivative 2.

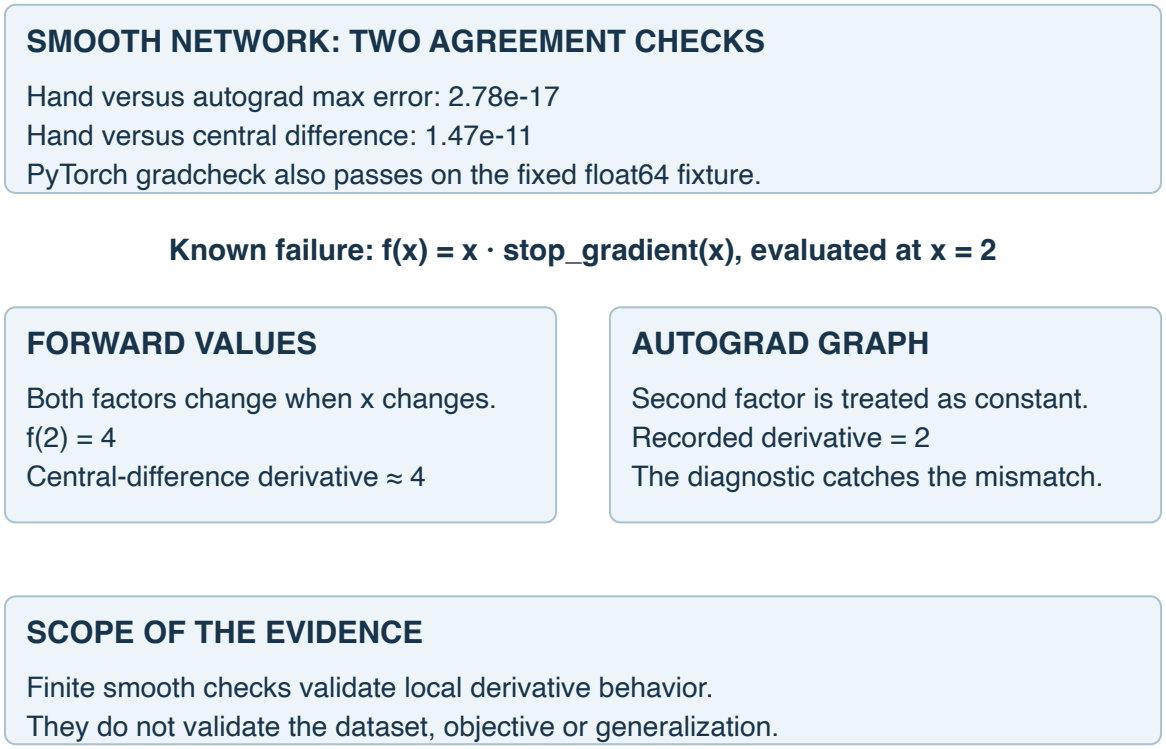


Figure 3.5: This is not evidence that autograd failed its contract: stop-gradient requested the altered derivative. It is evidence that the recorded gradient is not the derivative of the forward-value mapping under ordinary input perturbation. Whether that is a bug depends on the intended algorithm; the chapter's negative control makes the distinction observable.

The example guards against a weak testing habit: running a checker that has never detected any known failure. It also separates two tests from Chapter 2. A finite future-token intervention examines forbidden information flow. A finite parameter perturbation examines a derivative. Neither substitutes for the other.

3.9 Why a smaller difference step eventually gets worse

For a sufficiently smooth scalar function, Taylor expansion gives

$$L(\theta + h) = L(\theta) + hL'(\theta) + \tfrac{1}{2}h^2L''(\theta) + \tfrac{1}{6}h^3L'''(\theta) + O(h^4), \tag{3.20}$$

and a corresponding expansion at $-h$. Subtraction cancels the even terms; dividing by $2h$ leaves a leading truncation error proportional to h^2. Smaller h reduces that term.

But two close floating-point loss values are subtracted. If their absolute rounding errors are of scale $\epsilon|L|$, dividing their difference by h gives an error contribution of order $\epsilon|L|/h$. Thus the schematic competition is

$$\text{error}(h) \approx C_1h^2 + C_2\epsilon/h. \tag{3.21}$$

This is a local error model with scale-dependent constants, not a universal fitted law. It explains why arbitrarily small perturbations can be worse. For our fixture, the recorded error falls from about 2.39×10^{-4} at $h = .1$ to 1.47×10^{-11} at 10^{-5}, then rises to roughly 1.18×10^{-7} at 10^{-9}. No step size was tuned for model performance; these are derivative diagnostics.

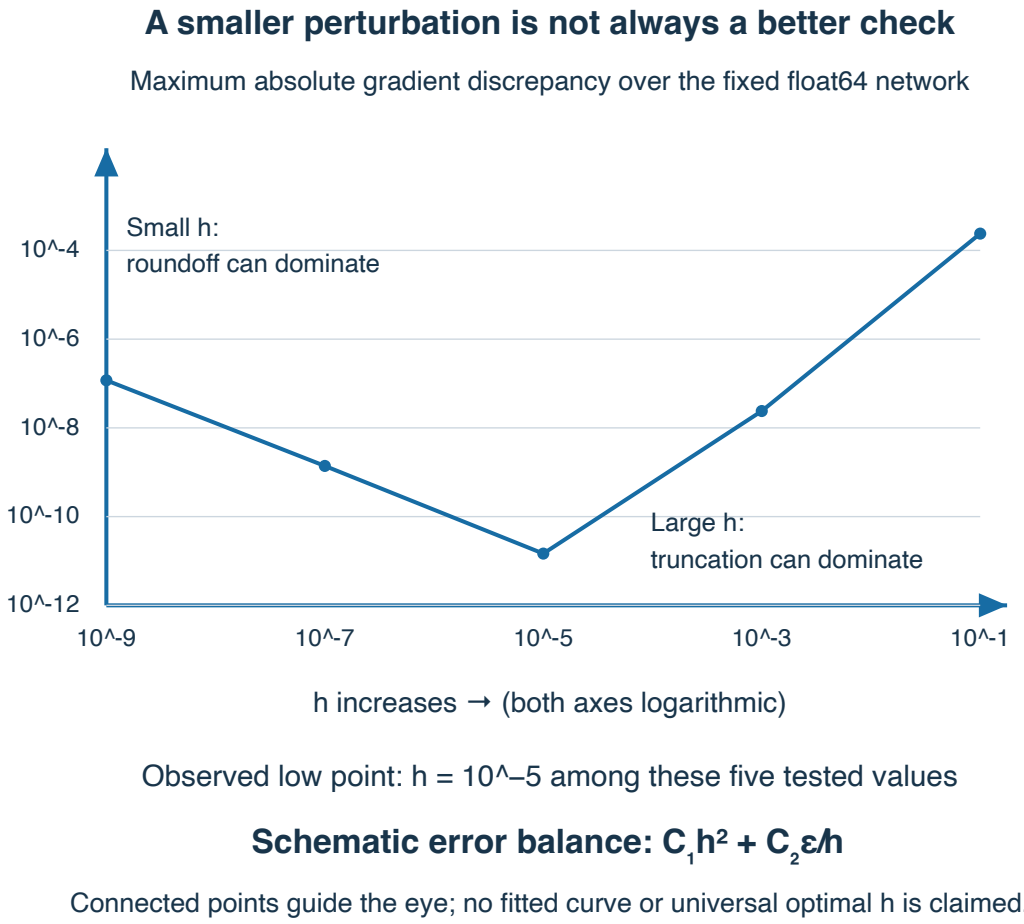


Figure 3.6: Both axes are logarithmic. The points compare central differences with the hand-derived gradients over every parameter coordinate of the fixed network. The lines connect measured diagnostics, not samples from a fitted universal error law. The smallest error among these five choices occurs at $h = 10^{-5}$; that is not an optimal-step theorem.

The schematic balance can also suggest a scale, provided its assumptions hold. Differentiating $C_1h^2 + C_2\epsilon/h$ with respect to positive h gives $2C_1h - C_2\epsilon/h^2$. Setting this to zero yields $h^3 = C_2\epsilon/(2C_1)$. This cube-root dependence explains why a useful difference step is generally much larger than machine epsilon. It cannot choose a universal number: the constants depend on local derivatives and the scale of the evaluated losses, and the model is not a bound on every floating-point computation.

3.10 Stable formulas still need finite inputs

For logits $(1000, 1001, 999)$, direct float64 exponentiation overflows. Yet the cross entropy for target class one is finite. Let $m = \max_j z_j$ and rewrite it as

$$\log \sum_j \exp(z_j - m) - (z_y - m). \tag{3.22}$$

All exponent arguments are nonpositive, and at least one exponent is one. The companion's `stable_nll` gives approximately 0.4076059644, agreeing with PyTorch's cross entropy. Adding the same constant to all logits leaves the result unchanged in exact arithmetic; the tests check a finite shifted example and its gradients.

Floating-point operations are not generally associative, and mathematically equivalent batched and sliced computations need not be bitwise equal. PyTorch's numerical-accuracy notes document such limitations. Our scalar cancellation example makes precision visible: $(10^8 + 1) - 10^8$ evaluates to zero in float32 but one in float64. Using float64 improves this example; it does not eliminate numerical conditioning, overflow or all roundoff.

The stable-loss helper rejects nonfinite inputs rather than treating a nonfinite comparison as success. It is a compact reference for finite logits, not a replacement for the full production loss contract, masking rules or mixed-precision policy.

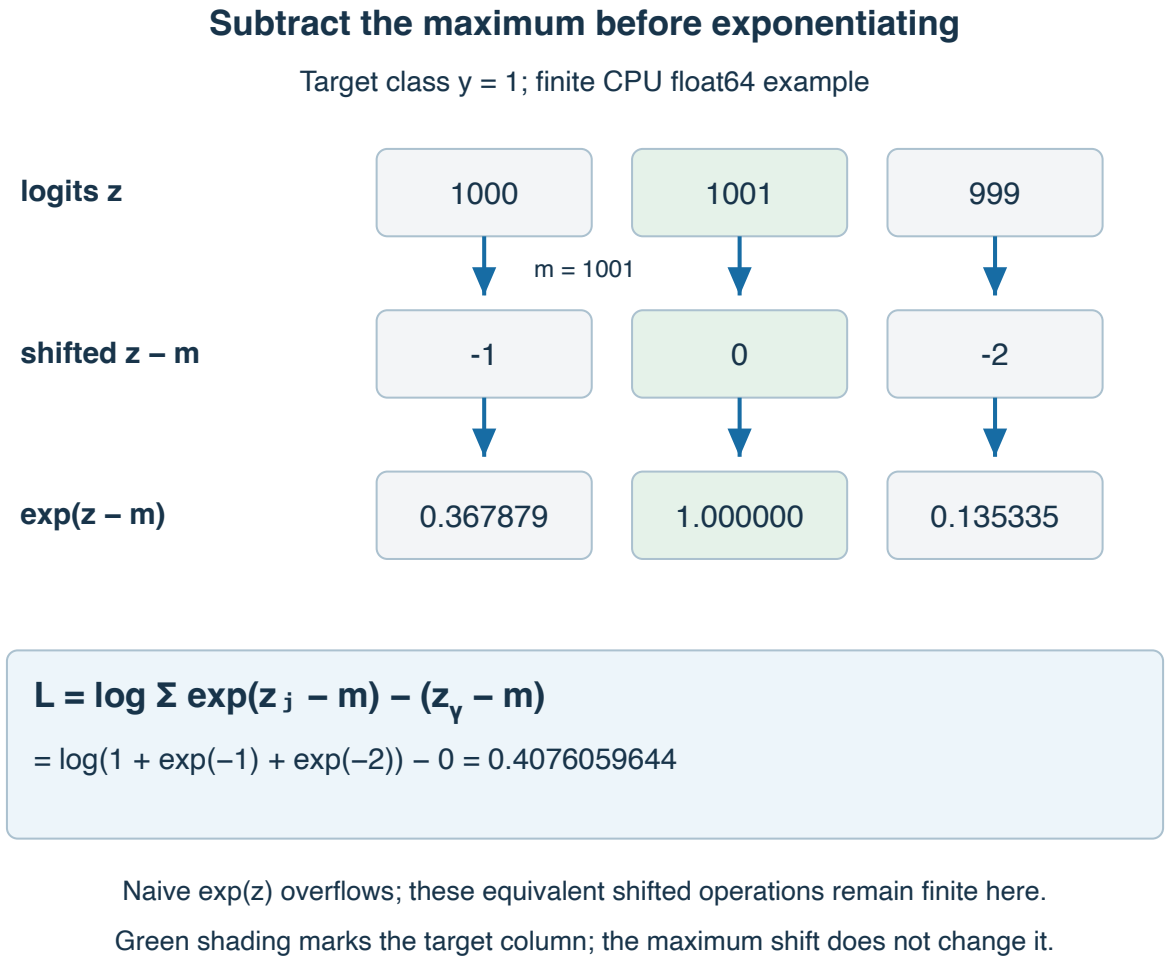


Figure 3.7: Subtracting 1001 maps the logits to $(-1, 0, -2)$. Their exponentials sum to $1 + \exp(-1) + \exp(-2)$; the target's shifted logit is zero. The aligned columns make it possible to reconstruct the final loss without interpreting a decorative softmax icon.

The algebra follows by factoring out $\exp(m)$ from the sum of exponentials: $\log \sum_j \exp(z_j) = m + \log \sum_j \exp(z_j - m)$. The m term cancels when we subtract z_y. Do not implement this proof by explicitly computing $\exp(m)$: that would reintroduce the overflow the rearrangement avoids.

There is a derivative subtlety at tied maxima. The maximum alone is nonsmooth, but the exact composite loss remains smooth at finite tied logits. Write $r_j = z_j - m$; the loss differential is $\sum_j q_j(dz_j - dm) - (dz_y - dm)$. The coefficient of dm is $1 - \sum_j q_j = 0$, leaving $\sum_j q_j dz_j - dz_y$. The final gradient does not depend on which maximizer supplied the shift. A dedicated tied-logit test checks the implemented value and gradient against the same unshifted cross-entropy contract.

3.11 An exact gradient does not choose a safe step

Gradient descent updates $x_{k+1} = x_k - \eta L'(x_k)$. Consider the fully analyzable scalar loss

$$L(x) = \frac{1}{2}\lambda x^2, \qquad \lambda > 0. \tag{3.23}$$

Since $L'(x) = \lambda x$,

$$x_k = (1 - \eta\lambda)^k x_0. \tag{3.24}$$

For a nonzero starting value, convergence to zero requires $|1 - \eta\lambda| < 1$, or $0 < \eta < 2/\lambda$. At $\eta = 2/\lambda$, the sign flips without shrinking. Above that boundary, the magnitude grows despite using the exact derivative at every step.

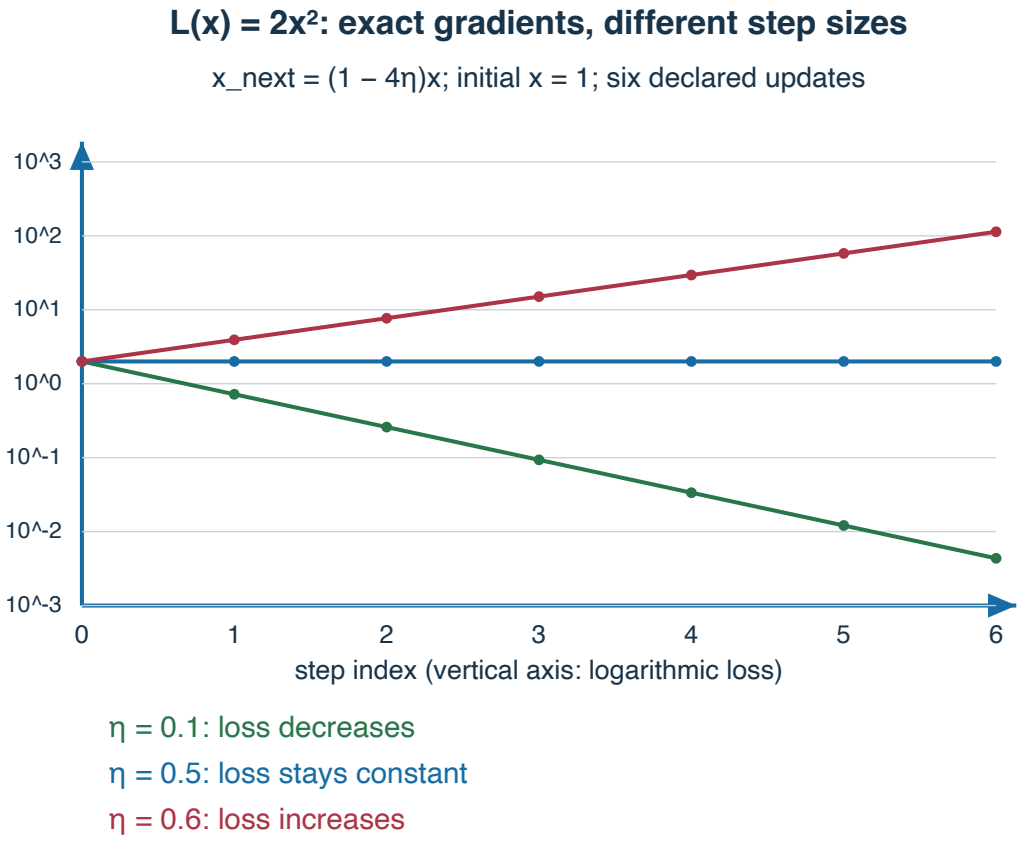


Figure 3.8: With $\lambda = 4$, learning rates .1, .5 and .6 respectively decrease, preserve and increase the loss magnitude sequence. The vertical axis is logarithmic so both decay and growth remain visible. These are analytic toy updates, not trained-model measurements.

For a positive-definite quadratic in several dimensions, orthogonal eigen-directions obey corresponding factors $1 - \eta\lambda_i$. A large eigenvalue restricts a shared stable step; a small eigenvalue can then decay slowly. This is the local geometry behind a simple conditioning difficulty. Nonlinear and stochastic objectives require additional assumptions and analysis.

The fixed network fixture takes one declared step with $\eta = .1$: its loss decreases from approximately 1.0250007902 to 1.0179571386. That confirms this update's arithmetic and local

effect only. It is not a training study or a generalization result. Optimizer state, momentum and other options add semantics beyond this plain update; see the official SGD specification before equating different implementations.

3.12 Accumulation must implement the declared objective

When multiple uses of a parameter contribute to one scalar loss, reverse differentiation sums their contributions. Separately, PyTorch's `.backward()` accumulates into leaf gradient buffers. In a small test, differentiating x^2 at $x = 2$ twice on freshly constructed forward graphs gives a stored gradient of eight unless the buffer is cleared between calls. Setting the buffer to `None` restores the next isolated result of four.

Microbatch accumulation creates another normalization question. Split our three rows into one row and two rows. If each microbatch computes its own mean gradient g_1 and g_2, the full mean is $(g_1 + 2g_2)/3$, not $(g_1 + g_2)/2$. The tests verify the weighted identity and require the incorrect unweighted version to disagree. For variable-length sequences, the relevant weights are the declared valid-target counts, not necessarily the number of sequences.

This is why a training loop is a tensor program implementing a statistical objective, not merely a repeated optimizer call. Chapter 4 must preserve those semantics across data loading, checkpointing and evaluation.

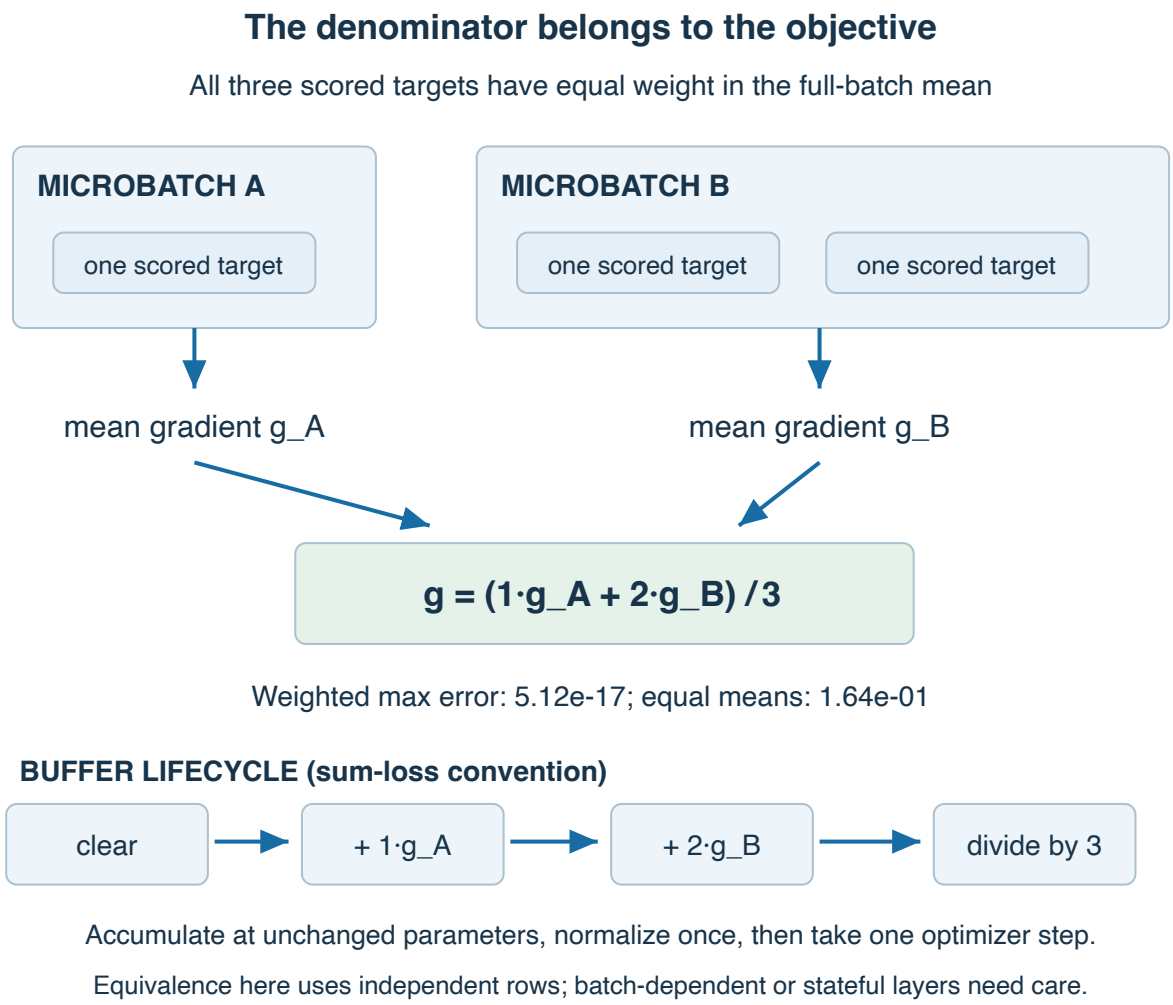


Figure 3.9: With the parameters held fixed, weighting the two mean gradients by target counts matches the full-batch gradient to approximately 5.12×10^{-17} maximum absolute error. Equal averaging instead differs by approximately 0.164. These figures come from the reference artifact; they diagnose objective weighting, not training quality.

More generally, let I_a partition the scored targets and $N_a = |I_a|$, with $N = \sum_a N_a > 0$. Define $L_a = N_a^{-1} \sum_{i \in I_a} \ell_i$. If the same per-target losses are evaluated at the same parameters in every partition, then

$$L = \frac{1}{N} \sum_a N_a L_a, \qquad \nabla L = \sum_a \frac{N_a}{N} \nabla L_a. \tag{3.25}$$

There are two consistent implementations. Accumulate the gradients of each sum loss $N_a L_a$, then divide the buffer by N; or multiply each mean loss by N_a/N before backward. Do not combine both normalizations. Do not step the optimizer between microbatches if the intended update is the gradient of the original full-batch objective.

The premise about the *same* per-target losses matters. Batch-dependent operations, changing hidden state or different random draws can change the function when the batch is partitioned. In that case count weighting alone does not establish equivalence. Empty masked microbatches require special handling too: zero valid targets contribute a zero sum, but their mean is undefined. Our fixed reference has neither batch-dependent layers nor empty batches, so its weighted identity has a precisely declared scope.

3.13 A biological motivation is not a derivative definition

Regulatory binding can affect transcription initiation, as the textbook genetic-switch discussion explains. A computational proposal must then specify its own operator. It must not infer differentiability from a biological name.

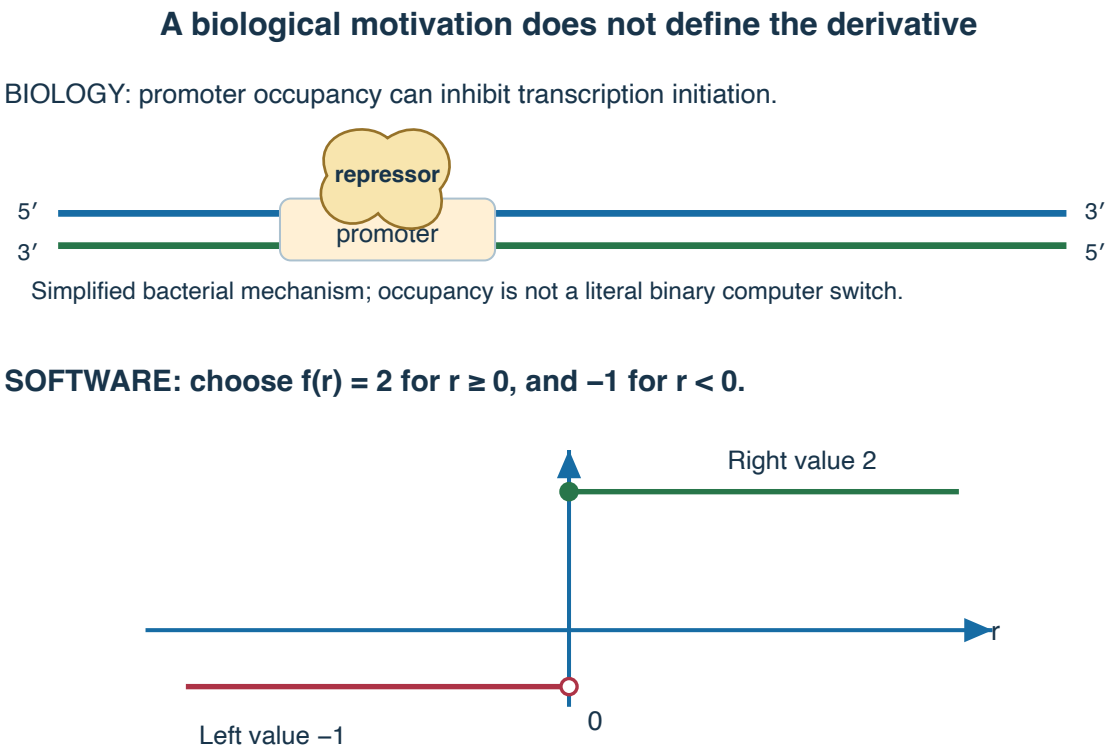


Figure 3.10: The upper panel is a simplified bacterial mechanism. The lower panel is an intentionally discontinuous mathematical example, not a biochemical response curve. Their connection is a research question, not an equivalence.

Consider $f(r) = 2$ when $r \geq 0$ and $f(r) = -1$ otherwise. Away from zero, f is locally constant and its ordinary derivative is zero. At zero its left and right limits disagree, so there is no classical derivative. A central difference across the boundary gives

$$\frac{f(h) - f(-h)}{2h} = \frac{3}{2h}, \qquad h > 0, \tag{3.26}$$

which diverges as h shrinks. This is not floating-point noise: the underlying function is discontinuous. A nonzero number returned by a numerical checker does not make a useful derivative exist.

Now replace the constants by trainable a and b. For a fixed branch, a loss can differentiate with respect to the selected parameter while assigning no contribution to the unselected one. That is a different question from how the loss changes when the branch choice changes. Our PyTorch check makes this explicit with `where`: the selected value receives its ordinary sensitivity, while the Boolean comparison does not create a differentiable path to r.

Later DOGMA research must decide how structural choices are proposed and evaluated. A smooth relaxation, a surrogate derivative and discrete search are different algorithms with different evidence obligations. This chapter adopts none as a validated genomic innovation. It establishes the boundary that their training claims must respect.

3.14 Optimizer state changes the update, not the derivative

Plain gradient descent uses the current derivative directly. Momentum retains information from earlier derivatives, so a reproducible update must include optimizer state as well as parameters. For the declared PyTorch SGD settings, the first buffer equals the first gradient; later steps use

$$v_k = \mu v_{k-1} + g_k, \qquad \theta_k = \theta_{k-1} - \eta v_k. \tag{3.27}$$

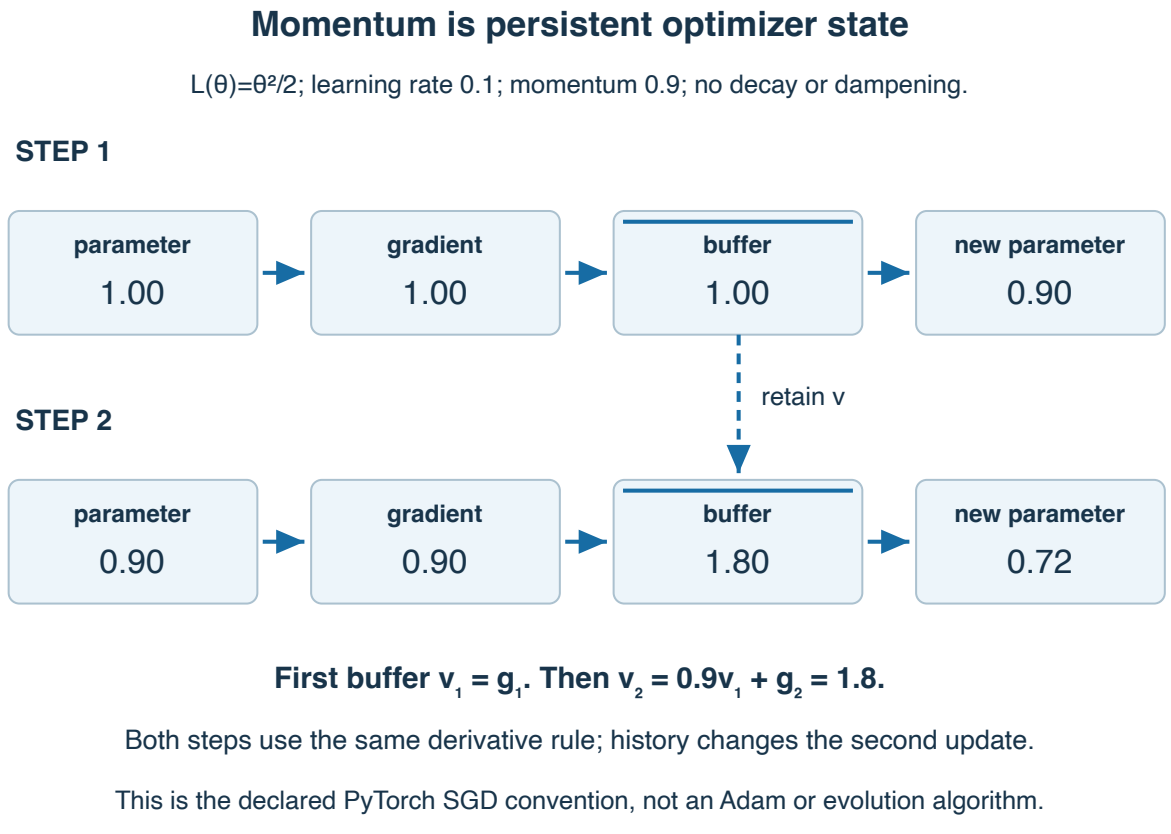


Figure 3.11: For $L(\theta)=\theta^2/2$, initial $\theta=1$, $\eta=.1$ and $\mu=.9$, the first gradient and buffer are both one, producing $\theta=.9$. The second gradient is .9, but the buffer becomes 1.8, producing $\theta=.72$. These are CPU float64 arithmetic checks, not a training benchmark.

The executable fixture compares the local derivation with `torch.optim.SGD`, using zero dampening and zero weight decay. It does not mix regularization into the objective or assume that every optimizer initializes its state identically. The versioned SGD documentation specifies that convention. Restoring only $\theta=.9$ and discarding the buffer would make the next first-step buffer .9 and yield $\theta=.81$ instead. The visible parameter value at a checkpoint is therefore insufficient to identify the training state.

3.15 Control the state in a numerical comparison

For $L(\theta;s)=(\theta+s)^2$, the derivative with respect to θ **at fixed s** is $2(\theta+s)$. At $\theta=2$ and $s=3$ it equals ten. If both finite-difference evaluations hold $s=3$, they compare the same scalar function at nearby parameters.

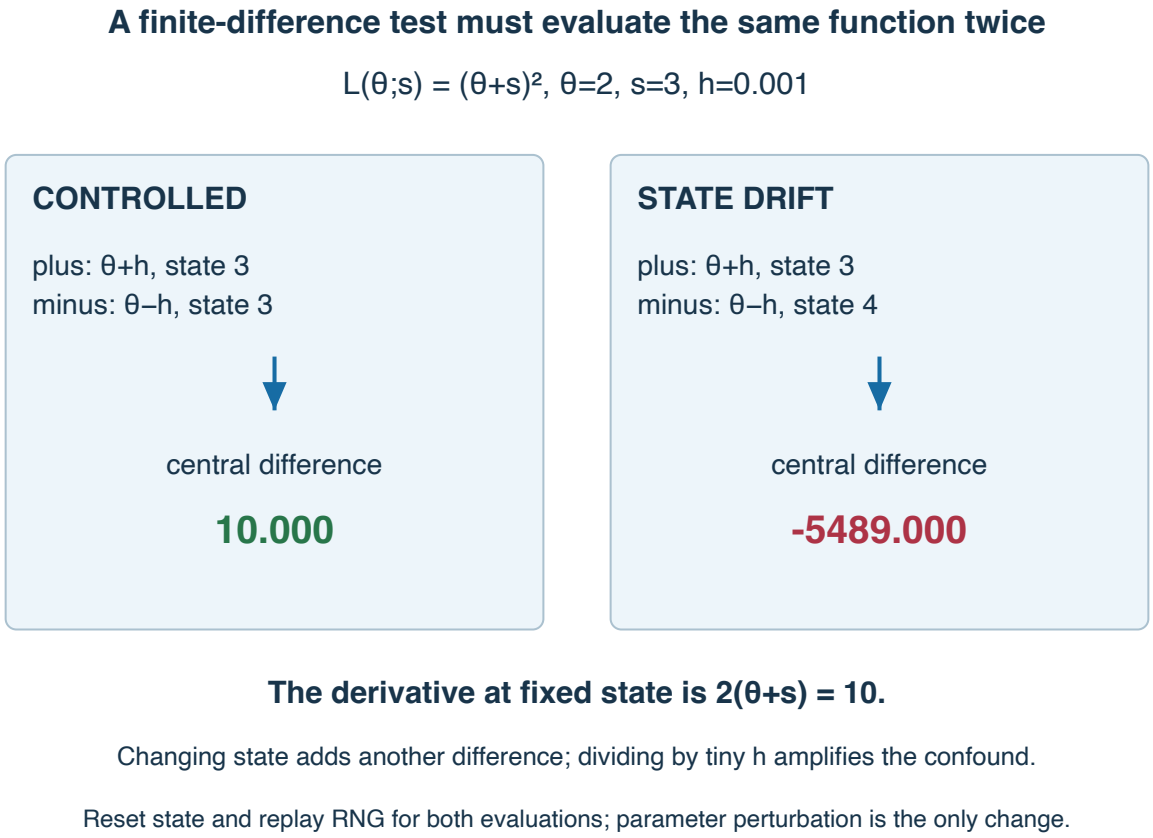


Figure 3.12: With $h=.001$, the controlled difference is approximately ten. Changing the second evaluation's state to four instead gives about -5489. The large discrepancy diagnoses an uncontrolled comparison, not a derivative implementation defect in the quadratic.

The calculation explains the failure. The uncontrolled numerator contains a finite difference between two states, not only an $O(h)$ difference between nearby parameters. Dividing that extra difference by $2h$ magnifies it. Reducing h can therefore make the diagnostic look worse even in exact arithmetic. RNG draws, mutable recurrent state, counters and batch-dependent statistics can cause analogous problems. They must be replayed or explicitly incorporated into the function being differentiated. These controls are separate from the future-token interventions that test causality.

3.16 The handoff to a reproducible training run

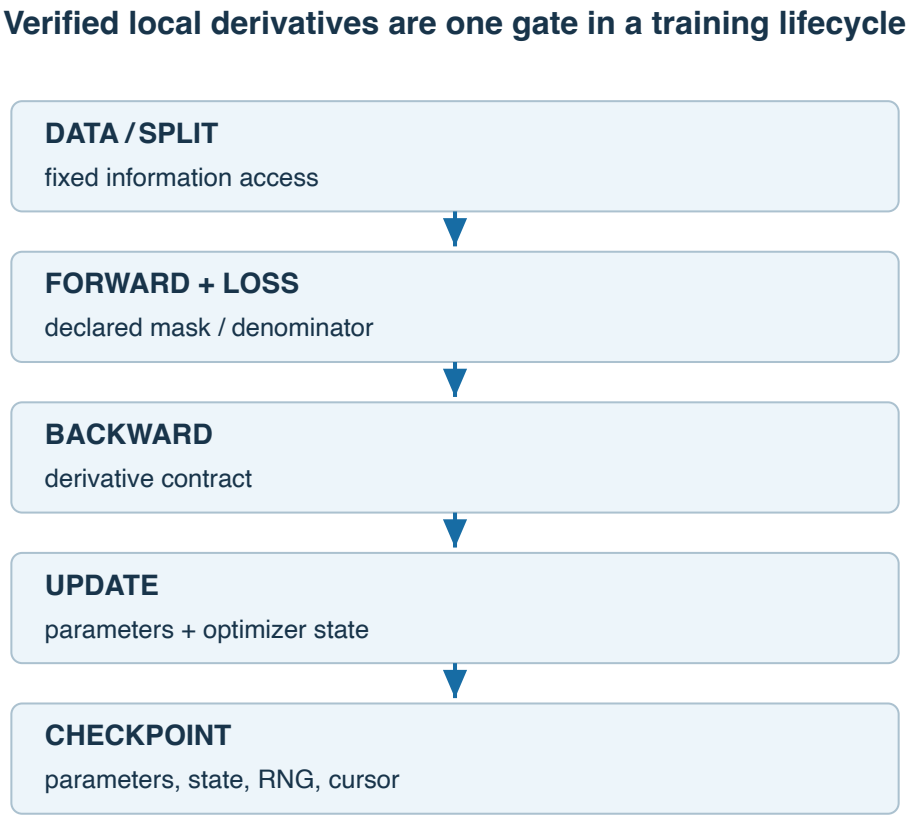


Figure 3.13: The figure is Chapter 4's contract, not an implemented end-to-end DOGMA or Hermon DNA training system. Parameters, optimizer state, relevant model state, random state and data position can all matter at restart.

Chapter 3 has established local claims: the declared smooth fixture has matching manual, automatic and numerical derivatives; known failures are detectable; normalization and state conventions affect updates. Chapter 4 must compose those facts with a data pipeline, explicit masking, held-out evaluation and a restart test. Lower training loss alone cannot establish genomic usefulness. A model can optimize a shortcut, memorize related sequences across splits or exploit forbidden future information while satisfying every derivative check here.

This is why these mathematical foundations belong in Evolutor. They provide tests that can reject a proposed genomic mechanism before its interpretation becomes an elaborate architecture story. DOGMA remains the non-Transformer DNA-native research track; Hermon DNA remains Transformer-based. Neither is established by the small neural network used in this chapter.

Key ideas

- A shared bias collects one gradient contribution from every row. Dividing that sum again by the batch size changes the update.
- Forward equality is not gradient equality: `stop_gradient` can preserve values while changing derivatives.
- An exact gradient does not choose a safe step. On $\frac{1}{2}\lambda x^2$, gradient descent converges only for $0 < \eta < 2/\lambda$.
- Weight microbatch gradients by their scored-target counts, not by the number of microbatches.

3.17 Exercises with worked reasoning

1. **Bias reduction.** Why sum the per-row projection errors for $\nabla_c L$? **Solution:** The same scalar bias coordinate contributes with derivative one to that coordinate in every row. The chain rule adds these contributions. Mean-loss normalization already entered the row errors.

2. **A symmetry check.** Why must each row of the logit gradient sum to zero? **Solution:** Both its predicted distribution and one-hot target sum to one; their difference sums to zero. This agrees with invariance to adding a constant to all logits in that row.

3. **A detached factor.** Is the negative control proof of a library bug? **Solution:** No. Stop-gradient deliberately changes differentiation semantics. The mismatch shows that the recorded gradient does not equal the derivative of rerun forward values; whether that is wrong depends on the specified algorithm.

4. **Stability boundary.** For $L(x) = 3x^2$, what positive rates converge from nonzero x_0? **Solution:** Here $\lambda = 6$, so the multiplicative factor is $1 - 6\eta$. Its magnitude is below one exactly for $0 < \eta < 1/3$. Equality at $1/3$ gives undamped sign flips.

5. **Unequal batches.** Ten scored targets give mean gradient a, and ninety give mean gradient b. What is the combined mean? **Solution:** $(10a + 90b)/100$. Equal averaging assigns ten times too much relative weight to each target in the smaller batch.

6. **A passing derivative check.** Does passing gradcheck establish that a genomic model is causal or scientifically useful? **Solution:** No. A function can have perfectly correct derivatives while reading forbidden future tokens or optimizing a shortcut. Parameter-derivative correctness, information access and empirical evidence are separate obligations.

7. **Broadcast bug with correct shape.** A reverse pass uses the row mean of E for the output bias. Why can all shape assertions pass while the update is wrong? **Solution:** Both the sum and mean produce a vector of vocabulary width. The mean introduces an extra factor $1/B$, despite E already representing the mean-loss sensitivity. Check values against a nonzero analytical fixture, not just dimensions.

8. **Maximum-shift algebra.** Why is calculating $\exp(m)$ unnecessary? **Solution:** Factoring it out gives $m + \log \sum_j \exp(z_j - m) - z_y$. Combining the two scalar terms yields the stable expression directly. Constructing the factored exponential numerically defeats the algebraic cancellation.

9. **Tied logits.** For one target in class 1 and logits $(0, 0, 0)$, what are the loss and gradient? **Solution:** The loss is $\log 3$, and the gradient is $(1/3, -2/3, 1/3)$. A tied maximum in the

stable intermediate does not make the final cross entropy nonsmooth; the maximum's differential cancels.

10. **Perturbation scale.** Minimize the schematic positive error $ah^2 + b\epsilon/h$ for positive a, b, ϵ. **Solution:** Setting its derivative to zero gives $h = (b\epsilon/(2a))^{1/3}$. The second derivative is positive. The result depends on unknown local constants and should motivate a diagnostic scan, not a universal checker default.

11. **Two equivalent accumulation loops.** Show why backpropagating $(N_a/N)L_a$ per microbatch matches accumulating $N_a L_a$ and dividing once. **Solution:** Differentiation and finite summation are linear at fixed parameters. Both yield $\sum_a (N_a/N)\nabla L_a$. Applying both methods' divisions produces an erroneous extra $1/N$. Updating parameters between terms breaks the fixed-parameter premise.

12. **Design a boundary test.** Does a passing microbatch check cover a layer whose output subtracts the current batch mean? **Solution:** No. Partitioning changes the batch mean, so a row's output and loss can change. Compare the per-row forward values first; if they differ, the two programs need not compute the same objective. Separate a weighting bug from a legitimate batch-dependent computation.

3.18 Selected glossary

Adjoint: reverse-mode sensitivity of the scalar objective to an intermediate.

Broadcast reduction: summation over uses of a shared value in the reverse pass.

Optimizer state: retained update-rule information distinct from parameters.

Discrete selector: a branch choice whose change need not have a classical derivative.

Controlled function: the same inputs, auxiliary state and randomness except for the coordinate deliberately perturbed in a numerical check.

3.19 Executable boundary diagnostics

These functions are reproduced from completion_diagnostics.py and exercised by the repository tests. The full reference program, exhaustive checks and machine-readable results remain alongside the manuscript in drafts/ch03/. These are teaching calculations, not calibrated biological measurements or a trained model.

```
import math

def momentum_trace():
    import torch
    p=torch.tensor(1.,dtype=torch.float64,requires_grad=True)
    opt=torch.optim.SGD([p],lr=.1,momentum=.9,weight_decay=0.,dampening=0.)
    rows=[]
    for _ in range(2):
        opt.zero_grad(set_to_none=True)
        before=float(p.detach())
        (.5*p.square()).backward()
        gradient=float(p.grad)
        opt.step()
        rows.append({"before":before,"gradient":gradient,
                     "buffer":float(opt.state[p]["momentum_buffer"]),
                     "after":float(p.detach())})
    return rows
```

```
def branch_value(r):
    return 2. if r >= 0. else -1.

def state_control(h=.001, advance_state=False):
    # L(theta;s)=(theta+s)^2. Only the second evaluation changes s in the bad control.
    if not math.isfinite(h) or h <= 0: raise ValueError("positive finite step required")
    plus=(2.+h+3.)**2
    minus=(2.-h+(4. if advance_state else 3.))**2
    return (plus-minus)/(2*h)
```

3.20 Sources and further reading

Baydin and colleagues survey forward- and reverse-mode automatic differentiation, including the distinction from finite differencing (JMLR 18, 2018, Section 3). The PyTorch 2.10 documentation for autograd mechanics, gradcheck, numerical accuracy, cross entropy and SGD defines the library conventions used in this chapter. The bacterial genetic-switch discussion in Alberts and colleagues (Molecular Biology of the Cell) supplies the biological example in the section on discontinuous selectors. The chapter's numerical fixtures and diagnostics are produced by the companion programs in `drafts/ch03/`; the chapter's source-access ledger is kept in the repository provenance records rather than in the reading text.

CHAPTER 4

Reproducible PyTorch training

In this chapter you will

- construct shifted, padded and masked next-base targets with an explicit denominator
- inventory everything a training transition depends on: parameters, momentum, sampler cursor and random state
- serialize and restore a checkpoint that resumes exactly in a declared environment
- use deliberately incomplete restores as negative controls

4.1 The weights are identical. Why does training diverge?

Two copies of a model have exactly the same parameter tensors. Both receive the same next batch. One was running before an interruption; the other was rebuilt from a saved weight file. Predict: must their next updates agree?

No. If the optimizer uses momentum, its next update also depends on a history-dependent buffer. If the model uses dropout, its next forward pass depends on random state. If the restarted data iterator selects a different batch, even the input assumption fails. A training computation has more state than its weights.

This chapter builds a small experiment in which that statement is directly testable. We train a one-base-context predictor on synthetic DNA strings, interrupt it after a completed update, and compare the resumed trajectory with an uninterrupted run. We then intentionally omit state and measure the resulting disagreement.

The model is small enough to inspect completely: 84 stored scalar parameters, CPU execution, and float64 arithmetic. It is not a genomic benchmark, an implemented DOGMA architecture, or a Hermon DNA result. DOGMA remains the non-Transformer DNA-native research track; Hermon DNA names the Transformer-based track; Evolutor is the broader research framework. A careful training contract is useful to all of them, but a working toy component is not evidence that the larger research systems exist or work.

4.2 A dataset small enough to understand

4.2.1 Specify the objects before choosing a loader

Our data consist of named strings. The six training rows and two validation rows are fixed before training:

Split	Row identity	Sequence
Training	train-a	`ACGTACGT`
Training	train-b	`ACGTT`
Training	train-c	`TTACGN`
Training	train-d	`CGTAC`
Training	train-e	`GATTACA`
Training	train-f	`TGCATGCA`
Validation	valid-a	`ACGACG`
Validation	valid-b	`TTGCA`

Each name is part of the data identity, not a biological annotation. The strings were constructed for this example. They do not represent independent donors, chromosomes, species, or experimentally characterized regulatory elements. The validation score therefore measures prediction on these held-out synthetic rows only.

The input alphabet is `A,C,G,T,N`, encoded as integers 0 through 4. Here `N` means an unspecified input base, not a fifth canonical base. Padding uses a separate input ID, 5. Output classes are only the four canonical bases. Targets that would be `N`, and positions added solely for padding, receive the sentinel -100 and are excluded from scoring.

The distinction is deliberate. An unknown input can still provide context for predicting a known next base. An unknown *target* does not provide a canonical answer against which to score this four-class predictor. Padding, by contrast, is an artifact of rectangular batching, not an observed biological symbol.

4.2.2 Split source groups before deriving windows

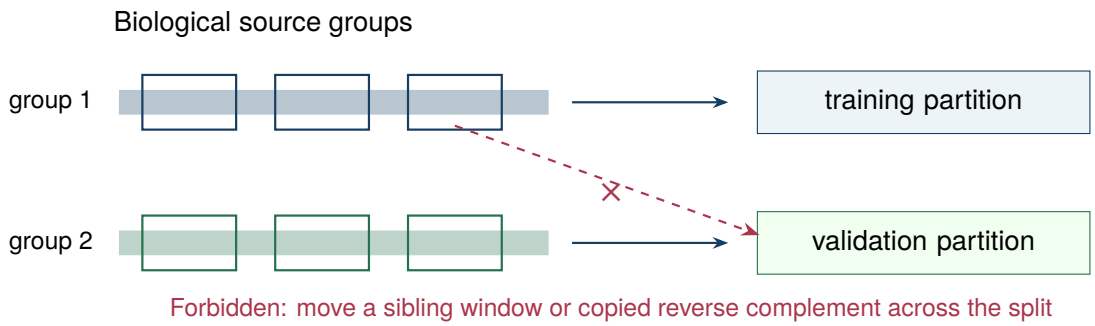


Figure 4.1: Partition source groups before creating overlapping windows or reverse-complement variants. The two sequence strips and their derived windows illustrate shared origin; crossed red links mark forbidden cross-split assignment of siblings. This is a data-lineage rule, not a depiction of molecular transport or an implemented homology-clustering algorithm.

If nearby windows from the same source appear on both sides of a split, a validation score can reflect recognition of shared sequence rather than the intended generalization. Reverse-complement augmentation can create a similar bookkeeping problem: changing orientation does not create a new independent origin.

Our executable guard checks disjoint row identities and excludes exact sequence or reverse-complement duplicates across splits. It canonicalizes a string using the lexicographically smaller of the string and its reverse complement. That is an exact duplicate

check only. It does not detect distant homology, shared organisms, overlapping coordinates, or batch effects. The diagram states a stronger design principle for future real datasets; the code implements the narrow checks that can actually be established for this toy corpus.

The checkpoint also stores a SHA-256 digest of the ordered training rows. A changed sequence, row identity, or row order is rejected on restoration. A digest detects a change in the serialized data identity; it does not prove the biological validity or independence of that data.

4.3 Turn a sequence into an explicit prediction task

4.3.1 Shift first, then pad

For a sequence $s_0, s_1, \ldots, s_{L-1}$ of length $L \geq 2$, construct inputs $x_t = s_t$ and targets $y_t = s_{t+1}$ for $0 \leq t < L - 1$. Thus `ACGT` becomes input `ACG` and target `CGT`. The last original base supplies a target, not another input position with an invented target.

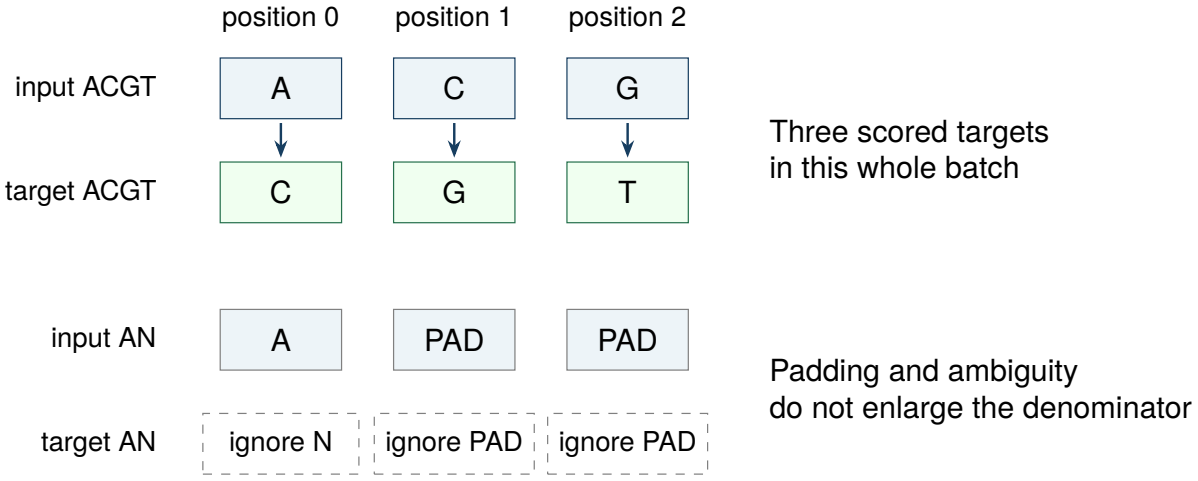


Figure 4.2: Exact alignment and scoring. The row `ACGT` contributes three canonical next-base targets. The row `AN` contributes no scored target: its real next base is unspecified, and its remaining cells are padding. Input `N`, input padding, and ignored targets have distinct roles. These are digital tensor cells, not chemical structures.

The collator pads each batch to the maximum input length in that batch, producing integer tensors $X, Y \in \mathbb{Z}^{B \times T}$. The model returns real logits $Z \in \mathbb{R}^{B \times T \times 4}$. Here B is the number of rows, T is padded sequence length, and 4 is the number of output classes. Tensor dimensions are counts, not physical units.

The model we use is deliberately weaker than a full sequence model. Its prediction at position t depends only on x_t, not on the entire prefix $x_0, \ldots, x_t$. It cannot learn a motif that requires several earlier positions. This restriction makes causality easy to test and prevents architecture complexity from obscuring the training mechanics.

4.3.2 Derive the loss and its denominator

For logits $z_{b,t,k}$, the canonical-base probability is

$$p_\theta(k \mid x_{b,t}) = \frac{\exp z_{b,t,k}}{\sum_{j=0}^{3} \exp z_{b,t,j}}. \tag{4.1}$$

Define $m_{b,t} = 1$ when the target is a known canonical base and 0 otherwise. If $C = \sum_{b,t} m_{b,t} > 0$, the unweighted token-mean cross-entropy is

$$\mathcal{L} = \frac{S}{C}, \qquad S = -\sum_{b,t:m_{b,t}=1} \ln p_\theta(y_{b,t} \mid x_{b,t}). \tag{4.2}$$

Natural logarithms express this information loss in nats per scored target. The denominator counts scored targets, not batch rows, padded cells, or all raw characters.

The implementation requests summed cross-entropy with an ignored target index, then divides by its independently counted non-ignored targets. This matches the unweighted class-index loss contract documented for PyTorch 2.10 [20]. We do not use class weights or label smoothing; those choices would require their own normalization audit.

For the illustrated batch, uniform logits imply probability $1/4$ for each class. Exactly three targets are scored, so $S = 3\ln 4$ and $\mathcal{L} = \ln 4 \approx 1.3862943611$. This is a **golden case**: an answer known independently of optimization that tests the implementation's meaning.

For a scored position, differentiating the softmax cross-entropy gives

$$\frac{\partial \mathcal{L}}{\partial z_{b,t,k}} = \frac{p_\theta(k \mid x_{b,t}) - \mathbf{1}[k = y_{b,t}]}{C}. \tag{4.3}$$

For an ignored position, the derivative is zero. The reference tests check this distinction directly. Ignoring target loss and setting an embedding's padding row are different operations; one does not substitute for the other.

If $C = 0$, the mean is undefined. Our code raises an error before backward or optimizer update. It does not replace the denominator with one and silently declare a successful zero-loss update. This is a fail-stop policy, not transaction rollback: if an epoch transition has already drawn a new permutation, that sampler action is not undone by the exception.

4.3.3 Why averaging averages can be wrong

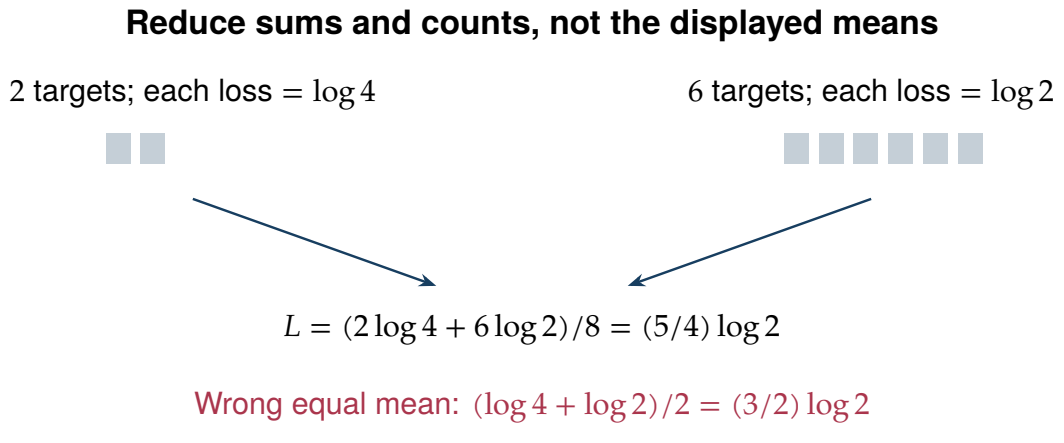


Figure 4.3: Two groups contribute unequal numbers of scored targets. Combining summed losses and dividing by the total count gives $(5/4)\ln 2$. Averaging the two group means gives $(3/2)\ln 2$, which weights groups equally rather than targets equally. The discrepancy is a change of objective, not floating-point noise.

Let group 1 contain two targets with mean loss $\ln 4$ and group 2 contain six targets with mean loss $\ln 2$. Their token mean is

$$\frac{2\ln 4 + 6\ln 2}{8} = \frac{5}{4}\ln 2, \tag{4.4}$$

whereas the mean of means is $(\ln 4 + \ln 2)/2 = (3/2)\ln 2$.

The same issue appears in validation aggregation and gradient accumulation. To reproduce one large token-mean batch using microbatches, sum each microbatch's loss contribution with the shared total target count as denominator. Then call the optimizer once. Equivalence also requires compatible forward behavior: dropout masks or batch-dependent normalization can make differently shaped executions disagree even when the loss algebra is correct. Our gradient-accumulation test disables dropout and uses a model with no batch normalization.

There is a subtler distinction. Uniformly sampling *rows* and using a token mean within each variable-length batch does not in general produce an unbiased gradient of the full-corpus token mean. A short row's targets can receive greater expected weight than a long row's. Our reference defines a specific batch-normalized training procedure; it does not claim an unbiased global-token estimator. Evaluation, in contrast, explicitly sums all validation token losses before dividing by their total count. If unbiased token sampling is the research objective, use an appropriate token-sampling or weighting design and test that separate contract.

4.4 One complete training transition

4.4.1 The 84-parameter predictor

The model embeds each of six input IDs into eight real coordinates, applies tanh, applies dropout with probability 0.25 during training, and maps the result to four logits. In symbols,

$$z = W\ \mathrm{Dropout}(\tanh E[x]) + b, \tag{4.5}$$

with $E \in \mathbb{R}^{6\times 8}$, $W \in \mathbb{R}^{4\times 8}$, and $b \in \mathbb{R}^4$. The stored parameter count is $48 + 32 + 4 = 84$. The padding embedding row is present in that count even though its ordinary lookup gradient is suppressed by the padding configuration.

This architecture has no attention, recurrence, convolution, or representation of a whole prefix. It is a small conditional-distribution learner, not a molecular simulation. Its biological alphabet gives us realistic masking and data-identity questions without claiming biological capability.

4.4.2 Momentum is state

We use stochastic gradient descent with learning rate $\eta = 0.15$, momentum $\mu = 0.9$, and no weight decay or Nesterov term. With zero initial momentum and gradient g_u at update u, the relevant recurrence is

$$v_u = \mu v_{u-1} + g_u, \qquad \theta_{u+1} = \theta_u - \eta v_u. \tag{4.6}$$

For the first nonempty update, the buffer equals that update's gradient, consistent with the chosen optimizer configuration. Model parameters θ and buffer v jointly determine the next parameter state.

Suppose $v_{u-1} \neq 0$ at an interruption. Restoring θ_u but resetting the buffer to zero changes the next update by $\eta\mu v_{u-1}$ even if the new gradient happens to be identical. Once parameters differ, future gradients can differ too. This explains why a checkpoint of weights alone is suitable for some inference uses but incomplete for exact continuation of this optimizer.

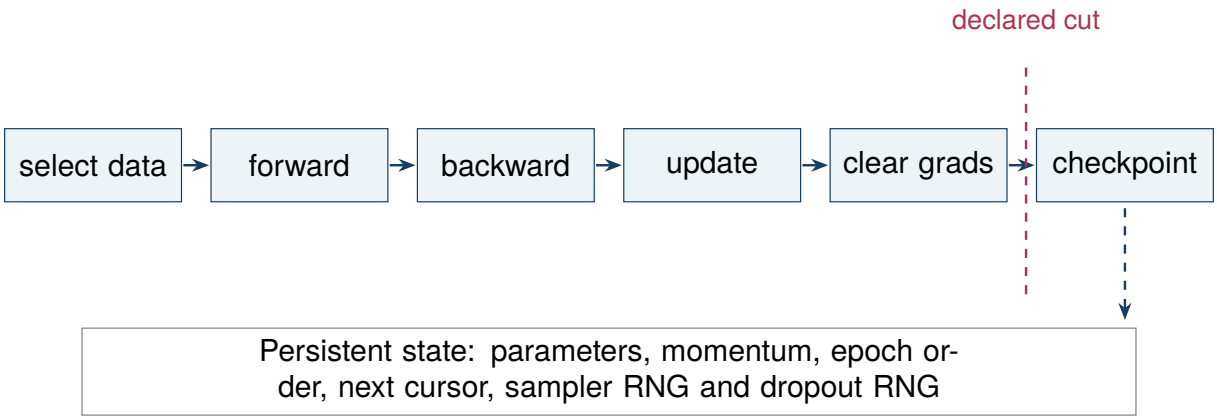


Figure 4.4: The supported checkpoint boundary is after a complete optimizer update, gradient clearing, and advancement of the data cursor and update counter. The lower state inventory supplies dependencies for the next transition. The dashed cut is a logical boundary in this teaching program, not a guarantee of atomic filesystem persistence. Checkpointing mid-backward or mid-accumulation is outside this implementation.

The update routine selects row indices, collates them, enters training mode, clears old gradients, computes S/C, runs backward, applies the optimizer, clears gradients again, and advances its cursor and update counter. Clearing at both boundaries makes the supported snapshot state unambiguous: no partial accumulated gradients need to be restored.

The sampler uses a separate seeded generator. It produces one permutation of the six row indices per epoch, with batches of two rows. The cursor points to the next unread position of the current permutation. At the end of a permutation, the next step draws a fresh permutation and advances the epoch counter. Saving only the seed would reconstruct the beginning of this random stream, not necessarily the current stream position.

4.5 Evaluation has two independent switches

Training versus evaluation mode and gradient recording answer different questions:

Module mode	Autograd context	Meaning in this model
`train()`	enabled	Dropout active; gradients recorded
`train()`	`no_grad()`	Dropout active; no gradient recording
`eval()`	enabled	Dropout inactive; gradients can be recorded
`eval()`	`no_grad()`	Dropout inactive; no gradient recording

Our validation routine uses the last row, combines loss sums and target counts, and then restores the model's previous mode. The test suite checks that this routine leaves parameters, the CPU random state, and the original module mode unchanged. It also checks that no parameter gradients appear.

This matters operationally. A validation pass that accidentally consumes the dropout random stream can perturb the next training step. A pass that leaves the model in evaluation mode can disable the intended training stochasticity. The safe behavior is a tested contract, not the mere presence of a method named `evaluate`.

4.6 A checkpoint is a serialized continuation state

4.6.1 What this implementation saves

A **checkpoint** is useful when it captures what is needed to perform the *next* supported transition. For this program, the state includes:

1. Model parameters and optimizer state, including momentum buffers.
2. The current permutation, its cursor, epoch, and completed-update count.
3. The sampler generator state and the CPU PyTorch random state used by dropout.
4. Schema version, training configuration, ordered-corpus digest, and exact PyTorch version string.

PyTorch's saving tutorial distinguishes model state from optimizer state and warns that an in-memory state dictionary can refer to changing tensors [22]. Our method serializes the snapshot immediately to a byte buffer. Continuing the original run therefore cannot mutate the already serialized snapshot. The test suite explicitly verifies this property.

The byte buffer is not a production durable checkpoint file. We do not implement temporary-file replacement, filesystem synchronization, remote-object checksums, crash recovery, or concurrent-writer coordination. A correct in-memory snapshot and crash-safe persistence are separate engineering obligations.

4.6.2 Restoration order matters

Restoration first verifies schema, configuration, corpus digest, and runtime version. It then constructs a new run, loads model and optimizer state, restores sampler state and cursor information, and restores the CPU random state *last*. Construction initializes parameters and therefore consumes randomness. Restoring the random state before construction would allow construction to consume the very stream positions intended for resumed training.

The loader uses `weights_only=True` with the tensor-and-primitive payload produced by this program. That is not a claim that arbitrary external files are trustworthy. The example restores its own local snapshots and provides no general untrusted-artifact validation system.

An incompatible corpus is rejected even if its tensor shapes happen to fit. Continuing with altered data may be a legitimate new experiment, but it is not exact continuation of the recorded one. A production system should make such a change explicit rather than disguising it as an identical resume.

4.6.3 Reproducibility has a boundary

Setting a seed specifies an initial random state. Saving generator state specifies a later position in a random stream. Neither alone fixes every possible platform-dependent behavior. PyTorch's versioned reproducibility notes explicitly limit guarantees across versions and platforms [21]. Accordingly, our claim is scoped to the tested CPU float64 environment with PyTorch 2.10.0 and one execution thread.

The example has no GPU kernels, distributed collectives, data-loader workers, automatic mixed precision, scheduler, gradient scaler, or asynchronous prefetch queue. Such components would add state and possibly nondeterministic operations. Listing them as future obligations is more useful than asserting a universal guarantee based on a small CPU test.

4.7 Measure the restart, including negative controls

4.7.1 The short training trace

We initialize once, evaluate the two fixed validation rows, execute exactly eight updates, and evaluate again. The eight-update budget is declared in advance; it is not chosen by selecting the best validation point. The generated trace is:

Update	Epoch	Targets	Training loss
1	0	8	1.530817
2	0	13	1.561957
3	0	11	1.329619
4	1	13	1.423917
5	1	11	1.346725
6	1	8	1.061230
7	2	8	0.972897
8	2	13	1.041860

The variable target counts arise from unequal sequence lengths and the ignored `N` target. Training loss is measured on the batch before its parameter update. It need not decrease monotonically across different batches with different dropout realizations.

The full validation token mean changes from approximately 1.523178 to 1.172270 nats per target. This is a descriptive result on two synthetic rows, not an estimate of genomic generalization or evidence of a new architecture's superiority. The recorded JSON contains the full precision trace, batch identities, configuration, and corpus digest.

4.7.2 A falsifiable exact-restart test

Interrupt the reference run after update 2. Serialize it, continue the original to update 8, and separately restore the snapshot and execute the same number of remaining updates. Compare parameter tensors, batch identities, loss traces, and optimizer tensors. Then repeat while deliberately omitting momentum, the dropout random state, or the correct cursor.

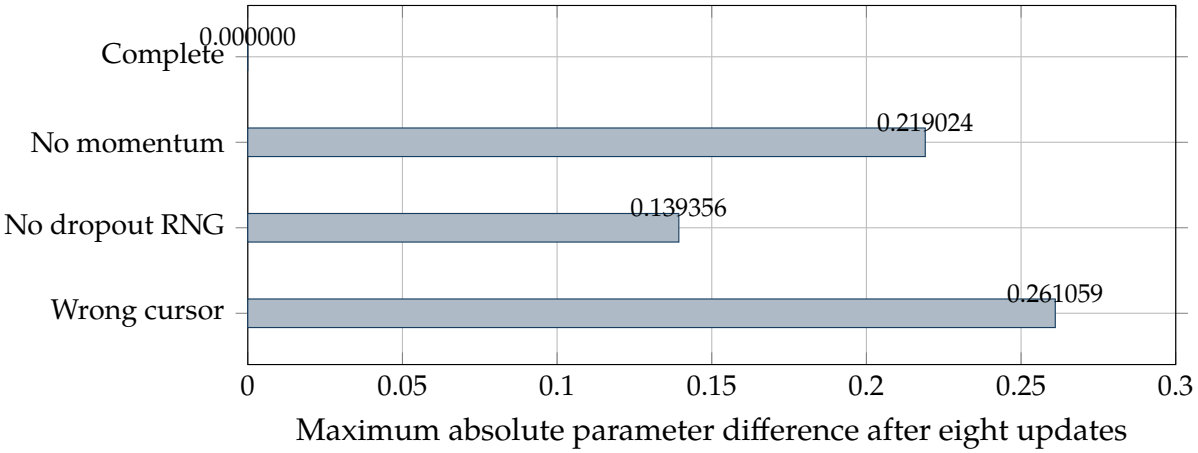


Figure 4.5: Measured restart errors in the synthetic CPU experiment. The complete snapshot produces zero maximum absolute parameter difference after eight updates. Removing momentum, omitting the dropout random state, or resetting the cursor produces distinct nonzero discrepancies. Values are generated from the executable negative controls, not hand-positioned illustrative bars.

Restored state	Max. error	Same rows?	Same losses?
complete	0.000000	yes	yes
missing_optimizer	0.219024	yes	no
missing_rng	0.139356	yes	no
missing_cursor	0.261059	no	no

The missing-momentum run still reads the same rows, but its losses and parameters diverge. The missing-random-state run also reads the same rows, isolating a different dependency. Resetting the cursor changes the rows themselves. These controls demonstrate that the test can detect several plausible restoration errors rather than merely checking that loading raises no exception.

The automated tests repeat exact restart at every cut after updates 1 through 7, including cuts at epoch boundaries. They compare final parameter values, optimizer state, row traces, and loss traces. Those are the tested equivalences; the experiment is not a proof about every possible model, interruption point, or execution platform.

Key ideas

- A training run has more state than its weights. Momentum, the data cursor and the dropout random stream all determine the next update.
- The token mean and the mean of group means are different objectives. Choose one deliberately and test it.
- Training mode and gradient recording are independent switches: `no_grad()` does not disable dropout.
- Exact replay is scoped to a tested environment, not promised across devices or library versions.

4.8 Exercises with worked answers

1. **Alignment.** Write the input and target arrays for `ACGT`.

Answer. Input IDs are $(0, 1, 2)$ and target IDs $(1, 2, 3)$. There are three predictions, not four. Appending an invented target for the last base changes the task.

2. **Unknown bases.** What happens for `NAC`?

   Answer. Inputs are $(4, 0)$ and targets $(0, 1)$. Both targets are scored: an unspecified input does not invalidate a known next-base label under this declared policy. For `AN`, the sole target is ignored.

3. **Golden loss.** Compute loss and one gradient coordinate for uniform four-class logits with three scored targets.

   Answer. The mean is $\ln 4$. At a scored position, the correct-class derivative is $(1/4 - 1)/3 = -1/4$ and each other-class derivative is $1/12$. An ignored position has zero derivative.

4. **Weighted aggregation.** Combine a two-target group with mean $\ln 4$ and a six-target group with mean $\ln 2$.

   Answer. The token mean is $(2 \ln 4 + 6 \ln 2)/8 = (5/4) \ln 2$. The mean of group means is a different objective, $(3/2) \ln 2$.

5. **Counterexample: global token weighting.** Uniformly choose one of a one-target row and a nine-target row, then optimize its row mean. Is this the full-corpus token mean in expectation?

   Answer. No. Each row receives probability $1/2$, whereas the global token mean weights the rows $1/10$ and $9/10$. This counterexample explains why batch normalization of variable-length rows is not automatically unbiased token sampling.

6. **Momentum.** If the previous buffer is $v = 2$, the new gradient is $g = 1$, $\mu = 0.9$, and $\eta = 0.15$, compare the parameter changes with and without restored momentum.

   Answer. Restored momentum gives $v' = 2.8$ and change -0.42. Reset momentum gives $v' = 1$ and change -0.15. The discrepancy is 0.27 before any later feedback through gradients.

7. **Mode switches.** Does `no_grad()` disable dropout?

   Answer. No. It controls gradient recording. Module evaluation mode disables dropout in this model. The switches are orthogonal, as the mode table shows.

8. **Snapshot aliasing.** Why not keep only `saved = model.state_dict()` and continue training?

   Answer. The resulting dictionary can reference live parameter storage. A later use may observe updated values rather than the intended earlier snapshot. Serialize immediately or make a suitable deep copy; the reference chooses immediate serialization.

9. **Random state.** Why is restoring the CPU random state the final restoration operation?

   Answer. Constructing the model initializes parameters and consumes random numbers. Restoring the intended continuation stream before construction would advance it prematurely.

10. **Empty target set.** Should a batch containing only unknown targets perform a momentum-only update?

    Answer. Not under this chapter's contract. Its mean objective is undefined, and the implementation raises before the optimizer step. A policy that intentionally performs some other transition would need a different explicit specification and tests.

11. **Data leakage.** Does an exact reverse-complement duplicate check establish a biologically independent validation split?

    Answer. No. It can reject exact equivalent strings, but cannot establish independence of overlapping loci, related organisms, homologous sequences, or experimental batches. Those require appropriate source metadata and study design.

12. **Investigation.** Extend the restart experiment to a scheduler or gradient accumulation.

    Rubric, ten points. Award two points each for a precise checkpoint boundary; an inventory of new mutable state; an uninterrupted/resumed comparison; an intentional missing-state control; and a declared environment plus reproducible test. For mid-accumulation restoration, partial gradients and accumulation counters must be addressed, not just the scheduler's scalar learning rate.

4.9 Read the implementation as a contract

The complete runnable companion is `drafts/ch04/reference.py`; its tests are in `tests/test_ch04_reference.py`. The following listings are extracted from the tested source at build time. Methods belong to the `Run` class and use its module's constants, imports, and helper functions. These are source excerpts, not separate standalone programs.

4.9.1 Alignment and target masking

```
def collate(rows):
    if not rows or any(
        len(seq) < 2 or set(seq) - set(ALPHABET) for _, seq in rows
    ):
        raise ValueError(
            "nonempty canonical ACGTN sequences of length at least two required"
        )
    length = max(len(seq) - 1 for _, seq in rows)
    x = torch.full((len(rows), length), PAD, dtype=torch.long)
    y = torch.full_like(x, IGNORE)
    for i, (_, seq) in enumerate(rows):
        ids = [ALPHABET.index(base) for base in seq]
        x[i, : len(ids) - 1] = torch.tensor(ids[:-1])
        targets = [v if v < 4 else IGNORE for v in ids[1:]]
        y[i, : len(targets)] = torch.tensor(targets)
    return x, y
```

```
def masked_loss(logits, targets):
    if logits.shape[:-1] != targets.shape or logits.shape[-1] != 4:
        raise ValueError("expected logits [B,T,4] and targets [B,T]")
    count = int((targets != IGNORE).sum())
    if count == 0:
        raise ValueError("an empty scored-target set has no mean loss")
    total = F.cross_entropy(
        logits.reshape(-1, 4),
        targets.reshape(-1),
        ignore_index=IGNORE,
        reduction="sum",
    )
    return total, count
```

4.9.2 A completed update

```
def step(self):
    if self.cursor == len(self.order):
        self.order = torch.randperm(
            len(self.corpus), generator=self.sampler
        ).tolist()
        self.cursor = 0
        self.epoch += 1
    ids = self.order[self.cursor : self.cursor + CONFIG["batch_size"]]
    x, y = collate([self.corpus[i] for i in ids])
    self.model.train()
    self.optimizer.zero_grad(set_to_none=True)
    total, count = masked_loss(self.model(x), y)
    loss = total / count
    loss.backward()
    self.optimizer.step()
    self.optimizer.zero_grad(set_to_none=True)
    self.cursor += len(ids)
    self.updates += 1
    return {
        "update": self.updates,
        "epoch": self.epoch,
        "ids": [self.corpus[i][0] for i in ids],
        "scored_targets": count,
        "loss": float(loss.detach()),
    }
```

4.9.3 Serialization and restoration

```
def checkpoint(self):
    state = {
        "schema": 1,
        "config": CONFIG,
        "corpus_sha256": corpus_digest(self.corpus),
        "torch_version": str(torch.__version__),
        "model": self.model.state_dict(),
        "optimizer": self.optimizer.state_dict(),
        "sampler_rng": self.sampler.get_state(),
        "torch_rng": torch.get_rng_state(),
        "order": self.order,
        "cursor": self.cursor,
        "epoch": self.epoch,
        "updates": self.updates,
    }
    stream = io.BytesIO()
    torch.save(
        state, stream
    )  # serialize now, not a live state_dict alias
    return stream.getvalue()
```

```
@classmethod
def restore(cls, blob, corpus=CORPUS, omit=None):
    state = torch.load(
        io.BytesIO(blob), map_location="cpu", weights_only=True
    )
    if state["schema"] != 1 or state["config"] != CONFIG:
        raise ValueError("checkpoint schema/config mismatch")
    if state["corpus_sha256"] != corpus_digest(corpus):
        raise ValueError("checkpoint corpus/order identity mismatch")
    if state["torch_version"] != str(torch.__version__):
        raise ValueError(
            "exact replay requires the recorded PyTorch version"
        )
    run = cls(corpus)  # initialization consumes RNG before restoration
    run.model.load_state_dict(state["model"])
    if omit != "optimizer":
        run.optimizer.load_state_dict(state["optimizer"])
    run.sampler.set_state(state["sampler_rng"])
    run.order = list(state["order"])
    run.cursor = 0 if omit == "cursor" else state["cursor"]
    run.epoch = state["epoch"]
    run.updates = state["updates"]
    if omit != "rng":
        torch.set_rng_state(
            state["torch_rng"]
        )  # restore after constructing modules
    return run
```

4.10 Selective glossary and the next question

Scored target A position whose known label contributes to the declared objective.

Token mean Sum of per-target losses divided by the number of scored targets.

Data lineage The source relationships among rows, windows, and derived variants.

Momentum buffer Optimizer memory that carries information from earlier gradients.

Checkpoint A snapshot sufficient for the next supported training transition.

Exact replay Equality under a specified execution environment and tested comparison contract, not a universal cross-platform guarantee.

Negative control An intentionally broken condition used to show that a verification procedure can detect the corresponding failure.

We have made a small training run understandable, interruptible, and testable. The next question is not how to rename this baseline as a richer architecture. It is what additional representation, computation, and evidence are needed when predictions should depend on longer genomic context. Each added capability must preserve an explicit objective, a valid data boundary, and a verifiable continuation state.

Evidence and illustration boundaries

The corpus, model, derivations, worked examples, and native vector figures are original teaching constructions. The references support library behavior, not genomic capability claims. The experiment is limited to a synthetic CPU float64 training run; no real genomic generalization, integrated research architecture, or universal restart guarantee is established. Technical and visual checks by the authoring workflow are not independent specialist review.

CHAPTER 5

Tokenization and sequence representation

In this chapter you will

- define a token by its base interval as well as its text
- build fixed-width base-four codes and a model vocabulary that keeps every short tail
- detect objectives in which overlapping tokens reveal part of their own target
- transform spans and token boundaries under reverse complementation

A model does not receive a DNA molecule, or even a string. It receives numbers whose meaning comes from a representation contract. Before we train a larger model, we must be able to explain every integer, every tensor axis, and every base visible at a prediction step.

Our running sequence is `ACGTACG`. We will turn it into tokens, integer IDs, and embedding vectors, then reconstruct it exactly. Along the way we will build two negative controls: an overlapping-token objective that partly reveals its own answer, and a strand transformation that breaks when token boundaries are treated as biological boundaries.

5.1 Decide what a token means

For base-level tokens, the sequence becomes seven tokens: `A`, `C`, `G`, `T`, `A`, `C`, `G`. A **k-mer** is a contiguous word of k bases. For $k = 3$, we have at least two reasonable segmentations:

- overlapping windows: `ACG`, `CGT`, `GTA`, `TAC`, `ACG`;
- disjoint chunks with the short tail retained: `ACG`, `TAC`, `G`.

Both preserve the sequence if their decoding rules are known. They are not interchangeable inputs to the same prediction objective.

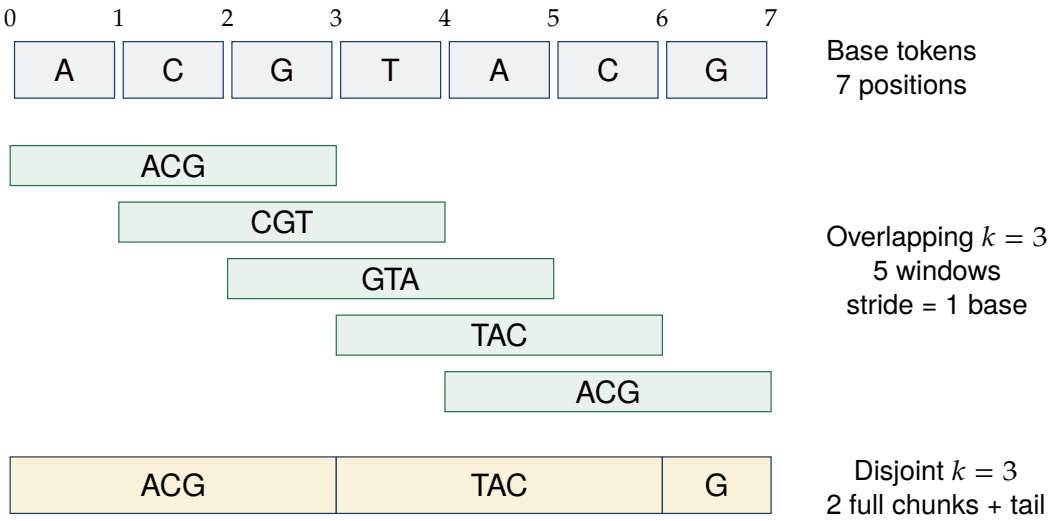


Figure 5.1: Three representations of `ACGTACG`. Colored bars denote source intervals, not chemical bonds. Overlapping windows share bases; disjoint chunks retain the terminal one-base token. Every token carries a half-open interval in the original sequence.

For a length-L sequence with $L \geq k$, stride-one full windows give

$$T_{\text{overlap}} = L - k + 1. \tag{5.1}$$

Disjoint chunks with a retained tail give

$$T_{\text{disjoint}} = \lceil L/k \rceil . \tag{5.2}$$

Our implementation additionally preserves short nonempty records with $L < k$ as one short token; empty records produce none. The first formula does not describe that fallback. Naming the edge case is better than forcing a negative token count into a formula written for a different domain.

Each `Token` contains `text`, `start`, and `stop`. Offsets count bases, not bytes or token positions.

```
def tokenize(sequence, k, mode="overlap"):
    validate(sequence)
    if type(k) is not int or k < 1 or mode not in ("overlap", "disjoint"):
        raise ValueError("positive k and a declared mode required")
    if mode == "disjoint":
        return [
            Token(sequence[i : i + k], i, min(i + k, len(sequence)))
            for i in range(0, len(sequence), k)
        ]
    if len(sequence) < k:
        return [Token(sequence, 0, len(sequence))] if sequence else []
    return [
        Token(sequence[i : i + k], i, i + k)
        for i in range(len(sequence) - k + 1)
    ]
```

We accept uppercase canonical A/C/G/T only. Ambiguity symbols are rejected, not silently mapped to a universal unknown token. Such a mapping would be lossy: N, R, and a malformed character would become indistinguishable. An ambiguity-aware extension should preserve the sets introduced in DNA Computing Chapter 5 and explicitly define how the model consumes them.

5.2 Build an integer code from first principles

Assign digits $d(A) = 0$, $d(C) = 1$, $d(G) = 2$, and $d(T) = 3$. For a fixed-width word $w = w_0 \dots w_{k-1}$, its base-four ID is

$$\mathrm{id}_k(w) = \sum_{j=0}^{k-1} d(w_j) 4^{k-1-j}. \tag{5.3}$$

For `ACG`, this is $0 \cdot 16 + 1 \cdot 4 + 2 = 6$. There are exactly 4^k fixed-width words and IDs 0 through $4^k - 1$. The ID orders symbols; it does not measure biochemical similarity.

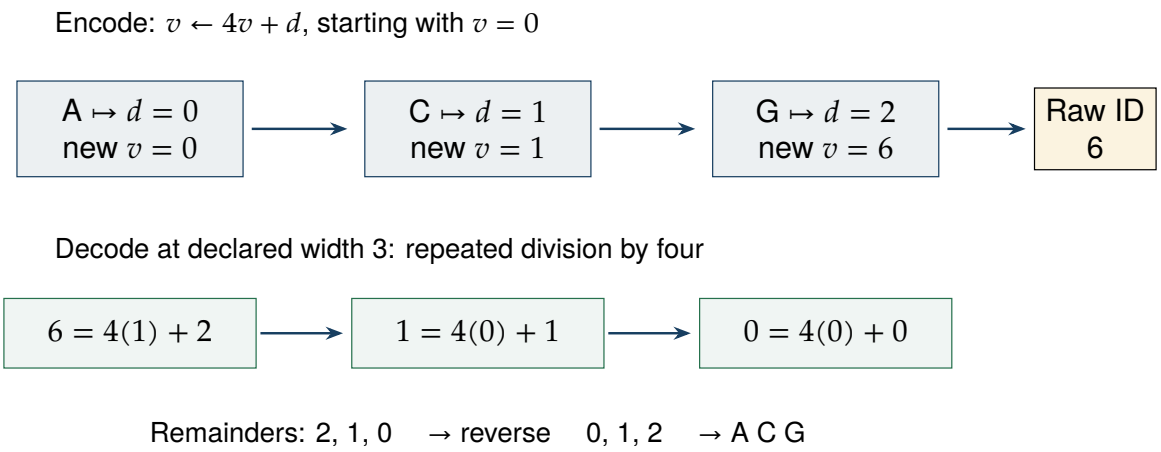


Figure 5.2: Encoding and decoding the same word. Horner's rule accumulates ID 6 from digits 0, 1, 2. Repeated division by four recovers remainders 2, 1, 0; reversing them restores ACG. Declared width preserves the leading A.

```
def encode_kmer(word):
    validate(word)
    if not word:
        raise ValueError("a k-mer must be nonempty")
    value = 0
    for base in word:
        value = 4 * value + BASES.index(base)
    return value
```

```
def decode_kmer(value, k):
    if (
        type(k) is not int
        or k < 1
        or type(value) is not int
        or not 0 <= value < 4**k
    ):
        raise ValueError("integer ID must fit the declared positive k")
    output = []
    for _ in range(k):
        value, digit = divmod(value, 4)
        output.append(BASES[digit])
    return "".join(reversed(output))
```

The decoder needs k. ID zero could mean `A`, `AA`, or `AAA` at different widths. A representation that discards width cannot recover leading zero digits. Our tests enumerate every word for $k = 1, 2, 3, 4$ and verify both directions of the bijection.

5.2.1 A model vocabulary is not the raw k-mer code

The teaching vocabulary reserves IDs 0, 1, and 2 for PAD, BOS, and EOS. It then lists all one-base words, all two-base words, and so on through width k, in base-four order. This policy retains every possible short tail. For a width-j word, the model ID is

$$\text{model_id}(w) = 3 + \sum_{r=1}^{j-1} 4^r + \text{id}_j(w). \tag{5.4}$$

At $k = 3$, `ACG` therefore has model ID $3 + 4 + 16 + 6 = 29$, not 6. `TAC` has ID 72 and the short token `G` has ID 5. Our disjoint record stores IDs $[29, 72, 5]$ alongside spans $[0, 3)$, $[3, 6)$, and $[6, 7)$.

Special symbols are reserved here; the sequence encoder does not insert BOS or EOS, and the sequence decoder rejects all special IDs. A later batching layer can add controls, but it must remove or interpret them before calling this sequence-only decoder. Reserving an ID and actually emitting it are different design decisions.

5.3 Make the round trip enforce a contract

For non-overlapping tokens, concatenation looks sufficient. For overlapping tokens, it would duplicate bases: `ACG` followed by `CGT` must reconstruct `ACGT`, not `ACGCGT`. Offsets tell us exactly how much overlap should agree.

```
def reconstruct(tokens):
    """Accept overlaps only when all shared bases agree; reject gaps."""
    output = ""
    for token in tokens:
        validate(token.text)
        if (
            type(token.start) is not int
            or type(token.stop) is not int
            or not token.text
            or not 0 <= token.start <= len(output)
            or token.stop != token.start + len(token.text)
            or token.stop <= len(output)
        ):
            raise ValueError(
                "tokens must extend an ordered gap-free sequence"
            )
        overlap = len(output) - token.start
        if output[token.start :] != token.text[:overlap]:
            raise ValueError("overlap disagrees")
        output += token.text[overlap:]
    return output
```

The function checks that each token extends a gap-free prefix, that its declared length matches its text, and that overlapping bases agree. For example, `ACG` at $[0, 3)$ followed by `CTT` at $[1, 4)$

must fail: the shared region is `CG` in the first token but `CT` in the second. A decoder that simply takes the final character would conceal the inconsistency.

This low-level function accepts any consistent extending spans. The higher level `decode_record` also checks that the decoded tokens equal the declared overlap or disjoint segmentation of the reconstructed sequence. It rejects mismatched vocabulary identities and control IDs. The record therefore specifies not just a string, but how that string became model input.

We hash a canonical serialization of alphabet order, k, tail policy, and the ordered vocabulary. The digest is an identity check, not authentication against a malicious party. Store the mode and offsets in the record too. Two vocabularies with equal size can give the same embedding row completely different meanings; shape compatibility alone is not checkpoint compatibility.

5.4 Count the representation cost

With retained short words and three special symbols, vocabulary size is

$$V(k) = 3 + \sum_{j=1}^{k} 4^j = 3 + \frac{4^{k+1} - 4}{3}. \tag{5.5}$$

A float32 embedding table with width D requires $4V(k)D$ bytes for its parameter values. This excludes gradients, optimizer state, activations, and allocation overhead.

k	4^k	Vocabulary	Disjoint T	Overlap T	Bytes
1	4	7	1000	1000	1792
2	16	23	500	999	5888
3	64	87	334	998	22272
6	4096	5463	167	995	1398528

The table uses $L = 1000$ and $D = 64$. At $k = 6$, a disjoint representation has 167 tokens instead of 1000, but the retained-tail vocabulary has 5,463 entries instead of seven at $k = 1$. Its embedding values occupy 1,398,528 bytes. Overlapping width-six windows still have 995 tokens: a larger vocabulary has barely shortened the sequence at all.

This is a parameter accounting exercise, not a throughput benchmark. For an architecture with dense all-pairs attention, reducing token count can reduce a quadratic interaction term; other costs scale differently. A recurrent model, a convolution, and a state-space model have different resource profiles. Tokenization should be evaluated against the actual architecture and objective, not declared superior from one column.

5.5 An embedding is a row lookup

Let $E \in \mathbb{R}^{V \times D}$ be a learned table and let integer IDs $I \in \{0, \dots, V-1\}^{B \times T}$ index its rows. The output is

$$X_{b,t,:} = E_{I_{b,t},:}, \qquad X \in \mathbb{R}^{B \times T \times D}. \tag{5.6}$$

Equivalently, if $Q_{b,t,:}$ is one-hot over the vocabulary, the forward calculation is $X_{b,t,:} = Q_{b,t,:}E$. We normally avoid materializing Q: integer lookup expresses the operation without the mostly-zero tensor. PyTorch documents the same input/output shape convention [23].

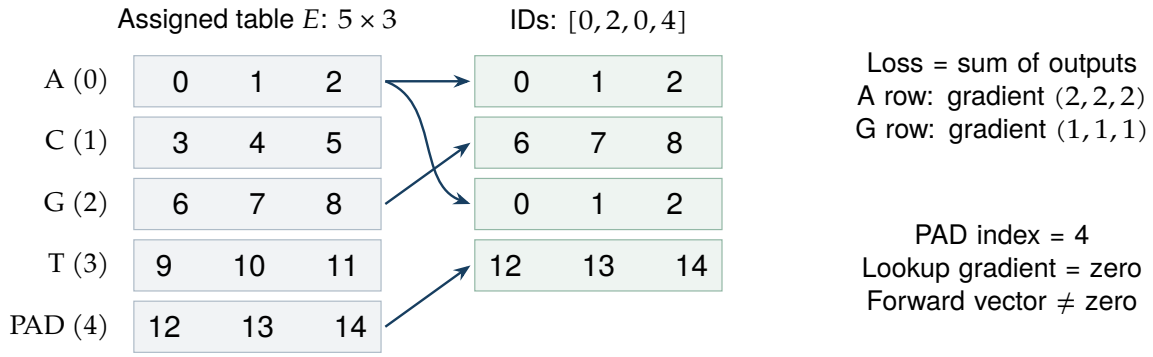


Figure 5.3: A tiny lookup with repeated IDs. A is selected twice, so both positions receive the same row before positional information is added. Under a sum loss, its lookup gradients accumulate twice. PAD's assigned nonzero vector illustrates why a padding-gradient rule is not an attention or loss mask.

For this isolated demonstration only, use the smaller ID scheme A=0, C=1, G=2, T=3, PAD=4. It is deliberately separate from the retained-tail vocabulary above. Set E to the numbers 0 through 14 in a 5×3 matrix and look up $I = [0, 2, 0, 4]$. The output is

$$X = \begin{bmatrix} 0 & 1 & 2 \\ 6 & 7 & 8 \\ 0 & 1 & 2 \\ 12 & 13 & 14 \end{bmatrix}. \tag{5.7}$$

These assigned values make the calculation inspectable. They are not learned biochemical coordinates or evidence that A and C differ by a distance of three.

```
def embedding_example():
    # Deterministic assigned numbers, not learned biochemical coordinates.
    weight = (
        torch.arange(15, dtype=torch.float64)
        .reshape(5, 3)
        .clone()
        .requires_grad_()
    )
    ids = torch.tensor([[0, 2, 0, 4]])
    selected = F.embedding(ids, weight, padding_idx=4)
    one_hot = F.one_hot(ids, 5).to(torch.float64) @ weight
    selected.sum().backward()
    return {
        "ids": ids.tolist(),
        "vectors": selected.detach().tolist(),
        "one_hot_matches": torch.equal(selected, one_hot),
        "gradient": weight.grad.tolist(),
        "padding_value": selected[0, 3].detach().tolist(),
    }
```

For a scalar loss $\mathcal{L} = \sum X$, an ordinary row receives one unit of gradient per occurrence in every feature:

$$\frac{\partial \mathcal{L}}{\partial E_{v,d}} = \sum_{b,t:I_{b,t}=v} 1, \tag{5.8}$$

except for the explicit padding rule. The row gradients in our example are $[2,2,2]$, $[0,0,0]$, $[1,1,1]$, $[0,0,0]$, and $[0,0,0]$.

The last row deserves close attention. PyTorch's `padding_idx` suppresses its gradient contribution from embedding lookup; it does not force an explicitly supplied weight row to zero [23]. Our PAD output remains $[12,13,14]$. Newly initialized `nn.Embedding` padding rows default to zero, but that initialization is not a universal guarantee about supplied weights or downstream operations.

The one-hot multiplication matches the *forward values* here, not the custom padding backward rule. Backpropagating through plain one-hot matrix multiplication would give the selected PAD row a gradient too. A padding ID also does not automatically mask attention, recurrent state updates, or the training loss. Chapter 4's valid-target mask remains necessary.

5.6 Track which bases a prediction can see

Suppose the current input token is `ACG` and the next overlapping token is `CGT`. Two of the three target bases, `CG`, are already present in the input. This is not necessarily an invalid objective, but its score must not be interpreted as predicting three unseen bases.

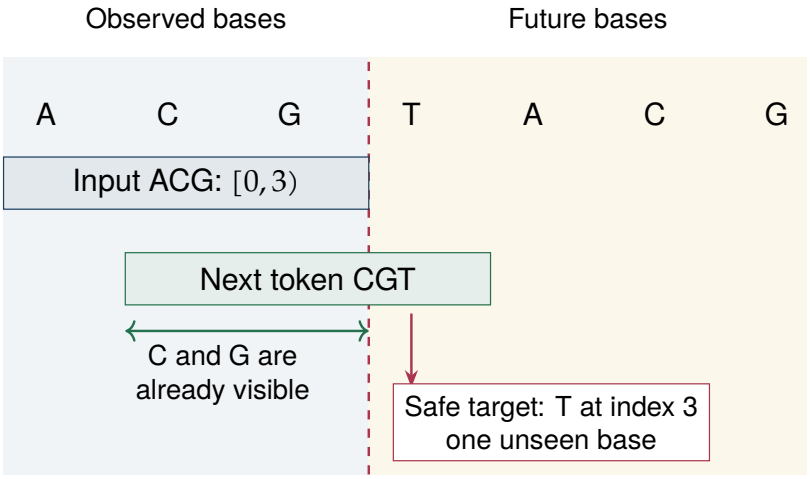


Figure 5.4: The information boundary is a base coordinate. ACG ends at boundary 3. The next overlapping token CGT contains two already-observed bases and only one new base. Predicting the base at index 3 makes the unit of novelty explicit. A token-level causal mask cannot remove information already inside an input token.

Under an illustrative independent uniform base process, a fresh three-base word has entropy $3\log 4 = \log 64$ nats. Conditional on the preceding overlapping word, two bases are fixed; only the new base is uncertain, so the conditional entropy is $\log 4$. This is an analytical toy result, not an empirical estimate of genomic entropy. Comparing those two losses without the conditioning context would exaggerate the improvement.

For a transparent baseline, let each window predict the next unseen base. Our function returns its input word, the target base, and the target index:

```
def safe_next_base_examples(sequence, k):
    return [
        (t.text, sequence[t.stop], t.stop)
        for t in tokenize(sequence, k)
        if t.stop < len(sequence)
    ]
```

For the running sequence, the examples are `ACG→T` at 3, `CGT→A` at 4, `GTA→C` at 5, and `TAC→G` at 6. The terminal `ACG` has no next-base target and is omitted. This defines local training pairs; it does not yet implement a full sequence model, its batching, or its evaluation split.

A useful negative control changes a future base while holding the prefix fixed. The input representation for a prediction must remain unchanged. Our test changes the base at index 3 and verifies that the first input token is still `ACG` while its target changes. If a tokenizer were allowed to include that base inside the current token, a later causal mask would not repair the leak.

The same reasoning applies to masked-token objectives: masking one k-mer may leave its bases visible inside neighboring overlapping k-mers. Define the intended hidden base interval first, then remove every input feature that reveals it. Also split raw records or biological groups before making windows; random window splitting can put near-duplicates across evaluation boundaries. Representation correctness is necessary but does not establish evaluation independence.

5.7 Reverse the strand, then reconsider the boundaries

For full stride-one windows of width k and $L \geq k$, reversing the token order and reverse-complementing each token produces the windows of the reverse-complemented sequence. A span $[a, b)$ becomes $[L - b, L - a)$, as derived in DNA Computing Chapter 5.

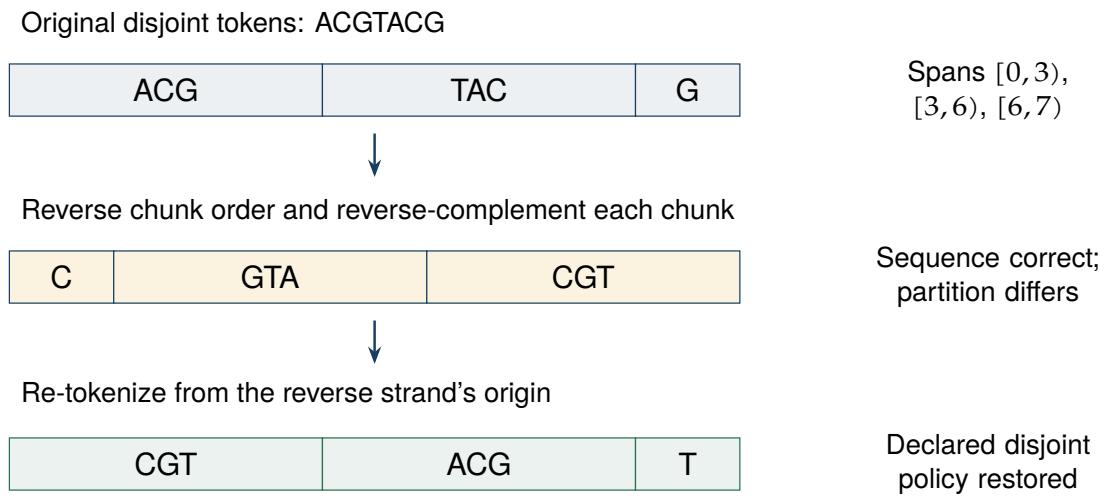


Figure 5.5: Reverse complementation changes the sequence coordinate system. With overlapping full windows, reversing window order and each window's bases works. With disjoint chunks and a short tail, that operation leaves the short chunk at the front; re-tokenizing from the new origin gives different boundaries.

For our length-seven sequence, disjoint tokens are `ACG|TAC|G`. Reversing token order and complementing gives `C|GTA|CGT`. Concatenation correctly yields `CGTACGT`, but disjoint re-tokenization from the new origin yields `CGT|ACG|T`. The sequence is right and the segmentation is wrong. Token boundaries are a software choice, not covalent breaks in DNA.

Token index alone is also insufficient to identify a base position. An overlapping width-three token at index j starts at base j; a disjoint full-width token at index j starts at $3j$. Preserve base spans and decide whether positions describe starts, centers, or something else before adding positional features. A position-aware embedding can distinguish equal tokens in different locations; the lookup table alone cannot.

Finally, consider **canonicalization**: store the lexicographically smaller of s and $\mathrm{rc}(s)$. It merges `AAA` and `TTT`. That may be suitable for a strand-invariant identity task, but it loses orientation unless we keep a flip flag. Direction-dependent labels must be transformed consistently. Even with a flag, a model that never receives or uses it cannot learn two different labels for identical canonicalized inputs. Palindromic ties use a deterministic no-flip convention.

5.8 A representation checklist before training

The completed artifact includes strict alphabet validation, fixed-width integer codes, two segmentation policies, retained tails, offset-checked reconstruction, vocabulary identity checks, an embedding gradient example, and causal-reach tests. To reproduce the golden results and test the contracts:

```
python3 drafts/ch05/reference.py
python3 -m pytest tests/test_ch05_reference.py
```

The computed record is saved in `drafts/ch05/results.json`.

Before reusing the data with Chapter 6's recurrent models, answer four questions. Can every record round-trip exactly? Does a checkpoint agree on what each row means? Does each target lie beyond the allowed input boundary? Are strand-dependent labels and coordinates still aligned? Training cannot recover information a representation has discarded.

These examples do not train a genomic foundation model. DOGMA remains the non-Transformer DNA-native research direction; Hermon DNA denotes the Transformer-based direction; Evolutor is the broader framework. Tokenization contracts apply to either architecture without proving either system exists as a trained or validated biological model.

Key ideas

- An integer ID decodes only together with its declared width: ID 1 is `C` at width one and `AAC` at width three.
- With stride-one k-mers, the next token contains only one unseen base. Conditional entropy is $\ln 4$, not $k \ln 4$.
- Token boundaries are a software choice, not covalent breaks. Disjoint chunks do not commute with reverse complementation.
- A checkpoint must carry the vocabulary identity. Equal table shapes can give embedding rows entirely different meanings.

5.9 Exercises with worked solutions

1. Predict the tail

Tokenize `ACGTA` disjointly at $k = 3$. **Solution:** `ACG` at $[0, 3)$ and `TA` at $[3, 5)$. Dropping the tail loses two bases; padding is a different policy requiring its own length or mask metadata.

2. Encode without a table

Find the raw width-three ID of `TGA`. **Solution:** $3 \cdot 16 + 2 \cdot 4 + 0 = 56$. Its model ID in our retained-tail vocabulary is $23 + 56 = 79$.

3. Recover leading zeros

Decode ID 1 at widths one and three. **Solution:** `C` and `AAC`. The integer alone does not encode width.

4. Detect inconsistent overlap

Why reject `ACG` at $[0, 3)$ followed by `CTT` at $[1, 4)$? **Solution:** shared bases should be `CG` but the second token claims `CT`. Taking only the new suffix would hide the disagreement.

5. Count vocabulary entries

Compute the retained-tail vocabulary for $k = 2$. **Solution:** $3 + 4 + 16 = 23$. At embedding width 64 in float32, parameter values occupy $23 \cdot 64 \cdot 4 = 5{,}888$ bytes, excluding training state.

6. Follow a repeated lookup

For IDs $[2, 2, 2]$ and a sum loss, what gradient reaches row 2? **Solution:** three in every feature, unless that row is the declared padding index. Repeated lookup shares parameters; it does not create three independent rows.

7. Separate padding mechanisms

Does a zero-gradient padding row imply padding cannot affect a model? **Solution:** no. Its forward vector can be nonzero, positional features can alter it, and downstream operations still need appropriate masks. Lookup-gradient suppression is not a universal padding exclusion rule.

8. Measure novelty

How many unseen bases are in the next stride-one width-six token? **Solution:** one; five overlap. Under independent uniform bases, its conditional uncertainty is $\log 4$, not $\log 4096$ nats.

9. Audit a causal mask

A token covering $[0, 3)$ is used to predict base 2. Can a token-level causal mask fix the problem? **Solution:** no. Base 2 is already inside the token. Change token reach or move the target beyond the token's stop.

10. Reverse an uneven partition

Reverse-complement the tokens `ACG|TAC|G`, then re-tokenize. **Solution:** transformed chunks `C|GTA|CGT` concatenate correctly, but the new disjoint partition is `CGT|ACG|T`.

11. Preserve directional labels

A dataset labels `AAA` and `TTT` differently. What happens after canonicalization without orientation metadata? **Solution:** both become `AAA`. A deterministic predictor sees identical inputs with incompatible labels. Keep and use orientation, transform labels appropriately, or choose a genuinely strand-invariant task.

12. Specify a checkpoint contract

What must accompany embedding weights? **Worked rubric:** vocabulary ordering and identity, alphabet policy, special-ID meaning, segmentation and tail policy, positional convention, architecture dimensions, and preprocessing version. Shape checks are necessary but insufficient. Add a golden raw-sequence-to-ID example and reject a deliberately permuted vocabulary even when its dimensions match.

Biological and modeling boundaries

The strand convention follows the explicitly derived orientation rules in DNA Computing Chapter 5. The tokenizer here accepts canonical uppercase DNA only and is not an ambiguity-aware biological model. Uniform-base entropy calculations are hypothetical baselines. Assigned embedding vectors are not trained representations; resource counts exclude optimizer state and are not measured runtime benchmarks.

CHAPTER 6

Recurrent models and gated state

In this chapter you will

- unroll a recurrence and derive the Jacobian product that governs long-range gradients
- implement LSTM and GRU cells and check values and gradients against PyTorch
- hold state through padding, reset it at record boundaries and carry it across chunks
- separate the numerical value of carried state from its gradient history

Read the bases A, C, G, T one at a time. After the last base arrives, what information about the first base is still available? A fixed-width window answers by retaining a specified recent interval. A recurrent model instead maintains a state that it updates at every step. The state is a learned representation of history, not an automatically lossless copy of it.

This chapter builds recurrent arithmetic from scratch. We first expose the state transition, then add gates, then test what changes at padding and chunk boundaries. The main implementation is a manually written GRU cell, checked against PyTorch in both forward values and gradients. A smaller scalar recurrence gives exact numbers that can be verified without a library.

We assume Chapter 5's embedding shapes and Chapter 4's loss and gradient conventions. All numerical probes use assigned weights or synthetic inputs. They establish properties of the computation, not trained genomic capability.

6.1 Unroll one state transition

Let $x_t \in \mathbb{R}^D$ be the embedding of the base arriving at step t, and let $h_{t-1} \in \mathbb{R}^H$ be the previous hidden state. A simple **recurrent neural network** updates

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \tag{6.1}$$

where $W_x \in \mathbb{R}^{H \times D}$, $W_h \in \mathbb{R}^{H \times H}$, and $b \in \mathbb{R}^H$. The hyperbolic tangent acts elementwise. The same parameters are reused at every step.

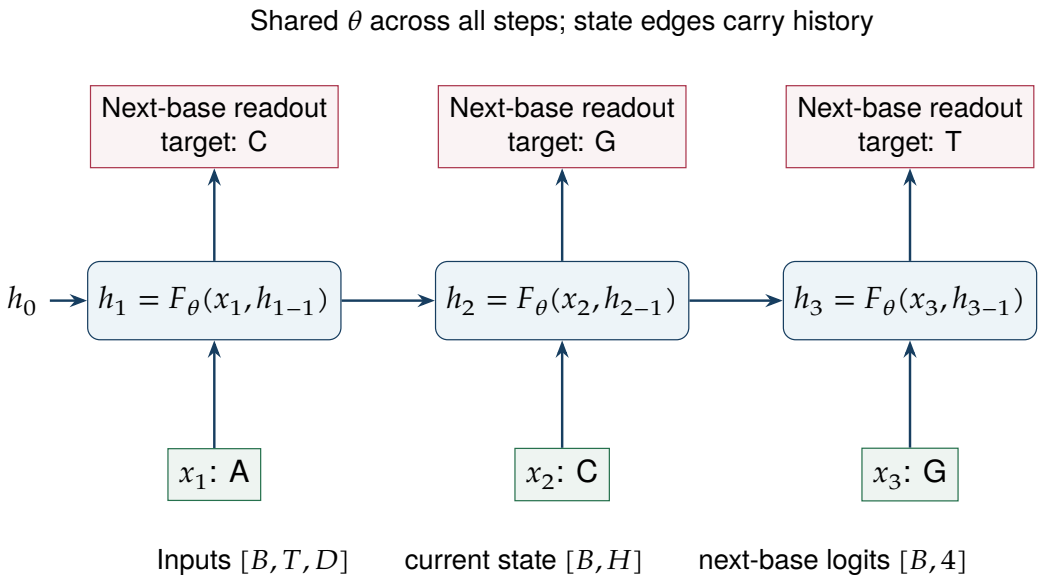


Figure 6.1: A causal recurrence unrolled over three observed bases. Each state uses the current embedding and the previous state; the shared transition parameters do not become separate trainable copies. A readout after observing A predicts the next base, C. Arrows show computation dependencies, not biological reactions.

For a batch, the code stores x as $[B, T, D]$ and each state as $[B, H]$. The mathematical column-vector multiplication becomes `F.linear(x, weight, bias)` for batch rows: PyTorch applies the transpose of the stored $[H, D]$ matrix to those rows. Confusing tensor storage with the mathematical map is a common source of accidentally transposed weights.

For next-base prediction, a readout produces

$$\ell_t = W_y h_t + b_y \in \mathbb{R}^4, \qquad p(s_{t+1} \mid s_{1:t}) = \mathrm{softmax}(\ell_t). \tag{6.2}$$

The state at step t has already consumed s_t. Scoring it against s_t would let it predict a base present in its own input. A unidirectional recurrence is causal only relative to correctly aligned inputs and targets.

We begin each independent record with a declared initial state, here zero. Two adjacent records in a data loader are not adjacent pieces of one genome unless the data contract says so. Carrying state across unrelated records creates an undeclared input channel.

6.2 Why the gradient can lose the distant past

For a single example, define $a_t = W_x x_t + W_h h_{t-1} + b$. Differentiating Eq. 6.1 gives

$$J_t = \frac{\partial h_t}{\partial h_{t-1}} = \mathrm{diag}(1 - h_t^2) W_h. \tag{6.3}$$

The influence of an earlier state on a later one is a product:

$$\frac{\partial h_T}{\partial h_k} = J_T J_{T-1} \cdots J_{k+1}. \tag{6.4}$$

Repeated contractions can suppress a signal; expanding directions can amplify it. Saturated tanh coordinates contribute small local derivatives. The foundational LSTM paper motivates gated memory through this long-range gradient problem [24].

A scalar linear recurrence makes the product visible without nonlinear complications. If $h_t = \alpha h_{t-1}$, then $h_T = \alpha^T h_0$ and $\partial h_T / \partial h_0 = \alpha^T$. For $\alpha = 0.5$, ten steps retain $1/1024$ of a perturbation; for $\alpha = 1.1$, amplification grows with the number of steps. Increasing a learning rate does not restore information removed by the forward state or turn a badly conditioned Jacobian product into a stable one.

Shared weights also receive contributions from multiple time steps. A gradient with respect to W_h is not merely the final state's local derivative: each earlier occurrence of the same parameter can affect later states. Our chunk experiment will expose what happens when some of these paths are deliberately cut.

6.3 Separate retention, writing, and reading

A modern **long short-term memory** cell carries two vectors, hidden output h_t and cell state c_t, both in $\mathbb{R}^H$. Let the input gate i_t, forget gate f_t, and output gate o_t be sigmoid outputs, and let g_t be a tanh candidate:

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i), \tag{6.5}$$
$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f), \tag{6.6}$$
$$g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g), \tag{6.7}$$
$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o). \tag{6.8}$$

The state update is

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{6.9}$$

Here $\odot$ is elementwise multiplication. These are the modern forget-gate equations used by PyTorch's unprojected cell [25], not a claim that every feature of this variant appeared in the 1997 paper.

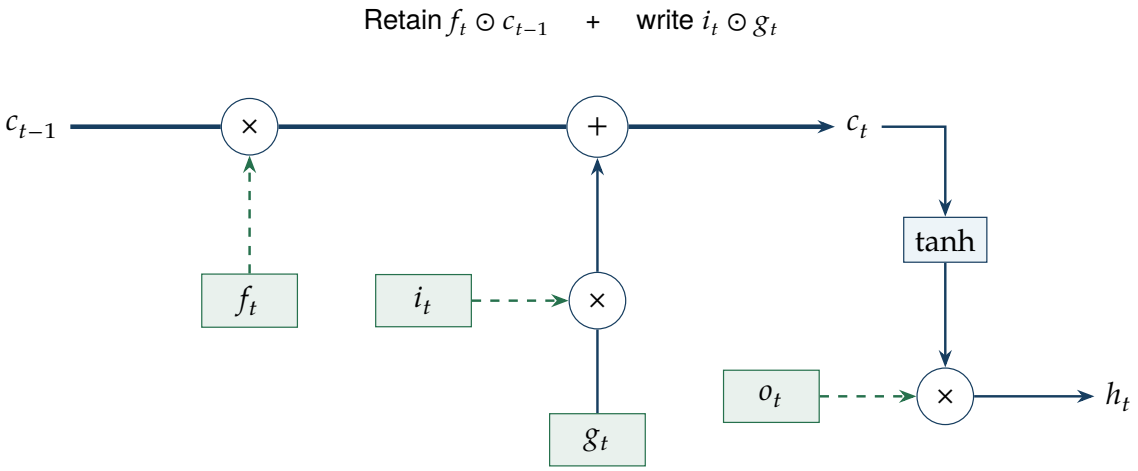


Figure 6.2: Separate LSTM state paths. The forget gate scales retained cell contents; the input gate scales a candidate write; their sum updates the cell. The output gate controls the exposed tanh-transformed state. Dashed arrows deliver gate values, while solid arrows carry state or candidate values. This diagram omits peepholes and projections.

Take one coordinate with $c_{t-1} = 2, f_t = 0.75, i_t = 0.2, g_t = -0.5$, and $o_t = 0.6$. Retention contributes 1.5, writing contributes -0.1, and $c_t = 1.4$. The output is $0.6 \tanh(1.4) \approx 0.531$. The cell contents and the exposed output are not the same number.

Holding gates and candidate fixed, the direct derivative along the cell path is $\partial c_t / \partial c_{t-1} = f_t$ coordinatewise. If f_t remains near one and writes are small, that path can preserve information. But the *total* derivative also includes paths through h_{t-1} and the gates. A direct retention path is not a theorem that all gradients remain constant or that a trained model will learn the desired memory.

```
def lstm_step(x, state, parameters):
    """Modern forget-gate LSTM, no peepholes or projection; i,f,g,o."""
    h, c = state
    wi, wh, bi, bh = parameters
    i, f, g, o = (F.linear(x, wi, bi) + F.linear(h, wh, bh)).chunk(
        4, dim=-1
    )
    c_new = torch.sigmoid(f) * c + torch.sigmoid(i) * torch.tanh(g)
    h_new = torch.sigmoid(o) * torch.tanh(c_new)
    return h_new, c_new
```

The code packs four gate blocks in the order input, forget, candidate, output. It accepts two bias vectors because PyTorch stores input-side and hidden-side biases separately. Their sum plays the role of each b above. Changing packing order preserves tensor shapes while changing the model. The parity tests therefore compare actual values, not just dimensions.

6.4 Build a GRU cell and name its convention

A **gated recurrent unit** keeps a single state vector. In the PyTorch convention used here, reset and update gates are

$$r_t = \sigma(W_{xr} x_t + b_{xr} + W_{hr} h_{t-1} + b_{hr}), \tag{6.10}$$

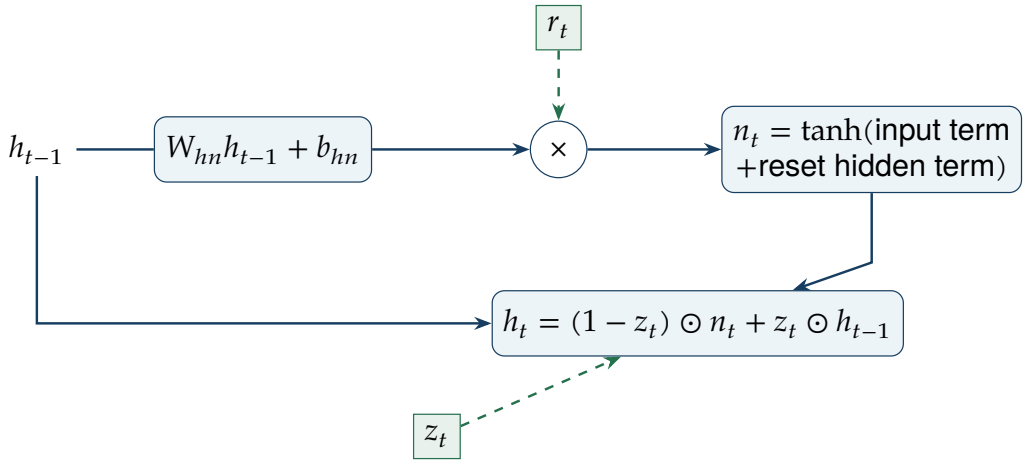
$$z_t = \sigma(W_{xz} x_t + b_{xz} + W_{hz} h_{t-1} + b_{hz}), \tag{6.11}$$

and the candidate and update are

$$n_t = \tanh\left(W_{xn} x_t + b_{xn} + r_t \odot (W_{hn} h_{t-1} + b_{hn})\right), \tag{6.12}$$

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}. \tag{6.13}$$

Thus a large z_t retains the old state in this convention. Some presentations attach the symbol z to the opposite mixing weight; read the equation rather than memorizing the gate name [26].



Reset-after: $r \odot Wh = (0.6, -0.8)$
Reset-before: $W(r \odot h) = (1.8, -1.4)$ for the example in the text.

Figure 6.3: The implemented GRU candidate and state merge. Reset acts after the hidden projection, including its bias. The update gate chooses coordinatewise retention versus candidate contribution. A separate numerical counterexample shows that resetting before a matrix multiplication generally produces different values.

For each coordinate, the final update is a convex combination of the candidate and previous state because a sigmoid lies between zero and one. That fact bounds the new coordinate between those two values; it does not bound every derivative of the state-dependent gates.

```
def gru_step(x, h, parameters):
    """PyTorch reset-after convention; gate packing r,z,n."""
    wi, wh, bi, bh = parameters
    ir, iz, inn = F.linear(x, wi, bi).chunk(3, dim=-1)
    hr, hz, hn = F.linear(h, wh, bh).chunk(3, dim=-1)
    r = torch.sigmoid(ir + hr)
    z = torch.sigmoid(iz + hz)
    n = torch.tanh(inn + r * hn)
    return (1 - z) * n + z * h
```

The implementation splits the input and hidden affine projections into three blocks ordered reset, update, candidate. The two candidate projections must remain separate until reset has been applied. Adding all affine outputs and then splitting would lose the information needed to apply the reset only to the hidden-side candidate contribution.

6.4.1 Reset-before and reset-after are not algebraic rearrangements

PyTorch documents a difference from the original GRU formulation: its reset is applied to the projected hidden state, rather than before that projection [27]. Generally,

$$r \odot (Wh + b) \neq W(r \odot h) + b. \tag{6.14}$$

For $h = (1, 2)$, $r = (0.2, 0.8)$, zero bias, and

$$W = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \tag{6.15}$$

reset-after gives $(0.6, -0.8)$ whereas reset-before gives $(1.8, -1.4)$. Both programs can execute and have identical tensor shapes. Only a convention-aware numerical test detects the mismatch.

This is why a checkpoint needs more than dimensions and vocabulary identity. It also needs an architecture definition: gate order, bias convention, activation functions, and where each gate acts. The name GRU is insufficient to guarantee parameter interchangeability across implementations.

6.5 Verify forward values and backward paths

Our reference artifact assigns deterministic weights. It compares our GRU step with the library's `GRUCell`. The test checks the output and gradients of a squared-output loss with respect to inputs, previous state, both weight matrices, and both biases. The tolerance is 10^{-12} in CPU float64; no bitwise cross-hardware guarantee is implied.

There are three complementary checks. A hand case sets all weights and biases to zero: both gates become 0.5, the candidate becomes zero, and the new state is half the old state. Library parity tests the full packed implementation. Numerical finite-difference gradient checking tests the derivatives independently of a second handwritten backward formula.

Forward parity alone is not enough. A misplaced detach can preserve every output while removing gradient paths. Conversely, a finite-difference check on one scalar output is not evidence that a data pipeline masks padding correctly. Tests should mirror distinct claims.

For $D = 4$ and $H = 8$, a cell with two bias vectors has $g(HD + H^2 + 2H)$ parameters, where g is the number of packed gate blocks. That gives 112 for a simple tanh cell, 336 for a GRU, and 448 for an LSTM. LSTM additionally carries both h and c at runtime. Parameter count is not a benchmark of speed, accuracy, or useful biological memory.

6.6 Measure a controlled retention path

Freeze a GRU update gate to scalar z and set its candidate to zero. Then $h_t = zh_{t-1}$ and both the state and its direct sensitivity decay as z^t when $h_0 = 1$. A half-life in **token steps** satisfies

$$z^\tau = \tfrac{1}{2}, \qquad \tau = \frac{\ln(1/2)}{\ln z}, \quad 0 < z < 1. \tag{6.16}$$

For $z = 0.5, 0.9, 0.99$, half-lives are approximately 1, 6.58, and 68.97 steps. At 100 steps, the retained factors are approximately 7.89×10^{-31}, 2.66×10^{-5}, and 0.366.

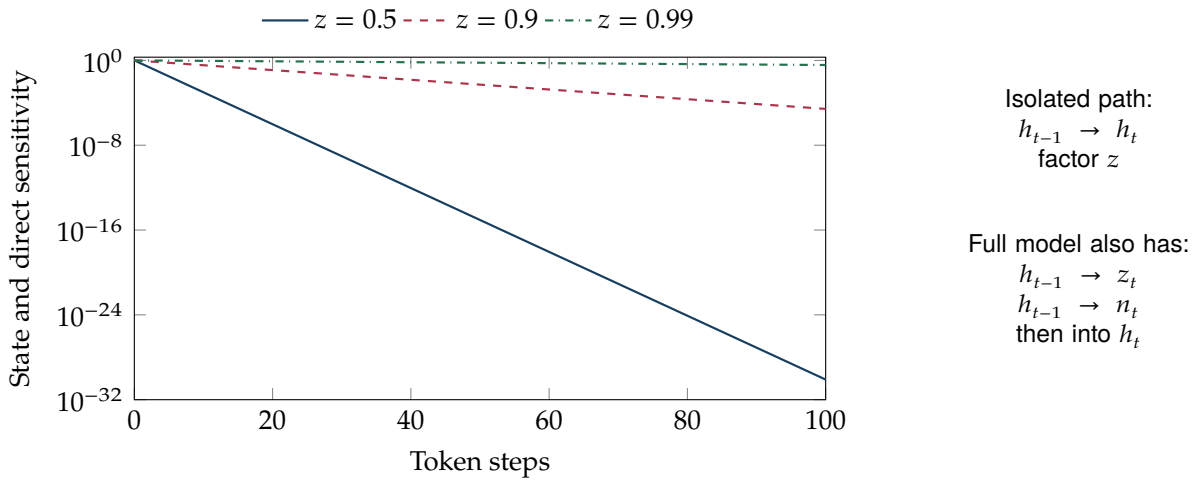


Figure 6.4: Computed retention powers for fixed gates and zero candidate, on a logarithmic vertical scale. This isolates one memory path, not a trained model's total Jacobian. The companion dependency sketch marks additional state-dependent gate paths that the simplified calculation holds fixed.

The time unit is part of the interpretation. A hundred base tokens span 100 bases; a hundred disjoint width-six tokens typically span 600 bases. An overlapping tokenizer advances differently again. A gate half-life cannot be reported as a genomic distance without the representation contract.

Nor is a slowly decaying state automatically useful. It can preserve an irrelevant feature, mix several signals destructively, or fail to expose stored information to the readout. These probes isolate retention mechanics; learning useful memory requires a task, data split, objective, and evaluation.

6.7 Batch sequences without advancing padded time

Our scan consumes x of shape $[B, T, D]$ and a valid-length vector. At a valid step, each row updates normally. Beyond its length, the row retains its previous state. This is a state-update mask, separate from the target-loss mask introduced earlier.

```
def scan_gru(x, h, parameters, lengths=None):
    """x is [batch,time,input]; padding holds state but is not a target mask."""
    if x.ndim != 3 or h.ndim != 2 or x.shape[0] != h.shape[0]:
        raise ValueError(
            "batch/time/input and batch/hidden tensors required"
        )
    batch, time, _ = x.shape
    if lengths is None:
        lengths = torch.full(
            (batch,), time, dtype=torch.long, device=x.device
        )
    if (
        lengths.shape != (batch,)
        or lengths.dtype != torch.long
        or torch.any(lengths < 0)
        or torch.any(lengths > time)
    ):
        raise ValueError("integer valid lengths required")
    states = []
```

```
    for t in range(time):
        candidate = gru_step(x[:, t], h, parameters)
        h = torch.where((t < lengths)[:, None], candidate, h)
        states.append(h)
    output = torch.stack(states, 1) if states else h[:, None, :][:, :0, :]
    return output, h
```

The output at padded positions repeats the last valid state; it is not automatically a zero vector. A downstream loss must still ignore invalid targets. A zero input is not a substitute for holding the state: recurrent weights and biases can change the state even when the input vector is zero.

The implementation computes candidates before selecting valid rows. Its padding tests use finite values. A mask does not generally sanitize NaNs introduced inside unselected computations, especially during backward propagation; validate inputs rather than relying on masking to repair them.

An empty time dimension returns an empty output sequence and the unchanged initial state. A zero-length row in a nonempty batch likewise remains at its initial state. Lengths must be integer counts between zero and the available number of steps.

6.8 Carry, detach, and reset mean different things

Consider the scalar recurrence

$$h_t = ah_{t-1} + x_t, \quad a = 0.5, \quad h_0 = 0, \quad x = (1, 2, 3, 4). \tag{6.17}$$

Its successive states are 1, 2.5, 4.25, and 6.125. Split the computation after step two. Carrying $h_2 = 2.5$ into the next chunk preserves the result. Replacing it with zero does not.

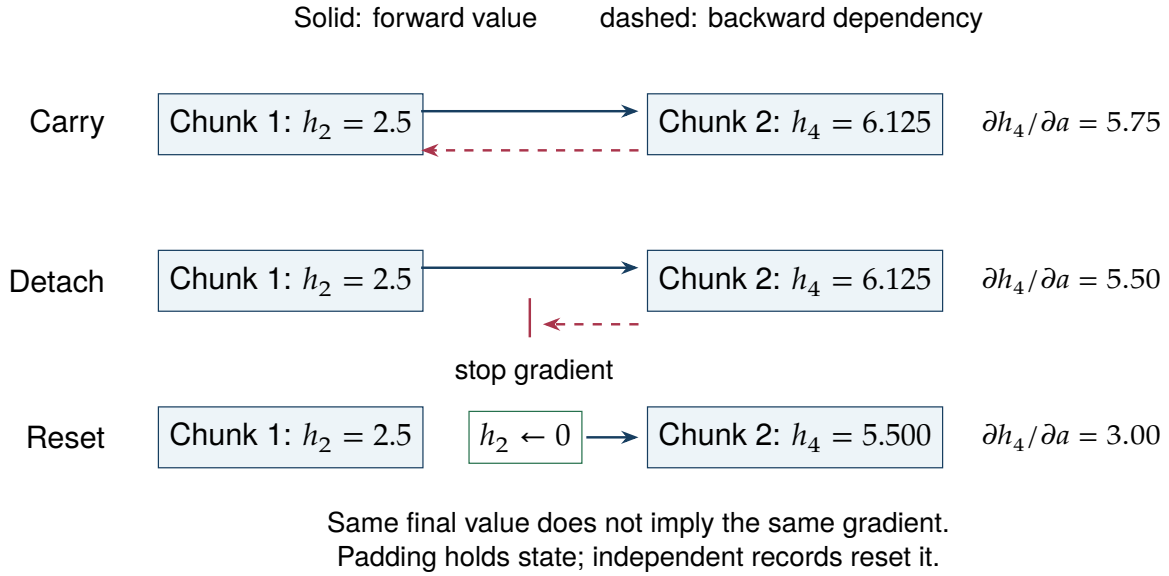


Figure 6.5: Three chunk-boundary policies. Carry keeps both numerical state and its gradient history. Detach carries the same value but cuts backward paths through the boundary. Reset replaces that value with zero and changes the forward computation. Record boundaries and truncated training boundaries therefore require different policies.

```python
def chunk_experiment(detach=False, reset=False):
    x = torch.tensor(
        [1.0, 2.0, 3.0, 4.0], dtype=torch.float64, requires_grad=True
    )
    a = torch.tensor(0.5, dtype=torch.float64, requires_grad=True)
    h = torch.zeros((), dtype=torch.float64)
    for t in range(4):
        if t == 2:
            h = (
                torch.zeros_like(h)
                if reset
                else h.detach()
                if detach
                else h
            )
        h = a * h + x[t]
    gx, ga = torch.autograd.grad(h, (x, a))
    return {
        "final": float(h.detach()),
        "input_gradient": gx.tolist(),
        "parameter_gradient": float(ga),
    }
```

With full history, the final state expands to $h_4 = a^3x_1 + a^2x_2 + ax_3 + x_4$. Hence

$$\frac{\partial h_4}{\partial x} = (a^3, a^2, a, 1) = (0.125, 0.25, 0.5, 1) \tag{6.18}$$

and

$$\frac{\partial h_4}{\partial a} = 3a^2x_1 + 2ax_2 + x_3 = 5.75. \tag{6.19}$$

If we detach h_2, the second chunk treats 2.5 as a constant: $h_4 = a(a \cdot 2.5 + 3) + 4$. Its derivative with respect to the shared a is $2a \cdot 2.5 + 3 = 5.5$, and the first two input gradients are zero. Resetting instead gives $h_4 = a \cdot 3 + 4 = 5.5$, with derivative 3.

Boundary policy	Final state	$\partial h_4/\partial a$	$\partial h_4/\partial x$
full	6.125	5.750	[0.125, 0.25, 0.5, 1]
detach	6.125	5.500	[0, 0, 0.5, 1]
reset	5.500	3.000	[0, 0, 0.5, 1]

This is **truncated backpropagation** in its most inspectable form: detachment limits credit assignment, not necessarily the numerical history carried forward. It is not equivalent to full-sequence training. Our demonstration keeps parameters fixed while scanning. If an optimizer updates weights between chunks, a carried state was computed under earlier weights; that is another distinction from a single full unroll.

For inference with fixed parameters, correctly carried GRU state produces the same outputs as an uninterrupted scan in our tests. Across independent records, reset instead. Across padded steps, hold. A single generic state-clearing rule cannot implement all three boundaries correctly.

6.9 What this chapter builds, and what comes next

Run the reference probes and tests:

```
python3 drafts/ch06/reference.py
python3 -m pytest tests/test_ch06_reference.py
```

The artifact covers manual GRU and modern LSTM cells, state masking, streaming equivalence, finite-difference checks, fixed-gate retention, and a detachment counterexample. It does not train or benchmark a genomic language model. DOGMA remains the non-Transformer DNA-native research direction, Hermon DNA the Transformer-based direction, and Evolutor the broader framework.

A sequential scan uses a fixed-size state, but nonlinear state dependence creates a dependency from one step to the next. Chapter 7 asks which state transitions can be reorganized into efficient scans and how selective state-space models change the design tradeoffs. The question is no longer whether there is memory, but how it is represented, updated, and computed.

Key ideas

- $\partial h_T / \partial h_k$ is a product of Jacobians. Contraction loses the distant past and expansion amplifies it.
- Gate names are conventions: PyTorch's GRU applies reset after the hidden projection, and the order changes the values.
- Detaching keeps the boundary value but cuts credit assignment; resetting changes the forward computation itself.
- A gate half-life in token steps is not a genomic distance until the tokenization fixes what a step means.

6.10 Exercises with worked solutions

1. Align the target

After consuming A, C, G, which base does the final readout predict? **Solution:** the next base after G, not G itself. The state has already received the G embedding.

2. Derive a scalar gradient

For $h_t = 0.8h_{t-1}$, find $\partial h_{10} / \partial h_0$. **Solution:** $0.8^{10} \approx 0.1074$. This is exact for the fixed linear recurrence, not for a learned nonlinear gate.

3. Trace an LSTM coordinate

Use $c = 2, f = 0.75, i = 0.2, g = -0.5, o = 0.6$. **Solution:** the new cell is $1.5 - 0.1 = 1.4$ and the hidden output is about 0.531. The output gate does not erase the underlying cell by merely hiding part of its readout.

4. Interpret the update gate

In Eq. 6.13, what happens as z approaches one? **Solution:** the previous state is retained. Approaching zero instead chooses the candidate. State the convention when comparing equations.

5. Find a shape-preserving bug

Swap the reset and update blocks in packed weights. **Solution:** tensor dimensions still match, but the equations change. Forward and backward parity with a nontrivial assigned-weight case should fail.

6. Test reset order

Evaluate the chapter's W, r, and h counterexample. **Solution:** $Wh = (3, -1)$ gives reset-after $(0.6, -0.8)$; $r \odot h = (0.2, 1.6)$ gives reset-before $(1.8, -1.4)$.

7. Count parameters

Find the GRU parameter count for $D = 4, H = 8$ with both biases. **Solution:** $3(32 + 64 + 16) = 336$. Excluding one bias vector would change the count and potentially the checkpoint convention.

8. Convert a memory horizon

What base distance corresponds to 50 disjoint full-width six-mer steps? **Solution:** 300 bases. Do not reuse that conversion for stride-one overlapping windows, which advance one base per step.

9. Explain padding drift

Why can feeding zero embeddings through padded steps change final state? **Solution:** recurrent projections and biases remain active. Hold the row's state explicitly and mask invalid targets separately.

10. Separate value and gradient

Why do full carry and detachment both yield 6.125 but different gradients? **Solution:** detachment preserves the boundary's numerical value while treating it as constant for backward propagation. It removes paths from the final loss through the first chunk.

11. Prevent cross-record leakage

A loader places two unrelated sequences consecutively in the same batch row. **Solution:** reset state at the record boundary. Detaching alone retains the earlier record's numerical information.

12. Design a memory evaluation

How would you test learned long-range genomic dependencies? **Worked rubric:** define the biological target, split related records appropriately, vary dependency distance, compare local-window baselines, audit strand and position conventions, and report uncertainty across runs. Separate synthetic retention probes from learned-task accuracy and from biological generalization.

Model scope

All API sources were inspected on 19 September 2026. Synthetic retention curves and assigned cell weights do not establish trained-model performance or biological generalization.

CHAPTER 7

State-space models, selective updates and scans

In this chapter you will

- derive zero-order-hold discretization and contrast it with an unstable Euler step
- expand a time-invariant recurrence into a causal convolution
- make updates input-selective while keeping them affine in the previous state
- compose interval summaries with an associative scan and check it against the sequential recurrence

A recurrence appears to demand one update after another. Before computing h_4, surely we must compute h_3, then h_2, then h_1? That is true of one way to execute the recurrence. It is not always true of the underlying computation. Sometimes a whole interval can be represented by a small summary that composes with the summary of another interval.

This chapter builds that summary from first principles. We begin with a scalar state, derive a causal convolution, introduce input-dependent updates, and implement a tree-based prefix computation. We then test forward values, gradients, padding, and chunk boundaries. The result is a reference implementation of diagonal affine scans, not a reproduction of Mamba, S4, S5, or RWKV, and not a trained genomic model.

We assume Chapter 6's distinction between a numerical state and its gradient history, and Chapter 5's sequence representation contract. The new mathematics is exponential decay and associative composition. The important conceptual distinction is between *linear in the previous state* and *linear in the entire input sequence*.

7.1 Start with a state whose arithmetic is visible

Consider the scalar update

$$h_t = a_t h_{t-1} + b_t. \tag{7.1}$$

The pair (a_t, b_t) describes a function: multiply the incoming state by a_t, then add b_t. The additive term may already include an input projection. For several independent state coordinates, multiplication is elementwise and $a_t, b_t, h_t \in \mathbb{R}^N$.

Here is a four-step example with $h_0 = 2$. Predict the final state before reading the last column. The second step is an identity update; it should not erase or advance the stored value.

t	a_t	b_t	P_t	Q_t	h_t
1	0.5	1	0.5	1	2
2	1	0	0.5	1	2
3	0.25	2	0.125	2.25	2.5
4	0.8	-1	0.1	0.8	1

The states are $2, 2, 2.5, 1$. Columns P_t, Q_t summarize the entire prefix: $h_t = P_t h_0 + Q_t$. We will derive those columns rather than accepting them as another execution trace.

```
def sequential(a, b, h):
    check_shapes(a, b, h)
    outputs = []
    for at, bt in zip(a, b):
        h = at * h + bt
        outputs.append(h)
    return torch.stack(outputs) if outputs else b + h[None, :]
```

The reference uses shape $[T, N]$ for one sequence, with initial state $[N]$. A batch adds an independent leading batch dimension; no sequence should share another row's state. The implementation validates shapes, finite inputs, dtype, and device. An empty sequence returns an empty state sequence. Its incoming state remains the caller's carry value.

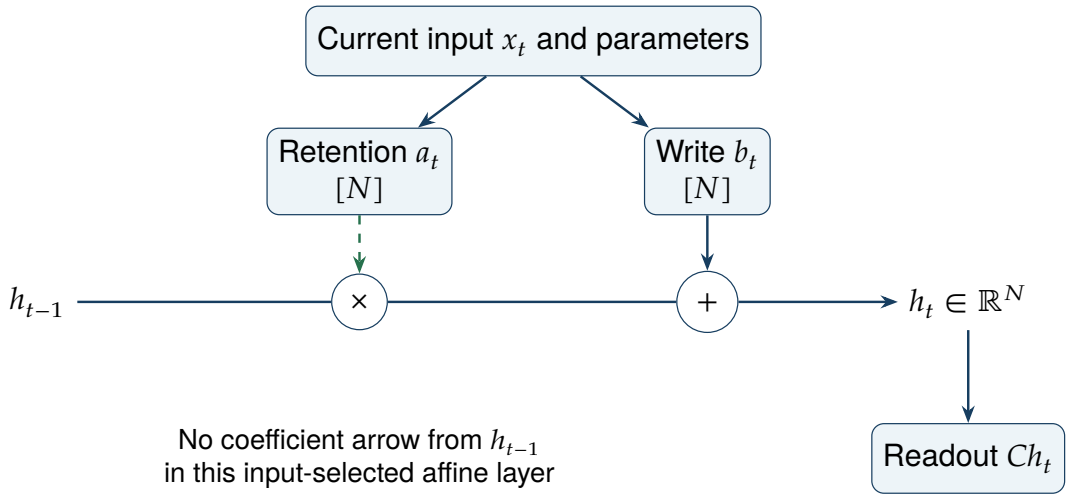


Figure 7.1: One diagonal affine state update. Retention and input-derived write values meet at an addition; a separate readout maps state to output. Shapes and dependencies are explicit. These are computational states, not biological concentrations or genes.

A full sequence model needs more than this recurrence: input representation, mixing across channels, nonlinear transformations, readout, loss, and a training procedure. We deliberately isolate one operation so its numerical meaning can be checked before adding those layers.

7.2 Derive a stable discrete update from continuous time

For one coordinate, let an internal continuous state satisfy

$$\frac{dh}{ds} = -\lambda h + u, \qquad \lambda > 0. \tag{7.2}$$

Hold u constant over an interval of length Δ. Multiplication by the integrating factor $e^{\lambda s}$ gives $\frac{d}{ds}(e^{\lambda s}h) = e^{\lambda s}u$. Integrating over that interval yields

$$h_{\text{new}} = e^{-\lambda\Delta}h_{\text{old}} + \frac{1 - e^{-\lambda\Delta}}{\lambda}u. \tag{7.3}$$

This is **zero-order hold** discretization: constant input on the interval, exact integration of this linear system on that interval. It is not the same as a forward Euler step, $h_{\text{new}} = (1 - \lambda\Delta)h_{\text{old}} + \Delta u$.

For $\lambda\Delta = 3$, Euler's homogeneous multiplier is -2: successive steps alternate sign and grow. The exact multiplier is $e^{-3} \approx 0.0498$, a contraction. The continuous system was stable; the wrong step approximation introduced the instability.

```
def zoh(decay, delta):
    """dh/ds=-decay*h+u, piecewise constant u; decay>0, delta>=0."""
    if not isinstance(decay, torch.Tensor) or not isinstance(
        delta, torch.Tensor
    ):
        raise ValueError("tensors required")
    if (
        not torch.isfinite(decay).all()
        or not torch.isfinite(delta).all()
        or torch.any(decay <= 0)
        or torch.any(delta < 0)
    ):
        raise ValueError(
            "finite positive decay and nonnegative interval required"
        )
    z = -decay * delta
    return torch.exp(z), -torch.expm1(z) / decay
```

The function uses `expm1` to compute the small difference $e^z - 1$ accurately. For very small positive Δ, the write gain approaches Δ. At $\Delta = 0$, the pair is exactly $(1, 0)$: no internal time passes and no input is written. We require $\lambda > 0$; the $\lambda = 0$ limiting case is an integrator and needs an explicit branch rather than division by zero.

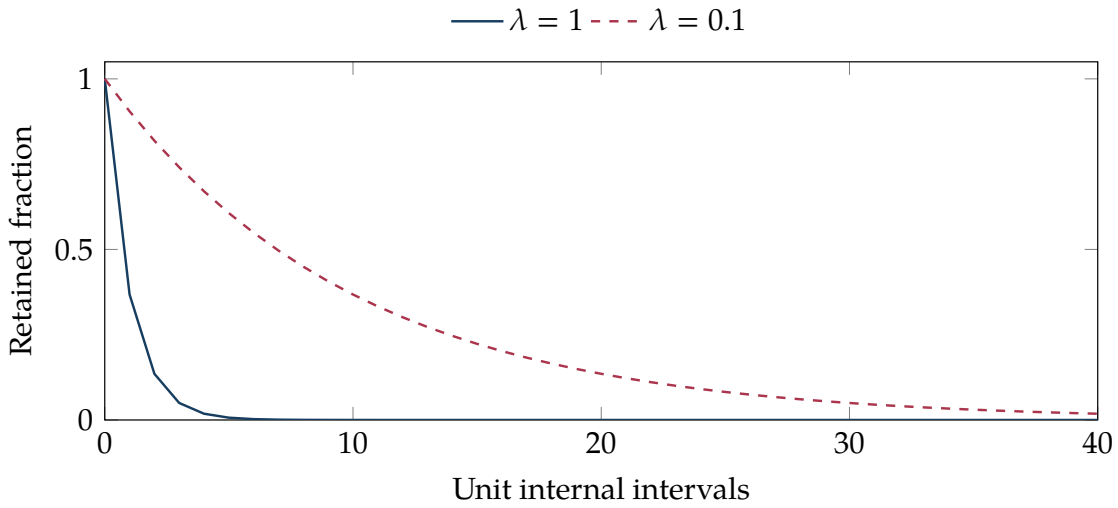


Figure 7.2: Computed homogeneous retention for two positive decay rates at unit intervals. Each curve starts from the same state and has zero drive. Different coordinates can represent different timescales; long retention alone does not establish useful memory or biological understanding.

For a learned sequence model, s and Δ can be internal computational coordinates. They need not represent physical seconds or genomic base pairs. Changing tokenization changes what one sequence step means. A mathematical decay constant should not be described as a measured biochemical lifetime.

In a vector system the familiar notation is $\dot{h} = Ah + Bx$, $y = Ch$, with a discretized update $h_t = \bar{A}h_{t-1} + \bar{B}x_t$. Our derivation covers a diagonal negative A and a scalar drive per

coordinate. General matrix exponentials and structured parameterizations require additional numerical machinery. The S4 paper introduces a structured sequence-model construction, not merely a new name for any exponentially decaying scalar [29].

7.3 Expand a fixed recurrence into a convolution

Freeze the coefficients: $a_t = a$, $b_t = gx_t$, and let $y_t = ch_t$. For zero initial state,

$$h_1 = gx_1, \tag{7.4}$$
$$h_2 = agx_1 + gx_2, \tag{7.5}$$
$$h_3 = a^2gx_1 + agx_2 + gx_3. \tag{7.6}$$

In general,

$$y_t = \sum_{j=1}^{t} cga^{t-j}x_j, \qquad K_\ell = cga^\ell. \tag{7.7}$$

The coefficient depends on lag $t - j$, not the absolute positions. This is a causal **linear time-invariant** convolution. It gives another way to compute the same transformation when the entire input is available.

```
def convolution_form(x, a, gain, readout):
    """Scalar LTI zero-state causal convolution, implemented directly."""
    return (
        torch.stack(
            [
                sum(
                    readout * gain * a ** (t - j) * x[j]
                    for j in range(t + 1)
                )
                for t in range(len(x))
            ]
        )
        if len(x)
        else x.clone()
    )
```

This direct implementation intentionally costs $O(T^2)$; it is an independent, readable oracle rather than a fast convolution kernel. For $a = 0.5$, $g = c = 1$, and $x = (1, 2, -1, 3)$, both execution forms yield $(1, 2.5, 0.25, 3.125)$. With a nonzero h_0, Eq. 7.7 also needs the term $ca^t h_0$. Omitting it is a state-contract error.

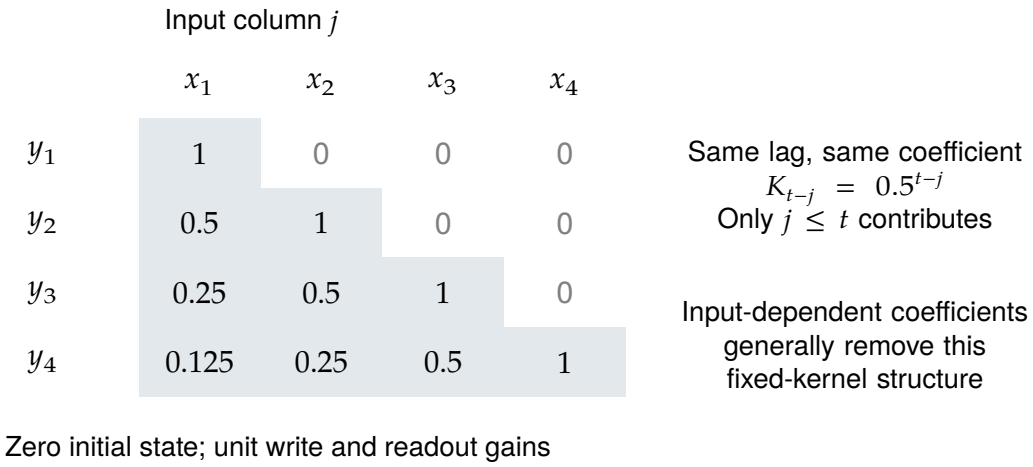


Figure 7.3: A causal kernel for $a = 0.5$ and unit input/readout gain. The lower-triangular coefficients depend only on lag. The diagram shows why future inputs are excluded and why fixed coefficients allow a shared convolution kernel.

A diagonal N-coordinate state produces a sum of exponential kernels, $K_\ell = \sum_n c_n g_n a_n^\ell$. Those coordinates may encode different decay rates or, in other parameterizations, oscillations. Their sum can represent richer filters than any individual coordinate. Efficient kernel computation, parameter initialization, and numerical conditioning are separate questions from this algebraic equivalence.

7.4 Make updates selective without making them state dependent

Suppose Δ_t is computed from the current input:

$$\Delta_t = \mathrm{softplus}(W_\Delta x_t + b_\Delta), \quad a_t = e^{-\lambda \Delta_t}, \quad b_t = \frac{1 - a_t}{\lambda} u_t. \tag{7.8}$$

The update remains affine in h_{t-1}, but its coefficients change with the input. Small Δ_t retains state and weakens the write. Large Δ_t forgets the old state and moves toward u_t/λ. It does not generally reset the state to zero.

```
def selected_coefficients(x, weights, bias, decay):
    """x [T,D], weights [D,N], bias/decay [N]; current-input selection only."""
    delta = torch.nn.functional.softplus(x @ weights + bias)
    a, gain = zoh(decay, delta)
    # Original teaching input projection, not an architecture reproduction.
    drive = x @ weights
    return a, gain * drive
```

Our small example reuses a projection for the drive and interval selection to keep the calculation inspectable. That choice is a teaching simplification, not the parameterization of a published architecture. Each coefficient depends on x_t and fixed parameters, not on the unknown previous recurrent state. All coefficients can therefore be formed before scanning this layer. They can be nonlinear functions of the input without breaking that property.

Why does a fixed convolution no longer suffice? Expanding the variable recurrence gives

$$h_t = \left(\prod_{k=1}^{t} a_k \right) h_0 + \sum_{j=1}^{t} \left(\prod_{k=j+1}^{t} a_k \right) b_j. \tag{7.9}$$

The contribution of input j depends on intervening inputs through the a_k. There is generally no single lag-only kernel shared across sequences. Even the one-step map $y(x) = (1 - e^{-\operatorname{softplus}(x)})x$ fails $y(2x) = 2y(x)$, so it is not a linear operator on the input.

Mamba's selective SSM makes certain state-space parameters input dependent and uses scan-based execution; its architecture and hardware-aware implementation add further mechanisms [10]. We use the principle to motivate an original toy, not to claim equivalent accuracy or speed.

Contrast this with Chapter 6's GRU: its gates and candidate depend nonlinearly on h_{t-1}. The GRU's next affine-looking pair cannot generally be precomputed independently of earlier states. Function composition is always associative abstractly, but composing arbitrary nonlinear functions need not yield a small, cheaply evaluable summary. The useful property here is closure in a fixed-size family of affine maps.

7.5 Compose intervals, preserving chronological order

Let an earlier interval map h to $ah + b$, and a later interval map its input to $ch + d$. The composition is

$$c(ah + b) + d = (ca)h + (cb + d). \tag{7.10}$$

Define the chronological binary operation

$$(a, b) \star (c, d) = (ca, cb + d). \tag{7.11}$$

The first argument occurs earlier in time. This convention is important: some software APIs reverse the argument order while expressing the same functional composition.

```
def compose(earlier, later):
    """Chronological composition: later(earlier(h))."""
    a, b = earlier
    c, d = later
    return c * a, c * b + d
```

For three pairs $(a, b), (c, d), (e, f)$, either parenthesization gives

$$(eca,\ ecb + ed + f). \tag{7.12}$$

Thus $\star$ is associative over exact real arithmetic. Its identity is $(1, 0)$. It is not commutative: $(0.5, 1) \star (0.25, 2)$ gives $(0.125, 2.25)$, while reversing them gives $(0.125, 2)$. Changing the grouping is legal; changing the order changes the sequence.

For vectors with diagonal transitions, the products are elementwise. For dense matrices the same argument uses ordered matrix multiplication. Associativity survives, but composing dense transitions costs much more than composing diagonal ones. A proof of parallelizability does not remove the cost of the operator used inside the scan.

7.6 Compute every prefix with a tree

An inclusive **prefix scan** returns each prefix composition: p_1, $p_1 \star p_2$, and so on. Blelloch develops scans and reductions of recurrences to associative operations [28]. Our implementation first composes adjacent pairs, recursively scans those pair summaries, then fills the intermediate prefixes.

For eight inputs, the recursive sizes are $8, 4, 2, 1$. At the size-eight level we form four pair summaries. The recursive result gives prefixes ending at positions 2, 4, 6, and 8. Position 1 is already known; positions 3, 5, and 7 each require one more composition. Nothing is reordered, and no future coefficient participates in an earlier prefix.

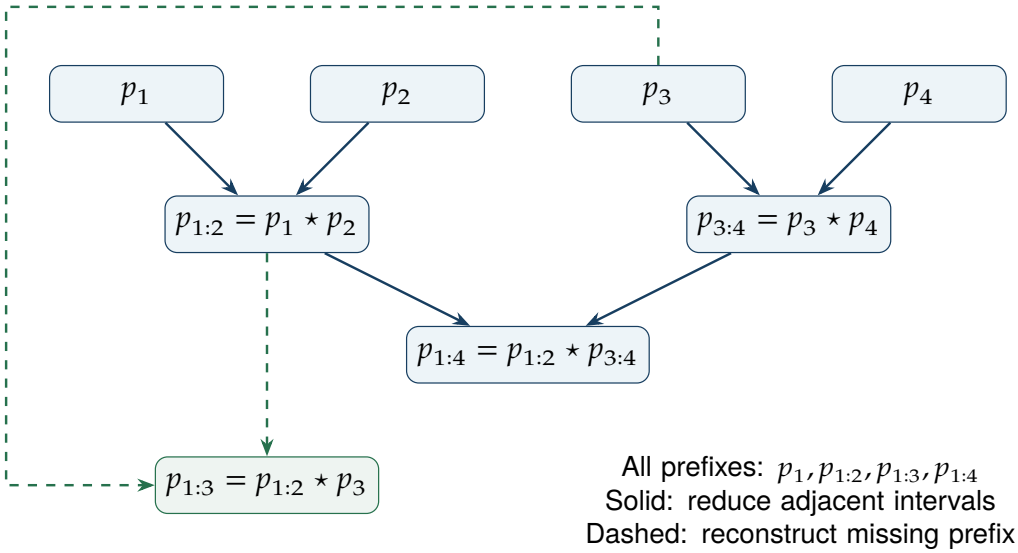


Figure 7.4: A four-element inclusive scan. The reduction combines adjacent intervals; reconstruction supplies the missing prefix ending at element three. Labels identify the interval represented by each summary. The arithmetic is associative, but chronological order remains fixed.

```
def prefix_pairs(a, b):
    """Work-efficient inclusive prefix DAG; Python executes its nodes serially."""
    size = a.shape[0]
    if size <= 1:
        return a, b
    paired = size // 2
    ra, rb = compose(
        (a[: 2 * paired : 2], b[: 2 * paired : 2]),
        (a[1 : 2 * paired : 2], b[1 : 2 * paired : 2]),
    )
    pa, pb = prefix_pairs(ra, rb)
    out_a, out_b = [a[0]], [b[0]]
    for i in range(1, size):
        if i % 2:
            ca, cb = pa[i // 2], pb[i // 2]
        else:
            ca, cb = compose((pa[i // 2 - 1], pb[i // 2 - 1]), (a[i], b[i]))
        out_a.append(ca)
        out_b.append(cb)
    return torch.stack(out_a), torch.stack(out_b)
```

```
def parallel_form(a, b, h):
    check_shapes(a, b, h)
    pa, pb = prefix_pairs(a, b)
    return pa * h[None, :] + pb
```

If prefix t is (P_t, Q_t), applying it to the initial state produces $P_t \odot h_0 + Q_t$. Keeping h_0 outside the prefix recursion makes nonzero initial-state handling explicit and supports chunk summaries. It also exposes a common bug: returning only Q_t accidentally assumes zero initial state.

For $T \geq 2$, our number of pair compositions satisfies

$$W(T) = \lfloor T/2 \rfloor + \lfloor (T-1)/2 \rfloor + W(\lfloor T/2 \rfloor), \quad W(1) = 0. \tag{7.13}$$

Hence $W(T) < 2T$. For $T = 8$, the count is 11, not the seven compositions needed merely to reduce to one final summary. A scan returns all prefixes, so it solves a larger output problem.

The dependency depth is $O(\log T)$ when independent nodes at each level are executed concurrently. *This Python reference does not do that.* Its loops execute on the host; the tensor operations expose the dependency structure but are not a fused accelerator kernel. The algorithm has linear arithmetic work for fixed state width, but these measurements do not establish wall-clock speedups.

7.7 Check gradients and finite-precision equivalence

Changing parenthesization can change floating-point rounding. Associativity is a real-arithmetic identity, not a promise of bitwise equality between execution orders. We compare CPU float64 outputs with tolerances, including empty, odd-length, and non-power-of-two sequences. Every prefix is checked, not only the last one.

The tests compare gradients of a squared-output loss with respect to all a_t, b_t, and h_0. They also apply finite-difference gradient checking to the scan and to the input-dependent coefficient path. This catches a hidden detachment that could preserve the forward result while changing the learned model.

For fixed coefficients, the direct state sensitivity is

$$\frac{\partial h_t}{\partial h_0} = \prod_{k=1}^{t} a_k. \tag{7.14}$$

When coefficients depend on inputs, input gradients additionally traverse the functions producing a_k and b_k. Treating those coefficients as constants during differentiation would train a different computation. Our out-of-place construction preserves those paths.

Positive decay and nonnegative intervals keep $0 < a_t \leq 1$ in exact arithmetic. This bounds the homogeneous retention path but does not bound every output or gradient: the drive, readout, projections, and their derivatives also matter. Extremely long products can underflow. Testing modest float64 examples establishes a reference, not universal mixed-precision stability.

7.8 Make padding, reset, and carry algebraic

For finite inputs, padding that must not advance state is the identity pair $(1, 0)$. It preserves the incoming state exactly at the mathematical level. Feeding a zero token through learned projections need not produce that pair. Loss masking remains separate: padded outputs must not become valid targets merely because their state was held.

A record boundary that resets the incoming state to zero *before* the current input's update replaces (a_t, b_t) by $(0, b_t)$. Then $h_t = b_t$, with no contribution from the preceding record. Resetting after the update would instead discard the current input. At a chunk boundary within one record, carry the final state; do not insert either padding or reset.

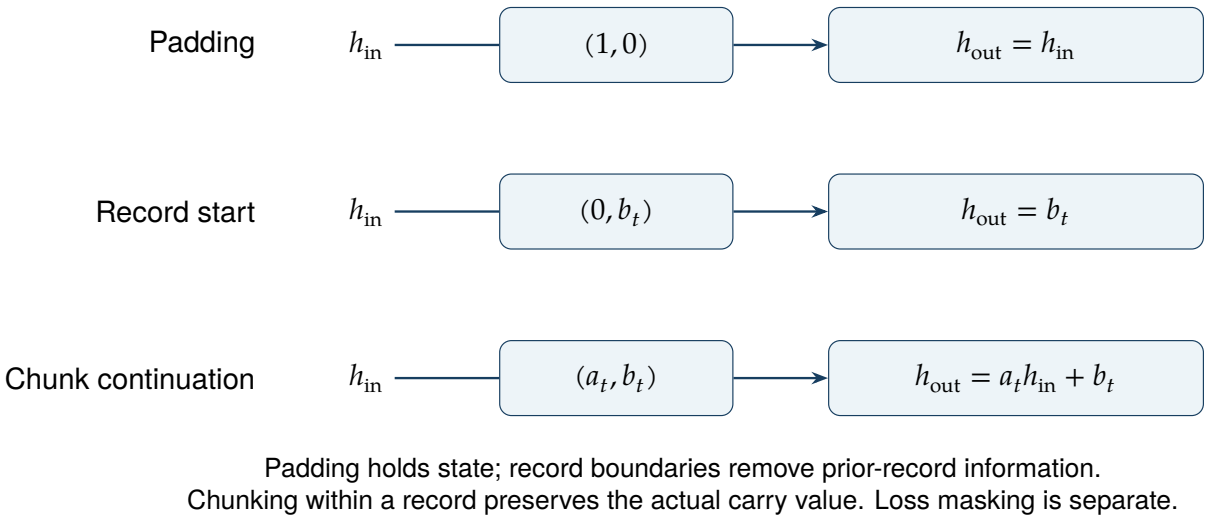


Figure 7.5: Three different boundary contracts expressed as affine maps. Padding is identity, record reset removes the incoming-state term, and chunk carry preserves the actual boundary state. These operations can have identical tensor shapes while representing different computations.

Full and chunked execution match when coefficients, token positions, model parameters, and initial-state policy match. A coefficient network with a local causal convolution would also need its own boundary buffer; carrying only the SSM state would then be insufficient. A change in parameters between chunks creates the same stale-state issue discussed in Chapter 6. These details become explicit runtime contracts later.

7.9 Place the mechanism in the broader architecture

S4, S5, Mamba, and RWKV should not be collapsed into one model merely because they use recurrence. S4 develops structured state-space sequence models. S5 uses a multi-input, multi-output state-space layer and parallel scan. The original Mamba combines selective state spaces with an architecture and a hardware-aware execution strategy. The original RWKV paper presents a recurrent language-model architecture with its own time-mixing and channel-mixing construction [29, 30, 10, 31]. These are high-level distinctions; this chapter does not reproduce their benchmark results or implement their complete blocks.

For Evolutor, the reusable lesson is a legality test: can an interval's effect be represented in a fixed-size family whose composition is associative and affordable? If yes, a compiler may have sequential, chunked, and scan execution choices. If not, a label such as state model does not make those transformations legal.

DOGMA remains the non-Transformer DNA-native research direction; Hermon DNA remains the Transformer-based direction. Neither identity is established by giving a generic diagonal recurrence a biological name. Claims about genomic usefulness require tasks, appropriate data splits, baselines, and mechanism-specific interventions.

Run the reference and tests:

```
python3 drafts/ch07/reference.py
python3 -m pytest -q tests/test_ch07_reference.py
```

Chapter 8 changes the question from how to compress history into a fixed-dimensional state to how to address stored content through attention. Chapter 26 will revisit scan parity at model scale, with training, precision, and runtime boundaries included.

Key ideas

- Affine maps $h \mapsto ah + b$ are closed under composition, and composition is associative. That closure is what makes a parallel scan legal.
- Associativity permits regrouping, never reordering: $(0.5, 1) \star (0.25, 2) \neq (0.25, 2) \star (0.5, 1)$.
- Padding is the identity pair $(1, 0)$; a reset before the current write is $(0, b_t)$; a chunk boundary carries the actual state.
- Logarithmic dependency depth describes a possible schedule. It is not a measured speedup.

7.10 Exercises with worked solutions

1. Trace an identity

Apply $(0.5, 1)$ and then $(1, 0)$ to $h_0 = 2$. **Solution:** the first state is 2; the identity leaves it at 2. An identity step is not a reset to zero.

2. Reverse two updates

Compare $(0.5, 1) \star (0.25, 2)$ with the reversed order. **Solution:** the multipliers both equal 0.125, but the writes are 2.25 and 2. Associativity permits regrouping, not permutation.

3. Audit Euler stability

For $\lambda = 2$ and $\Delta = 1$, compare Euler and exact retention. **Solution:** Euler gives -1, while the exact multiplier is $e^{-2} \approx 0.1353$. Euler does not decay in magnitude in this case.

4. Recover a limiting gain

Show that $(1 - e^{-\lambda\Delta})/\lambda \to \Delta$ as $\lambda\Delta \to 0$ with fixed positive λ. **Solution:** expand the exponential: $1 - e^{-\lambda\Delta} = \lambda\Delta + O(\Delta^2)$. The implementation uses `expm1` to preserve this small difference.

5. Restore a nonzero initial state

Which term is missing from a zero-state convolution when $h_0 \neq 0$? **Solution:** add $ca^t h_0$ to output y_t for fixed coefficients. The prefix form handles this as $P_t h_0 + Q_t$ before readout.

6. Find the hidden nonlinearity

Why can a model be affine in state but nonlinear in inputs? **Solution:** the coefficient functions may depend nonlinearly on the input. Holding those coefficients fixed is a conditional linear calculation, not a claim that the entire input-output map is linear.

7. Count a scan

How many compositions does our size-eight scan perform? **Solution:**
$(4+3)+(2+1)+(1+0)=11$. A final-only reduction needs seven, but does not return every prefix.

8. Check an odd length

At length five, where does the final prefix come from? **Solution:** scan the two complete adjacent pairs, then compose the prefix covering positions 1–4 with the original fifth pair. No fake padded input is needed.

9. Detect a gradient bug

The scan matches sequential outputs but its coefficient network has zero gradients. What should be tested? **Solution:** compare parameter and input gradients and check for detachment or conversion to Python numbers between coefficient creation and scanning. Forward parity alone cannot expose all such errors.

10. Encode a reset

Which pair resets to zero before applying the current write b_t? **Solution:** $(0, b_t)$. The new state is b_t. Using $(0, 0)$ would discard the current write too.

11. Challenge a speed claim

Does logarithmic dependency depth prove this Python code is faster? **Solution:** no. It specifies a parallel schedule's potential depth, not available processors, kernel overhead, memory traffic, or measured latency. Report both theoretical work and actual execution.

12. Design a fair memory comparison

How would you compare a selective recurrence with attention on DNA? **Worked rubric:** align tokenization, targets, splits, training budgets, and evaluation distances. Test state reset and causality; separate synthetic retention from learned accuracy. Measure inference memory and latency under declared batch and sequence lengths. Include negative controls for selection and readout rather than attributing every improvement to recurrence.

CHAPTER 8

Attention and content addressing

In this chapter you will

- compute attention by hand, separating addressing (queries and keys) from content (values)
- build packed causal masks that also respect record boundaries and padding
- derive the attention backward pass, including $\partial o/\partial z_j = p_j(v_j - o)$
- append to a key/value cache with offset masks and verify chunked–full parity

Chapter 7 compressed a sequence into a recurrent state and used an associative scan to compute a particular recurrence. This chapter asks a different question: what if each position could retrieve a mixture of information from explicitly retained earlier positions? The resulting operation is **attention**. Its central mechanism is content-dependent addressing, not another name for a recurrent update.

We will build the operation from one query, two keys, and two values; derive its gradients; establish which information is allowed to flow; and implement a cache whose chunked results agree with full-sequence computation. The reference uses small CPU float64 tensors so that arithmetic and failure cases are inspectable. No training run or biological performance claim is hidden inside the examples.

Within this book's taxonomy, DOGMA denotes the non-Transformer branch and Hermon the Transformer branch. Attention is a component needed to understand the latter, not evidence that it is always preferable to a state-based architecture. A fair comparison requires a task, data split, resource budget, and evaluation protocol.

8.1 Separate the address from the retrieved content

Suppose a query is the scalar $q = \ln 2$, the two keys are $k_1 = 1, k_2 = 0$, and their values are

$$v_1 = (3, 0), \qquad v_2 = (0, 6). \tag{8.1}$$

The dot-product scores are $(\ln 2, 0)$. Exponentiating and normalizing gives weights $(2/3, 1/3)$, so the retrieved vector is

$$o = \tfrac{2}{3}(3, 0) + \tfrac{1}{3}(0, 6) = (2, 2). \tag{8.2}$$

Before reading the implementation, predict what changes if the second value becomes $(0, 12)$. The weights stay the same, because keys determine compatibility with the query; the output becomes $(2, 4)$, because values determine the retrieved content.

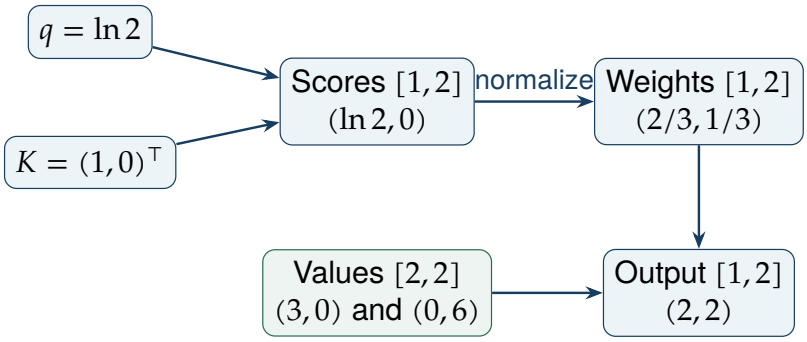


Figure 8.1: A complete two-key retrieval. Query–key compatibility determines normalized weights; the values supply content. The output is a weighted combination, not the selected key itself. Shapes and numbers make the addressing and retrieval stages independently checkable.

In a learned model the query, key, and value commonly come from different projections of token representations. They need not encode literal questions, database identifiers, or human-interpretable facts. Those names describe their mathematical roles.

8.1.1 Write the shapes before the equation

For L query positions, S key positions, key width D, and value width D_v, let

$$Q \in \mathbb{R}^{L\times D}, \quad K \in \mathbb{R}^{S\times D}, \quad V \in \mathbb{R}^{S\times D_v}. \tag{8.3}$$

The score matrix has one row per query and one column per key:

$$Z = \frac{QK^\top}{\sqrt{D}}, \quad P = \operatorname{softmax}_{\text{row}}(Z), \quad O = PV. \tag{8.4}$$

Consequently P has shape $[L, S]$ and O has shape $[L, D_v]$. Query length need not equal key length. This becomes essential when a new chunk attends to an existing history.

Scaled dot-product attention and multiple projected heads follow the original Transformer formulation [6]. Here we isolate the operation before surrounding it with residual connections, normalization, positional representations, and a feed-forward network.

8.1.2 Why scale, and why subtract the maximum?

Under a simplified initialization model, let the D products $q_d k_d$ be independent, zero-mean, and have variance one. Then

$$\operatorname{Var}\left(\sum_{d=1}^{D} q_d k_d\right) = D. \tag{8.5}$$

Dividing by $\sqrt{D}$ restores variance one. This motivates the scale: increasing feature width should not automatically drive logits toward more saturated softmax distributions. It is not a theorem that learned queries and keys stay independent or unit variance.

For any finite scalar a and nonempty allowed set A,

$$p_j = \frac{e^{z_j - a}}{\sum_{r\in A} e^{z_r - a}}, \quad j \in A. \tag{8.6}$$

The common factor cancels. Taking $a = \max_{r \in A} z_r$ keeps the largest exponent at zero, avoiding unnecessary overflow. The normalized weights are nonnegative and sum to one; each raw output row lies in the convex hull of its allowed value vectors. A later learned output projection or residual addition need not preserve that geometric restriction.

8.2 A mask is an information-flow contract

In **causal attention**, query position i may use key positions $j \le i$. The current token is visible because its representation predicts a later target in an autoregressive objective. If the target alignment instead asks it to predict the same visible token, a triangular mask does not repair the resulting label leakage.

For packed records we also require matching record identity and valid, nonpadding positions. Let r_i be a unique identifier for the record containing position i, and let u_i indicate validity. Define

$$A_{ij} = (j \le i) \wedge (r_i = r_j) \wedge u_i \wedge u_j. \tag{8.7}$$

Identifiers must be unique per record, not merely class labels reused by unrelated sequences. Otherwise equal labels would incorrectly open a path between records.

```
def packed_mask(records, valid):
    """One sequence; IDs are unique per record; True means an allowed edge."""
    if records.ndim != 1 or valid.shape != records.shape:
        raise ValueError(
            "record and validity vectors must have matching shape"
        )
    if records.dtype != torch.long or valid.dtype != torch.bool:
        raise ValueError("int64 record IDs and boolean validity required")
    if records.device != valid.device:
        raise ValueError("matching devices required")
    pos = torch.arange(len(records), device=records.device)
    return (
        (pos[None, :] <= pos[:, None])
        & (records[None, :] == records[:, None])
        & valid[None, :]
        & valid[:, None]
    )
```

	A0	A1	B0	B1	pad
A0	●	×	×	×	×
A1	●	●	×	×	×
B0	×	×	●	×	×
B1	×	×	●	●	×
pad	×	×	×	×	×

Row: query. Column: key.
●: allowed, ×: blocked.
A and B are distinct records.
No edge crosses the record boundary.
The padding query has no allowed key.

$$M_{ij} = [j \le i]\,[r_i = r_j]\,[\text{valid}_i]\,[\text{valid}_j]$$

Figure 8.2: A packed causal mask for records A and B followed by padding. A filled dot permits an edge; a cross blocks it. Causality, record separation, and query/key validity are separate requirements. The padding query has no permitted keys and receives an explicitly defined zero attention output.

The example uses IDs $(0,0,1,1,2)$ and validity $(1,1,1,1,0)$. The second A query sees both A keys. The first B query sees itself but neither A key. The padding row sees nothing.

For a DNA masked-reconstruction objective, bidirectional context may be intentional. Causality is not a universal biological requirement: the learning objective determines permitted information. Record boundaries and held-out-data boundaries still need independent protection.

8.2.1 Mask before normalization, not after it

Suppose two allowed scores are both zero and one forbidden score is 1000. The correct allowed weights are $(1/2, 1/2)$. If we first normalize all three logits, essentially all probability goes to the forbidden key. Setting its weight to zero afterward leaves almost no mass on either legal key. The error is in the denominator, not merely in a forbidden contribution to the output.

Masked softmax therefore excludes disallowed entries from normalization. A further edge case is a row with no allowed keys. Softmax of an all-negative-infinity row is undefined. We choose a clear policy: return all-zero weights, zero output, and zero gradients through that row. This is not a probability distribution; it is a sentinel behavior for an inactive query.

```
def masked_softmax(scores, allowed):
    """Empty rows return zeros with zero gradient; no softmax(-inf,...,-inf)."""
    if scores.ndim != 2 or scores.shape != allowed.shape:
        raise ValueError("scores and mask must be [queries,keys]")
    if allowed.dtype != torch.bool or allowed.device != scores.device:
        raise ValueError("boolean mask on score device required")
    if not scores.is_floating_point() or not torch.isfinite(scores).all():
        raise ValueError("finite floating scores required")
    if scores.shape[1] == 0:
        raise ValueError("at least one key required")
    has_key = allowed.any(dim=-1, keepdim=True)
    masked = scores.masked_fill(~allowed, -torch.inf)
    safe = torch.where(has_key, masked, torch.zeros_like(masked))
    weights = torch.softmax(safe, dim=-1)
    return torch.where(allowed, weights, torch.zeros_like(weights))
```

The temporary safe scores avoid evaluating an undefined all-masked softmax. The final selection removes every forbidden weight, including every weight in an empty row. In an active row, forbidden logits are replaced before softmax and cannot influence its denominator. The helper also rejects a physically empty key dimension and nonfinite input scores rather than silently propagating invalid values.

```
def attention(q, k, v, allowed):
    """Q[L,D], K[S,D], V[S,Dv] -> output[L,Dv], weights[L,S]."""
    if any(x.ndim != 2 for x in (q, k, v)):
        raise ValueError("rank-two tensors required")
    if (
        q.shape[1] == 0
        or q.shape[1] != k.shape[1]
        or k.shape[0] != v.shape[0]
    ):
        raise ValueError("query/key width and key/value length must agree")
    if any(x.dtype != q.dtype or x.device != q.device for x in (k, v)):
        raise ValueError("matching dtype and device required")
    if not q.is_floating_point() or not all(
```

```
        torch.isfinite(x).all() for x in (q, k, v)
    ):
        raise ValueError("finite floating tensors required")
    scores = (q @ k.T) / math.sqrt(q.shape[1])
    weights = masked_softmax(scores, allowed)
    return weights @ v, weights
```

A zero raw attention output does not guarantee a zero whole-block output. An output-projection bias or residual stream can contribute nonzero values. Nor does this mask automatically remove padding from the loss: the training objective must apply its own valid-target mask.

8.3 Derive the gradient through retrieval

Understanding a forward weighted average is only half the mechanism. Training adjusts both what is stored in the values and how the query addresses it. For one nonempty row, softmax has derivative

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j). \tag{8.8}$$

Substitute this into $o = \sum_i p_i v_i$:

$$\frac{\partial o}{\partial z_j} = \sum_i v_i p_i(\delta_{ij} - p_j) \tag{8.9}$$

$$= p_j v_j - p_j \sum_i p_i v_i = p_j(v_j - o). \tag{8.10}$$

Increasing one score shifts the output toward that key's value relative to the current mixture. If all allowed values are identical, changing the addressing weights cannot change the output: every score derivative is zero. If p_j is tiny, its score gradient is suppressed even when its value differs substantially from the mixture.

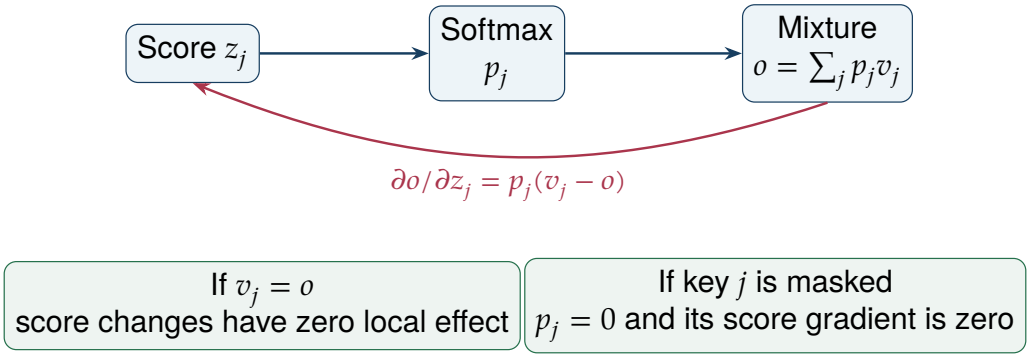


Figure 8.3: Backward sensitivity follows the retrieval mechanism. The derivative with respect to one score depends on both its probability and its value's difference from the current output. Identical values make addressing changes invisible; a tiny probability suppresses local score sensitivity.

8.3.1 The matrix backward pass

Let $\overline{O}$ denote the loss gradient at the output. The chain rule gives

$$\overline{V} = P^\top \overline{O}, \qquad \overline{P} = \overline{O} V^\top, \tag{8.11}$$

$$\overline{Z} = P \odot \left(\overline{P} - \operatorname{rowsum}(P \odot \overline{P}) \right), \tag{8.12}$$

$$\overline{Q} = \overline{Z} K / \sqrt{D}, \qquad \overline{K} = \overline{Z}^\top Q / \sqrt{D}. \tag{8.13}$$

The row sum is broadcast across each row. Forbidden entries have zero gradient because their scores cannot change the masked weights. The inactive-row policy also gives zero gradients. Autograd executes this chain rule for our tensor program; the equations explain what should be checked.

An attention weight alone is therefore not a complete attribution of a model prediction. Values, downstream projections, residual paths, and other layers affect the final output. Even within one head, Equation 8.10 shows why weight magnitude and output sensitivity are different quantities.

8.3.2 Test with more than one oracle

The test suite uses four complementary checks. First, the hand example fixes both weights and output. Second, finite-difference gradient checking tests the local derivative of the implemented operation. Third, PyTorch's scaled-dot-product attention supplies a separate forward and backward comparison with explicit masks and dropout probability zero. Finally, counterexamples perturb forbidden future or cross-record entries and require earlier valid outputs to remain unchanged.

The local reference was checked with PyTorch 2.10.0 on CPU float64. In the documented API, boolean true means an allowed attention edge; evaluation requires explicitly setting `dropout_p=0.0` when no dropout is wanted [32]. Other backends and precisions can have numerical differences. Agreement in this configuration is an arithmetic check, not a hardware benchmark or a proof that every deployment shares identical rounding.

8.4 Cache keys and values without changing the computation

For fixed parameters, consistent positional representations, and causal computation without stochastic dropout, earlier layer representations do not change when future tokens are appended. At the first layer, a past query cannot read future keys. Its output is therefore unchanged. Applying the same argument layer by layer establishes the property for a causal stack.

This explains a **KV cache**: retain each layer's past projected keys and values and append new ones. A current query reads those retained vectors. Past queries need not be stored merely to compute the new output. This argument fails if model parameters change, past tokens change, positions are reassigned, or the computation uses noncausal context.

Our reference receives already computed Q, K, and V. It does not implement a positional encoding or a complete multilayer decoder. The caller must compute position-dependent projections consistently and clear the cache at record boundaries.

8.4.1 The rectangular-mask trap

Suppose the cache has $P = 3$ past positions and the new chunk has $L = 2$ queries. The queries are at absolute positions 3 and 4, while the five keys are at positions 0 through 4. The correct rule is

$$A_{ij} = (j \le P + i), \quad 0 \le i < L, \quad 0 \le j < P + L. \tag{8.14}$$

The first query sees four keys; the second sees five. A naive upper-left triangle on a 2×5 array instead exposes only one and two keys.

```
def offset_mask(query_length, past, device=None):
    """Current chunk queries are at absolute positions past,...,past+L-1."""
    if any(type(n) is not int or n < 0 for n in (query_length, past)):
        raise ValueError("nonnegative integer lengths required")
    qpos = past + torch.arange(query_length, device=device)
    kpos = torch.arange(past + query_length, device=device)
    return kpos[None, :] <= qpos[:, None]
```

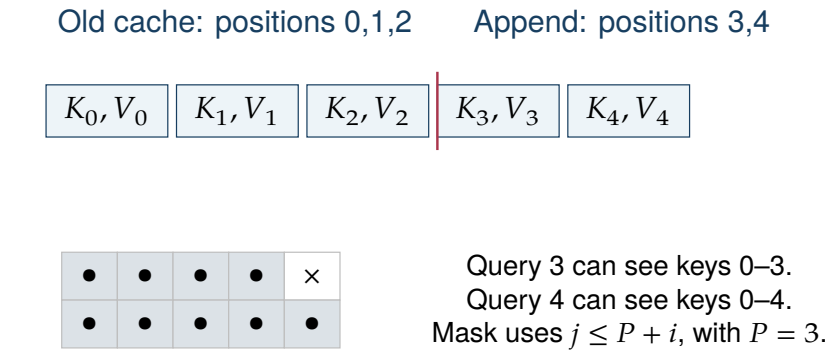


Figure 8.4: Chunked attention uses absolute positions. Three cached keys precede two new keys. The first new query sees positions 0–3; the second sees 0–4. A triangular rule based only on local row indices would incorrectly hide most of the available history.

PyTorch 2.10 documents upper-left alignment for its non-square `is_causal=True` behavior [32]. That flag is not a substitute for the explicit offset mask in this cache layout. Our negative-control test deliberately makes the naive call and confirms a different answer from the correctly aligned computation.

```
def cached_chunk(q, new_k, new_v, cache=None):
    """One record, fixed parameters/positions, no dropout; immutable
    ↪ concatenation."""
    if cache is None:
        old_k, old_v = new_k[:0], new_v[:0]
    else:
        old_k, old_v = cache
    if q.shape[0] != new_k.shape[0] or new_k.shape[0] != new_v.shape[0]:
        raise ValueError("one new key/value for each current query")
    if old_k.shape[0] != old_v.shape[0]:
        raise ValueError("cache key/value lengths must agree")
    keys = torch.cat((old_k, new_k), dim=0)
    values = torch.cat((old_v, new_v), dim=0)
    mask = offset_mask(q.shape[0], old_k.shape[0], q.device)
```

```
    output, _ = attention(q, keys, values, mask)
    return output, (keys, values)
```

The cache uses immutable concatenation so that the demonstration preserves autograd links to previous keys and values. A production inference cache would normally use allocated storage and controlled writes; repeated concatenation copies memory and is not an optimized serving implementation. Detaching cached tensors would preserve forward values but alter training gradients, so it must not be introduced into a backward-parity test without changing the intended contract.

8.4.2 Full-sequence and chunked equivalence

The tests process the same five positions as one chunk, five single-position chunks, and partitions $2 + 3$ and $3 + 2$. Concatenated outputs must match full causal attention. Backpropagating the same output objective must also produce matching gradients for Q, K, and V. This tests both the offset rule and preservation of graph connectivity.

The comparison assumes a single record, fixed input tensors, and nonempty chunks. It does not establish numerical identity for a full decoder that changes precision or kernel between prefill and decode. A correct attention cache is a necessary component, not a complete serving system.

8.5 Multiple heads: preserve feature ownership

Several attention heads apply the retrieval mechanism in different projected feature subspaces. For a projected tensor X of shape $[B, T, HD]$, each token's final dimension contains H consecutive groups of D features. Splitting heads requires two distinct operations:

$$[B, T, HD] \longrightarrow [B, T, H, D] \longrightarrow [B, H, T, D]. \tag{8.15}$$

The first operation groups features; the second swaps the head and time axes. Directly reshaping to $[B, H, T, D]$ would preserve the flat element order but assign elements to the wrong token/head coordinates.

```
def split_heads(x, heads):
    """[B,T,H*D] -> [B,H,T,D]; transpose, do not just reshape."""
    if (
        x.ndim != 3
        or type(heads) is not int
        or heads <= 0
        or x.shape[-1] % heads
    ):
        raise ValueError("rank three and width divisible by positive heads")
    b, t, width = x.shape
    return x.reshape(b, t, heads, width // heads).transpose(1, 2)
```

```
def merge_heads(x):
    """[B,H,T,D] -> [B,T,H*D]."""
    if x.ndim != 4:
        raise ValueError("rank four required")
    b, h, t, d = x.shape
    return x.transpose(1, 2).reshape(b, t, h * d)
```

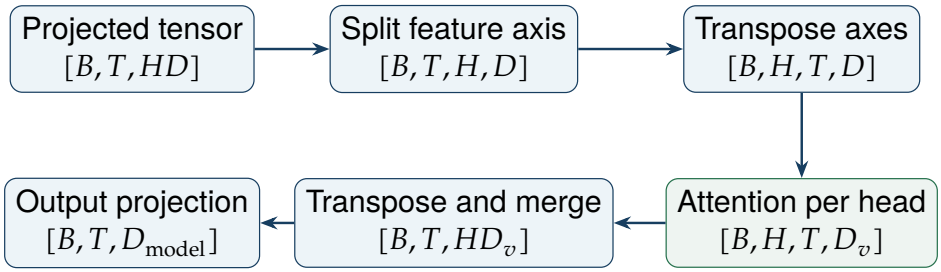


Figure 8.5: Multiple heads preserve token and feature ownership through reshape, transpose, per-head retrieval, and inverse transpose. Concatenation restores the feature axis before the learned output projection. Head splitting is an axis transformation, not an arbitrary rearrangement of the same element count.

If the model width is d_{model}, query and key projections can have shape $[d_{\text{model}}, HD]$, and a value projection $[d_{\text{model}}, HD_v]$. After independent head outputs are merged to $[B, T, HD_v]$, an output matrix of shape $[HD_v, d_{\text{model}}]$ maps back to the residual width.

A useful indexing test assigns distinct integers to every input element and verifies

$$X_{\text{split}}[b, h, t, d] = X[b, t, hD + d]. \tag{8.16}$$

A round trip alone is weaker: an incorrect split and its equally incorrect inverse could still cancel each other.

8.6 Count storage separately from work

For a full sequence of length T, our materialized score matrix contains T^2 entries per head. Only $T(T+1)/2$ causal query–key pairs are allowed, but the reference still allocates the dense matrix. For key and value width 64, a single head's cache stores $2T \cdot 64$ scalars:

T	Dense score cells	Causal pairs	KV scalars
16	256	136	2048
64	4096	2080	8192
256	65536	32896	32768
1024	1048576	524800	131072

These counts omit batch size, layers, other activations, and the bytes per scalar. Multiply by the appropriate factors before estimating memory. A cache stores retained keys and values; it does not store the entire attention matrix for each future query.

With one new query and T retained keys, scoring requires work proportional to TD and the value mixture proportional to TD_v. Caching avoids recomputing old projections and other past activations; it does not make a new query's full-history retrieval constant-time. Generating a growing sequence still accumulates a quadratic number of attention pairs.

The function also returns all weights for teaching and inspection. Memory-efficient fused implementations need not materialize that full matrix. This reference therefore cannot support claims about their throughput or memory use.

8.7 Assemble the next level only after the contracts hold

The operation now has explicit input shapes, normalized retrieval, legal information paths, gradients, cache positions, and head ownership. The next chapter will assemble these pieces with positional representations, normalization, residual paths, and a feed-forward sublayer into a Transformer block.

The design lesson applies to genomic modeling without turning a software diagram into a biological mechanism. Attention edges are permitted information paths between representations; they are not measured biochemical interactions. To argue that a model learns useful genomic dependencies, we will need an objective and evidence beyond an attractive attention map.

Run `python3 -m pytest tests/test_ch08_reference.py` and regenerate computed masks and tables with `python3 drafts/ch08/build.py --assets-only`. Inspect the negative controls as carefully as the passing examples: they show what a plausible implementation can get wrong.

Key ideas

- Masking must happen before normalization. Masking afterwards leaves the forbidden entry in the denominator.
- An all-masked row has no probability distribution. Its output needs an explicit policy.
- A cached chunk's queries sit at absolute positions. Applying a local triangular mask hides most of the available history.
- An attention weight is not an attribution: the values, later projections and residual paths also determine the output.

8.8 Exercises with worked reasoning

1. **Separate key and value roles.** Double the second value in Equation 8.2. *Solution:* weights remain $(2/3, 1/3)$; output becomes $(2, 4)$. The query and keys did not change.
2. **Shift every score.** Add 100 to both logits. What changes? *Solution:* nothing mathematically. The common exponential factor cancels; stable evaluation subtracts a common maximum.
3. **Check the scale argument.** Under the stated independence assumptions, what is the variance after division by $\sqrt{D}$? *Solution:* $D/D = 1$. Correlated learned features need not satisfy the assumptions.
4. **Break post-softmax masking.** Use legal scores $(0, 0)$ and forbidden score 1000. *Solution:* correct legal weights are $(1/2, 1/2)$. Normalizing all scores first gives the forbidden entry almost all mass; deleting it afterward does not repair the legal weights.
5. **Define an empty row.** Should its weights sum to one? *Solution:* not under our inactive-query policy. They sum to zero, and output and gradients are zero. This is a defined sentinel, not a distribution.
6. **Prevent record leakage.** Two unrelated records share class label 7. Can that label be their mask identity? *Solution:* no. Use unique record IDs; otherwise equality admits cross-record edges.
7. **Find a zero score gradient.** Let all allowed values equal v. *Solution:* $o = v$, so $p_j(v_j - o) = 0$ for every allowed score. Addressing changes cannot alter identical content.

8. **Construct an offset row.** With four cached positions, which keys can the first query in a two-position chunk see? *Solution:* keys 0 through 4. The query is at absolute position 4, not local position 0.

9. **Invalidate a cache.** Why not reuse old keys after updating projection weights? *Solution:* the cached vectors represent old parameters. Recomputed full attention would use different keys and values.

10. **Track a head feature.** For $D = 3$, where does head 2, feature 1 come from? *Solution:* input feature $2 \cdot 3 + 1 = 7$ at the same batch and time indices, using zero-based indexing.

11. **Count cache scalars.** Take $T = 256$ and $D = D_v = 64$ for one head. *Solution:* $2 \cdot 256 \cdot 64 = 32768$ scalars. At four bytes each this is 131072 bytes, before layers, batches, or allocator overhead.

12. **Set a comparison standard.** Does passing cache parity show that attention outperforms the preceding recurrence on DNA? *Solution:* no. It establishes an implementation property. A capability comparison needs matched data, objectives, resource accounting, independent evaluation, and uncertainty estimates.

Selective glossary

Query: a vector used to score compatibility with keys. **Key:** an address vector associated with a value. **Value:** the content vector mixed into the output. **Causal mask:** an information-flow rule preventing use of future positions. **KV cache:** retained projected keys and values reused by later queries under a fixed computation contract.

Sources and further reading

Sections 3.1–3.5 of the Transformer paper were inspected for attention, head projections, masking, and positional representations. The PyTorch 2.10 API documentation was inspected for mask polarity, dropout, rectangular causal alignment, and numerical-backend qualifications. The chapter's worked arithmetic, derivatives, counterexamples, and cache tests use the explicit contracts stated in the text. Access checked 20 September 2026.

CHAPTER 9

Assembling a Transformer block

In this chapter you will

- assemble a pre-normalized block from attention, LayerNorm, a GELU feed-forward branch and residual paths
- derive the LayerNorm backward pass, including the role of ϵ
- add sinusoidal absolute positions and keep them consistent across cached chunks
- count parameters by ownership and align next-token targets across records and padding

A working attention function is not yet a Transformer. It retrieves information from other positions, but it does not specify how that information modifies a persistent representation, how feature scales are controlled, how positions enter, or how predictions become a training objective. Those interfaces are the subject of this chapter.

Begin with a useful prediction test. If both learned residual branches output zero, should a block return its input exactly, or a normalized version of it? Our *pre-normalized* block must return its input. This small test will distinguish two superficially similar computational graphs before we train anything.

We will assemble a two-block reference with model width $D = 4$, two attention heads, feed-forward width $F = 8$, and a five-symbol synthetic vocabulary. The dimensions are small enough to audit every parameter. Symbols are abstract IDs, not a claim to model DNA. Within this book's taxonomy, Hermon is the Transformer-based direction and DOGMA is the non-Transformer direction. This chapter develops a component of the former, not comparative evidence that it is superior.

9.1 Name the residual stream and the two branches

Let $X \in \mathbb{R}^{T \times D}$ be the current representations of T token positions. With deterministic evaluation for now, define

$$U = X + \mathrm{Attn}(\mathrm{LN}_1(X)), \tag{9.1}$$

$$Y = U + \mathrm{FFN}(\mathrm{LN}_2(U)). \tag{9.2}$$

Every branch returns the same shape as its residual input. The two LayerNorm operations have different learned affine parameters. The attention branch mixes positions; the feed-forward branch applies the same feature transformation independently at each position.

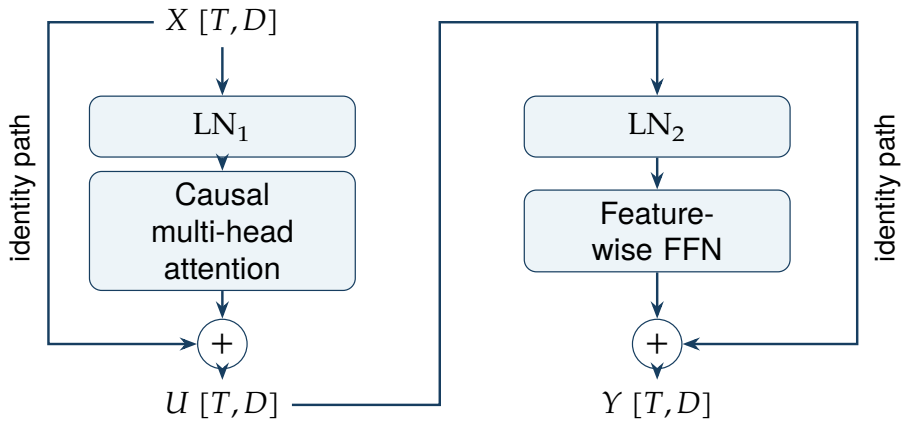


Figure 9.1: The chosen pre-normalized block. Normalization is inside each learned branch, while the identity paths carry the residual representations around those branches. Two additions preserve width D. Dropout, when enabled in this reference, acts on each branch output before its addition.

The original Transformer paper used normalization after residual addition [6]. Moving it inside the branch changes both forward values and gradients. Pre-normalization is an explicit architecture choice, not the definition of all Transformers. Xiong and colleagues studied the importance of this placement [33]; we use the distinction without importing their initialization theory as a guarantee about our small model.

For one sublayer $u = x + f(N(x))$, treating x as a column vector for differential notation,

$$J_u = I + J_f J_N, \qquad \bar{x} = \bar{u} + J_N^\top J_f^\top \bar{u}. \tag{9.3}$$

The bar denotes the derivative of a scalar loss with respect to a variable. The first term bypasses both learned transformation and normalization. It is a gradient route, not a proof that the total gradient cannot vanish: another term may oppose it.

Set $f = 0$. Then $u = x$ and $\bar{x} = \bar{u}$ exactly. A post-normalized operation $u = N(x + f(x))$ instead gives $u = N(x)$ under the same zero-branch condition. Our test zeros attention's output projection and the FFN's second projection, including their biases, and verifies identity values and identity input gradients through the whole block.

9.2 Layer normalization operates along features

For a single token vector $x \in \mathbb{R}^D$, define

$$\mu = \frac{1}{D}\sum_{j=1}^{D} x_j, \qquad c_j = x_j - \mu, \tag{9.4}$$

$$v = \frac{1}{D}\sum_{j} c_j^2, \qquad r = \sqrt{v + \epsilon}, \tag{9.5}$$

$$z_j = c_j / r, \qquad N(x)_j = \gamma_j z_j + \beta_j. \tag{9.6}$$

Here $\epsilon > 0$ stabilizes the denominator, and $\gamma, \beta \in \mathbb{R}^D$ are learned scale and shift. Variance divides by D, not $D - 1$; this matches the LayerNorm operation documented by PyTorch [34].

Each token uses its own feature statistics. No future token or other record contributes to its mean.

With $x = (1, 2, 3, 4)$, $\mu = 2.5$ and $v = 1.25$. For $\epsilon = 10^{-5}, \gamma = \mathbf{1}, \beta = \mathbf{0}$, the normalized values are approximately $(-1.341635, -0.447212, 0.447212, 1.341635)$. These are computed values, not manually chosen activations.

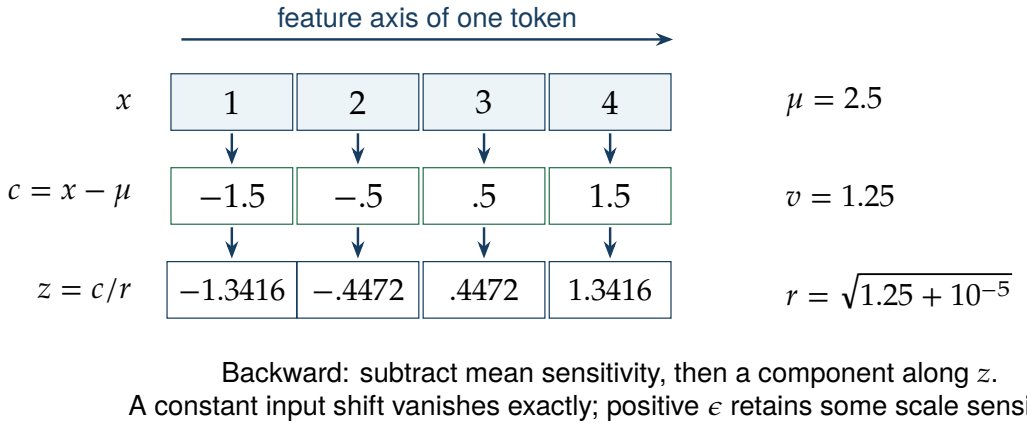


Figure 9.2: The four-feature normalization calculation. Subtracting a common mean and dividing by one token-specific scale couples that token's features, not its time positions. Positive ϵ is included in the displayed numbers and in the backward derivation.

```
def layer_norm(x, gamma, beta, eps=1e-5):
    """Normalize features only, separately at every token."""
    if (
        x.ndim != 2
        or gamma.shape != x.shape[-1:]
        or beta.shape != gamma.shape
    ):
        raise ValueError("expected x[T,D] and affine vectors [D]")
    if not math.isfinite(eps) or eps <= 0:
        raise ValueError("positive finite epsilon required")
    centered = x - x.mean(-1, keepdim=True)
    variance = centered.square().mean(-1, keepdim=True)
    return centered * torch.rsqrt(variance + eps) * gamma + beta
```

Adding a constant a to every feature changes μ by a, leaving c, v, and z unchanged. That is exact shift invariance. Scaling every feature by a positive a gives

$$z(ax) = \frac{ac}{\sqrt{a^2 v + \epsilon}}, \tag{9.7}$$

which is not exactly $z(x)$ when $\epsilon > 0$. The often-used intuition that normalization removes scale must retain this qualification. After affine scaling, even the output mean and variance need not be zero and one.

9.2.1 Derive the backward pass instead of memorizing it

Let $g_j = \bar{y}_j \gamma_j$, where $y = N(x)$. Mean subtraction gives $dc_j = dx_j - D^{-1} \sum_k dx_k$. Since $\sum_j c_j = 0$,

$$dv = \frac{2}{D} \sum_j c_j \, dx_j, \qquad dr = \frac{1}{Dr} \sum_j c_j \, dx_j. \tag{9.8}$$

Differentiate $z_j = c_j / r$:

$$dz_j = \frac{dx_j - \operatorname{mean}(dx)}{r} - \frac{c_j}{Dr^3} \sum_k c_k \, dx_k. \tag{9.9}$$

Multiplying by g_j, summing over j, and collecting coefficients of each dx_k yields

$$\bar{x} = \frac{1}{r} \left[g - \operatorname{mean}(g)\mathbf{1} - z \, \operatorname{mean}(g \odot z) \right]. \tag{9.10}$$

The affine derivatives are $\bar{\beta} = \sum_t \bar{y}_t$ and $\bar{\gamma} = \sum_t \bar{y}_t \odot z_t$, with sums over token positions in this reference. The code below exposes the input vector-Jacobian product; autograd computes all parameter gradients during model training.

```
def norm_vjp(x, gamma, upstream, eps=1e-5):
    """Analytic derivative with respect to x; affine parameters fixed."""
    centered = x - x.mean(-1, keepdim=True)
    scale = torch.rsqrt(centered.square().mean(-1, keepdim=True) + eps)
    z = centered * scale
    g = upstream * gamma
    return scale * (
        g - g.mean(-1, keepdim=True) - z * (g * z).mean(-1, keepdim=True)
    )
```

The components of $\bar{x}$ sum to zero, consistent with exact constant-shift invariance. But the centered direction is not generally an exact null direction:

$$J_z c = \frac{\epsilon}{r^3} c. \tag{9.11}$$

This follows by substituting $dx = c$ into the differential and using $\sum_j c_j^2 = Dv$. It also explains why setting ϵ to zero to make an algebraic argument prettier changes the operation.

For a constant input, $c = 0$ and $r = \sqrt{\epsilon}$. Output is β, while input perturbations can still have nonzero derivatives. Our tests include this case, finite-difference gradient checking, and direct forward/backward comparison with PyTorch's implementation.

9.3 Give position an absolute coordinate

Causal masking restricts which keys are visible, but it does not itself supply an explicit numerical position vector. We choose additive sinusoidal positions to make coordinate handling inspectable. This is one position scheme among many.

For even width D, frequency pair $i = 0, \ldots, D/2 - 1$, and absolute position t, let

$$\omega_i = 10000^{-2i/D}, \qquad P_{t,2i} = \sin(\omega_i t), \quad P_{t,2i+1} = \cos(\omega_i t). \tag{9.12}$$

The input stream is $X_t = E[\text{token}_t] + P_t$, with learned embedding table $E \in \mathbb{R}^{V \times D}$. This reference does not multiply embeddings by $\sqrt{D}$; the convention is deliberately explicit. For $D = 4$, the frequencies are 1 and 0.01.

t	$\sin t$	$\cos t$	$\sin(0.01t)$	$\cos(0.01t)$
0	0.00000	1.00000	0.00000	1.00000
1	0.84147	0.54030	0.01000	0.99995
2	0.90930	-0.41615	0.02000	0.99980
3	0.14112	-0.98999	0.03000	0.99955

```
def positions(length, width, offset=0, *, dtype=torch.float64, device=None):
    for value in (length, width, offset):
        if type(value) is not int or value < 0:
            raise ValueError(
                "nonnegative integer position dimensions required"
            )
    if width == 0 or width % 2:
        raise ValueError("positive even width required")
    pos = torch.arange(offset, offset + length, dtype=dtype, device=device)
    freq = torch.exp(
        -math.log(10000)
        * torch.arange(0, width, 2, dtype=dtype, device=device)
        / width
    )
    angle = pos[:, None] * freq[None, :]
    return torch.stack((angle.sin(), angle.cos()), dim=-1).flatten(1)
```

A fixed displacement δ has a useful exact relation:

$$\begin{bmatrix} \sin(\theta+\phi) \\ \cos(\theta+\phi) \end{bmatrix} = \begin{bmatrix} \cos\phi & \sin\phi \\ -\sin\phi & \cos\phi \end{bmatrix} \begin{bmatrix} \sin\theta \\ \cos\theta \end{bmatrix}, \qquad \phi = \omega_i \delta. \tag{9.13}$$

This follows from the angle-addition identities. The original Transformer paper motivates its sinusoidal construction with fixed-offset relationships [6]. Our rotation test verifies the arithmetic; it does not prove that trained attention extrapolates to unseen lengths.

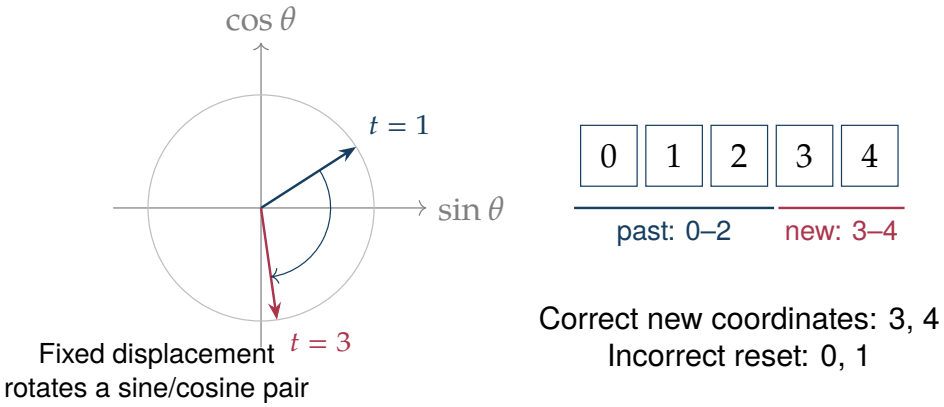


Figure 9.3: Position as a coordinate, not a chunk-local index. Each sinusoidal pair rotates under a fixed displacement. When three positions are cached, the next two tokens occupy positions 3 and 4. Resetting them to 0 and 1 changes the model inputs even if the causal mask remains correct.

9.4 Integrate attention and a nonlinear feature branch

Chapter 8 derived attention's normalized retrieval and masking. Now its projections become owned parameters of a block. With H heads and $d = D/H$, our combined projection matrix has shape $D \times 3D$. After projection, reshape $[T, 3D]$ into $[T, 3, H, d]$ and transpose each component to $[H, T, d]$.

Each head scores all legal cached and current keys, whose shape is $[H, S, d]$. The weights have shape $[H, T, S]$. Multiplication by values gives $[H, T, d]$, transposed and joined into $[T, D]$

before output projection. Below is the complete helper rather than a call that hides cache ownership.

```
def multihead(x, p, heads, cache=None, allowed=None):
    """Projected K,V cache [H,S,d]; immutable append preserves gradients."""
    t, width = x.shape
    if type(heads) is not int or heads <= 0 or width % heads:
        raise ValueError("heads must divide model width")
    d = width // heads
    qkv = (x @ p["wqkv"] + p["bqkv"]).reshape(t, 3, heads, d)
    q, k, v = (qkv[:, i].transpose(0, 1) for i in range(3))
    past = 0
    if cache is not None:
        old_k, old_v = cache
        if old_k.shape != old_v.shape or old_k.shape[::2] != (heads, d):
            raise ValueError("cache must contain matching [H,S,d] tensors")
        past = old_k.shape[1]
        k, v = torch.cat((old_k, k), 1), torch.cat((old_v, v), 1)
    if allowed is None:
        qi = past + torch.arange(t, device=x.device)
        kj = torch.arange(past + t, device=x.device)
        allowed = kj[None, :] <= qi[:, None]
    if allowed.dtype != torch.bool or allowed.shape != (t, past + t):
        raise ValueError("boolean mask must be [current queries, all keys]")
    scores = (q @ k.transpose(-2, -1)) / math.sqrt(d)
    masked = scores.masked_fill(~allowed, -torch.inf)
    active = allowed.any(-1, keepdim=True)
    safe = torch.where(active, masked, torch.zeros_like(masked))
    weights = torch.softmax(safe, -1).masked_fill(~allowed, 0)
    joined = (weights @ v).transpose(0, 1).reshape(t, width)
    return joined @ p["wo"] + p["bo"], (k, v)
```

Boolean true means an *allowed* edge in this helper. Empty query rows receive zero attention weights. The output projection bias, residual stream, and FFN can still produce nonzero padded-position outputs; the loss must independently exclude invalid targets. Custom packed masks are supplied explicitly for full-sequence execution. Default cached execution assumes one continuous record with valid tokens.

The feature branch is

$$\mathrm{FFN}(x) = \mathrm{GELU}(xW_1 + b_1)W_2 + b_2, \tag{9.14}$$

where $W_1 \in \mathbb{R}^{D\times F}$, $W_2 \in \mathbb{R}^{F\times D}$, $b_1 \in \mathbb{R}^F$, and $b_2 \in \mathbb{R}^D$. GELU is $a\Phi(a)$, where Φ is the standard normal cumulative distribution function [35]; the code requests its non-tanh approximation mode. Differentiation gives

$$\frac{d}{da}\left[a\Phi(a)\right] = \Phi(a) + a\varphi(a), \tag{9.15}$$

with φ the normal density.

Without a nonlinear activation, the two affine transformations collapse to a single affine map with weight W_1W_2. Expanding to width F would then not create this nonlinear feature interaction. With row-vector conventions, the local Jacobian is represented by $W_1 \operatorname{diag}(\mathrm{GELU}'(a))W_2$.

```
def feed_forward(x, p):
    return (
        F.gelu(x @ p["w1"] + p["b1"], approximate="none") @ p["w2"]
        + p["b2"]
    )
```

```
def block(
    x, p, heads=2, cache=None, allowed=None, dropout=0.0, training=False
):
    """Pre-LN; dropout only on the two residual-branch outputs."""
    if not 0 <= dropout < 1 or type(training) is not bool:
        raise ValueError("dropout in [0,1), explicit boolean training mode")
    normalized = layer_norm(x, p["g1"], p["n1"])
    branch, new_cache = multihead(normalized, p, heads, cache, allowed)
    u = x + F.dropout(branch, p=dropout, training=training)
    branch = feed_forward(layer_norm(u, p["g2"], p["n2"]), p)
    return u + F.dropout(branch, p=dropout, training=training), new_cache
```

The optional dropout is placed only at the two residual-branch outputs. We omit attention-weight dropout and FFN-hidden dropout. These are architecture choices. When dropout is disabled, the reference can be matched to PyTorch's pre-normalized `TransformerEncoderLayer` with GELU, affine biases, identical weights, and a causal mask [36]. The test compares output, input gradient, and every block-parameter gradient. Transposing linear weights and inverting boolean mask polarity are explicit parts of that comparison.

9.4.1 Count parameters by ownership

For this block, QKV weights contribute $3D^2$, attention output weights D^2, and FFN weights $2DF$. Biases contribute $3D + D + F + D$, while two affine normalizations contribute $4D$. Total:

$$P_{\text{block}} = 4D^2 + 2DF + 9D + F. \tag{9.16}$$

At $D = 4, F = 8$, this is 172. Two blocks use 344 parameters. Add $VD = 20$ embeddings, a final affine normalization with $2D = 8$, and an untied output head with $DV + V = 25$. The complete reference has 397 learned scalars, checked against actual tensor sizes.

This is a parameter count, not training memory. Optimizer state, activations, attention matrices, gradients, temporary buffers, and the number of bytes per scalar are separate quantities.

9.5 Carry one cache per block

For fixed weights, consistent absolute positions, and deterministic causal execution, past states do not change when future tokens are appended. Attention cannot read the future; normalization and the FFN act separately per token; residual addition is also local. Inducting through the two blocks preserves that property. Each block can therefore retain its projected past keys and values.

Crucially, block 2 receives the *output of block 1*. Its keys are projected from its own normalized input, not copied from block 1's key cache. Equal cache dimensions do not imply equal meaning.

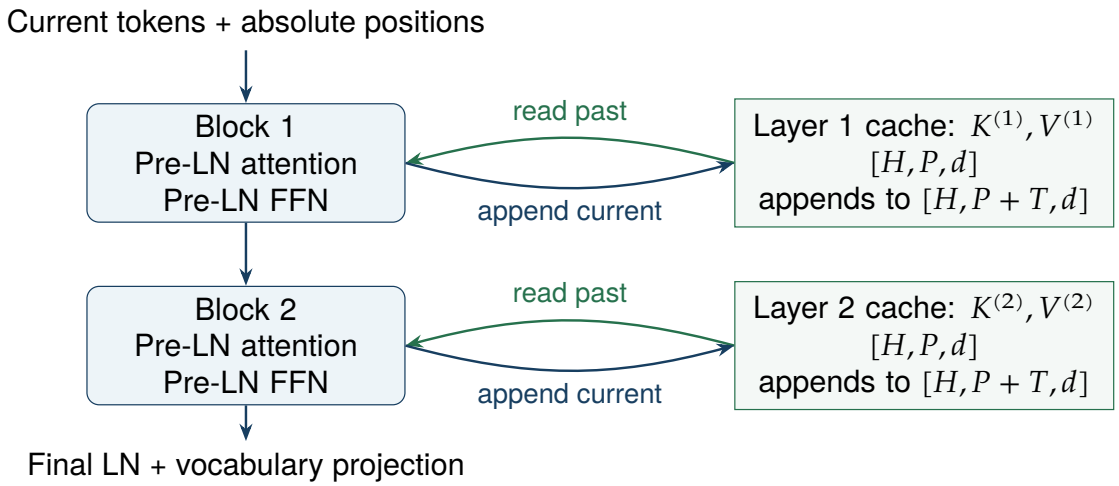


Figure 9.4: Two block-local histories. Each cache is derived from that block's normalized inputs and grows along its sequence dimension. Current representations pass through block 1 before entering block 2. Reading a past cache and appending current projections are different operations; neither permits interchanging caches between layers.

```
def model(
    tokens,
    p,
    heads=2,
    caches=None,
    allowed=None,
    dropout=0.0,
    training=False,
    position_offset=None,
):
    """One sequence/chunk; default absolute offset inferred from cache."""
    if tokens.ndim != 1 or tokens.dtype != torch.int64 or len(tokens) == 0:
        raise ValueError("nonempty int64 token vector required")
    if (tokens < 0).any() or (tokens >= p["embedding"].shape[0]).any():
        raise ValueError("token IDs must lie within the vocabulary")
    if caches is None:
        caches = [None] * len(p["blocks"])
    if len(caches) != len(p["blocks"]):
        raise ValueError("one cache slot per block required")
    lengths = [0 if c is None else c[0].shape[1] for c in caches]
    if len(set(lengths)) != 1:
        raise ValueError("all block histories must have the same length")
    past = lengths[0]
    offset = past if position_offset is None else position_offset
    x = p["embedding"][tokens]
    x = x + positions(
        len(tokens), x.shape[-1], offset, dtype=x.dtype, device=x.device
    )
    updated = []
    for weights, cache in zip(p["blocks"], caches):
        x, cache = block(
            x, weights, heads, cache, allowed, dropout, training
        )
        updated.append(cache)
```

```
    x = layer_norm(x, p["gf"], p["nf"])
    return x @ p["head"] + p["head_bias"], updated
```

The wrapper infers the positional offset from cache length. The optional override exists to construct a deliberate wrong-position counterexample; normal cached execution leaves it unset. The code checks that cache histories have the same length across blocks. Callers must retain parameter identity, record identity, dtype/device compatibility, and the intended position scheme; equal lengths alone do not establish those contracts.

For tokens $(0, 1, 2, 3, 4)$, the tests compare one full pass with five single-token chunks and partitions $2 + 3$ and $3 + 2$. They compare vocabulary logits and gradients with respect to every learned tensor. The caches use immutable concatenation without detaching, so backpropagation can follow the same past projections as in the full computation.

An inference runtime may use preallocated buffers and no autograd history. That is a different storage implementation, not a reason to detach caches in a training-gradient comparison. Repeated concatenation here copies memory and is intentionally not an optimized serving path.

9.5.1 Make the wrong implementation fail visibly

With the fixed seeded parameters, resetting the second chunk's positions to zero produces a maximum logit error of about 0.66348. Correct full/chunk differences are below the test tolerance. The negative control demonstrates that the test is capable of detecting a broken interface; it is not a benchmark result.

Dropout introduces another boundary. A random mask changes individual forward values even when its expectation preserves a branch's scale. PyTorch's functional dropout takes an explicit `training` flag [37]. Our wrapper passes that flag and the tests confirm that evaluation disables dropout even if its configured probability is nonzero. Two training runs with different random seeds differ.

Even resetting a random seed does not automatically align masks across different chunk partitions: call order and tensor shapes change random-number consumption. Full/chunk parity should first be established with dropout disabled, or with a separately specified mechanism for matching masks by absolute position. Deterministic parity is not a claim that all training executions commute with rechunking.

9.6 Shift targets and count only legal prediction edges

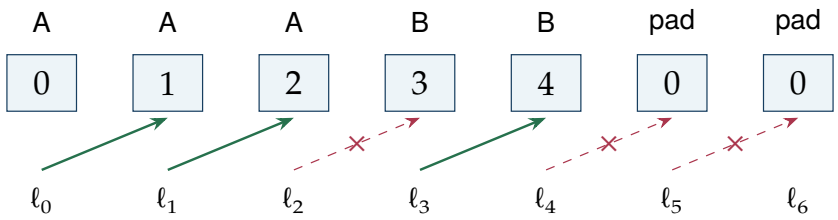
A logit vector $\ell_t \in \mathbb{R}^V$ at input position t predicts token s_{t+1}. Let u_t mark a valid token and r_t be a unique record identifier. Define the legal target indicator

$$m_t = u_t u_{t+1} [r_t = r_{t+1}], \qquad 0 \le t < T - 1. \tag{9.17}$$

The objective is

$$\mathcal{L} = -\frac{1}{M} \sum_{t=0}^{T-2} m_t \log \operatorname{softmax}(\ell_t)_{s_{t+1}}, \qquad M = \sum_t m_t > 0. \tag{9.18}$$

This mask answers which predictions enter the objective. The attention mask separately answers which inputs can influence each prediction. Either can be wrong while the other is correct.



Three legal targets: token 1, token 2, token 4.
Loss denominator = 3, not 7 and not 6.
ℓ_t is the logit vector at input position t, predicting token $t + 1$.
A causal attention mask and a target-validity mask do different jobs.

Figure 9.5: Target alignment across two records and padding. Three legal adjacent-token edges remain. The record boundary and padding edges do not enter the objective; the final input position has no next token. A mean over all seven slots would dilute the gradient by using the wrong denominator.

```
def next_token_loss(logits, tokens, valid, records):
    """Valid adjacent targets within one record; mean over legal edges."""
    t = len(tokens)
    if logits.ndim != 2 or logits.shape[0] != t or t < 2:
        raise ValueError(
            "logits[T,V] and at least two token positions required"
        )
    if tokens.shape != valid.shape or tokens.shape != records.shape:
        raise ValueError("token/validity/record vectors must match")
    if (
        tokens.dtype != torch.int64
        or records.dtype != torch.int64
        or valid.dtype != torch.bool
    ):
        raise ValueError(
            "int64 token/record vectors and boolean validity required"
        )
    keep = valid[:-1] & valid[1:] & (records[:-1] == records[1:])
    if not keep.any():
        raise ValueError("no legal next-token targets")
    return F.cross_entropy(logits[:-1][keep], tokens[1:][keep])
```

For uniform logits over five symbols, every legal target contributes $\log 5$. Mean loss is still $\log 5$, whether there are three legal edges or thirty. For one legal target class, the logit gradient is $(p - y)/M$. The unit test uses exactly three legal edges, checks a target-coordinate gradient of $-0.8/3$, and checks zero logit gradients at excluded positions. A batch with no legal targets raises an error rather than yielding a misleading zero loss.

An invalid position can retain nonzero hidden states without harming valid outputs if all information and target masks are correct. Conversely, zeroing padding embeddings alone does not prevent invalid targets or cross-record attention. Record IDs must identify records, not a class label shared by unrelated sequences.

9.7 Run a small, inspectable training experiment

The executable experiment uses one sequence $(0, 1, 2, 3, 0, 1, 2, 3, 0)$ followed by one invalid padding slot. It therefore has eight legal targets. The model uses two blocks, 397 parameters, a

fixed local initialization seed, CPU float64 arithmetic, and full-batch gradient descent with step size 0.1. No held-out generalization claim is attached to this repeated pattern.

```
def training_trace(steps=30):
    p = parameters()
    tokens = torch.tensor([0, 1, 2, 3, 0, 1, 2, 3, 0, 0])
    valid = torch.tensor([True] * 9 + [False])
    records = torch.zeros(10, dtype=torch.int64)
    mask = packed_mask(records, valid)
    trace = []
    for step in range(steps + 1):
        logits, _ = model(tokens, p, allowed=mask)
        loss = next_token_loss(logits, tokens, valid, records)
        trace.append(dict(step=step, loss=round(loss.item(), 10)))
        if step == steps:
            break
        loss.backward()
        with torch.no_grad():
            for value in leaves(p):
                value -= 0.1 * value.grad
                value.grad = None
    return trace
```

Initialization and parameter traversal are provided by `parameters` and `leaves` in the same reference file. Each step computes a new forward graph, differentiates the mean valid-target loss, updates every parameter under `no_grad`, and clears its gradient. The last trace row is evaluated after the thirtieth update; it is not a thirty-first update.

Step	Mean target loss
0	1.676427
10	1.350871
20	1.196540
30	1.067793

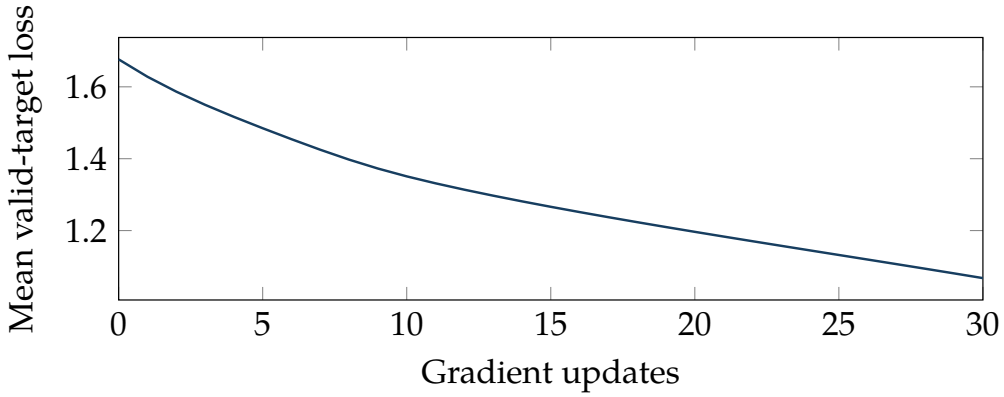


Figure 9.6: Computed training-set loss for the explicitly specified synthetic sequence. Each point uses the same eight-target mean. The trace tests the assembled learning path; it contains no held-out performance measurement.

The computed loss falls from approximately 1.67643 to 1.06779. This confirms that the assembled graph supports learning on the specified training sample. It does not establish genomic capability, superiority to recurrence, or useful out-of-distribution behavior.

Reproduce the data with `python3 drafts/ch09/build.py --assets-only` and the checks with `python3 -m pytest tests/test_ch09_reference.py`. The tests cover mathematical identities, a matched library implementation, full/chunk equivalence, legal information flow, objective alignment, and deliberately broken positions. Each check protects a different interface.

9.8 What is now assembled, and what remains

The reference contains embeddings, explicit positions, two causal pre-normalized blocks, a final normalization, an untied vocabulary head, and a valid-target training objective. It still excludes batching infrastructure, optimized caches, production precision, distributed execution, and evaluation on biological data.

This separation matters for Evolutor: a compiler or runtime cannot infer legal transformations from component names. It needs tensor shapes, parameter ownership, information-flow rules, and state lifetime. A rewrite that preserves a single attention call may still break position handling or the objective after composition.

Chapter 10 introduces conditional computation: when only selected subcomponents run, how are routing decisions, resource budgets, and gradients defined and tested? The dense block here provides the baseline against which that additional mechanism must be explained.

Key ideas

- With both branches zeroed, a pre-normalized block returns its input exactly; a post-normalized block does not.
- Normalization couples the features of one token, never positions or records. With $\epsilon > 0$ it is shift-invariant but not exactly scale-invariant.
- Each block owns its own cache. Equal shapes do not make block 1's keys usable by block 2.
- The attention mask decides which inputs a prediction may see; the target mask decides which predictions are scored. Either can be wrong while the other is right.

9.9 Exercises with worked reasoning

1. **Zero branches.** What does the pre-normalized block return if both residual branches are zero? *Solution:* $Y = X$ and the input Jacobian is identity. Post-normalization need not preserve X.

2. **Normalize a token.** Find mean and variance of $(1, 2, 3, 4)$. *Solution:* 2.5 and 1.25 using divisor four. Include ϵ before taking the square root.

3. **Find an exact invariance.** Why does $\sum_j \bar{x}_j = 0$? *Solution:* adding a constant to all input features leaves normalization unchanged. Equation 9.10 also sums directly to zero.

4. **Keep epsilon honest.** Is centered scale direction c an exact null direction? *Solution:* not generally: $J_z c = \epsilon c / r^3$.

5. **Position offset.** A cache contains seven tokens and the new chunk has three. *Solution:* new absolute positions are 7, 8, 9, not 0, 1, 2.

6. **Remove nonlinearity.** Replace GELU by identity. *Solution:* the FFN becomes affine with weight $W_1 W_2$ and bias $b_1 W_2 + b_2$.

7. **Count ownership.** Evaluate $4D^2 + 2DF + 9D + F$ for $D = 4, F = 8$. *Solution:* $64 + 64 + 36 + 8 = 172$ parameters per block.

8. **Separate cache identity.** Why cannot block 2 reuse block 1's keys? *Solution:* its keys have different projections and inputs; identical shapes do not imply identical values or semantics.

9. **Check target denominator.** Use the seven-slot example in Figure 9.5. *Solution:* three targets remain, so uniform five-way loss is $\log 5$ and a target logit derivative is $-0.8/3$.

10. **Distinguish masks.** Does excluding a cross-record target prevent cross-record attention? *Solution:* no; attention requires its own record-boundary mask.

11. **Challenge parity.** A cached test detaches old values and compares only logits. *Solution:* it may pass forward parity while breaking parameter gradients. Compare the same scalar objective's gradients before claiming training equivalence.

12. **Design an evaluation.** Does the synthetic loss curve justify choosing Hermon over DOGMA? *Rubric:* no. Specify matched tasks, held-out biological data, leakage controls, budgets, baselines, uncertainty, and failure cases. Arithmetic correctness is a prerequisite, not comparative capability evidence.

Sources and further reading

Sections 3.1–3.5 of the original Transformer paper were inspected for the baseline sublayers and sinusoidal positions. Xiong and colleagues' abstract and introduction were inspected for the pre-/post-normalization distinction; their mean-field theory and benchmarks are not reproduced. PyTorch 2.10 API descriptions were inspected for normalization statistics, block configuration, GELU, and explicit dropout mode. Access checked 20 September 2026.

The local tests use PyTorch 2.10.0, CPU float64. The seeded training trace is a synthetic training-set calculation, not a validation or biological benchmark.

Selective glossary

Residual stream: representation preserved and incrementally modified by branch additions. **Pre-normalization:** normalization of a branch input before its learned transformation. **VJP:** vector-Jacobian product propagating a scalar objective's sensitivity backward. **Absolute position:** sequence coordinate independent of chunk boundaries. **Legal target edge:** a next-token prediction admitted by validity and record identity.

CHAPTER 10

Conditional computation and routing

In this chapter you will

- separate router scoring, top-1 selection, capacity admission and probability weighting
- gather admitted rows to experts and scatter results back, verified against a dense all-expert oracle
- derive which gradients exist for a fixed routing decision, and which do not
- account for demand, overflow, unused slots and multiply–accumulate counts

Six tokens choose among three feed-forward networks. Each network has two available slots. Six tokens, six slots: should every token fit?

No. If their choices are $(0, 0, 0, 2, 1, 1)$, expert 0 receives three requests while expert 2 receives one. A spare slot at expert 2 does not help the third token requesting expert 0 unless the routing policy permits reassignment. A capacity budget is not a guarantee of balanced demand.

Chapter 9 assembled a dense Transformer block. Every valid token used the same feed-forward parameters. Here we replace only that branch with conditional expert execution, then make selection, admission, weighting, gradients, and state ownership explicit. Our small reference is not a Switch Transformer training reproduction or an optimized distributed runtime. Its purpose is to reveal the contracts a future Evolutor implementation must preserve.

10.1 Separate scoring, selection, admission, and weighting

Let $Z \in \mathbb{R}^{T \times D}$ contain the branch inputs for T token positions. There are E experts, each mapping $\mathbb{R}^D$ to $\mathbb{R}^D$. A router $R \in \mathbb{R}^{D \times E}$ produces logits $H = ZR$. For token t,

$$p_{tj} = \frac{\exp(h_{tj})}{\sum_{r=0}^{E-1} \exp(h_{tr})}, \qquad k_t = \arg\max_j p_{tj}. \tag{10.1}$$

Expert indices start at zero. Probabilities sum to one over experts, not tokens. Softmax is evaluated by the library's numerically stable operation. A row-wise constant shift in logits leaves the probabilities unchanged.

Top-1 selection chooses one expert index. It does not yet grant a capacity slot. Let v_t indicate a valid token and a_t indicate admission. Our policy visits tokens in their input order, excludes invalid positions, and admits a token if its selected expert has fewer than C occupied slots. It never reroutes overflow. The branch result is

$$b_t = a_t p_{tk_t} E_{k_t}(z_t), \qquad y_t = u_t + b_t. \tag{10.2}$$

u_t is the residual representation being updated. A dropped branch leaves $y_t = u_t$; it does not remove the token from the sequence or from a later loss.

This separation follows the useful top-1, probability-weighted expert mechanism described by Fedus, Zoph, and Shazeer [38]. Our deterministic first-arrival implementation and test cases are deliberately small. We do not inherit that paper's training, hardware, or quality results.

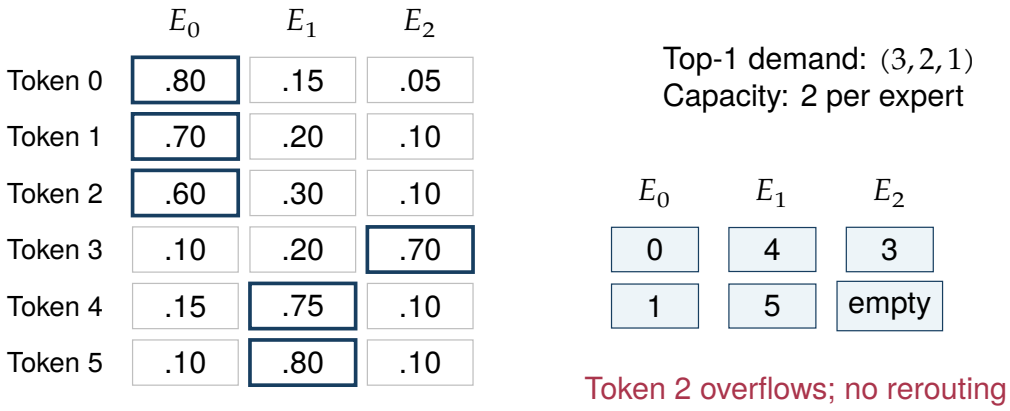


Figure 10.1: Six fully specified probability rows. The selected experts request $(3, 2, 1)$ slots; capacity two gives admitted counts $(2, 2, 1)$. Token 2 is rejected after tokens 0 and 1 occupy expert 0. The vacant slot at expert 2 remains vacant under this no-rerouting policy.

Token	Expert	Gate	Admitted
0	0	0.8	yes
1	0	0.7	yes
2	0	0.6	no
3	2	0.7	yes
4	1	0.75	yes
5	1	0.8	yes

10.1.1 Ties and padding are part of the definition

If logits tie, a result must still be chosen. This reference uses the first maximal expert index, matching the documented `torch.argmax` convention [39]. A tie is not broken by a random coin or by the least loaded expert. Those would be different policies with different reproducibility requirements.

An invalid padding position may have logits and a selected index in the intermediate tensors, but it consumes no slot, contributes no expert branch, and does not enter the load-balancing denominator. Attention validity and next-token target validity remain the separate Chapter 9 contracts.

The implementation returns both *demand* before admission and occupied *used* slots after admission. In a continued cohort, used counts include earlier chunks, while demand reports only the current call.

```
def plan(logits, valid, capacity, used=None):
    """Top-1, first-index ties, first-arrival admission; padding excluded."""
    if (
        logits.ndim != 2
        or min(logits.shape) < 1
        or not logits.is_floating_point()
        or not torch.isfinite(logits).all()
    ):
        raise ValueError("finite floating logits[T,E] required")
```

```
    t, e = logits.shape
    if valid.shape != (t,) or valid.dtype != torch.bool:
        raise ValueError("boolean validity[T] required")
    if type(capacity) is not int or capacity < 0:
        raise ValueError("nonnegative integer capacity required")
    used = [0] * e if used is None else list(used)
    if len(used) != e or any(
        type(n) is not int or not 0 <= n <= capacity for n in used
    ):
        raise ValueError("one bounded integer counter per expert required")
    p = torch.softmax(logits, dim=-1)
    selected = p.argmax(-1)
    accepted = torch.zeros(t, dtype=torch.bool, device=logits.device)
    demand = [0] * e
    for i in range(t):
        if bool(valid[i]):
            j = int(selected[i])
            demand[j] += 1
            if used[j] < capacity:
                accepted[i] = True
                used[j] += 1
    return dict(
        probability=p,
        selected=selected,
        accepted=accepted,
        demand=demand,
        used=used,
    )
```

The Python loop exposes control flow but synchronizes scalar decisions with the host. It is intended for transparent CPU tests, not GPU throughput. Real dispatch kernels can implement the same policy through prefix counts and indexed communication. They must first agree on exactly which policy they are implementing.

10.2 Gather only admitted rows, then restore token identity

Each expert is a feed-forward network:

$$E_j(z) = \mathrm{GELU}(zW_{1j} + b_{1j})W_{2j} + b_{2j}. \tag{10.3}$$

W_1 has stacked shape $[E, D, F]$, b_1 has $[E, F]$, W_2 has $[E, F, D]$, and b_2 has $[E, D]$, where F is hidden width. These are independent parameter sets. The word expert describes ownership and conditional use; it does not establish meaningful specialization before training and evaluation.

```
def expert(x, weights, j):
    return (
        F.gelu(x @ weights["w1"][j] + weights["b1"][j], approximate="none")
        @ weights["w2"][j]
        + weights["b2"][j]
    )
```

For expert j, form the ordered row index set

$$I_j = \{t : a_t = 1,\ k_t = j\}. \tag{10.4}$$

Gather $Z[I_j]$ with shape $[|I_j|, D]$, evaluate only that expert on those rows, multiply row t by p_{tj}, and scatter into the original token positions. Top-1 admission makes the I_j disjoint, so no token receives two writes. A top-k extension would require accumulation, not blindly reusing this single-write assumption.

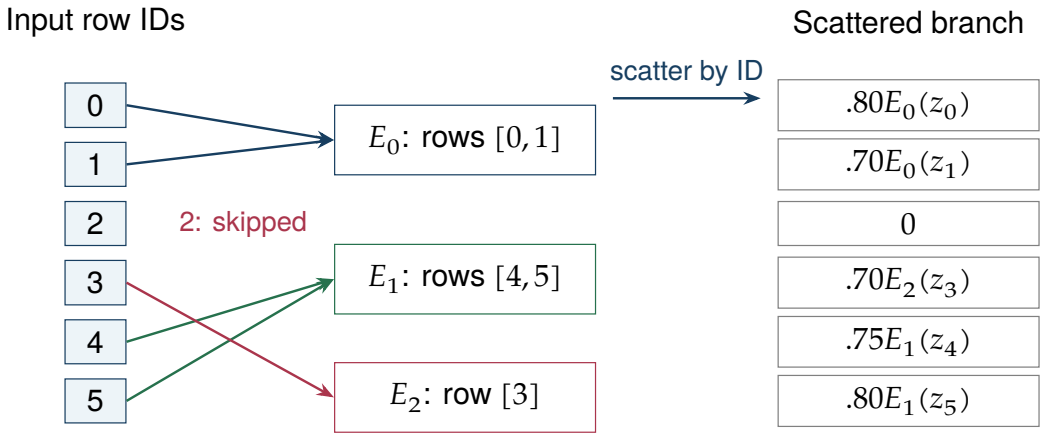


Figure 10.2: From token-major inputs to expert minibatches and back. Expert 0 receives rows 0 and 1; expert 1 receives rows 4 and 5; expert 2 receives row 3. The output preserves original token order, including a zero branch at row 2. The formulas use the probabilities in Figure 10.1.

```
def dispatch(x, weights, routing):
    """Gather admitted rows only; scatter weighted results to original rows."""
    p, chosen, keep = (
        routing[k] for k in ("probability", "selected", "accepted")
    )
    if x.ndim != 2 or x.shape[0] != p.shape[0]:
        raise ValueError("matching token axis required")
    out = x * 0 + p.sum() * 0
    for j in range(p.shape[1]):
        ids = torch.where(keep & (chosen == j))[0]
        if len(ids):
            values = expert(x[ids], weights, j) * p[ids, j, None]
            out = out.index_copy(0, ids, values)
    return out
```

The zero initialization retains a zero-valued autograd connection to inputs and router probabilities. This lets an all-overflow batch backpropagate zero branch gradients instead of failing because the output has no computational graph. An expert that is never executed can have an unused gradient represented as `None`. In comparisons, the tests interpret that as zero sensitivity, not as proof that every optimizer treats an absent gradient identically to an explicit zero tensor.

10.2.1 Construct a deliberately expensive oracle

A correctness oracle evaluates *every* expert at every token, producing $V \in \mathbb{R}^{T \times E \times D}$. Define

$$G_{tj} = a_t[\, k_t = j \,]p_{tj}, \qquad B_{td} = \sum_j G_{tj} V_{tjd}. \tag{10.5}$$

This formula and sparse dispatch should agree on values and derivatives when the routing plan is the same. The oracle wastes expert work but makes the weighted sum easy to inspect.

```
def dense_oracle(x, weights, routing):
    """Compute every expert, then mask. Correctness oracle, not sparse
     ↪  execution."""
    p, chosen, keep = (
        routing[k] for k in ("probability", "selected", "accepted")
    )
    all_values = torch.stack(
        [expert(x, weights, j) for j in range(p.shape[1])], dim=1
    )
    gate = F.one_hot(chosen, p.shape[1]) * keep[:, None] * p
    return (all_values * gate[:, :, None]).sum(1)
```

The tests compare outputs, input gradients, logit gradients, and all stacked expert-parameter gradients at capacities $0, 1, 2, 6$. A separate exact table tests the plan itself. Sharing a plan between two execution paths does not verify selection or admission, which is why both kinds of check are required.

An expert permutation provides another test. Away from ties, permute logit columns and the corresponding expert weights together. The physical result should stay the same. At ties, the first-index rule intentionally depends on index order, so unrestricted permutation invariance is not promised.

10.3 Differentiate the chosen computation, not a fictional smooth selector

Within a region where k_t and a_t do not change, consider one admitted token and write $b = p_k E_k(z)$. For upstream sensitivity $\overline{b} = \partial \mathcal{L}/\partial b$, define the scalar $s = \overline{b}^{\top} E_k(z)$. The softmax derivative is

$$\frac{\partial p_k}{\partial h_j} = p_k(\mathbf{1}_{j=k} - p_j). \tag{10.6}$$

The chain rule then gives

$$\frac{\partial \mathcal{L}}{\partial h_j} = a\, s\, p_k(\mathbf{1}_{j=k} - p_j). \tag{10.7}$$

Even an unselected expert's *router logit* can receive a gradient because it participates in the softmax denominator. That is different from evaluating the unselected expert network or updating its weights through this token's task loss.

For the expert parameters, the upstream sensitivity is scaled by ap_k. For router weights, $\overline{R} = Z^{\top}\overline{H}$. For the input z, one path passes through the selected expert and another through $h = zR$:

$$\overline{z} = ap_k J_{E_k}^{\top}\overline{b} + R\,\overline{h} \tag{10.8}$$

when writing individual feature sensitivities as column vectors. The dimensions are $R : D \times E$, $\overline{h} : E$, and $\overline{z} : D$. In the complete block, normalization and the identity residual add their own previously derived paths.

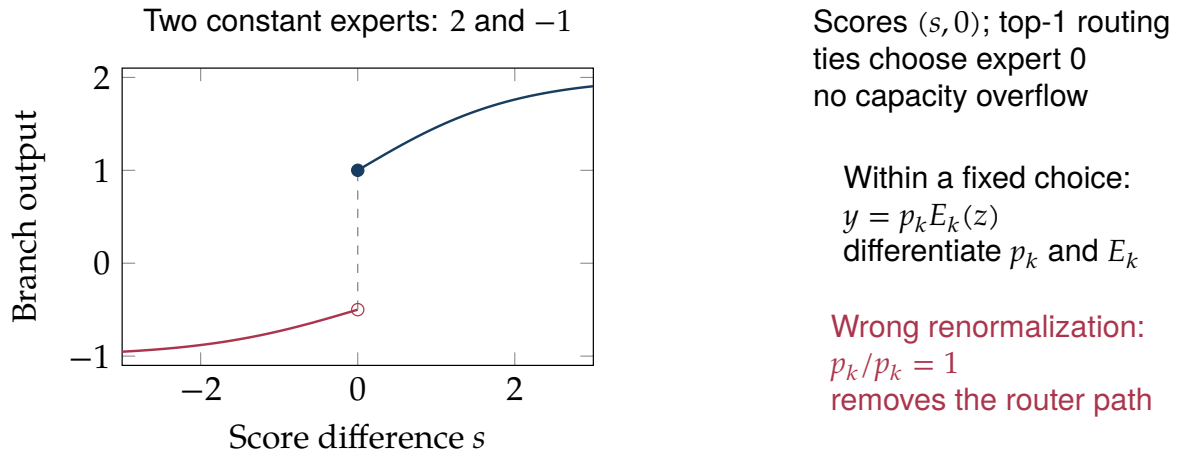


Figure 10.3: A routing boundary with no capacity pressure. For logits $(s, 0)$ and constant expert outputs 2 and -1, the selected output has smooth pieces but a jump at $s = 0$. First-index ties choose expert 0, giving output 1 exactly at the boundary. The open circle marks the other branch's limiting value, -0.5.

The argmax operation does not supply an ordinary derivative describing how a small parameter change swaps experts. Away from a boundary it is locally constant; at a tie the selected function may be discontinuous, as in Figure 10.3. Our autograd path differentiates the chosen smooth piece and does not invent a straight-through estimator for the discrete decision. Finite-difference checks use scores safely away from ties.

10.3.1 Why top-1 renormalization can erase a learning path

Suppose an implementation selects one expert and then renormalizes over only the selected set. Its gate becomes

$$\tilde{p}_k = \frac{p_k}{p_k} = 1. \tag{10.9}$$

The output is now $E_k(z)$. Away from selection boundaries, the task-loss gradient through the router scores vanishes. This is not equivalent to Equation 10.2. It might be a deliberate architecture with a separate router objective, but it cannot silently replace probability weighting.

One test uses logits $(1, 0, -1)$ and scalar expert value 2. It checks Equation 10.7 directly, then verifies that the renormalized expression has zero logit gradient. This negative control proves the test can detect the missing router path. Gradient correctness is more informative here than merely observing that the code runs.

10.4 Capacity creates state and grouping semantics

Let $d_j = \sum_t v_t[\, k_t = j \,]$ be demand in one cohort, with initially empty slots. Under the stated policy,

$$q_j = \min(d_j, C), \quad A = \sum_j q_j, \quad D_{\text{drop}} = T_v - A, \quad U = EC - A, \tag{10.10}$$

where $T_v = \sum_t v_t$ is the valid-token count. q_j counts admitted tokens, A counts all admitted tokens, and U counts unused slots. Demand is not admitted load, and neither is total physical device work.

For $d = (3, 2, 1)$ and $C = 2$, $A = 5$, $D_{\text{drop}} = 1$, and $U = 1$. Increasing total capacity helps only when it changes a constrained expert's available slots. Our capacity sweep is computed, not a hardware measurement.

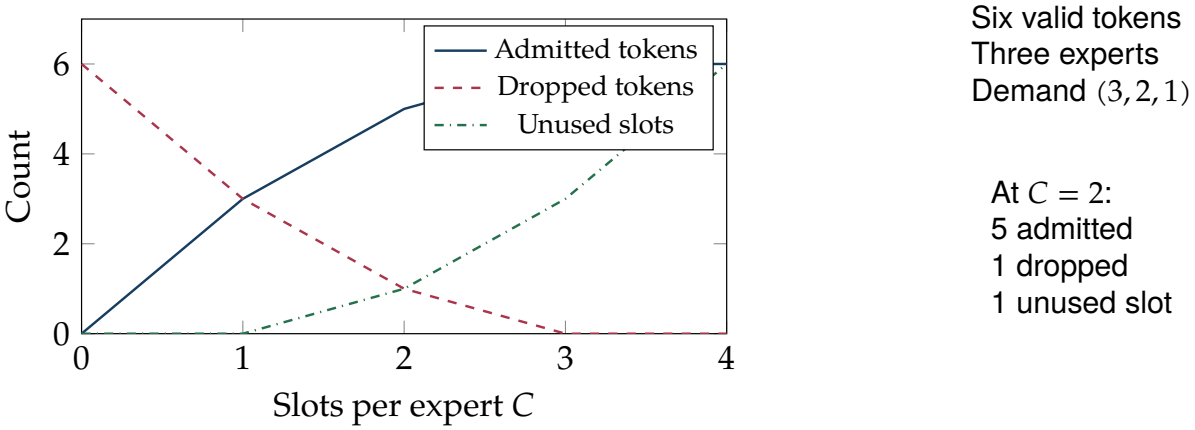


Figure 10.4: Demand, overflow, and unused capacity are different quantities. At two slots per expert, one token is dropped despite one unused slot. Increasing capacity to three admits all six; further capacity adds unused slots under the same deterministic demand. Lines connect discrete integer-capacity cases.

Capacity	Admitted	Dropped	Unused slots
0	0	6	0
1	3	3	0
2	5	1	1
3	6	0	3
4	6	0	6

A common family of designs sets capacity from a factor times average demand; the Switch paper describes this trade-off [38]. For a concrete integer policy one might choose $C = \lceil cT_v/E \rceil$. This reference instead accepts C explicitly so rounding and cohort size cannot remain hidden. Changing T_v or the capacity factor can change the forward computation.

10.4.1 Resetting counters is not a harmless implementation detail

Process the six-token example as a full cohort, then split after token 1. The first two tokens both select expert 0 and fill it. If the second call starts with fresh counters, token 2 is now admitted. If it receives the used counts $(2, 0, 0)$, token 2 remains an overflow and the original result is recovered.

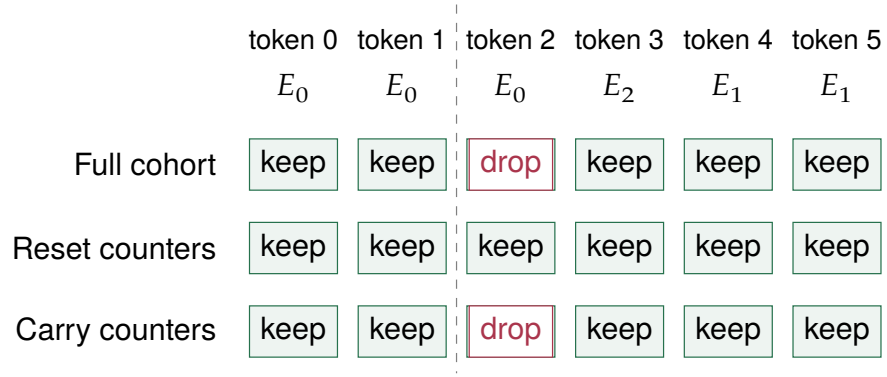


Figure 10.5: Capacity history under rechunking. Token logits and expert weights remain fixed. Resetting counters at the split changes admission of token 2; carrying used counts with the same capacity and order preserves the full-cohort decision. These counters are not an attention KV cache.

The code copies incoming counters rather than mutating the caller's list. Parity requires the same cohort, capacity, token order, logits, and parameters. Carrying counters alone cannot repair logits changed by incorrect position or attention handling. Nor does this demonstration establish an optimized autoregressive cache implementation.

This reveals a boundary of Chapter 9's token-local FFN argument. Expert scoring is token-local, but capacity admission couples tokens through a shared resource. Reordering the first three tokens changes which one is dropped. Combining otherwise isolated records into one capacity cohort can make one record's admission depend on another record's demand, even if attention is correctly record-masked.

With fixed capacity and first-arrival order, later tokens cannot change earlier admissions. But computing capacity from a changing total length or selecting admitted tokens by a global score ranking can introduce different dependencies. A runtime must document grouping and admission order before claiming causal or chunk-equivalent execution. If full-sequence training and token-at-a-time inference use different capacity conventions, matching attention alone does not prove model equivalence.

10.5 Load balancing is an objective, not a certificate

Use pre-admission demand fractions and mean router probabilities over valid tokens:

$$f_j = \frac{1}{T_v} \sum_t v_t [\, k_t = j \,], \qquad P_j = \frac{1}{T_v} \sum_t v_t p_{tj}. \tag{10.11}$$

A Switch-style auxiliary term is

$$\mathcal{L}_{\text{bal}} = \alpha E \sum_j f_j P_j, \qquad \alpha \geq 0. \tag{10.12}$$

The assignment fractions f_j are discrete and treated as fixed in the local gradient. The probabilities P_j are differentiable. Counting requests before overflow matters: an overloaded expert should not look less demanded merely because some requests were rejected.

```
def balance_loss(routing, valid, alpha=0.01):
    """Demand fractions before overflow; discrete assignment is not
     ↪  differentiated."""
    if not math.isfinite(alpha) or alpha < 0 or not bool(valid.any()):
        raise ValueError(
            "nonnegative coefficient and valid tokens required"
        )
    p = routing["probability"][valid]
    chosen = routing["selected"][valid]
    fraction = F.one_hot(chosen, p.shape[1]).to(p.dtype).mean(0)
    return alpha * p.shape[1] * (fraction * p.mean(0)).sum()
```

Apply the softmax derivative and collect terms for a valid token:

$$\frac{\partial \mathcal{L}_{\text{bal}}}{\partial h_{tj}} = \frac{\alpha E}{T_v} \sum_r f_r p_{tr} (\mathbf{1}_{r=j} - p_{tj}) \tag{10.13}$$

$$= \frac{\alpha E}{T_v} p_{tj} \left(f_j - \sum_r f_r p_{tr} \right). \tag{10.14}$$

The tests compare this formula with autograd. An invalid token contributes no term; a call with zero valid tokens raises an error rather than dividing by zero. A dropped token can still receive an auxiliary router gradient even though its expert branch contributed no task-loss gradient.

If $f_j = P_j = 1/E$, Equation 10.12 equals α. But this numeric value is not proof of uniform hard assignments. Set every logit to zero. Then $P_j = 1/E$, while first-index ties send every request to expert 0, giving $f = (1, 0, \ldots, 0)$. The same loss value α appears. Its gradient need not be zero. Therefore log actual demand, admissions, and overflow along with the scalar loss. An attractive auxiliary-loss curve does not certify balanced execution, expert specialization, or useful prediction.

10.6 Replace the feed-forward branch in the existing block

The integration keeps Chapter 9's attention and normalizations:

$$U = X + \text{Attn}(\text{LN}_1(X)), \tag{10.15}$$

$$Z = \text{LN}_2(U), \tag{10.16}$$

$$Y_t = U_t + a_t p_{tk_t} E_{k_t}(Z_t). \tag{10.17}$$

Our wrapper reuses `drafts/ch09/reference.py` for LayerNorm and attention instead of reprinting a subtly different implementation. The review record binds that reused source hash as well as the new chapter. The example fixes two attention heads and disables dropout.

```
def sparse_block(
    x,
    block_weights,
    router,
    experts,
    valid,
    capacity,
    allowed=None,
    used=None,
```

```
):
    """Replace only Chapter 9's FFN branch; no dropout in this reference."""
    normalized = base.layer_norm(
        x, block_weights["g1"], block_weights["n1"]
    )
    attention, _ = base.multihead(
        normalized, block_weights, heads=2, allowed=allowed
    )
    u = x + attention
    z = base.layer_norm(u, block_weights["g2"], block_weights["n2"])
    routing = plan(z @ router, valid, capacity, used)
    return u + dispatch(z, experts, routing), routing
```

The full-compute oracle is also inserted at this same branch location. Tests compare the integrated outputs and gradients through attention, normalization, router, experts, and inputs. At zero capacity, the result must be exactly U, not X and not a zero tensor: the attention branch has already run.

There is also an exact dense limit. With one expert, its softmax probability is one; if capacity admits all valid tokens and its weights equal the old FFN, the unpadded no-dropout block equals Chapter 9's dense block. Increasing to several experts changes the function and training problem; it does not merely speed up the same already trained dense network.

10.7 Count arithmetic without promising speed

Ignoring bias additions and nonlinearities, one expert evaluation on one token uses approximately $2DF$ multiply-accumulates (MACs). If A tokens are admitted, expert work is $2ADF$. Scoring all T positions costs TDE MACs in our implementation, including padded positions whose scores are later ignored. An all-expert oracle uses $2TEDF$ expert MACs.

```
def cost(tokens, experts, width, hidden, admitted):
    """Matrix MACs; excludes bias, activation, selection, and transfers."""
    for value in (tokens, experts, width, hidden, admitted):
        if type(value) is not int or value < 0:
            raise ValueError("nonnegative integer sizes required")
    if min(experts, width, hidden) == 0 or admitted > tokens:
        raise ValueError(
            "positive dimensions and admitted <= tokens required"
        )
    return dict(
        router_macs=tokens * width * experts,
        expert_macs=2 * admitted * width * hidden,
        all_expert_macs=2 * tokens * experts * width * hidden,
        expert_parameters=experts * (2 * width * hidden + hidden + width),
    )
```

At $T = 6, E = 3, D = 4, F = 8, A = 5$, the router uses 72 MACs and admitted experts use 320, versus 1152 expert MACs in the deliberately expensive oracle. But the original *single dense FFN* uses only 384 MACs on six tokens. The routed arithmetic total 392 already exceeds that dense baseline, before dispatch overhead. Confusing an all-expert oracle with the original dense baseline would manufacture a misleading speedup story.

The experts store

$$E(2DF + F + D) = 3(64 + 8 + 4) = 228 \tag{10.18}$$

parameters, plus $DE = 12$ router weights in this bias-free router. Stored parameters, active arithmetic, optimizer state, and communication are separate budgets. In a distributed design, gathering activations, transfers between devices, imbalanced expert batches, synchronization, and padding can dominate wall-clock time. None is measured by these MAC formulas.

Dropping work also changes the function. A low operation count obtained by rejecting all tokens is not a successful optimization of a model that should have evaluated their branches. Compare quality and semantics at matched admission policies before measuring systems performance.

10.8 What the reference establishes

Run `python3 drafts/ch10/build.py --assets-only` to regenerate the tables and `python3 -m pytest tests/test_ch10_reference.py` to run the checks. The evidence covers a declared top-1 policy, literal routing examples, forward/backward equivalence, deterministic ties, padding exclusion, counter history, and a dense one-expert limit. It contains no genomic training result or hardware benchmark.

Conditional expert routing is not itself biological regulation. In Evolutor's taxonomy, Hermon is the Transformer-based direction and DOGMA the non-Transformer direction; this chapter supplies a software mechanism that must be specified and evaluated in any architecture that uses it. Part III, beginning with Chapter 11, asks which biological observations motivate which abstractions, and where each mapping fails.

Key ideas

- A capacity budget is not balanced demand. With demand $(3, 2, 1)$ and two slots per expert, one token is dropped while a slot stays empty.
- Renormalizing over a single selected expert makes the gate identically one and erases the router's task-loss gradient.
- Capacity counters are state: resetting them at a chunk boundary changes which tokens are admitted.
- Compare routed arithmetic with the original dense layer, not with an all-expert oracle. Here the routed version does more work.

10.9 Exercises with worked reasoning

1. **Count before running.** Demand is $(3, 2, 1)$ and $C = 2$. *Solution:* admit $(2, 2, 1)$, drop one, leave one slot unused.

2. **Overflow output.** What remains for a dropped token? *Solution:* the incoming residual u_t, not a deleted token or necessarily the block's original x_t.

3. **Find the gate.** A chosen expert returns $(2, -1)$ and its probability is 0.7. *Solution:* branch $(1.4, -0.7)$ before residual addition.

4. **Unselected logit.** Why can it receive a gradient? *Solution:* it changes the selected probability through the softmax denominator; it does not imply the unselected expert ran.

5. **Renormalization.** Differentiate p_k/p_k within a fixed top-1 choice. *Solution:* the quotient is identically one, so its derivative is zero.

6. **Rechunking.** Split the example after token 1 and reset counters. *Solution:* token 2 changes from dropped to admitted. Carry $(2, 0, 0)$ with fixed capacity to preserve this policy.

7. **Tie permutation.** Can arbitrary expert relabeling preserve the result at an exact tie? *Solution:* not with index-based tie-breaking unless expert identity and tie order are also preserved.

8. **Load certificate.** Uniform probabilities give loss α. Must hard demand be balanced? *Solution:* no; identical logits route every token to expert 0 under first-index ties.

9. **Dense limit.** State sufficient conditions for the original FFN result. *Solution:* one expert with matching weights, probability one, no overflow, matching valid inputs, attention and normalization, and no dropout.

10. **Arithmetic baseline.** Compare 72 + 320 MACs with the original dense FFN's 384. *Solution:* the routed count is eight larger; comparison with an all-expert oracle does not establish a dense-model speedup.

11. **Independent check.** Why is sparse/dense equality alone insufficient? *Solution:* both may share a wrong plan. Test explicit selection, demand, admitted IDs and capacity history independently.

12. **Evaluation design.** Does adding experts improve a genomic model? *Rubric:* specify held-out biological tasks, leakage controls, matched training and inference grouping, dropped-token rates, parameter and compute budgets, baselines, uncertainty, and measured hardware results. A working dispatch function answers none of these empirical questions by itself.

Selective glossary

Cohort: tokens sharing one capacity budget and admission order. **Demand:** requests before capacity rejection. **Admission:** permission to execute the selected expert. **Gate:** scalar weighting of an executed expert's output. **Dispatch:** movement or indexing of token representations into and out of expert batches.

Sources and further reading

Access checked 21 September 2026. Sections 2.1–2.2 of the Switch paper, including its routing, capacity and auxiliary-loss equations, were inspected in the JMLR PDF. The PyTorch 2.10 argmax description was inspected for tie behavior. The local CPU float64 reference reuses the Chapter 9 normalization and attention implementation, which is included in the reviewed source closure.

The deterministic policy, six-token traces, negative controls, gradient derivations, and cost comparisons are teaching artifacts. They do not reproduce Switch's distributed implementation, training experiments or performance claims.

PART III

From biology to formal computational hypotheses

Which biological mechanisms motivate which computational abstractions, and where does each mapping stop?

Part III moves from biology to typed software. It begins with real regulatory dynamics, where history determines abundance and some parameters cannot be identified from protein data alone. It then builds generated structure, evolutionary search, typed genomes, operational semantics, resource ledgers and intervention-based evidence. Each chapter ends with a contract that later chapters can test. The recurring lesson is that a biological name supplies a question, not an equivalence: a software gate is not a promoter, and a credit score is not a mechanism.

CHAPTERS IN THIS PART

11 **Regulation and expression across levels** Map real regulatory mechanisms to explicitly limited abstractions.

12 **Development and generated computational structure** Compile developmental descriptions into typed executable structure.

13 **Learning, structural adaptation and evolution** Separate transient state, parameter learning and structural selection.

14 **Typed genomic computation systems** Define typed genes, genomes, tensor contracts and effects.

15 **Operational semantics and expression traces** Specify deterministic transitions, traces, halting and replay.

16 **Expression complexity and resource semantics** Count work, storage, communication and scheduling resources honestly.

17 **Traces, credit and mechanistic evidence** Distinguish prediction, intervention and mechanistic credit.

CHAPTER 11

Regulation and expression across levels

In this chapter you will

- normalize promoter-state weights and distinguish active occupancy from fold change
- solve two-stage RNA/protein dynamics exactly over a constant-input interval
- show that history, not present context alone, determines abundance
- identify a structural non-identifiability in protein-only observations

Two cells can carry the same gene and encounter the same present signal, yet contain different amounts of its protein. A model that chooses one stored function from the current input cannot explain that difference unless it also represents history. This is the first useful boundary between biological regulation and Chapter 10's instantaneous expert routing.

We will connect a promoter-state model to RNA and protein dynamics, derive a reproducible update, and ask which parameters observations can identify. Then we will extract a software contract without claiming that a neural module is a molecule. All numbers are assigned teaching values, not measurements or an Evolutor biological-performance result.

11.1 Stored sequence is not executed function

DNA supplies a substrate and information within cellular machinery. Transcription requires machinery interacting with DNA; translation requires RNA, ribosomes and other resources. Existing RNA and protein can persist after production changes. Protein abundance is distinct from protein activity, which may depend on localization, partners or chemical state. These distinctions make *turning a gene on* ambiguous: does it mean promoter occupancy, RNA synthesis, RNA abundance, protein synthesis, protein abundance, or a downstream effect?

Our setting is a simplified bacterial promoter with one overlapping repressor site. Simultaneous polymerase/repressor occupancy is excluded. Garcia and Phillips analyze such a three-state simple-repression motif under quasi-equilibrium and promoter-occupancy assumptions [40]. We use that limited starting point, not their fitted parameters or experimental performance.

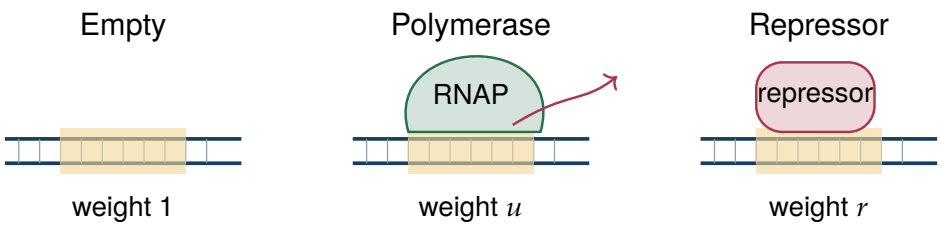


Figure 11.1: Empty, polymerase-bound and repressor-bound promoter states. The overlapping binding region explains exclusion in this model. DNA remains present in all states; only polymerase-bound occupancy is assigned transcriptional activity. Molecular silhouettes are schematic.

11.2 Normalize states before interpreting a gate

Assign dimensionless weights $1, u, r$ to the three states. These summarize effective availability and binding preference relative to an empty promoter. They are not probabilities until normalized:

$$Z = 1 + u + r, \qquad P_{\emptyset} = Z^{-1}, \quad P_{\mathrm{pol}} = u/Z, \quad P_{\mathrm{rep}} = r/Z. \tag{11.1}$$

If transcription is proportional to polymerase occupancy, the ratio with and without repressor is, for $u > 0$,

$$f(r) = \frac{u/(1+u+r)}{u/(1+u)} = \frac{1+u}{1+u+r}. \tag{11.2}$$

Only for $u \ll 1$ is $f(r) \approx 1/(1+r)$ a suitable approximation. At $u = 0$ both transcription rates vanish: the measured ratio is undefined, even though the simplified expression has a finite limit.

```
def occupancy(polymerase, repressor):
    """Weights relative to empty promoter; mutually exclusive binding."""
    finite_nonnegative(polymerase, repressor)
    # Scaling avoids overflow when summing large but finite weights.
    scale = max(1.0, polymerase, repressor)
    weights = [1.0 / scale, polymerase / scale, repressor / scale]
    total = sum(weights)
    return tuple(w / total for w in weights)
```

r	Active probability	Fold change
0	0.16667	1.00000
0.2	0.14286	0.85714
1	0.09091	0.54545
4	0.03846	0.23077
10	0.01786	0.10714

At $u = 0.2, r = 1$, active probability is $1/11$ and fold change is $6/11$. Confusing these quantities changes the interpretation of an experiment. The code scales weights before summation to avoid overflow when two finite weights are very large.

Unlike a generic softmax, the alternatives here are specified physical configurations. But many possible states and processes have been omitted. A cooperative mechanism needs its allowed configurations and weights; raising r to an arbitrary power does not itself derive cooperativity. A fitted response curve need not uniquely identify a mechanism.

11.3 Track production, retention and removal

Let $m(t), p(t)$ be coarse-grained RNA and protein concentrations in assigned nM units. Time is minutes, RNA production $s(t)$ is nM/min, translation k is min^{-1}, and effective loss rates $\gamma_m, \gamma_p > 0$ are min^{-1}. With fixed volume and first-order removal, define

$$\dot{m} = s(t) - \gamma_m m, \tag{11.3}$$

$$\dot{p} = km - \gamma_p p. \tag{11.4}$$

Translation does not consume RNA in this model. Each RNA can support repeated production; degradation and dilution enter separately through effective loss. This

mean-concentration model omits discrete fluctuations and does not describe each individual cell.

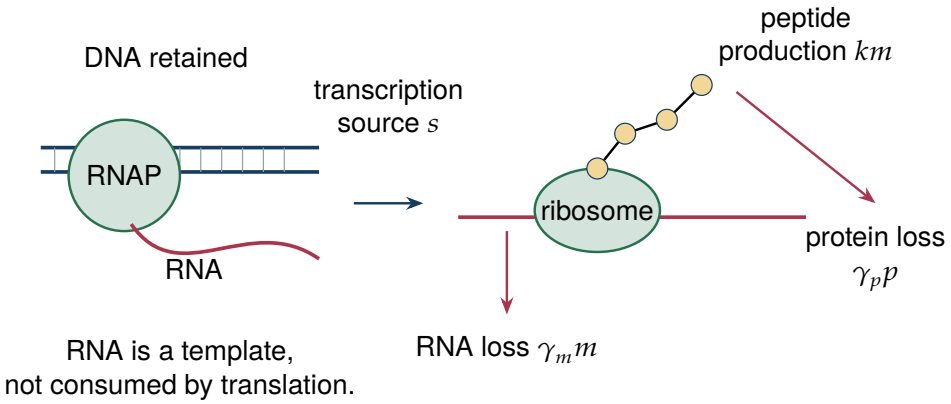


Figure 11.2: The objects behind the two-stage model. Polymerase uses DNA to synthesize RNA; a ribosome uses RNA while a peptide grows. Loss removes RNA and protein from their respective pools. This coarse bacterial mechanism omits detailed resources and regulatory factors.

Introduce α in nM/min and a competent-promoter fraction $a(t) \in [0,1]$, then set

$$s(t) = \alpha a(t) \frac{u(t)}{1 + u(t) + r(t)}. \tag{11.5}$$

Occupancy is conditioned on the competent subpopulation. The product treats a as separately supplied; it is not a derivation of chromatin accessibility or a universal bacterial rule. Gene copy number and fixed-volume conversion are absorbed into α. Altering them changes the model.

11.4 Solve a constant-input interval

Hold all coefficients fixed for duration Δ, and set $m_* = s/\gamma_m$. Multiplying the RNA equation by $e^{\gamma_m t}$ and integrating yields

$$m(\Delta) = m_* + (m_0 - m_*)e^{-\gamma_m \Delta}. \tag{11.6}$$

For protein, use the integrating factor $e^{\gamma_p t}$:

$$p(\Delta) = p_0 e^{-\gamma_p \Delta} + k \int_0^\Delta e^{-\gamma_p(\Delta-\tau)} m(\tau)\, d\tau \tag{11.7}$$

$$= p_0 e^{-\gamma_p \Delta} + k \left[m_* \frac{1 - e^{-\gamma_p \Delta}}{\gamma_p} + (m_0 - m_*) J(\Delta) \right], \tag{11.8}$$

where

$$J(\Delta) = \begin{cases} \dfrac{e^{-\gamma_m \Delta} - e^{-\gamma_p \Delta}}{\gamma_p - \gamma_m}, & \gamma_p \neq \gamma_m, \\ \Delta e^{-\gamma_m \Delta}, & \gamma_p = \gamma_m. \end{cases} \tag{11.9}$$

At equal rates, the integrand's τ dependence cancels. The apparent singularity is removable, not a physical divergence.

For $g = |\gamma_p - \gamma_m| > 0$, rewrite $J = e^{-\min(\gamma_m,\gamma_p)\Delta}(1 - e^{-g\Delta})/g$. This avoids subtracting nearly equal exponentials and avoids a large positive exponential. The implementation uses `expm1` for $1 - e^{-x}$. An exact mathematical formula still needs floating-point tests.

```
def expression_step(
    state, production, translation, rna_loss, protein_loss, dt
):
    """Exact constant-input two-stage linear dynamics, positive loss rates."""
    finite_nonnegative(
        state.rna,
        state.protein,
        production,
        translation,
        rna_loss,
        protein_loss,
        dt,
    )
    if rna_loss == 0 or protein_loss == 0:
        raise ValueError("strictly positive loss rates required")
    m_star = production / rna_loss
    em, ep = math.exp(-rna_loss * dt), math.exp(-protein_loss * dt)
    gap = abs(protein_loss - rna_loss)
    if gap == 0:
        mixed = dt * em
    else:
        mixed = (
            math.exp(-min(rna_loss, protein_loss) * dt)
            * -math.expm1(-gap * dt)
            / gap
        )
    integral = -math.expm1(-protein_loss * dt) / protein_loss
    m = m_star + (state.rna - m_star) * em
    p = state.protein * ep + translation * (
        m_star * integral + (state.rna - m_star) * mixed
    )
    return State(m, p)
```

The immutable `State` stores RNA/protein concentrations; immutable `Definition` stores four rate parameters. Both live in `reference.py`. The context wrapper is:

```
def regulated_step(
    definition, state, polymerase, repressor, accessible, dt
):
    """No hard selector: accessibility scales conditional promoter activity."""
    finite_nonnegative(accessible, definition.max_transcription)
    if accessible > 1:
        raise ValueError("accessibility must lie in [0, 1]")
    bound = occupancy(polymerase, repressor)[1]
    source = definition.max_transcription * accessible * bound
    return expression_step(
        state,
        source,
        definition.translation,
        definition.rna_loss,
        definition.protein_loss,
        dt,
    )
```

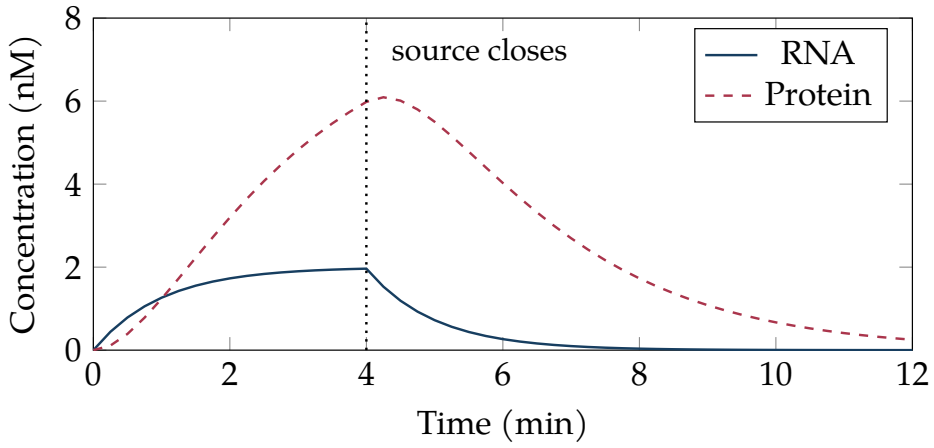
There is no winner selection or capacity budget here. The wrapper updates a continuous state. Multiple intervals equal one long interval only with unchanged input and parameters,

additive durations, and carried state. Tests verify this semigroup property and compare with an independent small-step fourth-order Runge–Kutta integration.

11.5 A stopped source can leave a rising output

Take $\alpha = 12, u = 0.2, r = 0, a = 1, k = 2, \gamma_m = 1, \gamma_p = 0.5$, so $s = 2$. Start at zero and set $a = 0$ after four minutes, retaining the RNA/protein state. The reference uses exact quarter-minute updates:

```
def simulate(definition, intervals, initial=State()):
    """Each interval is (duration, polymerase weight, repressor weight,
     ↪  access)."""
    time, state = 0.0, initial
    records = [dict(time=time, rna=state.rna, protein=state.protein)]
    for dt, u, r, access in intervals:
        state = regulated_step(definition, state, u, r, access, dt)
        time += dt
        records.append(
            dict(time=time, rna=state.rna, protein=state.protein)
        )
    return records
```



Assigned mean-dynamics model; not single-cell measurements

Figure 11.3: RNA and protein under an assigned four-minute production pulse. RNA falls after closure, but protein briefly continues rising because surviving RNA still drives translation. Solid and dashed lines encode the two quantities independently of color.

At four minutes, $m \approx 1.96337, p \approx 5.98116$. Immediately after closure, $\dot{m} = -1.96337$ nM/min, yet $\dot{p} = 2(1.96337) - 0.5(5.98116) \approx 0.93616$ nM/min. Rising protein does not establish current transcriptional activity.

Setting derivatives to zero for constant source gives $m_* = s/\gamma_m$ and $p_* = ks/(\gamma_m\gamma_p)$:

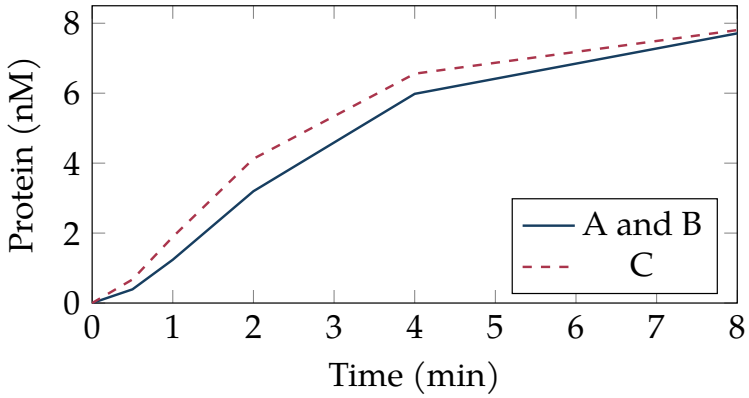
```
def steady_state(production, translation, rna_loss, protein_loss):
    finite_nonnegative(production, translation, rna_loss, protein_loss)
    if min(rna_loss, protein_loss) <= 0:
        raise ValueError("positive loss rates required")
    m = production / rna_loss
    return State(m, translation * m / protein_loss)
```

The assigned steady state is $(2, 8)$ nM. Closing its source for one minute gives approximately $(0.73576, 6.76146)$, whereas zero initial state under the identical closed context stays zero. Context alone cannot determine abundance. Production history is part of the computational state.

11.6 A perfect output trajectory can remain ambiguous

Consider three parameter sets, all with $\gamma_p = 0.5$: A has $(s, k, \gamma_m) = (2, 2, 1)$; B has $(1, 4, 1)$; C has $(4, 2, 2)$. All have $p_* = 8$. A and C approach it differently, so a transient can distinguish them. But A and B have identical protein trajectories from zero despite different RNA trajectories.

Time	m_A	m_B	$p_A = p_B$	p_C
0	0.0000	0.0000	0.0000	0.0000
0.5	0.7869	0.3935	0.3914	0.6738
1	1.2642	0.6321	1.2385	1.8912
2	1.7293	0.8647	3.1966	4.1248
4	1.9634	0.9817	5.9812	6.5573
8	1.9993	0.9997	7.7096	7.8046



All reach $p_* = 8$.
A and B: same curve, different RNA.
C: faster RNA response.

Figure 11.4: A and C share a steady protein value but differ during the transient. A and B coincide throughout: their transcription–translation product and loss rates match. RNA observations can separate A from B.

For any $c > 0$, replace s by s/c, k by ck, and m_0 by m_0/c. The RNA solution is divided by c, preserving $km(t)$. With unchanged p_0, the entire protein solution is unchanged. More precise protein measurements cannot remove this structural ambiguity. RNA measurements, an independent translation constraint, or an intervention that breaks the symmetry are needed.

Low protein therefore does not uniquely establish transcriptional repression. RNA turnover, translation, protein turnover and history offer alternatives. A genomic predictor should name its output and the evidence that distinguishes these explanations. Predictive association and mechanistic intervention are different tests.

11.7 Extract a typed software hypothesis

Separate a stored definition, context, eligibility, evolving state and executed effect. A definition is not a state; eligibility need not guarantee admission under a compute budget; abundance need not equal activity; an output is not the specification. One variable called a gene cannot safely stand for all five.

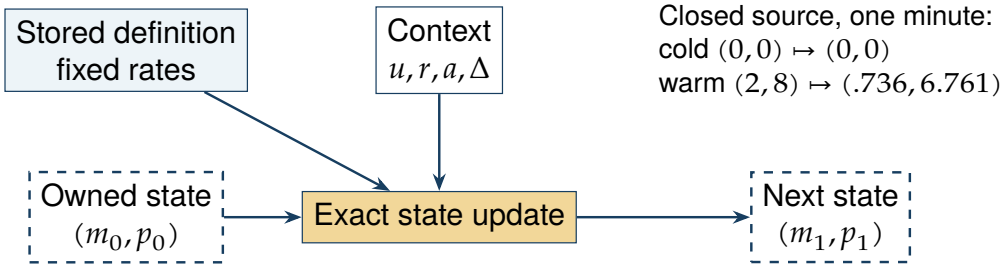


Figure 11.5: Equal present context with different initial state produces different next state. Solid arrows are update dependencies. Dashed boundaries mark state owned by one simulated system, not global state shared across unrelated records.

Chapter 10's gate weights a current-token expert branch under cohort and capacity rules. This chapter retains material/state after the source closes. Naming both mechanisms regulation does not make their semantics equivalent. A regulation-inspired software experiment must specify update time, reset boundaries, parameter ownership, observations and interventions. Accidental state carryover between unrelated genomic records is leakage, not useful memory.

These are research hypotheses above either Evolutor direction: Hermon DNA remains Transformer-based and DOGMA remains non-Transformer. The scalar model establishes no biological capability for either. A useful next study would compare stateful and stateless predictors with matched data and compute budgets, held-out interventions and explicit reset ablations. Chapter 12 asks a different question: how can a compact specification generate a larger computational structure before execution?

Key ideas

- At $u = 0.2$, $r = 1$ the active probability is $1/11$ while the fold change is $6/11$. These are different quantities.
- Protein can keep rising after transcription stops, because surviving RNA continues to be translated.
- Scaling $s \to s/c$ and $k \to ck$ leaves every protein trajectory unchanged. Better protein data cannot separate these parameter sets; RNA data or an intervention can.
- Expert routing (Chapter 10) is instantaneous selection. Regulation here is stateful production with turnover. Calling both "regulation" does not make them the same.

11.8 Exercises with worked reasoning

1. **Normalization.** Weights $(1, .2, 1)$ give which active probability? *Solution:* $.2/2.2 = 1/11$.
2. **Fold change.** Is $1/11$ the fold change? *Solution:* no; divide by unrepressed $1/6$ to obtain $6/11$.
3. **Zero polymerase.** Interpret $u = 0$. *Solution:* no active occupancy; the ratio of zero rates is undefined.
4. **Units.** Check km. *Solution:* $\text{min}^{-1} \cdot \text{nM} = \text{nM/min}$.
5. **Equal rates.** With both loss rates one, $s = 2, k = 3$ and zero state, derive $p(t)$. *Solution:* $6[1 - (1 + t)e^{-t}]$.
6. **Steady state.** Find the pulse's active-source equilibrium. *Solution:* $(2, 8)$ nM.
7. **Delayed peak.** Why can protein rise after source closure? *Solution:* km can still exceed $\gamma_p p$.
8. **Memory.** Compare cold and warm states one minute after closure. *Solution:* $(0, 0)$ versus approximately $(.73576, 6.76146)$.
9. **Ambiguity.** Can protein-only time series distinguish A and B? *Solution:* no; the scaling symmetry preserves all those observations.
10. **Chunking.** When does two-step propagation equal one step? *Solution:* same held coefficients, additive durations and carried state.
11. **Independent check.** Why also compare against RK4? *Solution:* the printed formula and its implementation might share a mistake.
12. **Study design.** Test a regulation-inspired predictor. *Rubric:* name quantities, reset boundaries, matched baselines, intervention splits, uncertainty and competing explanations.

Selective glossary

Occupancy: probability of a molecular configuration. **Expression:** a process whose measured level must be specified. **Turnover:** effective removal from a modeled pool. **Identifiability:** distinct parameters imply distinguishable observations. **Context:** supplied update conditions, distinct from carried state.

Sources and further reading

Access checked 24 September 2026. Garcia and Phillips, pp. 12173–12175, introduction and Theory and Experimental Design through equations 2–5, were inspected for the simple-repression states and assumptions. The supplementary derivation and full experimental dataset were not inspected or reproduced. Dimensionless weights, rates and trajectories here are assigned. The exact mean-dynamics solver and identifiability examples are independent teaching constructions, not fits to those experiments.

CHAPTER 12

Development and generated computational structure

In this chapter you will

- separate a stored specification, its construction procedure, the generated graph and its execution
- implement simultaneous rewriting and derive its exponential growth
- compile a recursive residual module into a typed, topologically ordered program
- show that shared parameters accumulate gradients from every occurrence

A stored specification says: apply a construction rule twice, use width two, and share the affine parameters. How many actual operations exist? How many parameter arrays exist? Are those the same question?

They are not. A compact description can expand into a large graph, and many operation occurrences can refer to one parameter owner. This chapter builds that distinction into an executable compiler and checks both forward values and gradients against a separate recursive interpreter.

Chapter 11 modeled changing amounts within a fixed mechanism. Here the construction process changes the number and arrangement of computational operations before numerical execution. The biological term development is a motivation for asking about generated structure, not a claim that the compiler reproduces embryogenesis or that its parameters are genes.

12.1 Three maps, not one biological metaphor

Write a stored specification as g, construction context as c, and construction procedure as $\mathcal{D}$. It produces a typed graph $G = \mathcal{D}(g, c)$. Parameter initialization assigns tensors θ to owners in G, and execution computes

$$y = \mathrm{Run}(G, \theta, x). \tag{12.1}$$

Changing g, changing θ, changing carried state, and changing x are different interventions. The first may alter graph structure; the second changes numerical behavior inside a fixed graph; the latter two can change a result without changing either structure or learned parameters.

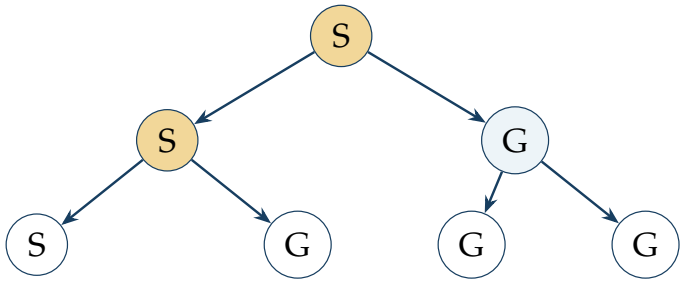
Calling g a genotype and G a phenotype is therefore only a restricted analogy. Biological development includes chemistry, spatial organization, mechanics, environmental interactions and history not present in this map. For software, the value of the analogy is a testable separation between a stored description and a realized structure.

Parallel rewriting provides one precise example of such a separation. Prusinkiewicz and Lindenmayer introduce simultaneous replacement of symbols in L-systems and distinguish it from sequential rewriting [41]. We use a small original teaching rule set rather than reproducing their plant models or figures.

12.2 Make a construction step atomic

Take alphabet $\{S, G\}$, initial word S, and rules $S \mapsto SG$, $G \mapsto GG$. In one generation, replace every symbol in the *old* word and concatenate the replacements in order.

Generation	Word
0	S
1	SG
2	SGGG
3	SGGGGGGG



$S \mapsto SG$
$G \mapsto GG$

Read one row;
construct the next.

Abstract symbols and provenance, not literal cell lineage

Figure 12.1: Parent-to-child provenance under simultaneous rewriting. Each old symbol produces two new symbols. All parents in one row are read before the next row exists. These are abstract construction states, not cell types or an experimentally observed lineage.

```
def rewrite(word, rules, generations, limit=1024):
    """Simultaneous, not in-place, symbol replacement."""
    if type(generations) is not int or generations < 0:
        raise ValueError("nonnegative generation count required")
    if type(limit) is not int or limit < 1 or len(word) > limit:
        raise ValueError(
            "positive budget covering the initial word required"
        )
    history = [word]
    for _ in range(generations):
        if any(s not in rules for s in word):
            raise ValueError("missing production rule")
        if sum(len(rules[s]) for s in word) > limit:
            raise ValueError("rewrite budget exceeded before allocation")
        word = "".join(rules[s] for s in word)
        history.append(word)
    return history
```

The function makes the timing convention explicit by creating a new string. An in-place loop that immediately rewrites newly inserted symbols would not be one generation of this system. The size guard checks the total replacement length before allocating the next word.

Let s_d, g_d be symbol counts after generation d. The rules imply

$$\begin{bmatrix} s_{d+1} \\ g_{d+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} s_d \\ g_d \end{bmatrix}, \qquad (s_0, g_0) = (1, 0). \tag{12.2}$$

Thus $s_d = 1$ and $g_{d+1} = 1 + 2g_d$, giving $g_d = 2^d - 1$ by induction. Total word length is 2^d. A short rule does not imply a small realization. Our budget counts symbols, not biological cells, bytes of a full model, or elapsed time.

The string example has syntax but not yet numerical semantics. Interpreting each symbol as a neural layer without defining inputs, outputs and ownership would leave the central question unanswered. We next specify a different recursive construction whose meaning is fully executable.

12.3 Define the graph before writing the compiler

Inputs are row-vector batches $x \in \mathbb{R}^{B\times D}$. An affine-activation occurrence is

$$H_\theta(x) = \tanh(xW + b), \qquad W \in \mathbb{R}^{D\times D}, \quad b \in \mathbb{R}^D. \tag{12.3}$$

The bias broadcasts over the batch axis only. Define a depth-d module recursively, with separate left and right occurrences:

$$R_{0,q}(x) = H_{\theta_q}(x), \tag{12.4}$$

$$R_{d,q}(x) = x + R_{d-1,qR}(R_{d-1,qL}(x)), \qquad d > 0. \tag{12.5}$$

The path q records construction provenance, not a tensor index. Left execution precedes right execution because the right child consumes the left child's result. The parent residual adds its own incoming x, not an arbitrary intermediate.

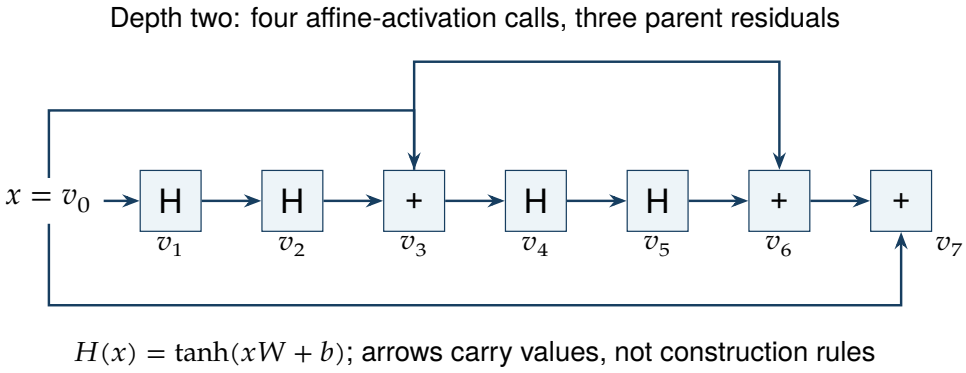


Figure 12.2: The depth-two graph with values numbered in execution order. Four affine-activation occurrences and three residual additions implement the recursive definition. Skip edges retain the correct parent input. Reusing a definition does not remove any shown evaluation.

There are two parameter policies. Under *shared* ownership, all leaves refer to the same (W, b). Under *independent* ownership, each leaf path owns distinct tensors of the same shape. Identical initial tensor values do not make independent owners into shared owners.

The compact specification consists of depth, width and ownership policy. Construction is deterministic and occurs outside differentiation. No architecture search has taken place: choosing one valid graph is not searching for a graph that performs well.

12.4 Emit a typed, topologically ordered program

Each emitted Node has a fresh integer ID, an operation, input IDs, an output width, and an optional parameter-owner key. Value zero denotes the external input. The node sequence is an

SSA-like representation: each ID is assigned once, and every input must already exist. The immutable Node data class is defined alongside the reference.

```
def compile_program(depth, width, shared=True, budget=1023):
    """Expand R_d(x)=x+R_(d-1),right(R_(d-1),left(x))."""
    if type(depth) is not int or not 0 <= depth <= 20:
        raise ValueError("depth must be an integer from 0 through 20")
    if type(width) is not int or width < 1 or type(shared) is not bool:
        raise ValueError(
            "positive integer width and boolean sharing required"
        )
    if type(budget) is not int or budget < 2 ** (depth + 1) - 1:
        raise ValueError("insufficient expansion budget")
    nodes = []

    def emit(level, source, path):
        if level == 0:
            key = "shared" if shared else path
            node = Node(len(nodes) + 1, "affine", (source,), width, key)
        else:
            left = emit(level - 1, source, path + "L")
            right = emit(level - 1, left, path + "R")
            node = Node(len(nodes) + 1, "add", (source, right), width)
        nodes.append(node)
        return node.id

    output = emit(depth, 0, "root")
    validate(nodes, width)
    return tuple(nodes), output
```

The nested emitter returns an output ID, not a computed tensor. At a leaf it creates an affine occurrence; otherwise it first emits the left subtree, then the right subtree consuming the left output, then a residual addition with the original source. Construction path labels such as `rootLR` keep distinct occurrences identifiable.

ID	Operation	Input IDs	Owner
1	affine	0	shared
2	affine	1	shared
3	add	0,2	–
4	affine	3	shared
5	affine	4	shared
6	add	3,5	–
7	add	0,6	–

For depth two and width two, the table contains four affine calls and three additions. Nodes 1, 2, 4 and 5 share an owner in the displayed configuration. Node 6 adds values 3 and 5, while node 7 adds values 0 and 6. Confusing those residual sources would still produce shape-correct output, but the wrong function.

12.5 Reject illegal structure before running tensors

The validator maintains known value widths and parameter-owner shapes. Every affine node has one already-defined input of the same width; every add has two already-defined inputs matching its output width. A shared owner must have one consistent width.

```python
def validate(nodes, input_width):
    """SSA-like topological order; node 0 is the external input."""
    if type(input_width) is not int or input_width < 1:
        raise ValueError("positive integer input width required")
    widths, owners = {0: input_width}, {}
    for node in nodes:
        if type(node.id) is not int or node.id < 1 or node.id in widths:
            raise ValueError("positive fresh node ID required")
        if type(node.width) is not int or node.width < 1:
            raise ValueError("positive integer node width required")
        arity = {"affine": 1, "add": 2}.get(node.op)
        if arity is None or len(node.inputs) != arity:
            raise ValueError("unknown operation or wrong arity")
        if any(i not in widths for i in node.inputs):
            raise ValueError(
                "inputs must already exist; forward edges forbidden"
            )
        if any(widths[i] != node.width for i in node.inputs):
            raise ValueError("square affine/residual widths must agree")
        if node.op == "affine":
            if not node.parameter:
                raise ValueError("affine parameter owner required")
            if owners.setdefault(node.parameter, node.width) != node.width:
                raise ValueError("shared owner has inconsistent shape")
        elif node.parameter:
            raise ValueError("add must not own affine parameters")
        widths[node.id] = node.width
    return owners
```

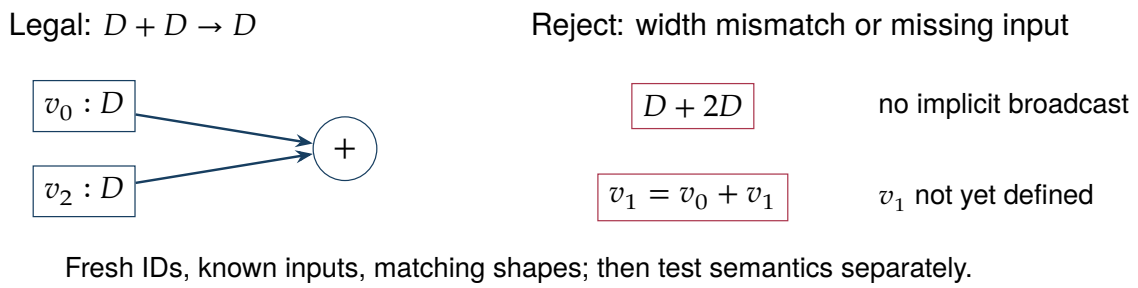


Figure 12.3: Structural contracts distinguish legal residual addition from a width mismatch and a forward/self reference. The reference requires topological order, so it rejects forward edges even if a different ordering could make a graph acyclic. Type validity does not establish the desired numerical semantics or task quality.

These checks prevent accidental broadcasting and dangling dependencies. They do not prove a compiler emitted the intended graph. For example, adding the wrong width-D value can pass all type checks. That is why we separately test graph execution against an interpreter of the mathematical recursion.

Biological feedback is not represented by inserting a cycle into this same-step DAG. A feedback model needs state, timing and an update rule: for example, a previous-time value as an explicit external input. Finite unrolling then produces an acyclic execution graph whose depth means time steps. Our construction depth means nested structure instead. Equal-shaped diagrams can hide different time semantics.

12.6 Run the graph and preserve gradient ownership

Execution stores a tensor for each value ID. An affine occurrence fetches its owner's tensors; an addition fetches its input values. The runtime checks actual parameter shapes and requires the owner set to match exactly.

```
def execute(nodes, output, x, parameters):
    """Row-vector tensors [batch, width]; retain autograd through all uses."""
    if x.ndim != 2:
        raise ValueError("rank-two input required")
    owners = validate(nodes, x.shape[1])
    if set(parameters) != set(owners):
        raise ValueError("parameter owners must exactly match the graph")
    for key, width in owners.items():
        w, b = parameters[key]
        if w.shape != (width, width) or b.shape != (width,):
            raise ValueError("incorrect parameter shape")
    values = {0: x}
    for node in nodes:
        if node.op == "affine":
            w, b = parameters[node.parameter]
            values[node.id] = torch.tanh(values[node.inputs[0]] @ w + b)
        else:
            a, b = node.inputs
            values[node.id] = values[a] + values[b]
    if output not in values:
        raise ValueError("unknown output")
    return values[output]
```

The simple dictionary intentionally retains intermediate values. It is not an optimized allocator, fused kernel, or memory-efficient training runtime. CPU float64 tests isolate semantic mistakes before systems work.

The separate interpreter follows the recursion without inspecting the compiled node list:

```
def recursive_oracle(depth, x, parameters, shared=True, path="root"):
    """Independent tree interpreter; does not read the compiled node list."""
    if depth == 0:
        w, b = parameters["shared" if shared else path]
        return torch.tanh(x @ w + b)
    left = recursive_oracle(depth - 1, x, parameters, shared, path + "L")
    right = recursive_oracle(
        depth - 1, left, parameters, shared, path + "R"
    )
    return x + right
```

Tests compare values, input gradients and every parameter gradient at depths zero through three under both ownership policies. A finite-difference check adds another independent gradient diagnostic. An order-swap negative test uses distinct leaf parameters so that exchanging left and right cannot pass by symmetry.

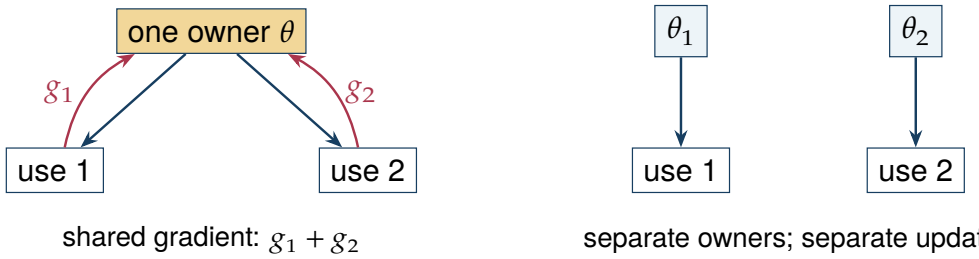


Figure 12.4: Two operation occurrences can read one parameter owner or two independent owners. Shared backward contributions add into the same gradient. Drawing one stored tensor does not mean only one forward call; drawing two equal tensors does not make them one owner.

Let an expanded graph have leaf parameters $\theta_1, \dots, \theta_L$. For a tied model set $\theta_i = \theta$ for every i. The chain rule gives

$$\frac{d\mathcal{L}_{\text{tied}}}{d\theta} = \sum_{i=1}^{L} \left.\frac{\partial \mathcal{L}}{\partial \theta_i}\right|_{\theta_1=\cdots=\theta_L=\theta}. \tag{12.6}$$

Each partial derivative includes all downstream dependencies from that occurrence. It is not merely a local derivative with the rest of the graph discarded. Our test clones one owner's initial values into independent owners, verifies equal forward results, then sums the independent gradients and compares with the tied gradient. A deliberate comparison with just one occurrence's gradient fails, catching overwrite instead of accumulation.

After an optimizer update, the two ownership policies generally diverge: the shared parameter receives the sum, while each independent parameter receives its own contribution. Thus shared and initially equal independent models are different training systems even if their first outputs match.

12.7 Count the expanded structure, not the description

Let L_d count affine calls and A_d additions. The construction gives $L_0 = 1, A_0 = 0$ and $L_d = 2L_{d-1}, A_d = 2A_{d-1} + 1$. Hence

$$L_d = 2^d, \qquad A_d = 2^d - 1, \qquad N_d = 2^{d+1} - 1. \tag{12.7}$$

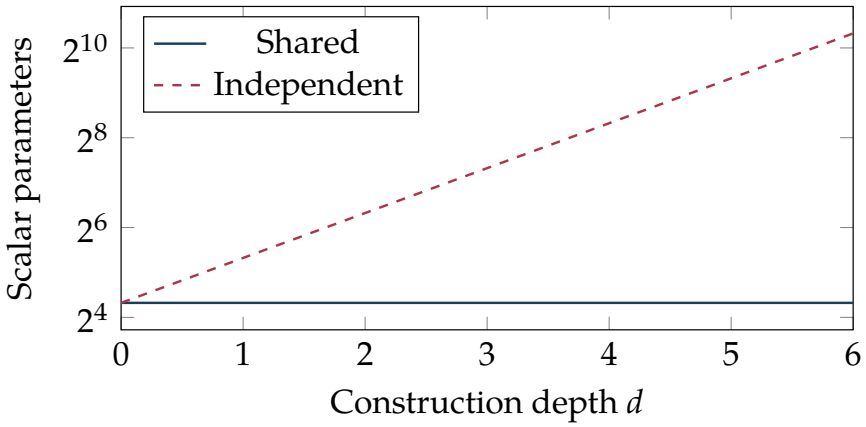
The compiler checks N_d against a budget before recursion. This prevents a tiny request from silently expanding into an enormous program. It also restricts the reference's admissible depth explicitly.

The number of scalar parameters is $D^2 + D$ under sharing and $2^d(D^2 + D)$ without sharing. In either case, affine matrix work per input row is $2^d D^2$ multiply-accumulates, excluding bias additions, nonlinearities and residual operations.

```
def costs(depth, width, shared):
    nodes, _ = compile_program(
        depth, width, shared, budget=2 ** (depth + 1) - 1
    )
    leaves = sum(n.op == "affine" for n in nodes)
    owners = len(validate(nodes, width))
    return dict(
        depth=depth,
        affine_calls=leaves,
        adds=leaves - 1,
        nodes=len(nodes),
```

```
        owners=owners,
        parameters=owners * (width**2 + width),
        matrix_macs_per_row=leaves * width**2,
    )
```

Depth	Calls	Nodes	Shared params	Separate params
0	1	1	20	20
1	2	3	20	40
2	4	7	20	80
3	8	15	20	160
4	16	31	20	320
5	32	63	20	640
6	64	127	20	1280



Width $D = 4$; both policies execute 2^d affine calls per row

Figure 12.5: At width four, sharing keeps scalar parameter count at 20 while independent ownership doubles it with each depth increment. Both graphs still execute 2^d affine calls per row. These counts are properties of the reference, not measured speed or memory benchmarks.

At depth six, the graph has 127 nodes and 64 affine calls. It stores 20 shared parameters or 1280 independent parameters, but both execute 1024 matrix multiply-accumulates per row. A compact specification or small parameter bank does not eliminate expanded computation.

Could common-subexpression elimination remove repeated shared calls? Not merely because they share W, b: each call generally receives a different input value. Memoization needs equal operation *and* equal relevant inputs/state, with compatible side-effect semantics. Likewise, depth reduction requires a proof of functional equivalence, not a diagram with repeated labels.

The deterministic initializer gives reproducible owner-key ordering and a local random generator:

```
def parameter_bank(nodes, width, seed=17):
    owners = validate(nodes, width)
    generator = torch.Generator().manual_seed(seed)
    return {
        key: (
```

```
            torch.randn(
                width, width, generator=generator, dtype=torch.float64
            )
            .mul(0.2)
            .requires_grad_(),
            torch.randn(width, generator=generator, dtype=torch.float64)
            .mul(0.1)
            .requires_grad_(),
        )
        for key in sorted(owners)
    }
```

An exact restart in a trained extension would also need optimizer state, data order, random-generator state, and the compiled specification. Rebuilding a different sharing policy from the same numerical arrays would not reproduce the original optimization process.

12.8 Separate construction correctness from empirical discovery

We have proved bounds, rejected malformed programs, and compared execution with a recursive oracle. None of those tests establishes that the generated architecture is useful for genomic prediction. A search method would need a space of specifications, proposal or mutation rules, an evaluation budget, selection criteria and held-out assessment. Training a chosen graph and searching over graphs are separate sources of cost and bias.

To compare direct and generated encodings fairly, record description size, expanded graph size, unique parameters, active computation, search trials, training budget and evaluation leakage controls. A compact encoding may restrict the search space beneficially or exclude the needed architecture. Either outcome must be measured on a declared task.

In Evolutor's taxonomy, this compiler is a general structural reference, not DOGMA Engine, Hermon DNA Engine, or a demonstrated genomic controller. Biological resemblance provides possible design questions; explicit semantics and independent task evidence determine what a resulting system actually does. The next conceptual step is to describe typed modules and interactions at larger scope without conflating structural generation, dynamic regulation and numerical learning.

Key ideas

- A short rule need not give a small realization: $S \mapsto SG$, $G \mapsto GG$ doubles the word every generation.
- Type validity is necessary but not sufficient. Adding the wrong width-D value passes every check and computes the wrong function.
- Sharing reduces stored parameters, not executed work. At depth six both policies run 1024 MACs per row.
- Initially equal independent owners and one shared owner are different training systems once the first update is applied.

12.9 Exercises with worked reasoning

1. **Parallel step.** Rewrite SG once. *Solution:* SGGG, reading both old symbols before replacing either.
2. **Growth.** Prove $g_d = 2^d - 1$. *Solution:* base $g_0 = 0$; $1 + 2(2^d - 1) = 2^{d+1} - 1$.
3. **Budget.** Can a seven-symbol limit reach generation three? *Solution:* no; the next word needs eight symbols and is rejected.
4. **Shape.** Give the affine input, weight and bias shapes. *Solution:* $B \times D$, $D \times D$, and D, respectively.
5. **Residual source.** At depth two, why does node 6 add node 3? *Solution:* node 3 is the input to the right depth-one module.
6. **Owner count.** At depth three, compare policies. *Solution:* eight affine occurrences; one shared owner or eight separate owners.
7. **Gradient.** Why is one occurrence's gradient insufficient? *Solution:* a tied parameter influences every occurrence; chain-rule contributions sum over all uses.
8. **Equivalence.** Do initially equal independent weights stay equal? *Solution:* generally no; their per-occurrence gradients differ.
9. **Counts.** At depth six and width four, report work and parameters. *Solution:* 64 calls, 1024 matrix MACs per row, 20 shared or 1280 independent scalar parameters.
10. **Caching.** Why not cache every call with the same owner key? *Solution:* its input tensors generally differ; ownership is not value identity.
11. **Feedback.** How could delayed feedback enter an acyclic program? *Solution:* expose previous-time state as input and specify update order; unroll time explicitly instead of inserting an unexplained same-step cycle.
12. **Evaluation.** Compare generated versus direct architectures. *Rubric:* control search and training budgets, graph/parameter/compute size, initialization, task splits and uncertainty; include a counterexample showing a useful architecture excluded by the chosen grammar.

Selective glossary

Construction context: values supplied while generating structure. **Occurrence:** one operation use in an expanded graph. **Owner:** identity of stored parameter tensors shared by uses.

Topological order: inputs precede their consumers. **Structural validity:** satisfaction of graph/type rules, distinct from semantic correctness and predictive usefulness.

Sources and further reading

Access checked 24 September 2026. The author-hosted electronic Chapter 1 of *The Algorithmic Beauty of Plants*, printed pp. 1–5, was downloaded and read for rewriting and D0L-system timing semantics. Later plant models and the remainder of the book were not inspected for this chapter. The S/G example, residual construction, compiler, ownership tests and figures are original teaching artifacts. No plant artwork, architecture-search results or biological developmental predictions are reproduced.

CHAPTER 13

Learning, structural adaptation and evolution

In this chapter you will

- distinguish state updates, parameter learning, structural proposals and population selection
- derive closed-form oracles for gradient descent on a two-feature problem
- run a deterministic elitist mask search with an explicit lineage
- state an inheritance map and account for validation reuse during search

A model can change its answer without changing its weights. It can change its weights without changing its architecture. It can change its architecture without becoming a population undergoing selection. Calling all three processes *adaptation* is acceptable in conversation, but inadequate for implementing an experiment.

We will build a small structural search whose task is deliberately transparent. Two observed features agree during training. Only one remains predictive when that agreement is broken. The learner fits the training set almost perfectly using either feature. Can a search over feature masks identify the robust representation? Which data choose the winner, and which pieces of a parent's implementation does a child inherit?

Prerequisites are gradients and evaluation from Chapters 2–4, transient state from Chapter 6, and generated graphs from Chapter 12. The example is a deterministic two-feature regression problem, not evidence of genomic capability or a general architecture-search advantage.

13.1 Four objects and four update contracts

Write a candidate as (G, θ), where G is a computational structure and θ its trainable parameters. Let h_t denote transient state during an input sequence and $\mathcal{P}_g$ a population at generation g. A useful separation is

$$h_{t+1} = f_{G,\theta}(h_t, x_t), \qquad \text{state update,} \tag{13.1}$$

$$\theta_{k+1} = U_G(\theta_k, D_{\text{train}}), \qquad \text{parameter learning,} \tag{13.2}$$

$$G' = M(G, \xi), \qquad \text{structural proposal,} \tag{13.3}$$

$$\mathcal{P}_{g+1} = \mathcal{S}(\mathcal{P}_g, C_g, E), \qquad \text{selection.} \tag{13.4}$$

Here t is an input step, k an optimization step, ξ a proposal choice, C_g candidate offspring and E an evaluation procedure. These indices name causal boundaries; they need not correspond to universally separated wall-clock times.

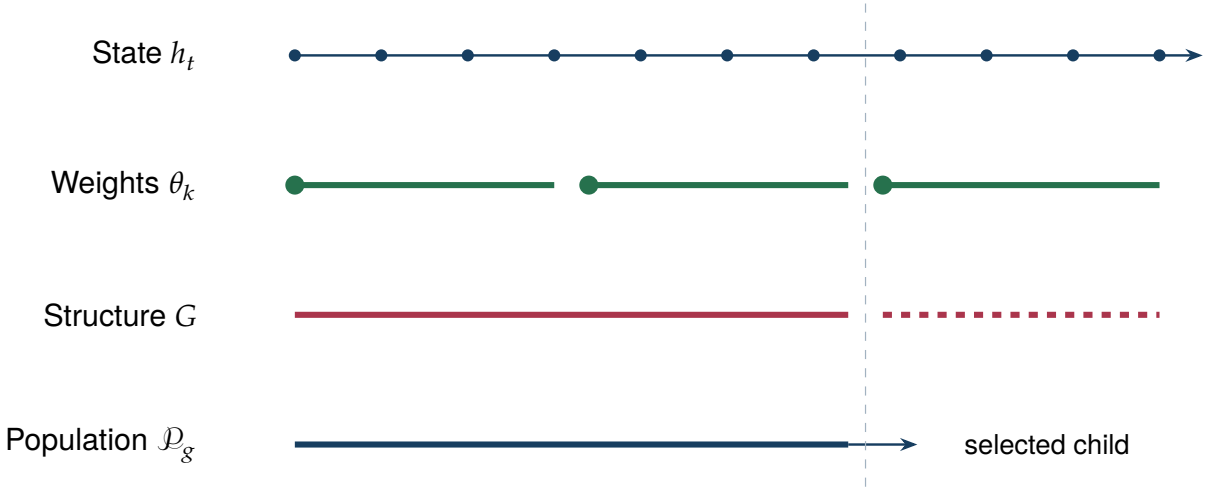


Figure 13.1: State, parameters, structure and population have distinct owners and update events. A state trajectory occurs under one parameter version; a training run occurs under one structural contract. The example uses explicit resets at these boundaries.

An attention cache changing during inference is not parameter learning. Turning an expert on for one token is not necessarily structural learning: the graph may already contain all experts and a fixed routing rule. A rewritten graph is a structural proposal even before anyone has measured its usefulness. Evolution requires an inheritance/variation/selection process over candidates, not merely an operation named `mutate`.

Biological inheritance motivates vocabulary but does not license an identity claim. A software mask has no cell machinery, metabolism or reproductive ecology. We will use **genotype** only as a declared encoding of a computational candidate, with its interpretation specified.

13.2 A two-feature problem that exposes the difference

Take training rows $(-1,-1)$ and $(1,1)$, with target $y = 2x_1$. The validation set contains all four sign combinations $(x_1, x_2) \in \{-1,1\}^2$, still with $y = 2x_1$. All features and targets are dimensionless synthetic values.

A binary mask $m = (m_1, m_2)$ defines the active structure:

$$\hat{y} = x^\top (m \odot w), \qquad L_{\text{train}}(w; m) = \frac{1}{N}\|X(m \odot w) - y\|_2^2. \tag{13.5}$$

$X \in \mathbb{R}^{N\times 2}$, $w, m \in \mathbb{R}^2$ and $y \in \mathbb{R}^N$. The mask is held fixed during gradient descent. We do not differentiate through its discrete selection.

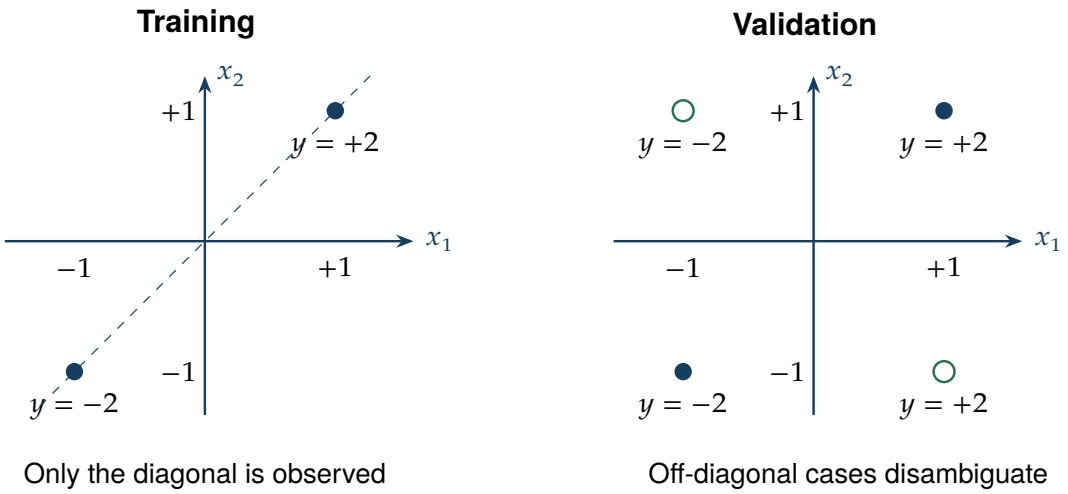


Figure 13.2: Training samples lie on $x_2 = x_1$, so either coordinate can explain them. The validation grid breaks that correlation while retaining $y = 2x_1$. The experiment tests a specified shift, not all possible future distributions.

Let $r = X(m \odot w) - y$. Since changing w_j changes predictions by $m_j X_{:,j}$, differentiation gives

$$\nabla_w L = \frac{2}{N} m \odot (X^\top r). \tag{13.6}$$

An inactive coordinate receives zero gradient. Starting all weights at zero therefore keeps inactive entries zero. That storage convention avoids hidden inherited values becoming active after a later mask change.

The reference uses 40 full-batch updates with learning rate $\eta = 0.1$. For a fixed mask the Hessian is $H = (2/N)\,\mathrm{diag}(m) X^\top X\,\mathrm{diag}(m)$. On every positive-eigenvalue mode, gradient descent contracts when $0 < \eta < 2/\lambda_{\max}(H)$. Zero-eigenvalue directions are unidentifiable and retain their initialization. For mask 11, $\lambda_{\max} = 4$; the chosen rate satisfies the bound.

```
def fit(mask, x, y, steps=40, rate=0.1):
    """Every candidate starts with zero weights and identical step count."""
    m, x, y = checked(mask, x, y)
    if (
        type(steps) is not int
        or steps < 0
        or not math.isfinite(rate)
        or rate <= 0
    ):
        raise ValueError("valid step budget and positive rate required")
    w = np.zeros(len(m))
    for _ in range(steps):
        residual = x @ (m * w) - y
        gradient = 2 * m * (x.T @ residual) / len(y)
        w -= rate * gradient
    return w
```

13.3 Derive a numerical oracle without running the loop

For a single active feature, the training loss is $(w-2)^2$. One update is $w_{k+1} = w_k - 0.2(w_k - 2) = 0.8w_k + 0.4$. With $w_0 = 0$, solve the geometric recurrence:

$$w_k = 2(1 - 0.8^k). \tag{13.7}$$

Either feature obtains the same training loss because the training columns are identical.

For mask 11, symmetry and zero initialization keep $w_1 = w_2 = a$. Each component's gradient is $2(2a-2)$, giving $a_{k+1} = 0.6a_k + 0.4$ and

$$a_k = 1 - 0.6^k. \tag{13.8}$$

These formulas are independent oracles for the iterative implementation, including its gradient scaling and initialization.

On the validation grid, $\mathbb{E}[x_1x_2] = 0$ and $\mathbb{E}[x_1^2] = \mathbb{E}[x_2^2] = 1$. Hence

$$L_{\text{valid}} = (m_1w_1 - 2)^2 + (m_2w_2)^2. \tag{13.9}$$

At convergence, masks 10, 01 and 11 have validation losses 0, 8 and 2. The all-zero predictor has loss 4. Perfect training fit does not identify the causal feature; the deliberately chosen validation intervention supplies information absent from the training set.

We select by

$$J(m) = L_{\text{valid}}(w_{40}(m); m) + 0.05\|m\|_0. \tag{13.10}$$

The second term penalizes active features, not weight magnitude. Its units match squared target units per active feature. It is a declared preference, not a calibrated hardware cost.

Mask	Train MSE	Validation MSE	Score
00	4	4.000000	4.000000
01	7.07e-08	7.998937	8.048937
10	7.07e-08	0.000000	0.050000
11	7.15e-18	2.000000	2.100000

The dense representation fits the training set better at step 40, yet the one-feature 10 mask wins the declared validation objective. Nothing here establishes that sparse models generally outperform dense ones.

13.4 Selection needs a reproducible lineage

Our **elitist search** retains the best candidate among a parent and its one-bit mutations. Generation zero evaluates only parent 01. At the next generation, candidates are 01, 00 and 11; 11 wins. Its next neighborhood contains 11, 01 and 10; 10 wins. All weights are freshly trained from zero for every evaluation, including the parent. Ties use lexicographic mask order, never completion order.

```
def select(candidates, xtrain, ytrain, xvalid, yvalid, penalty=0.05):
    if not math.isfinite(penalty) or penalty < 0 or not candidates:
        raise ValueError("candidates and nonnegative penalty required")
    rows = []
    for mask in sorted(set(candidates)):
```

```
        w = fit(mask, xtrain, ytrain)
        loss = mse(mask, w, xvalid, yvalid)
        rows.append(
            dict(
                mask=list(mask),
                weights=w.tolist(),
                train=mse(mask, w, xtrain, ytrain),
                valid=loss,
                score=loss + penalty * sum(mask),
            )
        )
    # Stable lexical tie break, not arrival order.
    return min(rows, key=lambda r: (r["score"], r["mask"])), rows
```

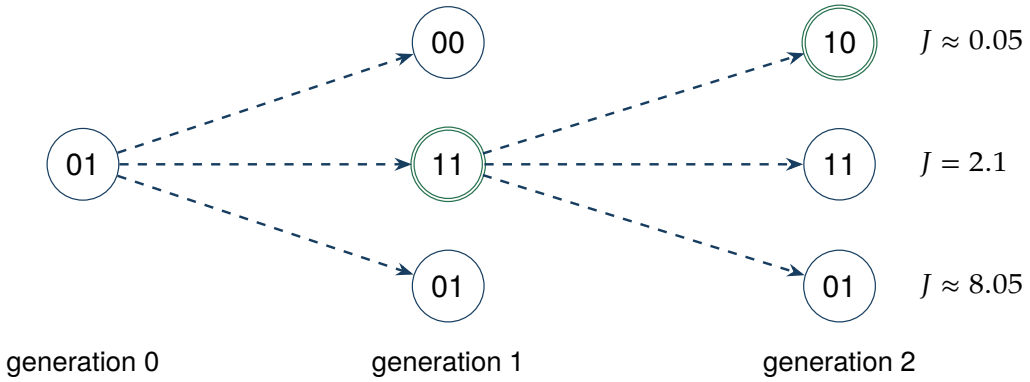


Figure 13.3: The lineage inherits masks only. Each candidate receives the same 40-step training recipe, followed by the same validation objective. Dashed descent edges encode structural inheritance; no trained weight crosses them in this experiment.

Generation	Parent	Winner	Score
0	01	01	8.048937
1	01	11	2.100000
2	11	10	0.050000

```
def evolve(rounds=2):
    """(1+lambda) elitist mask search; weights are never inherited."""
    if type(rounds) is not int or rounds < 0:
        raise ValueError("nonnegative rounds required")
    parent = (0, 1)
    history = []
    for generation in range(rounds + 1):
        candidates = [parent]
        if generation:
            candidates += [
                tuple(1 - v if j == i else v for j, v in enumerate(parent))
                for i in range(len(parent))
            ]
        best, rows = select(candidates, TRAIN_X, TRAIN_Y, VALID_X, VALID_Y)
        history.append(
            dict(
```

```
                generation=generation,
                parent=list(parent),
                candidates=rows,
                winner=best["mask"],
                score=best["score"],
            )
        )
        parent = tuple(best["mask"])
    return history
```

This is a $(1 + \lambda)$ population-selection scheme with one survivor, not a diverse many-survivor population. The parent remains eligible, so deterministic reevaluation under a fixed objective cannot worsen the best score. Noise, changing data or changed evaluation budgets break that argument. A monotone validation score also says nothing about test performance.

Equal step counts are not equal compute. A candidate with two active features performs more useful scalar products than one with one feature; our NumPy representation still stores both columns. This reference isolates optimization budget and semantics, not actual sparse-kernel efficiency. A serious comparison should report candidate evaluations, training steps, data exposures, active parameter count, measured compute and wall time. Caching repeated candidates could save work, but the cache key must include data, initialization, training recipe and code version, not just the mask.

13.5 Inheritance is a mapping, not a file copy

An alternative policy can inherit trained weights. For a stable feature coordinate i, define

$$w_i' = \begin{cases} w_i, & m_i = m_i' = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{13.11}$$

A newly enabled feature starts at zero; a disabled feature's stored weight is cleared. The policy needs a stable meaning for coordinate i. Equal vector lengths are not enough if features were reordered or replaced.

```
def inherit(old_mask, new_mask, weights):
    """Explicit alternative policy: retain only shared active coordinates."""
    if len(old_mask) != len(new_mask) or len(weights) != len(old_mask):
        raise ValueError("stable coordinate map required")
    if any(
        type(v) is not int or v not in (0, 1)
        for v in (*old_mask, *new_mask)
    ):
        raise ValueError("binary masks required")
    if not np.isfinite(weights).all():
        raise ValueError("finite weights required")
    return np.array(
        [
            w if old and new else 0.0
            for old, new, w in zip(old_mask, new_mask, weights)
        ]
    )
```

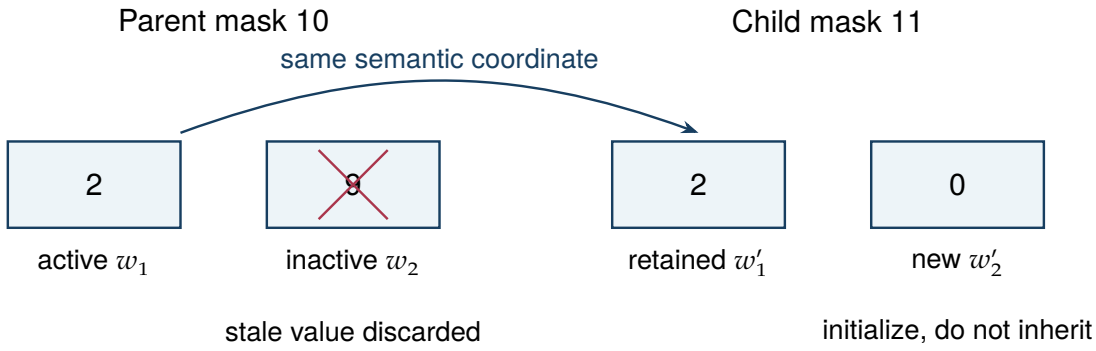


Figure 13.4: The inherited coordinate keeps its value, the newly enabled coordinate starts at zero, and transient sequence state resets. This is an alternative policy, not the policy used to generate the search table. Optimizer state would require a separate inheritance rule.

Weight inheritance changes the experiment: candidates no longer begin from an equally trained state. It may accelerate search, but inherited quality and structural quality become coupled. Momentum buffers, adaptive optimizer statistics, random-number state, normalization statistics and data order must also be addressed. Copying a weight without its optimizer history is a different algorithm from copying both.

In terminology often used for computational evolution, a **Baldwinian** evaluation can let within-candidate learning affect fitness without inheriting its learned weights; a **Lamarckian** variant can transmit them. These labels are useful policy shorthand here, not claims about biological mechanisms. Always publish the actual state map.

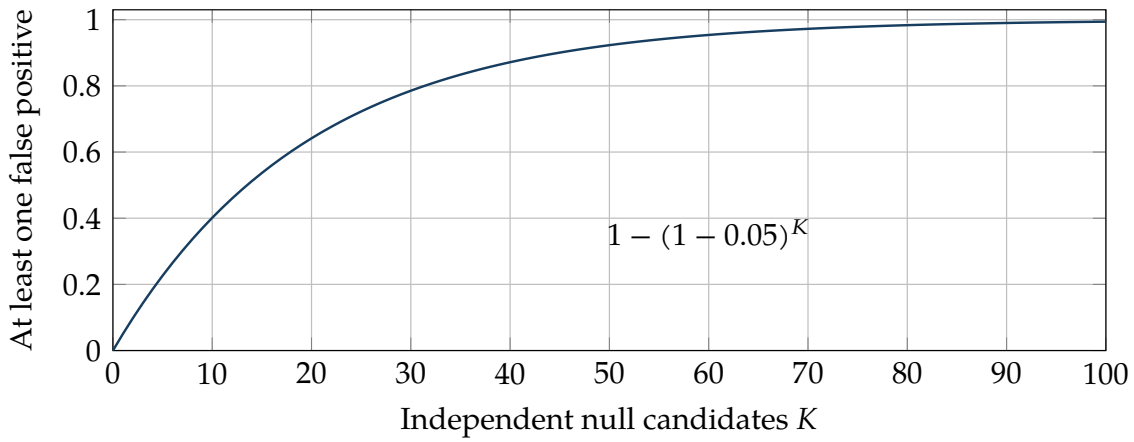
13.6 Search uses up evaluation information

The validation set participates in selection and is therefore part of the search process. It is not an untouched final test. Our complete four-point grid is an exhaustive check of one synthetic condition; we report no separate generalization estimate.

For intuition, suppose K independent null candidates each have probability α of a misleading apparent improvement. Then

$$\Pr(\text{at least one false positive}) = 1 - (1 - \alpha)^K. \tag{13.12}$$

At $\alpha = 0.05$ and $K = 20$, this is approximately 0.642. The calculation is a model of repeated testing, not a calibrated error rate for adaptive architecture search. Candidates and their errors are usually dependent; repeated querying can also make later candidates depend on earlier validation results.



Train: fit weights / Validation: choose candidates / Test: assess frozen choice

Figure 13.5: A larger search increases the chance of finding an apparently favorable null result under the stated independent-test model. Training, selection and final assessment must retain distinct roles. The curve is illustrative probability arithmetic, not measured search performance.

Use an untouched final set only after freezing the selection procedure. For genomic tasks, split by the scientifically relevant unit rather than assuming random windows are independent; related sequences and shared sources can leak information. Predeclare the task, grouping rule, baselines, search budget, stopping rule and uncertainty analysis. Repeating seeds does not repair a systematically leaked split.

13.7 A current frontier: evaluator-driven program evolution

AlphaEvolve's 2025 technical report describes an LLM proposing program changes, automated evaluation, and an evolutionary database that balances reuse and diversity [42]. Its optional evaluation cascade rejects unpromising programs before more costly stages. This is a relevant systems pattern, not the algorithm implemented by our two-bit reference.

The important engineering lesson is that the evaluator is part of the search system's specification. A candidate can exploit an incomplete metric, a timeout policy, a data leak or an implementation bug. Passing tests is evidence about those tests, not an unrestricted correctness proof. For executable candidate code, isolation, resource limits and evaluator integrity are separate requirements; this chapter's safe numeric masks avoid executing generated source.

A useful next experiment would compare a fixed model, random mask search and neighborhood evolution under equal total evaluation budgets. Report both best validation score and final held-out outcome. Include failed candidates and search overhead. Without that comparison, an attractive lineage diagram is not evidence that evolution was necessary.

Key ideas

- When two features agree in training, perfect training fit cannot identify the robust one. Only a validation set that breaks the agreement can.
- Inheritance is a mapping, not a file copy. Newly enabled coordinates start from a declared value and stale weights are cleared.
- A validation set that guides selection is part of the search, not an untouched test. With 20 independent null candidates at $\alpha = 0.05$, the chance of at least one false positive is about 0.64.
- In evaluator-driven program evolution the evaluator is part of the specification, and a candidate can exploit its gaps.

13.8 Exercises with worked answers

1. **Classify.** A recurrent state changes while weights are frozen. *Answer:* This is a state update, not parameter or structural learning.

2. **Derive.** Differentiate Equation 13.5. *Answer:* Apply the chain rule to each residual: $\partial L/\partial w_j = (2/N)\sum_n r_n X_{nj} m_j$. Stacking coordinates gives Equation 13.6.

3. **Oracle.** Compute the single-feature weight after one and two steps. *Answer:* $w_1 = 0.4$ and $w_2 = 0.72$, matching $2(1 - 0.8^k)$.

4. **Identifiability.** Why is mask 11's optimum not unique on the training set? *Answer:* Only $w_1 + w_2$ matters there. Every pair summing to 2 minimizes training loss; symmetric initialization and updates select the symmetric solution.

5. **Shift.** Derive the spurious feature's limiting validation loss. *Answer:* Predictions are $2x_2$; the mean squared error is $4\mathbb{E}[(x_2 - x_1)^2] = 8$ on the independent sign grid.

6. **Budget.** Does forty steps per mask imply equal FLOPs? *Answer:* No. Active work, tensor representation and execution kernels can differ. The controlled variable is the number of updates.

7. **Lineage.** Why does the first mutation round not choose 10? *Answer:* It differs from parent 01 in two bits and is not in the one-bit neighborhood. The search reaches it through 11.

8. **Inheritance.** Map $(1, 0)$ with stored weights $(2, 9)$ to $(1, 1)$. *Answer:* Equation 13.11 yields $(2, 0)$; the stale inactive value 9 must not silently become an inherited skill.

9. **Selection proof.** When is best score monotone? *Answer:* When the parent is included and all candidates are scored deterministically under the same objective. Changed conditions invalidate the comparison; noisy minima can improve without true quality improving.

10. **Multiple testing.** For $K = 2$ and $\alpha = .05$, compute the familywise probability. *Answer:* $1 - .95^2 = .0975$, not .05. The independence assumption must be stated.

11. **Reproducibility.** Specify a candidate cache key. *Rubric:* Include structural encoding, data/split identity, initialization and seed, optimization budget, hyperparameters, evaluator version, numeric precision and code/environment version.

12. **Research design.** Test whether evolution is useful. *Rubric:* Compare fixed, random-search and evolutionary baselines; match total budgets; freeze selection before

final testing; report multiple runs, failed candidates, overhead and uncertainty. Do not tune on final data.

13.9 From adaptation policies to typed candidates

Reproduce the chapter. From the repository root, run `python3 drafts/ch13/build.py --assets-only`, then `python3 -m pytest tests/test_ch13_reference.py`. The first command regenerates `results.json`, tables, plots and literal code listings; the tests compare the implementation with independent oracles and explicit failure cases. Run the builder without `--assets-only` to typeset the chapter on its own.

We can now distinguish state transition, learning, structural proposal and selection by inspecting which object changes and which evidence authorizes the change. The synthetic example demonstrates an evaluation failure mode, not a universal recipe for robust models.

Working vocabulary. A genotype encodes a candidate; phenotype is its interpreted computation; inheritance maps parent state to child state; fitness is an evaluator-defined quantity; elitism retains a qualifying parent; a holdout ceases to be untouched once it guides decisions. Chapter 14 makes candidate structure explicit enough to validate before execution: typed computational genes, owned state and an expression trace.

Sources and evidence notes

Access checked 24 September 2026. The AlphaEvolve report's method overview, evaluation, evolution, distributed-pipeline and limitations passages were inspected. Its reported benchmark improvements were not independently reproduced and are not numerical evidence for this chapter's reference. The synthetic dataset, closed-form gradient oracles, mask search and inheritance policy are original teaching artifacts.

The reference evaluates all points of one declared synthetic validation condition. It reports no untouched-test estimate, trained DNA-model capability, comparative search advantage or production throughput.

CHAPTER 14

Typed genomic computation systems

In this chapter you will

- define tensor types with semantic axis names, sizes and element type
- specify computational genes, genomes and explicit regulator decisions
- type-check a genome as an environment transformation before any arithmetic
- execute one expression step with snapshot reads and an atomic commit

Two arrays both have shape 2×2. In the first, rows are samples and columns are features. In the second, rows are features and columns are samples. A shape-only interface accepts one where it expects the other. The matrix operation can run and return plausible numbers while answering the wrong question.

This chapter builds a small language in which a computational candidate declares its input meanings, operations, parameter owners and state effects. We then execute a two-instruction genome, inspect its trace, and reject malformed variants before numerical work. The word *genomic* names an Evolutor research abstraction here; these are software instructions, not DNA molecules, biological genes or a trained genomic model.

Prerequisites are Chapter 12's graph construction and Chapter 13's separation of structure, parameters and transient state. The deliverable is an inspectable typed reference interpreter. Production compilation, distributed effects, automatic differentiation and deployment are not silently included.

14.1 Start with a contract that shapes alone cannot express

Define a tensor type as

$$\tau = (A, S, d), \qquad A = (a_1, \ldots, a_r), \quad S = (s_1, \ldots, s_r), \tag{14.1}$$

where A lists semantic axis names, S lists positive integer sizes, d is an element type and r is rank. In this reference, names within a type are distinct and $d =$ `float64`. We allow rank-zero scalar types but no zero-size or dynamic-size axes. Those are scope choices, not claims that empty or dynamic tensors are invalid in general.

Type equality requires all three fields to agree in order. Thus a tensor whose axes are `(sample, feature)` is not interchangeable with one whose axes are `(feature, sample)`, even when both have sizes $(2, 2)$ and dtype `float64`. An explicit transpose can convert both values and their types. Relabeling metadata without transposing data is not that conversion.

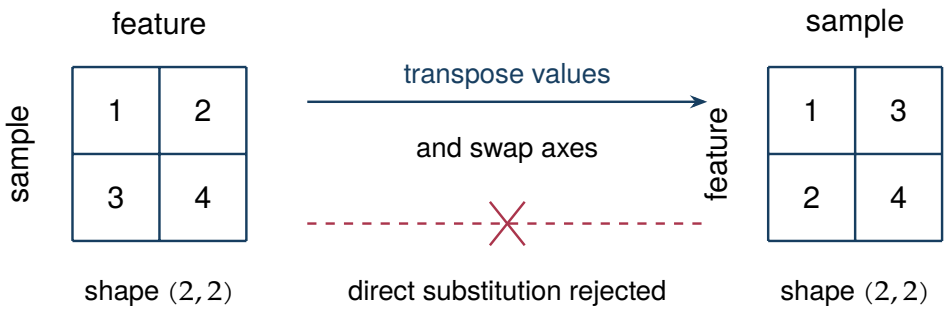


Figure 14.1: Equal shape does not imply equal meaning. An explicit transpose changes the array and swaps its semantic axes. The direct connection is rejected by exact type matching even though a shape-only runtime could execute it.

```
@dataclass(frozen=True)
class TensorType:
    axes: tuple[str, ...]
    sizes: tuple[int, ...]
    dtype: str = "float64"

    def __post_init__(self):
        if (
            type(self.axes) is not tuple
            or type(self.sizes) is not tuple
            or len(self.axes) != len(self.sizes)
            or len(set(self.axes)) != len(self.axes)
            or any(not isinstance(a, str) or not a for a in self.axes)
            or any(type(d) is not int or d < 1 for d in self.sizes)
            or self.dtype != "float64"
        ):
            raise ValueError(
                "distinct axes, positive sizes, float64 required"
            )

    def check(self, value):
        if (
            not isinstance(value, np.ndarray)
            or value.shape != self.sizes
            or str(value.dtype) != self.dtype
            or not np.isfinite(value).all()
        ):
            raise ValueError("runtime value violates declared type")
```

A NumPy array does not carry these semantic names. The runtime checks shape, dtype and finiteness; the caller supplies the truthful semantic declaration. The type system can check consistent use of that declaration, not prove that a file labeled `feature` actually contains the intended features. This trust boundary belongs beside the type definition.

14.2 Define the alphabet and the computational gene

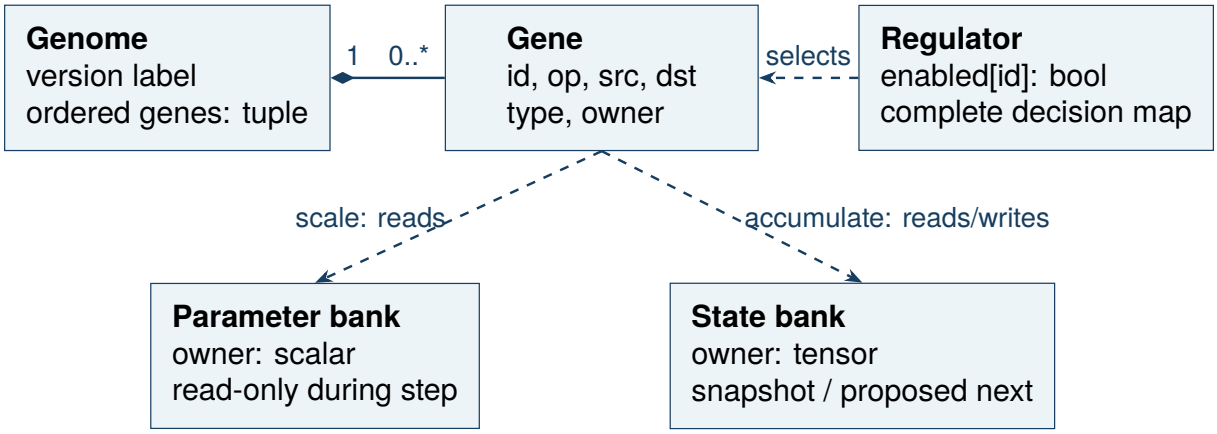
Our operation alphabet is deliberately small:

$$\Sigma = \{\texttt{scale}, \texttt{accumulate}\}. \tag{14.2}$$

A **computational gene** is an immutable instruction record $g = (\text{id}, \text{op}, \text{src}, \text{dst}, \tau, \text{owner})$. The record names a source register and a fresh destination, declares their common type, and identifies an external parameter or state owner. A **genome** is an ordered tuple $(g_1, \ldots, g_L)$. It is not yet an unordered graph or a self-modifying program.

```
@dataclass(frozen=True)
class Gene:
    identity: str
    op: str
    source: str
    output: str
    type: TensorType
    owner: str
```

For `scale`, the owner names a finite scalar parameter p: $y = px$. For `accumulate`, it names a state tensor h of type τ: $y = x + h$ and a proposed next state $h' = y$. State has no trainable update rule in this chapter. Parameter values are read-only during expression.



UML composition: genome owns records. Dashed arrows: references/control.

Figure 14.2: A genome owns an ordered collection of immutable instruction records. Genes reference external parameter or state owners; they do not duplicate those objects. Regulator decisions select whether each instruction uses its active operation or its declared bypass.

Separate **regulation** from instruction validity. Our regulator provides one explicit Boolean decision per gene per expression step. When disabled, either operation returns a copy of its input; an accumulating gene also leaves its state unchanged. This identity bypass is part of the language contract. Other languages might omit an output or return zeros, but cannot leave that choice implicit. A disabled malformed gene is still rejected: regulation is not a way to hide an invalid instruction.

The example genome contains

$$g_1 : u = \texttt{scale}_{\text{gain}}(x), \qquad g_2 : y = \texttt{accumulate}_{\text{memory}}(u).$$

Here $x, u, y, h \in \mathbb{R}^2$ share axis `feature`, gain $= 2$, and initial memory is $(0, 0)$.

14.3 Type checking as an environment transformation

Let Γ map defined value names to their tensor types, Θ contain declared parameter owners, and Δ map state owners to tensor types. A scale instruction is well-formed when

$$\Gamma(\text{src}) = \tau, \quad \text{dst} \notin \operatorname{dom}\Gamma, \quad \text{owner} \in \Theta. \tag{14.3}$$

It extends the environment with $\Gamma' = \Gamma[\text{dst} \mapsto \tau]$. An accumulation additionally requires $\Delta(\text{owner}) = \tau$ and no earlier instruction writing that state owner in this expression. Gene identities must also be unique.

These are **single static assignment** (SSA) value names: a value name is defined once. State ownership is a different constraint. Many genes may read the same scalar parameter; this reference permits only one writer to each state owner. A more expressive language could order multiple writes explicitly, but merely listing them in a file would not specify whether each reads the old or newly updated state.

```
def check_genome(genes, inputs, parameters, states):
    """SSA values, exact semantic types, and one writer per state owner."""
    env, ids, writers = dict(inputs), set(), set()
    for gene in genes:
        if not gene.identity or gene.identity in ids or gene.output in env:
            raise ValueError("duplicate identity or output")
        if gene.source not in env or env[gene.source] != gene.type:
            raise ValueError("missing input or incompatible semantic type")
        if gene.op == "scale":
            if gene.owner not in parameters:
                raise ValueError("undeclared parameter owner")
        elif gene.op == "accumulate":
            if states.get(gene.owner) != gene.type or gene.owner in writers:
                raise ValueError(
                    "state mismatch or competing state writers"
                )
            writers.add(gene.owner)
        else:
            raise ValueError("unknown operation")
        ids.add(gene.identity)
        env[gene.output] = gene.type
    return env
```

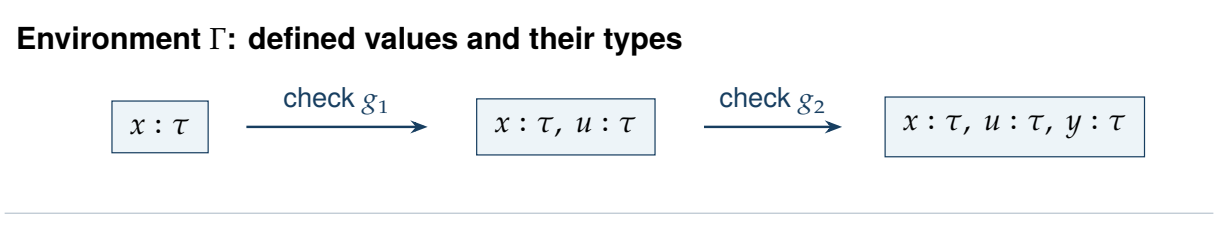


Figure 14.3: The type environment grows only after an instruction passes its checks. Missing producers, incompatible semantic axes, reused destinations and undeclared owners fail before arithmetic. State-writer conflicts are checked independently of value-name uniqueness.

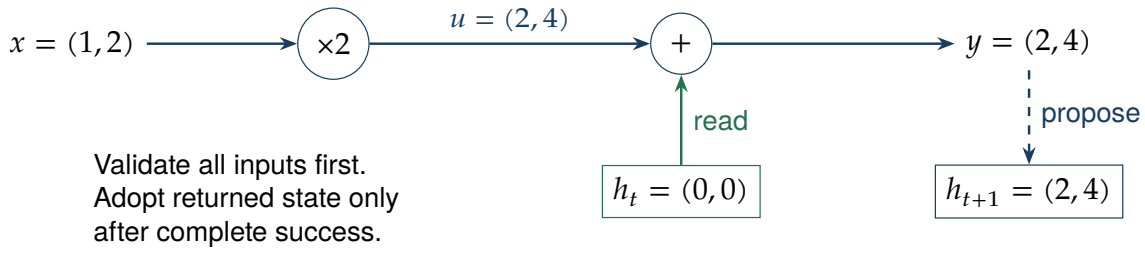
The checking pass is linear in instruction count under expected constant-time dictionary operations, apart from type comparisons proportional to rank. This is a logical work estimate, not a compiler benchmark.

A small preservation argument explains why the checks matter. Assume all inputs satisfy their declared types. A finite scalar multiplication preserves shape, axis interpretation and element dtype in this reference; addition of equal-typed arrays does likewise. Bypass copies the input. Each successful instruction therefore produces a value of its declared type. By induction, all defined values remain well-typed. Floating-point overflow can still violate finiteness, so the runtime checks each result and fails rather than committing invalid state. Type correctness is not numerical stability.

14.4 Expression has a snapshot and a commit boundary

For one expression step, the environment consists of input values, parameter values, a state snapshot and regulator decisions. Evaluation reads only the old snapshot h_t while producing a proposed h_{t+1}. The caller adopts the returned state only after successful completion. The input arrays and caller-owned state arrays are never mutated.

One expression step with snapshot semantics



Solid: data. Dashed: proposed state. Caller-owned arrays are unchanged.

Figure 14.4: The two-instruction example reads a stable state snapshot. With input $(1, 2)$ and gain 2, the first step proposes state $(2, 4)$ and the second proposes $(4, 8)$. Disabling accumulation on the third step bypasses to $(2, 4)$ while retaining memory $(4, 8)$.

The public `express` function checks the complete genome, exact input/state key sets, all runtime values, scalar parameters and an explicit Boolean decision for every gene. It then calls this internal evaluator. Calling the helper directly bypasses validation and is not a supported untrusted-input entry point.

```
def evaluate_checked(genes, inputs, parameters, states, enabled, version):
    """Internal evaluator; express() validates the complete contract first."""
    values = {k: v.copy() for k, v in inputs.items()}
    next_state = {k: v.copy() for k, v in states.items()}
    trace = []
    for gene in genes:
        x = values[gene.source]
        on = enabled[gene.identity]
        if not on:
            y = x.copy()  # declared bypass, not missing output
        elif gene.op == "scale":
            y = x * parameters[gene.owner]
```

```
        else:
            y = x + states[gene.owner]  # read old snapshot
            next_state[gene.owner] = y.copy()
        gene.type.check(y)
        values[gene.output] = y
        trace.append(
            dict(
                version=version,
                gene=gene.identity,
                enabled=on,
                output=y.tolist(),
            )
        )
    return values, next_state, trace
```

Tick	Accumulate?	Output	Next state
0	True	[2.0, 4.0]	[2.0, 4.0]
1	True	[4.0, 8.0]	[4.0, 8.0]
2	False	[2.0, 4.0]	[4.0, 8.0]

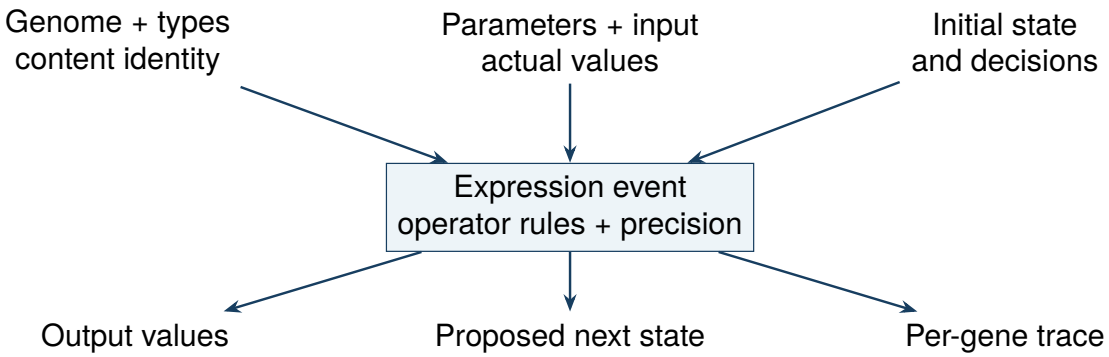
No mutation is committed if an exception occurs midway. This is an **atomic state transition** at the reference API boundary, not a durable database transaction. It offers no process-crash recovery, cross-worker consensus or protection against concurrent mutation by the caller. The snapshot contract assumes no external thread changes the arrays or parameters during a call.

Why copy output into next state? If the returned output and returned state shared writable storage, a caller changing its output buffer could change future model behavior. The reference tests that mutating a returned value does not mutate the returned state or original input.

Why read `states`, not `next_state`? This makes snapshot semantics explicit. Our one-writer restriction keeps the small language simple, but the distinction remains essential when later languages introduce more complex read/write dependencies.

14.5 A trace is evidence only if it identifies the experiment

The reference emits a compact per-gene trace containing the supplied genome version, gene identity, enabled decision and numeric output. This is enough to inspect our small worked example, not enough to replay an arbitrary experiment without its external context.



Reference trace: version label, gene ID, enabled decision, output.
Replay requires retaining the external context above as well.

Figure 14.5: Replay needs the genome, inputs, parameter values, initial state, regulator decisions and execution rules. The compact reference trace records only a subset of this context. A version string is a label, not a content hash or tamper-evident signature.

A practical replay record should bind immutable content identities for the genome, type declarations, parameter snapshot, input batch, initial state and regulator policy or realized decisions. Also record numeric precision, operator implementations, random seeds and environment. For stochastic operations, a seed alone may be insufficient if kernel scheduling changes random-number consumption.

Reproducibility has levels. A semantic replay can agree within a tolerance; a bitwise replay must agree exactly. Neither proves biological usefulness. A trace can establish that a specified program ran on specified data, while an external evaluation must establish whether its answers are good.

14.6 Current compiler practice and the research boundary

The current StableHLO specification defines programs using typed functions and operations, with tensor shapes and element types; it also distinguishes token values used for ordering [43]. That separation is relevant to the distinction between data and effects here. Our named semantic axes, gene records and snapshot policy are original teaching choices, not StableHLO syntax or a compatible dialect.

The industry-facing lesson is to make interfaces machine-checkable before attempting optimization. A lowering must preserve not only array shapes but also state-update order, parameter sharing, bypass semantics and failure behavior. Fusing two operations can be invalid if the unfused program rejects a nonfinite intermediate while the fused program hides it. Reordering stateful operations can change results even when their arithmetic costs match.

Evolutor remains the broader research/compiler/runtime direction above DOGMA (non-Transformer DNA-native model plus DOGMA Engine) and Hermon DNA (Transformer-based DNA model plus Hermon DNA Engine). This chapter supplies neither engine. It supplies a small contract against which future compiler transformations and runtime implementations can be tested.

Key ideas

- Two 2×2 arrays can be incompatible: `(sample, feature)` is not `(feature, sample)`.
- A disabled gene follows a declared identity bypass, and it must still be valid.
- Value names are single-assignment. State ownership is a separate constraint, with one writer per state owner.
- A compact trace is not a replay record. Replay needs content identities for the genome, inputs, parameters, initial state and decisions.

14.7 Exercises with worked answers

1. **Type mismatch.** Can two 2×2 arrays be incompatible? *Answer:* Yes. Reversing `sample` and `feature` changes the declared meaning despite equal dimensions.

2. **Conversion.** Is changing axis labels sufficient to transpose? *Answer:* No. Transpose changes the coordinate map of the data as well as the type. Metadata-only relabeling can assert a false interpretation.

3. **Environment.** Why must a destination be fresh? *Answer:* SSA avoids ambiguous value versions. Reusing `x` could conceal whether later instructions read the original or overwritten value.

4. **State.** Compute two steps with input $(1, 2)$, gain 2 and initial state zero. *Answer:* Each scale gives $(2, 4)$. Accumulation yields states $(2, 4)$ then $(4, 8)$.

5. **Bypass.** Disable accumulation on the next step. *Answer:* Output is the scaled input $(2, 4)$; memory remains $(4, 8)$. Disabled does not mean output zero or memory reset.

6. **Ownership.** Can two scale genes share one parameter owner? *Answer:* Yes; both read the same scalar. Two accumulating genes cannot share one state owner under this reference's one-writer rule.

7. **Negative control.** A disabled gene has an undeclared owner. Should it pass? *Answer:* No. Genome validity is checked independently of today's regulator decisions.

8. **Numerics.** Can a well-typed scale fail at runtime? *Answer:* Yes. Finite inputs and a finite parameter can overflow their product. The finite-result check rejects it without mutating caller state.

9. **Aliasing.** Why copy the accumulated output into next state? *Answer:* Otherwise a later caller mutation of the output array could alter stored memory through a shared buffer.

10. **Replay.** Is the version string alone sufficient? *Answer:* No. Bind content, values, decisions and execution rules. A mutable label identifies no immutable artifact.

11. **Extension.** Add an explicit transpose instruction. *Rubric:* Validate a rank permutation; derive output axes and sizes; transpose the value consistently; reject duplicate/out-of-range axes; test inverse permutation and a same-shape semantic mismatch.

12. **Compiler test.** Propose a legality test for fusion. *Rubric:* Compare outputs, proposed state, bypass decisions, parameter sharing and failures against the reference. Include nonfinite intermediates, disabled genes and rejected ownership conflicts. State the numerical tolerance and what it cannot prove.

14.8 The next step is an operational semantics

Reproduce the chapter. From the repository root, run `python3 drafts/ch14/build.py --assets-only`, then `python3 -m pytest tests/test_ch14_reference.py`. The first command regenerates `results.json`, tables, plots and literal code listings; the tests compare the implementation with independent oracles and explicit failure cases. Run the builder without `--assets-only` to typeset the chapter on its own.

The alphabet, instruction records, type environment, regulator decisions, state snapshot and trace now have distinct meanings. This prevents a biological analogy from standing in for an executable specification.

Working vocabulary. A tensor type declares axes, sizes and dtype; a computational gene names an operation and its dependencies; a genome orders genes; expression executes one regulated step; SSA versions values; ownership identifies shared resources; atomic return separates proposed from adopted state. Chapter 15 will turn these contracts into explicit transition rules and expression traces, then ask which different execution orders are genuinely equivalent.

Sources and evidence notes

Access checked 24 September 2026. The live StableHLO specification's Programs, Functions and Types sections, including tensor and token type definitions, were inspected. This is a dated documentation reading, not a claim to implement any particular release. The named-axis type system, gene records, regulation policy, snapshot semantics and reference interpreter are original.

Passing the reference tests establishes only the stated small-language contracts. It does not establish compiler compatibility, secure auditing, production durability or biological capability.

CHAPTER 15

Operational semantics and expression traces

In this chapter you will

- define configurations, events and a small-step relation for a register machine
- prove a loop correct with an invariant and terminating with a ranking function
- separate fuel exhaustion, explicit halt and runtime error
- state when two state updates commute, and replay a trace to check a claimed execution

A program adds the integers from N down to 1. For $N = 3$, its answer should be 6. But suppose an optimizer decrements the counter before adding it. The program is still well typed, every instruction is legal, and the answer becomes 3. Types alone did not preserve the computation.

Chapter 14 defined typed instructions and snapshot-based state ownership. Here we expose execution one transition at a time. We use scalar integer registers to make control and effects visible without tensor-library details. This is a new, explicitly sequential teaching machine, not a silent change to Chapter 14's snapshot semantics. Its instruction alphabet is deliberately small enough to derive, implement and exhaustively trace.

15.1 Specify configurations before instructions

A program P is an immutable tuple of instructions. A configuration

$$C = (p, \rho) \tag{15.1}$$

contains a program counter p and an environment ρ mapping register names to mathematical integers. The program is fixed during a run. The counter selects the next instruction, not the number of instructions already executed. An event records the instruction and its before/after configurations.

```
@dataclass(frozen=True)
class Configuration:
    pc: int
    registers: tuple
```

The helper `configuration` validates integer values and stores register pairs in sorted order, making snapshots immutable and comparisons deterministic. Python integers avoid fixed-width overflow in this reference; their arithmetic and storage are not constant cost for arbitrarily large values.

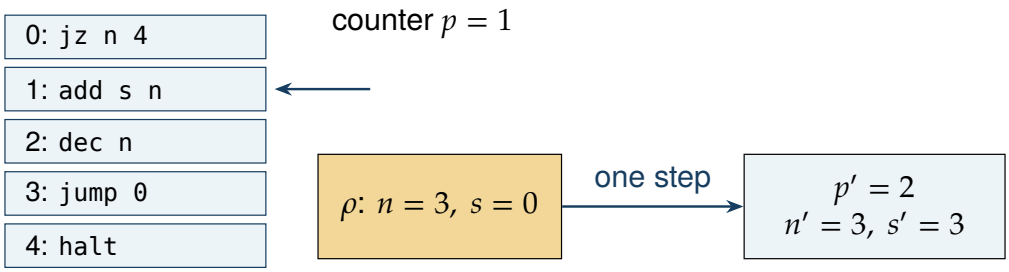


Figure 15.1: A configuration points to one instruction and stores register values. The step relation reads both and returns a new configuration plus an event. No biological process is implied by the term expression.

We use five instruction forms:

Instruction	Effect
add(a, b)	$\rho(a) \leftarrow \rho(a) + \rho(b)$; advance
dec(a)	$\rho(a) \leftarrow \rho(a) - 1$; advance
jump(j)	set counter to j
jz(a, j)	branch to j if $\rho(a) = 0$, otherwise advance
halt	return a halted outcome

The complete program is validated before execution: every register exists, each instruction has the right arity, and every branch target is a valid index. This prevents a disabled or unvisited branch from hiding malformed code.

```
def validate(program, env):
    if not isinstance(program, tuple) or not program:
        raise ValueError("nonempty program tuple required")
    for ins in program:
        if not isinstance(ins, tuple) or not ins:
            raise ValueError("instruction tuple required")
        op, args = ins[0], ins[1:]
        arity = {"jz": 2, "add": 2, "dec": 1, "jump": 1, "halt": 0}
        if op not in arity or len(args) != arity[op]:
            raise ValueError("unknown or malformed instruction")
        regs = (
            args[:1] if op in ("jz", "dec") else args if op == "add" else ()
        )
        if any(r not in env for r in regs):
            raise ValueError("undefined register")
        if op in ("jz", "jump"):
            target = args[-1]
            if type(target) is not int or not 0 <= target < len(program):
                raise ValueError("invalid branch target")
```

Validation is not termination analysis. A legal self-jump can execute forever, and a legal final non-halt instruction can fall off the program.

15.2 Derive the small-step relation

Write $C \xrightarrow{e}_P C'$ when executing the selected instruction produces event e and next configuration C'. For an addition,

$$\frac{P[p] = \operatorname{add}(a, b)}{(p, \rho) \longrightarrow (p + 1, \rho[a \mapsto \rho(a) + \rho(b)])}. \tag{15.2}$$

All right-hand-side values are read from the old environment; if $a = b$, this doubles the old value. Registers not named by the update remain unchanged. For zero-branch,

$$p' = \begin{cases} j & \rho(a) = 0, \\ p + 1 & \rho(a) \neq 0. \end{cases} \tag{15.3}$$

For decrement, the new value is $\rho(a) - 1$, including negative results. Halting is a terminal action, not ordinary fallthrough. The implementation uses counter $|P|$ as a halted checkpoint, together with a separate status.

```
def step(program, config):
    """One rule application; integer values are mathematical, not machine
     ↪ words."""
    env = dict(config.registers)
    if not 0 <= config.pc < len(program):
        raise ValueError("no instruction at program counter")
    ins = program[config.pc]
    op, args = ins[0], ins[1:]
    pc = config.pc + 1
    if op == "add":
        env[args[0]] += env[args[1]]
    elif op == "dec":
        env[args[0]] -= 1
    elif op == "jump":
        pc = args[0]
    elif op == "jz":
        pc = args[1] if env[args[0]] == 0 else pc
    elif op == "halt":
        pc = len(program)
    else:
        raise ValueError("unknown operation")
    nxt = configuration(pc, env)
    event = (config.pc, tuple(ins), config.registers, nxt.pc, nxt.registers)
    return nxt, event, op == "halt"
```

There is at most one applicable rule for a validated instruction and environment. Thus the machine is deterministic: the same program and configuration produce the same next configuration and event. The argument depends on exact integer operations and a fixed program. Random routing, asynchronous external effects or floating-point nondeterminism would require additional semantics.

The public runner validates inputs; `step` is an internal primitive assuming that validation. It cannot safely execute arbitrary unvalidated tuple structures merely because its body is short.

15.3 A loop invariant proves the result; a ranking function proves termination

The program has indices 0 through 4:

0	jz$(n, 4)$	check completion
1	add(s, n)	accumulate current counter
2	dec(n)	decrease counter
3	jump(0)	return to check
4	halt	finish.

Initialize $n = N \geq 0$, $s = 0$. At each visit to index 0, the invariant is

$$s + \frac{n(n+1)}{2} = \frac{N(N+1)}{2}, \qquad 0 \leq n \leq N. \tag{15.4}$$

Initially it is immediate. One body iteration replaces (s, n) by $(s + n, n - 1)$. Substitution gives

$$s + n + \frac{(n-1)n}{2} = s + \frac{n(n+1)}{2},$$

so the invariant is preserved at the next loop head. When $n = 0$, it implies $s = N(N+1)/2$.

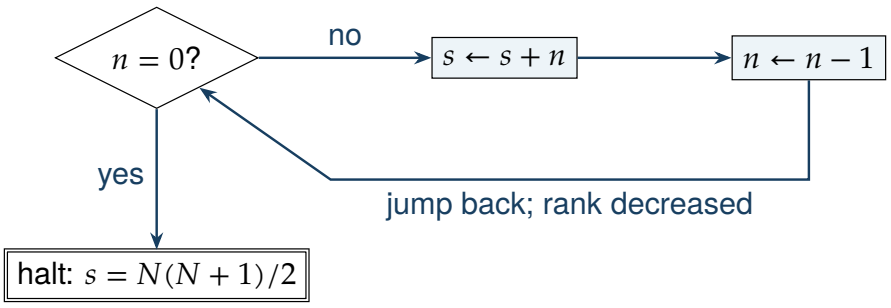


Figure 15.2: The guarded loop checks zero before changing state. The rank n decreases once per body traversal. It need not decrease on the guard or back edge individually.

Correctness on termination does not itself prove termination. At loop heads with $n > 0$, rank n is a nonnegative integer that decreases strictly after a body. It therefore reaches zero after exactly N bodies. Each body cycle executes four instructions, followed by a final zero check and halt, so

$$T(N) = 4N + 2. \tag{15.5}$$

If $N < 0$, decrement moves away from zero. The program remains syntactically valid but the precondition for this termination proof fails. The runner accepts integer states generally; the summation contract requires the caller to enforce $N \geq 0$.

Step	PC	Operation	n	s	Next PC
1	0	jz	3	0	1
2	1	add	3	3	2
3	2	dec	2	3	3
4	3	jump	2	3	0
5	0	jz	2	3	1
6	1	add	2	5	2
7	2	dec	1	5	3
8	3	jump	1	5	0
9	0	jz	1	5	1
10	1	add	1	6	2
11	2	dec	0	6	3
12	3	jump	0	6	0
13	0	jz	0	6	4
14	4	halt	0	6	5

This generated trace for $N = 3$ reaches $s = 6$ in fourteen transitions. An event is recorded for the halt instruction too. Changing that convention would change the exact cost formula.

15.4 Fuel bounds observation, not the program's meaning

```
def run(program, env, fuel, pc=0):
    """Validate first; return halt/exhausted/error and an immutable checkpoint."""
    config = configuration(pc, env)
    validate(program, env)
    if type(fuel) is not int or fuel < 0 or pc >= len(program):
        raise ValueError("invalid fuel or initial counter")
    trace = []
    for _ in range(fuel):
        try:
            nxt, event, halted = step(program, config)
        except ValueError:
            return "error", config, tuple(trace)
        config = nxt
        trace.append(event)
        if halted:
            return "halt", config, tuple(trace)
        if config.pc >= len(program):
            return "error", config, tuple(trace)
    return "exhausted", config, tuple(trace)
```

The runner returns one of three statuses. `halt` means an explicit halt was executed. `exhausted` means the provided transition budget was used without observing halt. `error` means no valid next instruction exists after a step, such as falling beyond the last non-halt instruction. Malformed programs are rejected before running, rather than being classified as a runtime outcome.

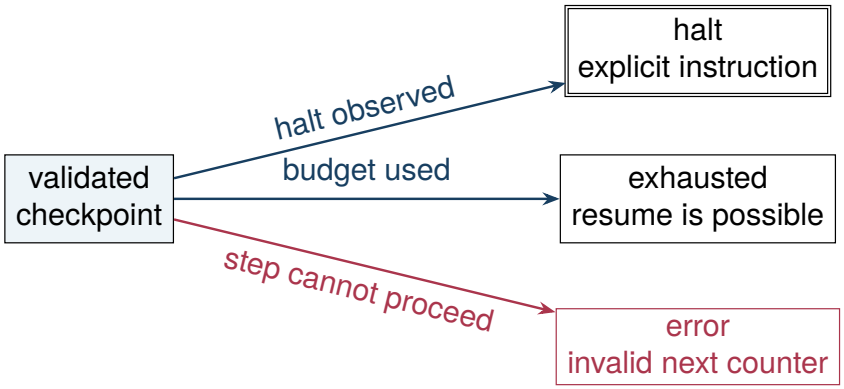


Figure 15.3: A bounded run separates an explicit halt, an invalid next step and budget exhaustion. Exhaustion preserves a checkpoint that can be resumed; it is not a proof of divergence.

For a valid initial configuration, zero fuel returns an empty trace and exhaustion. Fuel $4N + 1$ stops just before the halt instruction in the summation example; $4N + 2$ observes it. A self-jump also exhausts any finite fuel supply, but these equal statuses do not imply equal termination behavior.

The bound controls transitions, not memory or wall-clock time. Integer sizes, event storage and validation cost are outside its count. The implementation is not a sandbox for executing untrusted programs; it interprets a fixed, small instruction alphabet and performs no external I/O.

A resumable checkpoint contains the counter and complete register environment. Run seven steps at $N = 4$, then resume the checkpoint for eleven more: the concatenated event sequence equals the eighteen-step uninterrupted run. This follows by determinism and can be tested at every split before halt. A checkpoint after halt is terminal; the public API does not restart it as an ordinary instruction.

15.5 Well-typed transformations can violate effects

Compare $\mathrm{add}(s, n); \mathrm{dec}(n)$ with the reverse order. At $(s, n) = (0, 3)$ they yield $(3, 2)$ and $(2, 2)$. Both preserve integer types. They differ because addition reads n while decrement writes n.

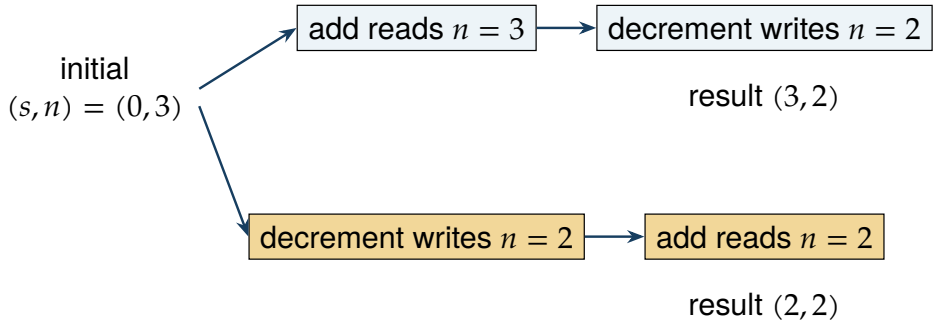


Figure 15.4: Reordering a read across a write changes the result. The dependency arrow is an ordering constraint, not a data tensor. Equal types and equal instruction counts do not establish equivalence.

For two pure deterministic state updates A, B with read sets R_A, R_B and write sets W_A, W_B, a sufficient noninterference condition is

$$W_A \cap (R_B \cup W_B) = \emptyset, \qquad W_B \cap (R_A \cup W_A) = \emptyset. \tag{15.6}$$

Under these assumptions neither update changes values the other observes or writes, so the two orders commute. This is sufficient, not necessary: some overlapping arithmetic updates commute for algebraic reasons.

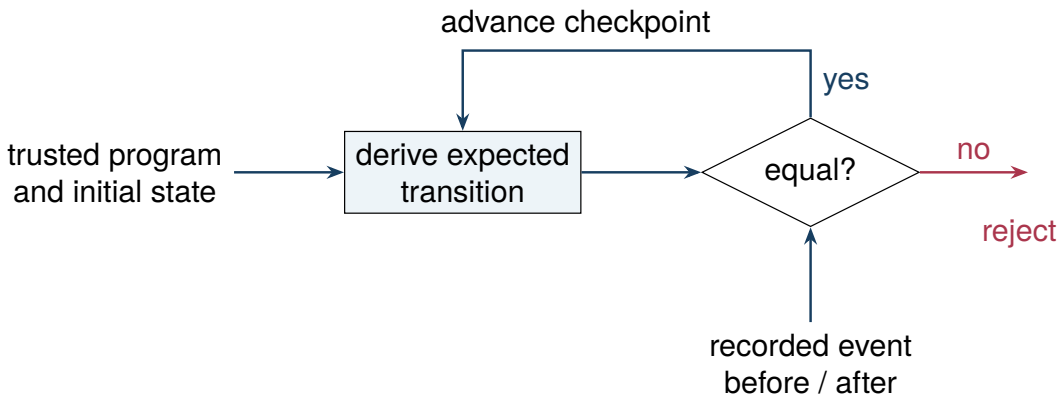
Control effects and observability also matter. Exchanging two independent register updates can preserve the final environment while changing a full instruction trace. If the specification exposes every event, trace equality is stronger than final-value equality. A compiler must declare which observations it preserves: results, exceptions, I/O, state, or a specified projection of a trace.

Modern MLIR structured-control-flow documentation makes region arguments, conditions and yielded values explicit [44]. StableHLO likewise states dataflow execution ordering and uses token dependencies for side effects [43]. These are relevant production design patterns, not evidence that this register machine implements either representation.

15.6 Replay checks a claim rather than trusting a log

An event stores before counter, instruction, before registers, after counter and after registers. Semantic replay recomputes the expected event from the current configuration, compares the entire record, and advances only on equality.

```
def replay(program, initial, events):
    """Recompute every transition. An empty or partial trace is allowed."""
    validate(program, dict(initial.registers))
    current = initial
    for event in events:
        nxt, expected, _ = step(program, current)
        if event != expected:
            raise ValueError("trace does not match transition semantics")
        current = nxt
    return current
```



Valid prefix ≠ completed execution. Check the final halt separately.

Figure 15.5: A replay checker recomputes each transition from trusted program and initial state. It compares the claimed event with the expected event. A plausible final answer alone cannot certify the intervening trace.

Duplicating or changing an event is detected when the next event no longer matches the derived transition. A valid prefix is still a valid partial trace: replay does not claim completion unless the caller separately checks that its last event is halt and its final configuration agrees with the reported result. Removing an entire suffix can therefore preserve replay validity while destroying a completeness claim.

The trace is not cryptographically authenticated. An adversary who changes the program, initial state and all events consistently can produce another valid trace. Retain content identities and an external trust mechanism when provenance matters. A textual version label does not supply either automatically.

Tests compare the summation result with $N(N+1)/2$ for $N = 0, \ldots, 50$, check exact fuel boundaries, resume a split run, replay its events and reject a tampered record. Negative controls include a self-loop, a reordered body, undefined registers, invalid jumps and fallthrough without halt.

Key ideas

- Types alone do not preserve a computation. Swapping `add` and `dec` keeps every type and gives $N(N-1)/2$ instead of $N(N+1)/2$.
- Running out of fuel does not prove divergence: one step too little fuel also exhausts.
- Disjoint read and write sets are sufficient for two updates to commute, but not necessary.
- Replay recomputes each event from a trusted program and start state. A valid prefix does not prove completion, and an unauthenticated trace proves nothing about its producer.

15.7 Exercises with worked answers

1. **Predict.** Run $N = 0$. *Answer:* Zero check and halt execute; $s = 0$, with two transitions.
2. **Derive.** Prove Equation 15.4. *Answer:* The body adds n and replaces its triangular remainder by $(n-1)n/2$; the total is unchanged.
3. **Terminate.** Why is n a valid rank only at loop heads? *Answer:* It decreases across each complete body, but not on every control step.
4. **Count.** Give exact fuel for $N = 10$. *Answer:* $4(10) + 2 = 42$.
5. **Distinguish.** Does exhausted fuel prove an infinite loop? *Answer:* No; the summation run with one step too little fuel is a counterexample.
6. **Precondition.** What fails for $N = -1$? *Answer:* The nonnegative rank premise; successive decrements never reach zero.
7. **Effects.** Reorder the loop's add and decrement. *Answer:* The new sum is $N(N-1)/2$, although the final counter is still zero.
8. **Aliasing.** Interpret $\mathrm{add}(s, s)$. *Answer:* Read the old value twice and store $2s$, not a sequentially updated right-hand side.
9. **Replay.** Can a valid prefix establish completion? *Answer:* No; require a final halt and agreement with the claimed final state.
10. **Implement.** Test every split of a finite run. *Rubric:* Resume counter and complete environment, compare concatenated events, handle zero-length prefixes, and treat a halted checkpoint as terminal.

11. **Optimize.** State what observations a scheduler preserves. *Rubric:* Specify final values, effects and trace projection; test a read/write conflict and an independent pair. Do not equate output parity with full trace parity.

12. **Budget.** Why is fuel not a memory sandbox? *Answer:* Integer payloads and retained events can grow; one logical step does not impose a byte or time limit.

15.8 Terms and the next contract

A **configuration** is the machine's current counter and environment. A **small step** is one rule application. A **ranking function** proves progress toward termination. **Observational equivalence** depends on which outcomes are visible. A **checkpoint** retains enough state to resume. README commands reproduce the reference and its tables. The next chapter attaches resource ledgers to executions without confusing a logical step with arithmetic work, peak memory or measured latency.

Sources and evidence

The instruction set, summation program, proofs, trace format and interpreter are original teaching artifacts. Current official documentation was inspected on 24 September 2026 for the specific production-design comparisons below. No MLIR lowering, StableHLO backend, distributed runtime or production compiler is implemented or benchmarked by this chapter.

CHAPTER 16

Expression complexity and resource semantics

In this chapter you will

- treat cost as a vector with units: parameters, MACs, routing, bytes, peak memory and events
- derive expert-routing ledgers with and without padding and capacity drops
- compute peak live memory for a schedule and show that legal schedules differ
- prove an amortized bound with a potential function and place a design under a roofline envelope

A system stores four experts but routes each token to one. Has it reduced inference cost by a factor of four? Only if we know what is counted. The router still runs; weights still occupy storage; padded execution may evaluate empty slots; moving representations may dominate arithmetic. A sparse logical graph is not a measured speedup.

Chapter 15 gave executions an exact transition semantics. Here we attach explicit resource models to computations. We use integer ledgers and a tiny dependency graph, not GPU timings. The purpose is to make a cost claim falsifiable: a reader should be able to identify both the counted work and the work omitted by the model.

16.1 A resource is a vector with units, not one adjective

For an execution trace τ, define separate components

$$C(\tau) = (P,\ M,\ R,\ D,\ H,\ J). \tag{16.1}$$

Here P counts stored scalar parameters, M multiply-accumulate operations (MACs), R routing decisions, D communicated bytes under a stated interface, H peak live buffer bytes, and J retained events. These quantities cannot be added without conversion factors and a purpose. A weighted objective $w^\top C$ introduces policy weights; it is not a universal physical cost.

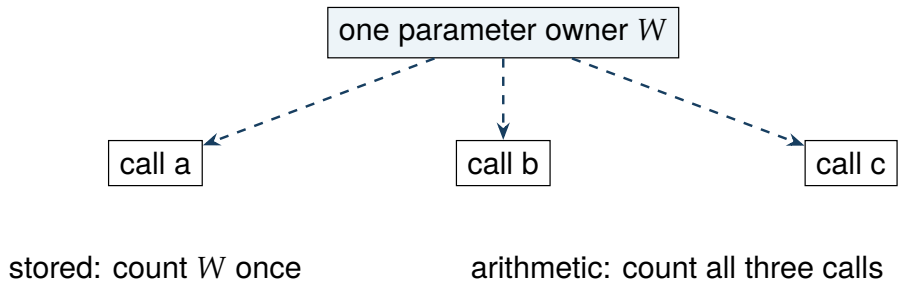


Figure 16.1: Resource ledgers have different units and aggregation rules. Shared ownership can deduplicate stored parameters while repeated calls still perform arithmetic. Peak memory is a maximum over live sets, not a sum over the entire trace.

For additive events, $M(\tau_1\tau_2) = M(\tau_1) + M(\tau_2)$. Peak storage needs the intermediate live state, so it cannot generally be computed from two isolated peaks alone. Parameter storage counts unique owners, not appearances in an instruction list. These distinctions prevent a repeated shared module from being mistaken for either free work or duplicated weights.

Every ledger needs a boundary. Does memory include optimizer state, activations, input storage, allocator fragmentation and kernel scratch space? Does communication count one-way payload, both directions, padding or protocol overhead? We will answer narrowly before computing anything.

16.2 Derive an expert ledger from tensor dimensions

Consider B tokens of width d, E experts, hidden width h, and top-k routing without repeated expert IDs per token. Each expert has bias-free matrices $W_1 \in \mathbb{R}^{d\times h}$ and $W_2 \in \mathbb{R}^{h\times d}$. The router has $W_R \in \mathbb{R}^{d\times E}$. Thus stored parameter count is

$$P = 2Edh + dE. \tag{16.2}$$

We count each multiply-accumulate as one MAC, including accumulation initialized at zero. Applying one expert to one token costs $2dh$ MACs, while router logits cost BdE MACs. Selection, activation, normalization and aggregation are outside this MAC count and must not be assumed free in elapsed time.

Without drops or padding, useful expert work is

$$M_{\text{useful}} = 2Bkdh. \tag{16.3}$$

If only U unique experts are called, their expert parameter count is $2Udh$. This is an active-owner count, not the number of parameters stored in the model and not a guarantee those bytes are fetched just once.

Suppose each expert is allocated C token slots. With dense padded kernels over every expert's slots, executed expert work is

$$M_{\text{padded}} = 2ECdh. \tag{16.4}$$

Let n_e count requested assignments to expert e. Accepted work uses $\min(n_e, C)$ assignments, and dropped assignments total $\sum_e \max(0, n_e - C)$. The reference drops overflow in input order; different admission policies change which tokens lose work.

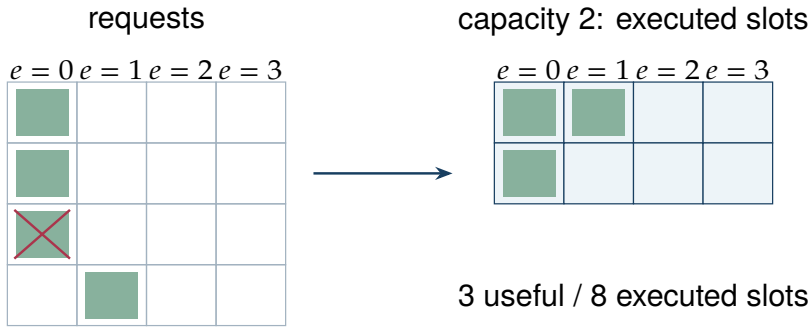


Figure 16.2: Four tokens request experts 0,0,0,1. A padded grid reserves slots for every expert, including unused ones. Capacity two drops one request to expert 0; it is not merely an implementation detail if output semantics depend on that assignment.

```
def routing_cost(
    routes, experts, width, hidden, capacity=None, padded=False
):
    """Top-k routes; overflow is dropped in input order; bytes are dispatch
     ↪  only."""
    positive(experts, width, hidden)
    if not routes or any(not r for r in routes):
        raise ValueError("nonempty routing batch required")
    k = len(routes[0])
    if any(
        len(r) != k
        or len(set(r)) != k
        or any(type(e) is not int or not 0 <= e < experts for e in r)
        for r in routes
    ):
        raise ValueError("equal top-k distinct expert IDs required")
    if capacity is not None:
        positive(capacity)
    if padded and capacity is None:
        raise ValueError("padding needs explicit capacity")
    counts = Counter(e for row in routes for e in row)
    accepted = {
        e: min(n, capacity) if capacity is not None else n
        for e, n in counts.items()
    }
    useful = sum(accepted.values())
    slots = experts * capacity if padded else useful
    return {
        "stored_parameters": experts * (2 * width * hidden + width),
        "active_expert_parameters": len(accepted) * 2 * width * hidden,
        "router_macs": len(routes) * width * experts,
        "useful_expert_macs": useful * 2 * width * hidden,
        "executed_expert_macs": slots * 2 * width * hidden,
        "dropped_assignments": len(routes) * k - useful,
        "dispatch_bytes": useful * 2 * width * 4,
    }
```

For $B = 4, E = 4, d = 8, h = 16, k = 1$, the model stores 1056 scalars: 1024 expert and 32 router parameters. Router work is 128 MACs. The two used experts own 512 scalars. Generated expert-work counts are

Policy	Useful MACs	Executed MACs	Dropped
ragged	1024	1024	0
padded	1024	4096	0
capped	768	2048	1

The padded $C = 4$ case executes four times the useful expert MACs. The $C = 2$ case drops one assignment yet still executes more expert MACs than the ragged case. Fewer accepted assignments can coexist with more executed work.

For a float32 dispatch interface that sends each accepted width-d activation and receives one width-d result, payload bytes are

$$D = 2 \cdot 4d \sum_{e} \min(n_e, C), \tag{16.5}$$

using n_e when capacity is unbounded. The ragged example gives 256 bytes. This excludes parameter transfer, routing metadata, collective overhead and padding on the wire. Local assignments may need no inter-device traffic at all; the formula describes a declared all-remote payload model, not measured network load.

16.3 Liveness turns a graph into a memory calculation

Consider dependencies $a = f(x), b = g(x), c = u(a), d = v(b), y = w(c, d)$. Let x, a, b each occupy 100 abstract bytes and c, d, y each occupy 10. These are invented allocation sizes, not tensor measurements. Inputs count as live from the beginning. Outputs remain live at the end. Each result is allocated before last-use operands are released.

A legal schedule must execute a node after its dependencies. Two legal choices are

$$S_1 = (a, b, c, d, y), \qquad S_2 = (a, c, b, d, y).$$

For S_1, allocating b temporarily leaves x, a, b live: 300 bytes. For S_2, computing c permits release of a before allocating b: the peak is 210 bytes. Both schedules execute every operation exactly once.

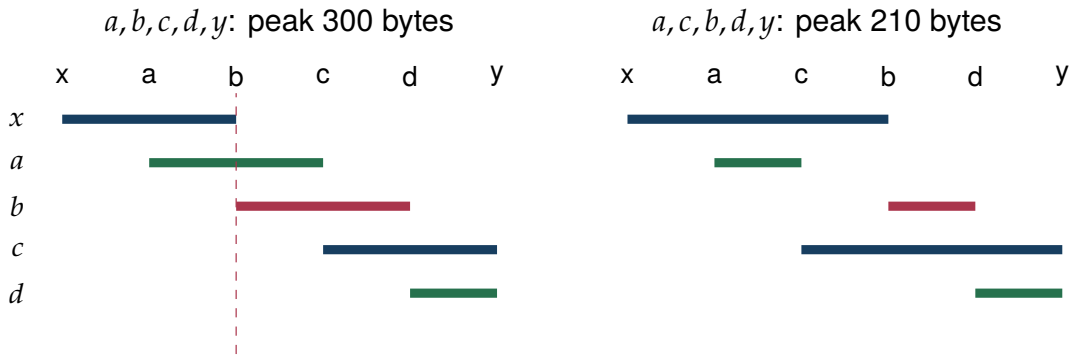


Figure 16.3: The same dependency graph admits different legal buffer lifetimes. The interleaved schedule shortens the lifetime of a large intermediate. Allocation occurs before last-use release, so the peak includes both old operands and a newly allocated result.

```
def peak_live(sizes, inputs, dependencies, schedule, outputs):
    """Abstract bytes; allocate result before freeing last-use operands."""
    validate_layout(sizes, inputs, dependencies, schedule, outputs)
    remaining = Counter(v for deps in dependencies.values() for v in deps)
    live = set(inputs)
    peak = sum(sizes[v] for v in live)
    trace = []
    for node in schedule:
        deps = dependencies[node]
        if any(v not in live for v in deps):
            raise ValueError("dependency not available")
        live.add(node)
        allocated = sum(sizes[v] for v in live)
        peak = max(peak, allocated)
        for v in deps:
            remaining[v] -= 1
            if remaining[v] == 0 and v not in outputs:
```

```
                live.discard(v)
        if remaining[node] == 0 and node not in outputs:
            live.discard(node)
        trace.append(
            {
                "node": node,
                "allocated": allocated,
                "after": sum(sizes[v] for v in live),
            }
        )
    return peak, trace
```

The companion's `validate_layout` checks declarations and schedule membership before execution. The algorithm initializes remaining-use counts from all dependency edges. After a node runs, decrement each operand's count and release non-output operands whose count reaches zero. Multiple uses of the same operand in one node count as separate uses but do not allocate duplicate buffers. A node with no future users is released immediately unless declared an output. Unused initial inputs remain retained in this narrow contract.

To validate the implementation independently, tests enumerate all legal topological schedules and compute interval overlap from producer and last-user indices. Both methods must agree; enumerated peaks range from 210 to 300 bytes. An invalid schedule is rejected rather than assigned an artificially small peak.

This analysis omits aliasing, in-place updates, recomputation, asynchronous execution, allocator reuse and operator workspace. An actual runtime may save or spend more memory. In-place reuse is legal only when alias and effect conditions hold; overwriting an operand before its final consumer changes the computation. Checkpointing can lower retained activation memory by recomputing values, changing both the arithmetic ledger and the schedule.

16.4 Amortized cost explains totals without hiding spikes

A trace recorder appends events to a dynamic buffer. Start with size $n = 0$ and capacity $c = 0$. If full, allocate capacity $\max(1, 2c)$ and copy all n old events, then write the new event. Count one unit per event copy or append; allocation overhead and variable-size event payload are excluded.

An append normally costs one unit. When the old buffer is full at size $n > 0$, it costs $n + 1$. The next append can therefore be expensive even if average cost remains bounded. Define a nonnegative potential over reachable buffer states:

$$\Phi(n, c) = 2n - c. \tag{16.6}$$

At empty state it is zero. After the first allocation it is one; after later doublings it is two; between doublings it rises by two per append. The amortized cost is

$$\hat{a}_i = a_i + \Phi_i - \Phi_{i-1}. \tag{16.7}$$

For a non-resizing append, $\hat{a}_i = 1 + 2 = 3$. For a resize with old $c = n > 0$, old potential is n and new potential is 2, so $\hat{a}_i = (n + 1) + 2 - n = 3$. The first append has amortized cost 2.

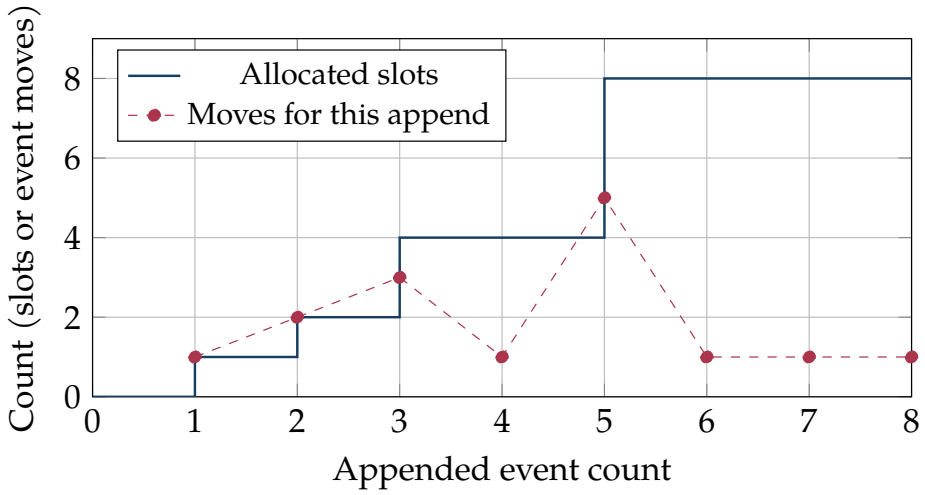


Figure 16.4: A doubling buffer trades occasional copy bursts for bounded aggregate work. The potential records prepaid work in the proof; it is not stored energy or a measured device counter.

```
def trace_buffer(events):
    """Count writes plus copies in a doubling vector; empty buffer has capacity
     ↪  0."""
    if type(events) is not int or events < 0:
        raise ValueError("nonnegative event count required")
    size, capacity, total = 0, 0, 0
    rows = []
    for _ in range(events):
        old_phi = 2 * size - capacity
        copied = 0
        if size == capacity:
            copied = size
            capacity = max(1, 2 * capacity)
        size += 1
        actual = 1 + copied
        phi = 2 * size - capacity
        amortized = actual + phi - old_phi
        total += actual
        rows.append(
            dict(
                size=size,
                capacity=capacity,
                actual=actual,
                potential=phi,
                amortized=amortized,
                total=total,
            )
        )
    return rows
```

Events	Capacity	Actual	Potential	Amortized
1	1	1	1	2
2	2	2	2	3
3	4	3	2	3
4	4	1	4	3
5	8	5	2	3
6	8	1	4	3
7	8	1	6	3
8	8	1	8	3
9	16	9	2	3
10	16	1	4	3
11	16	1	6	3
12	16	1	8	3
13	16	1	10	3
14	16	1	12	3
15	16	1	14	3
16	16	1	16	3

Summing Equation 16.7 telescopes:

$$\sum_{i=1}^{N} a_i = \sum_{i=1}^{N} \hat{a}_i + \Phi_0 - \Phi_N \leq 3N. \tag{16.8}$$

Nonnegative final potential is essential to the bound. Tests check every prefix through 128 events, not only powers of two.

Modern cost-aware verification work explicitly relates a program's behavior to its resource accounting [45]. Our dynamic-buffer example is an original small application of the potential method, not an implementation of an automatic resource-analysis tool. An amortized bound is neither a probabilistic average over workloads nor a worst-case per-event latency bound. A latency-sensitive recorder may preallocate, chunk storage or stream events; each alternative changes the contract.

The Chapter 15 trace contains complete environments, not fixed-size events. If each event stores r registers of up to b bits, its payload scales as $O(rb)$ before serialization overhead. The buffer's copy-count model cannot be reported directly as total bytes or constant-time logging.

16.5 A performance envelope is a hypothesis to measure

Let F be counted floating-point operations, D actual bytes transferred across one specified memory boundary, $P_{\max}$ an attainable compute ceiling in operations per second, and β an attainable bandwidth in bytes per second. Ignoring overlap overhead and other bottlenecks yields the necessary bound

$$t \geq \max\left(\frac{F}{P_{\max}}, \frac{D}{\beta}\right). \tag{16.9}$$

For $D > 0$, intensity $I = F/D$ gives

$$\frac{F}{t} \leq \min(P_{\max}, \beta I). \tag{16.10}$$

The crossover is $I^* = P_{\max}/\beta$. Below it, bandwidth limits this two-resource envelope; above it, compute does. Counting a MAC as two operations is a reporting convention and must be stated consistently.

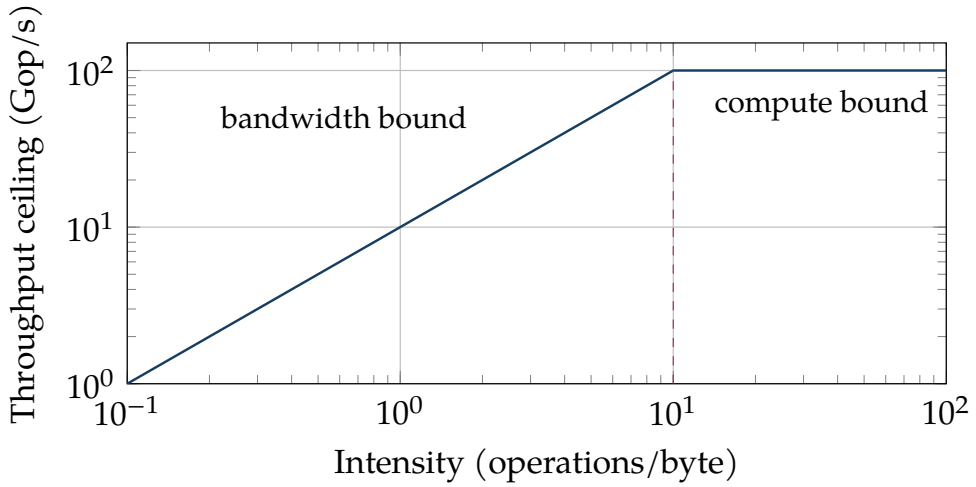


Figure 16.5: An illustrative envelope with compute ceiling 100 Gop/s and bandwidth 10 GB/s has crossover 10 operations/byte. These assigned values are not a hardware benchmark or a prediction for either Evolutor model direction.

Equation 16.9 is a lower bound on time, not a promise that either resource can be saturated. Launch overhead, dependency chains, synchronization, small matrices and communication can make observed time much larger. Do not insert the dispatch-only byte count from Equation 16.5 as total device-memory traffic without evidence: it crosses a different boundary.

Current PyTorch profiler documentation provides execution activity and memory recording, and limits its FLOP estimates to supported operations [46]. Use profiling to test a specified hypothesis after validating output parity. Declare warm-up, input shapes, dtype, device, synchronization, compilation state, sample count and uncertainty. Do not compare an eager reference with a compiled candidate and attribute every difference to the mathematical architecture.

A useful experimental sequence is: establish semantics; predict the ledger; measure actual operators and traffic; identify the gap; change one implementation choice; rerun correctness and measurements. This turns a resource estimate into an engineering question rather than an industrial performance claim.

Key ideas

- Sparse routing can execute more arithmetic than its useful work. With padded capacity, four times as many expert MACs run as are needed.
- Peak memory is a maximum over live sets, not a sum; the order a, c, b, d, y lowers the peak from 300 to 210 bytes.
- An amortized cost of three per append is not a latency bound. The append that triggers a resize still copies every element.
- A roofline is a lower bound on time under stated ceilings. It is a hypothesis to measure, not a performance result.

16.6 Exercises with worked answers

1. **Count.** Recompute stored parameters for $E = 4, d = 8, h = 16$. *Answer:* $2Edh + dE = 1024 + 32 = 1056$.
2. **Separate.** Why are 512 active expert scalars not the model size? *Answer:* Only two experts are called, but all four plus router remain stored.

3. **Derive.** What is ragged expert work for four top-1 tokens? *Answer:* $2(4)(1)(8)(16) = 1024$ MACs, excluding router and non-MAC work.

4. **Padding.** What does capacity four execute? *Answer:* $2(4)(4)(8)(16) = 4096$ expert MACs under all-expert dense padding.

5. **Overflow.** How many assignments are dropped with loads $(3, 1, 0, 0)$ and capacity two? *Answer:* One; capacity does not change the fact that its omitted contribution can change the output.

6. **Units.** Verify the ragged dispatch payload. *Answer:* Four accepted calls times two directions times eight float32 scalars times four bytes gives 256 bytes.

7. **Schedule.** Why is a, c, b, d, y better for the example's peak? *Answer:* It releases 100-byte a after producing 10-byte c before allocating b, reducing the peak from 300 to 210.

8. **Oracle.** Independently check liveness. *Rubric:* Enumerate legal orders, assign allocation/last-use intervals, include initial inputs and final outputs, and compare peaks at allocation boundaries.

9. **Potential.** Bound total append/copy work. *Answer:* Telescoping and $\Phi_N \geq 0$ give at most $3N$ counted units.

10. **Latency.** Does amortized cost three bound every append by three? *Answer:* No; appending to a full capacity-eight buffer copies eight old events and writes one, costing nine.

11. **Envelope.** Find crossover for 100 Gop/s and 10 GB/s. *Answer:* Ten operations/byte. Consistent decimal giga units cancel.

12. **Experiment.** Design an honest sparse-versus-dense comparison. *Rubric:* Match output contract, token cohort, precision and hardware; count router, padding, transfers and compilation; validate parity; report repeated timings and memory, including failures and dropped assignments.

16.7 Terms and next scope

A **cost model** chooses counted events and units. The **liveness** of a value depends on its future consumers. An **amortized cost** bounds aggregate sequences using a proof, not timing. The **arithmetic intensity** is operations per byte across a specified boundary. Using **padding** can turn sparse useful work into denser executed work. The README reproduces all ledgers. Later engine chapters can replace these assumptions with measurements; they must preserve the semantic contracts first.

Sources and evidence

The routing ledger, schedule comparison, buffer derivation, numerical examples and tests are original. The cost-aware verification comparison uses the author's exposition of his research, inspected through its potential and telescoping arguments on 24–25 September 2026. It does not claim an implementation of the associated formal-verification system. PyTorch profiler API documentation was checked on 25 September 2026; the stable URL resolved to version 2.14. No hardware measurements were collected. The roofline constants are assigned illustrative values, and dispatch payload is not total memory traffic.

CHAPTER 17

Traces, credit and mechanistic evidence

In this chapter you will

- distinguish traces, gradients, credit allocations and interventions
- implement structural interventions that recompute descendants
- expose redundancy and synergy with complete response tables
- compute exact Shapley credit and see what it hides

A trace says that a component ran. A gradient says that an output is locally sensitive to a variable. A credit rule assigns a share of a defined value. An intervention asks what changes when a mechanism is deliberately altered. These are different questions, even when their answers are drawn on the same graph. Treating one as another can make a convincing-looking explanation wrong.

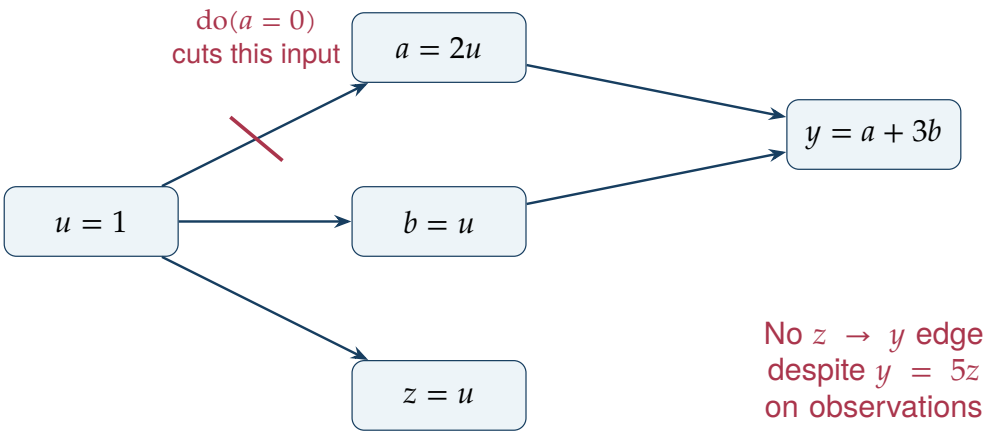
Chapters 15 and 16 gave executions semantics and resource ledgers. This chapter gives explanatory claims a similarly explicit contract. We begin with a synthetic system whose equations are known, implement interventions, and examine where simple attribution rules fail. No trained Evolutor model or experimentally discovered biological circuit is claimed here.

17.1 Observation can fit perfectly and still mislead

Let one externally assigned scalar u determine four internal quantities:

$$a = 2u, \qquad b = u, \qquad z = u, \qquad y = a + 3b. \tag{17.1}$$

The variable z is logged but never used to compute y. On every unmodified run, however, $y = 5z$. A predictor using only z can therefore fit these observations exactly. It has learned a predictive relationship, not a mechanistic dependency of y on z.



Replace the selected equation; preserve other equations; recompute descendants.

Figure 17.1: The declared structural mechanism has a common cause u, two paths into y, and a bystander z. An intervention replaces one equation and cuts its incoming dependency. It does not redraw the remaining equations merely to preserve correlations seen in the original trace.

A **structural intervention** $\mathrm{do}(a = 0)$ replaces $a = 2u$ with $a = 0$ while leaving $b = u$ and $y = a + 3b$ intact. Descendants must then be recomputed. At $u = 1$, the unmodified result is 5 and the intervention result is 3. Replacing the saved value of a in a log after execution would leave the saved y at 5; that is an edited record, not the declared intervention.

```
def evaluate(u, interventions=None):
    """Surgically replace selected equations, then recompute all descendants."""
    intervention = dict(interventions or {})
    if set(intervention) - {"a", "b", "z", "y"}:
        raise ValueError("unknown intervention target")
    if any(not math.isfinite(v) for v in [u, *intervention.values()]):
        raise ValueError("finite values required")
    values = {}
    for name, compute in (
        ("a", lambda: 2 * u),
        ("b", lambda: u),
        ("z", lambda: u),  # logged bystander; not a parent of y
        ("y", lambda: values["a"] + 3 * values["b"]),
    ):
        values[name] = (
            intervention[name] if name in intervention else compute()
        )
    return values
```

The evaluator follows topological order. A supplied intervention value wins over the corresponding equation; other equations use the resulting parent values. Unknown targets and nonfinite values are rejected. Its generated outputs at $u = 1$ are

Intervention	Output
none	5
do(a=0)	3
do(b=0)	2
do(z=0)	5

Intervening on z changes z but not y. Intervening directly on y is also allowed by this model: it replaces the output equation itself. That intervention answers a different question from modifying one of its causes. The target and boundary of an intervention belong in the report.

17.2 Necessity depends on what else remains available

Let two binary components A, B produce $y = A \vee B$. With both active, ablating either one leaves the output at one. It would be wrong to conclude that both components are irrelevant. They are redundant for this input and output criterion. Ablating both changes the answer.

Now let $y = A \wedge B$. Either single ablation changes the output from one to zero, but neither component alone is sufficient. These mechanisms have the same unmodified output and different intervention tables.

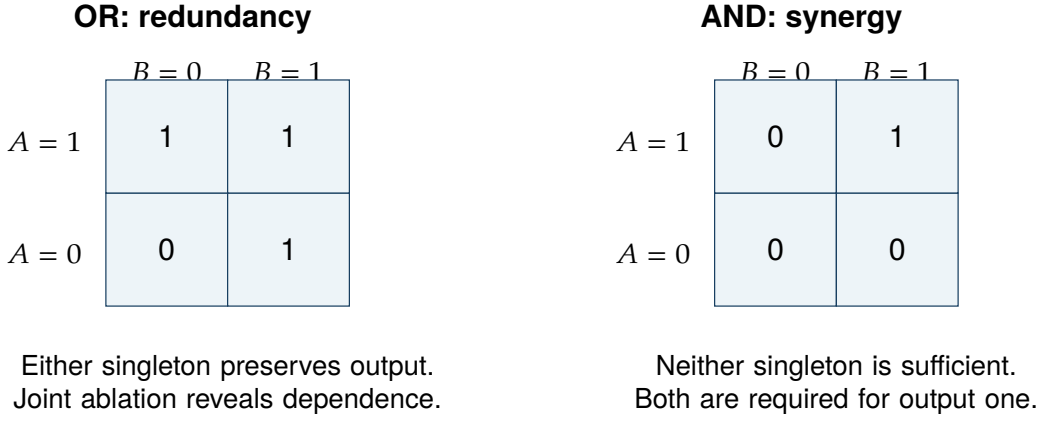


Figure 17.2: Complete binary response tables expose redundancy and synergy. Rows vary A, columns vary B. A single ablation from the upper-right jointly active case cannot characterize all four conditions. Logical response tables are model interventions, not biological knock-out measurements.

For a declared value function $v(S)$, with only components in coalition S enabled, define the two-component interaction

$$I_{AB} = v(\{A, B\}) - v(\{A\}) - v(\{B\}) + v(\emptyset). \tag{17.2}$$

It is -1 for OR and $+1$ for AND. For $v(S) = 2\,1_{A\in S} + 3\,1_{B\in S}$, it is zero. The sign describes departure from additivity under this coalition definition and baseline, not an intrinsic property of a component independent of task or context.

In a neural system, disabling a component also needs a definition: zeroing, mean replacement, resampling from a reference distribution and substituting another example need not have the same effect. Each may create states not encountered during training. A large effect can reveal sensitivity to the intervention artifact rather than the intended feature.

17.3 Derive credit as an average over ordering

Suppose we want to allocate the change $v(N) - v(\emptyset)$ among players in finite set N. For an ordering π, a player's marginal contribution is the value gained when it joins the players preceding it. Average over all orderings:

$$\phi_i = \frac{1}{|N|!} \sum_{\pi} \left[v(P_i^{\pi} \cup \{i\}) - v(P_i^{\pi}) \right]. \tag{17.3}$$

This is a **Shapley allocation** for the specified game. Along each ordering, the marginal terms telescope from the empty to the full coalition. Therefore $\sum_i \phi_i = v(N) - v(\emptyset)$. The efficiency property follows algebraically; it does not establish that the chosen game is causal.

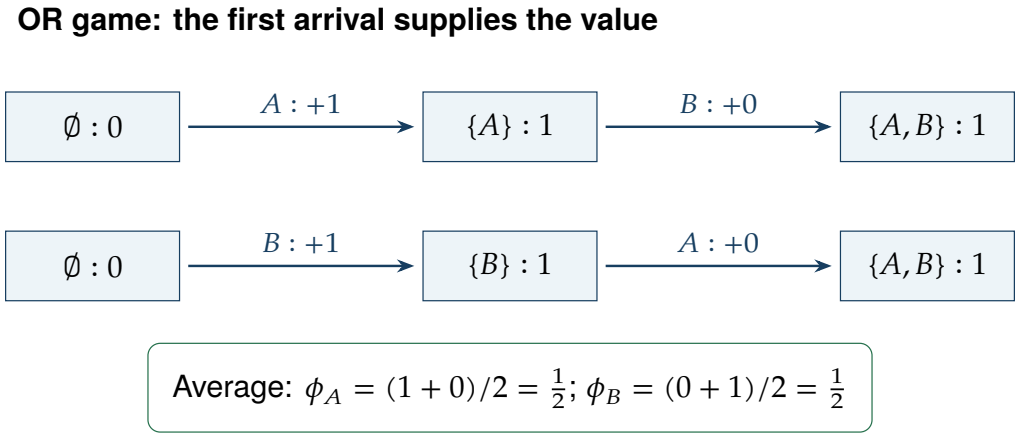


Figure 17.3: Two orderings allocate OR's unit value differently. Averaging assigns one half to each player. For AND the second arriving player receives the unit increment instead; averaging again gives one half each. Identical credit can hide different mechanisms.

```python
def shapley(players, value):
    """Exact permutation average; exponential/factorial teaching scope."""
    if not players or len(set(players)) != len(players):
        raise ValueError("nonempty unique players required")
    credit = dict.fromkeys(players, 0.0)
    count = 0
    for order in permutations(players):
        coalition = frozenset()
        before = value(coalition)
        for player in order:
            coalition = coalition | {player}
            after = value(coalition)
            credit[player] += after - before
            before = after
        count += 1
    return {p: c / count for p, c in credit.items()}
```

The reference requires unique player names and a nonempty player set. It assumes a deterministic, pure value function: the same coalition must mean the same intervention and evaluation. It enumerates permutations, so runtime is factorial rather than a scalable attribution service. Exact enumeration is useful here because it provides an inspectable oracle for small examples.

Game	A credit	B credit	Joint value
redundant	0.5	0.5	1.0
synergistic	0.5	0.5	1.0
additive	2.0	3.0	5

OR and AND both give $(0.5, 0.5)$, although their necessity and sufficiency patterns differ. The additive mechanism gives $(2, 3)$. A constant baseline of seven changes none of these marginal

credits because it cancels in every difference. Reporting credits without reporting the coalition semantics throws away the information needed to interpret them.

The tests independently compute the equivalent subset-weighted expression

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)].$$

The weight counts how many orderings place exactly S before i. Agreement between these implementations, plus efficiency and symmetry checks, guards against indexing mistakes. It does not validate a real system's intervention distribution.

17.4 A trace through time needs timed interventions

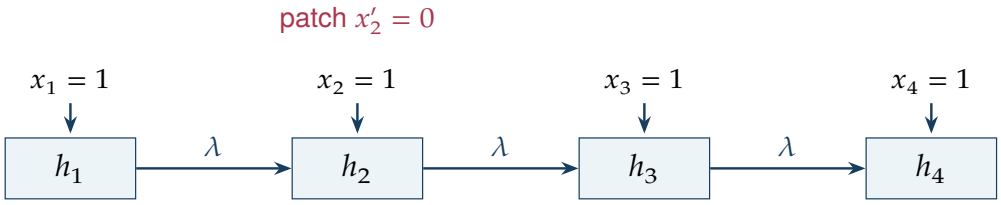
Consider a scalar memory with $h_0 = 0$:

$$h_t = \lambda h_{t-1} + x_t, \qquad 0 \le \lambda \le 1. \tag{17.4}$$

Replace input x_k by x'_k while holding all other inputs fixed and rerunning all later transitions. Let $\Delta_t = h_t - h'_t$. Before k the runs agree; at k, $\Delta_k = x_k - x'_k$; afterward, $\Delta_t = \lambda \Delta_{t-1}$. Thus

$$\Delta_t = (x_k - x'_k)\lambda^{t-k}, \qquad t \ge k. \tag{17.5}$$

This is an exact finite intervention result for this linear mechanism, not a general identity for nonlinear recurrent models.



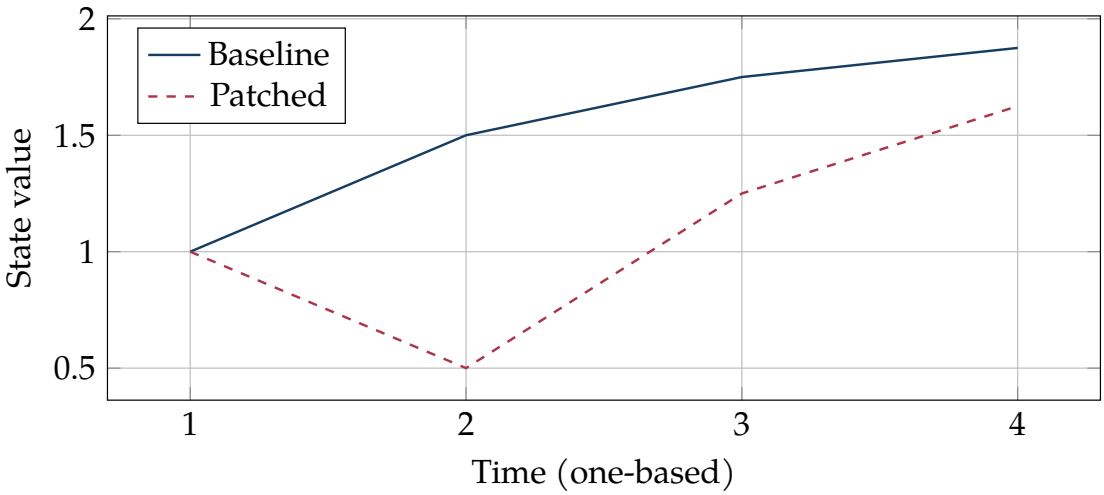


Figure 17.4: A patch to the second input propagates forward through the unrolled recurrence. The generated curves use four unit inputs and $\lambda = 0.5$. Earlier states remain unchanged; later differences decay by one factor of λ per transition.

```
def memory(inputs, retention=0.5, patch=None):
    """Patch one input by (zero-based time, replacement), then rerun."""
    xs = tuple(inputs)
    if not math.isfinite(retention) or not 0 <= retention <= 1:
        raise ValueError("retention in [0,1] required")
    if any(not math.isfinite(x) for x in xs):
        raise ValueError("finite inputs required")
    if patch is not None:
        k, replacement = patch
        if (
            type(k) is not int
            or not 0 <= k < len(xs)
            or not math.isfinite(replacement)
        ):
            raise ValueError("valid patch time and finite value required")
    h, states = 0.0, []
    for t, x in enumerate(xs):
        x = patch[1] if patch is not None and t == patch[0] else x
        h = retention * h + x
        states.append(h)
    return states
```

The code uses zero-based patch indices: `patch=(1,0)` changes the second input. The generated table uses one-based times:

Time	Baseline	Patched	Difference
1	1.0	1.0	0.0
2	1.5	0.5	1.0
3	1.75	1.25	0.5
4	1.875	1.625	0.25

Patching h_k directly is not the same interface as patching x_k. An input patch computes $h'_k = \lambda h_{k-1} + x'_k$; a state patch can assign any admissible h'_k. At later times both differences follow the same scalar propagation only because this model is linear and subsequent inputs are fixed. If future inputs depend on the changed state, the closed loop must be recomputed too.

With $\lambda = 0$, effects vanish after one further step. With $\lambda = 1$, they persist unchanged. A gradient $\partial h_t / \partial x_k = \lambda^{t-k}$ matches the finite effect divided by its magnitude here. In a nonlinear system, gradients are local and can miss threshold crossings, saturation or changed routes. Agreement in this example is a consequence of its equations, not a license to equate gradients with counterfactuals everywhere.

17.5 Test explanations on interventions, not only observations

Return to Equation 17.1. Surrogate $\hat{y} = 5z$ has zero error on all unmodified runs. At $u = 1$, $\mathrm{do}(z = 0)$ makes its prediction zero while the true output remains five. Intervening on a or b also breaks the surrogate's prediction. A perfect observational fit can therefore fail the simplest mechanistic tests.

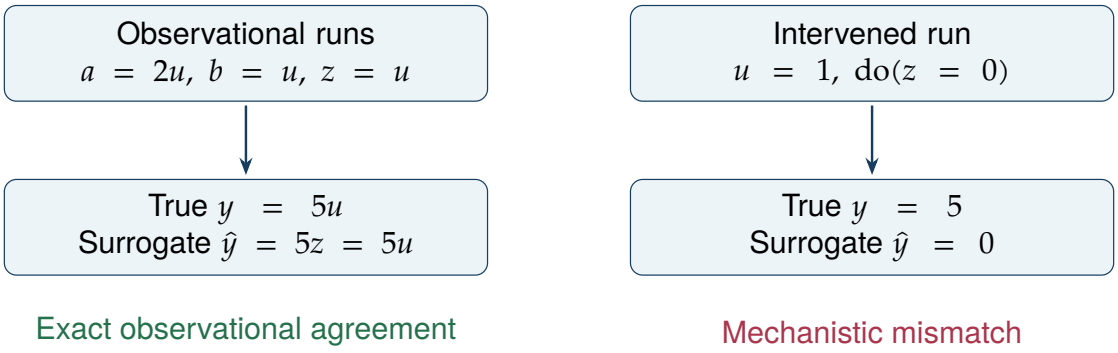


Figure 17.5: An explanation should be evaluated under a declared distribution of interventions. The bystander surrogate fits the observational line exactly but fails when the input mechanism is surgically changed. Intervention coverage is additional evidence, not implied by trace agreement.

A practical evaluation separates development interventions from held-out ones. Declare the output metric and tolerance, intervention targets and magnitudes, input distribution, random seeds, and invalid-state policy. Report failures by intervention family rather than only one average. A surrogate can reproduce single-node patches and still fail joint patches or long-horizon effects. Keeping a list of all failures makes the explanation falsifiable.

Current circuit-tracing work provides a useful comparison: replacement graphs can support interventions, but validation must examine whether those interventions match the underlying model. The 2025 methods report discusses fixed attention patterns, reconstruction errors and limits of replacement-model faithfulness [47]. Our synthetic evaluator avoids those approximation issues by declaring exact equations; that makes it a testing lesson, not an implementation or validation of that research system.

17.6 An evidence contract for Evolutor

For each proposed explanation, store the executable mechanism or exact model revision, the selected events, the intervention operator, the recomputation boundary, the response metric and the test set. A trace hash can bind what was observed; it cannot prove the trace is complete or that an unobserved component had no effect. A successful replay verifies reproducibility under the replay contract, not the truth of every causal interpretation attached to it.

Keep explanatory claims at their earned level. A route log supports which expert was selected. A zero-ablation measures sensitivity to removing its output in that context. A controlled replacement can test a more specific functional hypothesis. None alone establishes biological equivalence, general task competence or a complete decomposition of learned computation.

Key ideas

- A bystander can predict perfectly on observations and fail every intervention: $\hat{y} = 5z$ fits until z is set directly.
- Single ablations cannot separate OR from AND. Both give Shapley credit $(\frac{1}{2}, \frac{1}{2})$ despite opposite interaction signs.
- Editing a logged value is not an intervention. The declared equation must be replaced and its descendants recomputed.
- In a linear recurrence a patched input decays as λ^{t-k}. In nonlinear systems, gradients and finite counterfactuals can disagree.

17.7 Exercises with worked answers

1. Compute $\mathrm{do}(a = 4)$ at $u = 1$. **Answer:** $b = 1$, so $y = 4 + 3 = 7$. The equation for a is replaced, not added to.
2. Set both a and b to zero. **Answer:** $y = 0$ regardless of z. Descendant recomputation uses both replacements.
3. Why is z predictive? **Answer:** It shares cause u with the useful paths. On observations $y = 5u = 5z$, without a $z \to y$ edge.
4. Does zeroing a log field implement a causal patch? **Answer:** No; dependent computations must rerun under the replaced equation.
5. Derive OR's interaction. **Answer:** $1 - 1 - 1 + 0 = -1$. Either component substitutes for the other in this game.
6. Derive AND's interaction. **Answer:** $1 - 0 - 0 + 0 = 1$. The joint contribution exceeds the sum of singleton contributions.
7. Explain identical OR and AND Shapley credits. **Answer:** Each is symmetric with total value one. Averaging over orderings splits that total equally despite different response tables.
8. Add seven to every coalition value. **Answer:** Each marginal difference is unchanged; credits allocate the change above the baseline.
9. Patch the second unit input to zero at $\lambda = 0.5$. **Answer:** State differences at times one through four are $0, 1, 0.5, 0.25$, matching Equation 17.5.
10. Can the patch change time one? **Answer:** Not in this causal recurrence. Such a result would indicate indexing or replay leakage.
11. Construct a failed surrogate test. **Answer:** At $u = 1$, set $z = 0$. True y remains five; $5z$ predicts zero despite perfect observational fit.
12. Design a stronger evaluation than single-node ablation. **Answer:** Hold out inputs, paired interventions, patch magnitudes and temporal horizons; compare predicted and actual response changes with declared tolerances and report where the explanation fails.

17.8 Vocabulary and the next question

Counterfactual execution: a rerun under a specified altered mechanism with the other declared conditions fixed. **Redundancy**: alternative components can preserve a chosen

output. **Synergy**: a joint effect exceeds the additive comparison under a specified baseline. **Mechanistic faithfulness**: agreement of an explanation with the system's responses over the tested interventions, not merely agreement on predictions.

The next chapter defines a DNA-addressed state primitive for DOGMA research. It will earn simple claims through invariants, traces and counterexamples before asking whether a larger trained architecture can make useful use of it.

Sources and evidence

The structural-equation example, intervention evaluator, credit games, temporal-patch derivation, figures and exercises are original teaching artifacts. The Shapley formula is a standard mathematical allocation rule, derived here from permutations and independently checked with subset weights; its invention is not claimed. The synthetic causal graph is stipulated, not inferred from observational data. The circuit-tracing methods page's intervention-validation and limitations discussion was inspected on 25 September 2026, including fixed attention and replacement-model approximation limits. No underlying commercial model, replacement graph or experimental artifact was reproduced or evaluated.

PART IV

DOGMA: non-Transformer DNA-native computation

Can a non-Transformer, DNA-native state organization be specified precisely enough to be falsified?

DOGMA is assembled in five steps: a DNA-addressed state primitive, selective regulation, structured memory with explicit clocks, strand and complement actions, and finally a frozen reference model with a falsification matrix. Each step adds a capability only together with a counterexample it resolves and a test that could reveal its failure. The part ends with honest pilot results. The DOGMA reference sits near chance on a deliberately small motif task, and the chapter explains which experiments would decide whether its mechanisms are worth keeping.

CHAPTERS IN THIS PART

18 **DOGMA primitives and state semantics** Define native state, strand/complement actions and falsifiable hypotheses.

19 **DOGMA regulation and selective transformations** Analyze regulation, input selection and causal state updates.

20 **Structured memory, locality and timescales** Separate local memory, timescales and compressed history.

21 **Strands, complements and dual-state hypotheses** Derive strand symmetry and distinguish offline from causal access.

22 **DOGMA reference model and falsifiable research program** Implement DOGMA reference models and negative-control experiments.

CHAPTER 18

DOGMA primitives and state semantics

In this chapter you will

- define a DNA-addressed state primitive with ordered-pair channels
- separate padding identity, record reset and zero retention
- prove reverse-complement equivariance of the unit-retention histogram and construct its failure under decay
- show by construction that a fixed state can discard order information

Calling a model DNA-native does not define what it computes. Which input symbols select operations? What persists between positions? Does padding advance the state? What happens when a sequence is read from the opposite strand? These questions must have executable answers before an architecture can support meaningful training or benchmarking claims.

We use the book's taxonomy: DOGMA names the non-Transformer, DNA-native research direction and engine; HermonDNA names the Transformer-based direction; Evolutor is the broader framework. This chapter proposes one small state reference for DOGMA research. It is neither a completed DOGMA architecture nor a trained genomic model. Its deliberately limited representation makes its invariants and information loss visible.

18.1 Give each adjacent pair a stable address

Let $\Sigma = \{A, C, G, T\}$ and order the 16 dinucleotides lexicographically using that base order. A memory vector $h \in \mathbb{R}^{16}$ has one channel per pair. Maintain a previous symbol p and valid-position count t. The initial state is $(0, \varepsilon, 0)$.

For a valid incoming base x, use

$$h' = \lambda h + 1_{p \neq \varepsilon} e_{px}, \qquad p' = x, \qquad t' = t + 1, \quad 0 \leq \lambda \leq 1, \tag{18.1}$$

where e_{px} is the one-hot vector selecting the ordered-pair channel. For the first base there is no previous symbol, so nothing is added. The pair selects a memory address, not a biochemical binding reaction. The variable λ is a retention coefficient, not a measured reaction rate.

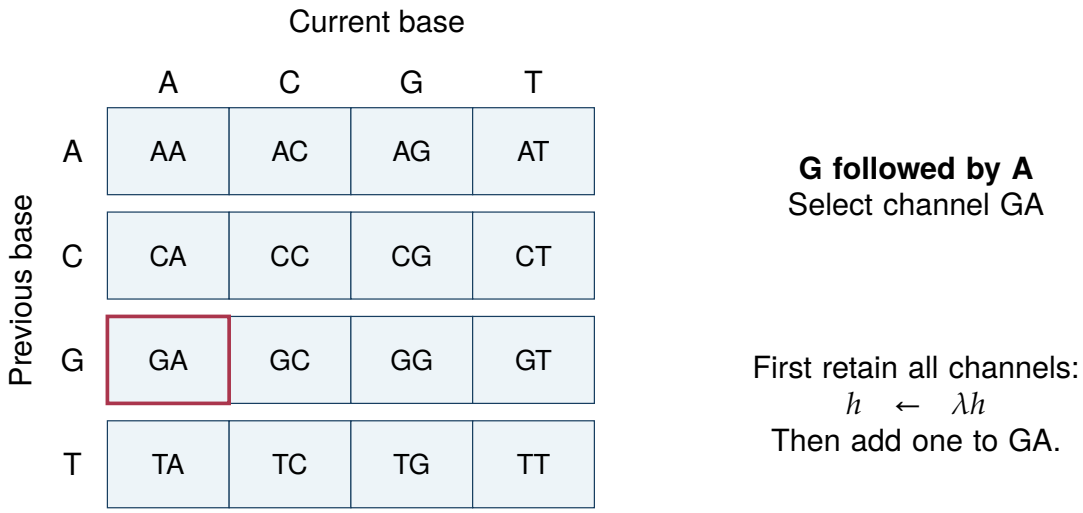


Figure 18.1: Previous and current bases select one of 16 ordered-pair channels. For the transition G followed by A, only GA receives a unit increment after all channels are multiplied by retention. Addressing is directional: GA and AG are distinct channels.

```
@dataclass(frozen=True)
class State:
    memory: tuple = (0.0,) * 16
    previous: str = ""
    position: int = 0
```

The frozen dataclass and tuple memory prevent later steps from overwriting an earlier state. Immutability makes replay comparisons easier; it is not a substitute for validation. The companion rejects negative or nonfinite memory, invalid previous symbols, and inconsistent zero-position state. The state validator checks representation validity, not whether every supplied state is reachable from the initial condition under one fixed λ.

18.2 Implement the transition before adding learned parameters

```
def step(state, symbol, retention=1.0, valid=True):
    validate_state(state)
    if (
        type(valid) is not bool
        or not math.isfinite(retention)
        or not 0 <= retention <= 1
    ):
        raise ValueError("Boolean mask and retention in [0,1] required")
    if not valid:
        return state, None
    if symbol not in tuple(BASES):
        raise ValueError("one canonical DNA base required")
    pair = state.previous + symbol if state.previous else ""
    memory = [retention * x for x in state.memory]
    if pair:
        memory[PAIRS.index(pair)] += 1.0
    nxt = State(tuple(memory), symbol, state.position + 1)
    event = (state, symbol, pair, nxt)
    return nxt, event
```

Every event contains the before state, consumed symbol, selected pair and after state. A trace can therefore be replayed by applying the same transition and checking equality, rather than trusting a textual explanation attached afterward. Padding emits no event and returns the identical state object. An unknown unmasked base is rejected instead of silently converted to A or treated as an absence of data.

For ACGAC with $\lambda = 1$, the generated trace is

Position	Previous	Selected pair	Memory mass
1	A	-	0.0
2	C	AC	1.0
3	G	CG	2.0
4	A	GA	3.0
5	C	AC	4.0

The previous column records the symbol retained *after* each step. The selected pairs are AC, CG, GA and AC. Hence final counts are AC twice, CG once and GA once. Five bases create four adjacent pairs, not five.

The complete scanner is similarly explicit:

```
def scan(sequence, state=None, retention=1.0, mask=None):
    current = State() if state is None else state
    validate_state(current)
    if not math.isfinite(retention) or not 0 <= retention <= 1:
        raise ValueError("retention in [0,1] required")
    active = (True,) * len(sequence) if mask is None else tuple(mask)
    if len(active) != len(sequence):
        raise ValueError("mask length mismatch")
    events = []
    for symbol, valid in zip(sequence, active):
        current, event = step(current, symbol, retention, valid)
        if event is not None:
            events.append(event)
    return current, tuple(events)
```

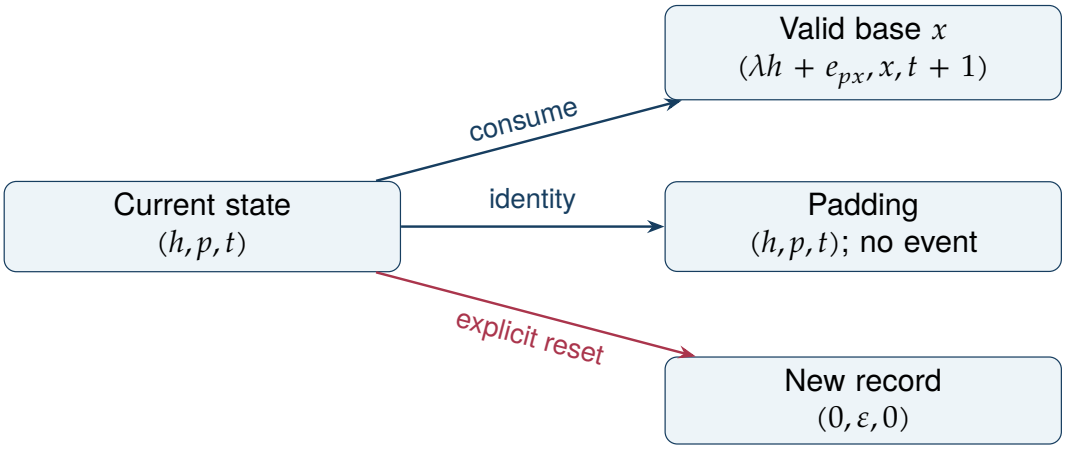
It returns both final state and immutable event sequence. Passing a final state into the next call continues the same record. Calling with no state begins a new record. The function validates retention even on empty input, so invalid configuration cannot be hidden by an empty batch.

18.3 Chunking, padding and reset are different operations

For a fixed λ, sequential composition gives

$$\operatorname{scan}(uv, s) = \operatorname{scan}(v, \operatorname{scan}(u, s)), \tag{18.2}$$

where the inner result denotes its final state. Concatenating the two event sequences also reproduces the uninterrupted trace. This property follows because each step depends only on its current state and symbol, not the call boundary. Changing retention between chunks changes the computation.



At the first valid base ($p = \varepsilon$), omit the pair increment.
A masked slot is not a model of an unknown nucleotide.

Figure 18.2: A valid base updates memory and previous symbol. A padding slot is the identity on all state fields and emits no event. A new biological record instead starts from an explicit fresh state. Skipping an unknown base as padding can create a pair across a real sequence gap.

The mask denotes storage padding, not an uncertain biological nucleotide. For symbols A, N, C with mask true, false, true, the model counts AC because the middle slot is absent from the logical sequence. If N represents an unknown base in a real record, this is usually the wrong semantics: the adjacency of A and C is not known. Alternatives include a separate unknown symbol, an explicit boundary, a distribution over bases or rejection. Each requires a new contract and tests.

Similarly, concatenating two independent records without resetting creates a spurious pair across their boundary. A reset clears memory, previous symbol and position. It is not equivalent to zero retention: at $\lambda = 0$, the previous symbol still creates the next pair. A batching interface should carry record identity separately from a padding mask.

18.4 Separate chemical polarity from string orientation

A DNA duplex is antiparallel. If one strand is written 5′ to 3′ as ACGAC, the aligned complementary strand runs 3′ to 5′ as TGCTG. Reading that complementary strand in its own 5′ to 3′ direction gives GTCGT. This last operation is the **reverse complement**, not just a characterwise complement.

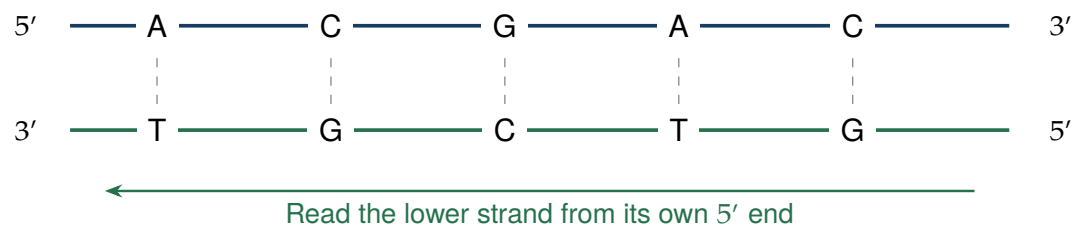


Figure 18.3: The upper and aligned lower strands have opposite chemical polarity. The software reverse-complement string is obtained by reading the lower strand from its 5-prime end. Pair channels must reverse order and complement both bases; complementing without reversal implements a different transformation.

```
def reverse_complement(sequence):
    if any(b not in COMP for b in sequence):
        raise ValueError("canonical DNA sequence required")
    return "".join(COMP[b] for b in reversed(sequence))
```

Let R denote reverse complementation of a sequence and P its channel permutation: pair ab maps to $c(b)c(a)$, where c complements a base. For example, AC maps to GT, GA maps to TC, and CG maps to itself. Applying either transformation twice returns the original: $R^2 = I$ and $P^2 = I$.

```
def permute_pairs(memory):
    if len(memory) != 16:
        raise ValueError("sixteen pair channels required")
    return tuple(memory[PAIRS.index(reverse_complement(p))] for p in PAIRS)
```

18.5 Prove a symmetry, then construct its failure

With $\lambda = 1$, the final memory $H(s)$ is a histogram of adjacent pairs. Every adjacent pair in s corresponds to exactly one reverse-complemented pair in $R(s)$. This bijection proves

$$H(R(s)) = PH(s). \tag{18.3}$$

This is **equivariance**: features change by a known permutation when the input orientation changes. It is not invariance, which would require the feature vector itself to stay unchanged.

Pair	Forward	RC mapped
AC	2.0	2.0
CG	1.0	1.0
GA	1.0	1.0

The generated table maps the reverse-complement memory back to forward channel coordinates before comparing it. Equality without that mapping would compare different biological orientations in different feature bases.

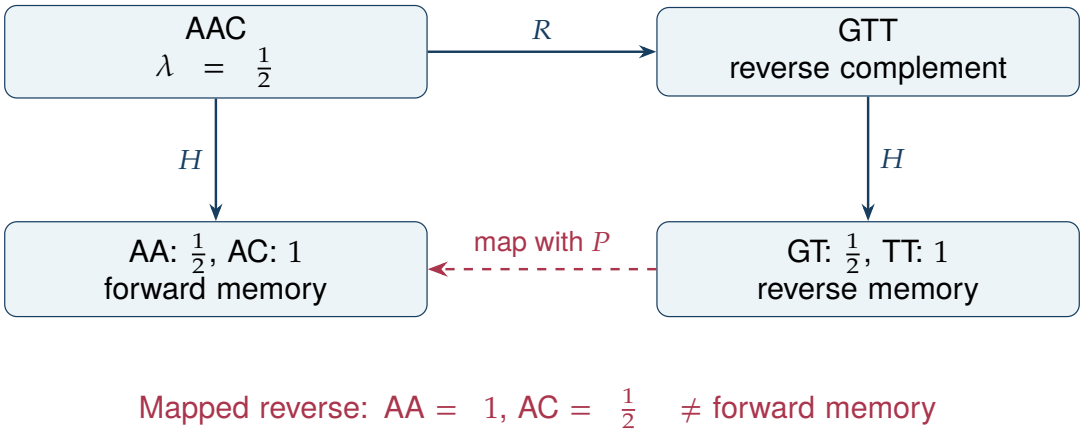


Figure 18.4: For unit retention, each observed adjacent pair contributes equally and reverse complementation permutes the histogram. For AAC at retention one half, older AA receives weight one half and newer AC receives weight one. Reversing orientation reverses their ages, breaking the one-pass symmetry.

For nonunit retention, earlier pairs receive more decay. AAC at $\lambda = 0.5$ has $H_{AA} = 0.5, H_{AC} = 1$. Its reverse complement GTT has $H_{GT} = 0.5, H_{TT} = 1$. Mapping back yields AA equal to one and AC equal to one half, not the forward memory. The addressing rule respects complement mapping, but the direction-dependent weighting does not. A model can fail strand symmetry even when every local base transformation looks correct.

One exact full-record repair is group averaging:

$$Z(s) = \tfrac{1}{2}(H(s) + PH(R(s))). \tag{18.4}$$

Because $P^2 = I$, we have $Z(R(s)) = PZ(s)$ for any H with these channel dimensions. This proof does not require H itself to be equivariant.

```
def bidirectional(sequence, retention=1.0):
    """Full-record feature; not an online causal update."""
    forward = scan(sequence, retention=retention)[0].memory
    reverse = scan(reverse_complement(sequence), retention=retention)[
        0
    ].memory
    mapped = permute_pairs(reverse)
    return tuple((a + b) / 2 for a, b in zip(forward, mapped))
```

The repair reads the full record in both orientations. It is not an online causal update and should not be sold as one. A scalar output $w^\top Z$ is invariant only if its head satisfies the corresponding symmetry, for example $w = Pw$. Strand-specific tasks may instead require equivariant outputs that swap labels. The correct property depends on what the labels mean, not on a universal preference for invariance.

18.6 Memory mass exposes retention and information loss

Let M_n be the sum of all channels after n valid bases from a fresh state. Then $M_0 = M_1 = 0$ and $M_n = \lambda M_{n-1} + 1$ for $n \geq 2$. Consequently,

$$M_n = \sum_{j=0}^{n-2} \lambda^j = \begin{cases} n-1, & \lambda = 1,\ n \geq 1, \\ (1-\lambda^{n-1})/(1-\lambda), & 0 \leq \lambda < 1,\ n \geq 1. \end{cases} \tag{18.5}$$

The empty case is defined separately as zero. For $\lambda < 1$, mass is bounded by $1/(1-\lambda)$, while unit retention accumulates without bound in the exact arithmetic model.

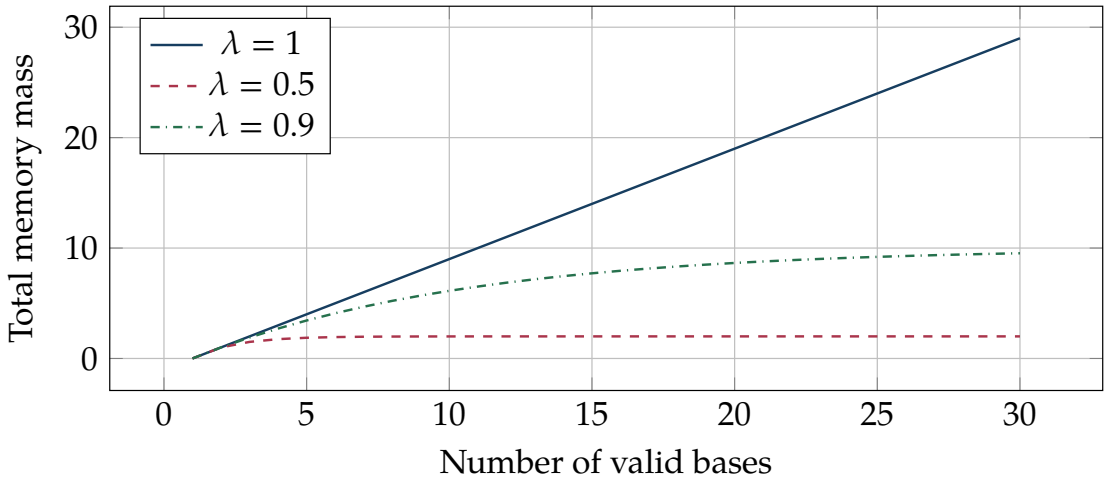


Figure 18.5: Generated memory mass for repeated A inputs at three retention settings. The sequence identity does not affect total mass in this reference; every adjacent pair contributes one before future decay. Bounded mass does not imply that all useful sequence information is preserved.

At unit retention, AACA and ACAA have the same final pair counts (AA, AC, CA once each), the same previous base A and the same position four. The complete final states are identical although the sequences differ. No readout of that state can distinguish them. This is a constructive information-loss proof, not a failure of training. Learning an output head cannot recover information that the state has already discarded.

18.7 Resource and numerical claims need their own tests

The reference updates 16 channels per valid step: $O(16n)$ scalar work for n bases and 16 memory slots in the recurrent vector. Materialized events retain before and after states, so a full trace requires $O(16n)$ stored scalar entries up to sharing and implementation overhead. The position counter also grows: its exact bit length is $O(\log n)$. Fixed vector width is not fixed exact information capacity at arbitrary sequence length.

Python floats cannot retain every unit increment once a channel exceeds the consecutive-integer range of binary64. Fractional retention introduces rounding much earlier, so tests use declared tolerances where needed. An accelerated implementation must compare chunking, masking, reset and orientation behavior against the reference, not merely match one aggregate loss. Vectorizing across records must never leak previous symbols between them.

The tests exhaustively compare pair counts with an independent counter for short sequences, test reverse-complement involution, check all chunk splits at several retentions, verify memory mass, and include the explicit decay counterexample. They establish the toy operator's

contract, not genomic prediction quality, hardware throughput or learned biological mechanisms.

18.8 Connect to research without borrowing its claims

Caduceus distinguishes bidirectional sequence processing from reverse-complement equivariance and constructs symmetry-aware DNA sequence models [48]. That architectural distinction motivates the questions asked here. Our pair histogram and two-pass average are not a Caduceus implementation and carry none of its experimental performance claims. A useful next DOGMA experiment would compare precisely specified state operators under matched data splits, parameter budgets and label transformations before making a performance claim.

The primitive's value is its inspectability. It gives a baseline with exact pair semantics, explicit loss of order, known symmetry conditions and a replay contract. More expressive operators can now be justified by showing which limitation they remove and what new cost or verification burden they add.

Key ideas

- Addresses are ordered: G followed by A selects GA, not AG.
- Equivariance means features transform by a known permutation; invariance means they do not change. Strand-specific labels need the first.
- With retention below one, reversing orientation changes each pair's age and breaks the symmetry. Averaging both orientations repairs it, but it reads the whole record.
- AACA and ACAA have identical final states, so no output head can tell them apart. Training cannot recover discarded information.

18.9 Exercises with worked answers

1. Scan ACG at unit retention. **Answer:** AC and CG each equal one; previous is G and position is three. All other channels are zero.

2. Why is AG not selected by G then A? **Answer:** Addresses are ordered previous–current pairs, so the selected channel is GA.

3. Split ACG after A. **Answer:** Carry previous A into the second chunk; consuming C must count AC. Resetting would lose that pair.

4. Pad between A and C. **Answer:** A masked slot changes nothing; C still creates AC. This assumes the slot is not a biological base.

5. Why is unknown N not ordinary padding? **Answer:** A real unknown position interrupts known adjacency. Skipping it asserts a different logical sequence.

6. Show retention zero is not reset. **Answer:** After A, a valid C with zero retention still adds AC using the retained previous base.

7. Reverse-complement ACGAC. **Answer:** Complement gives TGCTG in the aligned opposite polarity; reversing gives GTCGT.

8. Map channel GA under reverse complementation. **Answer:** Reverse AG and complement to TC. Applying the mapping again gives GA.

9. Explain decay's symmetry failure. **Answer:** Reverse orientation changes the age of each pair, so fixed forward decay assigns different weights even after channel permutation.

10. Prove the averaged feature is equivariant. **Answer:** $Z(Rs) = (H(Rs) + PH(s))/2 = P(H(s) + PH(Rs))/2 = PZ(s)$.

11. Compute mass for four bases at retention one half. **Answer:** $1 + 1/2 + 1/4 = 1.75$, independent of which canonical bases occur.

12. Can an output head distinguish AACA from ACAA at unit retention? **Answer:** Not from this final state: the counts, final base and length all agree. A richer representation or access to the trace is required.

18.10 Vocabulary and the next question

DNA-addressed state: software memory whose update address is selected by declared sequence symbols. **Padding identity**: a masked slot leaves all logical state unchanged. **Record reset**: explicit replacement with an initial state at a biological record boundary. **Bidirectionality**: access to sequence context in both directions, distinct from symmetry under reverse complementation.

The next chapter can add richer context and trainable operators while keeping these contracts visible. Every extra capability should arrive with a worked counterexample it resolves, a reference implementation and a test capable of revealing the new failure modes.

Sources and evidence

The pair-addressed state operator, examples, proofs, implementation and plates are original reference artifacts. DOGMA is a proposed research direction in this chapter; no trained model, genomic capability or throughput result is asserted. Reverse-complement symmetry and group averaging are mathematical properties, not newly claimed inventions. Caduceus v1's architectural discussion, particularly Sections 3.1–3.2, was inspected on 25 September 2026 to distinguish bidirectionality from reverse-complement equivariance. Its reported performance is not used as evidence for this reference operator. No source artwork or prose was copied.

CHAPTER 19

DOGMA regulation and selective transformations

In this chapter you will

- derive a bounded selective update from separate write and retention gates
- use the Chapter 18 collision as a constructive test of order sensitivity
- prove causality and chunk equivalence for input-only coefficients
- show how a state-dependent gate breaks both contraction and the affine scan

Chapter 18 exposed a collision: AACA and ACAA have identical dinucleotide histograms, length and final base. Any output head receiving only that state must give them the same answer. Can an ordered, selectively updated memory separate them without attention? We will construct one such operator, prove what it preserves, differentiate its gates, and then show why these successes still do not establish a useful genomic model.

DOGMA remains the book's proposed non-Transformer DNA-native research direction; Hermon DNA is the Transformer-based direction, and Evolutor is the broader framework. The scalar and vector operators here are explicit research references, not a released foundation model. Their merit is that their assumptions can be tested. Their biological relevance remains an empirical question.

19.1 A regulator chooses coefficients, not an explanation

Order the 16 pairs in $\{A, C, G, T\}^2$ lexicographically in that base order. For pair index i, store candidate vector $v_i \in \mathbb{R}^D$, write logit $u_i \in \mathbb{R}^D$, and retention logit $r_i \in \mathbb{R}^D$. These are three parameter tables of shape $16 \times D$. The recurrent state is $h \in \mathbb{R}^D$. All coordinates are dimensionless software quantities, not concentrations or binding energies.

At position t, the current base and the previous valid base select i_t. There is no pair at the first base. Selection is a deterministic address lookup, not a learned top-k expert decision. Chapter 10 owns that different routing problem. This chapter asks what transformation the selected row performs.

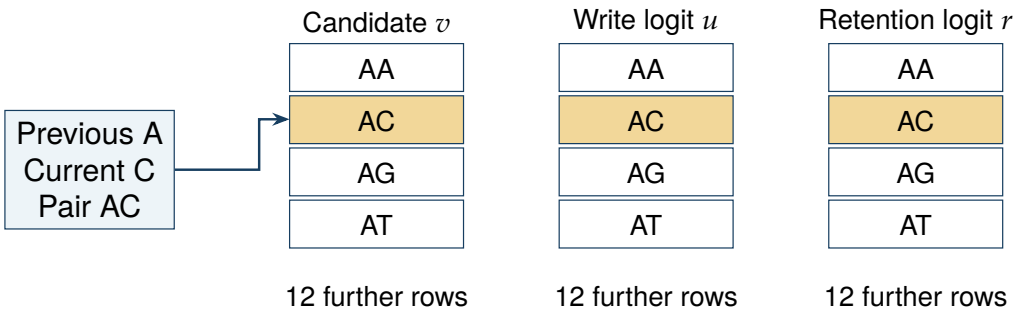


Figure 19.1: A pair selects corresponding rows in three separate parameter tables. The row index is shared, but candidate values, write gates and retention gates have different meanings. This is software addressing; the diagram does not represent complementary molecules binding together.

```
def pair_ids(sequence, previous=""):
    """Compile record symbols to causal adjacent-pair row IDs; -1 means first
     ↪ base."""
    if previous not in ("", *BASES) or any(
        x not in BASES for x in sequence
    ):
        raise ValueError("canonical DNA symbols required")
    ids = []
    for x in sequence:
        ids.append(PAIRS.index(previous + x) if previous else -1)
        previous = x
    return tuple(ids), previous
```

The returned previous base is part of the streaming contract. To continue a record, carry it into the next compilation call alongside the recurrent state. To start an independent record, clear both. Carrying one without the other creates a different model. The example accepts only canonical bases; an unknown biological base is not silently deleted as padding.

19.2 Derive a bounded selective transformation

The logistic function is $\sigma(z) = 1/(1 + e^{-z})$. Define coordinatewise gates

$$g_t = \sigma(u_{i_t}), \qquad \rho_t = \sigma(r_{i_t}), \qquad a_t = 1 - g_t \odot (1 - \rho_t). \tag{19.1}$$

The symbol $\odot$ denotes coordinatewise multiplication. Let $\tilde{h}_t = \rho_t \odot h_{t-1} + (1 - \rho_t) \odot v_{i_t}$ be a candidate state transformation. Interpolate between keeping the old state and applying that transformation:

$$h_t = (1 - g_t) \odot h_{t-1} + g_t \odot \tilde{h}_t \tag{19.2}$$

$$= a_t \odot h_{t-1} + (1 - a_t) \odot v_{i_t}. \tag{19.3}$$

This derivation matters: a write gate does not merely multiply the candidate. It changes the weight of the retained state too. Using $a_t h_{t-1} + g_t v_i$ would define a different, potentially

accumulating update with different bounds. For the first base or a padding slot, define the update as exact identity; an implementation should not approximate identity using a very negative logit.

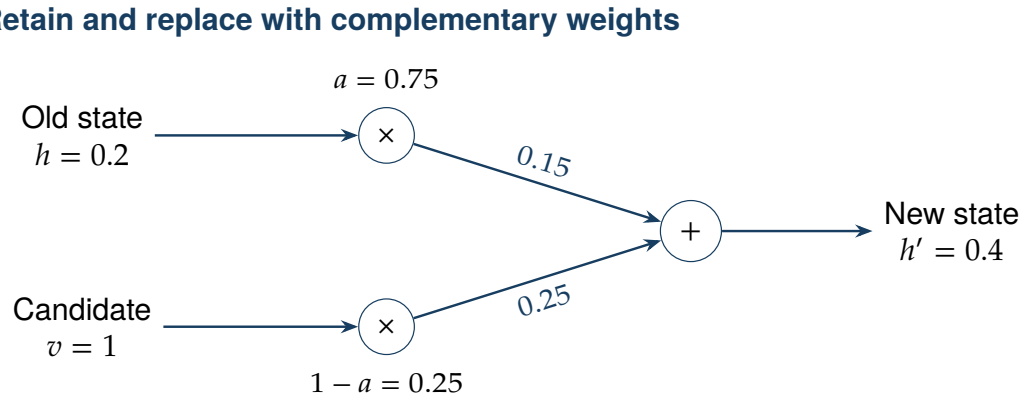


Figure 19.2: The two coefficients sum to one per channel. At $g = \rho = 1/2$, $a = 3/4$; old state 0.2 and candidate 1 produce 0.4. The two paths are arithmetic dependencies, not regulatory proteins.

```
def coefficients(write_logits, retention_logits):
    """Input-only gates: a in (0,1), fresh-candidate weight 1-a."""
    if write_logits.shape != retention_logits.shape:
        raise ValueError("matching gate shapes required")
    gate = torch.sigmoid(write_logits)
    retention = torch.sigmoid(retention_logits)
    a = 1 - gate * (1 - retention)
    return a, gate, retention
```

Proposition 19.1 (Coordinatewise range preservation)**.** *If each initial coordinate and every candidate coordinate lie in* $[-B, B]$ *for finite* B*, then every state coordinate also lies in* $[-B, B]$*.*

Proof. Since $0 \le g, \rho \le 1$, we have $0 \le a \le 1$. Each new coordinate is a convex combination of its old value and the corresponding candidate. The interval is convex, so induction proves the assertion. Identity slots preserve the same interval. □

For fixed finite tables such a B exists, but training unconstrained candidates does not preserve a chosen global bound across optimizer steps. One could parameterize candidates as $B \tanh(z)$; doing so introduces an additional derivative and a modeling choice. Bounded states also do not imply that all histories are distinguishable or that numerical gradients are well conditioned.

19.3 Use the lost-order example as a constructive test

Choose $D = 1$, $v_{AA} = 0$, $v_{AC} = 1$, $v_{CA} = -1$, and all other candidates zero. Set all logits to zero, so $g = \rho = 1/2$ and $a = 3/4$. Begin at $h_0 = 0$. AACA selects AA, AC, CA. Its states after pairs are $0,\ 1/4,\ -1/16$. ACAA selects AC, CA, AA and yields $1/4,\ -1/16,\ -3/64$. The final difference is $-1/64$.

Pair	Write gate	Retention	State
AA	0.5	0.5	0.0
AC	0.5	0.5	0.25
CA	0.5	0.5	-0.0625

Both strings still have the same pair counts. The difference arises because the update is ordered and earlier candidates are attenuated by later retention. It is not evidence that training discovered a motif or that these strings have different biological functions. A useful counterexample fixes the failure being repaired and states exactly which extra mechanism repairs it.

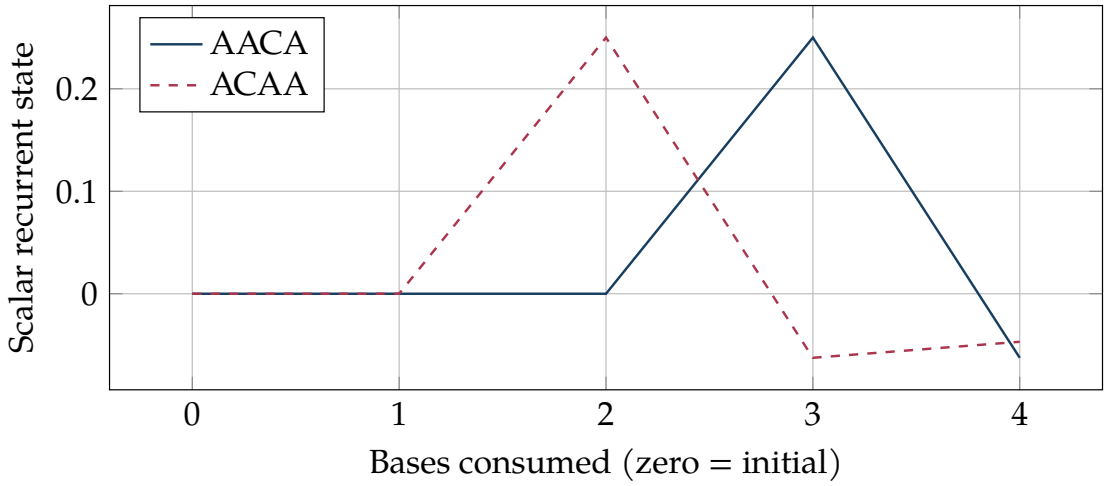


Figure 19.3: Generated traces separate two inputs that collide under the Chapter 18 histogram. Index zero is the initial state and index one is the first base, which creates no pair. The figure demonstrates order sensitivity of one assigned operator, not task accuracy or an optimized architecture.

This update also differs from Chapter 18 at nonunit retention: that reference adds a unit count, whereas Equation 19.3 interpolates toward a candidate. Its mass is therefore not a histogram count. Reusing the same state dimension or DNA address table does not make the state semantics interchangeable.

19.4 Implement one record with explicit ownership

Read the implementation in three stages: validate the whole request, prepare the input-addressed coefficients, then fold the selected transformations over the incoming state. The validation is outside the time loop. It prevents a bad late address from masquerading as a partially successful record; it is not intended as a high-throughput GPU validation strategy.

```
def selective_scan(
    ids, candidates, write_logits, retention_logits, state=None
):
    """One record, tables [16,D], state [D]. -1 is the exact no-pair identity."""
    ids = tuple(ids)
    if candidates.ndim != 2 or candidates.shape[0] != 16:
        raise ValueError("candidate table must have shape [16,D]")
    if any(
        t.shape != candidates.shape
```

```
        or t.dtype != candidates.dtype
        or t.device != candidates.device
        for t in (write_logits, retention_logits)
    ):
        raise ValueError("tables must share shape, dtype and device")
    if not candidates.is_floating_point() or any(
        not torch.isfinite(t).all()
        for t in (candidates, write_logits, retention_logits)
    ):
        raise ValueError("finite floating tables required")
    if any(type(i) is not int or not -1 <= i < 16 for i in ids):
        raise ValueError("pair IDs must be -1 or 0..15")
    h = (
        candidates.new_zeros(candidates.shape[1])
        if state is None
        else state
    )
    if (
        h.shape != candidates.shape[1:]
        or h.dtype != candidates.dtype
        or h.device != candidates.device
        or not torch.isfinite(h).all()
    ):
        raise ValueError(
            "state must match table channels, dtype and device"
        )
    a, _, _ = coefficients(write_logits, retention_logits)
    history = [h]
    for i in ids:
        if i >= 0:
            h = a[i] * h + (1 - a[i]) * candidates[i]
        history.append(h)
    return h, tuple(history)
```

The scanner returns a terminal tensor and a history that includes the initial state. No in-place write alters a caller-supplied state. Table shapes, dtype, device, finite values and pair IDs are validated before processing. A malformed later ID is rejected before any state update, avoiding partially applied work. The ID -1 denotes an exact no-pair operation. A padded storage slot may use the same identity, but the token-to-pair compiler must not advance its previous base for padding. Biological uncertainty requires a separate policy.

The scanner operates on compiled IDs rather than raw strings. This separation makes the state update easy to test, but it also exposes a responsibility: the caller must retain the previous base when compiling chunks. A production state object should bundle that field, the recurrent tensor, record identity, model revision and any position-dependent metadata. The teaching function does not supply a serving runtime merely because it accepts a state argument.

19.5 Prove causality and chunk equivalence

The pair at t depends only on bases up to t. The three selected table rows are functions of that pair and fixed model parameters. By induction, h_t depends only on the valid prefix through t and the initial state. If h_t predicts the next base, that is a causal next-token feature. If the target is the current base, feeding that same base into the feature before prediction is target leakage; causality alone does not settle label alignment.

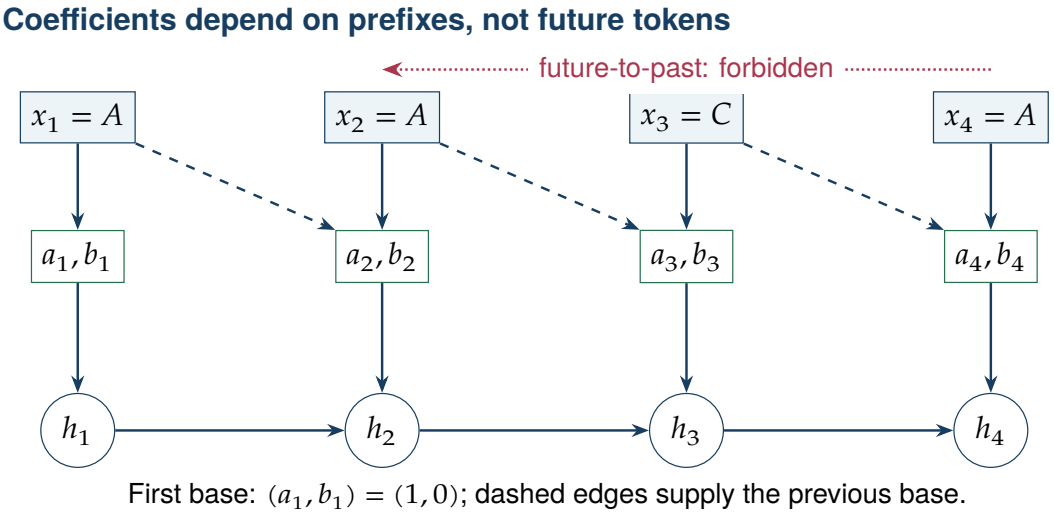


Figure 19.4: Gate coefficients can be prepared from known token pairs, while state values depend on earlier states. A future base must not contribute to an earlier predictive state. Changing the computation order does not permit changing these information dependencies.

For a fixed sequence of coefficients, the update is affine in the incoming state. The composition of (a_1, b_1) followed by (a_2, b_2) is

$$(a_2 \odot a_1,\ a_2 \odot b_1 + b_2), \qquad b_t = (1 - a_t) \odot v_{i_t}. \tag{19.4}$$

Chapter 7 proved associativity. Here the additional obligation is to establish that the coefficients are available without first evaluating the state they will update. Input-only pair lookup satisfies that obligation. It permits an affine prefix implementation; it does not prove that any particular GPU scan will be faster than sequential execution for a given size and precision.

Chunk equivalence follows by composing the same transitions in the same order and carrying the boundary state and previous base. Tests compare every split of short records and also compare gradients through the split computation. Detaching a chunk boundary can preserve forward values while changing the training objective's gradient. Matching output alone is therefore insufficient for a training-parity claim.

19.6 Differentiate the selected operation

For one scalar channel, hold the selected pair fixed. Write $h' = ah + (1 - a)v$, $g = \sigma(u)$ and $\rho = \sigma(r)$. Then

$$\frac{\partial h'}{\partial h} = a, \qquad \frac{\partial h'}{\partial v} = g(1 - \rho), \tag{19.5}$$

$$\frac{\partial h'}{\partial u} = g(1 - g)(1 - \rho)(v - h), \tag{19.6}$$

$$\frac{\partial h'}{\partial r} = g\rho(1 - \rho)(h - v). \tag{19.7}$$

To see the signs, increasing write strength moves toward v, while increasing retention moves toward h. If $h = v$, both gate derivatives vanish: changing the mixture cannot change its value. Saturated sigmoids also have small derivatives. A near-zero gradient may indicate this local geometry rather than a broken implementation or a useful learned biological switch.

Unselected rows receive no contribution from that event, but a row used several times accumulates gradient contributions from all those times. This is ordinary parameter sharing,

not one independent parameter per occurrence. The reference is checked against explicit scalar derivatives, central finite differences and an independently expanded recurrence. These checks establish arithmetic consistency; they do not establish that gradient descent will find a good model.

For input-only coefficients, the state-to-state Jacobian across positions is

$$\frac{\partial h_t}{\partial h_s} = \operatorname{diag}\left(\prod_{j=s+1}^{t} a_j\right). \tag{19.8}$$

No coordinate amplification occurs through this path when $a_j \in [0, 1]$, but products can vanish. Candidate and gate parameter gradients include their own local factors and repeated-use sums. It would be incorrect to extend the state-path bound to every parameter gradient without analyzing those terms.

19.7 A feedback gate changes both the proof and the scheduler

Suppose a regulator reads the old state, rather than only the input. For one channel let $g(h) = \sigma(wh)$ and set $f(h) = (1 - g(h))h + g(h)v$. Differentiation gives

$$f'(h) = 1 - g(h) + (v - h)wg(h)(1 - g(h)). \tag{19.9}$$

At $h = 0$, $v = 1$ and $w = 8$, $f'(0) = 1/2 + 8/4 = 2.5$. The output is still a convex combination of h and v, yet nearby inputs can separate locally. Bounded values are not the same property as contraction.

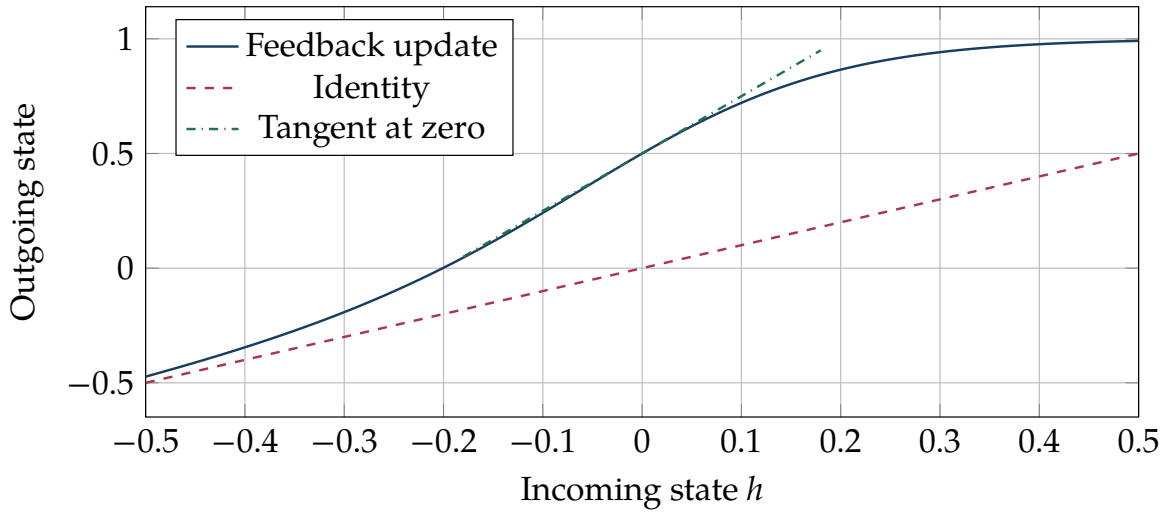


Figure 19.5: The declared feedback-gated map near zero has slope 2.5, whereas the identity has slope one. The gate's derivative is part of the Jacobian. Treating a computed gate as a constant would miss the amplification.

```
def feedback_step(h, weight=8.0):
    """Counterexample: bounded candidate, state-dependent gate, noncontracting
     ↪  map."""
    gate = torch.sigmoid(weight * h)
    return (1 - gate) * h + gate
```

Once coefficients depend on h_{t-1}, they cannot generally be prepared as a token-only affine scan input. Function composition remains associative in the abstract, but there need not be a closed,

fixed-size affine representation of those composed functions. A claim of parallel execution now needs a different algorithm or additional structure. This distinction prevents a common category error: importing a scan theorem after changing the regulator's dependencies.

19.8 Gate interpretations may be non-identifiable

The forward operator uses g and ρ only through $a = 1 - g(1 - \rho)$. For example, $(g, \rho) = (1/2, 1/2)$ and $(3/4, 2/3)$ both give $a = 3/4$. They can be represented by finite logits $(0, 0)$ and $(\log 3, \log 2)$. With identical candidates, every input and incoming state produces the same forward result. Output fitting alone therefore cannot identify these two mechanistically named gates separately.

This is structural **non-identifiability**, not merely too little training data. Separate gate semantics would need additional constraints, independent observations, or a different architecture. A regularizer can choose one parameterization but cannot by itself prove a biological interpretation. Displaying a gate trace is useful for debugging; calling it a discovered regulatory mechanism requires evidence beyond its name.

Variant	AACA	ACAA	Difference
baseline	-0.0625	-0.046875	-0.015625
write near zero	-9.43772e-12	-9.43769e-12	-2.89935e-17
retention near one	-9.43772e-12	-9.43769e-12	-2.89935e-17

The table fixes candidates and changes one logit family at a time. Near-zero write gates and near-unit retention both make state changes tiny. Neither intervention is exactly identity at finite logits. These are functional ablations of an assigned model, not trained accuracy results. A stronger study would retrain matched alternatives, preserve budgets, repeat seeds and report both utility and cost with uncertainty.

19.9 Place the operator against current research

Mamba makes selected state-space parameters input dependent and provides a hardware-aware execution strategy [10]. That motivates separating selection semantics from the implementation used to evaluate them. Our pair table and convex interpolation are not a Mamba block and inherit none of its benchmark results. They also make a different representational choice: a finite pair address is much narrower context than a learned contextual feature.

The 2026 Evo 2 paper describes a multi-hybrid architecture combining several input-dependent convolution operators and attention [49]. Its existence is a reason to compare complementary mechanisms carefully, not to declare a non-Transformer direction automatically superior. Evo 2 is an external model, not this book's Evolutor project. We do not equate a similar name with shared code, architecture, training data or validation.

An academically useful DOGMA claim should name the task, input and label semantics, hypothesis, baselines and failure criterion. An industrially useful claim additionally needs stable model/state versions, record isolation, memory and latency measurements on specified hardware, failure recovery and reproducible artifacts. Neither standard is met by synthetic arithmetic alone. The present operator supplies a small part of the correctness foundation on which those experiments could be built.

19.10 A falsifiable evaluation contract

For a benign sequence-prediction or regulatory-label study, first declare whether strand reversal preserves a scalar label or permutes strand-specific labels. Our directional recurrence is not automatically reverse-complement equivariant. The full-record averaging construction in Chapter 18 can restore a specified symmetry, but it changes access and streaming assumptions.

Then separate homologous or duplicated records before constructing train, validation and held-out test sets; otherwise related sequences can produce misleading generalization. Choose the split rule for the scientific question: held-out loci, individuals, chromosomes or species answer different questions. Fit normalization and selection rules on training data only, and keep model selection separate from final reporting. This is a proposed evaluation contract, not a claim that the reference has passed a genomic benchmark.

Compare pair histograms, fixed-retention recurrence, input-conditioned recurrence and a suitably budgeted sequence baseline. Record parameter count, valid tokens, training compute, peak memory, preprocessing and uncertainty across independent runs. Measure gate interventions separately from retrained ablations. Report failure examples, including long gaps, changed base composition, record boundaries and reverse-complement transformations. A frontier research program needs experiments that can reject its favored explanation.

The reference stores $48D$ scalar parameters and D recurrent values, plus the previous-base field in its caller. Forward arithmetic is $O(nD)$ after address compilation. Keeping every state for this teaching trace costs $O(nD)$; fixed recurrent width is not constant training memory. Dense Python validation, small gathered operations and autograd overhead also matter to runtime. An operation count is not measured throughput, and no optimized kernel is delivered by this chapter.

Key ideas

- A write gate reweights the retained state as well as the candidate: $a = 1 - g(1 - \rho)$.
- Convex updates keep every coordinate within the range of its inputs. A bounded state is not the same as a contraction.
- Once a gate reads h_{t-1}, its coefficients cannot be prepared from tokens alone, and the scan theorem no longer applies.
- $(g, \rho) = (\frac{1}{2}, \frac{1}{2})$ and $(\frac{3}{4}, \frac{2}{3})$ give the same forward map. Fitting outputs cannot identify the gates separately.

19.11 Exercises with worked answers

1. With $g = \rho = 1/2$, calculate a. **Answer:** $3/4$; the candidate weight is $1/4$, not the write gate $1/2$.

2. Update $h = 0.2$ toward $v = 1$. **Answer:** $0.75(0.2) + 0.25 = 0.4$.

3. Why do AACA and ACAA differ here? **Answer:** Their candidate orders differ. The final values are $-1/16$ and $-3/64$, despite equal pair counts.

4. What must cross a raw-sequence chunk boundary? **Answer:** Recurrent state and previous valid base, with the same tables and record semantics.

5. A detached boundary matches forward values. Is training parity proved? **Answer:** No; detachment removes earlier gradient paths. Check gradients too.

6. Differentiate with respect to u at $h = 0, v = 1, g = \rho = 1/2$. **Answer:** $(1/2)(1/2)(1/2) = 1/8$.

7. Differentiate with respect to r in the same case. **Answer:** $(1/2)(1/2)(1/2)(-1) = -1/8$: retention suppresses the new value.

8. Show non-identifiability using finite logits. **Answer:** $(u, r) = (0, 0)$ and $(\log 3, \log 2)$ both produce $a = 3/4$.

9. Why does the feedback example defeat a contraction claim? **Answer:** Its total derivative at zero is 2.5, including the gate path; the direct retention coefficient alone is only 0.5.

10. Does every nonlinear recurrence support the affine scan? **Answer:** No. A state-dependent regulator generally invalidates token-only coefficient preparation and closure of the fixed-size affine representation.

11. Count parameters for $D = 8$. **Answer:** $3 \cdot 16 \cdot 8 = 384$, excluding an output head and any richer context encoder.

12. Design a test of the claim that selective gates improve genomic utility. **Rubric:** Fix task and label semantics, leakage-resistant splits, matched baselines and resource budgets; compare trained controls and interventions, multiple seeds, uncertainty and out-of-distribution failures. Gate visualization or one synthetic separation is insufficient evidence.

19.12 Vocabulary and next question

Input-conditioned coefficient: an update parameter computed from the available input, not from a yet-unknown recurrent value. **State-path Jacobian**: sensitivity propagated through recurrent states. **Gradient parity**: agreement of derivative semantics, beyond forward values. **Functional ablation**: changing an operation while holding a declared parameter set fixed. Chapter 20 asks how to give different memory components different locality and timescales without hiding their costs or ownership.

Sources and evidence

Access checked 27 September 2026. Mamba arXiv v2: Sections 3.1–3.3 inspected for input-dependent coefficients and execution dependencies. Evo 2: publisher abstract and architecture/training section inspected, particularly the mixture of convolutional operators and attention. Neither paper's reported benchmark numbers are used as evidence for DOGMA. Full experimental reproduction, training-data auditing and independent hardware benchmarking were not performed.

The proposed pair-table operator, worked counterexamples, implementation, derivations and figures are original teaching artifacts. Convexity, logistic derivatives and affine composition are standard mathematics, not novelty claims. The small functional ablations are not genomic validation or production-engine results. The shared book front matter owns the general pedagogical acknowledgment; it is not repeated here. Independent specialist and reader review remain open.

CHAPTER 20

Structured memory, locality and timescales

In this chapter you will

- derive retention from a declared clock and convert it to e-fold time and half-life
- build a multiscale exponential bank and a block-rate coarse summary
- identify the complete carried state needed for exact chunk parity
- prove limits on lossless memory with an aliasing example and a counting argument

A model reads a long DNA record. It must preserve an exact recent motif, track a slowly changing compositional signal, and avoid mixing one record with the next. Should all three jobs belong to the same vector? A single state can be useful, but naming it memory does not specify which information it retains, which queries it can answer, or when it must reset.

Chapter 19 defined one selectively updated DOGMA reference. This chapter separates memory representation, retention and update clocks. We construct a small differentiable bank and a block-rate summary, prove their behavior, and expose failures that a final-state equality test can miss. DOGMA remains a proposed non-Transformer DNA-native research direction. The code below is a reference operator, not a trained genomic model or a biological mechanism.

20.1 Begin with the query a memory must answer

Three requests illustrate different contracts. Returning the last four canonical bases requires an ordered local buffer. Estimating a weighted recent feature average can use a compressed state. Retrieving the feature associated with a particular earlier location needs some notion of addressing and retention of that association. A long decay constant does not by itself provide an addressable record.

Fix an input feature sequence $u_t \in \mathbb{R}^D$. The features might come from Chapter 19's DNA-pair table, but here we take them as explicit inputs so memory behavior can be tested separately from the encoder. The reference state will contain an exponential bank $H \in \mathbb{R}^{K \times D}$, a coarse summary $s \in \mathbb{R}^D$, a pending sum $p \in \mathbb{R}^D$, and an integer phase q. A recent raw-token buffer is a contrasting design, not an unimplemented feature that the code secretly provides.

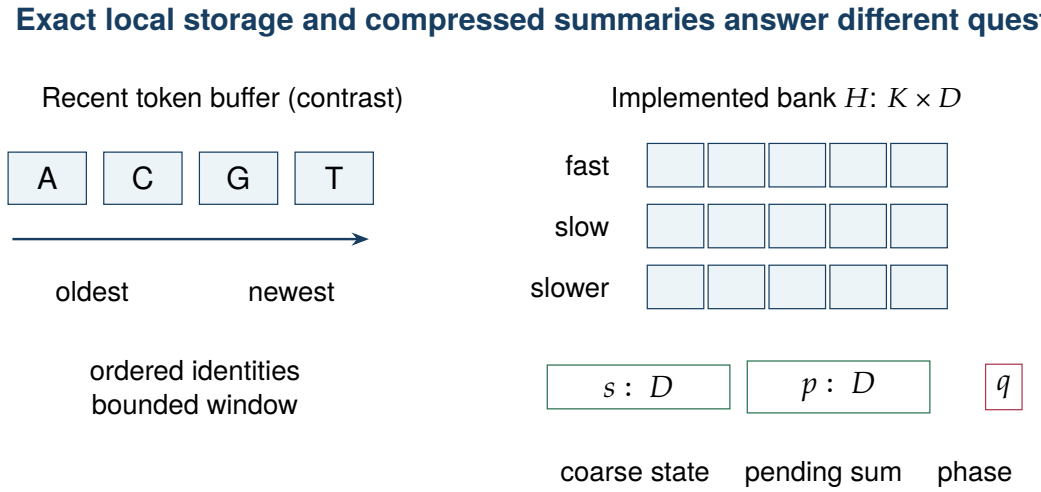


Figure 20.1: Memory layout expresses different promises. Ordered raw token slots support exact recent recall; the numerical bank supports weighted summaries. The implemented reference contains the bank, coarse state, pending sum and phase, not the illustrative token buffer.

Locality must also be named precisely. Adjacent genomic coordinates, neighboring positions in an encoded stream, adjacent tensor addresses, and recent update events are different neighborhoods. A tokenizer that combines several bases changes the relation between token distance and base-pair distance. Packed records can even make two adjacent tensor slots belong to different chromosomes or organisms. Memory ownership cannot be inferred from storage adjacency.

20.2 Derive a timescale from a declared clock

For one channel, consider the relaxation equation $dh/d\xi = (u - h)/\tau$, where ξ is a declared clock coordinate and u is constant over an interval of length $\Delta \geq 0$. Here $\tau > 0$ has the same units as Δ. Solving the equation gives

$$h_{\text{new}} = a h_{\text{old}} + (1 - a)u, \qquad a = e^{-\Delta/\tau}. \tag{20.1}$$

The clock could count valid tokens, genomic distance, or elapsed time. The equation alone does not choose among them. For a valid-token clock, we use $\Delta = 1$ on each valid feature and do nothing for padding. An unobserved stretch of genome is not automatically padding: if distance matters, it may need a nonzero Δ and a separately specified input policy.

The **e-fold time** is τ: a perturbation of the incoming state is multiplied by e^{-1} after this much elapsed clock distance under a fixed coefficient. The half-life is $\tau \log 2$. For constant per-event retention $a \in (0, 1)$, $\tau = -1/\log a$. These are sensitivity timescales, not measured biological half-lives or guaranteed task-relevant context lengths.

```
def retention(taus, delta=1.0):
    """taus and delta are measured in the same declared clock units."""
    if (
        taus.ndim != 1
        or taus.numel() == 0
        or not taus.is_floating_point()
    ):
```

```
        raise ValueError("nonempty floating timescale vector required")
    if not torch.isfinite(taus).all() or (taus <= 0).any():
        raise ValueError("positive finite timescales required")
    if not math.isfinite(delta) or delta < 0:
        raise ValueError("finite nonnegative elapsed distance required")
    exponent = -delta / taus
    return torch.exp(exponent), -torch.expm1(exponent)
```

Computing the write weight as $-\operatorname{expm1}(-\Delta/\tau)$ avoids subtracting two nearly equal numbers when the interval is very short. In exact arithmetic it equals $1 - a$. In floating point the separately computed coefficients may not sum to exactly one; tests use stated numerical tolerances. We do not promise bitwise equality across arbitrary kernels.

Retention	e-fold time	Half-life	Weight now
0.5	1.442695	1.0	0.5
0.875	7.488876	5.190893	0.125

Retention 7/8 forgets an existing perturbation more slowly than 1/2, but its immediate response to a fresh input is only 1/8 rather than 1/2. Longer retention trades immediate write amplitude for persistence in this normalized update. Simply increasing retention is not a free improvement.

20.3 Unroll the state and inspect what it remembers

For a fixed a and initial h_0,

$$h_t = a^t h_0 + \sum_{j=1}^{t} (1-a) a^{t-j} u_j. \tag{20.2}$$

The proof is substitution: multiply the previous expression by a and add $(1-a)u_t$. The input at lag $\ell = t - j$ has weight $w_\ell = (1-a)a^\ell$, with $\sum_{\ell=0}^{\infty} w_\ell = 1$. For a constant input and zero initial state, $h_t = (1 - a^t)u$. For one unit impulse followed by zeros, the response is exactly the kernel.

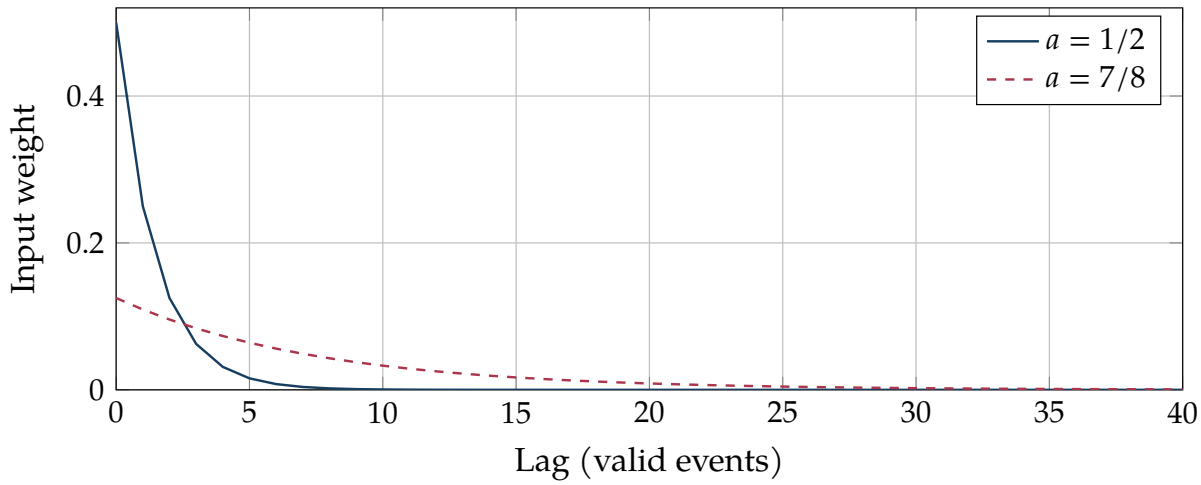


Figure 20.2: Two generated impulse kernels use the same unit input and zero initial state. Lag zero is the state immediately after that input. The slow bank writes less initially and retains a longer tail; neither curve is a genomic accuracy measurement.

With K timescales, update row k of the bank as $H_{k,t} = a_k H_{k,t-1} + (1 - a_k)u_t$. Each row summarizes the same feature history with a different kernel. A readout such as $y_t = \sum_k w_k H_{k,t}$ can combine those summaries. It cannot recover information that every row has discarded. If the readout weights are unconstrained, the readout need not be a convex average even though every bank row is one.

Proposition 20.1 (A finite bank is a bounded summary)**.** *If every input coordinate and initial bank coordinate lies in $[-M, M]$, then every bank coordinate remains in that interval in exact arithmetic. For two runs with the same inputs, their difference in row k after t unit-clock events is a_k^t times its initial difference.*

Proof. Each update is a convex combination, proving the range bound by induction. Subtract the recurrences of the two runs; the input terms cancel. Iterating $\delta H_{k,t} = a_k \delta H_{k,t-1}$ gives the second claim. □

The second statement concerns perturbations of the initial state, not arbitrary parameter gradients or separability of every pair of histories. Chapter 19 already showed how a state-dependent gate can invalidate this simple contraction argument. A bank of fixed timescales is deliberately chosen here so the memory kernel is inspectable.

20.4 Slow updates are not the same as slow retention

A second mechanism updates a coarse state only after a block of B valid features. Within the current block retain a sum p and phase q. For each valid input,

$$p \leftarrow p + u_t, \qquad q \leftarrow q + 1. \tag{20.3}$$

If $q = B$, commit

$$s \leftarrow \rho s + (1 - \rho)\frac{p}{B}, \qquad p \leftarrow 0, \quad q \leftarrow 0, \quad 0 \le \rho \le 1. \tag{20.4}$$

Otherwise s is held exactly. Padding changes none of these fields. The block boundary is determined by valid-event count, not by how a loader cuts a record into transport chunks.

After m complete blocks, an initial coarse-state perturbation is ρ^m times its original value. At block-aligned positions, an equivalent per-token envelope is $a_{\text{eff}} = \rho^{1/B}$. Between commits, however, the state is constant. Thus a per-token exponential with retention a_{eff} is not the same operator: it responds earlier, weights positions inside a block differently, and has different gradient paths.

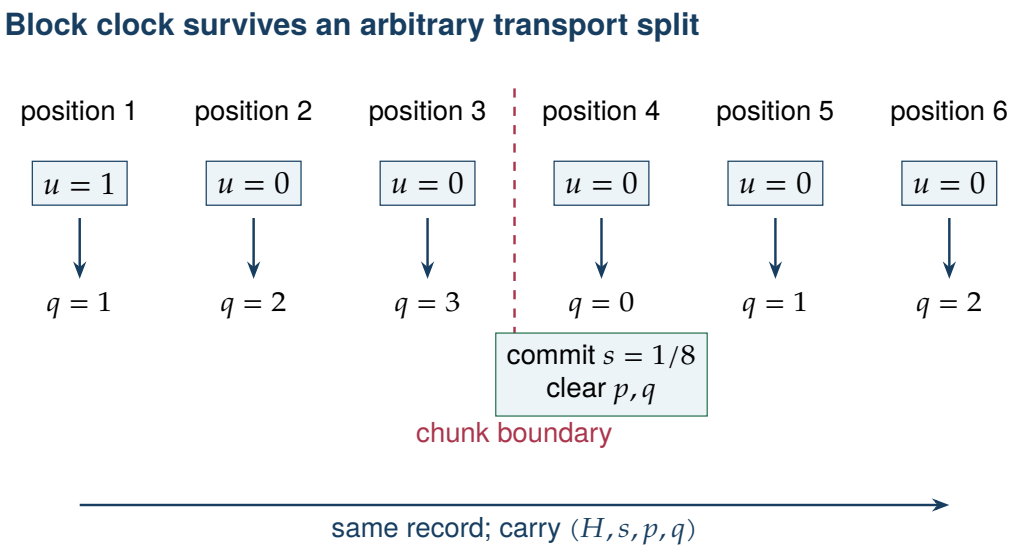


Figure 20.3: A chunk ending after position 3 carries phase 3 and the pending sum. The fourth valid position completes the block and triggers the coarse update. The transport boundary does not reset or flush the model's clock. A read before the commit cannot use a future block summary.

For $B = 4$, $\rho = 1/2$, and input $(1, 0, 0, 0, 0, 0, 0, 0)$, the first block mean is $1/4$ and the coarse state becomes $1/8$. It remains there until the next commit, then becomes $1/16$. The two exponential rows respond on every valid position instead.

Position	Fast	Slow bank	Block state	Phase
0	0.0	0.0	0.0	0
1	0.5	0.125	0.0	1
2	0.25	0.109375	0.0	2
3	0.125	0.095703	0.0	3
4	0.0625	0.08374	0.125	0
5	0.03125	0.073273	0.125	1
6	0.015625	0.064114	0.125	2
7	0.007812	0.056099	0.125	3
8	0.003906	0.049087	0.0625	0

What happens to a partial block at the end of a record? Our reference retains it without committing. A caller may inspect it as part of the complete state, or discard it when the record is finished. An explicit finalization operation could average by the actual count, but that would be a different output contract. Flushing every transport chunk is not valid finalization.

20.5 Implement the complete carried state

```
@dataclass(frozen=True)
class Memory:
    bank: torch.Tensor  # [K,D]
    coarse: torch.Tensor  # [D]
    pending: torch.Tensor  # [D], sum since last complete block
    phase: int  # valid positions since last commit
```

The container's fields name the sufficient state for this recurrence. The frozen dataclass prevents rebinding its attributes; it does not make PyTorch tensors deeply immutable. The implementation performs no in-place updates, and callers must not mutate tensors retained for later continuation or gradient computation.

Before executing, the validator checks feature shape $[T, D]$, positive timescales $[K]$, shared dtype/device, a Boolean mask, the allowed phase, and the shapes of all carried tensors. An empty pending block must have zero sum. It checks even masked values for finiteness so NaNs are not silently hidden. This is a correctness-oriented reference, not a device-efficient validation kernel.

```
def multiscale(
    values, taus, block_size=4, rho=0.5, mask=None, carry=None
):
    """Valid-token clock. Padding holds all fields; a chunk never flushes."""
    mask = (True,) * len(values) if mask is None else tuple(mask)
    validate(values, taus, block_size, rho, mask, carry)
    k, d = len(taus), values.shape[1]
    state = (
        carry
        if carry is not None
        else Memory(
            values.new_zeros(k, d),
            values.new_zeros(d),
            values.new_zeros(d),
            0,
        )
    )
    a, write = retention(taus)
    history = [state]
    for value, valid in zip(values, mask):
        if valid:
            bank = a[:, None] * state.bank + write[:, None] * value
            pending, phase = state.pending + value, state.phase + 1
            coarse = state.coarse
            if phase == block_size:
                coarse = rho * coarse + (1 - rho) * (pending / block_size)
                pending, phase = torch.zeros_like(pending), 0
            state = Memory(bank, coarse, pending, phase)
        history.append(state)
    return state, tuple(history)
```

The helper `validate` is included in full in the companion source, along with a separately expanded convolution oracle. The build extracts whole functions for printed listings. No line-number fragment is maintained independently of the executable file.

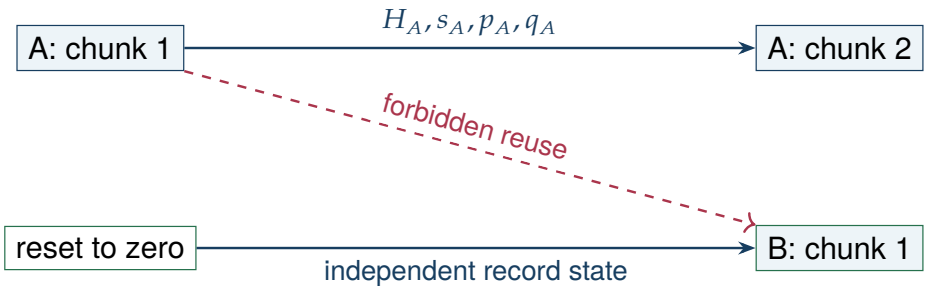
The returned history includes the initial state and one state per input slot. It is convenient for teaching and testing, but storing this history costs $O(TKD)$ values. A streaming inference implementation that retains only the terminal state can have bounded carried storage; that is not a claim that training through the reference uses constant memory.

20.6 Prove chunk parity, then test the missing-field failure

Each valid event maps (H, s, p, q) deterministically to its successor using fixed configuration (τ, B, ρ). A masked slot is the identity map. Applying the same sequence of maps with a temporary pause therefore gives the same final state. This proves chunk parity in exact arithmetic, provided all four fields cross the boundary and the configuration is unchanged.

The proof is short because the state is complete. If only H and s are saved after three valid positions of a four-position block, the next chunk cannot know whether its first token should trigger a commit. Dropping p, q changes the model even when a casual observer sees plausible bank values. The tests deliberately perform this broken continuation and check that both the coarse state and phase differ.

Record identity determines which state may continue



Model revision and configuration must also agree at continuation.

Figure 20.4: Record A's complete carried state follows only record A. Record B begins from its own reset state. Dashed forbidden edges mark cross-record reuse. In a serving system the record and model identities are additional ownership metadata, not implicit tensor properties.

Forward parity is only half of the training contract. A saved tensor that has been detached from the autograd graph may retain the right values but lose dependence on earlier inputs. Tests compare gradients into both features and timescales across a split. For independent records, resetting is correct; for a continuing record, resetting is a semantic change. Persistent storage must also bind model revision, dtype, timescales, block size, readout convention and any upstream pair-encoder state.

20.7 Coarse memory loses order: an exact counterexample

Take $B = 2$ and zero initial coarse state. The blocks $(1, -1)$ and $(-1, 1)$ both have mean zero. They leave the same coarse state for every ρ. Yet a fast bank with $a = 1/2$ ends at $-1/4$ and $+1/4$, respectively. Therefore the coarse state cannot answer a question about which sign came last, while the fast bank can distinguish these particular examples.

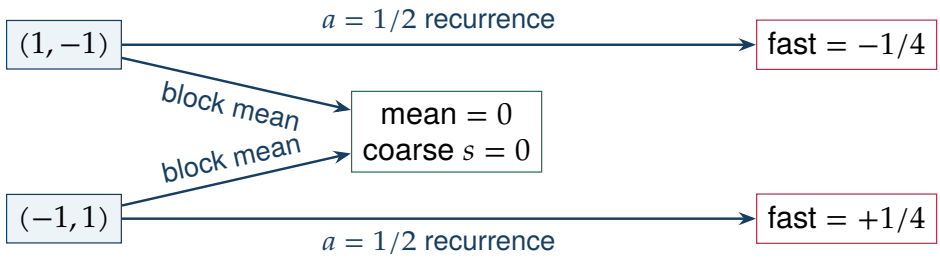


Figure 20.5: A block mean erases within-block order. The two histories meet at the same coarse value but reach different fast values. Adding a fast state repairs this counterexample; it does not prove lossless recovery of every possible history.

This is **aliasing** under the selected summary map, not numerical noise. No more accurate floating-point arithmetic can make the two exact block means different. A learned feature encoder might preserve relevant distinctions before pooling, but that is a hypothesis requiring a task and evidence. Declaring a summary global memory does not settle its adequacy.

A separate counting argument applies to any finite-precision implementation. If all carried state and metadata together have at most C bits, there are at most 2^C representable states. There are $4^n = 2^{2n}$ canonical DNA strings of length n. Whenever $2n > C$, two distinct strings must share a state. This proves a limit on lossless full-history encoding, not failure on every task. Many tasks need only a small sufficient statistic. Ideal exact real numbers do not satisfy the finite-bit premise.

20.8 Timescale gradients and useful information

For one update with $a = e^{-\Delta/\tau}$ and fixed incoming h, u,

$$\frac{\partial a}{\partial \tau} = a\frac{\Delta}{\tau^2}, \qquad \frac{\partial h_{\text{new}}}{\partial \tau} = a\frac{\Delta}{\tau^2}(h - u). \tag{20.5}$$

Increasing τ moves the update toward the old state. If old state and input coincide, this local derivative vanishes. Across time, the total derivative must include earlier states' dependence on τ; treating each step's incoming state as a constant misses those paths.

For a block commit, each of its B features has derivative $(1 - \rho)/B$ into the new coarse state before later attenuation. A subsequent m commits multiply this direct contribution by ρ^m. Before its own block commits, the feature has no effect on s, although it is present in p and in the fast bank. Reading a completed block summary back into earlier positions would leak future information.

Long retention preserves a sensitivity but does not select which information deserves preservation. A model may retain nuisance composition while losing a rare functional pattern. Conversely, short memory can be enough when the target depends only on local sequence. Report performance by declared dependency distance and perturbation type instead of turning τ into an unsupported effective-context-length headline.

20.9 Research comparisons without architectural substitution

HiPPO formulates recurrent memory through online polynomial approximation of a history under a specified measure [50]. Its approximation objective gives a more structured meaning to compression than an arbitrary collection of decay constants. Our independent exponential rows are not a HiPPO construction, and inherit no optimality theorem from that work.

Titans combines a learned test-time memory with other memory roles [51]. In its long-term module, an associative objective drives updates to memory parameters, with a momentum-like state and forgetting. Our recurrence updates activation state with fixed parameters during a record; it is not test-time learning. The shared lesson is to specify what is stored, how it is written and what a read returns, not to treat all mechanisms called memory as equivalent.

The distinction matters for DNA-domain systems. Molecular binding, cellular regulation, neural activation state, optimizer state and a persistent database have different units and causal mechanisms. A software timescale measured in valid tokens is not a cellular lifetime. Biological labels should not stand in for mechanism diagrams when the object is a tensor.

20.10 Build an evaluation contract that could reject the design

Compare at least a local-window baseline, a single-timescale recurrence, the multiscale bank, and the bank plus block summary. Hold the feature encoder and readout policy fixed when isolating memory effects. Match or report parameter count, carried-state bytes, arithmetic, training budget and actual hardware latency separately. A broader trained baseline may be needed to test usefulness; a controlled ablation tests a narrower causal claim.

Choose the split by the scientific question. Related sequences and overlapping windows should not cross a purportedly independent train/test boundary. State whether genomic coordinates, species, homologous groups or individuals are held out. Report more than an average score: probe rare patterns, long gaps, changed composition, ambiguous bases, record boundaries and reverse-complement transformations with the appropriate label semantics. Chapter 21 will own the strand-symmetry contract; a directional recurrence does not become strand invariant by being DNA-addressed.

For this reference, persistent numerical state contains $(K + 2)D$ scalars plus phase. At 32 bits per scalar, $K = 8, D = 64$ needs $4(8 + 2)64 = 2560$ bytes for those tensors, excluding allocator overhead, parameters, history, metadata and any raw-token buffer. Each valid event costs $O(KD)$ arithmetic for the bank and $O(D)$ for the pending sum. Coarse commits add $O(D)$ every B valid events. These counts are not measured throughput.

Operational tests should interleave records, change chunk boundaries, insert padding, serialize and restore complete state, and deliberately supply a mismatched model revision. Our local reference tests arithmetic, causality, masking and in-memory ownership; it does not implement a versioned serving store. Industrial claims require that additional system and its failure tests.

Key ideas

- Longer retention trades immediate write amplitude for persistence: at $a = 7/8$, a fresh input moves the state by only $1/8$.
- Slow updating is not slow retention. A block commit is constant between commits and has different gradient paths from a per-token exponential.
- Dropping the pending sum and phase at a chunk boundary silently changes the model.
- With C bits of state and $2n > C$, two distinct DNA strings of length n must share a state, so no fixed-size state stores every history.

20.11 Exercises with worked answers

1. Convert $a = 1/2$ into a unit-clock e-fold time and half-life. **Answer:** $\tau = 1/\log 2$; half-life 1 valid event.

2. Starting from zero, apply two unit inputs with $a = 3/4$. **Answer:** $h_1 = 1/4$, $h_2 = 7/16 = 1 - (3/4)^2$.

3. What is the weight of an input at lag three for $a = 1/2$? **Answer:** $(1-a)a^3 = 1/16$, not $1/8$.

4. Why is two-clock composition exact only under a stated input policy? **Answer:** With the same held input, $a(\Delta_1)a(\Delta_2) = a(\Delta_1 + \Delta_2)$ and the write weights combine. Different inputs produce different weighted contributions.

5. With $B = 4, \rho = 1/2$, what follows block $(1, 0, 0, 0)$ from reset? **Answer:** $s = 1/8$, $p = 0$, $q = 0$; the fast bank has its own values.

6. A chunk ends with $q = 3$. May the next chunk start with $q = 0$? **Answer:** Not for continuation. The next valid event must commit the old pending sum plus the new feature.

7. Show that equal block means need not imply equal fast states. **Answer:** $(1, -1)$ and $(-1, 1)$ have mean zero; at $a = 1/2$ their terminal fast values are $-1/4$ and $+1/4$.

8. Differentiate the update with respect to τ at fixed h, u. **Answer:** Equation 20.5; its sign follows $h - u$. Total sequence derivatives also propagate through previous states.

9. Can a C-bit state losslessly represent all length-n DNA strings when $2n > C$? **Answer:** No, by the pigeonhole principle. This does not rule out exact task-specific sufficient statistics.

10. Does a frozen dataclass prevent in-place tensor mutation? **Answer:** No. It prevents attribute rebinding, not mutation of a referenced tensor. Ownership rules and non-mutating updates remain necessary.

11. Compute carried tensor bytes for $K = 4, D = 32$ at float32. **Answer:** $4(4 + 2)32 = 768$ bytes, before phase and other overheads.

12. Propose a test of useful long-range memory. **Rubric:** Fix the task and label alignment; separate related training and test sequences; use matched baselines and encoder/readout controls; vary dependency distance and nuisance context; report uncertainty, state cost, runtime and negative cases. Synthetic kernel plots alone earn no credit as evidence of genomic capability.

20.12 Vocabulary and the next boundary

Update clock: the events or distance increments that advance state. **Retention**: attenuation of previous state in a declared recurrence. **Carried state**: all fields required to resume the same computation. **Information capacity**: the distinctions representable under the chosen precision and storage budget, not merely the length of input read. The next chapter asks how these directional memory states should transform when the same DNA is represented on the opposite strand.

Sources and evidence

Access checked 27 September 2026. HiPPO: arXiv v2 abstract and metadata inspected for the polynomial-projection memory objective; no reproduction of its optimality proof or experiments is claimed. Titans: v1 introduction and Section 3.1 inspected for the memory roles, associative objective and gradient/momentum update. Its benchmark results are not used as evidence for the proposed DOGMA reference.

No genomic training experiment or deployment benchmark is reported in this chapter. Tests establish the stated numerical contracts, not learned capability.

CHAPTER 21

Strands, complements and dual-state hypotheses

In this chapter you will

- express reverse complementation as position reversal composed with a channel permutation
- choose the output action (invariant, equivariant or label-swapping) that a task requires
- lift a shared causal operator into a paired dual-direction state and prove its transformation law
- prove that exact reversal symmetry and left causality together eliminate nonlocal context

Read the DNA strand 5′–ACGA–3′. Its paired strand, read in its own 5′-to-3′ direction, is TCGT, not TGCT and not AGCA. Suppose a model emits a score at every base. When the same duplex is presented from the other strand, should the scores stay fixed, reverse position, exchange labels, or change sign? The answer depends on the task. A biological relationship does not determine the output representation for us.

Chapter 18 introduced strand-aware hypotheses; Chapter 20 separated memory representations and clocks. We now construct a precise dual-direction state operator, prove its transformation law and test its implementation. We also prove a limitation: demanding exact reversal symmetry from an ordinary left-causal sequence-to-sequence map can eliminate nonlocal context. DOGMA remains a proposed non-Transformer research direction. The reference below tests semantics, not trained genomic capability.

21.1 Choose coordinates before choosing an architecture

Use channel order (A, C, G, T) and a record matrix $X \in \mathbb{R}^{T\times 4}$. For canonical DNA, each row is one-hot. The complement permutation is $c = (3, 2, 1, 0)$ in zero-based indexing. Let P be its 4×4 permutation matrix and R_T reverse the T positions. Then

$$\mathcal{R}X = R_T X P, \qquad P^2 = I, \quad R_T^2 = I, \quad \mathcal{R}^2 X = X. \tag{21.1}$$

Position reversal and channel complement commute because they act on different axes. Complement alone describes the aligned partner in the original left-to-right drawing; reversal is necessary to read that partner from its own 5′ end.

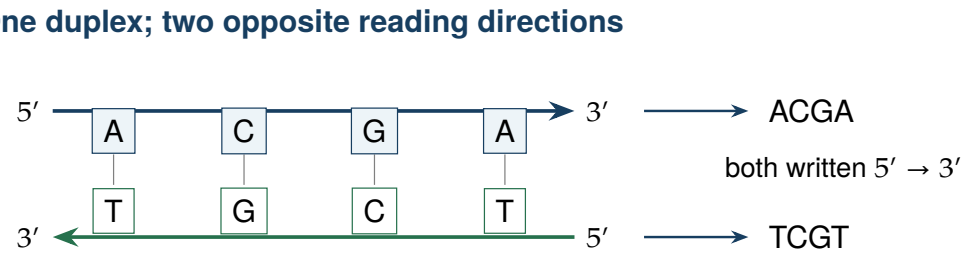


Figure 21.1: The physical duplex fixes the coordinate transformation. Continuous backbone arrows point toward 3′; short cross-strand marks denote base pairing. Reading the lower strand right to left gives TCGT. The tensor operation reverses rows and permutes base columns; it does not chemically reverse a molecule.

```
def rc(x):
    """Reverse positions and complement canonical base coordinates."""
    matrix(x, 4)
    return x.flip(0)[:, COMP]
```

The helper accepts finite real matrices with four channels so that continuous derivatives can be tested. The canonical encoder accepts only uppercase ACGT strings. It rejects N and lowercase input rather than silently assigning an ambiguity policy. A production tokenizer must separately define ambiguous bases, masking symbols, padding and record separators. Each extra symbol needs a specified transformation; a row of zeros is not automatically a nucleotide.

For zero-based coordinates, a base at i maps to $T - 1 - i$; a half-open interval $[a, b)$ maps to $[T - b, T - a)$. Applying the interval rule twice returns the original interval. A reference allele and its alternate allele must also be complemented when transforming a variant. Reversing an array of scores without transforming its coordinate metadata is a biological annotation error even if the tensor dimensions agree.

21.2 Invariance and equivariance answer different questions

A record-level scalar may be invariant: $f(\mathcal{R}X) = f(X)$. A per-position unstranded score should instead obey $f(\mathcal{R}X) = R_T f(X)$. A pair of strand-specific tracks may reverse position *and* swap its two labels. A four-base probability vector requires a complement permutation as well. None of these actions is interchangeable.

In general, choose an output action ρ with $\rho^2 = I$ and require

$$f(\mathcal{R}X) = \rho f(X). \tag{21.2}$$

This is **equivariance**. Invariance is the special case $\rho = I$. It is a structural constraint, not a claim that changing orientation cannot matter biologically. An oriented transcription task carries strand labels. An assay, genome coordinate frame or supplied directional context can break the symmetry unless that context is transformed too.

There is a generic full-record construction. Let the outputs lie in a vector space and let ρ be a linear involution, as all the permutation actions above are. For any deterministic map g into

that space, define

$$\mathcal{S}g(X) = \tfrac{1}{2}\{g(X) + \rho g(\mathcal{R}X)\}. \tag{21.3}$$

Then $\mathcal{S}g(\mathcal{R}X) = \rho\mathcal{S}g(X)$ by substitution and $\rho^2 = I$. Applying $\mathcal{S}$ again changes nothing: it is a projection onto equivariant maps. This construction is useful, but it evaluates g on both orientations and can discard orientation-sensitive distinctions. For classification, averaging probabilities and averaging logits are also different operations, even when each can be made equivariant.

Data augmentation is not this algebraic guarantee. It presents both orientations during training, encouraging rather than enforcing a relationship. Finite data, optimization, regularization and stochastic inference can still produce discrepancies. A symmetry test should measure the actual deployed operator under its actual numerical and randomness settings.

21.3 Lift a causal operator into paired directional states

Let $F_\theta : \mathbb{R}^{T\times 4} \to \mathbb{R}^{T\times D}$ be a deterministic sequence operator, shared between two evaluations. It need not itself be complement equivariant. Define the aligned dual representation

$$E_\theta(X) = [F_\theta(X) \mid R_T F_\theta(\mathcal{R}X)] \in \mathbb{R}^{T\times 2D}. \tag{21.4}$$

The second result must be reversed *back* into the original coordinates. Its hidden channels are arbitrary learned features, not nucleotide identities; we do not apply P to them. Instead define an explicit paired hidden action

$$\mathcal{J}[U \mid V] = [R_T V \mid R_T U]. \tag{21.5}$$

It reverses positions and swaps feature blocks, with $\mathcal{J}^2 = I$.

Proposition 21.1 (Shared dual lifting)**.** *For every input in the operator's domain,* $E_\theta(\mathcal{R}X) = \mathcal{J}E_\theta(X)$.

Proof. Using Equation 21.1, the left side is $[F_\theta(\mathcal{R}X) \mid R_T F_\theta(X)]$. Applying $\mathcal{J}$ to Equation 21.4 gives exactly these blocks, because $R_T^2 = I$. No special recurrence equation or learned weight value was needed. Shared parameters and compatible boundary conditions were needed. □

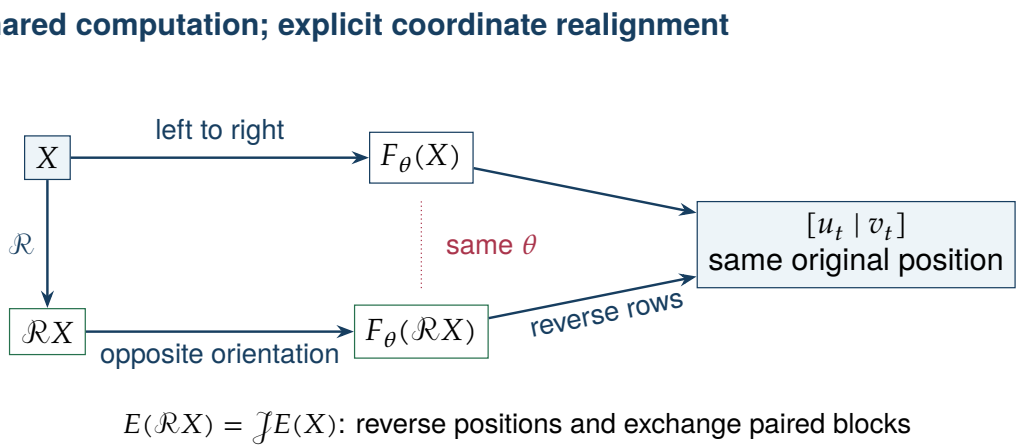


Figure 21.2: The two evaluations share one parameter set but traverse opposite orientations. Reverse-pass features are realigned before concatenation. Changing the input orientation exchanges the roles of the two halves. Hidden feature identity is preserved within each half; base complement and feature-block swap are separate operations.

Choose a deliberately transparent causal reference:

$$h_t = a \odot h_{t-1} + x_t W, \qquad h_0 = 0, \quad W \in \mathbb{R}^{4\times D}, \quad a \in [0,1)^D. \tag{21.6}$$

Here $\odot$ denotes componentwise multiplication; rows are feature vectors. Unlike Chapter 20's convex-average bank, this recurrence has no $(1-a)$ write factor. It sums decayed projected inputs. For bounded $|(x_t W)_d| \le M_d$, its zero-state magnitude is at most $M_d(1-a_d^t)/(1-a_d)$, not M_d. The change in normalization is explicit.

```
def causal(x, weight, retention, carry=None):
    """h_t = a*h_(t-1) + x_t W; this is NOT a convex-average update."""
    matrix(x, 4)
    matrix(weight)
    d = weight.shape[1]
    if weight.shape[0] != 4 or d == 0 or retention.shape != (d,):
        raise ValueError("W[4,D], a[D], D>0 required")
    if (
        not torch.isfinite(retention).all()
        or ((retention < 0) | (retention >= 1)).any()
    ):
        raise ValueError("retention must lie in [0,1)")
    if any(
        t.dtype != x.dtype or t.device != x.device
        for t in (weight, retention)
    ):
        raise ValueError("shared floating dtype/device required")
    h = x.new_zeros(d) if carry is None else carry
    if (
        h.shape != (d,)
        or h.dtype != x.dtype
        or h.device != x.device
        or not torch.isfinite(h).all()
    ):
        raise ValueError("incompatible carry")
    rows = []
    for row in x:
        h = retention * h + row @ weight
        rows.append(h)
    return (torch.stack(rows) if rows else x.new_empty((0, d))), h
```

The code validates shape, dtype, device, finiteness and retention range. It never updates caller-owned carry in place. Empty chunks produce no output rows and preserve carry. These checks favor inspectability, not accelerator throughput. The returned history uses $O(TD)$ storage; a streaming forward pass can retain only $O(D)$ state if intermediate outputs are consumed.

```
def dual(x, weight, retention):
    """Full-record operator: second half uses suffix information."""
    forward, _ = causal(x, weight, retention)
    reverse, _ = causal(rc(x), weight, retention)
    return torch.cat((forward, reverse.flip(0)), dim=1)
```

For $D = 1$, $W = (1,2,3,4)^\top$, $a = 1/2$ and ACGA, the forward projection sequence is $(1,2,3,1)$. The opposite strand TCGT projects to $(4,2,3,4)$. Applying the recurrence gives $(4,4,5,6.5)$ on that strand; realignment yields $(6.5,5,4,4)$. The generated trace checks the entire coordinate convention:

Position	Base	Forward	Aligned reverse	Sum
1	A	1.0	6.5	7.5
2	C	2.5	5.0	7.5
3	G	4.25	4.0	8.25
4	A	3.125	4.0	7.125

Adding the halves produces a position-equivariant unstranded representation. It does not mean that the two halves are redundant: their difference carries information that changes sign when the branch labels swap.

21.4 A readout must respect the hidden action

Write a paired feature at one position as $(u, v) \in \mathbb{R}^D \times \mathbb{R}^D$. A general linear paired readout has blocks $M = \left(\begin{smallmatrix} A & B \\ C & D_0 \end{smallmatrix}\right)$. Let J swap the two input blocks and J' swap the two output blocks. Commuting with the actions requires

$$MJ = J'M. \tag{21.7}$$

Multiplying both sides blockwise gives $A = D_0$ and $B = C$. Thus the tied form is

$$M = \begin{pmatrix} U & V \\ V & U \end{pmatrix}, \qquad b = \begin{pmatrix} b_0 \\ b_0 \end{pmatrix} \tag{21.8}$$

if an affine bias is included. The proof does not say that an arbitrary linear layer is safe just because its input is equivariant.

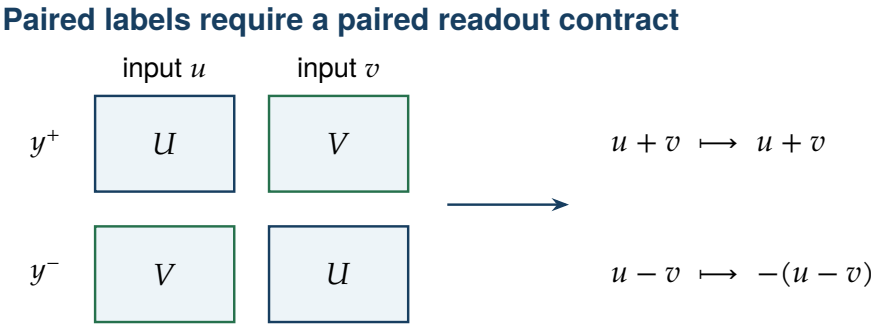


Figure 21.3: A tied paired readout commutes with branch exchange. Diagonal blocks share U and off-diagonal blocks share V. The sum channel is unchanged under exchange; the difference channel changes sign. Keeping both is useful when output labels are strand-specific.

```
def strand_head(z, u, v):
    """Tied two-channel head; RC reverses positions and swaps labels."""
    matrix(z)
    if z.shape[1] == 0 or z.shape[1] % 2:
        raise ValueError("even nonzero hidden dimension required")
    d = z.shape[1] // 2
    for p in (u, v):
        if (
            p.shape != (d,)
            or p.dtype != z.dtype
```

```
            or p.device != z.device
            or not torch.isfinite(p).all()
        ):
            raise ValueError(
                "one compatible weight vector per branch required"
            )
    f, b = z.chunk(2, dim=1)
    return torch.stack((f @ u + b @ v, f @ v + b @ u), dim=1)
```

Our reference implements two scalar tracks without bias, with $y^+ = u^\top U + v^\top V$ and $y^- = u^\top V + v^\top U$. Across a whole sequence the outputs reverse position and swap tracks. An unstranded per-position head can use $u + v$; averaging that head over a nonempty record makes a record-invariant summary. Empty-record pooling needs an explicit policy and is not implemented here.

This suggests a systematic architectural audit. A pointwise nonlinearity shared across coordinates commutes with a permutation. An arbitrary normalization with untied affine parameters may not. Absolute positional embeddings, branch-specific dropout masks, independent branch parameters and orientation-dependent initial states can all break the proof. At inference, disabled dropout is deterministic; at training, a claim of pathwise symmetry needs correspondingly transformed randomness, not merely equal mask distributions.

21.5 Why the reverse pass cannot be ordinary causal decoding

If F_θ is causal, its feature at position t depends on $x_1, \dots, x_t$. The aligned second half in Equation 21.4 depends on $x_t, \dots, x_T$. Together they see the entire record. Reversing a known record is therefore an offline operation, not a way to obtain future context from a prefix.

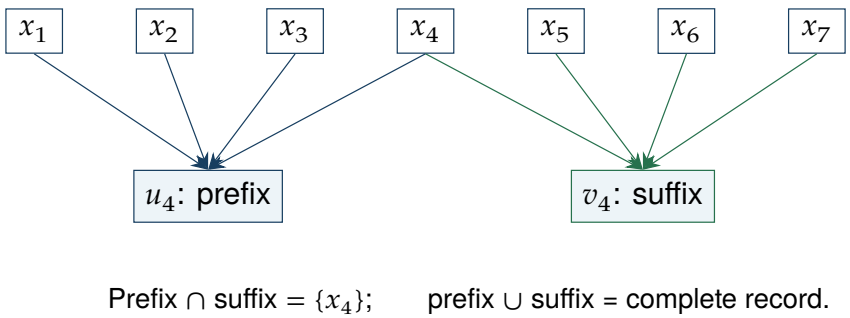


Figure 21.4: At a selected position, the forward branch reads the prefix and the aligned reverse branch reads the suffix. Their overlap is the current token; their union is the full record. The second branch is valid for offline inference but would leak future information into a left-to-right next-token predictor.

Proposition 21.2 (A causality–reversal limitation)**.** *Fix a record length T and a full product input domain. Suppose $f(X)_t$ is left-causal, and f is equivariant under reversal with invertible pointwise input/output channel permutations. Then $f(X)_t$ depends only on the input at t (and fixed metadata such as t, T), not on other positions.*

Proof. By left causality, $f(X)_t$ depends only on the prefix through t. Equivariance expresses this value as an invertible channel transform of $f(\mathcal{R}X)_{T+1-t}$. Left causality on the reversed input makes that quantity depend only on the original suffix from t onward. Now take two inputs that agree at t. Form a third using the first input's prefix and the second input's suffix; it exists because the domain is a full product. Prefix dependence equates its output with the first input, and suffix dependence equates it with the second. Hence only the shared token can matter. □

The assumptions matter. This is a theorem about same-position sequence-to-sequence outputs at fixed length, not a prohibition on all strand-aware causal models. The paired representation avoids the conclusion by being bidirectional, not left-causal. A forward-only branch can be used for causal decoding, but no longer provides the full paired-output guarantee. Record-level invariant outputs emitted after the record ends are another different contract.

For autoregressive training, a forward conditional $p(x_{t+1} \mid x_{\leq t})$ and a reverse-oriented conditional are different prediction problems. One may train both directions with shared weights and correctly transformed targets; this does not justify averaging their aligned logits while evaluating a forward conditional. A target mask must also be transformed with coordinates. Bidirectional masked-token objectives require masking the target information before either pass sees it.

21.6 Reverse the record, not the transport chunks

Let a complete record be partitioned as $X = C_1 C_2 \cdots C_m$. Reverse complement is an anti-homomorphism for concatenation:

$$\mathcal{R}(C_1 C_2 \cdots C_m) = \mathcal{R}(C_m) \cdots \mathcal{R}(C_2)\mathcal{R}(C_1). \tag{21.9}$$

The order of chunks and the coordinates within each chunk both reverse. For ACG|TA|CC, the correct opposite record is GG|TA|CGT. Independently transforming chunks in arrival order gives CGT|TA|GG, a different record.

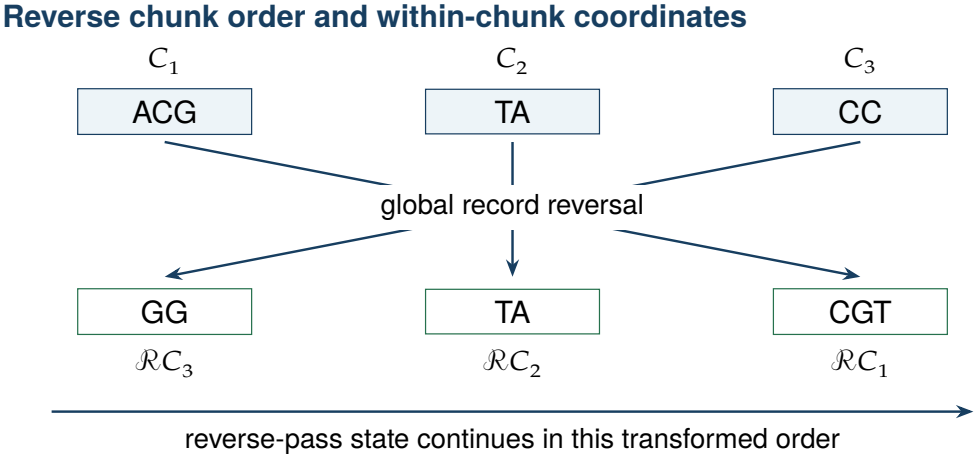


Figure 21.5: Chunk boundaries are a transport partition, not a new biological sequence order. The reverse pass starts with the final chunk and continues its state into the preceding chunk. Each record has its own pair of boundary states; concatenating unrelated records does not license state reuse.

```
def chunked_dual(chunks, weight, retention):
    """Offline: reverse global chunk order AND each chunk's coordinates."""
    if not chunks:
        raise ValueError("provide at least one chunk, which may be empty")
    forward, backward = [], []
    carry = None
    for part in chunks:
        out, carry = causal(part, weight, retention, carry)
        forward.append(out)
    carry = None
    for part in reversed(chunks):
        out, carry = causal(rc(part), weight, retention, carry)
        backward.append(out)
    return torch.cat(
        (torch.cat(forward), torch.cat(backward).flip(0)), dim=1
    )
```

The forward pass carries its final state left to right. The opposite pass starts from zero at the opposite record boundary, visits chunks in reverse order and then realigns the concatenated outputs. Exact-arithmetic parity with Equation 21.4 follows because the recurrence sees the same token order and the same state at every step. Tests check every three-way partition of several record lengths, including empty chunks, and compare parameter and input gradients as well as forward values.

This implementation is offline and retains output histories. In a real system, reverse traversal requires an available complete record, a reverse data source, or stored input that can be reread. It is not supplied by a constant-size forward state in general. For a scalar recurrence, the feature sequences $(1, -a)$ and $(0, 0)$ have the same forward final state zero, but their reverse final states are $1 - a^2$ and zero. This counterexample concerns the real-valued feature domain; it is not an assertion that these numbers are literal nucleotide tokens.

Padding requires similar care. Our reference processes unpadded records. If right padding is reversed along with the record, it becomes left padding. Advancing the recurrence on a zero row decays the state and is not the same as skipping a padding slot. A production implementation should transform the validity mask and obey a declared hold/advance policy. Likewise, a cached state needs record identity, orientation, model revision and boundary location, not just the correct vector dimension.

21.7 Differentiate the shared computation

Sharing weights means gradients add across both uses. If $E_\theta(X) = [F_\theta(X) \mid R_T F_\theta(\mathcal{R}X)]$ and a loss has branch cotangents (g_f, g_b), then schematically

$$\nabla_\theta \mathcal{L} = (\partial_\theta F_\theta(X))^\top g_f + (\partial_\theta F_\theta(\mathcal{R}X))^\top R_T g_b. \tag{21.10}$$

There is one parameter object, not two weights periodically copied after optimization. Position reversal and complement are permutations, whose adjoints are their inverses; in this case they are involutions. Consequently, gradients through the transformed input return to the corresponding original coordinates.

For a loss invariant under the output action and consistently transformed targets, the losses on X and $\mathcal{R}X$ are equal in exact arithmetic. Their parameter gradients are equal wherever differentiable. This does not mean the two branch contributions within one evaluation are equal. Finite differences, automatic-differentiation parity, and deliberately broken weight

sharing form separate checks. Our tests include all three forms of reasoning: a closed-form recurrence oracle, numerical gradient checking, and negative examples whose symmetry residual is nonzero.

21.8 A falsifiable strand hypothesis, not a biological shortcut

Caduceus provides a primary example of distinguishing bidirectionality, shared reverse-complement processing, equivariant embeddings and output heads [48]. It also separates architectural equivariance from post-hoc conjoining. Our paired-block action and simple recurrence are an independently specified teaching reference, not a reproduction of its MambaDNA implementation or reported benchmarks.

For DOGMA, formulate the scientific claim narrowly: does a declared strand constraint improve a specific task under matched budgets and correct labels? Compare a forward baseline, an unconstrained bidirectional baseline, augmentation, the shared dual lift and an invariant readout only when the task warrants invariance. Match or report parameters, FLOPs, input access, state storage and preprocessing. Sharing parameters does not eliminate the arithmetic of a second traversal.

Split data before orientation augmentation. Otherwise a record can appear in training while its reverse complement appears in testing. Group related loci and homologous examples according to the scientific question; exact RC deduplication alone does not remove biological relatedness. A palindromic record satisfies $\mathcal{R}X = X$, so its paired output must satisfy $E(X) = \mathcal{J}E(X)$. That relates opposite positions and branches, not necessarily the two branches at the same position.

Report task metrics alongside symmetry residuals and failure cases. Zero symmetry residual proves only the implementation's transformation contract. A constant predictor can satisfy it perfectly and still be useless. Conversely, a task with deliberately oriented labels can be harmed by an invariant head that erases needed distinctions. The output action is a scientific choice to test, not an architectural slogan.

Key ideas

- The paired strand of 5′-ACGA-3′, read 5′ to 3′, is TCGT. Complementing alone describes a different reading.
- A readout preserves the symmetry only if its blocks are tied: $\left(\begin{smallmatrix} U & V \\ V & U \end{smallmatrix}\right)$.
- The aligned reverse branch reads the suffix, so the dual representation is offline by construction.
- Reverse complementation reverses chunk order as well as each chunk. Transforming chunks in arrival order gives a different record.

21.9 Exercises with worked answers

1. Compute the reverse complement of ACGA. **Answer:** Complement TGCT, then reverse to TCGT; complement-only is not the required reading order.

2. Transform $[2, 5)$ in a record of length 9. **Answer:** $[9-5, 9-2) = [4, 7)$. Applying the rule again returns $[2, 5)$.

3. Prove $\mathcal{R}^2 = I$. **Answer:** Row reversal and channel complement commute, and each applied twice is identity.

4. What output action fits two strand-specific tracks? **Answer:** Reverse positions and swap track labels, provided the assay metadata transforms in the same way.

5. Why is the second branch reversed after evaluation? **Answer:** Its row $T + 1 - t$ corresponds to original position t; without realignment the concatenated rows describe different positions.

6. Derive h_3 for projected inputs $(1, 2, 3)$ and $a = 1/2$. **Answer:** $h_1 = 1$, $h_2 = 2.5$, $h_3 = 4.25$.

7. Solve the block constraints for a linear paired readout. **Answer:** $MJ = J'M$ implies $A = D_0$ and $B = C$; an affine bias must have equal paired blocks.

8. Why does equivariance not permit causal use of the entire dual state? **Answer:** Its aligned reverse branch uses the suffix. Symmetry does not remove those dependencies.

9. Reverse complement ACG|TA|CC. **Answer:** GG|TA|CGT; changing each chunk without reversing their order is wrong.

10. Does a zero padding row act as identity? **Answer:** Not in Equation 21.6: it maps h to $a \odot h$. Skipping an invalid slot needs an explicit mask rule.

11. Must a palindromic input have equal branch features at each position? **Answer:** No. It requires $u_t = v_{T+1-t}$; equality at t only follows where additional conditions, such as a central fixed position, apply.

12. Design a fair strand-ablation study. **Rubric:** Specify label transformations and orientation metadata; split related records before augmentation; compare forward, bidirectional and constrained variants; match or report computational budgets; test symmetry and task accuracy separately; include an oriented task and a constant-predictor control.

21.10 Vocabulary and the integration boundary

Reverse complement: reverse sequence order and complement base identities. **Output action**: the specified transformation of predictions when the input representation changes. **Parameter tying**: multiple computational uses of one parameter object. **Bidirectionality**: use of both prefix and suffix context. **Anti-homomorphism**: a transformation that reverses the order of composition, as in Equation 21.9.

Chapter 22 will assemble the DOGMA reference research model. Its first job is not to add more biological names: it is to reconcile the token, regulation, memory, strand, objective and execution contracts into a model whose successes and failures can be measured without changing the claim halfway through.

Sources and evidence

Access checked 27 September 2026. Caduceus arXiv v2, Sections 2.1, 3.1–3.2 and 4.1–4.2 were inspected for strand coordinates, bidirectionality, shared operators, equivariant embedding/head requirements and post-hoc conjoining. The paper's experimental results are not reproduced or used as evidence for DOGMA. The paired-block representation, numeric trace and tests in this chapter refer to the explicitly specified local implementation.

CHAPTER 22

DOGMA reference model and falsifiable research program

In this chapter you will

- freeze a small reference DOGMA with explicit ownership of fast state and memory loci
- trace one transition by hand and use it to detect read-before-write errors
- separate causal streaming from offline strand-symmetric encoding
- state ablations and a falsification matrix that could reject each DOGMA hypothesis

The previous four chapters introduced pieces: typed state, regulators, selective transformation, structured memory, strands and reverse-complement actions. A list of pieces is not yet a model. A model begins when every tensor has an owner, every transition has a legal information set, every output has an objective, and a failed experiment can force us to abandon a claimed benefit.

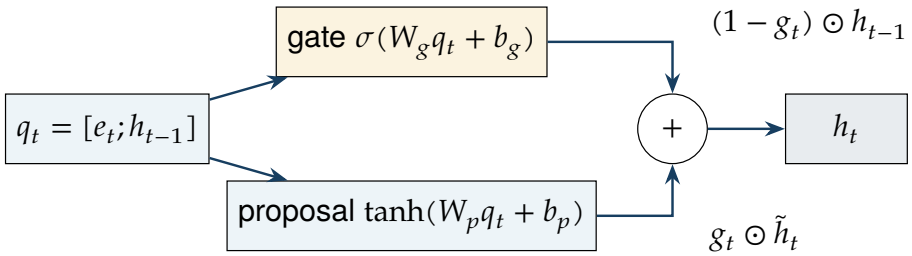
This chapter therefore freezes a small reference DOGMA. It is deliberately not the future DOGMA Engine. It is a scientific object: simple enough to trace, differentiated end to end, explicit about causality, and comparable with ordinary recurrent baselines. The implementation is a proposal assembled from established ingredients and earlier Evolutor contracts. Its value must come from experimentally useful organization, not from biological vocabulary. This freezes one additional reference variant. Chapter 18's exact pair-count state and Chapter 20's block-clock bank remain separate teaching models: learned loci here are not those objects renamed. Common contracts concern ownership, legal information access, complete carry and falsifiable evaluation. Architectural equivalence would require its own proof.

22.1 The computational contract

For batch size B, hidden width D, K memory loci and sequence length T, let $x_t \in \{0, \dots, V-1\}^B$. The model-owned parameters include an embedding $E \in \mathbb{R}^{V \times D}$ and learned affine maps. Request-owned state is

$$h_t \in \mathbb{R}^{B \times D}, \qquad M_t \in \mathbb{R}^{B \times K \times D}.$$

Both are zero-initialized at a request boundary unless a later runtime contract explicitly supplies persistent state. A trace is observational metadata; it is not silently fed back into the transition.



Model owns E, W, b; request owns (h, M).
The regulator depends on the current input and previous state.

The weighted branches are tensor operations, not biochemical reactions.
Bounded values do not imply bounded recurrent derivatives.

Figure 22.1: Reference DOGMA ownership and data flow. Biological terms such as regulator and locus denote computational roles here, not literal gene-expression chemistry.

With $e_t = E[x_t]$ and $q_t = [e_t; h_{t-1}]$, define a proposed fast-state update

$$\tilde{h}_t = \tanh(W_p q_t + b_p), \tag{22.1}$$

$$g_t = \sigma(W_g q_t + b_g), \tag{22.2}$$

$$h_t = (1 - g_t) \odot h_{t-1} + g_t \odot \tilde{h}_t. \tag{22.3}$$

This resembles established gated recurrence. The DOGMA claim is therefore *not* that Equation 22.3 invents gating. Its role is to provide a stable primitive on which the explicit memory and strand contracts can be tested.

Address the K memory loci with

$$a_t = \mathrm{softmax}(W_a h_t + b_a), \qquad r_t = \sum_{k=1}^{K} a_{t,k} M_{t-1,k}.$$

Writes use locus-wise gates $w_t = \sigma(W_w h_t + b_w)$ and value $v_t = \tanh(W_v h_t + b_v)$:

$$M_{t,k} = (1 - w_{t,k}) M_{t-1,k} + w_{t,k} v_t. \tag{22.4}$$

The emitted feature is $y_t = \tanh(W_m [h_t; r_t] + b_m)$. The reference code implements these equations literally.

For one example use column vectors: $W_p, W_g \in \mathbb{R}^{D \times 2D}$, $W_a, W_w \in \mathbb{R}^{K \times D}$, $W_v \in \mathbb{R}^{D \times D}$ and $W_m \in \mathbb{R}^{D \times 2D}$. Bias lengths are D, D, K, K, D, D. PyTorch batches examples as rows and applies transpose contractions inside `nn.Linear`. A C-class head produces C logits; causal base prediction needs $C = 4$, not the default two-class head.

```
    def step(self, ids, h, m, *, disable_read=False):
        e = self.emb(ids)
        q = torch.cat([e, h], -1)
        cand = torch.tanh(self.propose(q))
        g = torch.sigmoid(self.gate(q))
        h = (1 - g) * h + g * cand
        a = torch.softmax(self.read_key(h), -1)
        read = torch.einsum("bk,bkd->bd", a, m)
        if disable_read:
            read = torch.zeros_like(read)  # same-parameter path intervention
        wg = torch.sigmoid(self.write_gate(h))
```

```
        val = torch.tanh(self.write_val(h))
        m = (1 - wg[..., None]) * m + wg[..., None] * val[:, None, :]
        y = torch.tanh(self.mix(torch.cat([h, read], -1)))
        return y, h, m, g, wg
```

Reads use M_{t-1}, before the write at t. A new value cannot affect the same step through memory. Reading M_t instead changes the model even when shapes agree. The read distribution addresses fixed locus indices, not content-derived keys. Calling it content-addressed attention would erase a meaningful distinction from Chapters 8 and 23.

22.1.1 Trace a transition with numbers

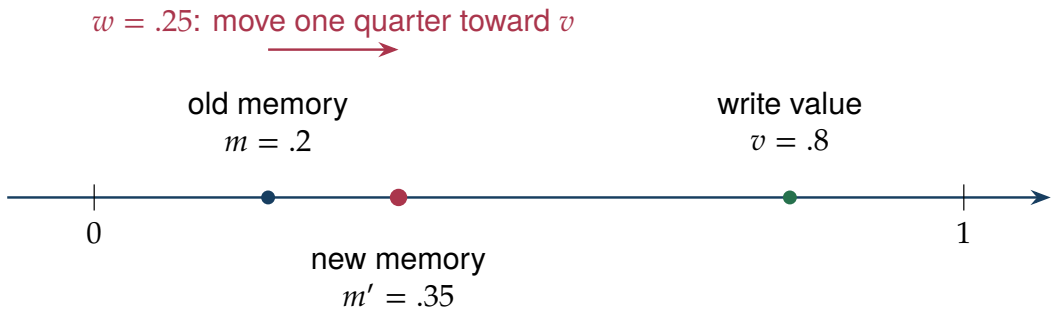
Set $D = K = 1$, $g_t = 1/2$, $\tilde{h}_t = 1/2$, $w_t = 1/4$ and $v_t = 4/5$. Choose $y_t = \tanh(h_t + r_t)$ and initialize both states to zero. Then

$$h_t = \tfrac{1}{2}(1 - 2^{-t}), \qquad M_t = \tfrac{4}{5}(1 - (3/4)^t), \qquad r_t = M_{t-1}. \tag{22.5}$$

Substitution verifies the recurrence and initial conditions; induction proves the formulas. Code fixes weights and biases to realize the constants and generates this trace:

Step	Fast state	Old memory	New memory	Feature
1	0.25	0	0.2	0.244919
2	0.375	0.2	0.35	0.519022
3	0.4375	0.35	0.4625	0.65699
4	0.46875	0.4625	0.546875	0.731176

The first feature is $\tanh(1/4)$, not $\tanh(1/4 + 1/5)$. This independent oracle detects read-before-write order, unlike copying the same update sequence into both implementations.



$m' = (1 - w)m + wv$ lies between the old state and the proposed value.
The feature at this step still reads $m = .2$, not the freshly written .35.

Figure 22.2: One memory-coordinate update: a write gate of one quarter moves the old value 0.2 toward the proposal 0.8, producing 0.35. The updated memory belongs to the next carry, while this step's read uses the old value. Positions on the line are values, not genomic coordinates.

22.1.2 Bounded state is not a gradient guarantee

Starting from zero, each fast-state and memory coordinate remains in $[-1, 1]$: every update is a convex combination of its old coordinate and a bounded tanh proposal. The addressed read

is a convex combination of bounded rows. This is a forward invariant, not a retention or gradient guarantee. Since the gate and proposal depend on h_{t-1}, their derivatives add Jacobian terms; the recurrent Jacobian is not simply $\mathrm{diag}(1 - g_t)$. Large learned weights can amplify perturbations despite bounded activations.

Including a biased C-class head, the model-owned parameter count is

$$P = 7D^2 + (V + 4)D + 2K(D + 1) + C(D + 1). \tag{22.6}$$

Proposal/gate maps contribute $4D^2 + 2D$; addressing contributes $2K(D + 1)$; value/mixing maps contribute $3D^2 + 2D$; embedding and head supply the rest. At $(V, D, K, C) = (4, 12, 3, 2)$ this is 1208. Carry stores $BD(K + 1)$ floats; histories, optimizer state, gradients, activations and input retention are additional costs. Sharing parameters across strands does not remove the second traversal's arithmetic.

22.2 Mode one: causal streaming

For causal prediction, the legal dependency set at position t is the prefix $x_{\le t}$ plus request state derived from that prefix. No reverse-complement full-record feature is legal, because Chapter 21 showed that its aligned reverse branch contains suffix information.

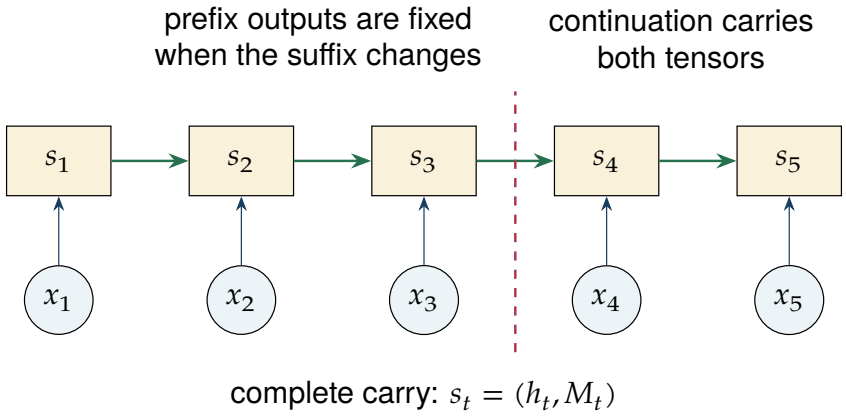


Figure 22.3: Causal mode. Recurrent state moves only from earlier to later positions. Appending or changing a suffix cannot change an already computed prefix feature.

For next-token training, one legal objective is

$$\mathcal{L}_{\mathrm{LM}} = -\sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\le t}).$$

The target is shifted; the current token cannot be used as its own label. State is reset between independent records unless the dataset explicitly defines continuity.

Chunking must carry *both* h and M. Method `run` returns a complete `DOGMAState` without mutating caller-owned tensors. An empty chunk emits a shape-$B \times 0 \times D$ history and preserves carry. Empty offline records are rejected because mean pooling has no defined divisor. This reference processes unpadded records; a later batching policy must define whether invalid positions hold or advance state.

```
    def run(self, x, state=None, *, trace=False, disable_read=False):
        if x.ndim != 2 or x.dtype != torch.long or x.shape[0] < 1:
            raise ValueError("tokens require nonempty batch and int64
             ↪  [batch,time]")
        if x.device != self.cell.emb.weight.device or bool(((x < 0) | (x >=
         ↪  4)).any()):
            raise ValueError("tokens violate device/vocabulary contract")
        b, t = x.shape
        state = self.initial_state(b) if state is None else state
        if not isinstance(state, DOGMAState):
            raise ValueError("carry requires the complete DOGMAState")
        h, m = state
        expected = ((b, self.cell.d), (b, self.cell.k, self.cell.d))
        for tensor, shape in zip(state, expected):
            if (
                tensor.shape != shape
                or tensor.dtype != self.cell.emb.weight.dtype
                or tensor.device != x.device
                or not bool(torch.isfinite(tensor).all())
            ):
                raise ValueError("carry shape/dtype/device/finiteness mismatch")
        ys = []
        traces = []
        for i in range(t):
            y, h, m, g, wg = self.cell.step(x[:, i], h, m,
             ↪  disable_read=disable_read)
            ys.append(y)
            if trace:
                traces.append(
                    {
                        "t": i,
                        "gate_mean": float(g.mean().detach()),
                        "write_mean": float(wg.mean().detach()),
                    }
                )
        output = torch.stack(ys, 1) if ys else h.new_empty((b, 0, self.cell.d))
        return output, DOGMAState(h, m), traces
```

Consecutive chunk execution equals a single pass when it carries the complete state, concatenates emitted features, and uses identical parameters and token order. Induction on the token transition proves this. Tests compare outputs, both carry tensors and parameter gradients at every split of a six-token record, including empty end chunks. Detaching carry preserves forward values but truncates gradients: that is a training policy, not a parity-preserving detail. State-dependent gates also prevent a general replacement by Chapter 7's input-only affine scan.

The test suite changes the final two tokens while preserving the first four and checks exact prefix invariance to numerical tolerance. This is more informative than checking only tensor shapes: it detects accidental suffix access.

22.3 Mode two: offline strand-symmetric encoding

Whole-record classification or annotation may legally use both orientations. Let $R(X)$ reverse positions and complement A/C/G/T. Apply the *same* causal operator F_θ to X and $R(X)$,

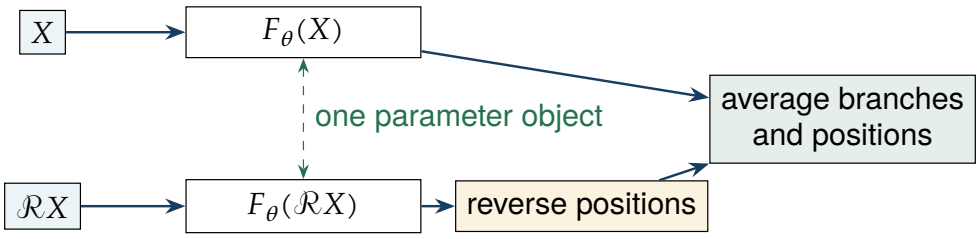
realign the reverse branch, and combine:

$$Z(X) = [F_\theta(X); \operatorname{rev} F_\theta(R(X))].$$

The compact reference uses an invariant pooled feature

$$\bar{z}(X) = \frac{1}{2T} \sum_t \left(F_\theta(X)_t + \operatorname{rev} F_\theta(R(X))_t \right).$$

Because parameters are shared and the two branches are exchanged by reverse complement, the pooled feature is invariant up to floating-point arithmetic.



Full-record access is legal for the declared offline objective.
It is not a causal next-token feature.

Figure 22.4: Offline dual mode shares parameters across both strand orientations and realigns coordinates. The resulting full-record feature is appropriate for offline objectives, not left-to-right generation.

This resolves the Chapter 21 choice rather than hiding it behind a flag. The two modes have different legal objectives. If a task requires orientation-sensitive labels, the output action must transform those labels rather than forcing invariance.

22.4 Why call the memory structured?

A single recurrent vector stores everything in one address. The reference adds K explicit loci and a learned address distribution. This does not by itself make the system genomic. It creates a testable structural hypothesis: tasks with sparse, reusable, differently timed information may benefit from explicit addressable loci relative to a capacity-matched recurrent state.

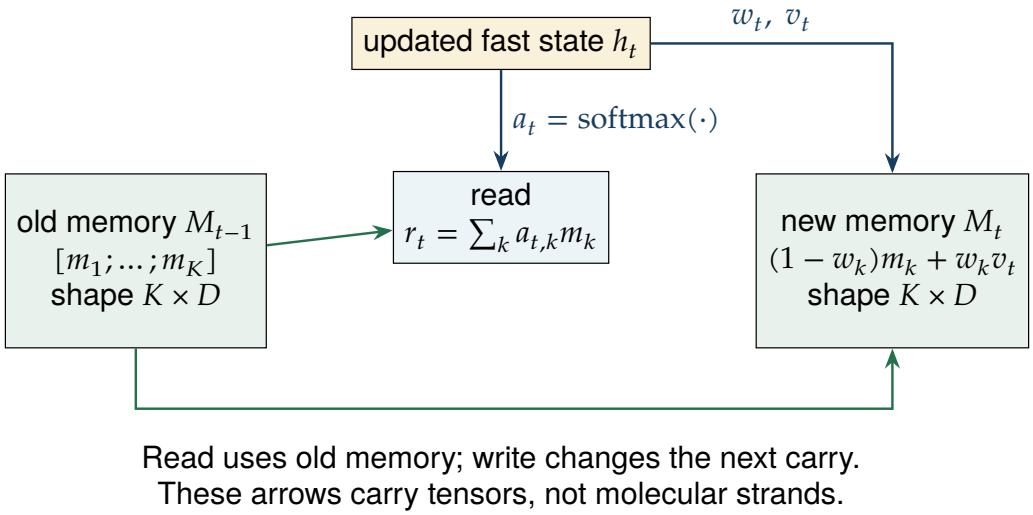


Figure 22.5: Fast state and explicit memory loci have different addresses but share request ownership. Writes are gated; reads are weighted. Persistence across requests is forbidden unless a runtime layer later supplies it explicitly.

The word *locus* is an analogy with a precise endpoint. In biology a genomic locus is a physical region with molecular regulation and inheritance. Here a locus is one row of a floating-point tensor. The analogy motivates modular addressing; it does not import chromatin, transcription, mutation or evolutionary semantics.

22.5 Trace as evidence, not explanation

For every step the reference can emit gate means and write-gate means. The step numbers restart within each chunk. They are local diagnostics, not a durable replay trace with record identity, absolute offsets and parameter hashes. A richer research implementation should record per-locus addresses, state norms, module choices and output actions. But a trace is not automatically a causal explanation. If a memory locus receives a high write gate whenever a motif appears, that is a correlation. A stronger test intervenes: clamp the gate, permute loci, erase a locus, or transplant state between matched examples and measure the output change.

A falsifiable mechanism claim therefore has three parts:

$$\text{claimed role} \quad + \quad \text{intervention} \quad + \quad \text{predicted change.}$$

For example: “locus 2 retains the first motif until the terminal decision” predicts that erasing locus 2 after the motif but before the decision will selectively reduce accuracy on examples requiring that motif. If erasure has no effect across seeds, the mechanistic story is wrong or incomplete.

22.6 A small executable experiment

The reference constructs a synthetic whole-record task. A record is positive when it contains motif ACG in either strand orientation; negatives are rejection-sampled so neither orientation occurs. Positives are explicitly implanted. This task is chosen because the offline dual mode has a legitimate symmetry to exploit. It is not a proxy for regulatory genomics.

The integrated pilot trains DOGMA offline dual, a GRU, a flattened MLP and a width-three CNN on the same 512 examples and evaluates on 128 held-out examples. A local motif problem needs a local-convolution baseline and an exact string oracle, not only weak generic

comparisons. An independent ACG/CGT detector checks every generated label and has perfect task accuracy by specification, not through training.

Data seeds 137 and 139 are fixed before the model runs. Exact records and their reverse complements are disjoint between splits. This excludes that synthetic leakage channel but is not a homology-aware biological split. Initialization seeds 11, 23 and 37 each run all model families for 30 full-batch epochs with AdamW at learning rate 3×10^{-3}. Artifacts record every run, parameter count, strand-pass count and the PyTorch environment.

```
def train_eval(model, xtr, ytr, xte, yte, dual=False, epochs=30, seed=0):
    torch.manual_seed(seed)
    opt = torch.optim.AdamW(model.parameters(), lr=3e-3)
    lossf = nn.CrossEntropyLoss()
    for _ in range(epochs):
        opt.zero_grad()
        logits = model.offline_dual_logits(xtr) if dual else model(xtr)
        loss = lossf(logits, ytr)
        loss.backward()
        opt.step()
    with torch.no_grad():
        logits = model.offline_dual_logits(xte) if dual else model(xte)
        acc = (logits.argmax(-1) == yte).float().mean().item()
        loss = lossf(logits, yte).item()
    return {"accuracy": acc, "loss": loss, "params": params(model)}
```

Model	Mean accuracy	Seed SD	Parameters
DOGMA dual	0.575521	0.0225527	1208
GRU	0.479167	0.0090211	1010
MLP	0.510417	0.0090211	18674
CNN width-3	0.669271	0.162066	182
DOGMA random	0.494792	0.0090211	1208

These are implementation-pilot means and sample standard deviations, not a matched-budget ranking. Equal epochs do not equalize FLOPs, parameter capacity, state storage or tuning opportunity. DOGMA traverses two orientations while the learned comparison models traverse one. This table licenses no architecture superiority claim; Chapter 25 supplies a shared comparison contract.

The generated `results.json` records every run's held-out loss, accuracy and parameter count. Read the table against chance before reading it against the other rows. Classes alternate, so the held-out set is exactly balanced and chance accuracy is $1/2$. With 128 held-out records, one run's accuracy under chance has standard error $\sqrt{0.25/128} \approx 0.044$, so a run needs about 0.59 to sit two standard errors above chance. By that yardstick the GRU and the MLP stay at chance, as does the random-label control. DOGMA dual's three runs $(0.56, 0.60, 0.56)$ lie one to two standard errors above chance, and their held-out losses $(0.66–0.68$ nats$)$ sit only slightly below $\ln 2 \approx 0.693$: weak learning at best. The width-three CNN learns clearly in two of its three seeds $(0.84$ and $0.64)$ and not in the third (0.52), which explains its large seed spread.

The honest reading is a negative pilot. Under this budget no model reliably learned a task that an exact string detector solves perfectly. The table therefore says something about the budget and the optimization, not about which architecture is better. The next diagnostic steps are those of Chapter 25: overfit a tiny subset, inspect gradients, and then extend training or tuning with every change declared. The pilot establishes no advantage on genomic data, language,

long context or production inference. The random-label control draws independent Bernoulli targets for both training and testing, rather than permuting training labels while scoring original motif labels. For independent uniform $Y \in \{1, \dots, V\}$ and predictor q,

$$\mathbb{E}[-\log q_Y] = \log V + D_{\mathrm{KL}}(\mathrm{Uniform}(V) \| q) \geq \log V. \tag{22.7}$$

This is a population expectation conditional on an independent predictor, not a bound on each finite-sample loss or training loss. Held-out values can fall below $\log V$ by chance; training labels can be memorized. Audit independence, uncertainty and repeated seeds before interpreting an exception.

A stronger study would add more seeds, tune each baseline under equal compute, include a Transformer/SSM baseline where appropriate, stratify by motif position, test longer sequences than training, and measure calibration as well as accuracy.

22.7 Ablations that can kill the hypothesis

The implemented read-path intervention replaces r_t with zero while retaining all parameter objects; the constructor requires $K \geq 1$ and does not implement a $K = 0$ architecture. It preserves fast-state transitions and memory writes, but changes emitted features after memory forms. Tests check these distinctions. This is not a retrained, capacity-matched ablation. A second proposed ablation freezes w_t to a constant, testing whether input-dependent writes matter. The third replaces the regulator g_t with a learned constant vector. The fourth unties the two strand branches in offline mode, testing whether symmetry is useful or merely extra capacity.

These interventions must be interpreted with capacity controls. Removing memory also removes parameters and computation; a weaker result may reflect smaller capacity rather than the mechanism. Conversely, giving DOGMA two strand passes and a baseline one pass is a compute advantage. Fair evidence reports these differences or compensates for them.

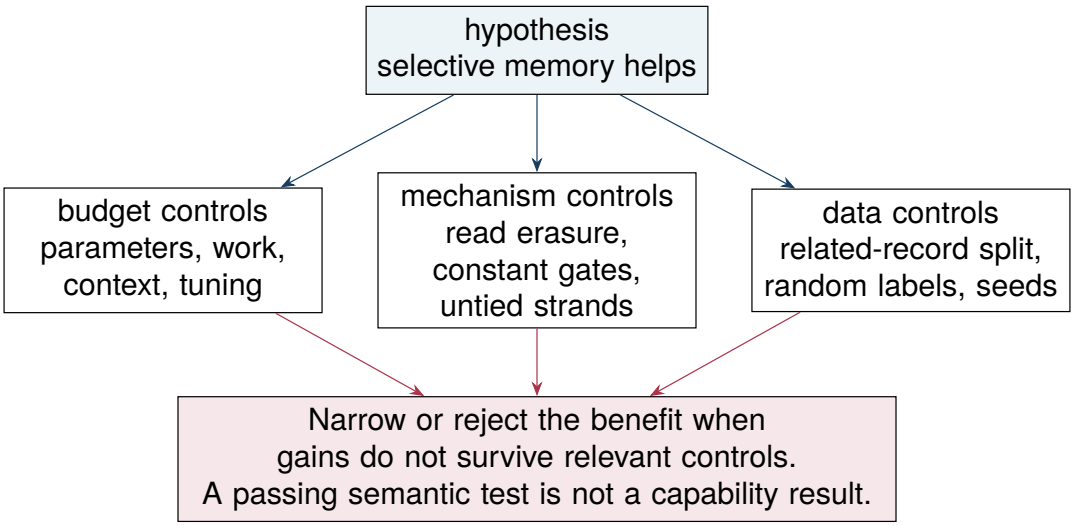


Figure 22.6: A mechanism claim survives only if its benefit remains after relevant baseline, budget and intervention controls. Negative evidence narrows the architecture rather than being discarded.

22.8 Closest established mechanisms

Gated recurrence is established. LSTM [52] introduced multiplicative gates to control information flow through recurrent state; GRU-like models likewise use learned update/reset behavior. Selective state-space models such as Mamba [10] make state updates input dependent and design parallel algorithms around the resulting recurrence. RWKV [31] also explores the training/inference trade between parallel sequence processing and recurrent execution.

DOGMA must therefore earn any distinction at a higher level of organization: typed state ownership, explicit addressable loci, strand actions, trace/intervention contracts, and eventually structural adaptation. If those pieces can be removed without loss after fair capacity matching, the proposed architecture should collapse toward the simpler established model. That is a scientifically useful outcome.

22.9 Falsification matrix

The research program can now be stated without slogans.

1. **Selective regulation.** Hypothesis: input-dependent gates improve tasks with sparse state changes. Reject or narrow if constant-gate capacity-matched models perform equivalently across seeds and interventions show no gate-specific effect.
2. **Addressable loci.** Hypothesis: explicit loci improve retention or mechanistic locality. Reject if a same-budget recurrent state matches performance, memory efficiency and intervention specificity.
3. **Strand sharing.** Hypothesis: parameter-tied dual processing improves sample efficiency or consistency on strand-symmetric tasks. Reject if ordinary augmentation achieves the same result at matched compute or if invariance hurts orientation-sensitive tasks.
4. **Trace utility.** Hypothesis: typed traces support reproducible causal diagnosis. Reject if trace variables are unstable across equivalent parameterizations and interventions do not validate their claimed roles.
5. **Structural adaptation.** Deferred hypothesis: changing active modules or topology can outperform fixed structure under a controlled adaptation budget. Chapter 22 does not implement this and therefore makes no positive claim about it.

22.10 What the reference model does not contain

There is no attention, no Transformer block, no DNA chemistry simulator, no evolutionary search, no persistent agent memory, no distributed runtime and no production kernel. Those omissions are intentional. Hermon DNA begins in Chapter 23 with a Transformer-based reference architecture. DOGMA Engine concerns begin in Chapter 30. Evolutor's compiler/runtime sits above both much later.

The present chapter has one responsibility: make the non-Transformer proposal specific enough that later engineering cannot retroactively change what an early experiment meant.

Key ideas

- Reads use M_{t-1} before the write at t. The assigned trace's first feature is $\tanh(1/4)$, not $\tanh(1/4 + 1/5)$.
- Bounded state is a forward invariant, not a gradient guarantee: state-dependent gates add Jacobian terms.
- Read the pilot honestly: on 128 balanced held-out records, a single run's accuracy has a chance-level standard error of about 0.044. Most learned models in this budget sit within two standard errors of 0.5.
- If loci, strand tying and typed traces add nothing after budget matching, DOGMA should collapse toward a GRU or SSM. That would be a useful result.

22.11 Exercises with worked answers

1. Who owns h_t and M_t? **Answer:** The request. Learned matrices and embeddings are model-owned.

2. Why is zero initialization part of the scientific contract? **Answer:** Hidden carry between independent records changes the function and can leak data across examples.

3. Is Equation 22.3 architecturally novel by itself? **Answer:** No. Learned gated recurrence is established.

4. Why is offline dual state illegal for next-token prediction? **Answer:** The aligned reverse branch contains suffix information.

5. Prove pooled Equation 22.3 is reverse-complement invariant. **Answer:** Applying R swaps the two shared-parameter branches; their sum and position average are unchanged after realignment.

6. Does invariance imply orientation-sensitive labels are impossible? **Answer:** For an invariant pooled head, yes. Use an equivariant output action or orientation metadata instead.

7. What does a high write gate prove? **Answer:** Only that the parameterized model emitted a high gate. A causal role requires intervention.

8. Why report parameter counts for baselines? **Answer:** Apparent mechanism gains can be capacity gains.

9. What is the random-label control expected to show? **Answer:** Independent held-out labels have chance population accuracy and cross-entropy at least $\log V$. Finite samples fluctuate; memorizing training noise establishes neither a violation nor the intended mechanism.

10. Design a locus-erasure intervention. **Answer:** Identify a time after the candidate information is written, zero one locus for matched examples, continue the same suffix, and compare output deltas against erasing other loci and sham edits.

11. When should DOGMA collapse toward a GRU/SSM? **Answer:** When explicit loci, strand contracts and typed interventions add no reproducible benefit after budget matching.

12. Design a genomic follow-up. **Rubric:** use a real provenance-tracked dataset; split homologous/related sequences safely; define strand label action; compare matched recurrent, SSM and Transformer baselines; use multiple seeds, calibration and

out-of-distribution lengths; pre-register ablations; distinguish task accuracy from biological mechanism claims.

13. Derive parameter count for $D = 12, K = 3, V = 4, C = 2$. **Answer:** $7(144) + 8(12) + 6(13) + 2(13) = 1208$. The orientations share parameters but perform separate arithmetic.

14. Which parity fails when carry is detached between chunks? **Answer:** Forward values can agree while gradients through prior writes disappear. The full gradient comparison requires undetached complete carry.

15. Does an independent random-label test loss below $\log 2$ prove leakage? **Answer:** No. That is a population expectation bound. Check finite-sample uncertainty, repeated seeds and independence before interpreting a deviation.

16. Compute the second feature of the assigned trace. **Answer:** $h_2 = 3/8$ and $M_1 = 1/5$, so $y_2 = \tanh(0.575) \simeq 0.519022$. Reading $M_2 = 0.35$ instead changes transition order.

17. One pilot run scores 0.5625 on 128 balanced held-out records. Is that evidence of learning? **Answer:** under chance the standard error is $\sqrt{0.25/128} \approx 0.0442$, so the run is $(0.5625 - 0.5)/0.0442 \approx 1.4$ standard errors above chance. That is weak evidence, and it says nothing about architecture without matched budgets and more seeds.

22.12 Vocabulary and next interface

Request-owned state: mutable state whose lifetime belongs to one inference or training record unless persistence is explicitly declared. **Model-owned parameter**: learned state shared across requests. **Mechanism intervention**: a controlled change to an internal variable intended to test a causal role. **Falsification criterion**: an observation that would reject or narrow a claimed benefit.

Chapter 23 starts a separate research direction inside Evolutor: Hermon DNA, a Transformer-based DNA model. Keeping the two reference architectures explicit makes their later comparison meaningful rather than turning "DNA-native" into a label that can absorb any successful mechanism.

Sources and evidence

Local verification on 6 October 2026 checked LSTM at publisher-indexed abstract/metadata level; direct full-page access failed. Mamba and RWKV were checked at arXiv abstract and version-metadata level, not as full reproductions. They establish neighboring recurrent/selective mechanisms, not DOGMA's novelty or measured superiority. Chapters 18–21 own the internal state, regulation, memory and strand contracts. This chapter's learned-locus variant, transition trace, tests, interventions and multi-seed synthetic pilot are locally specified artifacts. No external benchmark, trained genomic capability or production engine is reproduced.

PART V

Hermon DNA: Transformer-based DNA computation

What must a Transformer-based DNA model specify before its genomic claims mean anything?

Hermon DNA starts from an ordinary pre-norm Transformer over single nucleotides and makes every genomic convention explicit: masks, padding, reverse-complement readouts, base spans, span-level causality, strand embeddings, rotary positions in base units and long-context access graphs. The pilot experiment is reported as it came out: under its small budget, the Hermon reference did not fit the task while a four-wide convolution partly did. Treat this as a lesson, not an embarrassment. Correct algebra and a trained model are different achievements.

CHAPTERS IN THIS PART

23 Hermon DNA reference architecture Implement Hermon DNA as a Transformer-based research model.

24 DNA-aware Transformer mechanisms Bind Transformer mechanisms to genomic spans, coordinates and access.

CHAPTER 23

Hermon DNA reference architecture

In this chapter you will

- specify the Hermon DNA reference as a pre-norm Transformer over single nucleotides
- trace one attention head by hand and check attention against an independent row oracle
- separate offline and causal masks from padding, and build a shared-parameter reverse-complement-invariant head
- read a small trained pilot correctly, including a model that fails to learn

Chapter 22 made DOGMA executable by refusing to let its name substitute for a model. Hermon DNA deserves the same treatment. Its defining architectural commitment is different: **Hermon DNA** is the Transformer-based DNA-model research direction. It inherits DNA-specific data and symmetry questions from Evolutor, but its sequence mixing primitive is self-attention rather than DOGMA's recurrent fast state and addressable loci.

This immediately removes one possible novelty claim. Transformers are established, and Transformer genomic language models are established. DNABERT adapted BERT-style masked pretraining to DNA; Nucleotide Transformer scaled Transformer genomic pretraining and systematic downstream evaluation. [53, 54] Hermon DNA must therefore earn value through a precise architecture/evidence contract and later engineering choices, not by renaming a Transformer.

23.1 Input contract: begin with nucleotides

The reference vocabulary is

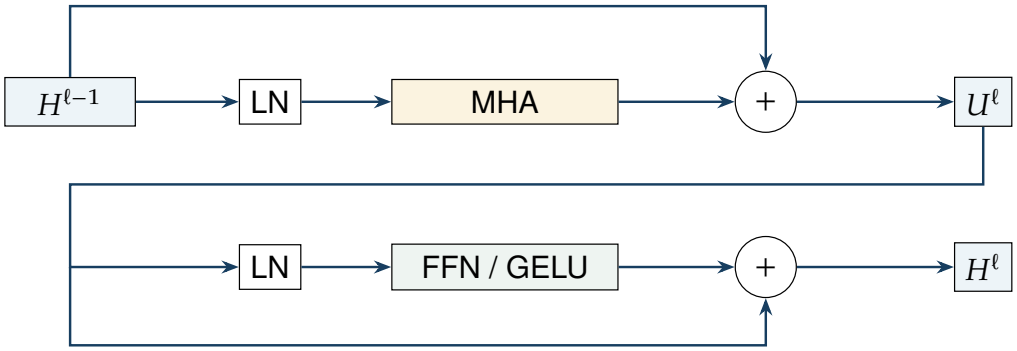
$$\mathcal{V} = \{A, C, G, T\} \leftrightarrow \{0, 1, 2, 3\}.$$

A batch is $X \in \{0, 1, 2, 3\}^{B \times T}$. We begin at single-nucleotide resolution because it makes position, complement and mutation semantics transparent. This is not a claim that nucleotide tokenization is universally optimal. DNABERT used overlapping k-mers; DNABERT-2 argues for BPE-like tokenization; long-context models have revisited single-base representations. Tokenization is therefore an experimental variable, not a biological truth. [16, 15]

Learn token embeddings $E_{\text{tok}} \in \mathbb{R}^{4 \times D}$ and position embeddings $E_{\text{pos}} \in \mathbb{R}^{T_{\max} \times D}$:

$$H^{(0)}_{b,t} = E_{\text{tok}}[X_{b,t}] + E_{\text{pos}}[t].$$

The reference uses learned absolute positions because they are easy to inspect. Later chapters may replace them with rotary, relative or other positional schemes only with an explicit long-context argument.



All main-path tensors have shape $B \times T \times D$.
Residual branches bypass normalization and their sublayer; they do not create persistent carry.

Figure 23.1: One Hermon DNA Transformer block. The reference is pre-norm: normalization precedes each sublayer and residual paths preserve the token matrix.

23.2 Self-attention from shapes to equations

For H heads, require $D = Hd_h$. From a normalized token matrix $U \in \mathbb{R}^{B\times T\times D}$, learned projections produce

$$Q, K, V \in \mathbb{R}^{B\times H\times T\times d_h}.$$

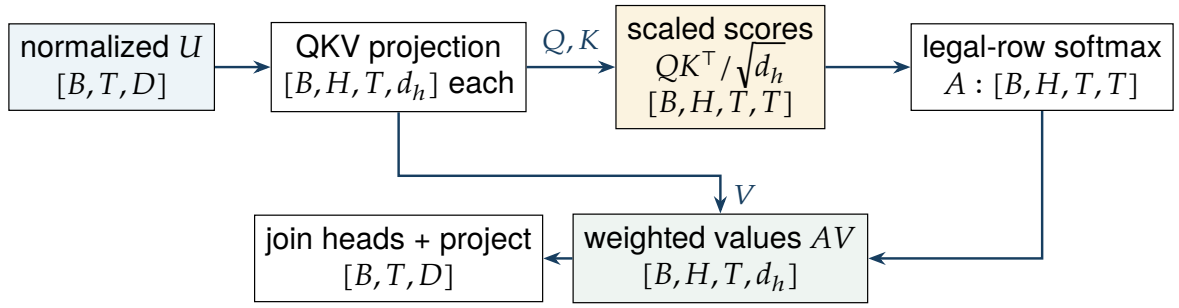
For one batch/head, the score between query position i and key position j is

$$S_{ij} = \frac{Q_i K_j^{\top}}{\sqrt{d_h}}. \tag{23.1}$$

If query/key coordinates are independent, centered and have unit variance, a dot product's variance is d_h; division by $\sqrt{d_h}$ makes it one. Learned features need not satisfy those hypotheses. The factor is a useful scale convention, not a theorem that every trained score has bounded magnitude. After adding a mask $M_{ij} \in \{0, -\infty\}$,

$$A_{ij} = \frac{\exp(S_{ij} + M_{ij})}{\sum_k \exp(S_{ik} + M_{ik})}, \qquad O_i = \sum_j A_{ij} V_j.$$

Each row of A is nonnegative and sums to one up to floating arithmetic. This statement requires at least one legal key. An entirely masked row has no probability distribution; $\mathrm{softmax}(-\infty, \ldots, -\infty)$ is undefined. The reference explicitly assigns zero attention and zero projected output to that row, including the output bias. It is a padding convention, not a new attention law.



Scores compare queries to keys. Probabilities mix value vectors.
No legal key: return zero attention/output rather than an undefined softmax.

Figure 23.2: Attention tensor contract. Shapes are part of correctness: mixing the head and position axes can produce numerically valid tensors with the wrong computation.

The companion code implements these equations without calling `nn.MultiheadAttention`. A separately written oracle reconstructs the learned linear projections, enumerates each row's allowed key indices independently, and checks both attention weights and output tensors.

```
    def forward(self, x, mask=None):
        if (
            x.ndim != 3
            or x.shape[0] == 0
            or x.shape[1] == 0
            or x.shape[-1] != self.d
            or x.dtype != self.qkv.weight.dtype
            or x.device != self.qkv.weight.device
            or not bool(torch.isfinite(x).all())
        ):
            raise ValueError("finite activation [B,T,D] matching model required")
        b, t, d = x.shape
        q, k, v = self.qkv(x).view(b, t, 3, self.h, self.dh).permute(2, 0, 3, 1,
         ↪  4)
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.dh)
        if mask is None:
            mask = torch.zeros(b, t, t, dtype=torch.bool, device=x.device)
        elif (
            mask.dtype != torch.bool
            or mask.device != x.device
            or mask.shape not in ((t, t), (b, t, t))
        ):
            raise ValueError("boolean [T,T] or [B,T,T] mask required")
        if mask.ndim == 2:
            mask = mask.expand(b, -1, -1)
        active = (~mask).any(-1)
        scores = scores.masked_fill(mask[:, None], float("-inf"))
        # Softmax on an all-masked row is undefined. Choose zero attention, not
         ↪  NaN.
        safe = torch.where(active[:, None, :, None], scores,
         ↪  torch.zeros_like(scores))
        a = torch.softmax(safe, -1) * active[:, None, :, None]
        y = (a @ v).transpose(1, 2).contiguous().view(b, t, d)
        return self.out(y) * active[:, :, None], a
```

23.2.1 Trace one head before stacking layers

Choose $B = H = 1$, $T = 3$, $d_h = 2$ with assigned, not learned, vectors

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}.$$

The causal first row has one allowed key, so $A_{1,:} = (1, 0, 0)$ and $O_1 = (1, 0)$. For the second query, allowed scores are $(0, 1/\sqrt{2})$. Thus $A_{2,1} = 1/(1 + e^{1/\sqrt{2}})$ and $A_{2,2} = 1 - A_{2,1}$; the third probability is exactly zero. Matrix multiplication yields $O_2 = (A_{2,1}, 2A_{2,2})$. All three rows below are generated directly from these assigned tensors.

Query	A1	A2	A3	O1	O2
1	1	0	0	1	0
2	0.330238	0.669762	0	0.330238	1.33952
3	0.248255	0.248255	0.50349	1.75872	1

Attention weights mix *value vectors*, not token IDs or probability of the next base. An LM head and its vocabulary softmax perform the latter decision. Changing a future value cannot affect an earlier causal output because its attention coefficient is zero. Changing an earlier key changes the normalization over every legal key in that query's row.

23.3 Residual block and parameter ownership

For block ℓ,

$$U^{(\ell)} = H^{(\ell-1)} + \mathrm{MHA}_\ell(\mathrm{LN}_{\ell,1}(H^{(\ell-1)})), \tag{23.2}$$

$$H^{(\ell)} = U^{(\ell)} + \mathrm{FFN}_\ell(\mathrm{LN}_{\ell,2}(U^{(\ell)})). \tag{23.3}$$

The feed-forward network is

$$\mathrm{FFN}(x) = W_2\ \mathrm{GELU}(W_1 x + b_1) + b_2.$$

All embeddings, projections, normalization scales and head parameters are model-owned. The token matrix is activation state scoped to one forward pass. Unlike DOGMA Chapter 22, Hermon DNA has no persistent recurrent state in this reference architecture.

For this implementation, a block uses $8D^2 + 11D$ parameters: QKV and output projections use $4D^2 + 4D$, the width-$2D$ FFN uses $4D^2 + 3D$, and the two LayerNorms use $4D$. With L blocks and two distinct heads, total count is

$$P = (4 + T_{\max})D + L(8D^2 + 11D) + 2D + 4(D + 1) + C(D + 1). \tag{23.4}$$

The LM head has four outputs, classification has C outputs, and neither head is tied to the embedding. A classifier-only optimizer gives no gradients to the LM head, although this head is included in the architecture's reported total. Weights are model-owned; padding masks and attention probabilities are temporary per-forward-pass objects, not request-persistent state.

23.4 Two masks, two scientific questions

An offline encoder and a causal language model are different functions. For an offline genomic classifier, every unpadded position may attend to every other unpadded position. For causal

next-token prediction,

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Padding is orthogonal: padded keys should be hidden in either mode, and padded query outputs should not be scored as biological positions.

Let $v_{b,t} \in \{0, 1\}$ declare validity, and define legal edges by $v_{b,i}v_{b,j} = 1$, additionally $j \le i$ for causal mode. The code accepts a boolean disallow mask of shape $T \times T$ or $B \times T \times T$; it rejects other shapes rather than relying on accidental broadcasting. The returned probabilities have shape $B \times H \times T \times T$. Valid rows sum to one; invalid-query rows sum to zero.

Positional embeddings use the ordinal among *valid* bases: $p_{b,t} = \sum_{s \le t} v_{b,s} - 1$ at valid positions. Inserting left or right padding therefore does not shift biological coordinates. Invalid query outputs are reset after each residual block and final normalization because learned biases otherwise revive them. The causal head also zeros invalid logits; a training loss must still exclude them. Zero logits are not predictions for an unknown biological base.

```
    def encode(self, x, causal=False, valid=None):
        valid = self.validity(x, valid)
        # Valid-token ordinal: inserting padding does not shift biological
         ↪ positions.
        pos = (valid.long().cumsum(1) - 1).clamp_min(0)
        z = (self.tok(x) + self.pos(pos)) * valid[:, :, None]
        mask = legality(valid, causal)
        attention = []
        for block in self.blocks:
            z, a = block(z, mask)
            z = z * valid[:, :, None]  # norm/FFN biases must not revive invalid
             ↪ queries
            attention.append(a)
        return self.norm(z) * valid[:, :, None], attention
```

Offline pooling divides the sum of valid activations by their count. A record with no valid bases is rejected rather than given an invented classification. Causal encoding allows an all-padding record with zero outputs. Empty time axes, out-of-alphabet tokens, non-int64 token IDs and overlength records are rejected. Ambiguous biological symbols and special masking tokens are outside this chapter's four-base vocabulary, not silently encoded as A.

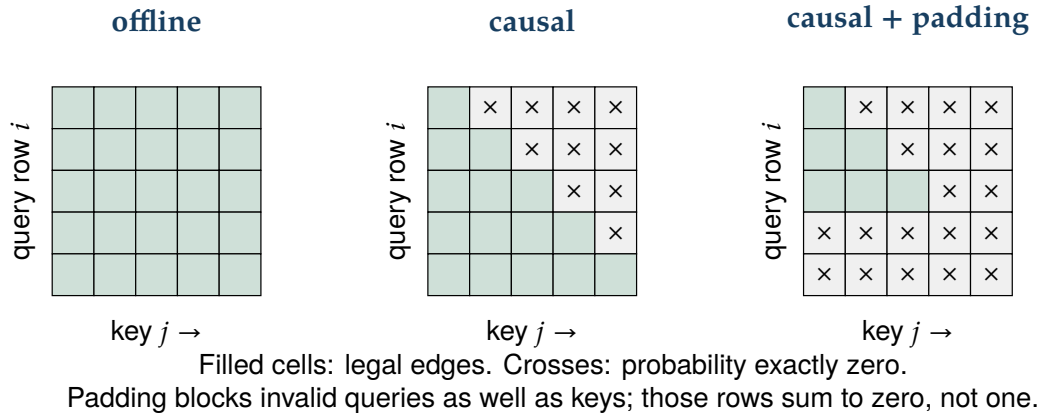


Figure 23.3: Offline and causal attention differ in legal information access. The causal triangle is not a performance optimization; it is part of the statistical objective.

The test suite mutates a suffix and verifies that causal logits for the unchanged prefix are invariant. It also inspects the attention matrix and confirms zero probability on masked future keys. These tests defend the Chapter 21/22 causality boundary at a different architecture.

For causal language modeling the target shift remains

$$\mathcal{L}_{\text{causal}} = -\sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\le t}).$$

An offline masked-token objective instead hides selected input positions and predicts their identities using legal bidirectional context. The dataset must record which positions were masked so the model cannot see the target token through an accidental copy path. The present code implements causal logits and whole-record classification, not a masked-LM training pipeline. Implementing the latter requires an expanded vocabulary/corruption contract or a different explicit base-obscuring representation. It is not accomplished merely by switching the triangular attention mask off.

23.5 Reverse complement is an action, not augmentation folklore

Define complement $c(A) = T, c(C) = G, c(G) = C, c(T) = A$. For a length-$T$ record

$$R(x_1, \dots, x_T) = (c(x_T), \dots, c(x_1)).$$

For tasks whose label is strand invariant, a simple reference construction shares one encoder across X and $R(X)$ and averages the two prediction logits:

$$f_{\text{sym}}(X) = \frac{f_\theta(X) + f_\theta(R(X))}{2}.$$

Applying R swaps the summands, so $f_{\text{sym}}(R(X)) = f_{\text{sym}}(X)$ exactly up to floating-point evaluation order. With padding, the reverse branch reverses the validity mask as well as the sequence. Otherwise a padded key can become a valid biological base, while a real base is silently discarded. Valid-token ordinals preserve the same sequence positions when padding moves from one end to the other. The paired-head invariance follows from an involution on the *sequence-and-mask pair*, not the sequence alone.

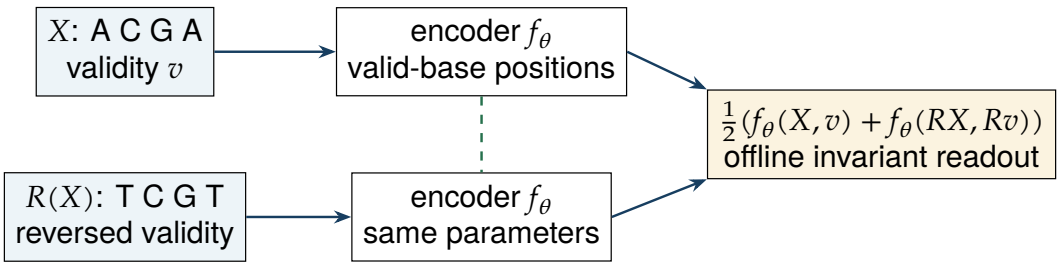


Figure 23.4: A shared-parameter offline symmetric head gives a transparent reverse-complement invariance contract. It doubles encoder work and is not used to smuggle suffix information into causal prediction.

This construction is intentionally simple, not claimed as frontier architecture. Caduceus explicitly develops bidirectional and reverse-complement-equivariant DNA sequence modeling on a Mamba-derived architecture, demonstrating that RC structure is established research territory and not specific to Transformers. [48] A Hermon DNA contribution must compare against such prior mechanisms rather than claiming the symmetry itself.

23.6 DOGMA and Hermon DNA remain separate

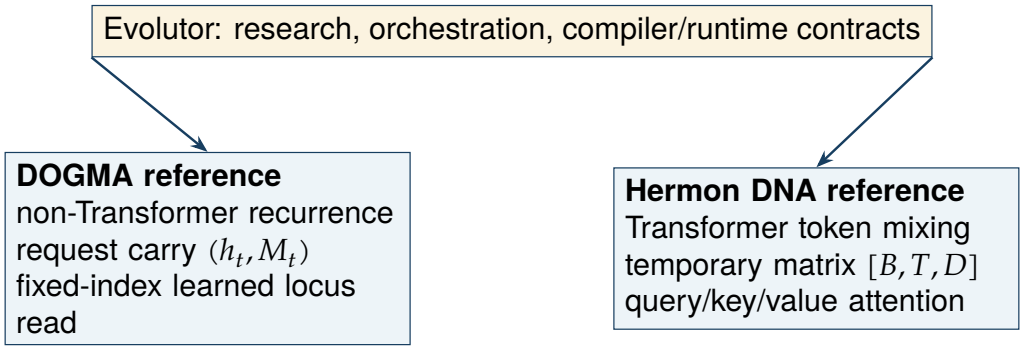


Figure 23.5: Evolutor sits above two research branches. DOGMA's primitive is typed recurrent/selective state with explicit loci; Hermon DNA's primitive is a Transformer token matrix mixed by self-attention.

This separation makes experiments interpretable. If a task favors Hermon DNA, the result supports some combination of token-level attention, optimization and model capacity; it does not disprove genomic organization as an inspiration. If DOGMA wins, the result does not show that Transformers are intrinsically unsuitable for DNA. Architecture comparisons require matched data, objective, parameter/compute budgets and tuning effort.

The engines are also different artifacts. **Hermon DNA Engine** will eventually refer to the inference/training systems implementation for this Transformer branch. The present PyTorch reference is a semantic oracle for that future engine, not the engine itself.

23.7 Complexity and the long-context boundary

Dense attention forms a $T \times T$ score matrix per head, giving $O(T^2D)$ attention arithmetic at the usual level of abstraction and $O(T^2)$ attention-score storage if materialized. Here batch and head factors matter: one layer retains BHT^2 score/probability entries, not merely T^2. QKV projections and the FFN also cost $O(BTD^2)$ at fixed expansion ratio. With element size s bytes, one materialized score tensor occupies $sBHT^2$ bytes before gradients and other activations. Doubling T quadruples this term, while a two-orientation head performs two encoder traversals even with shared weights. Genomic context can be much longer than ordinary short classification records. HyenaDNA was motivated in part by this quadratic attention boundary and explored long single-nucleotide contexts with a different sequence operator. Nucleotide Transformer likewise identifies long-input handling as an important open direction. [15, 54]

Therefore Hermon DNA cannot merely scale this pedagogical implementation to a whole chromosome. Later architecture chapters must choose among sparse/local attention, hierarchy, recurrence/state-space hybrids, chunking, retrieval or other mechanisms and measure what biological dependencies are lost or preserved.

23.8 A controlled executable experiment

The integrated pilot uses a small whole-record classification task: a record is positive if it contains ACGT. This pattern is its own reverse complement; there are not two distinct motifs.

Positives are implanted when necessary and negatives are rejection sampled. This differs from Chapter 22's three-base ACG/CGT task; scores across these chapters are not direct architectural comparisons.

Data seeds 137 and 139 freeze 512 training and 128 held-out records, respectively. Exact sequences and their reverse complements are disjoint between these sets. An independent Python string detector checks every label and has accuracy one by specification. This proves recoverable synthetic signal, not learned biology. Initialization seeds 11, 23 and 37 run the same declared 25 full-batch AdamW epochs at learning rate 3×10^{-3} without held-out tuning.

```
def train_eval(model, xtr, ytr, xte, yte, epochs=25):
    positive(epochs)
    opt = torch.optim.AdamW(model.parameters(), lr=3e-3)
    ce = nn.CrossEntropyLoss()
    losses = []
    model.train()
    for _ in range(epochs):
        opt.zero_grad()
        loss = ce(model.offline_rc_symmetric_logits(xtr), ytr)
        loss.backward()
        opt.step()
        losses.append(float(loss.detach()))
    model.eval()
    with torch.no_grad():
        z = model.offline_rc_symmetric_logits(xte)
        return {
            "loss": ce(z, yte).item(),
            "accuracy": (z.argmax(-1) == yte).float().mean().item(),
            "params": sum(p.numel() for p in model.parameters()),
            "train_loss": losses,
        }
```

Model	Mean accuracy	Seed SD	Parameters
Hermon	0.528646	0.0090211	10806
CNN width-4	0.723958	0.152761	154
Hermon random	0.541667	0.0225527	10806

These are means and sample standard deviations of locally rerun pilots. The width-four CNN tests the local-motif inductive bias. Both neural families use the same examples and two orientation passes, but differ in parameters and arithmetic; equal epochs are not equal compute. A separate Hermon control draws independent random Bernoulli targets for both training and test. For an independent uniform target, population expected cross-entropy is at least $\log 2$; individual finite samples or memorized training losses need not obey that bound. All per-seed losses, accuracies, parameter counts and training-loss traces are stored in the generated results, including weak or failed runs. No genomic, capacity-matched or compute-matched ranking follows from this table.

Now read the numbers. Classes alternate, so the held-out set is balanced and chance accuracy is $1/2$, with a single-run standard error of $\sqrt{0.25/128} \approx 0.044$. The Hermon runs score 0.52, 0.52 and 0.54, indistinguishable from the random-label control $(0.52–0.55)$. Their training losses fall from about 0.71 to 0.692 in the first epochs and then stay near $\ln 2 \approx 0.693$ for the rest of the run: under this budget the Transformer did not fit even its training set. The

four-wide CNN, whose inductive bias matches a local motif, learns in two seeds (0.71 and 0.88) and only weakly in the third (0.58).

This is a negative result, and a useful one. The algebra of the preceding sections is correct, and the tests below confirm it. A correct implementation is still not a trained model, and 25 full-batch epochs at this learning rate do not train this one. Chapter 25 uses the pilot as its worked diagnostic: overfit a tiny subset, inspect gradient norms and learning-rate sensitivity, then extend training under a declared budget. The task is deliberately synthetic. Even a successful run would not show promoter recognition, variant-effect prediction, regulatory reasoning or genomic foundation-model capability.

A proper genomic benchmark must address homology leakage, chromosome/species split policy, class balance, label provenance, sequence overlap, reverse-complement duplication, context length, parameter/compute budgets and repeated seeds. Nucleotide Transformer's systematic downstream benchmarking is relevant precedent for why a foundation-model claim needs broad controlled evaluation rather than one task. [54]

23.9 What current genomic models teach the design

DNABERT shows that Transformer masked pretraining can transfer to several genomic prediction tasks. DNABERT-2 makes tokenization efficiency itself an architectural question. Nucleotide Transformer shows the importance of scale, data diversity and consistent downstream protocols. HyenaDNA asks whether long genomic context is better served by a non-attention sequence operator. Caduceus makes bidirectionality and RC equivariance architectural. Evo 2 pushes single-nucleotide genomic modeling to very large model/data/context scales using a different modern architecture; its reported capabilities are frontier context, not evidence produced by Hermon DNA. [53, 16, 54, 15, 48, 49]

The resulting Hermon research questions are concrete: Which tokenizer preserves mutation-level semantics per unit compute? Which positional scheme extrapolates? When does RC symmetry improve sample efficiency? Which attention pattern captures distal regulation without quadratic cost? Which pretraining objective transfers across species without rewarding trivial composition statistics?

23.10 Evidence ladder

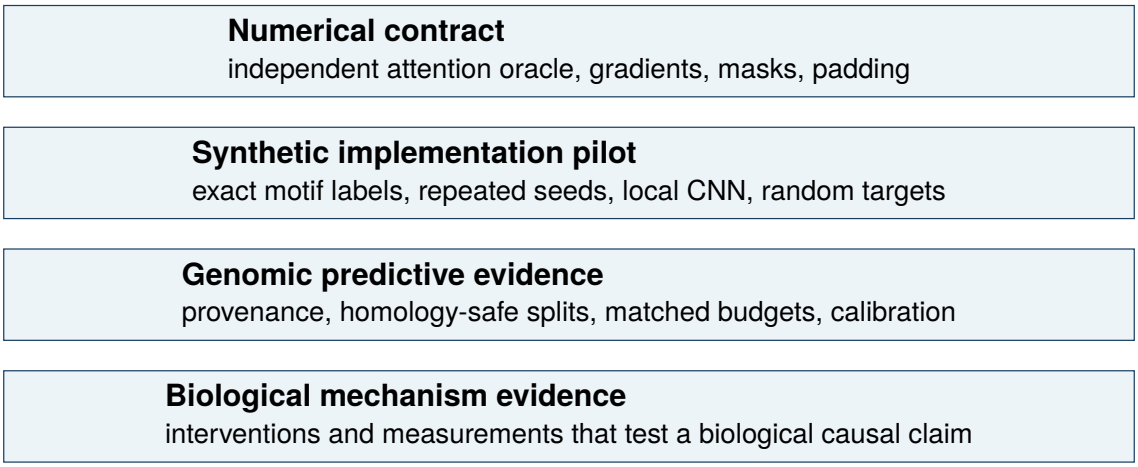


Figure 23.6: Correct tensor algebra is necessary but sits at the bottom of the evidence ladder. Synthetic learnability, genomic benchmark performance and biological mechanism claims require progressively stronger data and controls.

A unit test can establish that the causal mask works. It cannot establish that a model learned causality in biology. A synthetic motif task can establish whether the implementation trains under a stated budget; here it showed that it did not. It cannot establish regulatory grammar. A held-out genomic benchmark can establish predictive performance under its split. It does not by itself establish the molecular mechanism used by a cell. The chapter keeps those claims separate so later results have somewhere honest to land.

Key ideas

- An all-masked row has no softmax. The reference assigns it zero attention and zero output as an explicit padding convention.
- Reverse-complement invariance follows from an involution on the sequence-and-mask pair. Reversing the sequence without its validity mask breaks it.
- Dense attention keeps BHT^2 scores per layer. Doubling T quadruples that term, which is why long genomic context needs a different access pattern.
- In the 25-epoch pilot, Hermon's training loss stays at $\ln 2$ and its held-out accuracy equals the random-label control. A correct implementation is not a trained model.

23.11 Exercises and worked solutions

Independent implementation checks

The test suite compares attention values and gradients against an independent allowed-key row oracle in float64. Central finite differences additionally check the attention module's input derivatives. A future-feature perturbation changes offline prefix outputs but leaves causal prefix outputs fixed. Both left and right padding are tested; invalid queries/keys have zero probabilities and inserting padding leaves valid-token encodings unchanged. These are executed software contracts, not assertions of a biological mechanism.

```python
def attention_oracle(module, x, mask=None):
    """Independent row-by-row allowed-index oracle; no masked softmax reuse."""
    b, t, d = x.shape
    raw = torch.nn.functional.linear(x, module.qkv.weight, module.qkv.bias)
    q, k, v = [
        z.view(b, t, module.h, module.dh).transpose(1, 2) for z in raw.chunk(3, -1)
    ]
    weights = torch.zeros(b, module.h, t, t, dtype=x.dtype, device=x.device)
    rows = []
    for bi in range(b):
        heads = []
        for hi in range(module.h):
            output = []
            for i in range(t):
                allowed = (
                    torch.ones(t, dtype=torch.bool, device=x.device)
                    if mask is None
                    else ~(mask[i] if mask.ndim == 2 else mask[bi, i])
                )
                if bool(allowed.any()):
                    scores = torch.stack(
                        [
                            torch.dot(q[bi, hi, i], k[bi, hi, j])
                            for j in range(t)
                            if allowed[j]
                        ]
                    ) / math.sqrt(module.dh)
                    p = scores.softmax(0)
                    weights[bi, hi, i, allowed] = p
                    output.append((p[:, None] * v[bi, hi, allowed]).sum(0))
                else:
                    output.append(
                        torch.zeros(module.dh, dtype=x.dtype, device=x.device)
                    )
            heads.append(torch.stack(output))
        rows.append(torch.stack(heads))
    y = torch.stack(rows).transpose(1, 2).reshape(b, t, d)
    active = weights.sum(1).sum(-1) > 0
    return torch.nn.functional.linear(y, module.out.weight, module.out.bias) * active[
        :, :, None
    ], weights
```

Worked questions

1. Why require $D = Hd_h$? **Answer:** the reference partitions the model dimension evenly across heads before concatenating them.
2. What is the shape of attention scores? **Answer:** $[B, H, T, T]$.
3. Why divide by $\sqrt{d_h}$? **Answer:** to control dot-product scale as head dimension grows, stabilizing softmax behavior.
4. Prove each unmasked softmax row sums to one. **Answer:** divide each positive exponential by the sum of all allowed exponentials in that row.

5. What does the causal mask prevent? **Answer:** a query at i reading keys/values from $j > i$.

6. Why is padding masking separate? **Answer:** padding is invalid data in both causal and offline modes; causality concerns temporal/positional access.

7. Prove Equation 23.5 is RC invariant. **Answer:** R is an involution and applying it swaps the two shared-parameter summands.

8. Is RC invariance always correct? **Answer:** no; orientation-sensitive labels require an equivariant transformation or explicit orientation.

9. Why is single-base tokenization not declared optimal? **Answer:** tokenizer choice trades resolution, sequence length, statistical efficiency and compute; existing genomic models use different choices.

10. Compare DOGMA state and Hermon activations. **Answer:** DOGMA explicitly carries recurrent request state/loci across positions; this Hermon reference computes a token activation matrix inside stacked Transformer blocks without persistent recurrent state.

11. What falsifies a claimed Hermon advantage? **Answer:** disappearance of the gain under matched data, compute, capacity, tuning and multiple seeds, or a simpler baseline matching it.

12. Compute the second row of the assigned attention trace. **Answer:** with $a = 1/(1 + e^{1/\sqrt{2}})$, probabilities are $(a, 1-a, 0)$ and output is $(a, 2(1-a))$.

13. Why is an all-masked softmax not a uniform biological prediction? **Answer:** it has no legal key and is mathematically undefined. This reference assigns zero attention/output as a padding convention.

14. Why reverse the padding mask in the offline RC branch? **Answer:** validity belongs to the transformed sequence; leaving it fixed changes which bases are observed and violates the pair-action contract.

15. Does fixed-epoch comparison match FLOPs? **Answer:** no. Projection sizes, head counts and operator types change per-epoch work.

16. The Hermon pilot's training loss stays at 0.692 for most of training. What does that tell you? **Answer:** $\ln 2 \approx 0.693$ is the loss of a constant prediction of $(1/2, 1/2)$ on balanced labels. The model has not fit its training set, so held-out accuracy near 0.5 reflects optimization under this budget rather than overfitting or a test-set problem.

17. Derive Equation 23.4 at $D = 24$, $L = 2$, $T_{\max} = 32$, $C = 2$. **Answer:** embeddings use 864, blocks use 9744, final norm uses 48, and heads use 150: total 10806.

18. Design the first real benchmark. **Rubric:** provenance-tracked genomic dataset; leakage-safe chromosome/species/homology split; explicit strand label action; CNN/Transformer/non-attention baselines; matched budgets; multiple seeds; calibration and OOD context tests; ablate tokenization, RC handling and positional scheme.

23.12 Bridge

Chapter 24 tests DNA-aware Transformer mechanisms: token spans, strand-sensitive features, position contracts and long-context access. Chapter 25 then freezes shared data/model interfaces for comparisons across DOGMA, Hermon and conventional baselines. Dataset coordinate, masking, corruption, window, target and split contracts remain necessary; an architecture alone is not a reproducible genomic model.

Sources and evidence

Local checks on 6 October 2026 inspected Transformer, DNABERT-2, HyenaDNA and Caduceus arXiv abstracts/version records, not their full experiments. DNABERT's original author repository supplied its citation and README pretraining/tokenization contract; direct publisher access did not expose the methods text in this check. Nucleotide Transformer was verified through indexed primary abstract/metadata; direct publisher and PMC full-text access failed. Evo 2 was checked in publisher HTML for abstract and publication metadata. The bibliography cites its Nature version of record. Access depths and unresolved replication gates are recorded separately.

All equations, literal reference listings, assigned-tensor trace, tests and local three-seed pilots are teaching artifacts. Prior reported performance is not reproduced or attributed to Hermon. No biological mechanism, genomic capability, optimized engine or independent specialist review is established by this chapter.

CHAPTER 24

DNA-aware Transformer mechanisms

In this chapter you will

- give every token a base span and compute exact mutation footprints
- state span-level causality: a token containing the target base is illegal context
- build a tied strand embedding with an exact reverse-complement action
- derive rotary positions in base units and express long-context access as an attention graph with layer-counted paths

Chapter 23 made Hermon's Transformer algebra explicit. That does not yet make an architecture DNA-aware. DNA-specific information has to enter through mechanisms whose effect can be stated, implemented and tested: base spans, strand actions, physical coordinate distances and legal long-context access.

Consider a variant changing one base in a six-base record. Which tokens change? Does a position mean a token index or a base coordinate? If the record is reverse complemented, which features should change sign, which positions should reverse, and which label should remain the same? Finally, can distant positions communicate without allocating a dense attention matrix? This chapter answers those questions with small differentiable references, not a new untested architectural name.

24.1 A token must carry its genomic span

Let a canonical sequence be $x = (x_0, \dots, x_{L-1}) \in \{A, C, G, T\}^L$, $L > 0$. A token record is

$$t_i = (u_i, a_i, b_i), \qquad 0 \le a_i < b_i \le L,$$

where $u_i = x[a_i : b_i]$ and $[a_i, b_i)$ is a zero-based half-open base span. This local window convention is not automatically the coordinate system of a reference assembly; that mapping must also name the assembly, chromosome and orientation. Chapter 25 consolidates those dataset-level contracts.

For width k and stride s, the executable reference emits spans $[js, js + k)$ while the end stays inside the record. It rejects unsupported symbols, empty records and invalid widths/strides rather than silently converting ambiguity to a canonical base. Ambiguity handling is a distinct tokenizer policy.

```
def tokenize(sequence, k=1, stride=1):
    canonical(sequence)
    integer(k, 1)
    integer(stride, 1)
    if k > len(sequence):
        raise ValueError("token width exceeds record")
    return [
        Span(sequence[i : i + k], i, i + k)
        for i in range(0, len(sequence) - k + 1, stride)
    ]
```

Token	Start	End
ACG	0	3
CGT	1	4
GTA	2	5
TAC	3	6

The generated table uses ACGTAC, $k = 3$, $s = 1$. A substitution at base coordinate two changes tokens zero, one and two. Token three does not include that base. The exact mutation footprint is

$$\mathcal{F}(p) = \{i : a_i \leq p < b_i\}.$$

This is a coordinate statement, not a claim that all affected model activations remain local after attention.

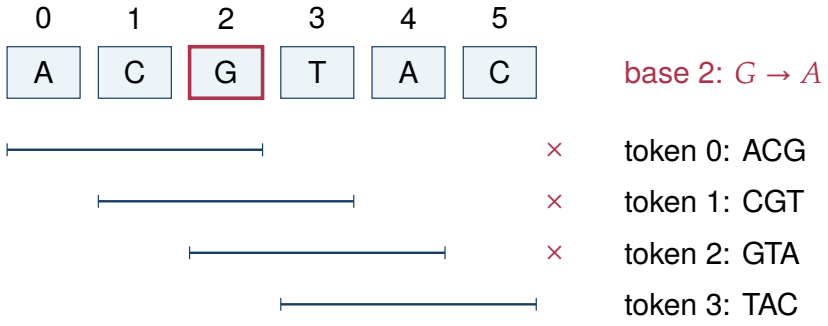


Figure 24.1: One substitution changes every overlapping token whose span contains the altered base. The base axis and token spans remain visible, so the number of changed tokens is not confused with the number of changed nucleotides.

Overlapping k-mers do not necessarily shorten token count: at stride one, $T = L - k + 1$. Larger stride can shorten the sequence but leave terminal bases unrepresented. A learned variable-span tokenizer must expose an analogous mapping if variant attribution and physical distances are to remain meaningful. DNABERT and DNABERT-2 supply established tokenizer design points, not evidence that Hermon's chosen tokenizer is optimal [53, 16].

24.1.1 Reverse complement acts on spans, not only strings

A local span transforms under reverse complement as

$$(u, a, b) \longmapsto (R(u), L - b, L - a). \tag{24.1}$$

Applying this twice returns the original span. Reversing a list of all stride-one k-mers after this transformation equals tokenizing $R(x)$ with the same policy.

That equality can fail for strided tokenization. For ACGTA, $k = 2$, stride two, forward spans are $[0, 2)$ and $[2, 4)$, leaving base four uncovered. Their reversed spans are $[1, 3)$ and $[3, 5)$, whereas independently tokenizing the RC record starts again at zero. The tokenizer-grid origin changed. RC consistency requires a boundary/grid policy; complementing token IDs alone does not repair the missed-base geometry.

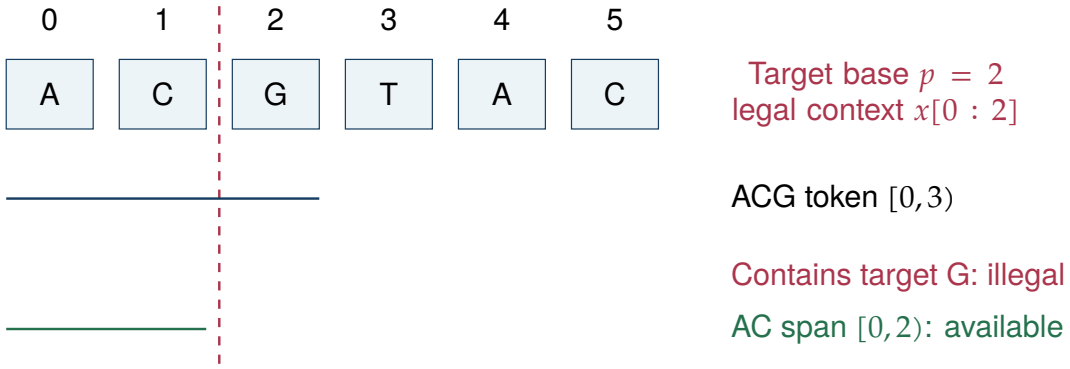
24.2 Token causality is weaker than base causality

Suppose the prediction target is base x_p and legal context is $x[0:p]$. An input token containing any base at coordinate p or later is illegal, even if its token index lies below the diagonal in a token-attention matrix.

For example, token ACG spans $[0,3)$ and already contains target G at base two. Using that token to predict the third base is leakage. A triangular attention mask cannot remove information encoded inside the input token itself. The necessary span-availability condition is

$$b_i \le p \quad \text{for every feature used to predict } x_p. \tag{24.2}$$

```
def causal_span_leaks(spans, prefix_end):
    """Prediction of base prefix_end may use only bases at coordinates <
     ↪  prefix_end."""
    integer(prefix_end)
    return [i for i, span in enumerate(spans) if span.end > prefix_end]
```



Token-level causality is insufficient if an input token already encodes the target base.

Figure 24.2: The information boundary is a base coordinate. A token ending after that boundary is illegal even when the token-level attention edge is causal. No full-record RC, corruption cache or pooled suffix feature may bypass this gate.

For causal token prediction, the prediction target can instead be the next token identity. Then overlapping adjacent tokens may reveal most of that next identity; the conditional entropy and interpretation differ from next-base prediction. Declare which target is scored. A low overlapping-token loss is not automatically a comparable nucleotide likelihood.

Masked-token objectives need a related check. Masking one token can leave its target bases visible inside neighboring overlapping tokens. To obscure base p, hide or consistently corrupt all spans in $\mathcal{F}(p)$ and any derived cached features carrying the same information. This chapter computes the footprint; it does not implement a masked pretraining pipeline. Causal, masked and whole-record objectives remain distinct, and Chapter 21's offline dual-state representation must not silently feed the causal case.

24.3 Build a strand action into a feature space

An unrestricted embedding lookup gives each base an unrelated learned vector. It does not impose complement symmetry. A small alternative makes the symmetry inspectable by separating even and odd feature channels.

Define pair class $g(A) = g(T) = 0, g(C) = g(G) = 1$ and signs $\sigma(A) = \sigma(C) = +1$, $\sigma(T) = \sigma(G) = -1$. Let $E_g, O_g, s \in \mathbb{R}^D$ be learned vectors, and let record orientation $o \in \{-1, +1\}$ be declared metadata, not a target-derived label. At base i, define

$$F_i(x, o) = \begin{bmatrix} E_{g(x_i)} \\ \sigma(x_i) O_{g(x_i)} + os \end{bmatrix} \in \mathbb{R}^{2\times D}. \tag{24.3}$$

Complement preserves pair class and reverses sign. Define the joint record action $(x, o) \mapsto (R(x), -o)$ and feature action $(JF)_i = (F^{\text{even}}_{L-1-i}, -F^{\text{odd}}_{L-1-i})$. Then the construction satisfies

$$F(R(x), -o) = JF(x, o).$$

This is an exact algebraic equivariance contract. It is not a representation of a physical DNA strand's electronic structure or a newly discovered biological law.

```
    def forward(self, ids, orientation):
        validate_ids(ids)
        if ids.device != self.even.device:
            raise ValueError("IDs and parameters must share device")
        if (
            orientation.shape != (ids.shape[0],)
            or orientation.device != ids.device
            or not bool(((orientation == 1) | (orientation == -1)).all())
        ):
            raise ValueError("orientation is +1/-1 for each record")
        # A/T share an even pair, as do C/G; complement flips odd sign.
        group = torch.tensor([0, 1, 1, 0], device=ids.device)[ids]
        sign = torch.tensor([1, 1, -1, -1], device=ids.device)[ids]
        even = self.even[group]
        odd = self.odd[group] * sign[..., None] + (
            orientation[:, None, None] * self.strand
        )
        return torch.stack([even, odd], dim=2)
```

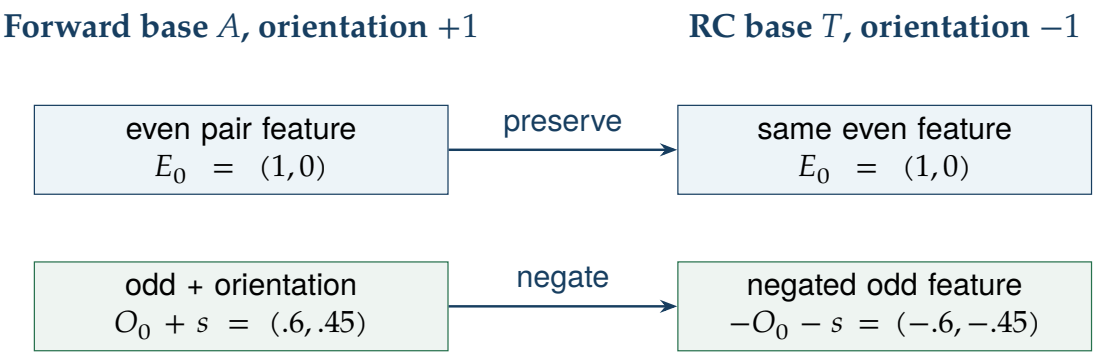


Figure 24.3: Complement-paired bases share even features and reverse odd features. Record reversal permutes positions; orientation metadata changes sign. The symmetry is imposed by parameter tying, not inferred from the colors.

Base	Even 1	Even 2	Odd 1	Odd 2
A	1	0	0.6	0.45
C	0	1	-0.15	0.7
G	0	1	0.35	-0.3
T	1	0	-0.4	-0.05

The generated trace assigns $E_0 = (1, 0)$, $E_1 = (0, 1)$, $O_0 = (.5, .25)$, $O_1 = (-.25, .5)$ and $s = (.1, .2)$ with $o = +1$. For A the odd channel is $(.6, .45)$; for its complemented T in orientation -1 it is $(-.6, -.45)$. Every value is generated from the same reference that tests the joint action.

24.3.1 Equivariant features do not make the whole model equivariant

Flattening the two channels and applying an arbitrary linear map can mix even and odd components in a way that does not commute with J. An unrestricted Transformer can therefore break the property despite a correct embedding. A pointwise linear map that commutes with the sign action must preserve the parity subspaces; nonlinearities and attention require their own action contracts. Ordinary additive positional embeddings can also break reversal symmetry.

Chapter 23's shared two-pass offline readout provides a simple invariant baseline: average the same function on a record and its joint RC transform. By contrast, using different learned strand embeddings in two branches and merely averaging does not guarantee invariance. Swapping the records does not swap identical functions unless the parameter/metadata action has been specified.

Caduceus establishes RC-equivariant, bidirectional sequence modeling as prior art [48]. Our tied feature reference is a small teaching construction, not a reproduction of its architecture or evidence of improved genomic accuracy. For orientation-sensitive labels, invariant pooling can discard the signal the task requires. Use equivariant outputs or an explicit label action instead.

24.4 Position should measure the distance the claim uses

Chapter 23 used ordinal positions of valid bases to make inserted padding inert. For variable-span tokens, token ordinal is not base distance. An optional representative coordinate is a span's base-center midpoint,

$$p_i = (a_i + b_i - 1)/2.$$

Even-width spans can have half-integer centers. A distance-sensitive feature should state whether its positions are base coordinates, normalized window coordinates, token ordinals or a multi-scale combination.

For example, tokens spanning $[0, 2)$ and $[2, 10)$ have centers .5 and 5.5: their base distance is five, not the ordinal distance one. Compressing unknown bases or omitted regions changes those distances unless the original coordinate mapping is retained. Padding is not a biological deletion.

24.5 Derive rotary positions as pair rotations

Rotary position embedding is an established Transformer mechanism [55]. We derive its small reference to make the unit contract clear. For a pair of coordinates and angular frequency ω in

radians per base, let

$$\mathsf{R}(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}, \quad \tilde{q}_i = \mathsf{R}(\omega p_i) q_i, \quad \tilde{k}_j = \mathsf{R}(\omega p_j) k_j.$$

Orthogonality and the angle-addition law give

$$\tilde{q}_i^\top \tilde{k}_j = q_i^\top \mathsf{R}\big(\omega(p_j - p_i)\big)\, k_j. \tag{24.4}$$

Thus simultaneous translation $p_i \mapsto p_i + a$ cancels in pair scores. The norm of each vector is unchanged. These claims hold pair by pair in an even head dimension D_h, using $D_h/2$ frequencies.

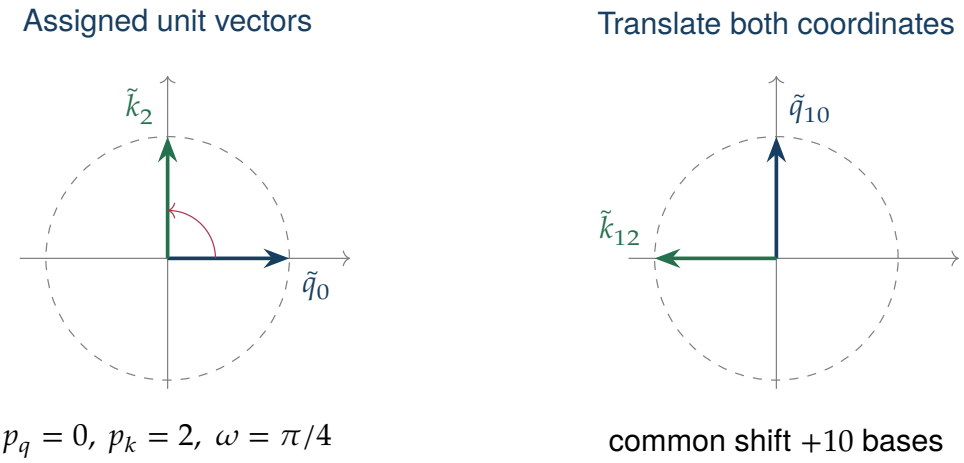


Figure 24.4: Assigned two-dimensional query/key vectors are rotated by their declared base positions. The dot product depends on relative angle; a common coordinate translation cancels. This geometric identity is not a guarantee of long-context generalization.

```
def rotary(x, positions, frequencies):
    """Pair rotation for [B,H,T,Dh]; positions are declared base coordinates."""
    if (
        x.ndim != 4
        or any(size == 0 for size in x.shape)
        or x.shape[-1] % 2
        or not x.is_floating_point()
        or not bool(torch.isfinite(x).all())
    ):
        raise ValueError("finite floating [B,H,T,even Dh] required")
    if (
        positions.shape != (x.shape[2],)
        or frequencies.shape != (x.shape[3] // 2,)
        or positions.device != x.device
        or frequencies.device != x.device
        or positions.dtype != x.dtype
        or frequencies.dtype != x.dtype
        or not bool(torch.isfinite(positions).all())
        or not bool(torch.isfinite(frequencies).all())
    ):
        raise ValueError(
            "finite coordinate/frequency vectors matching dtype/device required"
```

```
        )
    angle = positions[:, None] * frequencies[None, :]
    a, b = x[..., 0::2], x[..., 1::2]
    rotated = torch.stack(
        [a * angle.cos() - b * angle.sin(), a * angle.sin() + b * angle.cos()],
         ↪ dim=-1
    )
    return rotated.flatten(-2)
```

For $q = k = (1, 0)$, positions zero and two, and $\omega = \pi/4$ rad/base, the rotated vectors are $(1, 0)$ and $(0, 1)$ up to numerical error. Their score is zero. Translating both positions by ten changes the individual directions but not their dot product. The tests check this assigned case, norm preservation, explicit block-diagonal matrix parity and input gradients in float64.

Relative-position algebra does not establish RC equivariance. Reversal sends $p_j - p_i$ to its negative; content features also transform. Without a compatible feature/score action, Equation 24.4 can change. Similarly, a rotation formula accepting large coordinates does not prove that a trained model extrapolates to those coordinates. High-frequency phase aliasing and unseen distance distributions require evaluation.

A multi-scale variant could place one rotary block on base distance and another on larger genomic bins. Bin size, origin and boundary behavior are then model assumptions. Such a variant is a research extension here, not an implemented accuracy gain. An ablation must compare it against plain base positions under matched records, objective and tuning budgets.

24.6 Long context is an attention graph before it is a speed claim

Let $\mathcal{N}(i)$ be the legal key set for query i. For local radius w, a declared static global-position set G and valid-position indicators v_i, an offline edge exists when

$$v_i v_j = 1 \quad \text{and} \quad (|i - j| \le w \ \vee \ i \in G \ \vee \ j \in G).$$

The causal case additionally requires $j \le i$. Global edges do not override causality. Padded query rows and padded keys remain blocked as in Chapter 23.

```
def edge_lists(length, radius, causal=False, globals_=(), valid=None):
    """Local + static global edges; no suffix-dependent selection. Query->key."""
    integer(length, 1)
    integer(radius)
    if not isinstance(causal, bool):
        raise ValueError("causal must be boolean")
    for index in globals_:
        integer(index)
        if index >= length:
            raise ValueError("global index outside record")
    globals_ = set(globals_)
    if valid is None:
        valid = [True] * length
    if len(valid) != length or any(type(v) is not bool for v in valid):
        raise ValueError("one boolean validity per position required")
    rows = []
    for i in range(length):
        if not valid[i]:
            rows.append([])
            continue
```

```
        start, stop = max(0, i - radius), min(length, i + radius + 1)
        local = set(range(start, stop))
        if i in globals_:
            local.update(range(length))
        local.update(globals_)
        rows.append(sorted(j for j in local if valid[j] and (not causal or j <=
         ↪ i)))
    return rows
```

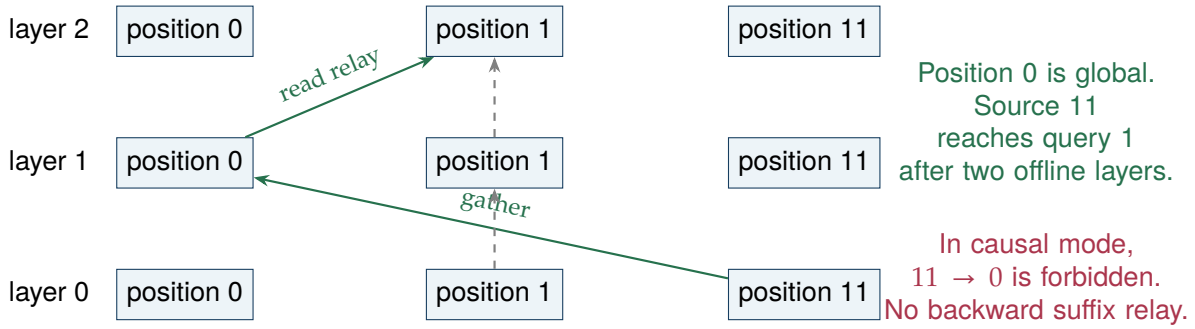
Local-plus-global attention is established prior art, including Longformer [56]. Our graph is a transparent reference, not a novel mechanism merely because its input is DNA. At fixed w and $g = |G|$, an edge-count upper bound is

$$|\mathcal{E}| \le T(2w + 1 + 2g).$$

One Tg term accounts for global keys available to local queries; the other accounts for global queries attending broadly. Intersections and boundaries reduce the actual count.

Length	Dense edges	Offline edges	Causal edges
32	1024	374	203
128	16384	1622	875
512	262144	6614	3563
1024	1048576	13270	7147

The table counts actual edges at radius four with the first and last positions global. It does not report measured latency, FLOPs or peak memory.



A first-layer query cannot read the global node's concurrently computed first-layer update.

Figure 24.5: A distant source reaches a local query through a global relay in two layers, not one. At layer one the global query gathers; at layer two another query reads the updated global representation. Causal graphs cannot relay future information backward.

24.6.1 Dependency paths have a layer count

Let $\mathcal{D}_0(i) = \{i\}$ because a residual path preserves self-information. With the same graph in each layer,

$$\mathcal{D}_{\ell+1}(i) = \mathcal{D}_\ell(i) \cup \bigcup_{j \in \mathcal{D}_\ell(i)} \mathcal{N}(j).$$

In a length-twelve offline record with radius one, query one reaches positions zero through three after two local layers. With position zero global, query one still cannot access source

eleven through that global node in the first layer: the global node's update is being computed concurrently. It can access source eleven after the second layer. The reference expands the dependency sets exactly.

```
def reachable_sources(edges, query, layers):
    """Dependency expansion includes residual self-access at each layer."""
    integer(query)
    integer(layers)
    if query >= len(edges):
        raise ValueError("query outside graph")
    sources = {query}
    for _ in range(layers):
        sources |= {j for i in sources for j in edges[i]}
    return sorted(sources)
```

At causal query one, the same global policy cannot expose any source above one, regardless of layer count. This is a pathwise proof that all edges respecting causality preserve causality; the finite suffix-intervention test checks the actual numerical implementation as well.

Global positions selected by inspecting the full sequence can create a different leak even if edges look triangular. A future motif could determine which prefix features are designated global. The causal policy must depend only on legal metadata or available prefix information. Known full-record length and a streaming endpoint are also different interface assumptions, addressed in Chapter 26.

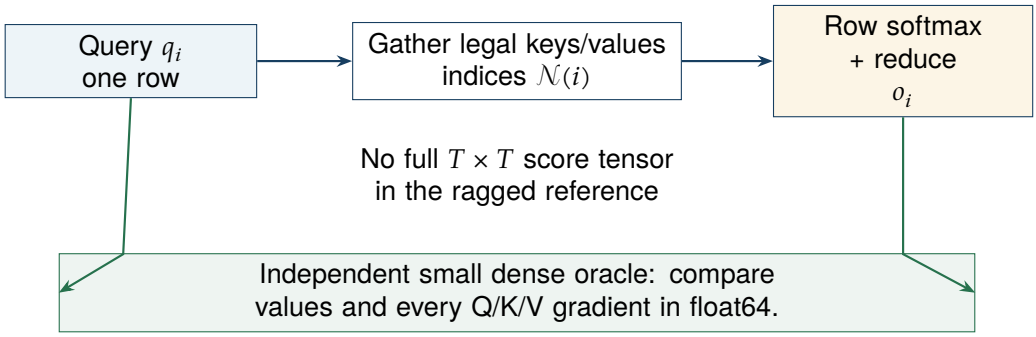
24.7 Implement sparse semantics without allocating dense scores

A dense $T \times T$ score matrix filled with masked values is semantically sparse but still consumes dense score storage. The reference instead stores ragged legal key lists. For each query, it gathers only allowed keys and values, computes the row scores, normalizes over those keys and reduces their values:

$$a_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{D_h})}{\sum_{j' \in \mathcal{N}(i)} \exp(q_i^\top k_{j'} / \sqrt{D_h})}, \qquad o_i = \sum_{j \in \mathcal{N}(i)} a_{ij} v_j.$$

An empty row returns zero under the declared padding convention.

```
def sparse_attention(q, k, v, edges):
    """Single-head ragged-row gather reference; no dense score matrix."""
    validate_qkv(q, k, v, edges)
    output = []
    for i, allowed in enumerate(edges):
        if not allowed:
            output.append(v.new_zeros(v.shape[1]) + q[i].sum() * 0)
            continue
        logits = (k[allowed] @ q[i]) / math.sqrt(q.shape[1])
        output.append(logits.softmax(0) @ v[allowed])
    return torch.stack(output)
```



Parity is implemented. Optimized-kernel speed, full-model training and genomic usefulness are not.

Figure 24.6: Ragged gather processes only declared edges; the independent dense oracle materializes a small full score matrix for value and gradient checks. Semantic parity does not imply that a Python loop is a fast production kernel.

For fixed D_h, score and weighted-value work scales with the number of edges. Edge construction, sorting, projection work, autograd buffers and output storage remain additional costs. The Python implementation is a pedagogical single-head reference: launches and irregular gathers can make it slower than an optimized dense kernel at modest lengths. We report no speedup.

A dense vectorized oracle independently reconstructs legal rows on small tensors. Tests compare output values and all Q/K/V gradients in float64, including a padding row. Finite differences check sparse derivatives. A future suffix intervention changes offline prefix outputs but leaves causal prefix outputs exactly unchanged under the tested graph. These are software contracts, not evidence that the model learned regulatory grammar.

24.8 Retrieval adds information, not free context

A retrieval mechanism can supplement local context with selected external sequences or annotations. It must declare the corpus version, coordinate mapping, selection rule and permitted information. Retrieving a test label, a homologous training duplicate or a suffix-derived feature changes the evaluation contract. Static external knowledge and dynamic within-record access are not the same.

For causal prediction, retrieval queries and retrieved content must respect the available-prefix and knowledge-cutoff assumptions. For offline variant scoring, paired reference/alternate retrieval must not receive different target-derived evidence. Retrieval is not implemented by the local-global graph above: a global token summarizes the current record, not an external database. We keep retrieval as a clearly labeled extension rather than calling ordinary attention external memory.

24.9 Design the experiment that could reject a mechanism

The executable chapter establishes span, strand, rotary and sparse-graph correctness. It does not train a new full genomic model. A serious evaluation now asks separate questions:

1. Does a span-aware tokenizer improve variant-local attribution relative to the same model with ordinal-only positions?

2. Does the declared RC action help invariant labels without damaging orientation-sensitive tasks?

3. Does base-coordinate rotary position encoding improve the chosen context distribution relative to Chapter 23's position baseline?

4. Does sparse access preserve task accuracy while meeting a declared memory/latency budget?

Each comparison freezes dataset, split groups, objective, width/depth budget, training tokens, tuning effort and evaluation examples. Compare conventional Transformer, local CNN and an appropriate non-attention long-context baseline, not only DOGMA versus Hermon.

A broken mutation footprint falsifies coordinate correctness. A nonzero joint-RC equivariance error falsifies the tied-feature construction. Gradient disagreement falsifies implementation parity. Matched-budget failure to improve the task weakens a usefulness claim even when every algebraic test passes. Chapter 23's pilot is the cautionary example: every tensor contract there passes, yet the model did not train under its budget.

Key ideas

- A triangular token mask cannot remove information already inside a token: ACG over $[0,3)$ contains the base it would predict.
- Equivariant input features do not make a model equivariant. Every later layer must commute with the same action.
- Rotary scores depend only on relative position, but that algebra is not evidence of length extrapolation.
- A global relay needs two layers. Sparse semantics implemented with dense masked scores still uses dense memory.

24.10 Exercises and worked solutions

1. List width-three spans of ACGTAC. **Answer:** ACG $[0,3)$, CGT $[1,4)$, GTA $[2,5)$ and TAC $[3,6)$.

2. Which tokens change if base two changes? **Answer:** zero, one and two, because exactly those spans contain two.

3. Derive Equation 24.1. **Answer:** reversal sends base p to $L-1-p$, mapping $a,\dots,b-1$ to $L-b,\dots,L-a-1$; complement transforms the substring.

4. Why can a strided grid break RC commutation? **Answer:** uncovered terminal bases move to the other end, changing the grid origin; the ACGTA width-two stride-two example gives different spans.

5. Why can causal attention on ACG leak target G? **Answer:** target base two is already inside span $[0,3)$.

6. To obscure base two in the width-three example, which tokens must be hidden? **Answer:** all three overlapping spans zero, one and two, including cached features derived from them.

7. Prove the tied-feature RC law. **Answer:** complement preserves g and negates σ; orientation negates; reversal permutes positions, leaving even and negating odd channels.

8. Must orientation metadata also transform? **Answer:** yes for the declared joint action. Keeping o fixed leaves the learned os contribution with the wrong sign.

9. Does an arbitrary downstream linear map preserve parity? **Answer:** not generally; it must commute with the declared sign action, which forbids unrestricted even/odd mixing.

10. When can invariant pooling be scientifically wrong? **Answer:** when the target changes under RC, such as an orientation-specific label; invariance can erase the required distinction.

11. Compute base centers of $[0, 2)$ and $[2, 10)$. **Answer:** .5 and 5.5, distance five bases.

12. Prove Equation 24.4. **Answer:** $\mathsf{R}(a)^\top = \mathsf{R}(-a)$ and $\mathsf{R}(a)\mathsf{R}(b) = \mathsf{R}(a + b)$.

13. Is rotary translation invariance an extrapolation benchmark? **Answer:** no; it is an algebraic score property, not learned capability at unseen distance or sequence distributions.

14. Does rotary positioning automatically impose RC equivariance? **Answer:** no; reflection negates relative angles and content features also transform. Their compatible action must be specified.

15. Derive the local-global edge bound. **Answer:** local rows contribute at most $T(2w + 1)$, global keys at most Tg, and global queries at most gT, before overlap removal.

16. Why does a global relay require two layers? **Answer:** the first layer computes the global aggregation; only a later layer can read that new representation.

17. Can causal global edges bypass the triangle? **Answer:** no; all edges retain $j \le i$, and any dependency path remains nonincreasing in base/token time under the legal representation contract.

18. Is a dense masked implementation memory-sparse? **Answer:** no; allocating all T^2 scores retains dense score storage.

19. What does value/gradient parity establish? **Answer:** the sparse and dense references implement the same tested function and derivatives, not superior speed or genomic prediction.

20. Plan a genomic mechanism ablation. **Rubric:** immutable reference/alternate pairs and split groups; explicit target and strand action; position units; legal information access; conventional baselines; matched budget and tuning; repeated seeds; attribution/accuracy metrics; OOD context tests; real memory/latency measurements and retained negative results.

24.11 Bridge

The mechanism contracts now have something concrete to expose: token spans, orientation action, positions and access edges. Chapter 25 puts those contracts into a shared experiment envelope while preserving DOGMA's native carry and Hermon's activation/mask semantics. Fair comparison means the same scientific question, not erasing each architecture's defining mechanism.

Sources and evidence

Primary access checks distinguish established mechanisms from local software evidence. DNABERT's author implementation README and DNABERT-2's versioned abstract provide tokenization context. Caduceus HTML Sections 3.2, 4.1 and Appendix B provide RC architecture context. RoFormer HTML Section 3.2 and its pair-rotation equations support the relative-position derivation. Longformer's versioned abstract establishes local/global attention as prior art. No cited training experiment was reproduced. The small strand and ragged-attention references are original teaching constructions, not reproductions of those models. Independent scientific and reader review remain open.

PART VI

Training systems and fair experiments

How do we compare unlike architectures fairly, train them reproducibly and know what a benchmark result supports?

Before either model family is scaled or served, Part VI fixes the experimental contract: estimands, transitive leakage closure, paired and cluster uncertainty, training/inference parity, memory-efficient optimization that preserves the objective, distributed reductions, reproducible artifacts and benchmark design with matched opportunity. The DOGMA and Hermon DNA references are compared without forcing their internal states into a false common form.

CHAPTERS IN THIS PART

25 **Shared experiments without false equivalence** Share experiments without forcing unlike states into false equivalence.

26 **Training/inference parity and parallel recurrence** Check sequential, chunked and parallel full-model execution.

27 **Optimization and memory-efficient training** Preserve the objective under accumulation and memory-saving training.

28 **Distributed training and reproducible model artifacts** Derive distributed reduction and reproducible artifact boundaries.

29 **Benchmark design and comparative evidence** Design paired, budgeted comparisons and quantify uncertainty.

CHAPTER 25

Shared experiments without false equivalence

In this chapter you will

- state an estimand before comparing DOGMA, Hermon DNA or baselines
- close leakage relations transitively with union–find and audit the split
- select preprocessing and thresholds from training data only
- quantify paired and cluster-level uncertainty and keep seed variance separate

DOGMA and Hermon DNA should be compared without making them identical. A common question, dataset and evidence standard can coexist with different state types, legal execution schedules and resource costs. The experiment must preserve the mechanism whose value is under test.

This chapter builds that contract rather than announcing an architecture winner. We implement transitive split closure, train-only baseline selection, paired metrics, cluster resampling and native-model execution probes. The numerical classification example compares a GC threshold with a known motif oracle. The DOGMA and Hermon probes check semantic execution, not learning performance.

25.1 Define the estimand before building the leaderboard

An estimand is the quantity the experiment intends to estimate. For two fixed predictors f_A, f_B and an evaluation distribution $\mathcal{D}$, one possible estimand is the accuracy difference

$$\Delta(f_A, f_B; \mathcal{D}) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\mathbf{1}\{f_A(x) = y\} - \mathbf{1}\{f_B(x) = y\}]. \tag{25.1}$$

This differs from the expected difference between two *training procedures*, where initialization, sampled training records, data order and tuning are random. It also differs from deployment utility under a fixed latency or memory budget.

Write the target before inspecting results. Does the claim concern an offline record classifier, a causal next-token model, a variant effect predictor or a serving system? Which species, chromosomes, contexts and shifts should generalization cover? A metric without its task and population is an incomplete scientific statement. Benchmark selection can change apparent algorithmic rankings; it is not a neutral wrapper [59].

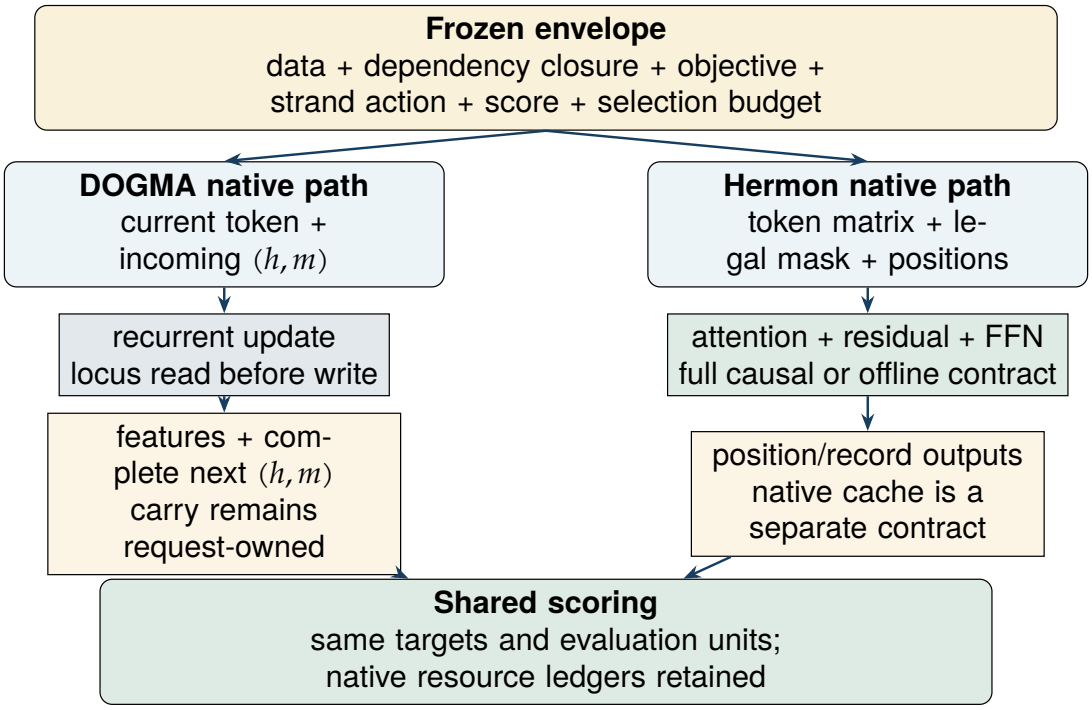


Figure 25.1: The shared experiment envelope freezes the scientific question and legal evidence. Separate adapters retain DOGMA's complete recurrent/locus state and Hermon's token/mask representation. Shared outputs do not require shared internal state.

25.2 Freeze a common envelope, not a common machine

The common envelope records dataset version and provenance, split/group artifacts, target construction, legal information access, strand-label action, scored positions, primary metric, uncertainty method, tuning budget and stopping rule. Its digest binds exact content, not the scientific adequacy of that content.

Chapter 5 supplies basic record/token contracts. Chapter 24 supplies physical base spans, reverse-complement span actions and attention legality. We now add the comparison-level split and evaluation contract. These are complementary layers, not a reason to rename Chapter 24 as a generic data chapter.

Native execution remains different. DOGMA owns fast state h and locus memory m, read-before-write semantics and reset/carry boundaries. Hermon owns token activations, positional representation and attention masks; a production serving engine may additionally own a KV cache. Evolutor organizes these models and experiments. It does not turn every mechanism into one architecture.

For causal next-token prediction both models receive the same legal prefix and target shift, but DOGMA advances state while Hermon uses a triangular mask. Neither can import Chapter 21's full-record reverse-complement features. For offline classification both may inspect the full record under a declared strand policy. Their feature construction may still differ; that is an architectural variable to measure, not erase.

25.3 Leakage prevention is a transitive graph problem

A genomic collection can contain exact duplicates, reverse complements, overlapping windows, homologous sequences and samples sharing an individual or experimental source.

Preventing exact duplication alone is insufficient. Represent records by vertices and prohibited-cross-split relationships by edges. A connected component must lie wholly in one partition.

This includes transitivity. If A is the reverse complement of B, B shares a declared homology group with C, and C has a curated overlap link to D, all four belong to one split component. Checking each relationship type in isolation and performing successive splits can separate the chain.

The reference joins exact/RC equivalents, globally namespaced declared group tags and explicit record-ID relations with union–find. A component's representative is a deterministic minimal ID; the final component order is sorted before seeded shuffling. Reordering source rows therefore does not silently alter the partition.

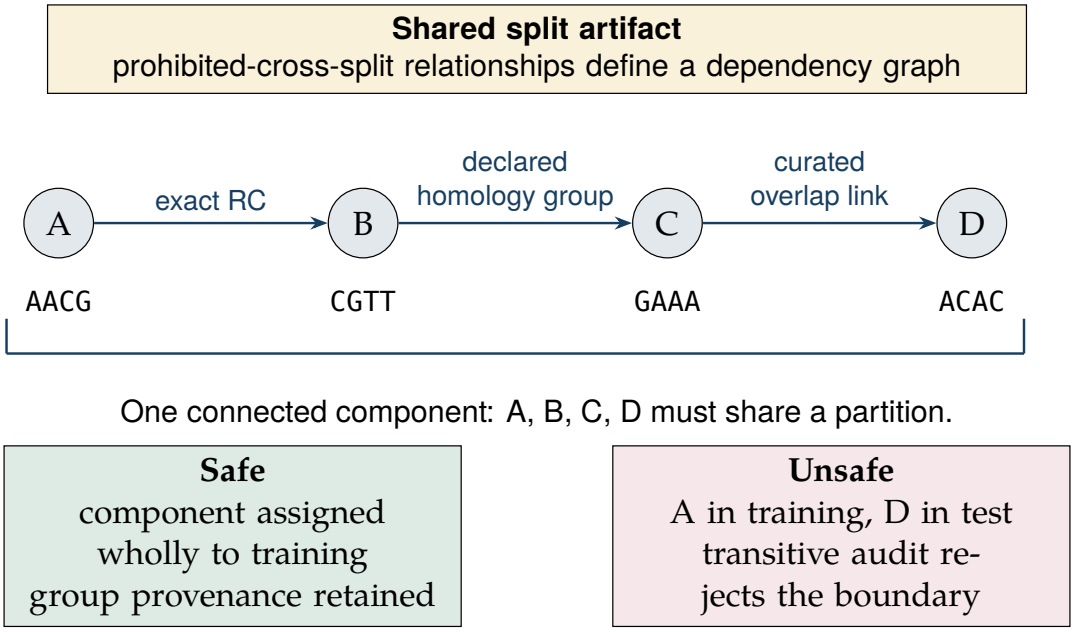


Figure 25.2: A shared split artifact binds transitive dependency closure. Exact reverse complements, declared groups and curated links can jointly connect records. The entire component must share one partition; native model semantics remain separately specified.

```
def closure(records, relations=()):
    """Transitive exact/RC, declared-group and explicit-relation components.

    Relations/group tags are supplied biological curation, not inferred homology.
    Each tuple in relations joins record IDs. A group tag is globally namespaced.
    """
    ids = [record["id"] for record in records]
    if len(ids) != len(set(ids)) or not all(isinstance(i, str) and i for i in
     ↪  ids):
        raise ValueError("Unique nonempty string record IDs required")
    parent = {i: i for i in ids}

    def find(i):
        while parent[i] != i:
            parent[i] = parent[parent[i]]
```

```
            i = parent[i]
        return i

    def union(a, b):
        if a not in parent or b not in parent:
            raise ValueError("Relation mentions an unknown record")
        a, b = find(a), find(b)
        if a != b:
            parent[max(a, b)] = min(a, b)

    seen = {}
    for record in records:
        sequence = canonical(record["sequence"])
        if type(record["label"]) is not int or record["label"] not in (0, 1):
            raise ValueError("Binary integer labels required")
        groups = record["groups"]
        if not isinstance(groups, list) or not all(
            isinstance(g, str) and g for g in groups
        ):
            raise ValueError("Group tags must be nonempty strings")
        keys = [("strand", min(sequence, rc(sequence)))]
        keys += [("group", group) for group in groups]
        for key in keys:
            if key in seen:
                union(record["id"], seen[key])
            else:
                seen[key] = record["id"]
    for a, b in relations:
        union(a, b)
    components = defaultdict(list)
    for i in ids:
        components[find(i)].append(i)
    return tuple(sorted(tuple(sorted(v)) for v in components.values()))
```

We do not infer homology from strings here. The biological curation artifact must supply those edges or tags, including the similarity criterion, overlap policy and version. Over-grouping can merge a whole chromosome or collection; under-grouping can permit near-duplicate leakage. Both consequences belong in the experiment report.

Our finite splitter assigns 60/20/20 percent of *components*, rounded to integer boundaries, not guaranteed percentages of records. Large components may yield uneven sample counts or class balance. It rejects empty partitions, and a separate audit checks every record and component.

```
def audit_splits(records, assignment, relations=()):
    ids = {record["id"] for record in records}
    if set(assignment) != ids or any(
        v not in ("train", "validation", "test") for v in assignment.values()
    ):
        raise ValueError("Every record needs exactly one declared partition")
    for component in closure(records, relations):
        if len({assignment[i] for i in component}) != 1:
            raise ValueError("Dependency component crosses a split boundary")
    return True
```

The test oracle independently constructs an adjacency graph and finds its components by breadth-first traversal. A union–find roundtrip alone would not expose a shared linkage

defect. We deliberately test a cross-partition reverse complement and a transitive curated relation; both must fail.

Exact RC grouping prevents cross-split access even when strand-specific labels are legitimate. Label equality under RC is a separate task property: an invariant classification task should check it; an orientation-sensitive target should specify its transformation. Grouping and label action must not be conflated.

25.4 Preprocessing and selection belong to the training side

A transform may be mathematically harmless while leaking information through how its parameters were chosen. Estimate vocabulary, scaling statistics, feature selection, thresholds and imputation from permitted training data. Choose hyperparameters by the declared validation rule; freeze the selected pipeline before the held-out test.

The executable baseline scores GC fraction and selects one threshold using only training labels. Candidate thresholds include midpoints between distinct observed scores and rules that predict every record positive or negative. The all-negative rule must exceed one: threshold one would still predict a pure-GC sequence positive. This boundary case appears in the tests.

```
def fit_threshold(train):
    if not train:
        raise ValueError("Training records required")
    scores = sorted({composition(record["sequence"]) for record in train})
    # Include all-positive and all-negative rules, including score exactly one.
    candidates = [-1.0, *((a + b) / 2 for a, b in zip(scores, scores[1:])), 2.0]
    return max(
        candidates,
        key=lambda t: sum(
            int(composition(r["sequence"]) >= t) == r["label"] for r in train
        ),
    )  # first candidate resolves ties deterministically
```

For the fixed synthetic example, 400 length-32 sequences have label one exactly when `ACGT` occurs. This motif is its own reverse complement. The source alternates intended classes, implants positive motifs and rejects negative candidates that contain the motif. Split assignment, not final test labels, determines the partitions. The resulting counts are 240 training, 80 validation and 80 test records.

The selected threshold is 0.390625. On the frozen test set, the string oracle gets all 80 labels; the GC baseline gets 39, giving accuracy 0.4875. This is a diagnostic for this deliberately weak statistic. It is not proof that neural models learn the task, nor that genomic classification is easy. The baseline need not consume its validation budget when no additional tuning is performed.

25.5 Accuracy, class balance and missing denominators

For a binary task, count true positives, true negatives, false positives and false negatives before reducing them to a scalar:

$$\text{accuracy} = \frac{TP + TN}{N}, \qquad \text{balanced accuracy} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right).$$

Balanced accuracy requires both classes to be represented. The implementation returns an explicit unavailable value if one class is absent, rather than manufacturing a denominator or silently assuming sensitivity one.

For the motif test the GC baseline has $TP = 32, TN = 7, FP = 34, FN = 7$. Thus sensitivity is $32/39$, specificity $7/41$, and balanced accuracy is about 0.495622. Always include sample counts and class composition: a majority-class predictor can have impressive accuracy while missing every positive record. Other tasks need other metrics and their own target definitions; this binary contract is not a universal genome benchmark.

25.6 Derive paired uncertainty from correctness differences

Two predictors evaluated on the same examples produce paired evidence. Define

$$d_i = \mathbf{1}\{f_A(x_i) = y_i\} - \mathbf{1}\{f_B(x_i) = y_i\}, \qquad \widehat{\Delta} = \frac{1}{N}\sum_{i=1}^{N} d_i. \tag{25.2}$$

Here d_i is $-1, 0$, or 1. Under iid evaluation records and fixed predictors,

$$\widehat{\mathrm{Var}}(\widehat{\Delta}) = \frac{\sum_i (d_i - \overline{d})^2}{N(N-1)}.$$

The covariance between model outcomes is already captured by their paired difference. Resampling model A and B separately destroys that pairing and generally changes the uncertainty calculation.

For two identical prediction vectors every $d_i = 0$, so the paired difference and bootstrap interval are exactly zero. This is an important implementation invariant, not evidence of perfect predictions: both models could make the same errors.

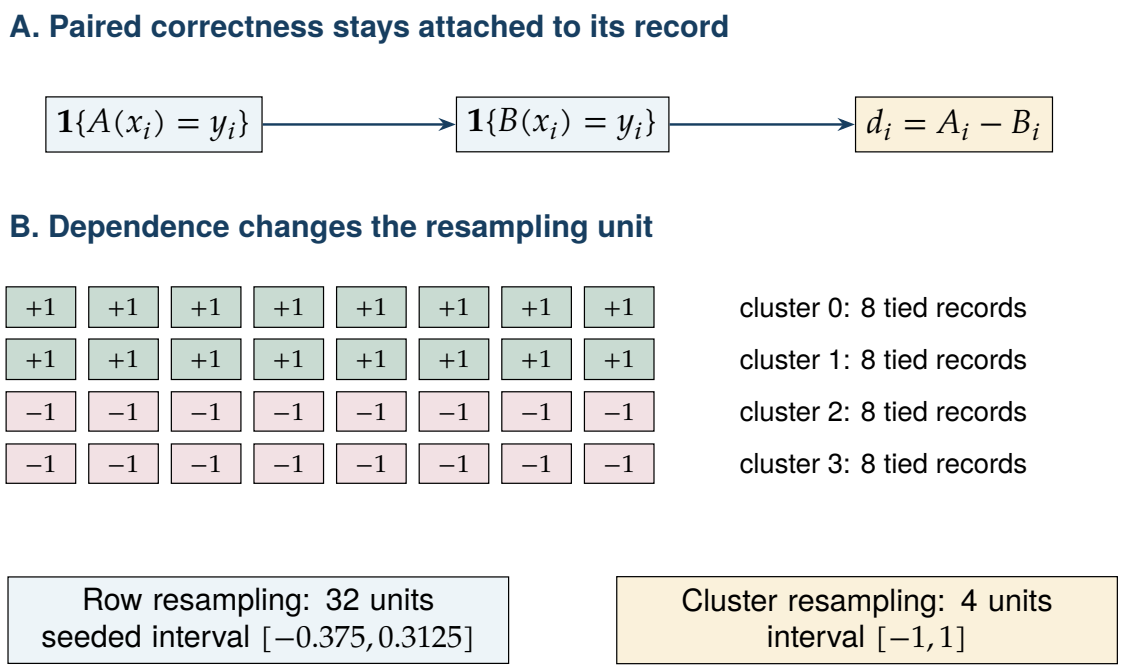


Figure 25.3: Keep the two predictions attached to one evaluation unit. Resample independent units at the record or cluster level. Repeat training seeds separately; conditional test uncertainty is not training-procedure uncertainty.

25.7 Cluster resampling and the cost of a false sample size

Suppose records within a family, individual or source group remain dependent after the split. Resample independent clusters, retaining every within-cluster record and both model outcomes. Let cluster g have difference sum D_g and size n_g. For bootstrap cluster draws $G_1^*, \ldots, G_K^*$,

$$\widehat{\Delta}^* = \frac{\sum_{k=1}^{K} D_{G_k^*}}{\sum_{k=1}^{K} n_{G_k^*}}. \tag{25.3}$$

This targets a record-weighted difference via a ratio of totals. It is not the equal-weight mean of cluster means, which is a different estimand when cluster sizes vary.

```
def paired_bootstrap(labels, a, b, clusters=None, repetitions=2000, seed=25):
    """Uniform cluster resampling; keep A/B pairing and all rows within a draw.

    Target is record-weighted accuracy difference. Unequal cluster sizes produce
    a ratio of resampled totals, not an equal-weight mean of cluster means.
    The percentile interval is descriptive/approximate, not guaranteed coverage.
    """
    positive(repetitions, "bootstrap repetitions")
    difference = deltas(labels, a, b)
    clusters = list(range(len(labels))) if clusters is None else list(clusters)
    if len(clusters) != len(labels):
        raise ValueError("One cluster ID per evaluation record required")
    groups = defaultdict(list)
```

```
    for d, group in zip(difference, clusters):
        groups[group].append(d)
    groups = list(groups.values())
    if len(groups) < 2:
        raise ValueError("At least two independent resampling units required")
    rng = random.Random(seed)
    draws = []
    for _ in range(repetitions):
        sample = [groups[rng.randrange(len(groups))] for _ in groups]
        draws.append(sum(map(sum, sample)) / sum(map(len, sample)))
    draws.sort()
    return {
        "observed_delta": statistics.mean(difference),
        "bootstrap_mean": statistics.mean(draws),
        "lo95": quantile(draws, 0.025),
        "hi95": quantile(draws, 0.975),
        "units": len(groups),
        "records": len(labels),
        "repetitions": repetitions,
    }
```

The reference draws 2,000 replicates with seed 25 and reports empirical 2.5/97.5 percentiles using linear interpolation. These are approximate, conditional percentile intervals, not a universal coverage theorem. At least two independent resampling units are required; four clusters still provide fragile evidence.

A counterexample fixes 32 records in four equal clusters. Differences are $+1$ throughout two clusters and -1 throughout two others. Both analyses observe $\widehat{\Delta} = 0$. Resampling individual rows yields the displayed narrow interval; resampling four clusters yields $[-1, 1]$. The latter wide interval is appropriate to the assigned dependence model.

Resampling	Delta	2.5 percentile	97.5 percentile
32 iid rows	0	-0.375	0.3125
4 clusters	0	-1	1

The independent oracle enumerates all $4^4 = 256$ cluster-draw sequences. It confirms the exact bootstrap support and extreme percentile endpoints. No bootstrap can undo train/test leakage; closure precedes resampling.

In the motif example each generated test record is treated as an independent synthetic sampling unit. The oracle-minus-GC observed difference is 0.5125 and the seeded percentile interval is $[0.4, 0.625]$. This is a conditional comparison of two fixed rules, not a learned DOGMA/Hermon performance interval.

25.8 Seeds, tuning and variance are separate experimental axes

Let S denote training randomness and D an evaluation sample. The law of total variance illustrates the separation:

$$\mathrm{Var}_{S,D}(\widehat{\Delta}) = \mathbb{E}_S[\mathrm{Var}_D(\widehat{\Delta} \mid S)] + \mathrm{Var}_S(\mathbb{E}_D[\widehat{\Delta} \mid S]).$$

A test-record bootstrap conditional on one fitted pair addresses the first term under its sampling assumptions. It does not estimate the second. Repeat prespecified seeds, retain each run and avoid reporting only the best one. Reporting and variance studies motivate exposing these budgets and sources of randomness [57, 58].

Using the same seed label for two architectures does not guarantee shared random draws: they may initialize different tensor shapes and consume random numbers differently. Pairing should follow shared data and justified randomization blocks, not a cosmetic matching integer. A seed summary requires its own inferential design when training sets or test cohorts are also varied.

A tuning trial is a resource. Record its selected metric, validation access, stopping rule and failed runs. The model receiving many unreported manual iterations has a hidden advantage even if the final parameter counts match.

25.9 A resource budget is a vector with provenance

Use a budget vector rather than a single size:

$$\mathcal{B} = (P, N_{\text{base}}, N_{\text{token}}, F, M_{\text{peak}}, M_{\text{persistent}}, T_{\text{train}}, L_{\text{infer}}, E).$$

Count base positions separately from model tokens: overlapping k-mers can repeatedly expose the same base. Distinguish static parameter bytes, live activations, optimizer state and request-owned carry/cache. An attention edge count is not measured FLOPs or latency; an absent KV cache in a stateless reference is not proof that production inference uses no state.

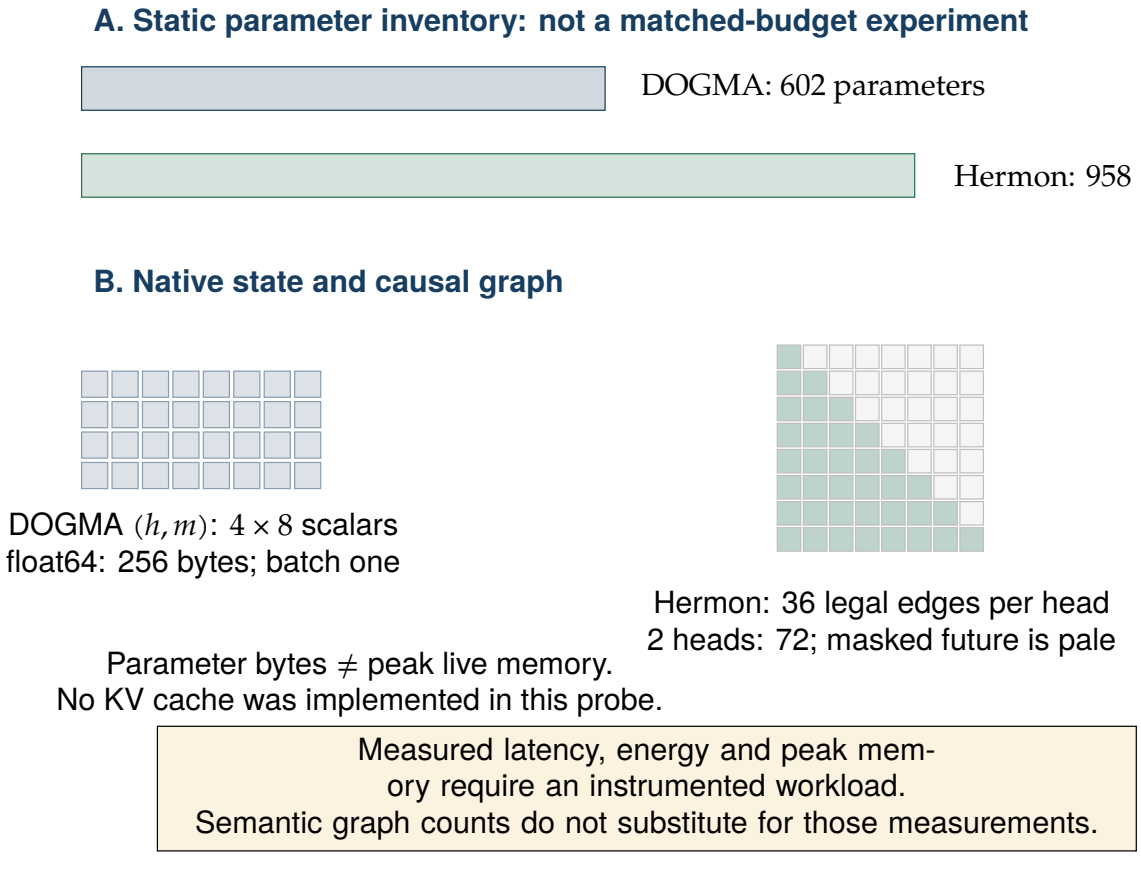


Figure 25.4: Different budget components answer different questions. Static inventories and semantic graph counts are computed here. Peak memory, latency, energy and accelerator throughput need a separately specified instrumented workload.

Reference	Parameters	Param. bytes	State bytes	Causal edges
DOGMA reference	602	4816	256	0
Hermon reference	958	7664	0	72

The native probe uses one eight-base record and float64 CPU models. DOGMA has 602 parameters; its fast/locus state contains $1 \cdot 8(1 + 3) = 32$ scalars or 256 bytes. Hermon has 958 parameters including its unused classification head; two attention heads give $2(1 + \cdots + 8) = 72$ legal causal edges per layer. These models are not parameter-matched. The table is an auditable inventory, not evidence of a fair performance comparison.

25.10 Native adapters must pass execution parity first

For DOGMA, run all eight tokens and compare with a three-token chunk followed by five tokens carrying both h and m. Features and terminal state match exactly in the assigned probe. Carrying only fast state would discard memory and define another model.

For Hermon, compare full causal logits with the last-position logits from each repeated prefix. Maximum absolute difference is about 2.22×10^{-16} in float64. The repeated-prefix path is deliberately slow and stateless: it is an oracle for information legality, not a KV-cache implementation or a serving benchmark.

```
def native_probe():
    """No training or accuracy comparison: exercise actual native model contracts."""
    import torch

    torch.set_num_threads(1)
    dogma_module, hermon_module = load_prior(22), load_prior(23)
    torch.manual_seed(25)
    dogma = dogma_module.DOGMA(d=8, k=3, classes=4).double().eval()
    hermon = hermon_module.HermonDNA(d=8, h=2, layers=1, max_len=32).double().eval()
    x = torch.tensor([[0, 1, 2, 3, 0, 2, 1, 3]], dtype=torch.long)
    with torch.no_grad():
        full, state, _ = dogma.run(x)
        first, carry, _ = dogma.run(x[:, :3])
        second, end, _ = dogma.run(x[:, 3:], carry)
        dogma_error = float((full - torch.cat([first, second], 1)).abs().max())
        carry_error = max(float((a - b).abs().max()) for a, b in zip(state, end))
        causal = hermon.causal_logits(x)
        # Repeated-prefix oracle, deliberately NOT a KV-cache implementation.
        prefix = torch.stack(
            [hermon.causal_logits(x[:, : i + 1])[:, -1] for i in range(x.shape[1])], 1
        )
        hermon_error = float((causal - prefix).abs().max())
    return {
        "dogma_chunk_feature_error": dogma_error,
        "dogma_terminal_state_error": carry_error,
        "hermon_repeated_prefix_error": hermon_error,
        "budgets": [
            {
                "model": "DOGMA reference",
                "parameters": sum(p.numel() for p in dogma.parameters()),
                "parameter_bytes": sum(
                    p.numel() * p.element_size() for p in dogma.parameters()
                ),
                "native_state_bytes": sum(s.numel() * s.element_size() for s in state),
                "per_layer_causal_edges": 0,
            },
```

```
            {
                "model": "Hermon reference",
                "parameters": sum(p.numel() for p in hermon.parameters()),
                "parameter_bytes": sum(
                    p.numel() * p.element_size() for p in hermon.parameters()
                ),
                "native_state_bytes": 0,
                "per_layer_causal_edges": 2 * x.shape[1] * (x.shape[1] + 1) // 2,
            },
        ],
        "dtype": "float64",
        "batch": 1,
        "bases": 8,
        "budget_scope": "static inventories/semantic edges, no measured
        ↪ peak/latency/FLOPs",
        "hermon_state_note": "stateless repeated-prefix call, not a production KV
        ↪ inventory",
    }
```

These checks use actual Chapter 22/23 references, not surrogate models named after them. They establish finite execution consistency for this probe. Chapter 26 broadens schedule, state and gradient parity. A serving engine additionally needs request ownership, reset, cancellation and cache-position tests before its timing results are interpretable.

25.11 Diagnose negative results without rewriting history

Chapter 23 supplies a real negative result to diagnose. Over three seeds the Hermon reference's mean held-out accuracy was about 0.529, its random-target control about 0.542 and a four-wide CNN about 0.724. The Hermon training loss stayed near $\ln 2$: the model never fit its training data. A negative result like this must be recorded as observed and then diagnosed. Rerunning until a better number appears, or relabelling the task afterwards, would rewrite history. These runs use their own frozen conditions and are not budget-matched architecture evidence.

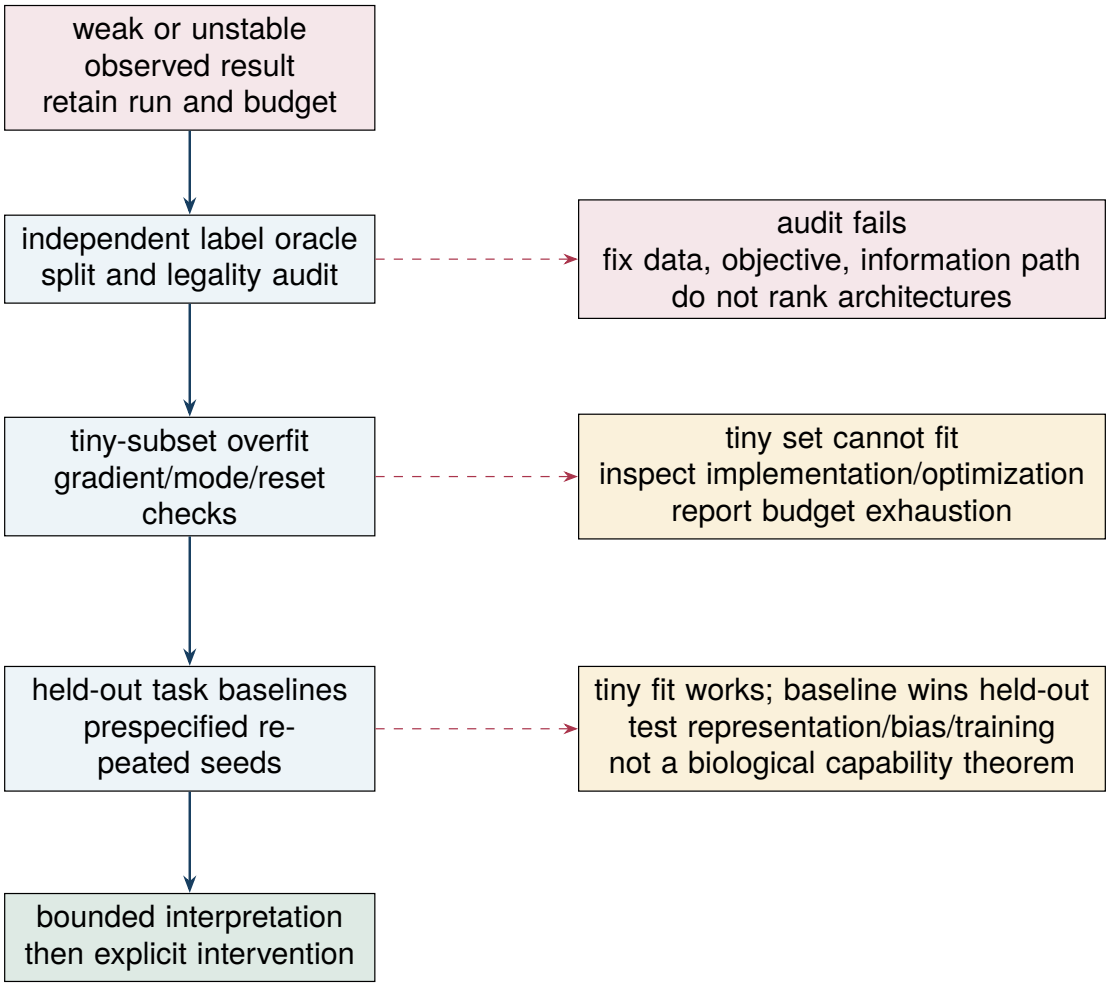


Figure 25.5: Verify labels and legal execution; try tiny-subset overfit; inspect gradients; compare task-relevant baselines; repeat the frozen design. Each branch localizes a failure without automatically declaring an architecture inferior.

First verify record IDs and labels with an independent oracle. Next train on a tiny subset and report whether near-zero training error is reached under the stated budget. Check gradients, logits, mode/reset and masks. If tiny-set overfit succeeds but held-out performance fails while a local CNN succeeds, representation or inductive-bias/training hypotheses strengthen. If it fails, implementation or optimization remains a primary suspicion. No branch automatically proves a biological capability or its absence.

25.12 Ablation, substitution and transplant are different interventions

An ablation removes a native mechanism; a matched substitution replaces it with a simpler legal mechanism; a transplant inserts it into another architecture. These ask different causal questions. Chapter 22's disabled memory-read path holds parameters fixed. A zero-locus model would change a different contract. A Hermon strand-averaging ablation must preserve dataset/objective and report its changed strand policy.

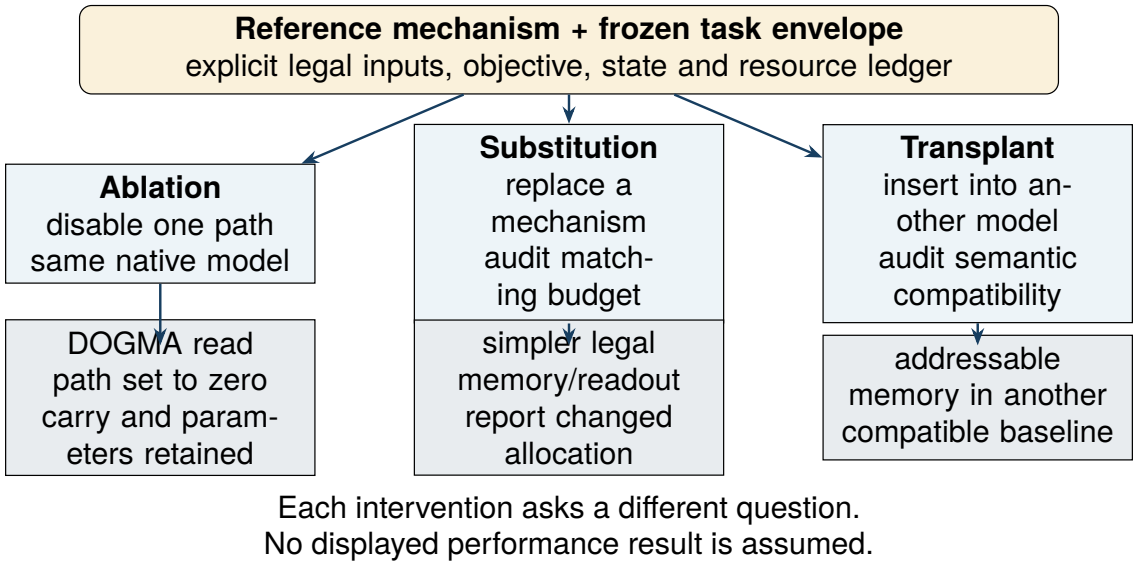


Figure 25.6: Mechanism interventions preserve the declared scientific envelope while changing a specific computational path. Parameter-matched substitution and transplant need their own budget and legality audits.

Attention weights and regulatory gates are not equivalent simply because both lie in $[0, 1]$. Their normalization, inputs and intervention semantics differ. Likewise a favorable full-model delta does not isolate which mechanism caused it. Retain negative, null and unstable interventions alongside improvements.

Key ideas

- A shared experiment envelope fixes the question and the evidence. It need not give the models a shared internal state.
- Reverse complements, group tags and curated links can chain together. The whole connected component must stay in one split.
- Keep both predictions attached to each resampled unit. With four perfectly correlated clusters, the honest interval is $[-1, 1]$, not the narrow row-level one.
- A test-set bootstrap conditional on one fitted pair says nothing about variation across training seeds.

25.13 Twenty worked exercises

1. Define a fixed-predictor estimand. **Answer:** the population mean of paired correctness differences under a declared distribution.

2. Why not hide DOGMA carry? **Answer:** reset/recomputation changes the tested function and cost.

3. Which Chapter 24 fields matter? **Answer:** base spans, legal information access, RC action and graph masks.

4. A–B by RC, B–C by group, C–D by overlap: split units? **Answer:** one transitive four-record component.

5. Does this code infer homology? **Answer:** no; curated groups/links must supply the biological criterion.

6. Does 60 percent of components mean 60 percent of records? **Answer:** not when component sizes differ.
7. Why threshold greater than one for all-negative GC? **Answer:** a pure-GC sequence has score one.
8. Can test labels select that threshold? **Answer:** no; selection belongs to training/declared validation access.
9. Always-negative accuracy on 3 negatives, 1 positive? **Answer:** 0.75; balanced accuracy 0.5.
10. One class absent: balanced accuracy? **Answer:** unavailable under this definition, not a fabricated number.
11. Identical wrong predictors: paired interval? **Answer:** exactly zero; agreement is not accuracy.
12. Derive paired sample variance. **Answer:** sample variance of d_i divided by N under iid-record assumptions.
13. Why keep both predictions in each bootstrap draw? **Answer:** separate draws destroy outcome covariance.
14. Four perfectly correlated eight-row clusters: independent n? **Answer:** four clusters, not 32 rows.
15. Unequal clusters: ratio of totals versus mean of means? **Answer:** record-weighted and cluster-weighted estimands differ.
16. Does record bootstrap estimate seed variation? **Answer:** no; it is conditional on fitted predictors.
17. DOGMA state bytes in the probe? **Answer:** $32 \cdot 8 = 256$.
18. Hermon causal edges? **Answer:** $2 \cdot 8 \cdot 9/2 = 72$.
19. Is repeated-prefix parity a KV-speed benchmark? **Answer:** no; it is a stateless finite correctness oracle.
20. Design a defensible comparison. **Rubric:** fixed estimand; immutable data/closure; objective and strand action; native adapters/parity; train/validation selection; task baselines; paired/cluster uncertainty; prespecified seeds/tuning; measured versus estimated budgets; ablations; negative-result ledger and reproducible artifacts.

25.14 Bridge

Chapter 26 derives parallel and chunked schedules for one recurrence and checks gradients as well as outputs. This chapter establishes why that parity is an experiment gate: a faster schedule that changes the function is not an optimization of the same model.

Evidence and source boundaries

The split-closure algorithm, synthetic comparison, paired/cluster calculations and native execution probes are inspectable teaching references. The uncertainty derivation is conditional

on its declared sampling unit; it is not a clinical or general biological inference guarantee. Model probes use the actual integrated Chapter 22/23 references, but neither train nor compare their capability here.

Primary abstract/metadata records for Dodge, Bouthillier and Dehghani were inspected on 6 October 2026. They support reporting, variance and benchmark-choice context, not a claim that this harness reproduces those studies. Chapters 22–24 were reread in full for native state, mask/span and pilot continuity. Independent specialist and reader review remain separate from these checks.

CHAPTER 26

Training/inference parity and parallel recurrence

In this chapter you will

- derive chronological affine composition and check associativity with an independent matrix oracle
- build an inclusive Hillis–Steele scan and fold resets and padding into effective pairs
- carry chunk state without cutting its gradient, and derive a reverse-mode adjoint oracle
- exhibit a real floating-point counterexample to unrestricted scan parity

A recurrent architecture is defined by its transition and state boundaries, not by a particular Python loop. If a different execution schedule implements the same function, training can exploit parallel work while inference updates one step at a time. The difficult part is proving equivalence without overlooking gradients, resets, padding, floating-point order or request ownership.

Chapter 25 tested actual Chapter 22 DOGMA carry and actual Chapter 23 Hermon causal prefix behavior. Here we isolate an affine recurrent core and build four independent views of it: a literal sequential loop, an inclusive scan, a chunked loop and a closed-form product/sum oracle. We then derive a reverse-mode gradient oracle. These are algebra and software experiments, not trained genomic capability or accelerator performance evidence.

26.1 Specify the transition before changing the schedule

Consider

$$h_t = a_t \odot h_{t-1} + b_t, \qquad h_t \in \mathbb{R}^D, \quad t = 1, \dots, T. \tag{26.1}$$

The elementwise coefficients a_t, b_t can be learned or input-dependent. For a whole-sequence parallel scan they must be available without first computing the recurrent states that the scan is supposed to produce. Causal legality alone is not sufficient: $a_t = g(h_{t-1}, x_t)$ may be causal yet require sequential evaluation. Local input-derived coefficients are one scan-compatible case.

This distinction matters for DOGMA. Its Chapter 22 complete state contains native hidden state and memory loci, including state-dependent read/write interactions. The entire nonlinear state machine does not become scan-compatible merely because one subexpression is affine. A scan proof for a frozen or input-conditioned core does not prove a scan implementation of that full model. HermonDNA remains the Transformer branch; Evolutor coordinates both rather than conflating their state.

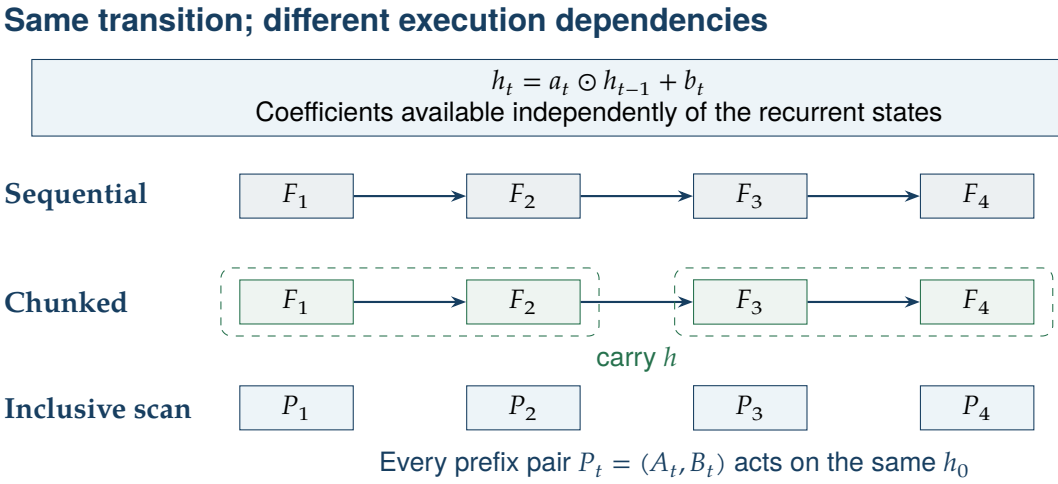


Figure 26.1: One affine transition has sequential, chunked and scan schedules. Coefficient availability is a prerequisite; this is not a claim that every DOGMA transition admits the same scan.

The reference uses a, b shaped $[T, *\text{state}]$ and h_0 shaped $[*\text{state}]$. Extra state axes can hold independent batch lanes. Shapes, floating dtype, device and finite inputs must match. It does not mutate PyTorch's global default dtype. The empty sequence returns no per-position states and final state h_0, rather than inventing a transition.

26.2 Derive chronological affine composition

Let $F = (a, b)$ denote $F(h) = a \odot h + b$. Applying F_L then F_R gives

$$F_R(F_L(h)) = (a_R \odot a_L) \odot h + (a_R \odot b_L + b_R).$$

Define chronological composition

$$F_L \otimes F_R = (a_R \odot a_L,\ a_R \odot b_L + b_R). \tag{26.2}$$

The left operand happens earlier even though function composition is written $F_R \circ F_L$. This convention must remain consistent in the diagram, code and tests.

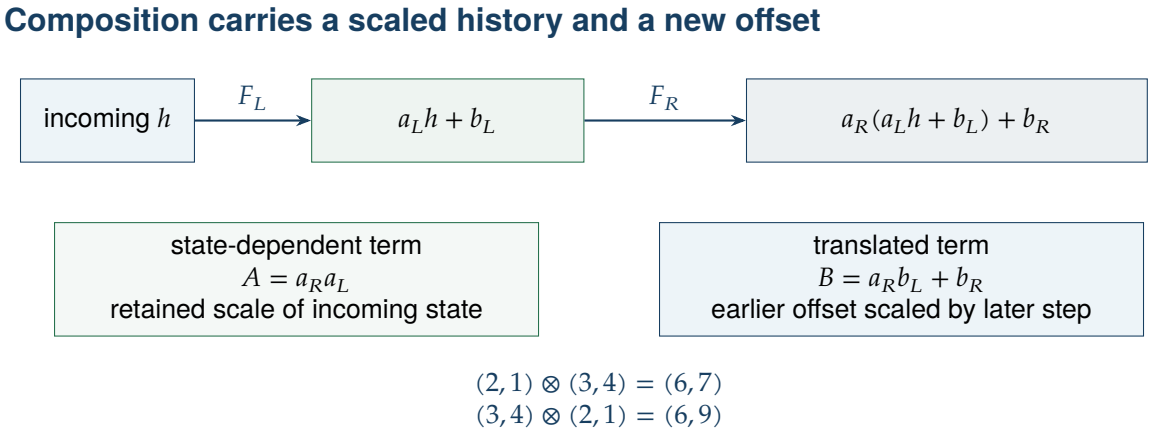


Figure 26.2: Chronological order fixes the translated term $a_R b_L + b_R$. Associativity permits regrouping; it never permits reordering.

```
def compose(left, right):
    """Chronological left then right; noncommutative affine composition."""
    a_left, b_left = left
    a_right, b_right = right
    if (
        a_left.shape != b_left.shape
        or a_right.shape != b_right.shape
        or a_left.shape != a_right.shape
    ):
        raise ValueError("Equal affine state shapes required")
    return a_right * a_left, a_right * b_left + b_right
```

For three steps both parenthesizations produce

$$(a_3a_2a_1,\ a_3a_2b_1 + a_3b_2 + b_3),$$

componentwise. Thus the pairs with identity $(\mathbf{1}, \mathbf{0})$ form an associative monoid over exact real arithmetic. They are generally not commutative. For $F_1 = (2, 1)$, $F_2 = (3, 4)$, chronological $F_1 \otimes F_2 = (6, 7)$, whereas $F_2 \otimes F_1 = (6, 9)$. A parallel algorithm that swaps operands is wrong.

An independent test represents scalar maps by homogeneous matrices $\begin{pmatrix} a & b \\ 0 & 1 \end{pmatrix}$. Their multiplication supplies an oracle without calling `compose`. Matrix-affine recurrences also compose in principle, but their operation cost and coefficient structure differ from this elementwise implementation; the scalar cost model must not be transferred blindly.

26.3 Prefixes, not just one tree reduction

The **affine prefix summary** at position t is

$$P_t = F_1 \otimes \cdots \otimes F_t = (A_t, B_t), \qquad h_t = A_t \odot h_0 + B_t.$$

Training often needs every h_t, not just the terminal state. A balanced reduction that returns only P_T is therefore not the required algorithm.

Our transparent Hillis–Steele scan starts with one-step pairs. At offsets $1, 2, 4, \ldots$, a position combines its earlier interval with its current interval using arrays from the *previous* stage. Unaffected leading positions are copied. After stage k, each position summarizes up to 2^{k+1} preceding steps including itself. This invariant proves complete prefixes after $\lceil \log_2 T \rceil$ stages.

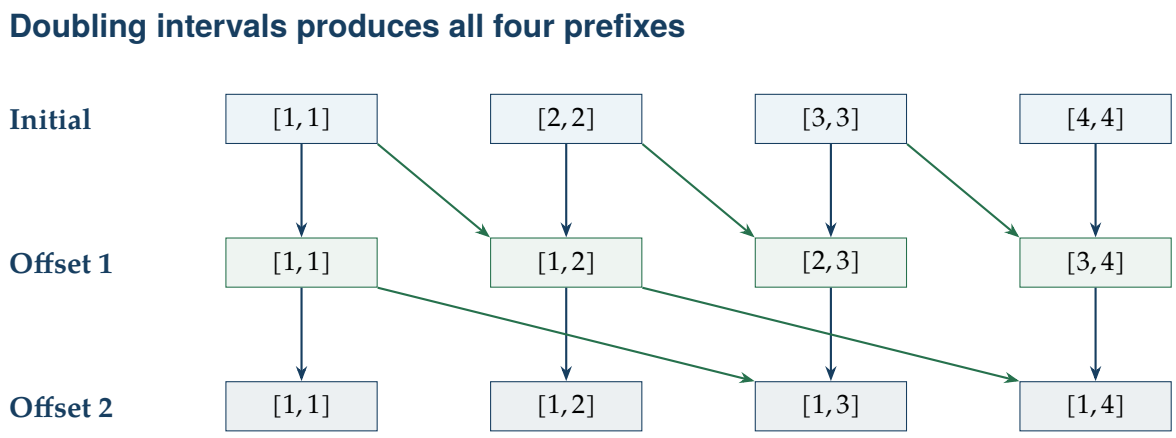


Figure 26.3: An inclusive scan produces every prefix. At each doubling stage all updates read the preceding stage; in-place partial updates would violate the interval invariant.

```
def hillis_steele_scan(a, b):
    """Inclusive scan: every stage reads the previous stage, never partially
     ↪ updated data."""
    if a.ndim < 2:
        raise ValueError("Time and state dimensions required")
    validate(a, b, a.new_zeros(a.shape[1:]))
    aa, bb = a, b
    offset = 1
    while offset < a.shape[0]:
        tail_a = aa[offset:] * aa[:-offset]
        tail_b = aa[offset:] * bb[:-offset] + bb[offset:]
        aa = torch.cat((aa[:offset], tail_a), dim=0)
        bb = torch.cat((bb[:offset], tail_b), dim=0)
        offset *= 2
    return aa, bb
```

For $h_0 = 2$, $a = (0.5, 2, 0, -1)$ and $b = (1, 0.5, 3, 2)$, direct updates give $h = (2, 4.5, 3, -1)$. The prefix summaries expose the same result:

Step	a	b	A	B	h
1	0.5	1	0.5	1	2
2	2	0.5	1	2.5	4.5
3	0	3	0	3	3
4	-1	2	-0	-1	-1

A zero coefficient discards the incoming state. Negative and expansive coefficients are legal algebraically; imposing $(0, 1)$ coefficients in a model is an additional parameterization choice, not a requirement of affine composition.

Blelloch's chapter establishes associative prefix computation as prior art [28]. S5's author abstract describes a multi-input, multi-output state-space layer compatible with parallel scans [30]. Mamba's selected mechanism sections distinguish input-dependent state-space computation and hardware-aware scan, fusion and recomputation [10]. These sources

motivate engineering questions; our reference neither reproduces their trained results nor implements their production kernels.

26.4 Resets and padding belong in the algebra

Let r_t mark **reset-before-update** and v_t mark a valid position. With an explicit reset state h_{reset}, a valid reset step computes $a_t \odot h_{\text{reset}} + b_t$ independently of h_{t-1}. Fold these semantics into effective pairs:

$$(\tilde{a}_t, \tilde{b}_t) = \begin{cases} (\mathbf{1}, \mathbf{0}), & v_t = 0, \\ (\mathbf{0}, a_t \odot h_{\text{reset}} + b_t), & v_t = 1,\ r_t = 1, \\ (a_t, b_t), & v_t = 1,\ r_t = 0. \end{cases}$$

Now the same scan machinery handles segmented records and identity padding. Our schema rejects reset requests on padding rather than allowing conflicting instructions. Masks are Boolean time vectors shared across state lanes; per-lane ragged resets require a separately extended mask contract, not accidental broadcasting.

```
def effective_maps(a, b, h_reset, resets=None, valid=None):
    """Fold reset-before-valid-step and padding identity into affine
     ↪  coefficients."""
    validate(a, b, h_reset)
    t = a.shape[0]
    if resets is None:
        resets = torch.zeros(t, dtype=torch.bool, device=a.device)
    if valid is None:
        valid = torch.ones(t, dtype=torch.bool, device=a.device)
    for mask in (resets, valid):
        if (
            not isinstance(mask, torch.Tensor)
            or mask.shape != (t,)
            or mask.dtype != torch.bool
            or mask.device != a.device
        ):
            raise ValueError("Boolean time masks on coefficient device required")
    if (resets & ~valid).any():
        raise ValueError("Padding cannot request a reset")
    shape = (t,) + (1,) * (a.ndim - 1)
    r, v = resets.reshape(shape), valid.reshape(shape)
    aa = torch.where(r, torch.zeros_like(a), a)
    bb = torch.where(r, a * h_reset + b, b)
    return torch.where(v, aa, torch.ones_like(a)), torch.where(
        v, bb, torch.zeros_like(b)
    )
```

For $h_0 = 2$, $h_{\text{reset}} = 10$, $a_t = 0.5$, $b_t = 1$, three valid steps with a reset at step two give $(2, 6, 4)$. If step three is padding, its state remains 6. Setting padded $a = b = 0$ would erase the state, not preserve it. A reset cuts dependence on the earlier record, but may preserve dependence on a shared learned reset parameter. When $h_{\text{reset}} = h_0$, gradients to that common parameter still accumulate across reset segments.

26.5 Carry the state and preserve its gradient

A chunk boundary is not a biological sequence boundary. For a chunk starting at s, its incoming state must be the previous chunk's terminal state h_{s-1}. Reset masks retain their original positions, and h_{reset} stays the global reset parameter, not the new chunk's incoming state.

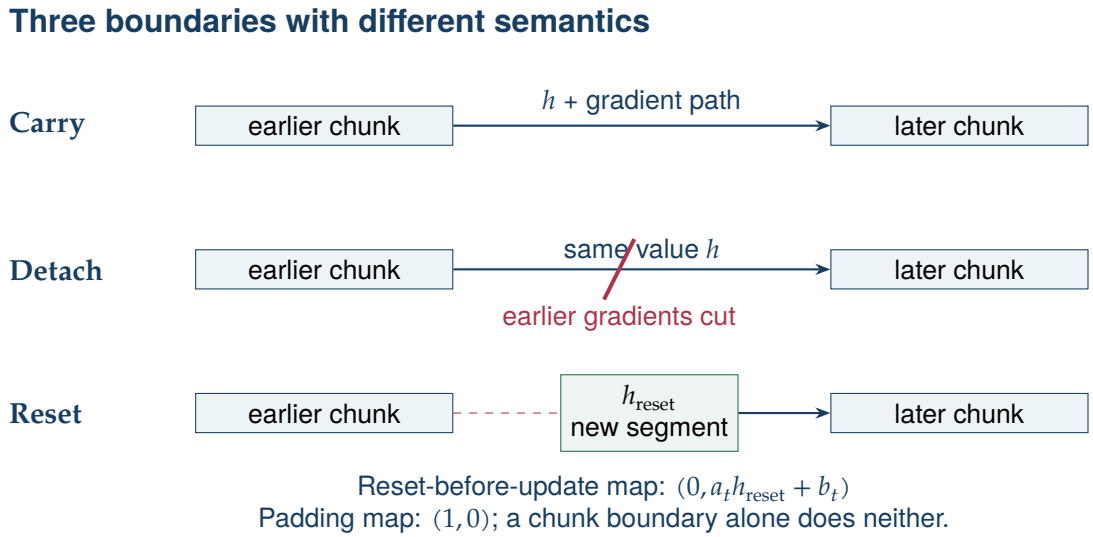


Figure 26.4: Ordinary chunking carries both the value and its differentiation path. Reset starts a declared new segment; detach keeps values but deliberately changes the gradient.

```
def chunked(a, b, h0, chunk, resets=None, valid=None, h_reset=None, detach=False):
    """Chunk boundaries carry state; optional detach is an explicit
     ↪  gradient-changing mutant."""
    validate(a, b, h0)
    if type(chunk) is not int or chunk <= 0:
        raise ValueError("Positive integer chunk size required")
    if type(detach) is not bool:
        raise ValueError("Detach intervention must be Boolean")
    h_reset = h0 if h_reset is None else h_reset
    effective_maps(a, b, h_reset, resets, valid)
    states, h = [], h0
    for start in range(0, a.shape[0], chunk):
        end = start + chunk
        xx, h = sequential(
            a[start:end],
            b[start:end],
            h,
            None if resets is None else resets[start:end],
            None if valid is None else valid[start:end],
            h_reset,
        )
        states.append(xx)
        if detach and end < a.shape[0]:
            h = h.detach()
    return torch.cat(states) if states else a.clone(), h
```

The reference tests sizes $1, 2, 3, 5$, larger-than-sequence chunks and uneven tails. It includes an explicit `detach=True` intervention for a counterexample. With $a = (1, 1)$, $b = (0, 0)$, $h_0 = 1$ and

loss $L = h_2$, both ordinary and detached chunking produce $h_2 = 1$. Ordinary differentiation gives $\partial L/\partial b_1 = 1$; detaching after step one gives zero. Forward parity alone therefore cannot establish training parity.

Truncated backpropagation may intentionally detach boundaries. It is a different gradient contract, not a transparent memory optimization. Chapter 27 will separate gradient accumulation and activation recomputation from such changes.

26.6 An independent closed-form oracle

Unroll Eq. 26.1 to obtain

$$h_t = \left(\prod_{j=1}^{t} a_j\right) \odot h_0 + \sum_{k=1}^{t} \left(\prod_{j=k+1}^{t} a_j\right) \odot b_k. \tag{26.3}$$

An empty product is $\mathbf{1}$. This formula handles $a_j = 0$ without division. Rewriting every suffix product as a ratio of cumulative products would fail at zero, precisely where reset semantics become important.

The `closed_form` reference evaluates each prefix using products and a reverse suffix-product loop, rather than the composition operator or forward state loop. It costs $O(T^2D)$ and exists for verification, not serving. Tests compare all four schedules across empty, singleton, power-of-two and non-power-of-two lengths. Hand traces, homogeneous matrices and this product/sum formula reduce the risk of two implementations sharing the same mistake.

26.7 Derive a reverse-mode gradient oracle

For an assigned linear loss $L = \sum_t w_t^\top h_t$, let $\lambda_t = \partial L/\partial h_t$. Reverse accumulation gives

$$\lambda_T = w_T, \qquad \lambda_t = w_t + a_{t+1} \odot \lambda_{t+1}.$$

Consequently

$$\frac{\partial L}{\partial b_t} = \lambda_t, \quad \frac{\partial L}{\partial a_t} = \lambda_t \odot h_{t-1}, \quad \frac{\partial L}{\partial h_0} = a_1 \odot \lambda_1. \tag{26.4}$$

These derivatives are with respect to the provided coefficient tensors. If a coefficient generator has parameters θ, an end-to-end implementation must also differentiate through $a(\theta, x), b(\theta, x)$ and compare those gradients.

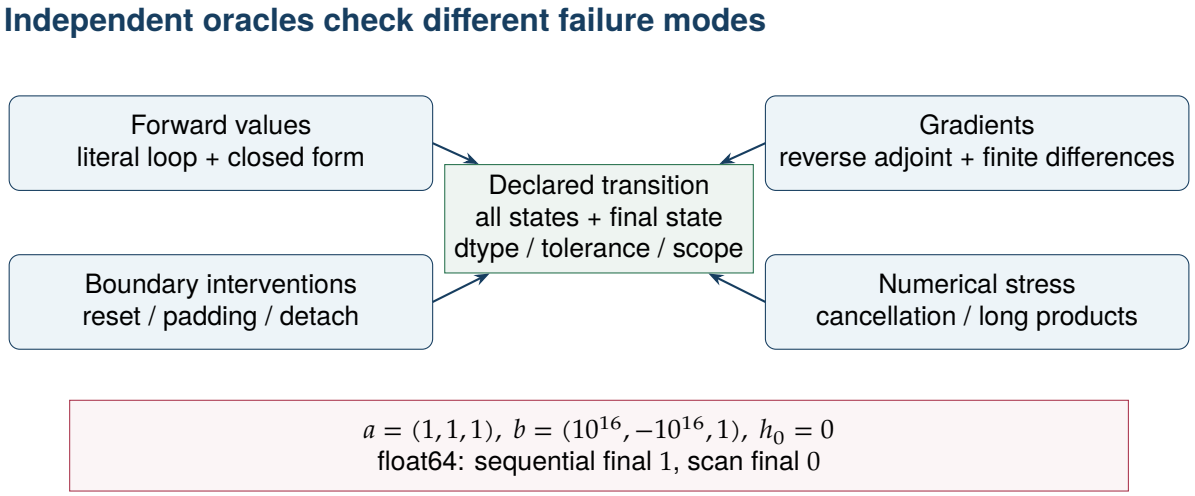


Figure 26.5: Parity requires independent forward and derivative checks, including state-boundary interventions. Agreement on one convenient trace is not a universal stability result.

```
def analytic_adjoint(a, b, h0, weights):
    """Independent reverse derivative for L=sum_t,d weights[t,d]*h[t,d]."""
    states, _ = sequential(a, b, h0)
    if weights.shape != states.shape:
        raise ValueError("One loss weight per state element required")
    lam = torch.zeros_like(h0)
    ga, gb = [], []
    for index in range(len(a) - 1, -1, -1):
        lam = weights[index] + (a[index + 1] * lam if index + 1 < len(a) else 0)
        prior = h0 if index == 0 else states[index - 1]
        ga.append(lam * prior)
        gb.append(lam)
    if not len(a):
        return a.clone(), b.clone(), torch.zeros_like(h0)
    return torch.stack(ga[::-1]), torch.stack(gb[::-1]), a[0] * lam
```

The reverse oracle implements Eq. 26.4 without asking autograd for these derivatives. Central finite differences provide another check, including a zero coefficient. Reset tests compare all participating gradients, including the reset state, against the literal reset loop. Padding coefficient gradients are zero because its effective map is independent of those coefficients.

For seed 26, length 17, dimension 5 and float64 coefficients in $[0.8, 0.9)$, the generated result file reports maximum forward discrepancy 2.22045×10^{-16}, gradient discrepancy 4.44089×10^{-16} and manual-adjoint discrepancy zero. This is a bounded deterministic fixture, not a general theorem. Long 1024-step bounded-coefficient tests use declared float64 and float32 tolerances; no BF16 or FP16 certification is inferred.

26.8 Numerical equivalence has a real counterexample

Exact-real affine composition is associative. Machine addition and multiplication are not exactly associative. Consider $a = (1,1,1)$, $b = (10^{16}, -10^{16}, 1)$, $h_0 = 0$ in float64. Sequential evaluation gives final state 1; this scan's regrouping gives 0. The absolute difference is one, not a negligible last-bit artifact at the output scale.

The example exposes cancellation: adding one to a large-magnitude partial sum can round it away before the opposite large term arrives. It does not refute the real-arithmetic proof; it refutes an unrestricted numerical-parity claim. Declare dtype, coefficient/state scales, context lengths and both absolute and relative tolerances. An allclose pass on contractive random coefficients says little about a cancellation-heavy or expansive workload.

Long products also express model stability. Values $|a_t| < 1$ tend to attenuate older contributions; values above one can amplify state and gradients. A gate parameterization may reduce amplification but does not eliminate cancellation, underflow, accumulation error or data-distribution shift. The software validates finite inputs; it does not guarantee that every finite input yields finite intermediates. Production paths need output monitoring and explicit non-finite failure handling.

26.9 Depth, work and measured latency are different quantities

At offset 2^k, our scan performs $T - 2^k$ pair compositions. For $T > 1$,

$$W_{\mathrm{HS}}(T) = \sum_{k=0}^{\lceil \log_2 T \rceil - 1} (T - 2^k), \qquad D_{\mathrm{HS}}(T) = \lceil \log_2 T \rceil.$$

This is $O(T \log T)$ pair work and $O(\log T)$ idealized stage depth. It is not a work-efficient Blelloch up-sweep/down-sweep implementation. For $T = 1024$, the reference performs 9217 pair compositions across ten stages; a literal sequence makes 1024 state updates. A pair composition and a state update are not identical FLOP units.

Length	Scan stages	Pair ops	Loop depth
0	0	0	0
1	0	0	1
4	2	5	4
17	5	54	17
1024	10	9217	1024

Tensor allocation, concatenation, synchronization and kernel dispatch remain outside this pair-operation count. With unlimited ideal parallel lanes, lower dependency depth can help; with finite hardware, extra work and memory traffic can dominate. No measured speedup is reported. A serious benchmark would specify hardware, shape, batch, dtype, compilation/fusion, warmup, device synchronization, peak memory and forward/backward scope.

Mamba's selected implementation discussion emphasizes kernel fusion and avoiding large intermediates in the memory hierarchy [10]. It would be misleading to assign those properties to this intentionally transparent PyTorch teaching scan.

26.10 Serving parity includes ownership

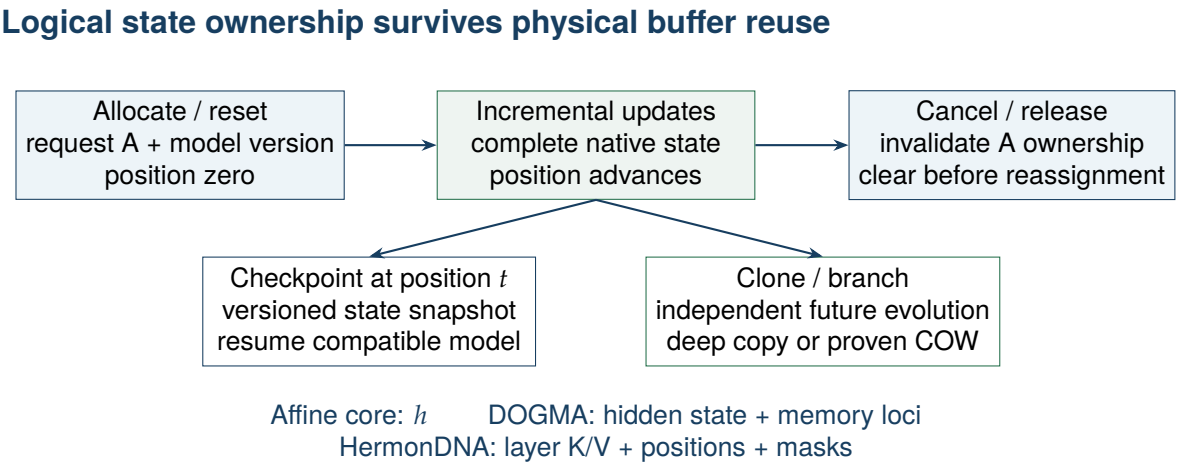


Figure 26.6: State belongs to a logical request, not merely a reusable buffer. Versioned position, complete state and reset/clone/cancel behavior determine the meaning of incremental execution.

State ownership means a request state identifies its model/configuration version, sequence position, dtype/device and all architecture-specific tensors. For the affine reference that is h; for actual DOGMA it includes native hidden state and memory loci; for HermonDNA cached attention it includes layer-specific K/V, position and masks. A common interface may wrap them but cannot replace their semantics.

Clone should preserve independent future evolution through deep copy or a proven copy-on-write contract. A checkpoint names both position and model version. Cancellation invalidates the logical owner before buffer reuse. A reset on a new request is not an optional performance hint. Otherwise serving can leak previous request information even if the mathematical recurrence is correct.

Chapter 25's Hermon prefix test repeated causal prefixes; it did not implement KV-cache decoding. This chapter likewise makes no Hermon KV-parity claim. That requires actual cache construction, correct positions/masks and every-position logit comparison against the complete causal model.

26.11 Failure-oriented verification

The chapter tests reject swapped composition order, partially updated in-place scan stages, wrong padding maps, reset-on-padding instructions and detached boundary gradients. Future-coefficient perturbations must not change earlier states; independent batch lanes must not interact. Invalid shapes, dtypes, non-finite inputs and Boolean chunk sizes are rejected.

No one oracle suffices. Matrix products check composition; hand arithmetic checks a trace; the closed form checks prefix values; the reverse recurrence and finite differences check derivatives; boundary interventions check lifecycle semantics. The cancellation fixture remains an expected disagreement, ensuring that tests do not erase a real numerical limitation merely to make every case pass.

Key ideas

- The left operand happens first: $(2,1) \otimes (3,4) = (6,7)$, while the reversed order gives $(6,9)$.
- Padding is $(\mathbf{1},\mathbf{0})$ and reset-before-update is $(\mathbf{0}, a_t h_{\text{reset}} + b_t)$. Setting padded coefficients to zero erases the state.
- Detaching a chunk boundary keeps forward values and changes gradients, so forward parity cannot establish training parity.
- In float64, $(10^{16}, -10^{16}, 1)$ sums to 1 sequentially and to 0 under the scan's regrouping. State the tolerance and workload for any parity claim.

26.12 Twenty worked exercises

1. Compose $(2,1)$ then $(3,4)$. **Answer:** $(6,7)$; reversing gives $(6,9)$.
2. What is the identity? **Answer:** $(\mathbf{1},\mathbf{0})$.
3. Expand a three-step translated term. **Answer:** $a_3a_2b_1 + a_3b_2 + b_3$.
4. Why is a terminal reduction insufficient for training? **Answer:** losses may require every position's state.
5. Trace the four-step example from $h_0 = 2$. **Answer:** $(2, 4.5, 3, -1)$.
6. What does $a_3 = 0$ erase? **Answer:** dependence of h_3 on earlier incoming state, while retaining b_3.
7. Why not divide cumulative products to compute suffixes? **Answer:** zero coefficients make the ratio undefined.
8. Give the pair for reset-before-update. **Answer:** $(0, a_t h_{\text{reset}} + b_t)$.
9. Give the pair for padding. **Answer:** $(1,0)$, not $(0,0)$.
10. Compute states with $a = .5, b = 1, h_0 = 2, h_{\text{reset}} = 10$ and reset at step two. **Answer:** $(2,6,4)$.
11. What if step three is padding? **Answer:** its state is 6.
12. Does resetting cut all gradients to a shared reset parameter? **Answer:** no; it cuts earlier-record state dependence, not the new reset contribution.
13. Why preserve the global reset state across chunks? **Answer:** using the incoming chunk state as reset changes segment semantics.
14. Give a forward-equal, gradient-unequal detach example. **Answer:** two identity-coefficient steps with $L = h_2$ yield $\partial L/\partial b_1 = 1$ ordinarily, zero after boundary detach.
15. Derive $\partial L/\partial b_t$. **Answer:** the direct state derivative is one, hence λ_t.
16. Why can causal state-derived coefficients prevent a whole-sequence scan? **Answer:** producing coefficients still requires previous recurrent states.
17. Count pair compositions at $T = 4$. **Answer:** $(4-1) + (4-2) = 5$ across two stages.
18. At $T = 1024$? **Answer:** $10(1024) - (1 + \cdots + 512) = 9217$.

19. Evaluate the cancellation example under the two schedules. **Answer:** float64 sequential final 1, reference scan final 0.

20. What would justify a production parity claim? **Rubric:** actual model/coefficient gradients, complete native state, resets/padding/positions, representative dtype and length tests, independent oracles, disclosed tolerances and separately measured systems performance.

26.13 Bridge to memory-efficient training

The next chapter changes how training work and activations are scheduled. Accumulation must preserve the declared loss weighting; recomputation must preserve the transition and stochastic behavior; detach changes the gradient unless it is explicitly part of the objective. The present chapter supplies the value, derivative and boundary contracts against which those changes must be tested.

Evidence and source boundaries

The affine derivations, reference code, reset/padding contract, independent gradient oracles and numerical counterexamples are original teaching artifacts. They do not establish full-DOGMA scan eligibility, Hermon KV parity or hardware performance. The inspected Blelloch source is the chapter PDF associated with the author's 1993 publication listing, not a claimed reading of the separate 1990 technical report. Mamba inspection covers selected mechanism sections; S5 inspection covers its author abstract and metadata. No external training or benchmark result is reproduced.

CHAPTER 27

Optimization and memory-efficient training

In this chapter you will

- separate persistent array inventories from peak memory
- accumulate weighted microbatch numerators against one global denominator
- check Adam's moments, bias-correction clock and updated parameters, not only gradients
- use activation checkpointing without changing stochastic behaviour

A memory optimization is correct only relative to a declared training procedure. This chapter treats the procedure as an invariant: fix the objective, parameters, randomness policy, and optimizer-update clock, then change how the computation is scheduled. We derive weighted accumulation, compare every gradient, verify Adam's moments and updated parameters, and deliberately break stochastic recomputation.

The executable experiment is a small CPU Float64 MLP, not DOGMA or Hermon DNA training, a mixed-precision certification, or a hardware benchmark. Its value is a sharp correctness oracle that later implementations must match where their assumptions apply. Chapter 26's distinction between equal outputs and equal gradient graphs remains essential.

27.1 Persistent arrays are not peak memory

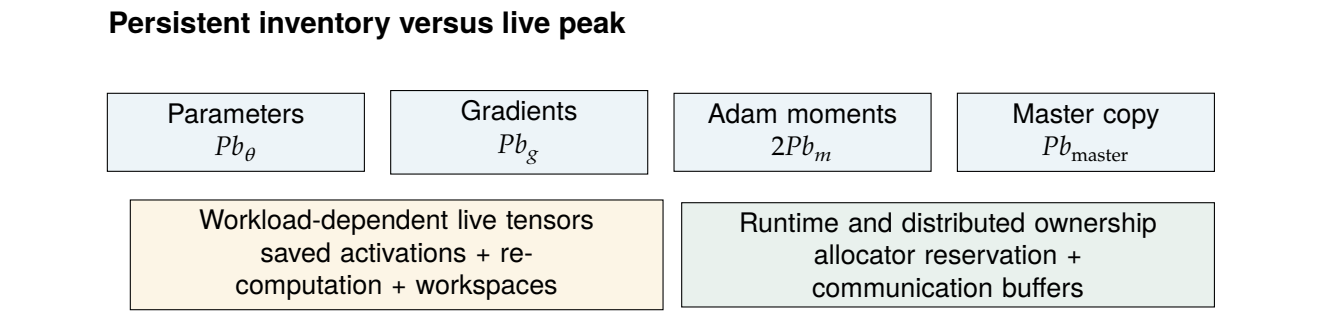


Figure 27.1: Memory has multiple owners and lifetimes. Static parameter and optimizer inventories do not include saved activations, temporary workspaces, or allocator reservation.

For P parameters, parameter/gradient widths b_θ, b_g, two Adam moments of width b_m, and optional master-copy width b_{master},

$$M_{\text{arrays}} = P(b_\theta + b_g + 2b_m + b_{\text{master}}).$$

The default all-FP32 layout gives $16P$ bytes. The reference MLP has $8 \cdot 16 + 16 + 16 \cdot 3 + 3 = 195$ parameters: this hypothetical inventory is 3,120 bytes. Its actual Float64 parameter, gradient, and moment arrays would total 6,240 bytes after all arrays exist, before object overhead or temporary tensors.

A low-precision layout with two-byte weights, two-byte gradients, four-byte moments, and four-byte master weights also totals $16P$. Lowering the weight dtype alone does not prove lower total training memory. Peak memory additionally depends on activation lifetimes, kernel workspaces, optimizer temporaries, cached allocations, and communication. These components need measurement, not a formula substitution.

27.2 Define the reduction before choosing microbatches

Let $p_{ij}(\theta)$ predict output j for record i, and let fixed nonnegative weights w_{ij} include both validity masks and importance weights. Define

$$Z = \sum_{i,j} w_{ij} > 0, \qquad L(\theta) = \frac{1}{Z} \sum_{i,j} w_{ij}(p_{ij}(\theta) - y_{ij})^2.$$

This **weighted reduction** treats a mask as a zero weight, not a target value. Target, prediction, and weight shapes must match exactly: silent broadcasting can turn a three-output objective into a different computation while still producing a scalar.

```
def weighted_sum(prediction, target, weights):
    """No broadcasting: numerator is sum of weighted squared output errors."""
    if prediction.shape != target.shape or weights.shape != target.shape:
        raise ValueError("Prediction/target/weight shapes must match exactly")
    return ((prediction - target).square() * weights).sum()
```

The batch validator requires finite floating inputs with matching dtype/device, fixed weights without gradients, no negative weights, and a finite positive global mass Z. Individual finite weights can sum to infinity, so the sum itself must be checked. Intermediate predictions or squared errors can still overflow; finite input validation is not a universal numerical-stability guarantee.

For cross-entropy over genomic tokens, replace squared error with token loss and specify whether the denominator counts valid tokens, sequences, or importance mass. Averaging each sequence mean weights short and long sequences equally; averaging all valid token losses does not. Neither is inherently wrong, but they are different experiments.

27.3 Accumulate numerators with one global denominator

Partition records into disjoint microbatches B_k. Keep θ fixed throughout the accumulation window and define

$$N_k = \sum_{i \in B_k, j} w_{ij}(p_{ij} - y_{ij})^2, \qquad L = \sum_k N_k / Z.$$

Linearity of differentiation gives

$$\nabla_\theta L = \sum_k \nabla_\theta (N_k / Z).$$

This **gradient accumulation** adds contributions in parameter gradient buffers. A microbatch with zero valid weight contributes zero; it does not divide by a local zero denominator. The complete window must still have positive mass.

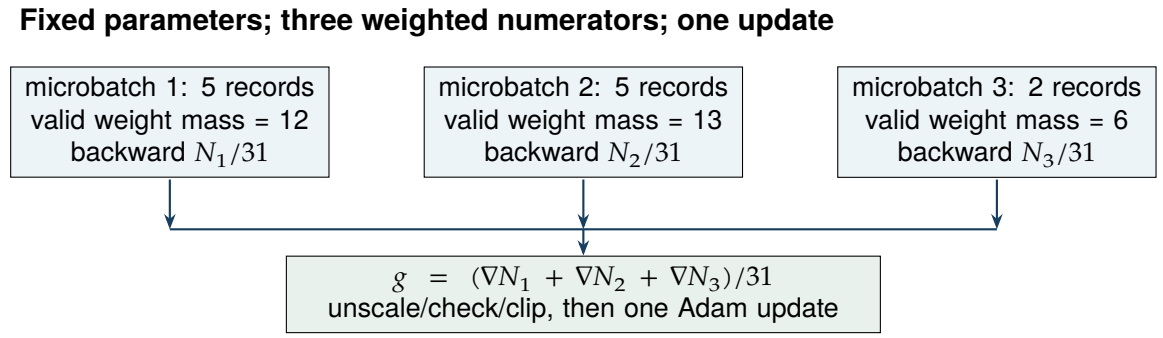


Figure 27.2: For a 5/5/2 record split, valid weight masses are 12, 13, and 6. Every numerator divides by 31, not by its local mass and then by three.

```
def accumulated_grad(model, x, y, weights, micro):
    """Each numerator uses the same global valid-weight denominator."""
    denominator = validate_batch(x, y, weights)
    if type(micro) is not int or micro < 1:
        raise ValueError("Positive integer microbatch size required")
    model.zero_grad(set_to_none=True)
    total = 0.0
    for start in range(0, len(x), micro):
        end = start + micro
        loss = (
            weighted_sum(model(x[start:end]), y[start:end], weights[start:end])
            / denominator
        )
        total += loss.item()
        loss.backward()
    return total, grads(model)
```

The fixed fixture has 12 records, eight features, and three outputs. Some outputs are masked and selected first outputs have weight two; total mass is 31. The batch loss is 1.099552069480512.

Micro	Weighted loss	Gradient error
1	1.09955	5.55112e-17
2	1.09955	6.93889e-17
5	1.09955	8.32667e-17
12	1.09955	0
20	1.09955	0

Microbatch size five creates a partial last microbatch. The measured errors are maximum absolute differences over *all* parameter gradients. Float64 roundoff changes addition order, so a tolerance is appropriate; bitwise equality is not the mathematical contract.

The shortcut $\frac{1}{K}\sum_k N_k/Z_k$ changes the objective unless local masses Z_k are equal or special cancellations occur. Its gradient differs from the correct gradient by about 0.141178 in this fixture. This is an executable counterexample, not an argument that every equal-sized record split is wrong: equal record counts can still have unequal valid token or weight mass.

27.4 An independent derivative oracle

For this MLP, $H = \tanh(XW_1^\top + b_1)$ and $P = HW_2^\top + b_2$. Holding H fixed for the last-layer derivative,

$$D = \frac{2}{Z} W \odot (P - Y), \qquad \nabla_{W_2} L = D^\top H, \qquad \nabla_{b_2} L = \sum_i D_{i,:}.$$

Tests compare these manual expressions with autograd without calling the accumulation helper. A separate scalar loop computes the loss element by element. A central finite difference checks one first-layer weight, exercising the nonlinear chain rule through tanh.

Scaling every weight by the same positive constant leaves L and its gradient unchanged because numerator and denominator scale together. This metamorphic test also checks the zero-valid-microbatch case. It is stronger than comparing two implementations that accidentally share the same reduction mistake.

27.5 The optimizer clock advances once per window

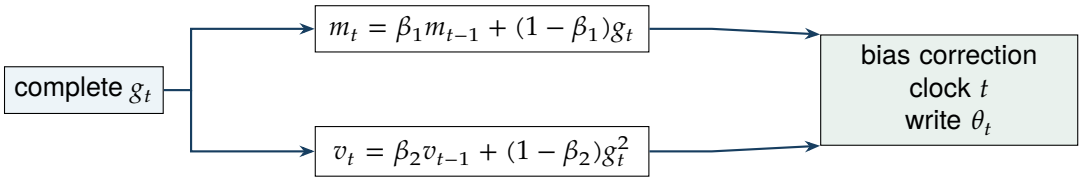
For Adam without weight decay or AMSGrad,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_t = \theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Here t counts parameter updates, not microbatches. Calling `step()` after each microbatch changes the parameters used for later forwards and changes the moment histories. It no longer implements the fixed-θ derivation.

One complete gradient advances one optimizer clock



Independent gates compare loss, gradients, both moment arrays and updated parameters. A microbatch is not an optimizer step.

Figure 27.3: The update boundary consumes the complete gradient once, advances both moments and bias-correction clock, and writes the new parameters.

```
def adam_step(
    theta, gradient, m, v, step, lr=0.01, beta1=0.9, beta2=0.999, epsilon=1e-8
):
    """Out-of-place scalar/vector Adam oracle, no weight decay or AMSGrad."""
    values = (theta, gradient, m, v)
    if not all(
        isinstance(t, torch.Tensor)
        and t.is_floating_point()
        and torch.isfinite(t).all()
```

```
        for t in values
    ):
        raise ValueError("Finite floating optimizer tensors required")
    if (
        len({t.shape for t in values}) != 1
        or len({t.dtype for t in values}) != 1
        or len({t.device for t in values}) != 1
    ):
        raise ValueError("Matching optimizer tensor shape/dtype/device required")
    if (v < 0).any() or type(step) is not int or step < 1:
        raise ValueError(
            "Nonnegative second moment and positive integer update clock required"
        )
    for value in (lr, beta1, beta2, epsilon):
        if type(value) not in (int, float) or not math.isfinite(value):
            raise ValueError("Finite optimizer hyperparameters required")
    if lr < 0 or epsilon <= 0 or not 0 <= beta1 < 1 or not 0 <= beta2 < 1:
        raise ValueError("Invalid Adam hyperparameter domain")
    next_m = beta1 * m + (1 - beta1) * gradient
    next_v = beta2 * v + (1 - beta2) * gradient.square()
    corrected_m = next_m / (1 - beta1**step)
    corrected_v = next_v / (1 - beta2**step)
    next_theta = theta - lr * corrected_m / (corrected_v.sqrt() + epsilon)
    if not all(torch.isfinite(t).all() for t in (next_theta, next_m, next_v)):
        raise ValueError("Non-finite optimizer result")
    return next_theta, next_m, next_v
```

The oracle is out of place and validates tensor shape/dtype/device, finite values, nonnegative second moments, and hyperparameter domains. It checks finite outputs as well. Tests compare full-batch and accumulated PyTorch Adam updates, every first/second moment, and the formula above. A two-step scalar calculation independently checks that the bias-correction clock is not stuck at one.

Gradient parity alone can miss an optimizer bug. A meaningful equivalence gate is (L, g, m, v, θ) parity under the same initial optimizer state. For restart tests, the stored update count and moments are therefore part of the checkpoint, not optional metadata.

27.6 Clipping and loss scaling have an order

If gradients are scaled by factor s, first accumulate the consistently scaled contributions, then unscale the complete gradient, check finiteness, clip if required, and perform one optimizer update. The update is skipped under an overflow policy when nonfinite gradients are detected.

Clipping is nonlinear: generally $\text{clip}(g_1) + \text{clip}(g_2) \neq \text{clip}(g_1 + g_2)$. For scalar threshold one, $g_1 = 2, g_2 = -1.5$ gives zero after separate clipping, but 0.5 after summing and clipping. Microbatch-local clipping changes the algorithm. Equally, clipping a scaled gradient at the unscaled threshold quietly changes the threshold by $1/s$.

This chapter specifies this order but does not execute an accelerator AMP training run. A production implementation must test actual scaler behavior, skipped-update schedules, dtype policies, and optimizer states under its installed framework.

27.7 Checkpointing changes retention, not the derivative

Reverse-mode differentiation needs intermediate values to evaluate vector–Jacobian products. **Activation checkpointing** retains selected boundary inputs and recomputes missing values during backward. It trades storage for extra execution; it must not change the function being differentiated.

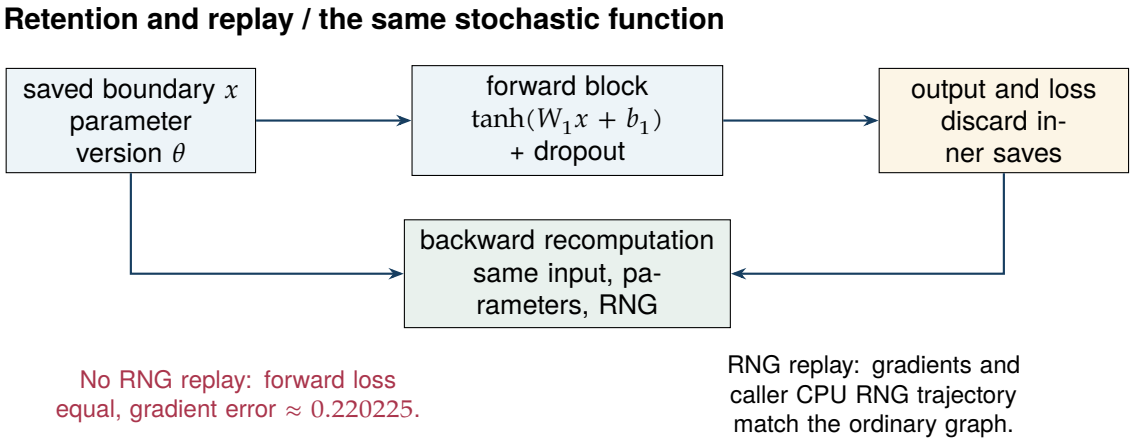


Figure 27.4: Forward retention and backward recomputation are different phases of one derivative graph. Random state must replay the same stochastic function, not merely produce tensors with matching shapes.

```
def checkpoint_grad(model, x, y, weights, dropout=0.0, preserve_rng=True):
    """Non-reentrant recomputation with explicit CPU stochastic-state policy."""
    from torch.utils.checkpoint import checkpoint

    denominator = validate_batch(x, y, weights)
    if (
        type(preserve_rng) is not bool
        or type(dropout) not in (int, float)
        or not 0 <= dropout < 1
    ):
        raise ValueError("Boolean RNG policy and dropout in [0,1) required")
    model.zero_grad(set_to_none=True)

    def block(z):
        hidden = torch.tanh(model.l1(z))
        return torch.nn.functional.dropout(hidden, p=dropout, training=True)

    hidden = checkpoint(block, x, use_reentrant=False,
     ↪  preserve_rng_state=preserve_rng)
    loss = weighted_sum(model.l2(hidden), y, weights) / denominator
    loss.backward()
    return loss.item(), grads(model)
```

The reference explicitly uses PyTorch's non-reentrant checkpoint path. For the deterministic block, both loss and every parameter gradient match ordinary execution in the Float64 fixture. The first-layer parameters still receive gradients even though input data do not require gradients.

The PyTorch 2.10 documentation distinguishes the variants and describes default recomputation checks based on shapes, dtypes, and devices [62]. Those checks are not a

numerical equality oracle. The tested runtime is PyTorch 2.10.0; compiler modes and additional devices require separate validation.

27.8 Same forward, wrong stochastic backward

Dropout evaluates a function of both activations and a random mask. Backward recomputation must reuse the appropriate stochastic state. With the same initial CPU RNG state, ordinary execution, checkpointing with preservation, and checkpointing without preservation have identical first forward losses. Yet the last version recomputes with a different mask and produces a maximum gradient discrepancy of about 0.220225.

The preserved version matches the ordinary gradients and leaves the caller's CPU RNG trajectory in the same post-operation state. This tests both the derivative and the surrounding random-state contract. Model initialization uses a local `fork_rng` context and data use a dedicated generator; importing the reference does not change the global default dtype or seed.

Random replay is only one hidden-state problem. A block that appends logs, updates running statistics, writes an external memory, or mutates its input can behave differently when executed again. Functionalization is preferable:

$$(h_{\mathrm{out}}, s_{\mathrm{out}}) = F(h_{\mathrm{in}}, s_{\mathrm{in}}; \theta),$$

with state transitions explicit and externally committed at the intended boundary. A snapshot/restore strategy must account for every relevant state, not only the visible tensor.

27.9 Derive a checkpoint-placement toy before measuring a system

Consider a chain of L equal-size activation stages and blocks of at most k layers. An illustrative inventory retains $\lceil L/k \rceil$ boundary slots and at most $\min(k, L)$ slots while recomputing one block:

$$I(L, k) = \lceil L/k \rceil + \min(k, L).$$

Ignoring ceilings for $1 \le k \le L$, $I \approx L/k + k$, whose derivative vanishes at $k = \sqrt{L}$. This explains why neither one huge block nor tiny blocks necessarily minimizes the toy inventory.

```
def chain_checkpoint_inventory(layers, block):
    """Equal-size chain toy: boundary plus one recomputed block activation
     ↪  slots."""
    if type(layers) is not int or type(block) is not int or layers < 1 or block <
     ↪  1:
        raise ValueError("Positive integer chain and block lengths required")
    boundaries = math.ceil(layers / block)
    return {
        "layers": layers,
        "block": block,
        "boundary_slots": boundaries,
        "recompute_slots": min(block, layers),
        "toy_live_slots": boundaries + min(block, layers),
    }
```

Block	Boundaries	Recompute	Toy live slots
1	64	1	65
4	16	4	20
8	8	8	16
16	4	16	20
64	1	64	65

For $L = 64$, block size eight gives 16 toy slots; sizes four and sixteen both give 20. These slots are a declared upper inventory, not measured PyTorch saved tensors or device bytes. At $k = 1$ the toy reports 65 slots, so it must not be marketed as an automatic improvement over retaining 64 ordinary stage outputs.

Checkpoint tradeoffs have a substantial research history [60]. Real architectures have unequal tensor sizes, residual branches, recomputation dependencies, attention matrices, and fused kernels. Our square-root derivation does not determine their optimal placement or runtime speed.

27.10 Mixed precision needs an operator-level policy

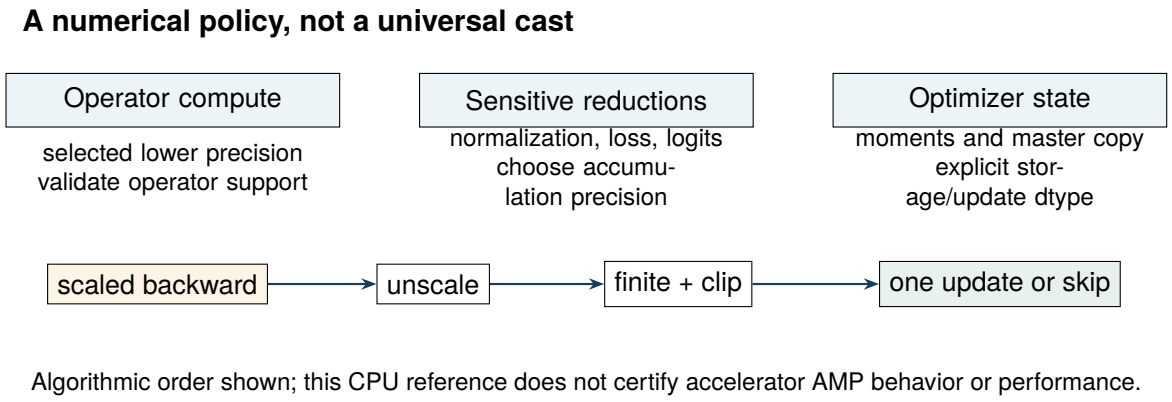


Figure 27.5: Low-precision compute, higher-precision reductions and optimizer state, scaling, and finite-gradient gates form a policy. Simply casting every tensor is not an equivalent training algorithm.

FP16 has a smaller exponent range than FP32; BF16 retains an FP32-sized exponent field but fewer significand bits. Loss scaling can help represent small gradients in FP16; it cannot fix an incorrectly normalized objective or recover information already lost in unstable arithmetic. Master weights and scaling are established mixed-precision techniques [61], not a guarantee for every optimizer/operator combination.

Sensitive reductions, normalization, attention logits, recurrence coefficients, and optimizer moments deserve explicit dtype choices and reference comparisons. Report finite-value incidence, skipped updates, loss/metric distributions across seeds, and restart equivalence. A successful single training run is weak evidence about numerical robustness.

27.11 Apply the contracts to both model families

DOGMA is the non-Transformer line. Its complete recurrent state, locus reads, gates, and writes must preserve Chapter 22's ordering. Sequence chunking carries state through time;

microbatching partitions examples. Detaching a carried state changes gradients and is truncated backpropagation, not activation checkpointing.

Hermon DNA is the Transformer line. Checkpointed blocks must preserve attention masks, positional conventions, and stochastic policy. Decode-time KV cache is an inference resource; ordinary training activation accounting should not quietly substitute it for the actual saved graph.

Chapter 26's affine scan applies only to the stated input-conditioned recurrence. Checkpointing that kernel does not establish that a state-dependent full DOGMA controller can be parallelized identically. Evolutor must record which path was selected and which state/numerical contract it actually implements.

27.12 A measurement protocol that can fail

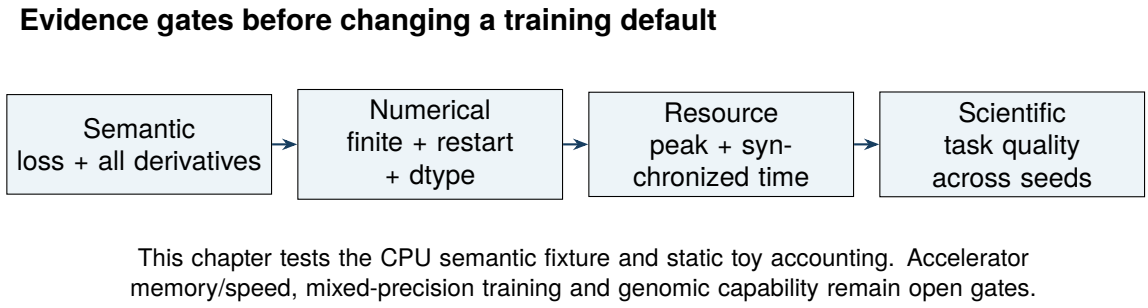


Figure 27.6: Semantic, numerical, resource, and task-quality gates are distinct. Passing a CPU derivative test does not report an accelerator memory saving or a genomic capability.

Specify workload shapes, valid-token counts, dtype policy, optimizer, software, hardware, and accumulation window. Warm up the complete step, including optimizer state initialization. Reset peak statistics at a declared point, include forward, backward, recomputation and update, and synchronize device execution around timing.

Distinguish live allocated bytes, allocator-reserved bytes, and external device usage. Report repeat measurements and workload-dependent failure boundaries. Selective retention, offload, and distributed sharding may move a peak rather than remove it. A bandwidth bottleneck or extra communication can outweigh the saved activation storage.

Key ideas

- Dividing each microbatch by its own mass and averaging changes the objective. In the fixture the gradient error is about 0.14.
- The optimizer clock advances once per complete update. Stepping after each microbatch is a different algorithm.
- Clipping is nonlinear: clipping 2 and -1.5 separately at one gives 0, but clipping their sum gives 0.5.
- Checkpointing without preserving the random state gives an identical forward loss and a wrong backward pass.

27.13 Worked problems and implementation exercises

1. Count the reference parameters. **Answer:** $128 + 16 + 48 + 3 = 195$.
2. Compute hypothetical FP32 Adam arrays. **Answer:** 3,120 bytes.
3. Compute all-Float64 arrays. **Answer:** 6,240 bytes, excluding overhead.
4. Why can two low-precision arrays still leave $16P$ bytes? **Answer:** moments and an FP32 master copy can dominate the inventory.
5. Derive the global valid-weight mass. **Answer:** sum all fixed nonnegative w_{ij}, including masks and importance weights.
6. Handle a zero-valid microbatch. **Answer:** its numerator contributes zero divided by positive global Z.
7. Why reject target broadcasting? **Answer:** it can silently change which output errors the objective includes.
8. Derive $\nabla_{W_2} L$. **Answer:** $D^\top H$, with $D = 2W \odot (P - Y)/Z$.
9. Scale all weights by seven. **Answer:** loss and gradient are unchanged.
10. Why is local-mean/K wrong for masses 12,13,6? **Answer:** it weights each microbatch equally rather than each weighted element.
11. When should Adam's clock advance? **Answer:** once per complete update.
12. What is missing from a gradient-only test? **Answer:** moment, bias-correction, and parameter-update correctness.
13. Clip 2 and -1.5 separately at one, then together. **Answer:** separate sum zero; summed-and-clipped value 0.5.
14. What precedes clipping under loss scaling? **Answer:** unscale and check the complete accumulated gradient.
15. Why can equal checkpoint forward losses hide a bug? **Answer:** backward recomputation can use a different random mask.
16. What does a shape/dtype/device check not prove? **Answer:** equal recomputed values or equal derivatives.
17. Minimize $L/k + k$ for $L = 64$. **Answer:** $k = 8$, under the toy assumptions.
18. Is 16 toy slots measured peak memory? **Answer:** no.
19. Functionalize a DOGMA write. **Rubric:** explicit incoming state, returned outgoing state, preserved read-before-write order, no duplicate external commit.
20. Design a deployment gate. **Rubric:** loss/gradient/moment/update parity, RNG and restart contracts, realistic dtype failures, synchronized peak/time measurement, repeated seeds, and explicit workload/hardware evidence.

27.14 Bridge to distributed ownership

Chapter 28 assigns arrays and reductions to ranks. Global normalization, optimizer-update boundaries, and stochastic-state provenance remain the same questions, but now communication and ownership can invalidate them. An ideal per-rank storage formula is only the beginning of that analysis.

Evidence and source boundaries

The weighted reduction, manual derivative and Adam oracles, CPU stochastic-replay counterexample, and equal-size-chain inventory are original teaching artifacts. The reference uses PyTorch 2.10.0 and Float64; it does not reproduce external hardware benchmarks or certify mixed-precision training. Paper inspection covers version-specific author abstracts and metadata, not full experimental sections. PyTorch inspection covers the 2.10 checkpoint API's selected execution, randomness and determinism notes. Current documentation for another version is not silently substituted for the tested runtime.

CHAPTER 28

Distributed training and reproducible model artifacts

In this chapter you will

- declare ownership of data, parameters, gradients and optimizer state before choosing a parallelism strategy
- derive one global weighted objective and its collective scaling
- shard arrays with explicit ranks, offsets and empty owners, and derive tensor-parallel operators
- pin, validate and reload a model artifact from raw bytes in a fresh process

Chapter 27 fixed the objective, optimizer clock, and stochastic replay policy before changing memory retention. Distribution adds another boundary: who owns each data element, parameter shard, gradient, and checkpoint byte? A computation can look plausible on every rank while optimizing the wrong global objective or producing an artifact that cannot be reconstructed.

We derive collective algebra locally, verify gradients independently, partition matrix operators, and load an inference fixture from raw bytes in a fresh process. These are executable correctness oracles, not an implementation or benchmark of DDP, NCCL, FSDP, ZeRO, or a safetensors parser. They establish invariants that an actual backend must preserve under its installed version and hardware.

28.1 Declare ownership before choosing a parallelism strategy

Data parallelism partitions examples while replicating model parameters. Tensor parallelism partitions an operator's operands or outputs. Pipeline parallelism partitions sequential layers and communicates intermediate activations. State sharding partitions persistent parameter, gradient, or optimizer arrays; it may coexist with any of the other strategies.

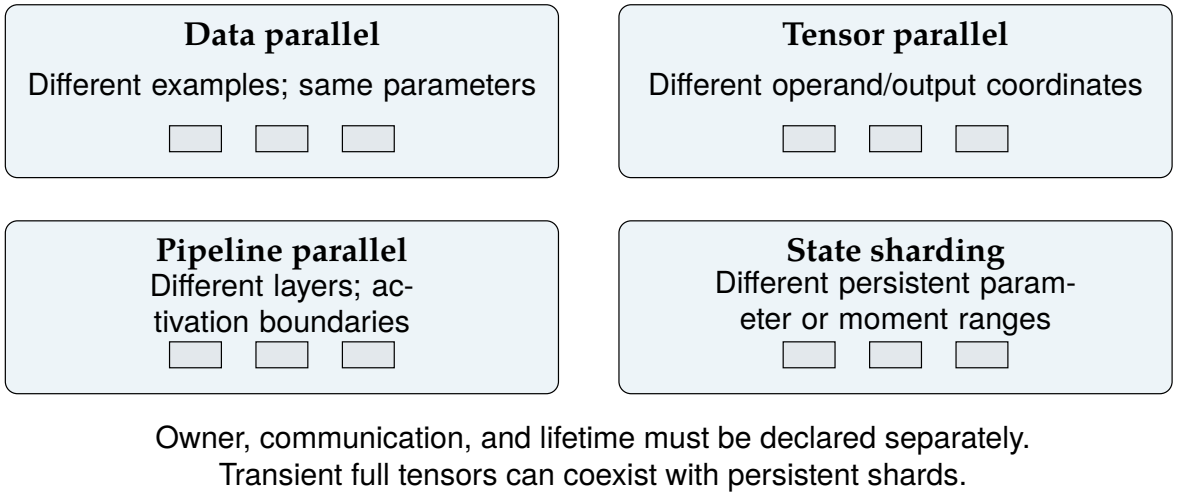


Figure 28.1: Four forms of partitioning have different owners and communication boundaries. A smaller persistent shard does not imply that all intermediate tensors remain sharded.

The primary DDP paper describes replicated training with gradient communication [63]. ZeRO partitions optimizer state, then gradients, then parameters cumulatively [64]. FSDP describes a practical parameter-sharded system [65]. These sources motivate the architecture categories. None of their published performance results is reproduced by this chapter.

An ownership table should include the data sample IDs, valid-token weights, parameter names and shapes, optimizer slots and update count, activation boundaries, and random-state ownership. A genomic record repeated on two ranks does not become two independent biological observations. Chapter 25's dependency closure and split provenance still apply after a distributed sampler is introduced.

28.2 One global weighted objective

Let rank r own a disjoint set of valid elements $\mathcal{B}_r$ and fixed nonnegative weights w_i. Define

$$N_r(\theta) = \sum_{i \in \mathcal{B}_r} w_i \ell_i(\theta), \qquad Z_r = \sum_{i \in \mathcal{B}_r} w_i, \quad Z = \sum_r Z_r > 0,$$

$$L(\theta) = \frac{\sum_r N_r(\theta)}{Z}, \qquad g = \frac{\sum_r \nabla_\theta N_r}{Z}.$$

The **rank mass** counts the declared statistical mass: valid tokens, records, or importance weight. Equal record counts do not imply equal mass when records have different lengths or masks.

For D ranks and a backend that *means* parameter gradients, each rank can backpropagate DN_r/Z. The subsequent mean is

$$\frac{1}{D} \sum_r \nabla_\theta (DN_r/Z) = g.$$

For a backend that sums gradients, the factor D is absent. Inspect the actual collective semantics rather than treating the word all-reduce as a normalization contract. Loss reporting similarly reduces numerators and masses separately.

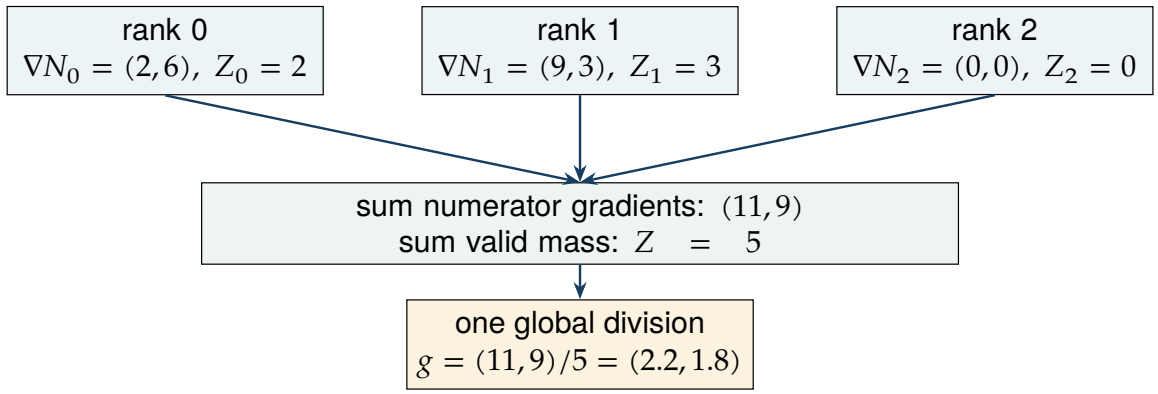


Figure 28.2: Each rank contributes a numerator gradient and a mass. Global normalization occurs once. A rank with zero valid mass contributes zero but still participates in the collective.

```
def weighted_reduce(numerators, masses):
    """Global numerator-gradient sum / global valid-element mass."""
    if len(numerators) != len(masses) or not numerators:
        raise ValueError("Matching nonempty rank axis required")
    masses = tuple(finite(v) for v in masses)
    if min(masses) < 0:
        raise ValueError("Negative rank mass")
    total = math.fsum(masses)
    if not math.isfinite(total) or total <= 0:
        raise ValueError("Finite positive global mass required")
    mean = allreduce_mean(numerators)
    return tuple(v * len(numerators) / total for v in mean)
```

The executable fixture has numerator gradients $(2,6)$, $(9,3)$, $(0,0)$ and masses $2,3,0$. It gives $(11,9)/5 = (2.2,1.8)$. Averaging rank-local means instead produces $(4/3,4/3)$ when the empty rank contributes zero. Excluding it and averaging the remaining means gives $(2,2)$, which is also wrong. A locally empty shard is not permission to omit a rank from a collective expected by its peers.

28.3 Test the derivative, not just vector addition

A second fixture uses a Float64 linear predictor $p_i = x_i^\top \theta$ and weighted squared error. Its independent scalar derivative is

$$g_j = \frac{2}{Z} \sum_i w_i (p_i - y_i) x_{ij}.$$

Tests compute the full loss with PyTorch autograd, separately differentiate rank-local numerators, apply the local collective oracle, and compare every parameter component with the scalar expression. One rank has zero valid mass. Additional tests permute ranks and multiply all weights by a common positive factor; neither operation should change the objective gradient.

These tests cannot detect deadlocks, missing process participation, network failure, or backend scaling conventions: no process group is launched. They deliberately isolate the mathematical contract. A production test must then execute the same fixture on real ranks and verify gradient, optimizer moment, and parameter-update parity against the single-process oracle.

Parameters must remain fixed throughout an accumulation window across ranks. Clipping, unscaling, skipped-update decisions, scheduler advancement, and optimizer updates occur at the declared global boundary. If one rank steps early or clips its local gradient independently, algebraic reduction does not restore the intended procedure. The counterexamples from Chapter 27 remain relevant.

28.4 Balanced shards require identities and offsets

For an array of length n over D ranks, write $n = qD + s$, $0 \le s < D$. Rank r receives

$$n_r = q + \mathbf{1}[r < s], \qquad o_r = rq + \min(r, s).$$

Its range is the half-open interval $[o_r, o_r + n_r)$. For $n = 10, D = 3$, extents are 4, 3, 3, with offsets 0, 4, 7. When $D > n$, some ranks own empty ranges; those descriptors are still required.

Rank	Offset	Extent
0	0	4
1	4	3
2	7	3

```
def reconstruct(parts):
    """Rank/offset coverage checked; never concatenate unlabeled rank files."""
    parts = tuple(parts)
    if not parts or not all(isinstance(s, Shard) for s in parts):
        raise ValueError("Typed shard descriptors required")
    world = positive_int(parts[0].world)
    total = parts[0].total
    if type(total) is not int or total < 0 or len(parts) != world:
        raise ValueError("Missing rank or invalid total")
    if any(type(s.rank) is not int for s in parts) or {s.rank for s in parts} !=
     ↪  set(
        range(world)
    ):
        raise ValueError("Duplicate or missing rank identities")
    out = []
    offset = 0
    for s in sorted(parts, key=lambda p: p.rank):
        expected = total // world + int(s.rank < total % world)
        if (
            type(s.world) is not int
            or type(s.total) is not int
            or s.world != world
            or s.total != total
            or type(s.start) is not int
            or s.start != offset
        ):
            raise ValueError("Incompatible shard metadata or coverage
             ↪  gap/overlap")
        if type(s.values) is not tuple or len(s.values) != expected:
            raise ValueError("Unexpected shard extent")
```

```
        out.extend(s.values)
        offset += expected
    if offset != total:
        raise ValueError("Incomplete global coverage")
    return tuple(out)
```

A shard descriptor stores rank, world size, offset, total length, and immutable values. Reconstruction validates the complete rank set, exact integer metadata, balanced extents, and continuous global coverage before concatenating in rank order. File discovery order is not rank order. A complete collection with one offset shifted by one is invalid even when its total byte count looks correct.

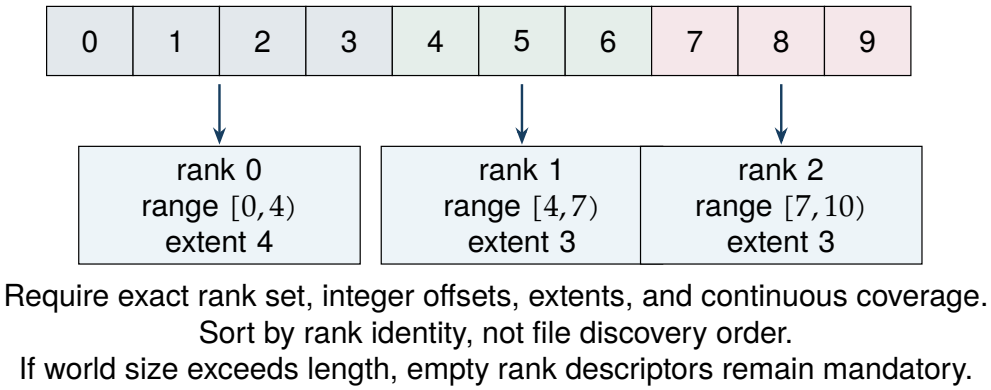


Figure 28.3: Uneven ranges reconstruct one logical tensor only when rank identity, offset, extent, and global contract agree. Empty owners are explicit, not silently missing files.

This is an intentionally simple flat-array protocol. A production checkpoint also needs tensor names, original shapes, dtypes, layouts, shard axes, mesh coordinates, and serialization version. Sharding a parameter does not specify how its optimizer moments, master copy, or recurrent inference carry are stored.

For DOGMA, a restart may need hidden, memory, and read/write carry as well as weights when continuing a stream. For Transformer-based Hermon DNA, a serving session may need cache position and cache tensors. A weight artifact alone is not a complete continuation checkpoint. Whether such transient state belongs in a portable model release or only a session snapshot is an explicit product and scientific contract.

28.5 Derive tensor-parallel operators from matrix algebra

Use $X \in \mathbb{R}^{B \times I}$ and $W \in \mathbb{R}^{I \times O}$. Partition the contraction axis into disjoint sets $\mathcal{I}_r$:

$$X = [X_0 \ \cdots \ X_{D-1}], \qquad W = \begin{bmatrix} W_0 \\ \vdots \\ W_{D-1} \end{bmatrix}, \qquad Y = XW = \sum_r X_r W_r.$$

Each partial output has shape $B \times O$; the required combination is a sum, not a mean. The reference names this contraction-axis split **row-parallel linear** because it partitions rows of W under the stated XW convention. Frameworks storing W transposed may name the same operation differently; dimensions are more reliable than names.

```
def row_parallel(x, w, world):
    """Split contraction axis; each rank produces a full-size partial output."""
    world = positive_int(world)
    parts = shard(tuple(range(len(w))), world)
    if not x or not w or any(len(row) != len(w) for row in x):
        raise ValueError("Matching nonempty contraction axis required")
    width = len(w[0])
    if width == 0 or any(len(row) != width for row in w):
        raise ValueError("Nonempty rectangular weight required")
    outputs = []
    for part in parts:
        outputs.append(
            tuple(
                tuple(
                    math.fsum(finite(row[k]) * finite(w[k][j]) for k in
                     ↪  part.values)
                    for j in range(width)
                )
                for row in x
            )
        )
    return tuple(
        tuple(math.fsum(out[i][j] for out in outputs) for j in range(width))
        for i in range(len(x))
    )
```

For $X = (1, 2, 3)$ and weight rows $(1, 2)$, $(3, 4)$, $(5, 6)$, the output is $(22, 28)$. A four-way split includes an empty owner and still returns the same result. Mistakenly averaging partial outputs gives $(5.5, 7)$.

For an output-axis split $W = [W_0 \ \cdots \ W_{D-1}]$, $Y = [XW_0 \ \cdots \ XW_{D-1}]$: combine by concatenation along the output axis. This is a different collective. A following elementwise nonlinearity may operate locally on output shards, but an operation that mixes that axis needs communication or a different decomposition. Softmax over a sharded vocabulary, for example, requires a global normalization with a numerically stable global maximum.

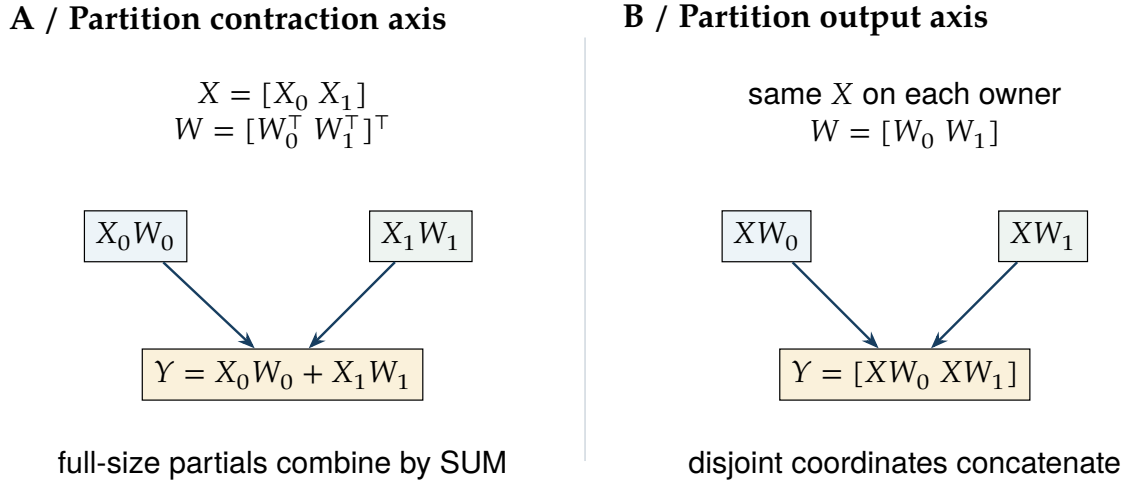


Figure 28.4: A contraction-axis partition combines full-size partial outputs by summation. An output-axis partition combines disjoint output coordinates by concatenation. Shapes reveal which operation is valid.

For contraction-axis splitting and upstream gradient G, $dW_r = X_r^\top G$ and $dX_r = GW_r^\top$. The backward ownership and communication must follow this derivation, not merely copy the forward collective. Bias is added once after the output sum unless its placement is otherwise proven equivalent.

28.6 Persistent memory and transient materialization

Let each parameter have stored weight width b_θ, gradient width b_g, and optimizer/master-state width b_s. A declared idealized inventory per rank is

$$M_{\text{rep}} = P(b_\theta + b_g + b_s), \quad M_{\text{state}} = P(b_\theta + b_g) + Pb_s/D,$$

$$M_{\text{state+grad}} = Pb_\theta + P(b_g + b_s)/D, \quad M_{\text{all}} = P(b_\theta + b_g + b_s)/D.$$

These formulas concern persistent ownership, not measured peak memory. Activations, all-gathered parameter groups, reduction buffers, padding, allocator reservation, and temporary optimizer tensors can dominate the gap. Communication overlap extends some lifetimes while hiding latency; it is not a free memory reduction.

For a simple ring all-reduce of a dense buffer of M bytes on D ranks, reduce-scatter and all-gather each communicate approximately $(D-1)M/D$ bytes sent per rank under the standard evenly partitioned model. This is a communication volume model, not a throughput prediction. Latency, topology, contention, kernel fusion, precision, and gradient readiness determine measured time.

In genomic workloads, compare runs with the same valid-token objective and dependency-aware data split. Larger global batch size, changed optimizer steps, or changed sequence truncation can change convergence even when the distributed operator arithmetic is correct. A speed figure without an equivalence contract can reward doing a different scientific experiment faster.

28.7 Model artifacts are byte-level contracts

Our demo format is explicitly named `evolutor-tensor-demo/1`. Each tensor records name, shape, F32/F64 dtype, exact byte extent, and SHA-256. The root records architecture, tokenizer identity, little-endian order, and contiguous row-major layout. For shape $(d_1, \ldots, d_k)$ and element width b,

$$n_{\text{bytes}} = b \prod_j d_j.$$

A scalar shape `()` has one element; a zero dimension gives an empty tensor. An unsupported dtype or a negative/fractional/Boolean dimension is rejected.

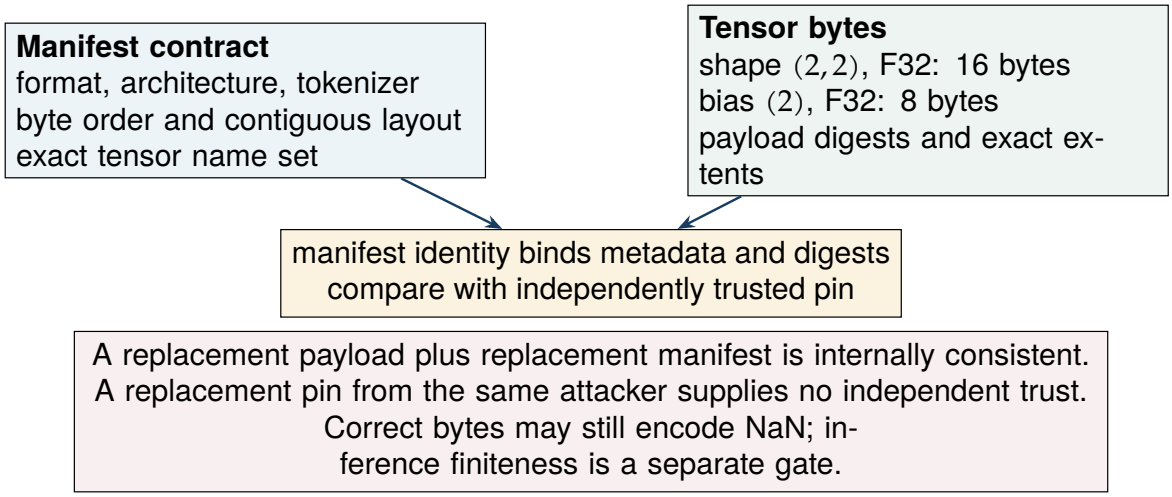


Figure 28.5: A tensor's numerical meaning depends on exact bytes plus shape, dtype, layout, architecture, and tokenizer contract. Hashes detect a changed artifact only relative to a trusted identity held outside that artifact.

The official safetensors specification describes a non-executable tensor format with JSON metadata and byte offsets [66]. Our demo is not that format and does not implement its header parser. It uses already decoded Python dictionaries and raw tensor blobs, then validates a fixed schema. Avoiding pickle removes executable object deserialization from this example, not every denial-of-service or provenance risk from a real loader.

```
def verify_artifact(manifest, blobs, architecture, tokenizer, specs, trusted_id):
    """Fail closed on content, schema, exact tensor set, config and pinned
     ↪ identity."""
    if not isinstance(manifest, dict) or set(manifest) != {
        "format",
        "byte_order",
        "layout",
        "architecture",
        "tokenizer",
        "tensors",
    }:
        raise ValueError("Exact manifest schema required")
    if (
        manifest["format"] != FORMAT
        or manifest["byte_order"] != "little"
        or manifest["layout"] != "contiguous-row-major"
    ):
        raise ValueError("Unsupported tensor encoding")
    if (
        canonical_json(manifest["architecture"]) != canonical_json(architecture)
        or manifest["tokenizer"] != tokenizer
    ):
        raise ValueError("Architecture/tokenizer contract mismatch")
    if (
        not isinstance(manifest["tensors"], dict)
        or set(manifest["tensors"]) != set(specs)
        or set(blobs) != set(specs)
    ):
        raise ValueError("Missing/unexpected tensor identity")
    if manifest_id(manifest) != trusted_id:
        raise ValueError("Manifest differs from independently pinned identity")
```

```
    for name, (shape, dtype) in specs.items():
        meta = manifest["tensors"][name]
        if not isinstance(meta, dict) or set(meta) != {
            "shape",
            "dtype",
            "nbytes",
            "sha256",
        }:
            raise ValueError("Exact tensor metadata schema required")
        size = tensor_extent(shape, dtype)
        if (
            type(meta["shape"]) is not list
            or meta["shape"] != list(shape)
            or meta["dtype"] != dtype
        ):
            raise ValueError("Expected tensor shape/dtype mismatch")
        if (
            any(type(d) is not int for d in meta["shape"])
            or type(meta["nbytes"]) is not int
            or meta["nbytes"] != size
        ):
            raise ValueError("Invalid tensor byte extent")
        if (
            type(blobs[name]) is not bytes
            or len(blobs[name]) != size
            or digest(blobs[name]) != meta["sha256"]
        ):
            raise ValueError("Corrupt tensor bytes")
    return True
```

Artifact integrity requires the exact expected tensor set in both manifest and payloads, the expected shape/dtype, exact payload length, content digests, architecture and tokenizer contract, and a manifest identity independently pinned by the caller. An extra tensor is rejected rather than ignored. The JSON identity uses the declared sorted, compact Python JSON profile; it is not an assertion of RFC 8785 cross-language canonicalization. Integer and floating configuration values encode differently and are compared through this profile.

28.8 Integrity is not authenticity or numerical validity

An attacker can replace tensor bytes and recompute their hashes in a replacement manifest. Without an independently trusted manifest identity, the altered artifact is internally consistent. Our coherent-forgery test demonstrates both cases: the old **trusted pin** rejects it, while a replacement pin supplied by the attacker accepts the altered bytes. Trust must come from an authenticated release channel, signature policy, or other independent provenance mechanism. A hash alone does not identify an author.

A tensor of the correct shape containing NaN can pass byte-integrity validation. The fixed inference loader separately rejects nonfinite weights and biases. A general model loader may need additional policies for dtype conversion, integer ranges, tensor contiguity, resource limits, and version compatibility. No single digest substitutes for those checks.

28.9 Reload in an independent process

```
def linear_reload(manifest, blobs, trusted_id, x):
    """Independent raw-byte inference loader for a fixed two-input/two-output
     ↪ fixture."""
    config = {"kind": "linear", "inputs": 2, "outputs": 2, "bias": True}
    specs = {"weight": ((2, 2), "F32"), "bias": ((2,), "F32")}
    verify_artifact(manifest, blobs, config, "nt-demo-v1", specs, trusted_id)
    if len(x) != 2:
        raise ValueError("Two features required")
    w = struct.unpack("<4f", blobs["weight"])
    b = struct.unpack("<2f", blobs["bias"])
    if not all(math.isfinite(v) for v in w + b):
        raise ValueError("Nonfinite inference tensor")
    return tuple(
        math.fsum((w[2 * i] * finite(x[0]), w[2 * i + 1] * finite(x[1]), b[i]))
        for i in range(2)
    )
```

The fixture stores a two-input, two-output linear layer in little-endian F32: weight rows $(1, 2)$, $(-1, 0.5)$ and bias $(0.25, -0.5)$. For input $(2, -1)$, independent raw-byte decoding gives logits $(0.25, -3)$. The test launches a fresh Python interpreter, passes manifest JSON and hexadecimal payloads, validates the pinned identity, decodes with `struct`, and performs scalar inference. No training object or optimizer state is reused.

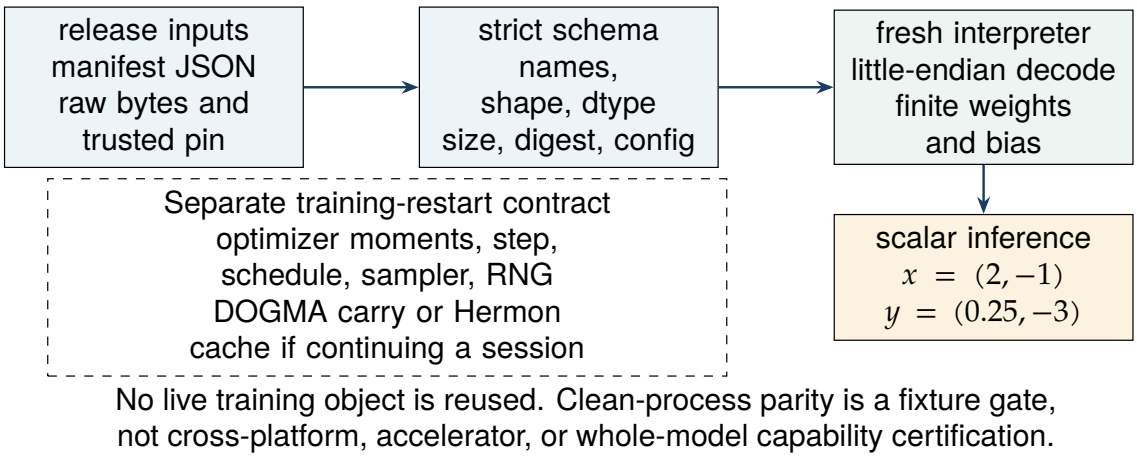


Figure 28.6: Separate artifact validation from numerical decoding and inference parity. A clean process catches dependencies hidden in a live training object, but does not by itself certify another platform or backend.

For a real DOGMA or Hermon DNA release, extend this gate to held-out tokenization, reference logits, carry/cache reset and continuation, precision tolerances, and supported hardware. For training restart, add optimizer moments, update count, schedule state, sampler progress, and random-state policy. Never label weight-only inference parity as full training-restart equivalence.

28.10 Boundary of the evidence

The 46 tests establish local weighted-gradient algebra, exact flat-shard coverage, matrix decomposition, strict demo byte validation, and clean-process inference for the declared fixtures. They include wrong normalization, missing ranks, offset shifts, malformed metadata,

corrupted bytes, coherent forgery, and nonfinite inference counterexamples. They do not measure distributed deadlocks, communication overlap, accelerator speed, or genomic predictive capability.

Chapter 29's benchmark design and comparative evidence should carry these equivalence gates into paired experiments. DOGMA remains the non-Transformer lineage; Hermon DNA remains the Transformer lineage. Comparing them requires controlled objectives, dependency-aware splits, and actual measurements, not an assumption that one ownership strategy establishes biological intelligence.

Key ideas

- Average rank numerators against the global mass. Averaging rank-local means, or dropping an empty rank, gives the wrong gradient.
- A contraction-axis split combines partial outputs by summation; an output-axis split combines them by concatenation. A replicated bias is added once.
- Persistent $1/D$ storage is not $1/D$ peak memory, because gathered parameters, activations and buffers remain.
- A hash detects change only relative to a trusted pin. A coherent forgery with a matching manifest passes any check that lacks one.

28.11 Worked exercises

1. Derive the global loss from rank numerators. *Solution:* $L = (\sum_r N_r)/(\sum_r Z_r)$, not the unweighted mean of N_r/Z_r.
2. Derive backward scaling for a mean collective. *Solution:* each rank uses D/Z times its numerator; the collective cancels D.
3. What if the backend sums? *Solution:* use $1/Z$ instead.
4. Compute the fixture gradient. *Solution:* $(2+9, 6+3)/5 = (2.2, 1.8)$.
5. Should a zero-mass rank skip a collective? *Solution:* no; it contributes a zero numerator and participates in the declared process schedule.
6. Explain why equal sequence counts are insufficient. *Solution:* unequal lengths or masks create unequal valid-token mass.
7. Partition length ten across three owners. *Solution:* sizes 4/3/3 and half-open intervals $[0,4), [4,7), [7,10)$.
8. Partition length one across four owners. *Solution:* one size-one shard and three explicit empty shards.
9. Why is file order not enough? *Solution:* discovery order does not prove rank identity or global coverage; metadata must reconstruct the intended order.
10. Compute the matrix fixture. *Solution:* $1(1,2) + 2(3,4) + 3(5,6) = (22, 28)$.
11. Falsify a mean of four partial outputs. *Solution:* it gives $(5.5, 7)$, scaling the operator by $1/4$.
12. Which collective combines output-axis shards? *Solution:* concatenation along output coordinates, not a sum.

13. Where should replicated bias be added? *Solution:* once after the contraction-axis sum; adding it on every owner multiplies it by D.

14. Derive the local weight gradient. *Solution:* for shared upstream G, $dW_r = X_r^\top G$.

15. Does persistent $1/D$ storage prove $1/D$ peak memory? *Solution:* no; materialized parameters, activations, and buffers remain.

16. Compute a $(2, 2)$ F32 payload length. *Solution:* $2 \cdot 2 \cdot 4 = 16$ bytes.

17. Interpret scalar and empty shapes. *Solution:* $()$ has one element; $(0, 3)$ has zero.

18. Explain a coherent hash forgery. *Solution:* replacing both payload and manifest preserves internal integrity; only an independent trust anchor rejects it.

19. Does a valid hash exclude NaN? *Solution:* no; finite inference is a separate policy checked after byte interpretation.

20. Specify restart evidence beyond logits. *Solution:* compare optimizer moments, update clock, schedule, sampler progress, stochastic state and next update under the same declared training procedure.

Primary materials inform the systems taxonomy and format boundary. The local algebra and demo loader do not reproduce their systems experiments.

CHAPTER 29

Benchmark design and comparative evidence

In this chapter you will

- freeze a falsifiable comparative claim before ranking models
- compute paired accuracy differences with an exact bootstrap oracle and a conditional sign test
- resample crossed seed and group dimensions, and recognize pseudoreplication
- gate resource budgets coordinate by coordinate and design ablation, transplant and sham interventions

A benchmark is a measurement instrument, not a verdict on an architecture. Before asking whether DOGMA outperforms HermonDNA, specify the behavior measured, the legal information available, the training opportunity and the population to which the result is meant to generalize. This chapter turns that specification into executable scoring, uncertainty calculations, resource controls and mechanism interventions.

Chapter 22 checked DOGMA's full carry and read-before-write semantics; Chapter 23 checked masked multi-head attention and gradients. Chapter 24 established span, reverse-complement, causal-availability and positional contracts. Chapter 25 audited dependency-closed splits, paired cluster estimates and architecture-native accounting. Chapter 26 derived a restricted affine scan, not a general replacement for full DOGMA. Chapters 27–28 added optimization, resource and artifact checks. These checks enable a benchmark; they do not rank trained genomic models. The fixtures below are synthetic supplied outcomes, not hidden training runs or industrial measurements.

29.1 Freeze a claim that can fail

Write the claim before selecting a winner:

$$C = (\mathcal{T}, \mathcal{D}, M, B, V, \delta_{\min}, H).$$

Here $\mathcal{T}$ is a task family; $\mathcal{D}$ fixes records, split/dependence units and legal access; M is the metric and estimand; B fixes resource coordinates and tuning opportunity; V names variation sources; $\delta_{\min}$ is a practically meaningful improvement; and H is the hypothesis and disconfirming criterion.

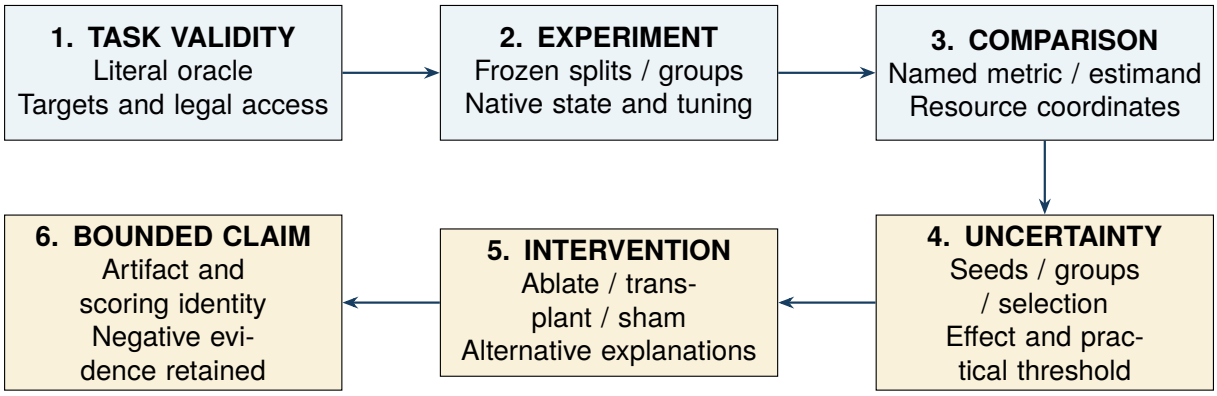


Figure 29.1: A comparative claim depends on task validity, legal access, resource opportunity and statistical design. A high terminal score cannot repair an earlier broken link.

A concrete protocol might ask: under a frozen donor-group split, a declared training-token ceiling and prespecified tuning budget, does a new mechanism improve equal-donor mean accuracy by at least $\delta_{\min}$? Changing to row-weighted accuracy changes the question if donors contribute unequal numbers of windows. Choosing the threshold after inspecting scores changes the test. Record amendments and distinguish exploratory from confirmatory results.

29.2 Same question, native execution

DOGMA is non-Transformer; HermonDNA is Transformer-based. Evolutor covers broader orchestration and research. Their comparison should share data, target, legal access and scoring interfaces without pretending that native carry and attention/cache are the same internal object.

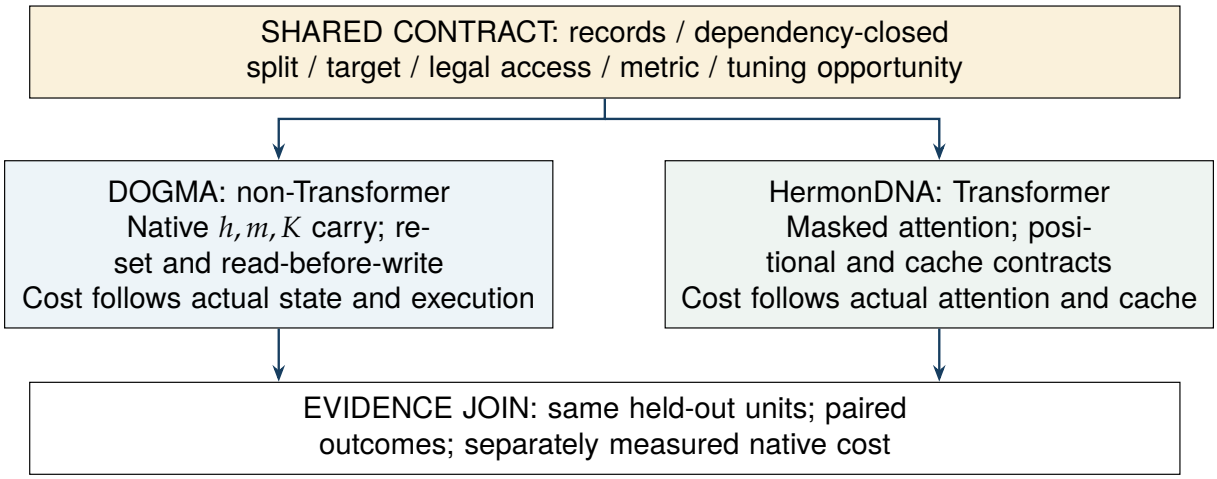


Figure 29.2: The shared envelope fixes evidence semantics; native state, reset and execution contracts remain distinct. Scores and costs are joined only after identity and legality checks.

An offline suffix-aware or reverse-complement representation may be legal for whole-record classification and illegal for causal next-token prediction. Neither a good score nor a compact implementation changes that boundary. For causal tasks, test target shifts, padding and the

availability of every feature at each position. For genomic generalization, audit exact duplicates, reverse complements, overlaps and shared biological sources before the split.

Begin with a literal oracle and a baseline ladder: majority/random, GC or length heuristic, simple learned model, established strong baseline, then proposed mechanism. A dataset solved by GC content has not established sophisticated sequence reasoning. Failed or null runs belong in the evidence ledger with configuration and artifact identity; do not invent earlier experiments or rewrite unsuccessful results as different tasks.

29.3 Make the estimand explicit

For fixed models on the same binary-label test cases, define

$$c_i^A = \mathbf{1}[\hat{y}_i^A = y_i], \quad c_i^B = \mathbf{1}[\hat{y}_i^B = y_i], \quad d_i = c_i^A - c_i^B.$$

Then $\hat{\Delta} = n^{-1} \sum_i d_i$ is the paired accuracy difference.

```
def paired_deltas(y, pred_a, pred_b):
    """Same held-out cases; correctness A minus B, not independent samples."""
    binary(y)
    binary(pred_a)
    binary(pred_b)
    if not len(y) == len(pred_a) == len(pred_b):
        raise ValueError("paired equal-length arrays required")
    return [int(a == pa) - int(a == pb) for a, pa, pb in zip(y, pred_a, pred_b)]
```

The strict binary schema rejects booleans and ambiguous label encodings. Multiclass, ranking, calibration and sequence likelihood require their own metric contracts; accuracy is not a universal proxy for biological usefulness. For masked likelihood, aggregate loss numerator and valid-token denominator, not an unweighted mean of differently sized batch means.

The eight-case fixture gives:

Case	A correct	B correct	d_i
1	1	1	0
2	1	0	1
3	1	0	1
4	1	1	0
5	1	1	0
6	0	1	-1
7	1	0	1
8	1	1	0

A is correct on seven cases and B on five, hence $\hat{\Delta} = .25$. There are three discordant A wins, one discordant B win and four ties. Pairing retains the common case difficulty. Treating the two accuracy estimates as independent discards that information.

29.4 An exact bootstrap oracle

For independent identically sampled cases and *fixed* trained models, the paired percentile bootstrap draws n differences with replacement. This **paired percentile interval** preserves

fixed-model case pairing. The empirical probabilities here are

$$P(D = 1) = 3/8, \quad P(D = 0) = 4/8, \quad P(D = -1) = 1/8.$$

Let $H_j(s)$ be the bootstrap sum distribution after j draws:

$$H_0(0) = 1, \quad H_{j+1}(s) = \tfrac{3}{8}H_j(s-1) + \tfrac{4}{8}H_j(s) + \tfrac{1}{8}H_j(s+1).$$

Exact rational convolution avoids Monte Carlo uncertainty for this tiny problem.

```
def exact_bootstrap(deltas, alpha=0.05):
    """Exact n-out-of-n IID paired percentile law via convolution."""
    deltas_ok(deltas)
    alpha_ok(alpha)
    n = len(deltas)
    if n > 100:
        raise ValueError("teaching convolution limited to n<=100")
    category = {v: Fraction(deltas.count(v), n) for v in set(deltas)}
    law = {0: Fraction(1)}
    for _ in range(n):
        nxt = {}
        for total, mass in law.items():
            for value, p in category.items():
                nxt[total + value] = nxt.get(total + value, Fraction(0)) + mass *
                 ↪  p
        law = nxt

    def quantile(q):
        cumulative = Fraction(0)
        for total, mass in sorted(law.items()):
            cumulative += mass
            if cumulative >= Fraction(str(q)):
                return total / n
        return max(law) / n

    return {
        "mean": sum(deltas) / n,
        "interval": [quantile(alpha / 2), quantile(1 - alpha / 2)],
        "law": [{"sum": s, "mass": float(p)} for s, p in sorted(law.items())],
    }
```

With the inverse-CDF quantile convention, the nominal 95% percentile interval is $[-.25, .75]$. The deterministic 4000-draw bootstrap with seed 29 gives the same endpoints. Independent Cartesian enumeration checks small fixtures, and an independent multinomial expansion checks this eight-case distribution.

The calculation is exact for the *empirical resampling law*; its nominal interval need not have exact frequentist coverage. A one-case sample $d = [1]$ gives a degenerate bootstrap interval $[1, 1]$, which plainly does not certify population certainty. Small samples, boundary effects and dependence demand caution.

For a different inferential question, an exact paired sign/McNemar-style calculation conditions on four discordant pairs and assumes exchangeable A/B outcomes under the null:

$$p = \min\left(1,\ 2\sum_{j=0}^{\min(3,1)}\binom{4}{j}2^{-4}\right) = .625.$$

This is not a proof that the methods are equivalent, nor an interchangeable interpretation of a bootstrap interval. Report assumptions, effect size, uncertainty and practical threshold rather than collapsing them to a star.

29.5 Seeds, groups and crossed dependence

Training outcomes vary with initialization, data order, augmentation, sampling, kernel nondeterminism and hyperparameter choices. Bouthillier et al. analyze these benchmark variance sources [67]. A shared integer seed does not guarantee that two different architectures consume comparable random streams. Pair shared splits and examples; document architecture-specific randomness.

Five supplied scores .71, .74, .69, .73, .72 have mean .718 and sample standard deviation .0192354. Report all five, not only the best .74. Their spread is training variation under the declared experiment, not the uncertainty of a new biological population.

Now define D_{sg} as an A-minus-B group metric for seed pair s and independent group g. Our teaching estimand gives equal weight to both axes:

$$\hat{\Delta} = \frac{1}{SG} \sum_{s=1}^{S} \sum_{g=1}^{G} D_{sg}.$$

It is not a row-weighted metric and not automatically Chapter 25's ratio of aggregated numerators/denominators. Unequal group sizes must not silently change it.

Seed fixture	G1	G2	G3	G4	Mean
Fixture 1	1	1	0	0	0.5
Fixture 2	0	0	-1	-1	-0.5
Fixture 3	1	1	0	0	0.5

The matrix is a hand-specified fixture, not scores from three neural training runs. Its mean is 1/6. Resample seed rows and group columns independently, sharing the same group draw across every chosen row:

```
def crossed_bootstrap(matrix, draws=4000, seed=29, alpha=0.05):
    """Equal seed/group estimand; resample both shared axes, not pooled cells."""
    matrix_ok(matrix)
    integer(draws, "draws", 1)
    integer(seed, "seed")
    alpha_ok(alpha)
    ns, ng = len(matrix), len(matrix[0])
    rng = random.Random(seed)
    means = []
    for _ in range(draws):
        rows = [rng.randrange(ns) for _ in range(ns)]
        cols = [rng.randrange(ng) for _ in range(ng)]
        # Shared group draw across seed rows preserves the crossed design.
        means.append(sum(matrix[s][g] for s in rows for g in cols) / (ns * ng))
    return {
        "mean": sum(map(sum, matrix)) / (ns * ng),
        "interval": [percentile(means, alpha / 2), percentile(means, 1 - alpha /
         ↪  2)],
        "draws": draws,
        "seed": seed,
```

```
        "estimand": "equal-seed equal-group mean",
    }
```

This **crossed bootstrap** retains shared group difficulty across seeds. It estimates uncertainty for the stipulated exchangeable seed/group design. It does not include hyperparameter search uncertainty, new-species shift, arbitrary interactions or a universal coverage guarantee.

For 4000 draws, seed 29, its interval is $[-.5, .75]$. Resampling seed means only gives $[-.5, .5]$; resampling group means only gives $[-1/3, 2/3]$. These are different variation models, not three interchangeable confidence levels.

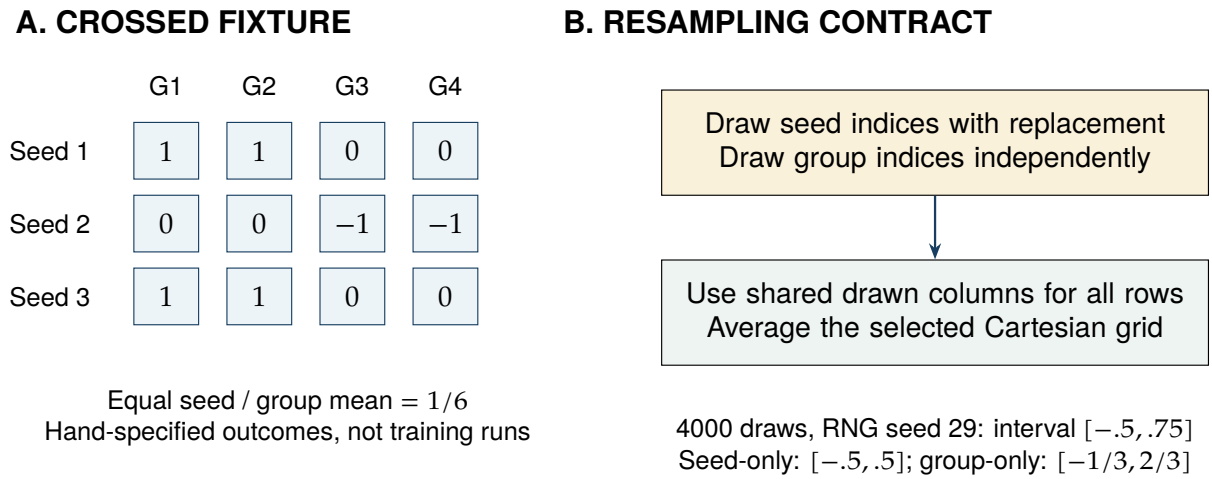


Figure 29.3: Rows represent training variation; columns represent independent biological groups. Shared column resampling preserves the crossed design. The figure shows actual fixture entries, not decorative random dots.

29.5.1 A pseudoreplication counterexample

If all three seed rows are $[1, 1, -1, -1]$, seed means are all zero and a seed-only interval is $[0, 0]$. Yet group uncertainty remains: the crossed interval is $[-1, 1]$ for this fixture. Conversely, identical group effects within each row can conceal training uncertainty from a group-only analysis.

Duplicating each of two independent effects $+1, -1$ one hundred times creates 200 rows but still only two independent groups. A false IID standard error falls below .071, while the two-unit standard error is 1. Added windows are not added independently assigned organisms. The **experimental unit** remains the independently assigned or sampled group under the declared design. Declare donor, preparation, lineage, chromosome or family boundaries according to the actual generalization claim.

29.6 The resource budget is a vector

Use a declared set of coordinates, for example

$$B = (P, N_{\text{tok}}, F, M_{\text{peak}}, M_{\text{state}}, W_{\text{infer}}).$$

Parameter count, training tokens, forward steps, peak bytes, persistent-state bytes and inference work are not convertible without an explicit model.

```
def within_budget(candidate, ceiling):
    """All controlled dimensions pass; missing measurements are not zeros."""
    if not isinstance(candidate, Budget) or not isinstance(ceiling, Budget):
        raise ValueError("complete measured budget vectors required")
    return all(
        getattr(candidate, k) <= getattr(ceiling, k)
        for k in Budget.__dataclass_fields__
    )
```

Our strict budget type requires nonnegative integer entries and all coordinates. Unknown peak memory is *unknown*, not zero. A candidate $(10, 10, 10, 10, 10, 11)$ fails a ceiling $(100, 100, 100, 100, 100, 10)$ despite passing the other five coordinates.

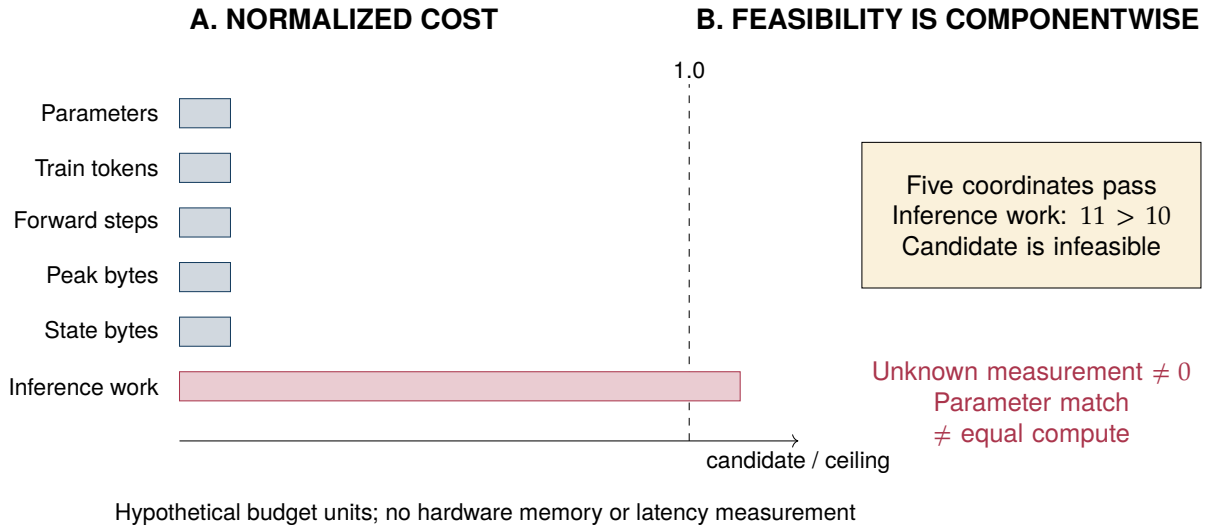


Figure 29.4: Each controlled coordinate has a separate gate. A parameter advantage cannot compensate for an undeclared inference-work violation. Hypothetical units are labeled; these values are not measurements.

Parameter-matched, compute-matched and latency-matched comparisons answer different questions. Equal forward-step counts can hide different sequence lengths or work per step. DOGMA carry and HermonDNA KV/attention costs must be accounted natively. Measured timing requires hardware, warmup, precision, batch/context lengths, synchronization, percentile reporting and failed-run treatment. Chapter 25's static arithmetic is not such a timing benchmark.

Our capacity matcher chooses the nearest parameter count with declared tie-breaking; it does not make every other coordinate fair. A **Pareto comparison** also requires score noninferiority and no worse controlled costs, with at least one strict improvement. Otherwise report a tradeoff rather than declaring a universal winner.

29.7 Tuning and task selection change conclusions

Giving one architecture 200 validation trials and another one default learning rate changes its opportunity. Freeze search spaces, trial counts or compute budget, selection metric and stopping rule. Retain failed configurations and consumed work. Dodge et al. emphasize validation/development reporting beyond test headlines [57].

Selection among many noisy results makes the selected maximum optimistic. For independent estimates with common CDF F, $P(\max_{j \le m} X_j \le x) = F(x)^m$. The selected maximum is not an unbiased estimate of a random configuration's performance. A reserved final test, evaluated after selection freezes, addresses a different role than the validation set; repeated test-set tuning invalidates that role.

Task selection can even reverse a ranking:

Task selection	A mean	B mean	Winner
Task 1 only	0.9	0.8	A
Both tasks, equal weight	0.7	0.8	B

A has .9, .5 across two tasks and B .8, .8. Task one alone favors A; an equal-weight suite favors B. Dehghani et al. discuss this benchmark-choice fragility [59]. Freeze task inclusion and aggregation weights. For multiple confirmatory endpoints, prespecify multiplicity treatment and distinguish exploratory discoveries. Do not report only tasks on which a proposed architecture wins.

29.8 Mechanism claims require interventions

An ablation compares a full source model with its mechanism disabled, while checking changed parameter count, optimization and execution cost. A transplant asks whether the mechanism helps an interface-compatible recipient. A matched sham checks whether extra capacity or work explains the gain.

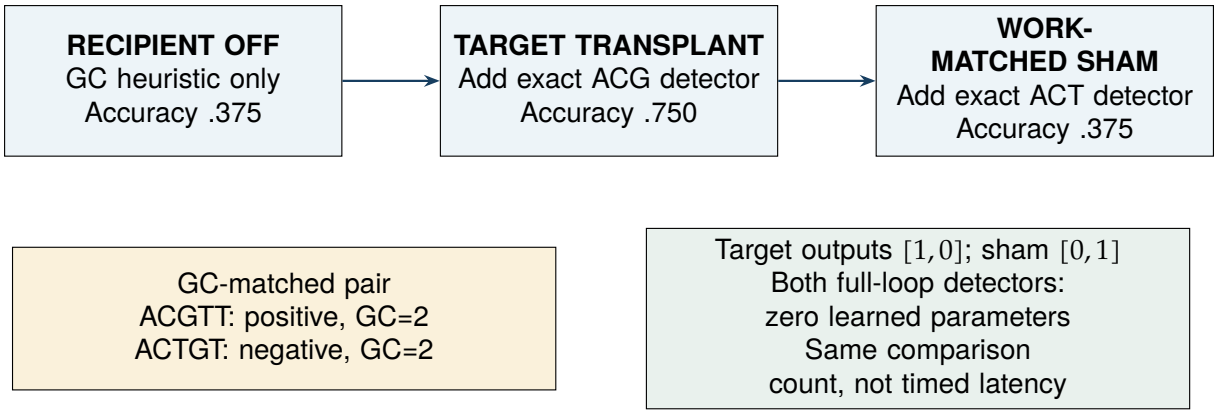


Figure 29.5: Target removal, recipient transplant and a work-matched sham are distinct interventions. A GC-matched positive/negative pair exposes whether the detector responds to the intended motif.

Our recipient is a deterministic GC-majority heuristic; the target is a literal ACG detector. The sham detects ACT using the same full window and character loops. Both detectors have zero learned parameters and the same static comparison count for a given length, not a claimed equal hardware latency.

```
def intervention(seq, mode):
    """No trainable parameters; source/recipient/sham toy Boolean program."""
    if mode not in ("off", "target", "sham"):
        raise ValueError("declared intervention mode required")
    base = base_model(seq)
    if mode == "off":
        return base
    return max(base, detector(seq, "ACG" if mode == "target" else "ACT"))
```

On the eight-string fixture:

Setting	Accuracy	Gain vs off
off	0.375	0
target	0.75	0.375
sham	0.375	0

The target improves accuracy from .375 to .75; the sham does not improve it. This is a Boolean-program intervention, not a trained DOGMA ablation. The target is supplied exact label logic, so the example demonstrates the experiment's causal structure, not learning ability.

The strings ACGTT and ACTGT both have GC count two. Target outputs are $[1, 0]$; sham outputs $[0, 1]$. This control defeats a simple GC explanation. But the OR-combined target still falsely accepts GCGCG: adding a correct positive detector does not remove the heuristic's false positives. Exhaustive length-five enumeration checks both detectors against independent substring membership.

A random-label sanity check must distinguish expected population chance from every finite empirical score; sufficiently expressive models can also memorize training labels. The executable negative control here simply inverts the matched pair's labels: target accuracy falls from one to zero. It is explicitly *not* a multi-seed shuffled-label training experiment.

29.9 Artifacts and claims need separate evidence

Chapter 28's byte-pinned artifact oracle supports reconstruction for its declared linear fixture. It is not a full DOGMA/HermonDNA deployment loader. For a real comparative result, record immutable model/config/tokenizer identities, dataset and split hashes, inference/reset options, all seeds, selection opportunity, software/hardware, scoring code and resource observations.

A benchmark card should permit recomputation of every numerator and denominator, describe group weighting and resampling, state exclusions and failed trials, and quote the exact bounded claim. A hash authenticates neither biological labels nor dataset independence; those need their own provenance and audits.

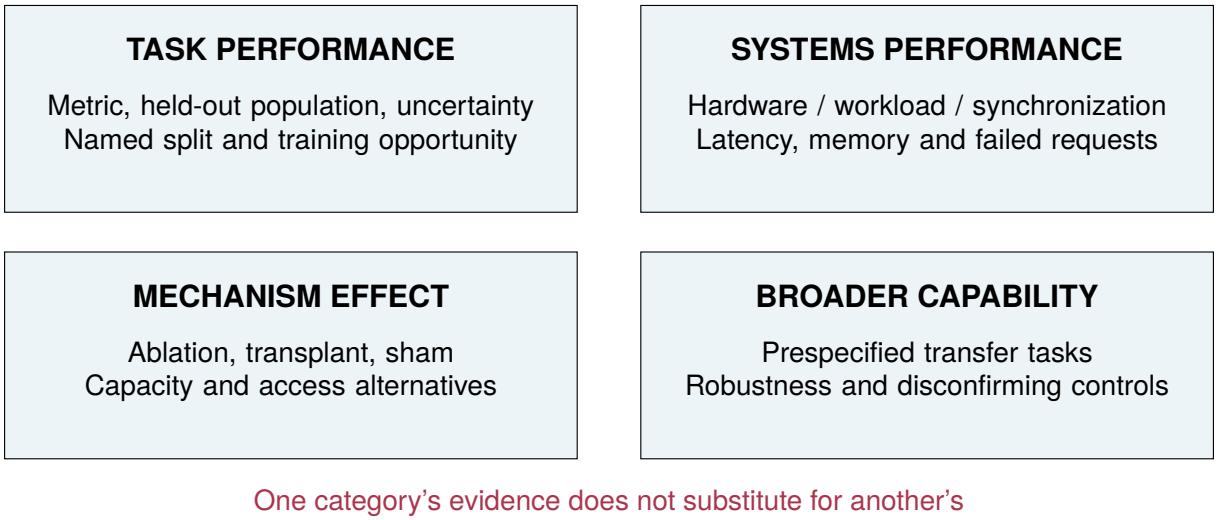


Figure 29.6: Task score, systems performance, mechanism effect and broad capability are separate claim categories. Extending a conclusion requires new tasks and controls, not stronger adjectives.

Tokens/s is a systems quantity, not genomic competence. One motif benchmark is not general genomic reasoning. A mechanism intervention in a hand-coded program is not proof of a novel learned architecture. Keep negative evidence, deployment limitations and uncertainty visible when moving from an experimental score to a professional claim.

Key ideas

- A benchmark is a measurement instrument, not a verdict on an architecture.
- Two hundred duplicated windows from two independent groups are still two experimental units. A naive i.i.d. standard error is fourteen times too small here.
- Task selection alone can reverse a ranking: A wins task one, while B wins the equal-weight two-task suite.
- A matched sham separates a mechanism's effect from extra work, and a GC-matched pair rules out a composition shortcut.

29.10 Twenty worked exercises

1. What is frozen before ranking? **Answer:** task, access, data/dependence, metric, budgets, variation and claim criterion.
2. Why keep native carry/cache distinct? **Answer:** their legal state and work semantics differ.
3. Does whole-record classification permit suffix features? **Answer:** possibly; a causal next-token task does not.
4. Paired fixture accuracy difference? **Answer:** $7/8 - 5/8 = .25$.
5. Discordant counts? **Answer:** three A-only wins and one B-only win.
6. Derive H_{j+1}. **Answer:** convolve H_j with empirical masses $3/8, 4/8, 1/8$ at $1, 0, -1$.
7. What is exact about the bootstrap oracle? **Answer:** its empirical resampling distribution, not nominal coverage.

8. Interpret $[-.25, .75]$. **Answer:** substantial uncertainty under fixed-model IID-case resampling; not equivalence or proof of A superiority.
9. Conditional sign-test p? **Answer:** $2(1+4)/16 = .625$.
10. Five-score mean and sample SD? **Answer:** .718 and .0192354.
11. Why not pick only .74? **Answer:** it selects the best trial and hides variation.
12. Crossed matrix estimand? **Answer:** equal-seed equal-group mean, here $1/6$.
13. Why share column draws across rows? **Answer:** models share the same biological-group difficulty.
14. Do 100 duplicate windows create 100 independent units? **Answer:** no.
15. May unmeasured peak bytes be zero? **Answer:** no; the budget is incomplete.
16. Why does the illustrative candidate fail its budget? **Answer:** inference work 11 exceeds ceiling 10.
17. Explain the lottery reversal. **Answer:** A wins task one but loses the frozen two-task mean.
18. What does the ACT sham control? **Answer:** equal static detector work without the target motif operation, not hardware latency.
19. Why is target accuracy not perfect? **Answer:** OR preserves GC heuristic false positives.
20. Design a publication gate. **Rubric:** independent oracle, leakage/access audits, matched opportunity, named estimand and units, variation/selection plan, unused final evaluation, intervention/sham, artifact identity, reproducible results and bounded conclusion.

29.11 Bridge to runtime contracts

Chapter 30 defines request identity, state ownership, streaming, cancellation, deadlines and terminal cleanup. Statistical rigor cannot rescue a runtime that changes model identity halfway through a request. Correct lifecycle and native step contracts precede any service-performance comparison.

PART VII

DOGMA Engine: state-native inference

How does a native recurrent state become a correct, owned and batched serving object?

The DOGMA Engine chapters wrap the restricted Chapter 31 recurrence in a serving lifecycle. The pieces are request identity and terminal arbitration, artifact-bound native state, generational state pools with in-flight pins, checkpoints and independent branches, and request-axis batching with fenced completion. Every optimization is stated as a refinement of a simpler oracle. The engine is a CPU reference: these chapters establish contracts and accounting, not measured throughput.

CHAPTERS IN THIS PART

30 **Runtime contracts and request lifecycles** Specify request ownership, terminal arbitration and output accounting.

31 **DOGMA state construction and execution** Bind actual native transitions to model identity and legal position.

32 **DOGMA state pools and memory ownership** Manage bounded reusable state with generation/version fences and pins.

33 **DOGMA checkpoints, branching and prefix state** Restore independent branches and cache bounded prefix snapshots.

34 **DOGMA batching and state-native kernels** Lift native transitions across requests without changing per-owner state.

CHAPTER 30

Runtime contracts and request lifecycles

In this chapter you will

- bind a request to tenant, pinned model identity, deadline and independent output ceilings before allocating state
- separate lifecycle phase, terminal reason and cleanup
- arbitrate one terminal winner and fence late results with generation-stamped leases
- reserve output capacity before launching work, and reconcile work and output counters

A model step finishes just as its client cancels. May the engine publish the result? May it reuse the step's state buffer for another tenant before the device acknowledges completion? What if the output stream still contains two committed tokens? These are semantic questions before they are performance questions.

Chapter 29 froze evidence and comparison contracts. This chapter constructs the serving shell: immutable request identity, admission, exclusive state leases, bounded output, commit fences, terminal arbitration and cleanup. The reference does not execute DOGMA or HermonDNA mathematics. It specifies the lifecycle that later native execution must preserve. A correct shell is not a trained genomic model, a network service or evidence of industrial throughput.

30.1 Bind a request before allocating state

Represent a request by

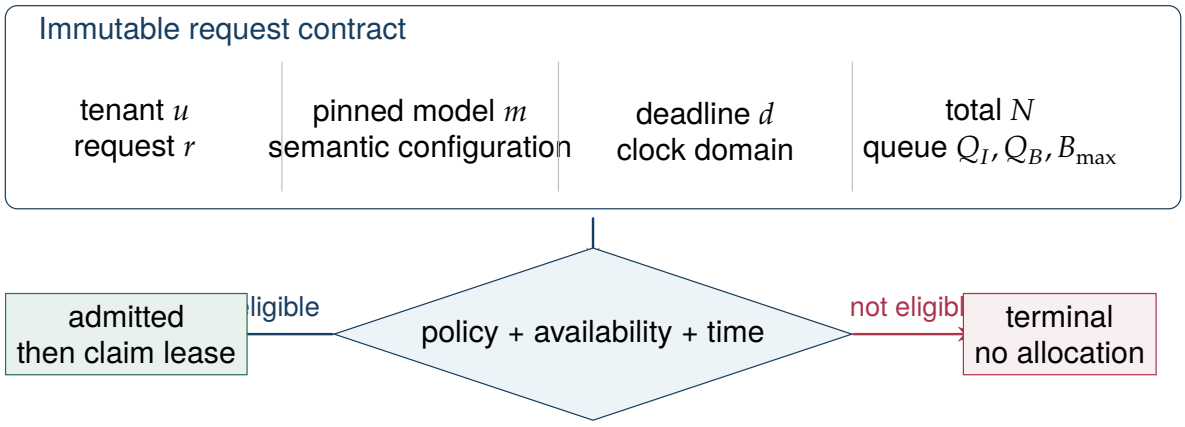
$$R = (r, u, m, d, N, Q_I, Q_B, B_{\max}).$$

r is request identity, u tenant/security scope, m a pinned model-artifact identity, and d a deadline in a declared monotonic clock domain. N bounds total committed output items; Q_I bounds queued items; Q_B bounds queued UTF-8 bytes; $B_{\max}$ bounds one item. These integer ceilings are independent. An empty queue does not mean the request has unused generation budget.

```
@dataclass(frozen=True)
class Request:
    request_id: str
    tenant_id: str
    model_id: str
    deadline_ms: int
    max_tokens: int = 3
    queue_items: int = 2
    queue_bytes: int = 8
    item_bytes: int = 4

    def __post_init__(self):
        identity(self.request_id, "request")
        identity(self.tenant_id, "tenant")
        if not isinstance(self.model_id, str) or not re.fullmatch(
            "[0-9a-f]{64}", self.model_id
        ):
            raise ValueError("pinned SHA-256 identity required")
        integer(self.deadline_ms, "deadline")
        for field in ("max_tokens", "queue_items", "queue_bytes", "item_bytes"):
            integer(getattr(self, field), field, 1)
        if self.item_bytes > self.queue_bytes:
            raise ValueError("one maximum item must fit")
```

The identity format is lowercase 64-character SHA-256. Chapter 28 checked pinned bytes for its linear artifact fixture; a production engine must extend that binding to the actual model, configuration, vocabulary, precision and semantic options. A correctly shaped string does not authenticate an artifact. The reference assumes a trusted binding supplied by its caller.



A syntactically valid SHA-256 name is not authentication. Fields are bindings, not causal arrows.

Figure 30.1: The envelope binds independent fields; arrows do not pretend that tenant identity causes model identity. Policy and availability checks precede state publication. Failed admission leaves no expensive state lease.

A hot deployment may choose a new identity for *new* requests. An existing request does not follow a mutable current-model pointer. Migrating its native state needs an explicit, verified conversion protocol. Matching tensor shape alone does not establish state compatibility.

30.2 Admission is a decision, not a scheduler

The oracle requires an explicit policy result and model-availability result. Before its deadline, an allowed request becomes admitted; a policy or availability failure becomes rejected. Deadline expiry terminates without allocation. Scheduling subsequently chooses execution order and placement among admitted work. Chapter 34 owns scheduling and batching.

Admission must account for resource reservation if concurrent requests compete. A check-then-allocate sequence without synchronized reservation can overbook a pool. Our registry's exclusivity contract prevents state-ID aliasing, not global GPU capacity exhaustion. There is no memory allocator, quota database, authentication service or scheduling fairness proof here. Unknown available capacity is not permission to treat it as infinite.

30.3 Terminal outcome and cleanup are separate coordinates

Use phases received, admitted, active and terminal. Streaming is a property of an active or terminal request's queue, not a phase that erases ownership. Keep a terminal reason and a separate cleaned flag:

$$S = (\phi, \rho, c, \ell, v, p, \mathcal{Q}, \mathcal{A}).$$

ϕ is phase, ρ terminal reason, c cleanup state, ℓ the lease, v the committed revision, p the pending ticket, $\mathcal{Q}$ the queue and $\mathcal{A}$ accounting counters.

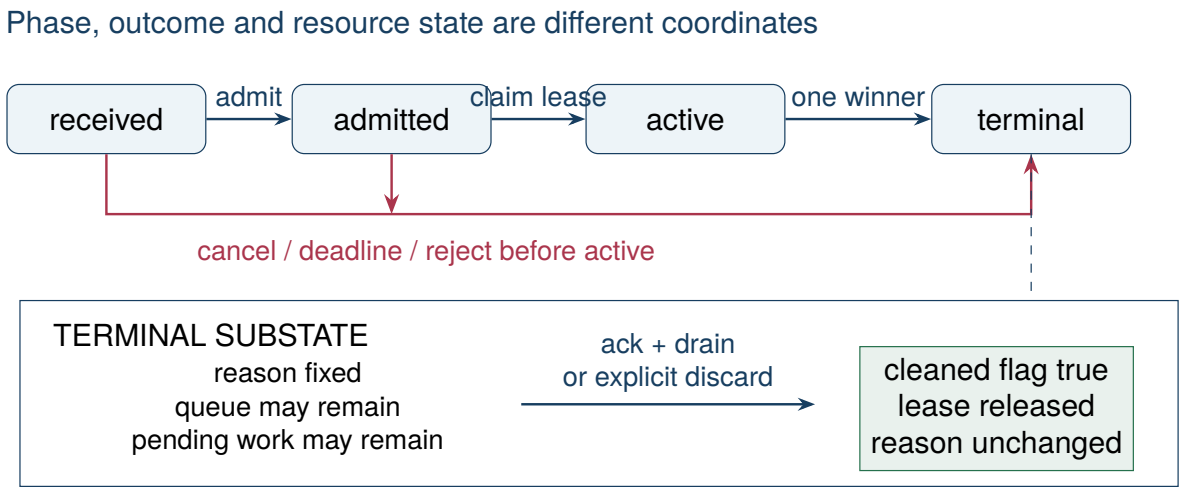


Figure 30.2: Lifecycle phase, immutable terminal reason and resource cleanup are separate. A completed request may still have queued output. A cancelled request may still have physical work awaiting acknowledgement.

Replacing completed by cleaned loses the outcome unless it is stored elsewhere. Similarly, reporting cancellation as complete because some bytes streamed hides a truncated operation. The reference retains exactly one reason from completed, cancelled, deadline, failed or rejected. Cleanup does not overwrite it.

30.4 One serialized terminal winner

All operations below are stipulated to execute serially. A terminal transition is the **linearization point** for outcome selection: before it, eligible work may commit; after it, no model result may create a new committed output or revision.

```
    def terminate(self, reason, now):
        """One serialized winner; terminal reason is not overwritten by
         ↪  cleanup."""
        self._clock(now)
        if reason not in ("completed", "cancelled", "deadline", "failed",
         ↪  "rejected"):
            raise ValueError("stable reason required")
        if self.phase == Phase.TERMINAL:
            return False
        if now >= self.req.deadline_ms:
            reason = "deadline"
        elif reason == "deadline":
            raise ValueError("deadline has not arrived")
        if reason == "completed" and (
            self.phase != Phase.ACTIVE or self.pending is not None
        ):
            raise RuntimeError("completion requires active quiescent state")
        self.phase, self.reason = Phase.TERMINAL, reason
        self._event("terminal", reason=reason)
        return True
```

Before the deadline, the first eligible terminal proposal wins. At or after the deadline, a newly observed terminal transition records deadline instead of pretending to complete on time. Already-terminal requests retain their earlier winner. Equality at d means expired, not one last allowed commit.

The six permutations of completion, cancellation and failure before expiry are tested against an independent first-winner rule. Four proposed reasons at exactly d all select deadline. Completion requires active state and no pending work; cancellation may terminate a received or admitted request too. This is an explicit arbitration policy, not a rule forced on every inference API.

For a concurrent implementation, serialize the *whole* check/update protocol or prove a suitable atomic transaction, including lease, deadline, revision, queue and counters. An atomic terminal flag alone cannot protect an unguarded output append. A test with twelve threads under one external lock checks terminal serialization only; the reference itself is not advertised as thread-safe.

30.5 Ownership survives physical placement changes

The registry maps each live state ID to

$$\ell = (s, g, (u, r, m)),$$

where g is a monotonically increasing allocation generation for that ID. A request owns its logical state; a worker is a residency label. The registry rejects even a second claim by the same owner while the ID is live. A shared prefix or fork is not created accidentally by repeated claim.

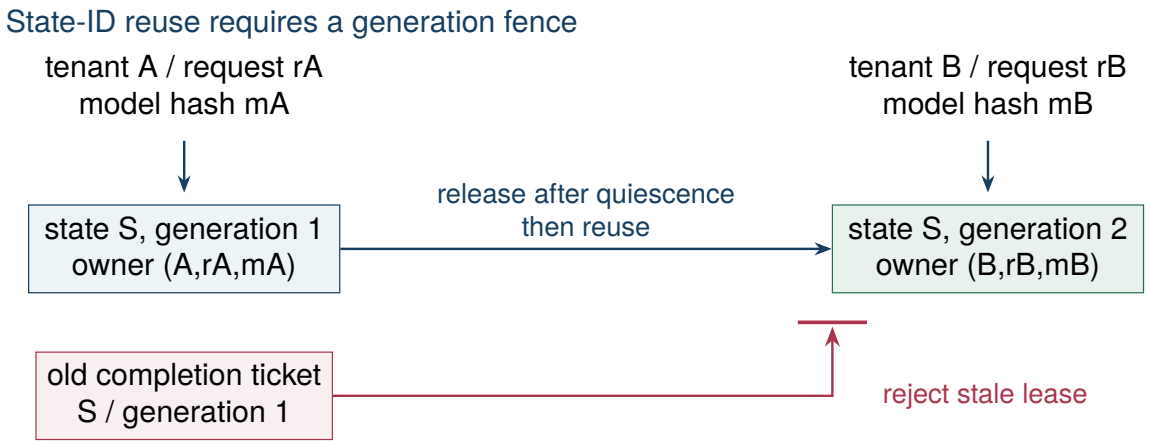


Figure 30.3: The same physical state ID is released and later receives a new generation. A completion carrying the old generation must not write into the new tenant's allocation. Worker placement is separate from logical ownership.

Allocation claims the registry entry before publishing active local state. An ownership conflict leaves the request admitted and unallocated. A foreign release fails. After release and reuse, the old lease also fails even if the new allocation has the same request/model names. This counters the **ABA reuse** problem: the string S may look unchanged while the resource it denotes has changed.

Generations persist only in this in-memory oracle. Across a crash or worker restart, a production design needs globally unique allocation epochs or a durable generation scheme and fencing at the executor. These tokens are correctness identities, not cryptographic capabilities. The registry assumes trusted code; it cannot prevent an attacker mutating process memory.

30.6 Backpressure before model work

To avoid silently running a step whose result has nowhere to go, reserve a worst-case output slot before launch:

$$|Q| < Q_I, \qquad \text{bytes}(Q) + B_{\max} \le Q_B.$$

At most one ticket is pending for this request. This simple policy is conservative: a two-byte item in a five-byte queue leaves three bytes free, but another item with declared maximum four bytes cannot launch.

```
    def launch(self, now):
        """Reserve worst-case output capacity before starting one logical step."""
        self._require(Phase.ACTIVE)
        if self.check_deadline(now):
            return None
        self.registry.validate(self.lease)
        if self.pending is not None:
            raise RuntimeError("one in-flight step per request")
        queued_bytes = sum(len(v) for _, v in self.queue)
        if (
            len(self.queue) >= self.req.queue_items
            or queued_bytes + self.req.item_bytes > self.req.queue_bytes
        ):
```

```
            self._event("backpressure", queued=len(self.queue))
            return None
        self.launched += 1
        self.pending = Ticket(self.lease, self.revision, self.launched)
        self._event("launch", launch=self.launched)
        return self.pending
```

The blocked path logs backpressure but creates no ticket and increments no launched work counter. A producer cannot repeatedly advance hidden state or consume sampling RNG while calling those attempts blocked output. A native engine must make state and RNG commit part of the same operation or preserve a pending snapshot.

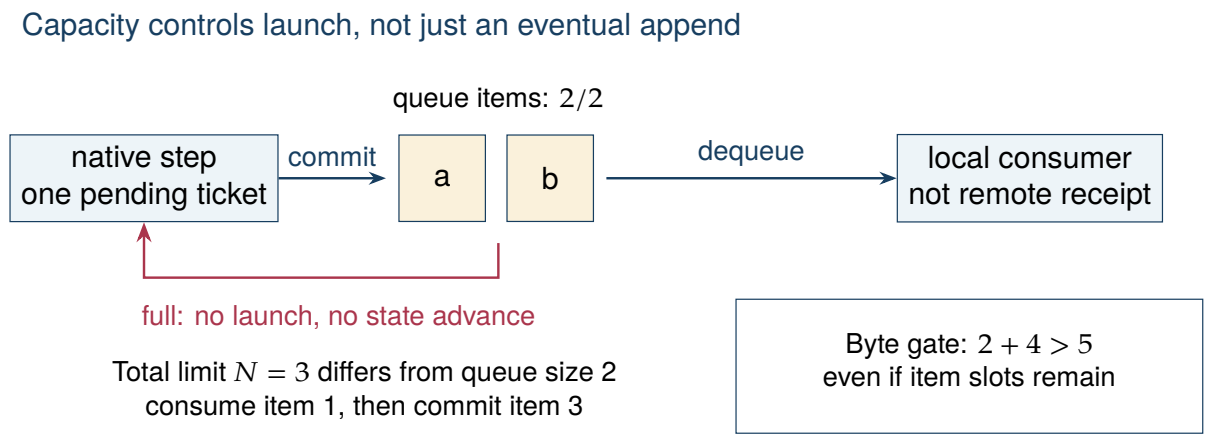


Figure 30.4: Item and byte ceilings both constrain launch. Total committed output is a third limit. A slow consumer holds only bounded buffered bytes; the backpressure path does not perform another model step.

UTF-8 character count is not byte count. The item *é* encodes to two bytes; a supplementary-plane emoji commonly needs four. The reference stores immutable encoded bytes and validates each produced item's maximum. It does not claim to bound the original Python string's transient allocation, native state, event log, allocator overhead or all process memory. The trace itself is unbounded here; a production system needs retention, sampling or bounded/durable export policies.

30.7 Fence a late result, then commit atomically

A launch ticket binds the exact lease, current revision and unique launch number:

$$p = (\ell, v, j_{\text{launch}}).$$

A result must match the pending ticket and a still-live lease. Duplicate, foreign and stale tickets cannot settle a different step.

```
    def commit(self, ticket, item, now):
        """Fence completion, then commit state revision and output together."""
        self._clock(now)
        if ticket != self.pending or self.pending is None:
            raise RuntimeError("foreign, stale or duplicate completion")
        self.registry.validate(ticket.lease)
        self.check_deadline(now)
        self.pending = None
        self.settled += 1
        if self.phase == Phase.TERMINAL:
            self.discarded_work += 1
            self._event("discard_work")
            return False
        if not isinstance(item, str):
            self.discarded_work += 1
            self.terminate("failed", now)
            return False
        try:
            payload = item.encode("utf-8")
        except UnicodeEncodeError:
            payload = b""
        if not payload or len(payload) > self.req.item_bytes:
            self.discarded_work += 1
            self.terminate("failed", now)
            return False
        self.revision += 1
        self.generated += 1
        self.queue.append((self.generated, payload))
        self._event("commit", item=self.generated, bytes=len(payload))
        if self.generated == self.req.max_tokens:
            self.terminate("completed", now)
        return True
```

After deadline/control checks, a successful result advances revision and output count together and appends one ordered byte payload. A result after a terminal winner settles the physical-work ticket but increments discarded-work accounting; it does not advance state or queue. Invalid type, empty output, malformed Unicode or oversized output terminates as failed with no new revision. The oracle does not model a fault during event append or crash halfway through the Python mutation sequence; production atomicity must cover such failures separately.

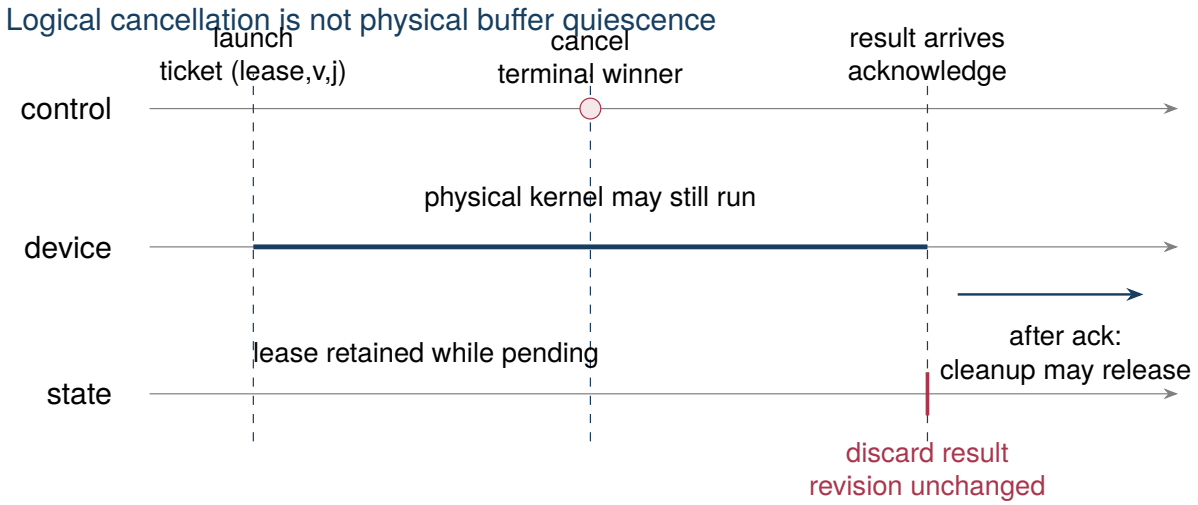


Figure 30.5: A launched step can finish physically after cancellation. Its acknowledgement settles the ticket, but the output/state commit is fenced. The lease cannot be released while physical work may still access its buffer.

Cancellation is therefore not instantaneous device preemption. If a kernel still holds a pointer, logically cancelling the request does not make that memory safe to reassign. A timeout waiting for acknowledgement is an unresolved resource, not a license to reuse the buffer. Device synchronization, abort guarantees or a quarantined allocation are required.

30.8 Deadlines use one explicit clock domain

Times in the reference are exact nonnegative integer milliseconds, passed by tests. Time never decreases. All admission, allocation, launch, commit, consumption and control paths observe the deadline where relevant. A step launched before d but returning at d cannot commit a timely result.

Production intervals should use a monotonic clock, not civil time that may jump. Monotonic epochs on different hosts are not automatically comparable. Transmit a deadline using a documented synchronization or remaining-budget protocol, include transit/error bounds, and check the receiving clock domain. A local service cannot guarantee a remote client's receipt time merely by meeting its own commit deadline.

HTTP/3 distinguishes rejecting an unprocessed request from cancelling one whose application processing has begun [68]. That protocol distinction motivates careful retry semantics; it does not define this model's state machine.

30.9 Draining is not success, and cleanup is not delivery

Our chosen queue policy keeps previously committed bytes readable after every terminal reason until cleanup. No new model outputs can commit. A client must receive the terminal reason separately to distinguish partial output from success. A different API may discard on cancellation, but it must declare and count that loss.

For the three-output fixture, the first two commits fill the queue; a launch is blocked. Consuming item one allows item three to commit and automatically reach the total output ceiling. The completed request still has two buffered items.

Sequence	UTF-8 item
1	a
2	b
3	c

Sequence numbers increase one through three regardless of consume timing. A local dequeue is not proof of remote receipt, application processing or exactly-once delivery. Transport reconnect and acknowledgement need their own protocol.

```
    def cleanup(self, now, discard_queue=False):
        """Physical quiescence precedes lease release; queued-output loss is
         ↪  explicit."""
        self._clock(now)
        if type(discard_queue) is not bool:
            raise ValueError("explicit queue disposition")
        if self.cleaned:
            return False
        if self.phase != Phase.TERMINAL or self.pending is not None:
            raise RuntimeError("terminal and quiescent required")
        if self.queue and not discard_queue:
            raise RuntimeError("drain or explicitly discard committed output")
        if self.lease is not None:
            self.registry.release(self.lease)
        self.discarded += len(self.queue)
        self.queue.clear()
        self.worker = self.lease = None
        self.cleaned = True
        self._event("cleaned", discarded=self.discarded)
        return True
```

Cleanup requires terminal phase and no pending physical work. It also requires an empty queue or an explicit discard decision. The registry releases the lease before local handles are cleared. A repeated successful cleanup returns a no-op. If release fails, local handles are retained so the failure remains diagnosable. This is logical idempotence in one live process, not a crash-resilient finalizer.

30.10 Reconcile every counter with an invariant

Let L, S, P be launched, settled and pending work, with $P \in \{0, 1\}$. Let G, Q, D, X be committed outputs, queued, locally dequeued and discarded outputs. Then

$$L = S + P, \qquad G = Q + D + X, \qquad v = G, \qquad G \le N.$$

Discarded *work* is distinct from discarded *output*. A cancelled in-flight step can be settled and discarded without ever creating a committed output; a committed but undelivered item can be discarded during cleanup.

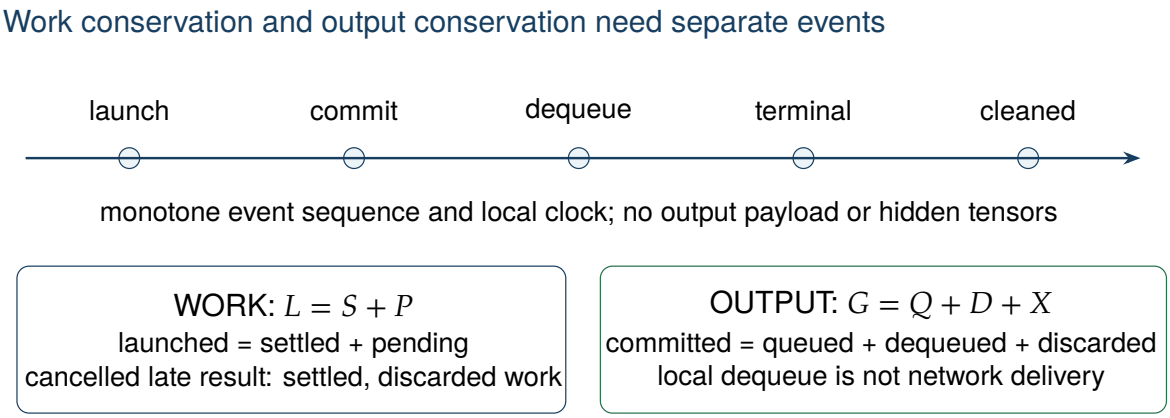


Figure 30.6: The trace records launches, commits, local dequeues, terminal selection and cleanup as distinct events. Separate work and output conservation identities expose a missing acknowledgement or an unreported queue loss.

The normal fixture finishes with $L = S = G = D = v = 3$, $P = Q = X = 0$. The cancelled pending fixture finishes with $L = S = 1$, discarded work one, $G = v = 0$. Counters measure declared logical events, not FLOPs, device time, billable tokens or a general resource cost. Billing and scheduler reservations require additional accounting contracts.

30.11 Independent interleavings before load tests

The test alphabet is launch, commit, consume, cancel, complete and cleanup. Enumerate all $6^5 = 7776$ length-five sequences from an allocated request. For each prefix, compare accepted/no-op/rejected operations and final state with a separate small specification that does not call the runtime's methods. Check queue bounds, one terminal event, lease release and both conservation identities after every operation.

This is bounded exhaustive verification for a stipulated serialized oracle. It is not exploration of every distributed execution, weak-memory ordering or device race. Extend the alphabet with worker crashes, loss/reorder, scheduler reservation changes, model-step failure and durable recovery before claiming production correctness. Chapter 26's restricted affine-scan checks do not supply complete DOGMA/HermonDNA engine parity; those model-specific checks remain distinct.

The trace contains immutable event records with monotone sequence numbers and times, but its list is in memory. It omits output payloads and hidden activations. It is neither an authenticated audit log nor a replay-complete durable journal. A production design must reconcile transactional publication and trace persistence, including crash windows, privacy, retention and recovery.

30.12 Retries and execution authority are additional contracts

A generation request is not automatically idempotent because it uses HTTP. RFC 9110 distinguishes method-level idempotence and unsafe automatic retries [69]. A client retry might reconnect to one retained operation or create a new one. Durable tenant-scoped idempotency keys must bind normalized request content and retention policy; a cleaned request ID alone is insufficient. If the same key accompanies different content, reject rather than silently reuse.

An inference shell also does not authorize actions. For an agent integration, model output proposes; a policy/kernel authorizes; a deterministic service executes; a durable trace records. Tool execution needs separate capabilities, revocation, side-effect idempotence and recovery. Neither a token nor a runtime trace creates execution authority or proves intelligence.

Key ideas

- Completion, cancellation and failure race; the serialized terminal transition is the linearization point. At the deadline itself, the deadline wins.
- Cancelling is not preemption. A buffer still referenced by in-flight work cannot be reused until its completion is acknowledged.
- Generation numbers defeat ABA reuse: a stale lease must not write into a new tenant's allocation that has the same slot name.
- $L = S + P$ and $G = Q + D + X$ are invariants. A missing acknowledgement or a silent queue loss breaks one of them visibly.

30.13 Twenty worked exercises

1. Why pin m? **Answer:** avoid mid-request model drift; identity is not authentication.
2. Is $Q_I = N$ required? **Answer:** no; queue occupancy and total output are different ceilings.
3. Why check availability before allocation? **Answer:** rejected work should obtain no native-state lease.
4. Does admission schedule the request? **Answer:** no; it decides eligibility.
5. Can completed imply cleaned? **Answer:** no; buffered output and physical resources may remain.
6. Where is the terminal linearization point? **Answer:** the serialized transition that fixes phase and reason.
7. Completion proposed exactly at d? **Answer:** deadline wins if the request was not already terminal.
8. Cancellation after successful completion? **Answer:** no reason rewrite or resurrection.
9. What does a generation number fix? **Answer:** stale leases after state-ID reuse, including same-name ABA.
10. Why isn't it a security capability? **Answer:** trusted callers and process integrity are assumed.
11. Queue bytes two, ceiling five, item maximum four: launch? **Answer:** no; worst-case reservation would require six bytes.
12. How many bytes is é in UTF-8? **Answer:** two; one character does not mean one byte.
13. A pending ticket after cancellation: release its buffer? **Answer:** no; acknowledge physical quiescence first.
14. A late result's state revision? **Answer:** unchanged; settled/discarded work increases instead.

15. Why retain committed bytes after cancellation? **Answer:** it is this API's explicit drain policy, not universal semantics.
16. Is local dequeue remote delivery? **Answer:** no; transport/application acknowledgement is separate.
17. Derive output conservation. **Answer:** each committed item is queued, dequeued or explicitly discarded, so $G = Q + D + X$.
18. What does 6^5 verification establish? **Answer:** bounded serialized-sequence agreement, not all concurrent schedules.
19. Is the event list durable? **Answer:** no; storage/recovery/privacy contracts remain unimplemented.
20. Design a serving release gate. **Rubric:** native-step parity, pinned identity, tenant ownership, synchronized commit/terminal protocol, clock domain, queue and resource bounds, physical quiescence, fault injection, durable trace/retry protocol, isolation, measured workload and bounded performance claims.

30.14 Bridge to native state

Chapter 31 constructs and advances model-native state inside this shell. A ticket's committed revision must correspond to an actual legal prefix and a verified state transition; it cannot merely increment a counter. Full DOGMA carry and HermonDNA KV semantics remain separate, and Chapter 21's suffix-aware offline representation must not enter causal left-to-right generation.

CHAPTER 31

DOGMA state construction and execution

In this chapter you will

- bind native state to tenant, request, content-addressed artifact and position
- derive and hand-check an input-gated diagonal recurrence step
- ingest masked chunks with full preflight, and verify a closed form and its derivative independently
- stage a decode step so that cancellation cannot advance native state

31.1 Make the native state executable

Chapter 30 supplied a serialized request lifecycle: admission, ownership, capacity reservation, completion fencing and cleanup. It did not specify how a native model consumes a token. This chapter fills that narrow gap with a fully auditable recurrence, then attaches its state changes to the shell.

Our implementation is an **input-gated diagonal recurrence**, not a complete DOGMA model. DOGMA denotes the non-Transformer research direction; HermonDNA denotes the Transformer branch; Evolutor is the broader orchestration and research system. The toy here establishes arithmetic and interface properties, not training quality, genomic capability, architectural novelty or production performance. Its gate depends on current input, not previous hidden state. This restriction is precisely why an independent affine closed form is possible.

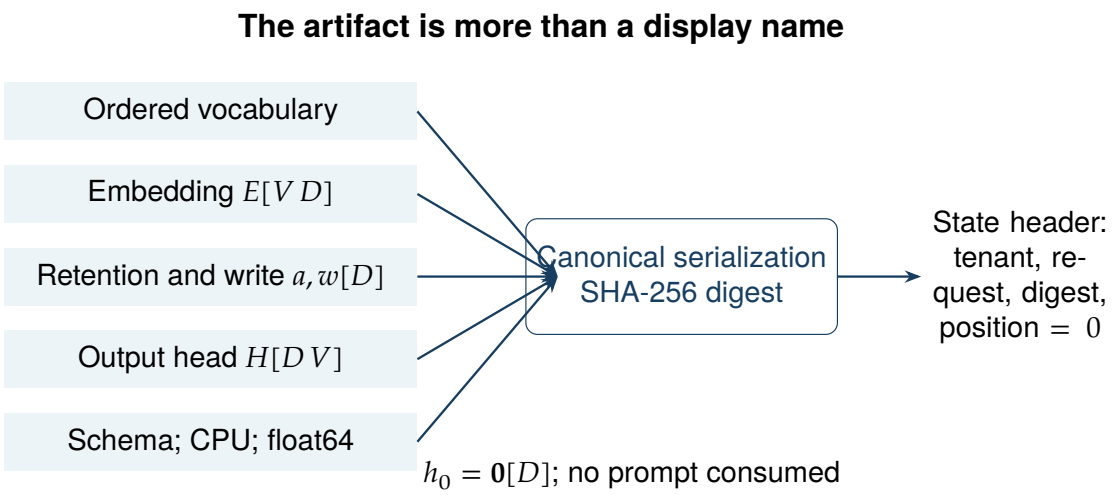
The reader will construct a content-bound state, ingest a masked prompt without emitting output, calculate logits by hand, verify derivatives independently, and stage a decode step that cannot advance native state after cancellation. All tables and literal code listings come from `drafts/ch31/reference.py`.

31.2 Declare the artifact, not only its name

The artifact contains an ordered vocabulary of size V, embedding $E \in \mathbb{R}^{V \times D}$, retention $a \in [0, 1)^D$, write parameter $w \in \mathbb{R}^D$ and output head $H \in \mathbb{R}^{D \times V}$. The reference requires finite CPU float64 tensors and an explicit schema version. Reordering vocabulary strings while leaving the matrix unchanged changes the meaning of every token ID.

A **content-bound artifact** is identified by canonical serialization of schema, dtype, device, vocabulary and all parameter values. The SHA-256 digest is recomputed at execution boundaries in this teaching oracle. It detects changes, including in-place tensor edits, rather than trusting a human-readable model label. It does not authenticate the publisher, prove that a checkpoint was trained, or certify that a hidden state was reached by the advertised prefix.

Production systems should validate immutable loaded artifacts once and use protected handles. Hashing all parameters on every token is intentionally expensive here and is not a proposed high-throughput serving design.



Execution rechecks parameter content; a frozen dataclass does not make tensor storage immutable.

Figure 31.1: Artifact construction binds data representation and token semantics. A frozen Python record does not freeze the storage of its tensors.

```
def artifact_id(model):
    if not isinstance(model, Model):
        raise ValueError("model required")
    vocab = model.vocabulary
    if (
        not isinstance(vocab, tuple)
        or len(vocab) < 2
        or len(set(vocab)) != len(vocab)
        or any(not isinstance(v, str) or not v for v in vocab)
    ):
        raise ValueError("ordered unique vocabulary required")
    if model.embedding.ndim != 2:
        raise ValueError("embedding rank")
    v, d = model.embedding.shape
    if v != len(vocab) or d < 1:
        raise ValueError("embedding vocabulary/width")
    tensor(model.embedding, (v, d))
    tensor(model.retention, (d,))
    tensor(model.write, (d,))
    tensor(model.head, (d, v))
    if not ((model.retention >= 0) & (model.retention < 1)).all():
        raise ValueError("0 <= retention < 1")
    payload = {
        "schema": "evod31-input-gated-diagonal-v1",
        "dtype": "float64",
        "device": "cpu",
        "vocabulary": vocab,
        "parameters": [
            p.detach().tolist()
            for p in (model.embedding, model.retention, model.write, model.head)
        ],
    }
    return hashlib.sha256(
        json.dumps(
            payload, sort_keys=True, separators=(",", ":"), allow_nan=False
        ).encode()
    ).hexdigest()
```

31.3 Construct and validate the state

The state header is

$$S = (\text{tenant}, \text{request}, \text{artifact digest}, p, h), \qquad p \in \mathbb{N}_0, \quad h \in \mathbb{R}^D.$$

Construction initializes $p = 0$ and $h = \mathbf{0}$ without consuming a prompt. Tenant and request form an exact ownership pair. A valid header, correctly shaped finite hidden vector and matching artifact digest are checked on the actual native-step path, including an empty ingestion call.

This is stronger than an optional assertion that callers may forget to invoke. It is weaker than authentication: trusted Python code can construct or alter objects. State reachability, signed checkpoint provenance and authorization are separate obligations. The reference also has no GPU/device-transfer policy, mixed precision, batch allocator or persisted checkpoint format.

31.4 Derive one native step

For a token ID $x_t \in \{0, \dots, V-1\}$,

$$e_t = E[x_t] \in \mathbb{R}^D, \tag{31.1}$$
$$q_t = \sigma(w \odot e_t) \in (0,1)^D, \tag{31.2}$$
$$b_t = q_t \odot e_t, \tag{31.3}$$
$$h_t = a \odot h_{t-1} + b_t, \tag{31.4}$$
$$z_t = h_t H \in \mathbb{R}^V. \tag{31.5}$$

$\odot$ denotes elementwise multiplication. The gate weights the current embedding's write, not the historical state. Signed embeddings still produce signed writes; a sigmoid gate does not turn h into probabilities.

The state position counts consumed live tokens, not predicted tokens or delivered bytes. Each accepted native step produces a new tensor and increments p once. No input tensor is updated in place. A nonfinite hidden vector or logit vector is rejected even when all parameters were initially finite.

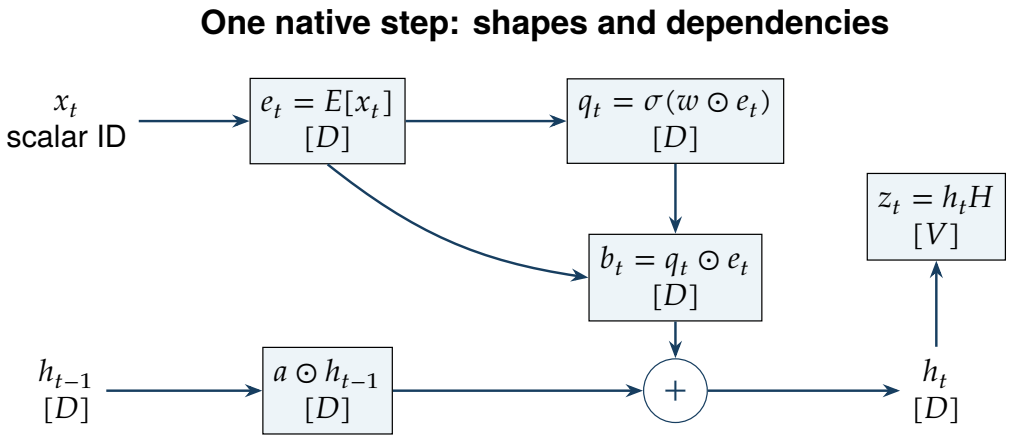


Figure 31.2: Shape-annotated native dataflow. Retention transports history; the input-only gate contributes a new write. There is no attention matrix or KV cache.

```
def native_step(model, state, token, owner):
    validate(model, state, owner)
    if type(token) is not int or not 0 <= token < len(model.vocabulary):
        raise ValueError("integer token in vocabulary")
    e = model.embedding[token]
    hidden = model.retention * state.hidden + torch.sigmoid(model.write * e) * e
    logits = hidden @ model.head
    tensor(hidden, tuple(state.hidden.shape))
    tensor(logits, (len(model.vocabulary),))
    return Result(State(owner, state.artifact, state.position + 1, hidden),
     ↪  logits)
```

31.4.1 Bound state magnitude without promising useful memory

If $|b_{t,j}| \le M_j$, then repeated substitution yields

$$|h_{t,j}| \le a_j^t |h_{0,j}| + M_j \sum_{k=0}^{t-1} a_j^k = a_j^t |h_{0,j}| + M_j \frac{1 - a_j^t}{1 - a_j}.$$

For this finite vocabulary, $M_j = \max_v |E_{v,j}|$ is sufficient because $0 < q_{t,j} < 1$. The bound can become large when a_j approaches one. It bounds magnitude, not the semantic quality or recoverability of remembered information. Fixed D also does not imply constant execution cost: the dense output projection costs $O(DV)$ per live token, and this oracle's digest checks read the complete parameter artifact.

31.5 Ingest chunks and masks

The ingestion function validates all live token IDs before entering the loop. A malformed tail therefore cannot mutate the caller's input state. It also requires a Boolean mask with exactly one flag per token row. A false row performs no embedding lookup, holds the hidden state and position, and emits an all-zero row vector by convention. That vector is a placeholder, not a probability distribution or a next-token prediction.

Masking is not request reset. A new request requires fresh construction and a new owner; silently resetting at padding would corrupt continuation semantics. An empty chunk retains the state but still checks owner and artifact.

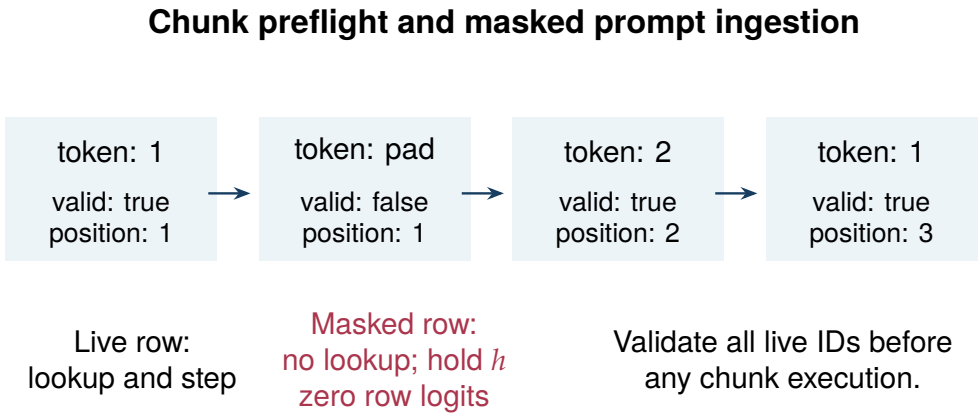


Figure 31.3: Masked chunk example. Padding neither consumes a token nor advances position. The full chunk is preflighted before live-row execution.

```
def ingest(model, state, tokens, owner, valid=None):
    """Preflight complete chunk; padding rows do not consume tokens or advance
     ↪  position."""
    validate(model, state, owner)
    tokens = tuple(tokens)
    valid = (True,) * len(tokens) if valid is None else tuple(valid)
    if (
        len(tokens) > 10000
        or len(tokens) != len(valid)
        or any(type(v) is not bool for v in valid)
    ):
        raise ValueError("bounded chunk with boolean mask")
    for token, live in zip(tokens, valid):
        if live and (type(token) is not int or not 0 <= token <
         ↪  len(model.vocabulary)):
            raise ValueError("invalid live token")
    outputs = []
    for token, live in zip(tokens, valid):
        if live:
            result = native_step(model, state, token, owner)
            state = result.state
            outputs.append(result.logits)
        else:
            outputs.append(torch.zeros(len(model.vocabulary),
             ↪  dtype=torch.float64))
    return state, tuple(outputs)
```

31.6 Verify exact arithmetic before optimizing

Choose vocabulary strings $(\texttt{0}, \texttt{1}, \texttt{2})$ and

$$E = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad a = (1/2, 1/4), \quad w = (0, 0), \quad H = \begin{pmatrix} 0 & 1 & -1 \\ 0 & -1 & 1 \end{pmatrix}.$$

Every gate equals $1/2$. Starting from zero, the prefix $(1, 2, 1)$ gives:

Position	Token	h_1	h_2	z_1	z_2
1	1	0.5	0	0.5	-0.5
2	2	0.25	0.5	-0.25	0.25
3	1	0.625	0.125	0.5	-0.5

For example, the second state is $(\frac{1}{2} \cdot \frac{1}{2}, \frac{1}{4} \cdot 0 + \frac{1}{2}) = (\frac{1}{4}, \frac{1}{2})$. The third is $(\frac{5}{8}, \frac{1}{8})$, and its logits are $(0, \frac{1}{2}, -\frac{1}{2})$. These rational values provide an independent oracle stronger than comparing two calls to the same implementation.

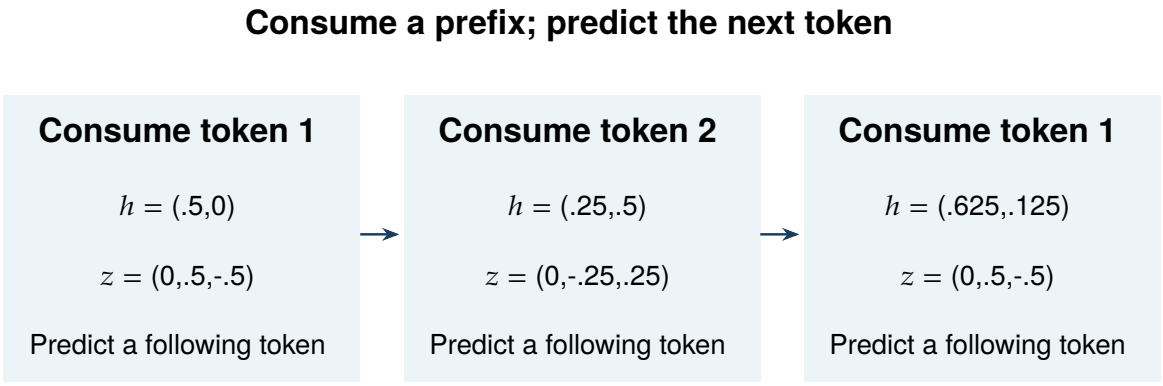


Figure 31.4: An exact untrained fixture. Consuming a prefix and predicting a following token are distinct operations. Values are computed arithmetic, not a genomic benchmark.

31.7 A closed form and ordered chunk composition

Since b_t depends only on the token,

$$h_{T,j} = a_j^T h_{0,j} + \sum_{i=1}^{T} a_j^{T-i} \sigma(w_j E_{x_i,j}) E_{x_i,j}. \tag{31.6}$$

The scalar implementation expands this sum using Python arithmetic. It never calls `native_step`; therefore it can detect indexing, retention and head-projection mistakes in the tensor path.

```
def scalar_closed_form(model, tokens, initial=None):
    """Independent scalar expansion, not repeated native_step calls."""
    artifact_id(model)
    e = model.embedding.detach().tolist()
    a, w, head = (
        p.detach().tolist() for p in (model.retention, model.write, model.head)
    )
    initial = [0.0] * len(a) if initial is None else initial
    t = len(tokens)
    h = []
    import math

    for j in range(len(a)):
        value = a[j] ** t * initial[j]
```

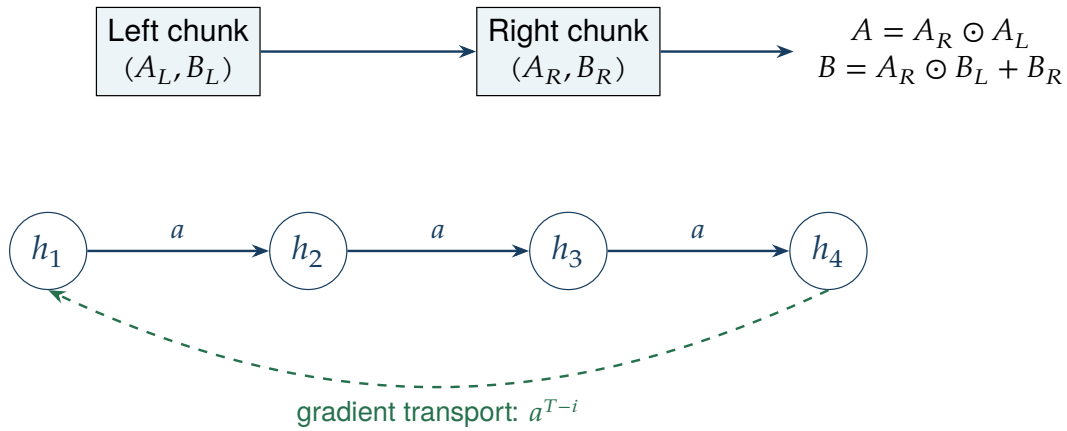
```
        for k, token in enumerate(tokens):
            x = w[j] * e[token][j]
            gate = 1 / (1 + math.exp(-x)) if x >= 0 else math.exp(x) / (1 +
             ↪  math.exp(x))
            value += a[j] ** (t - 1 - k) * gate * e[token][j]
        h.append(value)
    return h, [
        sum(h[j] * head[j][k] for j in range(len(a))) for k in range(len(head[0]))
    ]
```

A chunk represents $h \mapsto A \odot h + B$. Applying left chunk then right chunk gives

$$(A_R, B_R) \circ (A_L, B_L) = (A_R \odot A_L,\ A_R \odot B_L + B_R).$$

Composition is associative because these are affine functions. It is generally not commutative: changing token order changes the accumulated writes. This recovers the restricted affine algebra of Chapter 26, not proof that a full state-dependent DOGMA implementation supports the same scan.

Chunk composition preserves order, not arbitrary commutativity



Copying a tensor preserves an autograd edge; detaching removes it and may still alias its storage.

Figure 31.5: Ordered affine chunk composition and gradient transport. Forward parity, derivative parity and storage independence are separate properties.

31.8 Check derivatives independently

For a scalar final-state component,

$$\frac{\partial h_{T,j}}{\partial w_j} = \sum_{i=1}^{T} a_j^{T-i} \sigma(w_j E_{x_i,j})(1 - \sigma(w_j E_{x_i,j})) E_{x_i,j}^2.$$

At $w = 0$ for prefix $(1, 2, 1)$, this gives $\partial(h_{T,1} + h_{T,2})/\partial w = (\frac{1}{4}(1 + \frac{1}{4}), \frac{1}{4} \cdot \frac{1}{4}) = (.3125, .0625)$. The tests compare autograd with both this derivation and central finite differences at multiple write values.

For the initial state, $\partial h_{T,j}/\partial h_{0,j} = a_j^T$. A small a_j can erase gradients even though forward values remain bounded. Near-one retention trades slower forgetting for a larger magnitude envelope. Neither observation says that the model has learned useful long-range dependencies.

31.8.1 Copying, detaching and aliasing are not synonyms

PyTorch distinguishes a differentiable copy from detachment [70, 71]. A clone preserves the graph connection. Detachment removes that connection but shares underlying storage. Consequently, replacing a chunk boundary with `detach()` can preserve forward outputs while breaking training parity, and may still permit aliasing. A detached, storage-independent inference snapshot requires an explicit copy; a training continuation must retain the appropriate graph instead.

The chunk tests split the prefix at every boundary and compare the derivative against the analytical result. This does not verify gradients of a larger learned architecture or its optimizer.

31.9 Define next-token semantics

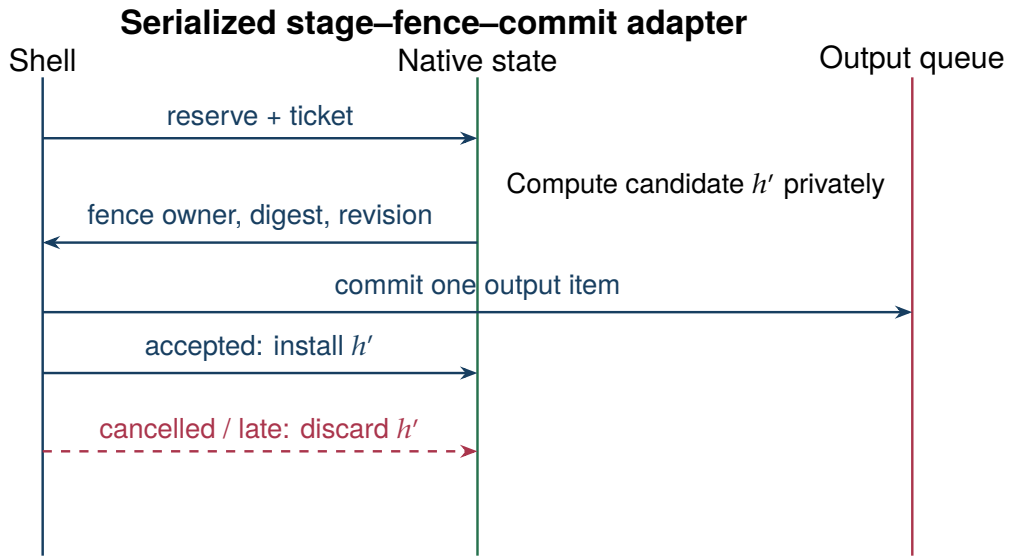
z_t predicts a token following the consumed prefix through t. Greedy decoding chooses $\hat{x}_{t+1} = \arg\max z_t$ and then consumes that selected token to prepare the next prediction. For the fixture after $(1, 2, 1)$, greedy ID 1 is new output; it is not an instruction to re-emit the last prompt token because both happen to be equal.

The adapter's state includes every generated token after its successful commit. Thus $p = p_{\text{prompt}} + r$, where r is the shell's committed decode revision. The fixture uses one-character token strings; it has no EOS convention, trained tokenizer, stochastic sampler or multi-token text detokenizer. A session requires an already consumed prompt or an explicitly supplied BOS token. It does not invent a BOS ID or silently treat zero-state logits as a prompt-conditioned prediction.

31.10 Stage the native result before shell publication

Chapter 30's `Runtime.launch` reserves queue capacity and returns a ticket bound to a lease generation and revision. Our **staged decode** computes a candidate result privately. Native state remains unchanged until `Runtime.commit` accepts the corresponding output.

The adapter checks owner, artifact, current state stamp, candidate stamp, position and head consistency. The stamps detect accidental tensor mutation while work is pending. Parameter changes are rejected by the digest. Duplicate or foreign completion objects are rejected before publication.



Backpressure: no ticket, no candidate. This local ordering is not a durable distributed transaction.

Figure 31.6: Serialized in-process commit ordering. Late or cancelled work settles its ticket but never installs the candidate hidden state.

```
    def finish(self, pending, now):
        if pending is not self.pending or pending is None:
            raise RuntimeError("foreign or duplicate staged completion")
        try:
            self._validate()
            if state_stamp(self.result.state) != pending.before:
                raise ValueError("native state changed while work was pending")
            validate(self.model, pending.candidate.state, self.owner)
            if (
                state_stamp(pending.candidate.state) != pending.after
                or tuple(pending.candidate.logits.detach().tolist()) !=
                 ↪  pending.logits
            ):
                raise ValueError("staged candidate was mutated")
            if pending.candidate.state.position != self.result.state.position + 1:
                raise ValueError("candidate position")
            if not torch.equal(
                pending.candidate.logits,
                pending.candidate.state.hidden @ self.model.head,
            ):
                raise ValueError("candidate logits")
        except Exception:
            self.runtime.commit(pending.ticket, "", now)
            self.pending = None
            raise
        accepted = self.runtime.commit(
            pending.ticket, self.model.vocabulary[pending.token], now
        )
        self.pending = None
        if accepted:
            self.result = pending.candidate
        return accepted
```

Three distinct outcomes matter:

1. Capacity unavailable: no ticket, no native computation, no state advance.
2. Valid accepted completion: commit one output item, then install its already validated candidate state.
3. Cancelled, deadline-expired or invalid completion: settle/discard work without installing the candidate.

For a live prompt state at position three and zero decode revision, a successful step produces position four and revision one. A cancelled step leaves position three and revision zero. The tests check both facts, not just the terminal reason.

This is a serialized, trusted, in-process adapter. Publication and Python assignment are not a durable distributed transaction: a process crash between them requires recovery machinery that is absent here. The adapter also does not schedule prefill, run GPU kernels, authenticate callers, protect Python memory from hostile code, or implement a network transport.

31.11 What the tests establish

All length-three fixture sequences are checked against independent scalar arithmetic. Additional tests exercise mask semantics, invalid tails, vocabulary reordering, parameter mutation, wrong owners, malformed states, derivative continuity, finite differences, cancellation, deadlines, backpressure and duplicate completion. Suffix changes do not alter earlier prefix outputs.

These are bounded deterministic checks of a declared teaching implementation. They do not measure throughput, prove security against a malicious caller, demonstrate biological validity or establish equivalence to a trained DOGMA model. Trace metadata should not log token IDs or hidden values in an untrusted production environment; the printed fixture is deliberately public toy data.

Key ideas

- This is a restricted teaching recurrence, not full DOGMA. Its input-only gate is exactly what makes the closed form possible.
- A frozen dataclass does not freeze tensor storage, so the oracle rechecks the artifact digest at every step.
- Masking holds state and position; it is not a reset. A new request needs a fresh state and owner.
- `detach()` keeps values but cuts gradients and still shares storage. An independent snapshot needs an explicit copy.

31.12 Worked exercises

1. **Shapes.** For $V = 3, D = 2$, give embedding and head shapes. *Answer:* $[3, 2]$ and $[2, 3]$.
2. **Gate.** What is q when $w = 0$? *Answer:* $1/2$ in each coordinate, independently of the embedding.
3. **First token.** Consume ID 1 from zero. *Answer:* $h = (.5, 0)$ and $z = (0, .5, -.5)$.
4. **Second token.** Then consume ID 2. *Answer:* $h = (.25, .5)$ and $z = (0, -.25, .25)$.

5. **Order.** Reverse the prefix $(1, 2)$. *Answer:* $(2, 1)$ gives $h = (.5, .125)$, demonstrating noncommutativity.
6. **Magnitude.** With $M = .5, a = .5, h_0 = 0$, give the asymptotic bound. *Answer:* $M/(1 - a) = 1$.
7. **Semantics.** Are signed hidden values probabilities? *Answer:* no; neither the embedding nor output projection is normalized.
8. **Masked row.** What does a false mask do? *Answer:* no lookup, unchanged hidden state and position, zero placeholder logits, not a prediction.
9. **Invalid tail.** Does a valid first row authorize consuming an invalid second row? *Answer:* no; full live-ID preflight rejects the chunk.
10. **Owner.** Can another tenant use the same request string? *Answer:* not with this state; the ownership pair must match exactly.
11. **Empty chunk.** Can it bypass artifact validation? *Answer:* no; validation occurs before the loop.
12. **Vocabulary.** Does swapping strings preserve artifact identity? *Answer:* no; ordered strings are part of canonical content.
13. **Mutation.** Does a frozen dataclass freeze its tensors? *Answer:* no; storage remains mutable, so the oracle rechecks its digest.
14. **Closed form.** Find the coefficient transporting write b_1 through a length-three prefix. *Answer:* a^2.
15. **Derivative.** Find the fixture's write gradient. *Answer:* $(.3125, .0625)$ for the sum of final hidden components.
16. **Graph.** Can detachment preserve forward values but break training? *Answer:* yes; it removes the gradient edge.
17. **Storage.** Does detachment produce independent storage? *Answer:* no; use an explicit copy when independence is required.
18. **Position.** A prompt consumes three tokens and one decode commits. *Answer:* native position four, shell decode revision one.
19. **Cancellation.** A staged step completes after cancellation. *Answer:* discard the candidate; native position and committed revision remain unchanged.
20. **Durability.** Is local commit-plus-assignment a crash-safe transaction? *Answer:* no; a persistent recovery protocol is a separate implementation obligation.

31.13 Handoff to state pools and memory ownership

Chapter 32 makes logical state reusable in bounded physical storage. Its generational handles, scrubbing and clone semantics must preserve the owner, artifact, position and hidden-state contract established here. An allocator slot is not a request identity. The gradient distinction above also constrains whether a pool copy is an inference snapshot or a training continuation.

CHAPTER 32

DOGMA state pools and memory ownership

In this chapter you will

- separate logical state identity from physical slot residency
- use generations to reject handles after reuse and versions to reject lost updates
- pin in-flight work and defer reclamation until every pin is acknowledged
- clone by value, bridge to native state, and state an honest scrubbing threat model

32.1 The address is not the state

Chapter 31 defined a tenant/request-owned state bound to artifact content, position and a hidden vector. Reusing physical storage introduces a different problem. Request A writes slot zero, finishes, and request B receives that slot. A stale reference from A still names the same address. Address equality does not mean logical-state equality.

A safe design needs both temporal identity and a reclamation rule. Generation numbers reject old handles after reuse. They do not, by themselves, prevent an already-running kernel from touching memory before it has finished. This chapter therefore adds explicit in-flight pins and deferred release to a generational pool.

The reference implementation is a serialized CPU inference oracle using copied scalar snapshots. It is not a concurrent GPU allocator, full DOGMA state layout, authentication mechanism or secure-erasure implementation. We use it to state and test the obligations that those systems would have to refine.

32.2 Map logical identity to bounded storage

A pool has capacity C, hidden width D and one artifact digest. Every occupied slot retains owner $(\text{tenant}, \text{request})$, consumed-token position, hidden payload, slot generation, update version and in-flight pin records. The pool instance has a unique identity separate from its display name.

A **generational handle** contains

$$\eta = (\text{pool instance}, i, g, \text{owner}, \text{artifact}), \qquad 0 \le i < C.$$

A lookup validates the caller's owner pair, handle fields and current slot metadata before exposing a snapshot. Reading with a handle carrying owner A while supplying caller B is rejected: owner metadata in a handle is not itself caller authorization.

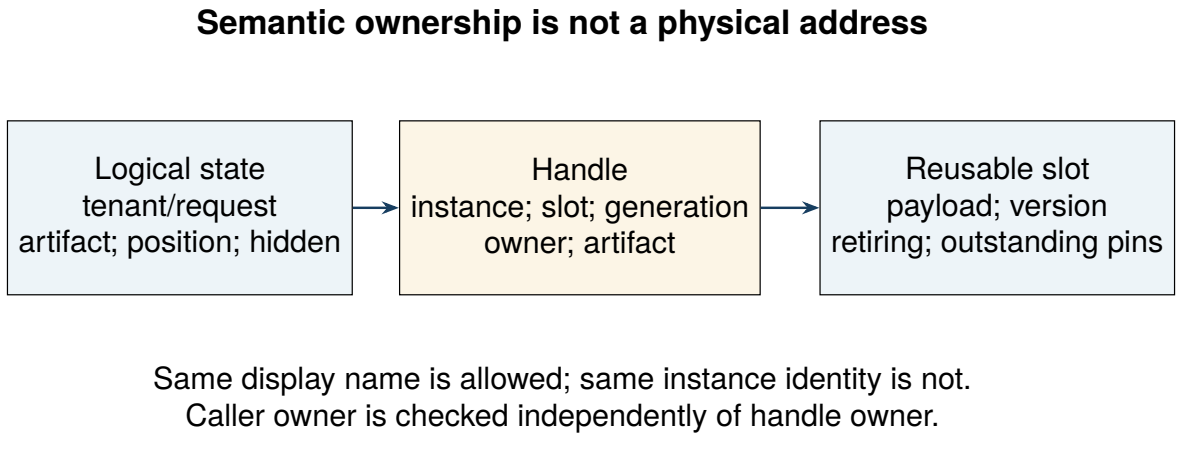


Figure 32.1: Logical state and physical residency are separate identities. The slot can be reused only under a new generation after reclamation.

```
    def _slot(self, handle, owner, allow_retiring=False):
        owner_pair(owner)
        if (
            not isinstance(handle, Handle)
            or handle.pool != self.identity
            or type(handle.slot) is not int
            or not 0 <= handle.slot < self.capacity
        ):
            raise RuntimeError("foreign handle")
        slot = self._slots[handle.slot]
        if type(handle.generation) is not int or slot.generation !=
         ↪  handle.generation:
            raise RuntimeError("stale generation")
        if (
            slot.owner != owner
            or handle.owner != owner
            or handle.artifact != self.artifact
        ):
            raise PermissionError("owner/artifact mismatch")
        if slot.retiring and not allow_retiring:
            raise RuntimeError("state is retiring")
        return slot
```

Exact identity checks within trusted Python code are not authentication. A malicious caller controlling the process could inspect private fields or forge objects. Production authorization must derive caller identity from a trusted boundary, not accept it from arbitrary request input.

32.3 Type the pool before allocating

The pool is bound to the artifact's digest and declared width, with an inference payload of finite float64-compatible scalars and nonnegative integer position. Boolean dimensions, Boolean positions, nonfinite values and wrong widths are rejected. Validating a complete payload before publication prevents a failed initialization from consuming a slot.

One pool class does not imply that full DOGMA has one vector. A real state may contain multiple banks, finite windows or other carried fields. Their layouts, dtypes and semantics

must be part of the typed artifact contract. Matching the shape of one tensor does not make two architectures interchangeable.

The reference uses monotonically growing Python integers for generations. A bounded hardware counter would require an explicit wraparound policy. Allowing an ancient handle to regain validity after counter wrap would undo the identity guarantee.

32.4 Allocation has an all-or-nothing publication point

Allocation first validates owner and optional snapshot, then selects a free slot and publishes its payload/owner/header. Exhaustion raises a capacity error instead of borrowing another live request's state.

```
    def allocate(self, owner, initial=None):
        owner_pair(owner)
        payload = (
            self._snapshot(initial)
            if initial is not None
            else Snapshot(0, (0.0,) * self.width)
        )
        for index, slot in enumerate(self._slots):
            if slot.owner is None:
                slot.payload, slot.owner, slot.version = payload, owner, 0
                slot.retiring = False
                return Handle(
                    self.identity, index, slot.generation, owner, self.artifact
                )
        raise MemoryError("exhausted, including retiring pinned slots")
```

The single-threaded reference can treat owner installation as its publication point. A concurrent implementation needs synchronization covering reservation, initialization and handle publication. If initialization fails after reservation, the implementation must scrub and return the slot without publishing an incomplete handle. Our oracle validates before changing slot metadata.

A pool instance identifier also prevents two different pools with the same display name from accepting each other's handles. This is a correctness namespace, not a secret credential.

32.5 Generation and version solve different stale-state problems

Generation changes when a slot is reclaimed. Update version changes when a live slot's payload changes. They answer different questions:

1. Does the handle still refer to this allocation? Check generation.
2. Was the result computed from the current live payload? Check version.

Two computations can pin version zero of the same allocation. If the first installs version one, the second must not overwrite it with a result computed from version zero. Both handles have the same valid generation; generation alone cannot reject the lost update.

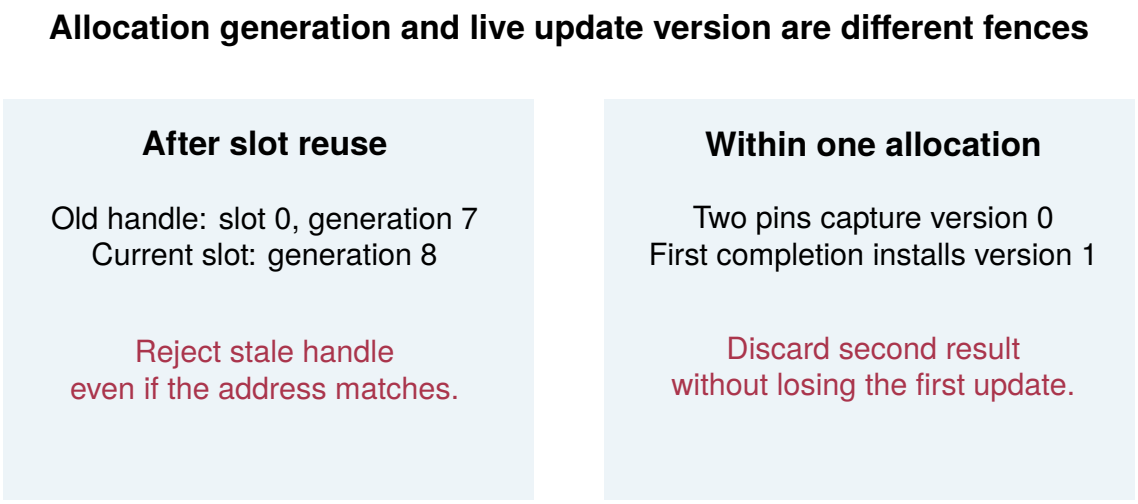


Figure 32.2: Two fences: allocation generation rejects address reuse; live update version rejects a result based on obsolete state.

Tagged pointer/counter structures occur in established concurrent algorithms [73]. Our pool is not an implementation of their queue, nor a claim of lock-free progress. Tagged identity and safe memory reclamation remain separate design obligations.

32.6 Pin in-flight work before release

A **state pin** captures a handle, unique completion serial, live update version and immutable input snapshot. The slot records that pin until its completion or explicit discard is acknowledged. Pinning a retiring allocation is forbidden.

Calling release on an unpinned allocation scrubs its API-visible payload, clears its owner and advances the generation. Calling release while pinned marks the slot retiring but does not expose it as free. Ordinary read, write and new-pin operations then fail. Existing pin completions may settle, but their candidate results cannot be installed.

This lifecycle prevents a cancellation request from being mistaken for a kernel-completion signal.

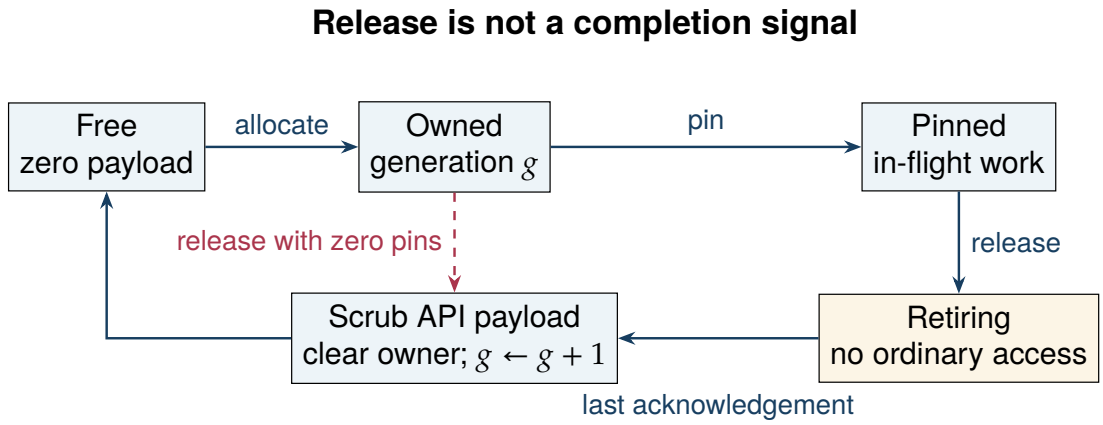


Figure 32.3: Deferred reclamation. A retiring slot remains occupied until the last in-flight acknowledgement; only then may it be scrubbed and reused.

32.7 Fence completion and settle every pin

The completion path validates the exact pin object, its allocation, caller and payload. A candidate may install only when the slot is not retiring and its live version matches the captured input version. Its position must be the pinned position plus one live token.

```
    def complete(self, pin, owner, candidate=None):
        if not isinstance(pin, Pin):
            raise RuntimeError("pin required")
        slot = self._slot(pin.handle, owner, allow_retiring=True)
        if slot.pins.get(pin.serial) is not pin:
            raise RuntimeError("stale or duplicate completion")
        payload = None if candidate is None else self._snapshot(candidate)
        accepted = (
            payload is not None and not slot.retiring and slot.version ==
             ↪  pin.version
        )
        if accepted and payload.position != pin.snapshot.position + 1:
            raise ValueError("candidate must consume one live token")
        del slot.pins[pin.serial]
        if accepted:
            slot.payload = payload
            slot.version += 1
        if slot.retiring and not slot.pins:
            self._scrub(slot)
        return accepted
```

A discarded completion still removes its pin. When a retiring slot loses its last pin, reclamation becomes legal. Duplicate acknowledgement is an error. An invalid candidate leaves the pin live so that a caller can explicitly discard it through the same completion API; silently dropping the pin would hide unfinished-work accounting.

The pool does not verify that arbitrary candidate values actually satisfy the model recurrence. The trusted executor must produce those values using Chapter 31's native semantics. The pool verifies ownership, layout, temporal identity and update boundary, not mathematical reachability of a hidden vector.

32.7.1 A cancellation trace with explicit capacity

For one slot initialized at position three:

Event	Free	Owned	Retiring	Pins
initial	1	0	0	0
allocate	0	1	0	0
pin	0	1	0	1
release	0	0	1	1
late result	1	0	0	0
reuse	0	1	0	0

Release after pinning leaves zero free slots. The late result is rejected, but acknowledging it removes the last pin and permits scrubbing. The next owner receives zero state under a new generation. The old handle cannot access the new allocation even though the index is unchanged.

This trace is a deterministic event ordering, not a concurrent stress result or GPU latency measurement. A leaked pin causes retained capacity; it must be diagnosed rather than bypassed by forcibly reassigning the live slot.

32.8 Clone values without sharing writable storage

The conservative clone reads an immutable snapshot and allocates a new slot for a new owner in the same tenant. Later writes to the clone do not affect the source. Cross-tenant clone is rejected because no export authorization policy is implemented.

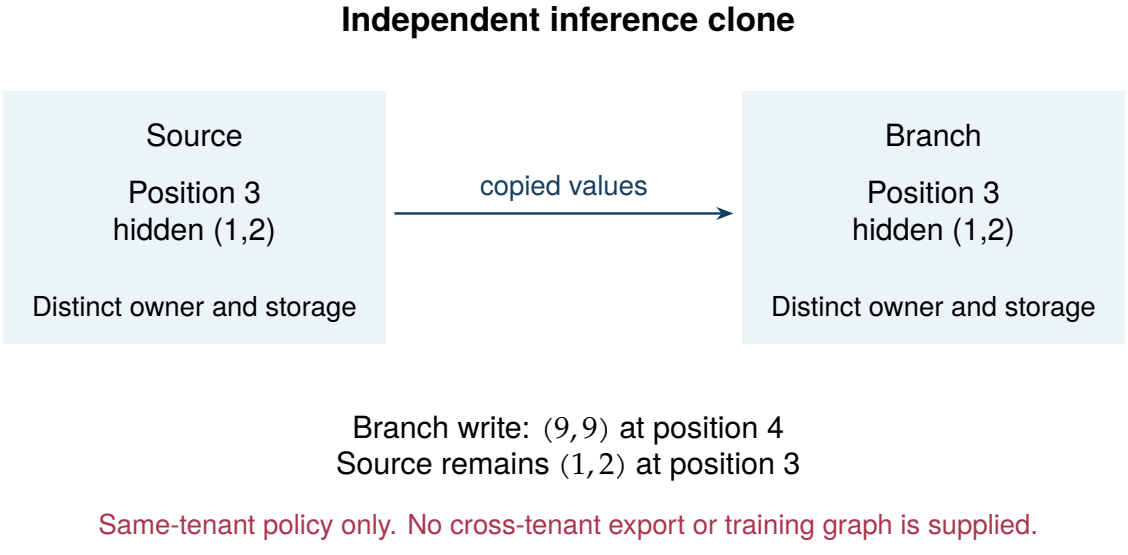


Figure 32.4: Inference clone preserves values and token boundary but allocates independent logical ownership. Same-tenant copying is not permission to export another tenant's state.

```
    def clone(self, handle, owner, new_owner):
        owner_pair(new_owner)
        source = self.read(handle, owner)
        if new_owner[0] != owner[0]:
            raise PermissionError("cross-tenant export policy not provided")
```

```
        return self.allocate(new_owner, source)
```

The reference copies scalars on input and output. Mutating a caller-provided list after allocation cannot mutate the slot. A returned snapshot may survive release, because it is an independent value, not a writable view of reused storage. Clone-by-copy is the semantic oracle for later copy-on-write work.

Clone costs an additional occupied slot. On exhaustion, it fails without changing the source. Identical prefixes alone do not justify mutable aliasing.

32.9 Bridge to the actual Chapter 31 native state

The implementation imports Chapter 31's reference rather than inventing a second recurrence. A fresh native tensor is constructed from the pool snapshot, with identical owner, artifact, position and hidden values.

```
def to_native(pool, handle, owner):
    """Fresh inference tensor; no writable alias and no training graph."""
    native = native_module()
    snap = pool.read(handle, owner)
    return native.State(
        owner,
        pool.artifact,
        snap.position,
        native.torch.tensor(snap.hidden, dtype=native.torch.float64),
    )
```

Saving a native state validates it against its model and owner, checks the pool's artifact digest, then copies its values. A separately materialized native tensor cannot change the pool through in-place mutation. Tests consume a real fixture token, save the resulting state and verify exact values on reload.

This is an **inference snapshot**, not a differentiable training continuation. Converting tensors to scalar data and rebuilding them removes the graph. Chapter 31's derivative parity is not preserved by this pool serialization, and we do not claim otherwise. A training-state allocator needs a distinct graph-lifetime contract.

The bridge's ordinary save path requires no live pins; asynchronous candidates must instead use the fenced completion API. Combining the pool with the shell's output publication requires a joint serialized policy for both resources. This chapter does not claim that two independent APIs compose into a durable transaction automatically.

32.10 Workspace has a shorter lifetime

A **step workspace** holds temporary projections or reductions, while the persistent state must survive continuation boundaries. The teaching workspace returns immutable copied values, validates shape and finiteness, and refuses use after close. Closing is idempotent and resets its API-visible payload.

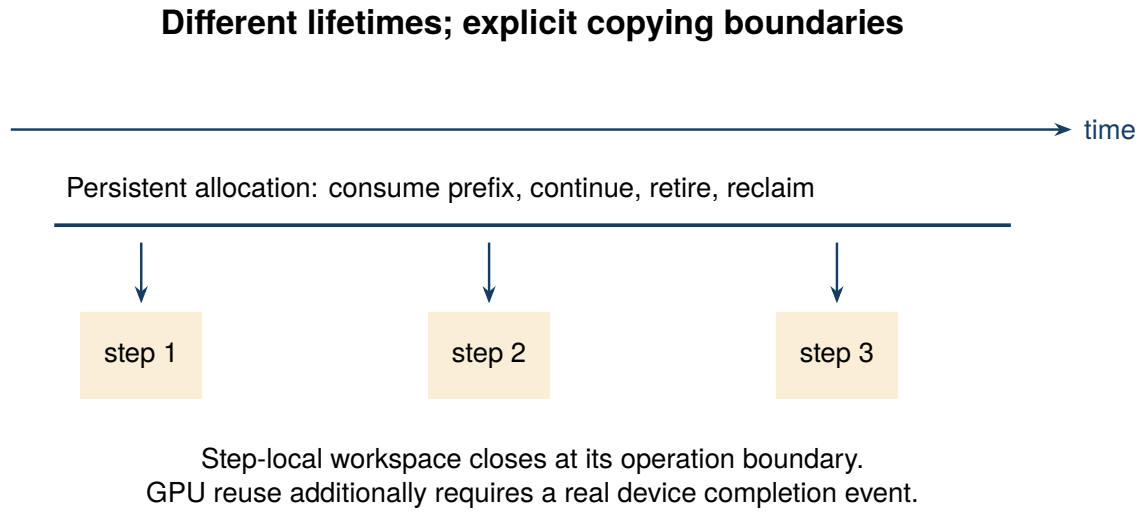


Figure 32.5: Persistent state and step-local scratch have different lifetimes. Captured input/output snapshots are copied values, not borrowed writable addresses.

An actual GPU workspace may be borrowed by an asynchronous kernel. Its reuse must wait for the corresponding completion event or synchronization, just as persistent state reclamation waits for pins. A lexical function return is not necessarily a device-completion boundary.

32.11 Scrubbing: specify the threat model honestly

The reference replaces payload with zeros and clears owner/position metadata before exposing a free slot. This prevents its ordinary API from returning a previous request's values after reuse.

It does not securely overwrite every copy in Python memory. Old immutable snapshots can remain referenced, and garbage collection is not cryptographic erasure. Nor does it overwrite VRAM, clear driver caches or protect crash dumps. A production erasure claim requires its own platform-specific threat model and verification. Resetting logical position alone is still insufficient: every persistent carried field must be initialized consistently before reuse.

32.12 Account for capacity and physical costs separately

At every event,

$$C = C_{\text{free}} + C_{\text{owned}} + C_{\text{retiring}}.$$

Retiring slots count as occupied even if their request is terminal. Pins are counted separately because multiple pins may refer to one occupied slot.

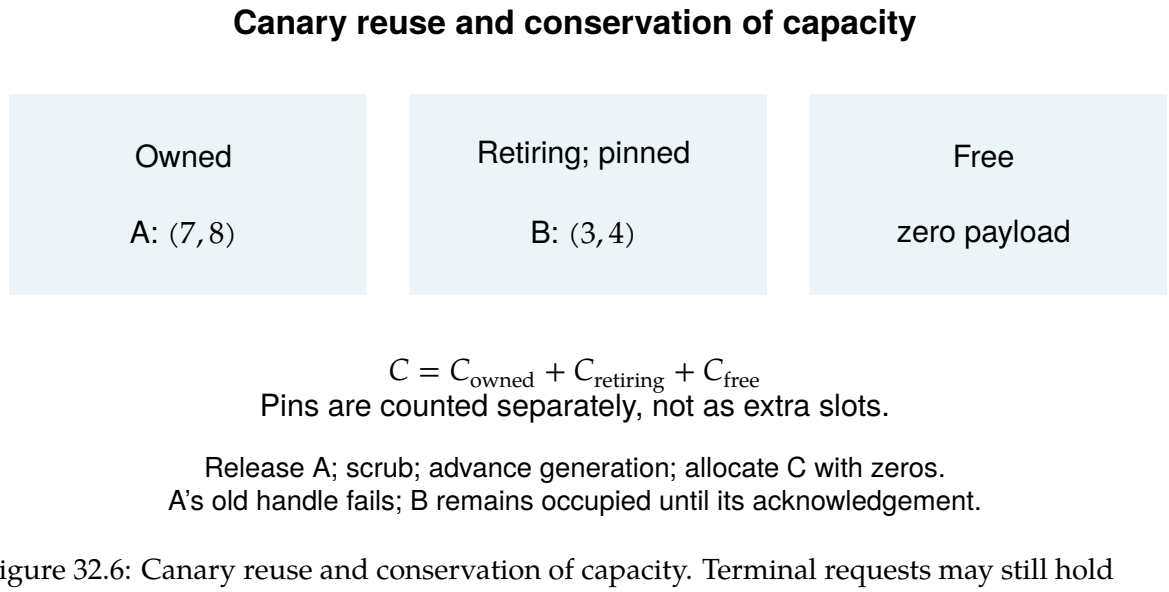


Figure 32.6: Canary reuse and conservation of capacity. Terminal requests may still hold retiring slots until work settles.

For a contiguous float64 arena, payload size would be $8CD$ bytes. The reference's `payload_bytes` reports this logical scalar-payload quantity, not Python heap usage. Actual memory also includes metadata, object overheads, retained snapshots, kernel buffers, allocator reservations and copies. If cloning every branch, those extra occupied slots must enter admission policy.

A useful metric panel reports free/owned/retiring counts, outstanding pins, failed allocations, stale-handle events and reclamation delay. It should not dump hidden values or prompt-derived state into routine telemetry.

32.13 Test against an independent ownership ledger

The tests maintain a separate map of owners to expected snapshots while performing deterministic randomized allocation/release sequences. Distinct canaries identify each request. After reuse, recently stale handles must fail and all new zero-initialized allocations must satisfy their initialization contract.

Targeted cases add deferred cancellation, multiple outstanding pins, duplicate completion, live version conflict, malformed allocation, clone exhaustion, caller mismatch, artifact mismatch and native-bridge storage independence. These tests are serialized: no race, device stream ordering, synchronization primitive or production performance is claimed.

For a concurrent implementation, show that each operation can be linearized to a permitted sequential outcome. Force allocate/release, pin/cancel, clone/release and completion/reuse races, rather than assuming a high-volume throughput test covers those interleavings.

Key ideas

- Generation answers "is this still the same allocation?"; version answers "was the result computed from the current payload?" Neither substitutes for the other.
- Release under a live pin marks the slot retiring. It is still occupied, so capacity is free + owned + retiring.
- Clone-by-copy is the semantic oracle that any later copy-on-write optimization must refine.
- Writing zeros clears what the API exposes. It is not cryptographic erasure of Python heap, VRAM or crash dumps.

32.14 Twenty worked exercises

1. **Identity.** Is slot zero a stable request identity? *Answer:* no; reuse changes its logical owner.
2. **Generation.** What rejects a pre-release handle? *Answer:* its old generation differs from current metadata.
3. **Version.** What rejects a second result from obsolete live state? *Answer:* its pinned update version differs from the current version.
4. **Namespace.** Can equal pool display names alias? *Answer:* not here; pool-instance identity is distinct.
5. **Caller.** Does a handle's owner field authenticate the caller? *Answer:* no; caller identity requires a trusted authorization boundary.
6. **Allocation failure.** An input snapshot has NaN. *Answer:* validation fails before any slot is published.
7. **Capacity.** The only slot is retiring with one pin. *Answer:* there are zero free slots; allocation must fail or queue externally.
8. **Cancellation.** Can release scrub a pinned slot immediately? *Answer:* no; mark retiring and wait for acknowledgements.
9. **Late result.** May a retiring slot install a candidate? *Answer:* no; completion settles its pin but discards the candidate.
10. **Two pins.** The first settles after release. *Answer:* retain the slot until the second also settles.
11. **Duplicate acknowledgement.** Can the same pin settle twice? *Answer:* no; it is absent after the first completion.
12. **Lost update.** Two pins capture version zero; one installs. *Answer:* the second result is discarded because version is now one.
13. **Position.** A pin captured position three. Is candidate five legal? *Answer:* not for this one-live-token completion contract.
14. **Clone.** Does modifying a branch alter its source? *Answer:* no; storage is independent.
15. **Export.** Does clone allow a different tenant? *Answer:* no; an explicit export policy is absent.

16. **Snapshot lifetime.** Can a copied snapshot outlive release? *Answer:* yes; it is an independent value, not a handle to reused storage.
17. **Training.** Does scalar snapshot reload preserve autograd? *Answer:* no; this is an inference bridge.
18. **Memory.** For $C = 4, D = 2$, what is logical float64 payload size? *Answer:* 64 bytes, excluding metadata and Python overhead.
19. **Erasure.** Does replacement with zeros prove secure erase? *Answer:* no; copies and platform storage require separate verification.
20. **Concurrency.** Do these serialized tests prove GPU race freedom? *Answer:* no; a device implementation must refine the contract and test real completion/reclamation races.

32.15 Handoff to checkpoints and shared prefixes

Chapter 33's checkpoint, branching and prefix-state candidate must restore fresh ownership rather than revive a stale pool handle. Its optimized sharing must refine independent-copy semantics and retain model, semantic version and exact token boundary. Integrity, authorization and graph lifetime remain separate obligations. The bounded pool here supplies the ownership and reclamation foundation for that next chapter.

CHAPTER 33

DOGMA checkpoints, branching and prefix state

In this chapter you will

- cut a computation at a semantic token boundary and prove continuation equivalence
- bound checkpoint approximation error for a contracting recurrence
- separate byte integrity, structural compatibility and replay consistency in a bounded data-only envelope
- restore into fresh owners, look up the longest validated prefix and count checkpoint storage

Three requests begin with the same prompt. Each has consumed tokens $(1, 2, 1)$, but one will continue with token 0 and another with token 2. Should the engine repeat the prompt computation? Should both requests point to the same writable state? Neither choice is necessary. We can preserve one immutable description of the prefix and restore independent mutable states. The difficult part is specifying exactly what that description means.

Chapter 31 supplied an artifact-bound native recurrence; Chapter 32 supplied owned storage with generational handles and in-flight pins. This chapter connects them through a bounded, data-only checkpoint protocol. The executable reference imports the actual Chapter 32 pool and Chapter 31 model. We derive continuation equivalence, deliberately break it, then account for the storage and replay costs of reusable prefixes.

33.1 Cut the computation at a semantic boundary

Let $x_{1:t} = (x_1, \dots, x_t)$ be the ordered integer token prefix already consumed, and s_t the complete model state immediately after token t. For fixed parameters θ and declared state semantics, write

$$s_{t+1} = F_\theta(s_t, x_{t+1}), \tag{33.1}$$

$$z_{t+1} = G_\theta(s_{t+1}). \tag{33.2}$$

Here z_{t+1} denotes logits after consuming the next token, not the token sampled from those logits. A transport packet, scheduler iteration and model-token boundary are different clocks. We snapshot only at the last one, when no partially accepted native operation is outstanding.

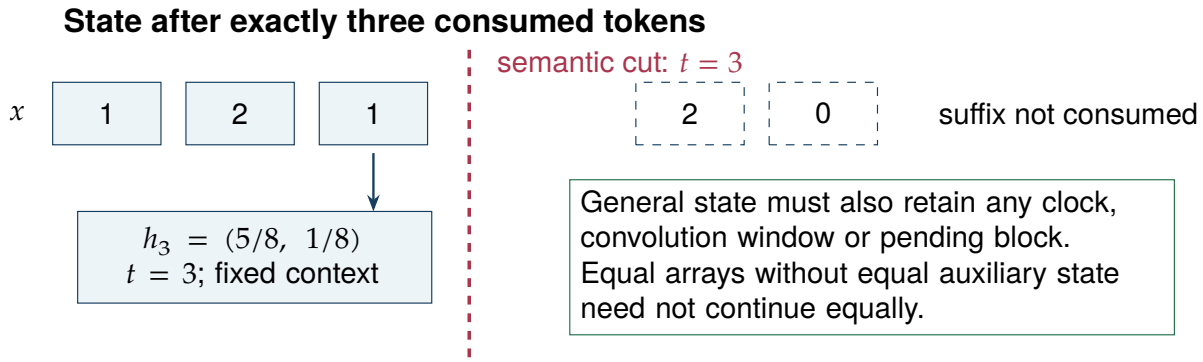


Figure 33.1: A checkpoint cuts after the consumed prefix, not after a network packet. The pending suffix is excluded. Position and any additional state needed for continuation belong at the cut.

For the restricted Chapter 31 reference, $s_t = (t, h_t)$ and $h_t \in \mathbb{R}^D$. The model artifact fixes an embedding matrix $E \in \mathbb{R}^{V \times D}$, coordinate retentions $a \in [0,1)^D$, write coefficients $w \in \mathbb{R}^D$, and a head $U \in \mathbb{R}^{D \times V}$. Its update is

$$e = E_{x_{t+1},:}, \quad h_{t+1} = a \odot h_t + \operatorname{sigmoid}(w \odot e) \odot e, \quad z_{t+1} = h_{t+1}^{\mathsf{T}} U. \tag{33.3}$$

The printed vector orientation in the final product is notation; Python represents h as a one-dimensional tensor. Native arithmetic here uses finite CPU float64 values. This is a concrete non-Transformer inference component, not a trained complete DOGMA model.

33.1.1 Tensor equality is not state sufficiency

Consider a machine with hidden value $h = 0$ and auxiliary phase $p \in \{0, 1\}$. On the next token it emits $h + (-1)^p$. Two snapshots containing only h are identical, yet one emits 1 and the other -1. No hash of h can recover the missing phase. A model with a finite convolution window, unfinished memory block, positional clock or segment-dependent transition requires those objects in its continuation state.

The reference does not serialize every future DOGMA architecture. Its semantic identifier names the restricted causal diagonal state. A new recurrence or additional state requires a new schema/semantic contract and tested migration, not a permissive shape check. An offline reverse-complement suffix state from Chapter 21 cannot silently become a causal prefix state just because its tensor has the same dimensions.

33.2 Derive the continuation obligation

Define $F_\theta^*(s, y)$ by repeated application of F_θ over a finite suffix y. A snapshot decoder D_C and encoder E_C should satisfy

$$D_C(E_C(s_t)) \equiv s_t, \tag{33.4}$$

where $\equiv$ means equality of all semantically relevant values under the declared numerical interpretation. Request IDs and physical addresses are excluded: restore must create new ownership.

Proposition 33.1 (Continuation equivalence)**.** *Suppose the transition/readout are deterministic, model artifact and semantics are unchanged, and equation 33.4 holds for complete state. For any legal suffix $y_{1:k}$, original and restored execution produce the same successive states and logits.*

Proof. At suffix length zero the semantic states agree by equation 33.4. Assume equality after j suffix tokens. Both executions next apply the same F_θ to equal states and equal token y_{j+1}, so their next states agree. Applying the same G_θ gives equal logits. Induction covers all $0 \le j \le k$. □

This proof locates the engineering obligations. It does not prove that an arbitrary tensor dump contains complete state, that two floating-point kernels use the same reduction order, or that sampled generation replays without decoder state. Those are premises to check. Here finite binary64 values round-trip, and restore uses the same native operator as ordinary continuation. Independent rational tests check fixture arithmetic. Another precision policy needs its own parity contract.

33.2.1 If restore is approximate

Suppose a compressed checkpoint introduces hidden error $\|\delta_t\|_\infty \le \epsilon$. In equation 33.3, the additive write depends on input, not on h_t. Under exact arithmetic,

$$\delta_{t+k} = a^{\odot k} \odot \delta_t, \qquad \|\delta_{t+k}\|_\infty \le \alpha^k \epsilon, \quad \alpha = \max_j a_j < 1. \tag{33.5}$$

If each subsequent update adds error at most η in that norm, summing the geometric series gives

$$\|\delta_{t+k}\|_\infty \le \alpha^k \epsilon + \eta \frac{1 - \alpha^k}{1 - \alpha}. \tag{33.6}$$

For $\alpha = 1/2$, the limiting additive term is 2η. This bound is specific to the stated recurrence. State-dependent gates need their own Jacobian/stability argument. Even a small logit perturbation can change an argmax near a tie; this norm bound is not an identical-generation guarantee.

33.3 The envelope binds interpretation to bytes

Use the context tuple

$$\kappa = (\tau, m, \nu, \xi, D, V, \text{dtype}),$$

where τ is the tenant namespace, m the model-content digest, ν the state semantics and ξ the tokenization contract identifier. D and V declare hidden width and vocabulary size. The body and envelope are

$$B_t = (f, \kappa, t, x_{1:t}, h_t), \quad C_t = (b_t, d_t), \quad b_t = \text{encode}(B_t), \quad d_t = \text{SHA256}(b_t). \tag{33.7}$$

The format f governs parsing; ν governs the meaning of parsed state. Neither replaces the model artifact. SHA-256 supplies the digest, not a producer-authentication protocol.[80]

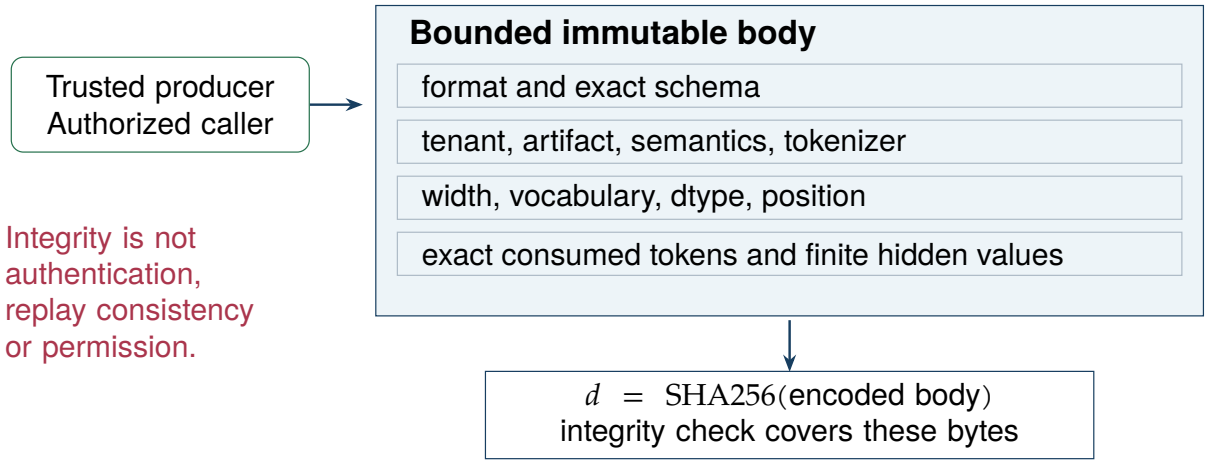


Figure 33.2: A digest binds the envelope's bytes. Context and exact tokens determine how the payload may be used. Trusted provenance remains a separate boundary: a self-consistent envelope is not an authorized one.

Chapter 31's artifact function hashes ordered vocabulary and parameter content. The local ξ hashes the ordered vocabulary mapping. It is not a universal tokenizer: deployment must bind preprocessing, normalization, special-token policy, adapters and any conditioning that changes transitions. A display name such as `model-latest` is not a content identity.

Tenant IDs here arrive through a trusted in-process caller. Including τ prevents accidental namespace reuse in the reference; it does not authenticate an API request. Authorization must precede construction of the context.

33.3.1 Three different meanings of validity

1. **Byte integrity:** bytes match the supplied digest. Someone able to replace both can compute a new matching pair.
2. **Structural compatibility:** schema, dimensions, finite values, model, semantics, tenant and exact prefix satisfy the local contract.
3. **Replay consistency:** hidden values equal those obtained by executing the prefix from its prescribed initial state.

The first two do not establish the third. Replace $h_1 = 0.625$ by 11 and recompute the digest: the object can remain well-formed. Optional `replay_audit` recomputes the actual native prefix and detects this fixture forgery, but costs the work caching aims to avoid. In ordinary use, trusted production and storage supply the missing provenance. Neither replay consistency nor hashing grants permission to read another tenant's state.

33.4 Parse bounded data, not executable objects

The decoder accepts immutable bytes and a lowercase hexadecimal digest. It rejects bodies above 262,144 bytes before JSON parsing. The schema allows at most 4,096 tokens and 256 hidden coordinates: teaching limits, not measured production requirements.

Duplicate object names, unknown or missing fields, nonfinite values, booleans where integers are required, out-of-vocabulary tokens, inconsistent position, undeclared dtype and noncanonical encoding are rejected. Default Python JSON accepts some forms this protocol refuses, including repeated names and nonfinite constants.[75] Integer hidden values must

also be exactly representable in binary64 before conversion. An overflow spelling such as `1e999` also requires a finite-value check; rejecting the literal word `Infinity` is insufficient.

The encoder sorts keys, removes optional whitespace, emits ASCII escapes and forbids NaN. This is a Python wire profile, not universal cross-language canonicalization. Interoperability needs an explicitly standardized encoding and numerical representation.

```
def checked(checkpoint, context, tokens):
    validate_context(context)
    tokens = tokens_copy(tokens, context)
    body = decode(checkpoint)
    if body["context"] != asdict(context):
        raise ValueError("checkpoint context mismatch")
    if tuple(body["tokens"]) != tokens:
        raise ValueError("exact consumed prefix mismatch")
    return body
```

This printed function checks context and exact prefix after `decode` has enforced schema and bytes. Whole functions are extracted from `drafts/ch33/reference.py`; the parser and negative tests remain available there even when not all code is printed.

Avoiding executable loading narrows one risk surface, but bounded JSON is not a sandbox. Resource limits and trustworthy surrounding code still matter. PyTorch's restricted-loading documentation similarly separates reduced code-execution exposure from complete protection against denial of service and downstream memory hazards.[76] The reference calls neither pickle nor `torch.load`.

33.5 Capture only a settled owned state

Chapter 32's handle identifies pool instance, slot, allocation generation, owner pair and artifact. Those physical handles do not belong in a portable checkpoint. A restarted process might reuse an integer slot for an unrelated request. Restoring the handle would confuse historical address with current ownership.

Capture validates the actual pool handle, refuses pinned/retiring storage, checks $t = |x_{1:t}|$, and copies immutable snapshot values into the envelope. This conservative quiescence policy can later be replaced by a consistently captured immutable version under an explicit synchronization protocol.

```
def capture(pool, handle, owner, context, tokens):
    _pool_context(pool, context)
    pool_module().owner_pair(owner)
    if owner[0] != context.tenant:
        raise PermissionError("tenant scope")
    tokens = tokens_copy(tokens, context)
    slot = pool._slot(handle, owner)
    if slot.pins:
        raise RuntimeError("snapshot requires a quiescent semantic boundary")
    snapshot = pool.read(handle, owner)
    if snapshot.position != len(tokens):
        raise ValueError("pool position/prefix mismatch")
    body = {
        "format": FORMAT,
        "context": asdict(context),
```

```
        "position": snapshot.position,
        "tokens": tokens,
        "hidden": hidden_copy(snapshot.hidden, context.width),
    }
    wire = canonical(body)
    checkpoint = Checkpoint(wire, sha(wire))
    decode(checkpoint)
    return checkpoint
```

Calls are serialized and exclusive. A private pool-validation helper checks the pin ledger; this is a teaching bridge, not a public allocator API. Generation/owner/artifact fences remain intact. The supplied prefix is trusted bookkeeping: the pool stores position and hidden values, not a transcript proving their origin. The replay falsifier makes this limitation visible. Authoritative request records could strengthen the interface without removing the need for trusted execution.

33.6 Restore allocates; it does not resurrect

Restoring to owner o' creates a fresh handle ℓ':

$$\mathrm{read}(\ell', o') = (t, h_t), \qquad \mathrm{owner}(\ell') = o'.$$

It may occupy another slot or a later generation of the old slot. The released handle stays stale even when semantic values are reconstructed exactly.

For K branches, validate the entire owner group/checkpoint and ensure K free slots exist before allocating. Retiring pinned slots are not free. Under serialized calls, ordinary validation and capacity failures leave the pool unchanged.

```
def restore_many(checkpoint, pool, owners, context, tokens):
    """Preflight all branches before mutation, under serialized exclusive
     ↪ calls."""
    _pool_context(pool, context)
    body = checked(checkpoint, context, tokens)
    if type(owners) not in (tuple, list) or not 1 <= len(owners) <= pool.capacity:
        raise ValueError("bounded nonempty branch group")
    for owner in owners:
        pool_module().owner_pair(owner)
        if owner[0] != context.tenant:
            raise PermissionError("cross-tenant restore not provided")
    if len(set(owners)) != len(owners):
        raise ValueError("distinct branch owners")
    if any(slot.owner in owners for slot in pool._slots if slot.owner is not
     ↪ None):
        raise ValueError("new request owners must not already own pool slots")
    if pool.stats()["free"] < len(owners):
        raise MemoryError("branch group cannot fit, including retiring states")
    snapshot = pool_module().Snapshot(body["position"], tuple(body["hidden"]))
    return tuple(pool.allocate(owner, snapshot) for owner in owners)
```

This is preflight failure atomicity, not a crash transaction. A process failure midway through allocation or another thread's simultaneous allocation requires rollback, reservation or different synchronization. The teaching loop does not establish distributed or hardware atomicity.

33.7 Work through two actual native branches

Use Chapter 31's fixture with $a = (1/2, 1/4)$ and $w = (0, 0)$. Both gates are $1/2$. Tokens $0, 1, 2$ have embeddings $(0, 0)$, $(1, 0)$, $(0, 1)$ respectively. Starting from zero:

$$\begin{aligned} h_1 &= (1/2, 0) && \text{after token 1,} \\ h_2 &= (1/4, 1/2) && \text{after token 2,} \\ h_3 &= (5/8, 1/8) && \text{after token 1.} \end{aligned}$$

The left branch consumes 0, giving $(5/16, 1/32)$. The right consumes 2, giving $(5/16, 17/32)$. Both positions become four; the checkpoint stays at three.

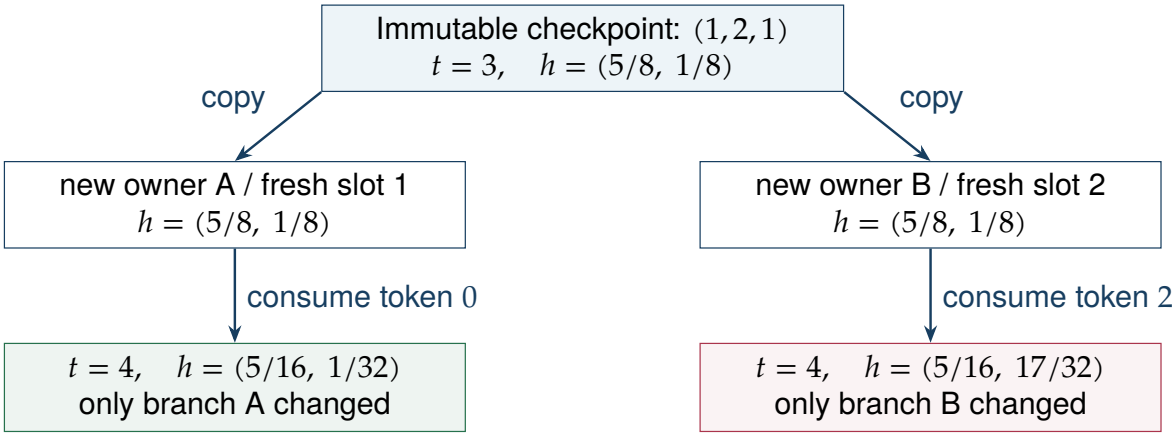


Figure 33.3: Equal prefix values are copied into independently owned slots. Suffix transitions change only their respective branches. The checkpoint is an immutable value, not shared writable state.

Boundary	h_1	h_2
shared prefix	0.625	0.125
left: token 0	0.3125	0.03125
right: token 2	0.3125	0.53125

These numbers come from the imported recurrence. Tests independently use rational arithmetic, then enumerate every prefix of length zero through three and every two-token suffix. For each case, replay from zero and restore-plus-suffix must match. This is a finite implementation check, distinct from the induction proof.

Fresh CPU tensors prevent writable aliases with source/sibling storage. Loading scalar values also discards the old computation graph. Equal inference logits do not establish resumed backpropagation. Training restart needs parameters, optimizer, gradient-accumulation, RNG and data-order state beyond this protocol.

33.8 A prefix key is an index, not an acceptance decision

Define

$$k_t = H(\text{encode}(\kappa, x_{1:t})).$$

Unlike the checkpoint digest, this key excludes h_t; it names the prefix/context being sought. The cache can therefore detect conflicting state bodies for one deterministic prefix instead of silently replacing them.

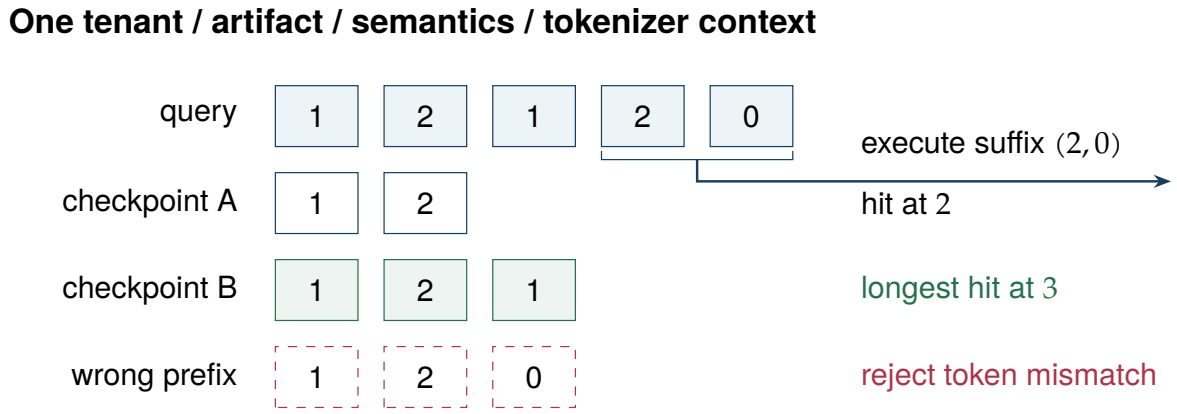


Figure 33.4: Longest-prefix matching uses exact token identity within one context. Reuse stops at token three here; only the two remaining tokens are executed. A shorter hit is valid but saves less work.

For query $q_{1:T}$ choose

$$t^* = \max\{t \le T : \text{a validated checkpoint for } (\kappa, q_{1:t}) \text{ exists}\}.$$

Otherwise start from initial state. A zero-prefix checkpoint is a real hit, so the API returns both length and an optional object; zero alone cannot distinguish that hit from a miss.

```
    def longest(self, context, query):
        validate_context(context)
        query = tokens_copy(query, context)
        for length in range(len(query), -1, -1):
            prefix = query[:length]
            key = prefix_key(context, prefix)
            checkpoint = self._items.get(key)
            if checkpoint is not None:
                checked(checkpoint, context, prefix)
                self._items.move_to_end(key)
                return length, checkpoint
        return 0, None
```

After a hash-table hit, the reference checks full envelope, context and token identity. A test replaces the key function by a constant to force an indexing collision: a wrong prefix is rejected. This tests the acceptance path, not SHA-256's empirical collision resistance. Sampling seeds are not required in this deterministic model-state key; transition-changing conditioning is. Stochastic state updates need relevant random state or an explicitly nondeterministic reuse contract.

33.8.1 Isolation and bounded eviction

The LRU budget sums immutable envelopes' wire lengths. Reuse updates recency. An oversized object is rejected before eviction; otherwise oldest envelopes are removed until it fits.

This counts encoded bytes, not Python heap, allocator capacity, device buffers or total resident memory.

Restored branches have independent pool values, so source-envelope eviction cannot invalidate them. Physical sharing would instead require live branches to retain shared pages. Cache and branch ownership are separate reachability roots.

Different tenants receive different keys and cross-tenant restore is absent. Timing leakage, malicious callers and authorization still need service-level policy. Current vLLM documentation offers comparison: prefix-dependent block hashes, extra identity inputs and optional salting for isolation.[77] This reference inherits no production guarantee from that precedent.

33.9 Copy-on-write must refine the copy oracle

Suppose branches A and B initially map to immutable physical page P. A write by A may allocate P_A, copy P, redirect A's mapping, then modify P_A. B still observes original P.

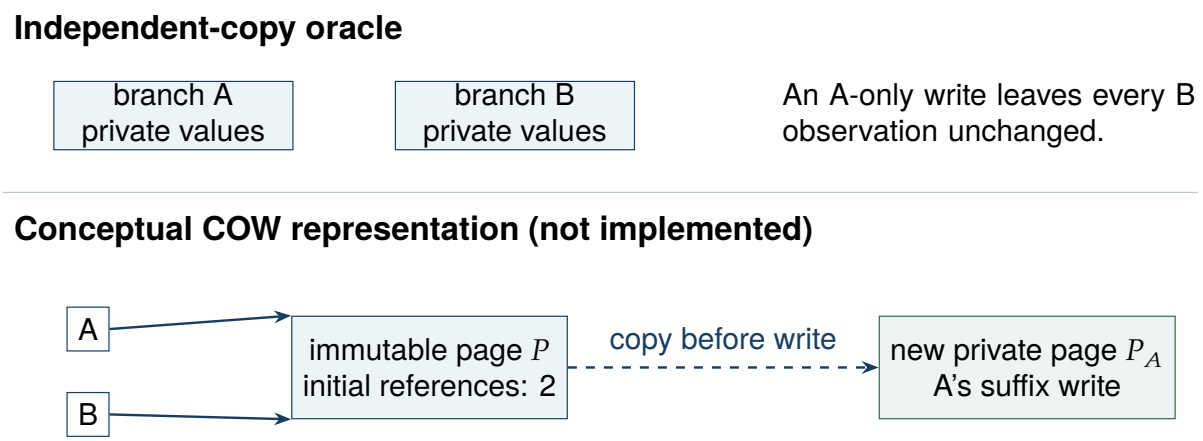


Figure 33.5: A possible copy-on-write representation beside the independent copy oracle. Mapping publication, reference counts and page lifetime add obligations. Physical COW is not implemented in this chapter.

Let R expand physical representation into the logical independent states visible through the API. For permitted operation u, require

$$R(\mathrm{COWstep}(P, u)) = \mathrm{Copystep}(R(P), u).$$

Reads must agree. An A-only write changes only A's expanded state. After release/eviction, remaining live mappings must resolve. Tests need partial pages, two writers, copy-allocation failure and cancellation between copy and mapping publication.

For single-page state size S and K branches, independent copies use KS state bytes. If every branch writes that page, COW can also need roughly KS, plus mappings. Smaller independently writable pages reduce copied bytes only for suitable write patterns and add bookkeeping. Memory savings are workload-dependent.

33.10 Derive checkpoint spacing and the metadata bill

Let stream length be T tokens, checkpoint interval B tokens, and hidden width D. With eight bytes per coordinate and $n = \lfloor T/B \rfloor$ nonzero periodic checkpoints, state payload alone uses $8Dn$ bytes.

Our envelope also stores each complete prefix. Under a separate logical convention of four bytes per token ID, repeated prefix storage is

$$M_{\text{tokens}} = 4 \sum_{j=1}^{n} jB = 2Bn(n+1) \text{ bytes}, \tag{33.8}$$

$$M_{\text{payload}} = 8Dn + 2Bn(n+1). \tag{33.9}$$

These are not actual JSON sizes: framing, decimal spellings, digests and containers add bytes. The formula still exposes a common mistake: fixed-width recurrent state does not make a cache of full-prefix envelopes constant-memory. With $B = 1$, copied token metadata grows quadratically in T.

If a desired boundary's residual offset from its preceding checkpoint is uniform on $\{0, \dots, B-1\}$, expected replay is

$$\mathbb{E}[r] = \frac{1}{B} \sum_{r=0}^{B-1} r = \frac{B-1}{2}. \tag{33.10}$$

Uniformity is a workload assumption. Prefix junctions may cluster at templates or block boundaries; finite-stream edges may change the residual distribution.

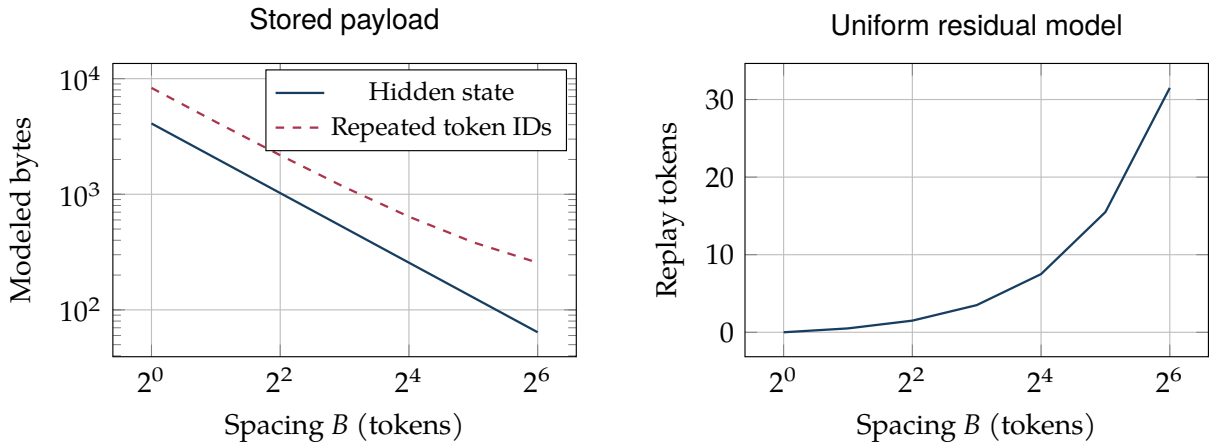


Figure 33.6: Modeled storage and replay for $T = 64$, $D = 8$. Token metadata counts repeated complete prefixes with four-byte IDs. These curves are arithmetic, not measured latency or actual JSON bytes.

B	States	State B	Token B	Replay
1	64	4096	8320	0
2	32	2048	4224	0.5
4	16	1024	2176	1.5
8	8	512	1152	3.5
16	4	256	640	7.5
32	2	128	384	15.5
64	1	64	256	31.5

At $B = 8$, eight hidden payloads use 512 bytes, copied prefixes 1,152 bytes, and expected replay 3.5 tokens. At $B = 1$, these totals rise to 4,096 and 8,320 bytes to eliminate replay. Chained hashes and a verified shared token store can avoid transcript duplication, but introduce lifetime/integrity dependencies.

The simple longest-prefix loop serializes successively shorter prefixes. Since $\sum_{t=0}^{T} t = T(T+1)/2$, a miss can require $O(T^2)$ token-processing work for keys despite expected constant-time dictionary lookup per candidate. Tries or incremental indices can improve that cost without replacing checkpoint validation.

33.11 State snapshots, generation restart and durability

A model-state checkpoint resumes F_θ. Sampled-text restart may also need RNG algorithm/state, sampler settings, grammar automaton, stop policy and committed-output cursor. Training adds optimizer and data-order state. These are different restart contracts; one tensor is not unqualified resumability.

Our cache is process-local. Durable publication must specify survived failures. One candidate sequence is: encode/validate; write an unadvertised object; persist contents under the storage policy; publish its identity atomically; make it discoverable. Checksums alone do not make writes atomic or durable. Filesystem rename/directory persistence and object-store transactions require platform-specific evidence; no such implementation is shipped here.

Current serving systems illustrate semantic granularity. vLLM documents recurrent-state reuse at selected block boundaries. Additional checkpoints at shared-prefix junctions require specific configurations.[78, 79] Those are comparison points, not evidence that this reference implements those models or outperforms their engines.

Key ideas

- Tensor equality is not state sufficiency: two snapshots with equal h can continue differently if a phase or clock is missing.
- A rehashed forgery passes the integrity and schema checks. Only replay or trusted provenance detects it.
- Restore allocates a new owner; it never revives a stale handle. Branches own copied values, so evicting the source cannot break them.
- Fixed-width state does not make a prefix cache constant-memory. Storing each complete prefix grows quadratically when $B = 1$.

33.12 Exercises with worked reasoning

1. **Predict a branch.** Continue the fixture with token 1. *Solution:* $(5/16 + 1/2, 1/32) = (13/16, 1/32)$ at position four.

2. **Prove a cut.** What is the induction's base case? *Solution:* equality of complete semantic state, not physical address or only the hidden array.

3. **Omitted clock.** Construct equal arrays with unequal next outputs. *Solution:* $h = 0$, phase $p \in \{0, 1\}$, output $(-1)^p$.

4. **Position.** Why keep t if this recurrence does not use it algebraically? *Solution:* to bind the consumed-token boundary and catch inconsistent request bookkeeping.

5. **Rehashed forgery.** Change h and recompute SHA-256. *Solution:* integrity/schema checks can pass; replay consistency and authorization remain separate.

6. **Boolean token.** Why not accept `True` as 1? *Solution:* subtype convenience is not the declared exact-integer wire type.

7. **Overflow.** Is rejecting `NaN` sufficient? *Solution:* no; `1e999` may parse to a nonfinite value.

8. **Fresh ownership.** Restore into a released slot. *Solution:* the new generation does not reactivate the stale handle.

9. **Capacity.** Two slots exist; one is retiring with a pin. Can two branches restore? *Solution:* no, until final acknowledgement.

10. **Group restore.** Why preflight all owners? *Solution:* a later duplicate/unauthorized owner must not leave earlier branches allocated after ordinary validation failure.

11. **Autograd.** Does equal hidden state preserve gradients? *Solution:* no; scalar serialization and fresh tensors cut the graph.

12. **Collision.** Force identical lookup keys for two prefixes. *Solution:* full token/context comparison must reject the wrong envelope.

13. **Zero hit.** Why return an optional object as well as length? *Solution:* zero can mean an initial-state checkpoint or no hit.

14. **Eviction.** Why can a restored branch outlive its source envelope? *Solution:* it owns copied values. Shared pages would require references.

15. **Storage.** Verify $T = 64, D = 8, B = 16$. *Solution:* $n = 4$; hidden 256 bytes, token IDs 640, total 896, expected replay 7.5 tokens under the declared model.

16. **Index cost.** Do T dictionary probes imply $O(T)$ work? *Solution:* not when encoding prefixes processes quadratically many IDs.

17. **Approximation.** Bound error for $\alpha = 1/2$, $\epsilon = 0.01$, $\eta = 0$ after three steps. *Solution:* at most 0.00125 in hidden infinity norm; identical argmax is not established.

18. **COW falsifier.** Design a two-writer test. *Rubric:* compare every read with independent copies; interleave distinct coordinate writes/releases; inject allocation failure; check mappings and live references during eviction.

19. **Benchmark.** Compare periodic and junction-aware checkpoints. *Rubric:* fix model, precision, request traces and quotas; report reuse, replay, metadata-inclusive bytes, restore traffic and latency distributions. Include low-overlap and cache-thrashing controls.

20. **Restart.** Extend the protocol to sampled generation. *Rubric:* enumerate RNG, sampler, constraint and output-commit state; specify failure boundaries, reject incompatible revisions, and test uninterrupted versus restarted traces. Durable publication requires crash tests.

33.13 What the next engine transformation must preserve

We have immutable inference snapshots, exact context/token binding, fresh-owner restore, independent branches and bounded prefix caching. Physical COW, authentication, durable publication and optimized device code remain outside the executable copy oracle.

Chapter 34 will gather several owned states into a batched transition and scatter results back. It must preserve each branch's values, exact token position and ownership despite padding, cancellation and changing batch membership. Throughput transformations are legal only when that per-request contract survives.

Selective glossary

Semantic boundary: point after a complete declared transition. **Checkpoint envelope**: bounded body plus integrity metadata. **Continuation equivalence**: equal behavior on a common legal suffix. **Prefix key**: context/token content index, not state acceptance. **Copy-on-write refinement**: physical sharing that preserves independent-copy observations. **Replay audit**: recomputation consistency check, not authentication.

Sources and further reading

Access checked 17 October 2026. The reference imports the actual Chapter 31 recurrence and Chapter 32 pool. It adds strict schema validation, native continuation, tenant-scoped restoration, a bounded LRU cache and independent rational, collision and failure oracles.

The primary documentation sections below were read for the specific contracts cited, not to reproduce an entire external engine or serialization system. The PyTorch stable URL redirected to its 2.14 documentation; local tests use installed PyTorch 2.10 CPU and do not call its object loader. Python documentation was read on the 3.13 branch. Live vLLM pages are dated access snapshots, not a pinned implementation or a measured comparative benchmark.

The continuation induction, approximation bound, branch arithmetic, cost model, code, figures and exercises are original teaching artifacts. Physical COW, authenticated checkpoint exchange, crash durability and production GPU concurrency are not implemented. Local tests and render inspection do not replace independent specialist or reader review.

CHAPTER 34

DOGMA batching and state-native kernels

In this chapter you will

- lift one native transition to independent batch rows and prove permutation legality
- implement inactivity as an identity, not as a zero token
- separate scheduler ticks from request-local token clocks and gather and scatter through fenced lanes
- bound numerical divergence and count bytes before claiming a kernel speedup

A request arrives with a valid checkpoint and two tokens left. Another has one token left; a third is waiting. We could execute each request separately. We would like to place independent transitions into a common operation, let short requests leave promptly, and reuse the vacated capacity. The question is not initially how many tokens per second this produces. It is whether changing the grouping changes any request's answer, ownership or token position.

Chapter 31 supplied a native transition; Chapter 32 supplied bounded owned storage; Chapter 33 supplied independent checkpoint branches. This chapter connects those *actual implementations*, rather than introducing a second detached recurrence. Its reference is CPU float64, inference-only and serialized. A state-native device kernel is a future lowering of this contract, not an implementation claimed by a Python matrix expression.

After working through the chapter, you should be able to derive the batched operator, explain the two relevant clocks, follow a lane from a pool handle to fenced installation, invalidate a plausible but wrong mask, and specify what a credible kernel benchmark would measure.

34.1 Begin with one state transition

Let the ordered vocabulary have V entries and the hidden state have D components. Chapter 31's restricted teaching model contains an embedding $E \in \mathbb{R}^{V\times D}$, retention $a \in [0,1)^D$, write parameter $w \in \mathbb{R}^D$, and output matrix $W \in \mathbb{R}^{D\times V}$. For token u, define

$$e = E_u, \qquad f(h,u) = a \odot h + \sigma(w \odot e) \odot e, \qquad z = f(h,u)W.$$

The carried record is not merely h. It also binds tenant/request owner, artifact identity and consumed-token position t. A live transition installs $(t+1, f(h,u))$. The pool handle additionally names a pool instance, slot and allocation generation. Shape-compatible state belonging to another owner is not an admissible input.

This model is an input-gated diagonal recurrence, not the complete proposed DOGMA architecture. It is nevertheless rich enough to expose state decay, nonlinear input gating, dense output reduction, continuation and ownership errors. Batching a larger native state would require lifting *every* carried field and proving that no omitted field couples otherwise independent requests.

34.1.1 A hand calculation before tensor code

Use the actual tiny model from Chapter 31:

$$E_0 = (0,0), \quad E_1 = (1,0), \quad E_2 = (0,1), \quad a = (1/2, 1/4), \quad w = (0,0).$$

The gate is $1/2$ in each component. Starting at zero, request A consumes $(1,2,1)$:

$$(0,0) \mapsto (1/2,0) \mapsto (1/4,1/2) \mapsto (5/8,1/8).$$

Request B consumes (2) and ends at $(0,1/2)$. Request C consumes $(1,1)$ and ends at $(3/4,0)$. These are exact binary fractions, so a rational oracle can validate both arithmetic and request accounting without calling the batched implementation.

34.2 The row lift and its legality

Stack B independent states into $H \in \mathbb{R}^{B\times D}$ and their embeddings into $U \in \mathbb{R}^{B\times D}$. Then

$$F_B(H,u) = H \odot a + \sigma(U \odot w) \odot U, \qquad Z = F_B(H,u)W.$$

Parameters broadcast across rows; they are shared read-only model data. Nothing averages, normalizes or communicates *between* requests.

Proposition 34.1 (Rowwise equivalence and request permutation)**.** *For independent rows under the same artifact, row r of F_B equals $f(H_r, u_r)$. For a row permutation matrix P,*

$$F_B(PH, Pu) = PF_B(H,u), \qquad Z(PH,Pu) = PZ(H,u).$$

These are real-arithmetic identities. Floating-point implementations may require a numerical tolerance for the output reduction.

Proof. Each hidden component uses only H_{rj}, embedding $E_{u_r j}$ and shared parameters a_j, w_j. The output matrix product contracts over the component index j, not the request index r. Permuting rows therefore merely renames independent evaluations. □

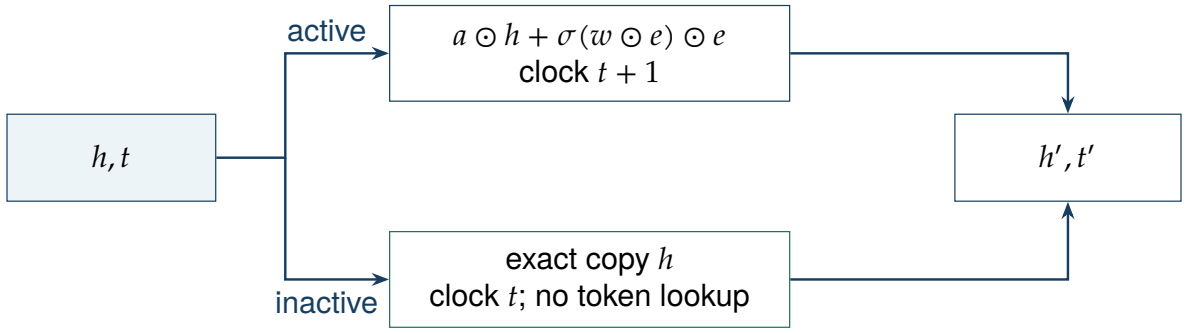
This proof fails for training-mode batch normalization over requests, a cross-request attention operator, a batch-wide top-k capacity decision or a shared mutable random stream whose consumption changes with lane order. Those operations may be useful, but they need a different equivalence statement. Row independence is a property to establish, not a synonym for having a batch dimension.

34.3 Inactivity is an identity, not a zero token

A **masked state transition** has the explicit semantics

$$h'_r = \begin{cases} f(h_r, u_r), & m_r = 1, \\ h_r, & m_r = 0, \end{cases} \qquad t'_r = t_r + m_r.$$

An inactive lane neither looks up a token nor publishes logits. The numerical array returned by the reference uses zero logits as an internal placeholder; the engine returns `None`, not that placeholder, for an unpublished lane.



Zero input is not inactivity: $(1, 2) \mapsto (1/2, 1/2)$ under retention $(1/2, 1/4)$.

Figure 34.1: The inactive path bypasses transition arithmetic and clock advancement. A zero embedding does neither.

For $h = (1, 2)$, token 0 has zero embedding, yet a live transition produces $(1/2, 1/2)$. Treating padding as token zero would silently decay the state and advance its local clock. Conversely, computing both branches of an unsafe expression and applying a final selection is not sufficient: an inactive out-of-range ID must not trigger an embedding lookup first.

The implementation validates live IDs before arithmetic and gathers only active embeddings. Hidden payloads must still be finite, declared-width CPU float64 tensors, even for inactive lanes. This makes inactivity an identity on a valid state, not a mechanism for hiding corrupted state.

```
@torch.no_grad()
def batched_step(model, hidden, tokens, active):
    """Only active lanes read token IDs; inactive state is an exact identity."""
    native = native_module()
    native.artifact_id(model)
    if hidden.ndim != 2 or not 1 <= hidden.shape[0] <= 256:
        raise ValueError("bounded nonempty [B,D] hidden")
    b, d = hidden.shape
    native.tensor(hidden, (b, model.embedding.shape[1]))
    if (
        type(tokens) not in (tuple, list)
        or type(active) not in (tuple, list)
        or len(tokens) != b
        or len(active) != b
        or any(type(x) is not bool for x in active)
    ):
        raise ValueError("matching concrete tokens and boolean activity")
    for token, live in zip(tokens, active):
        if live and (type(token) is not int or not 0 <= token <
         ↪  len(model.vocabulary)):
            raise ValueError("live token outside vocabulary")
    rows = [i for i, live in enumerate(active) if live]
    result = hidden.clone()
    logits = torch.zeros(b, len(model.vocabulary), dtype=torch.float64)
    if rows:
        e = model.embedding[[tokens[i] for i in rows]]
        candidate = model.retention * hidden[rows] + torch.sigmoid(model.write *
         ↪  e) * e
        z = candidate @ model.head
        native.tensor(candidate, (len(rows), d))
        native.tensor(z, (len(rows), len(model.vocabulary)))
        result[rows], logits[rows] = candidate, z
```

```
    return result, logits
```

This function has no ownership authority. It only transforms validated numerical arrays. Ownership is enforced by the adapter around it.

34.4 Two clocks and changing membership

A **scheduler tick** selects a group of ready requests. A **request-local token clock** counts accepted transitions for one owner. The clocks need not have equal values. In the worked trace, a stable-priority selector chooses the first two unfinished requests. It is deliberately simple enough to calculate by hand; it is not a fairness policy.

A. Scheduler tick is not a request's token position

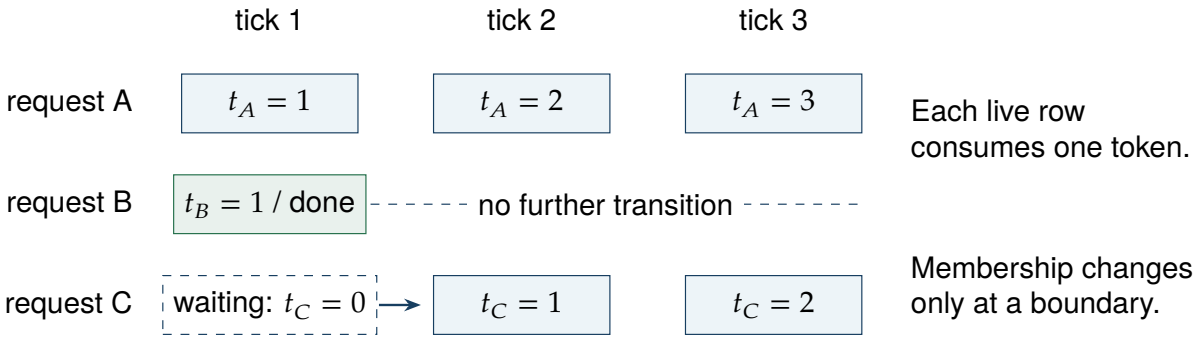


Figure 34.2: A short request leaves and another enters without resetting the continuing request's clock.

Tick	Members	t_A	t_B	t_C
1	A,B	1	1	0
2	A,C	2	1	1
3	A,C	3	1	2

Tick 2 is C's first transition, not its second. B's completed state is not advanced at ticks 2 or 3. The final hidden states agree with the three scalar calculations in Section 34.1.

A scheduler may group work by artifact, state geometry, dtype and backend compatibility. Within a group, requests can have different prefix lengths: this particular native recurrence carries its sufficient hidden state and position, not a padded tensor of every previous token. That does not imply that every possible DOGMA extension has constant-size sufficient state.

34.4.1 Fairness needs an explicit policy

The supplied `RoundRobin` chooses ready-list indices from the position after the last chosen index and wraps around. For a fixed set of N continuously ready entries and capacity B, a sweep visits every entry within $\lceil N/B \rceil$ selection calls. This is a bound in *selection calls*, not milliseconds, and assumes the caller does not reorder or replace entries. An indexed ready list is not a durable request registry.

```
    def choose(self, ready, limit):
        if (
            type(ready) not in (tuple, list)
            or any(type(x) is not bool for x in ready)
            or type(limit) is not int
            or not 1 <= limit <= 256
        ):
            raise ValueError("boolean ready list and positive bounded limit")
        if not ready:
            return ()
        order = [(self.next + i) % len(ready) for i in range(len(ready))]
        selected = tuple(i for i in order if ready[i])[:limit]
        if selected:
            self.next = (selected[-1] + 1) % len(ready)
        return selected
```

New admission, request priority, variable-duration kernels, quotas and preemption can invalidate a simplistic latency interpretation. A real service needs stable request identities, admission control, backpressure and tail measurements. Iteration-level scheduling in Orca is a relevant systems comparison, not a source of DOGMA speed claims [81].

34.5 Gather with identity; scatter through a fence

A temporary row number is not an allocation identity. A **lane map** preserves each row's original handle, caller owner, pin, candidate state and output. Changing the row order for locality must change the lane map in the same way.

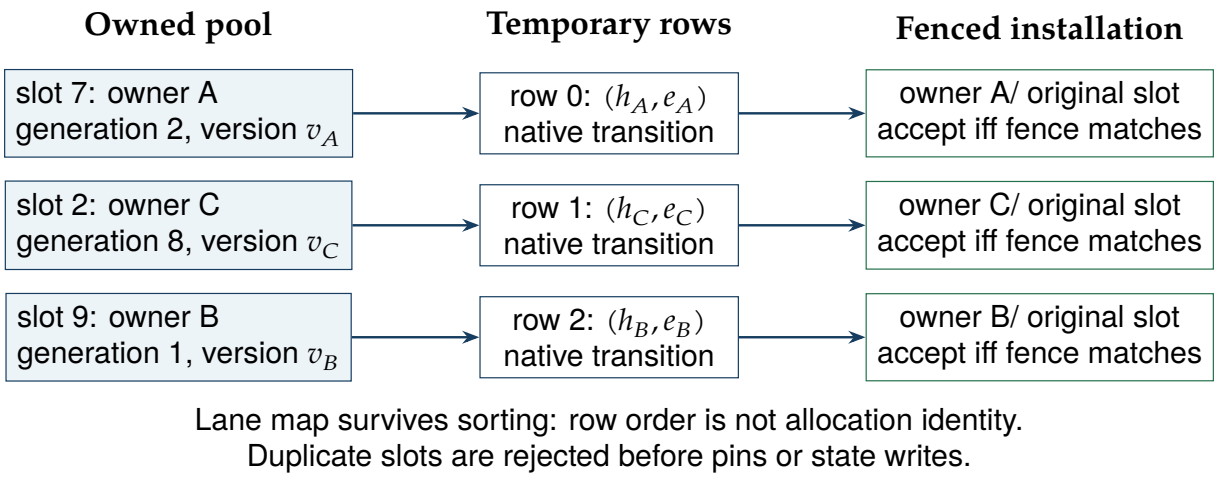


Figure 34.3: Temporary row order may change. Pool, owner, generation and completion version may not be guessed from that order.

The adapter imports Chapter 33's bridge, which imports the actual Chapter 32 pool and Chapter 31 model. Preparing a batch has four conceptual phases: validate the entire group; gather copied snapshots; compute finite candidates; then acquire pins and register the exact prepared job. Under the reference's serialized exclusive-call assumption, the snapshots cannot change between these phases. A concurrent backend would need a reservation/locking protocol or a retry on snapshot versions; this ordering alone is not such a protocol.

Every handle is checked even for inactive lanes. Duplicate slots are rejected, including an active/inactive alias, because otherwise the promised inactive identity could refer to a slot

simultaneously changed by its sibling. An already pinned slot is rejected by this adapter, although Chapter 32's more general pool can represent multiple outstanding pins. One in-flight batch per slot is a deliberately narrower engine policy.

```
    def prepare(self, lanes):
        """Preflight and compute before acquiring pins; serialized calls only."""
        self._context()
        if type(lanes) not in (tuple, list) or not 1 <= len(lanes) <= 256:
            raise ValueError("bounded nonempty lane group")
        slots, snapshots = set(), []
        for lane in lanes:
            if not isinstance(lane, Lane) or type(lane.active) is not bool:
                raise ValueError("lane contract")
            slot = self.pool._slot(lane.handle, lane.owner)
            if lane.handle.slot in slots:
                raise ValueError("duplicate slot, including inactive aliases")
            if slot.pins:
                raise RuntimeError("one in-flight batch per slot")
            slots.add(lane.handle.slot)
            snapshots.append(self.pool.read(lane.handle, lane.owner))
        hidden = torch.tensor([s.hidden for s in snapshots], dtype=torch.float64)
        h, z = batched_step(
            self.model, hidden, [x.token for x in lanes], [x.active for x in
             ↪  lanes]
        )
        p = pool_module()
        candidates = tuple(
            p.Snapshot(s.position + int(x.active), tuple(row.tolist()))
            for s, x, row in zip(snapshots, lanes, h)
        )
        pins = tuple(
            self.pool.pin(x.handle, x.owner) if x.active else None for x in lanes
        )
        job = Job(
            self.pool.artifact,
            tuple(lanes),
            pins,
            candidates,
            tuple(tuple(row.tolist()) for row in z),
        )
        self._pending[id(job)] = job
        return job
```

A failed live token, foreign caller, stale generation, busy slot, wrong artifact or numerical overflow is detected before this function acquires any pins. This is preflight atomicity under expected validation failures, not a guarantee of recovery from process death or an allocation failure at an arbitrary machine instruction.

34.5.1 Completion is independently conditional

A **fenced scatter** installs a lane only through its original pool pin. The pool checks the registered pin identity, owner, generation and snapshot version. A retired request cannot accept a result, even when the computed numbers are correct. An inactive lane has no pin and cannot commit.

```
    def finish(self, job, discard=False):
        """Settle each lane once; only accepted lanes expose model logits."""
        if self._pending.get(id(job)) is not job or type(discard) is not bool:
            raise RuntimeError("foreign/duplicate completion or discard policy")
        try:
            self._context()
            context_ok = True
        except ValueError:
            context_ok = False
        accepted, published = [], []
        for lane, pin, candidate, logits in zip(
            job.lanes, job.pins, job.candidates, job.logits
        ):
            ok = (
                False
                if pin is None
                else self.pool.complete(
                    pin, lane.owner, candidate if context_ok and not discard else
                     ↪  None
                )
            )
            accepted.append(ok)
            published.append(logits if ok else None)
        del self._pending[id(job)]
        if not context_ok:
            raise ValueError("model changed; all prepared pins discarded")
        return tuple(accepted), tuple(published)
```

The engine registers the exact prepared job object; an equal-looking copied dataclass is not an issued completion capability. Its numerical candidates and logits are immutable tuples. This is accidental-misuse protection inside a trusted Python process, not authentication against hostile code that can modify engine internals.

Logits returned for an accepted lane are available to a caller; this reference does not implement Chapter 30's network-output reservation and publication transaction. Integrating the two must bind state acceptance and output acceptance to the same request revision. Do not infer that simply returning a tuple has already solved streaming-service publication.

34.6 Cancellation separates rejection from reclamation

Suppose A and B are prepared together. A is cancelled before acknowledgement. Chapter 32's release marks A's slot retiring; its pin prevents physical reuse. On completion, A's candidate is rejected and its pin settles. Only then can its storage be scrubbed and its generation advanced. B may commit normally.

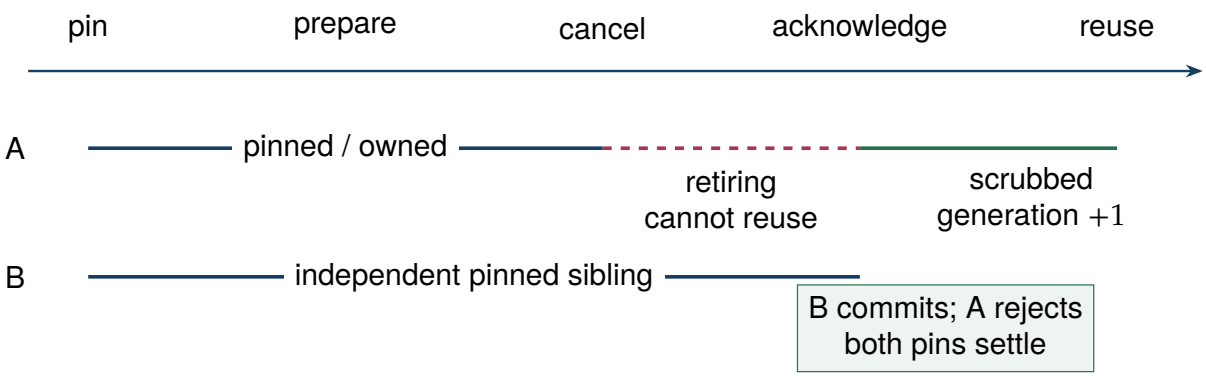


Figure 34.4: A cancelled lane rejects its result; a sibling can commit. Both outstanding pins must still be acknowledged.

This is not an all-or-nothing transaction across a batch. The two lanes represent different requests, with independently decided acceptance. A caller that needs group atomicity must build and prove that policy separately. Neither a Python loop over lanes nor a single GPU launch creates a cross-request transaction.

If a model's contents change between prepare and finish, the adapter detects the changed artifact, discards every registered pin and raises an error. Prepared logits must not escape as current-artifact outputs. A production system normally avoids this by retaining immutable versioned model instances until all users have finished.

The caller must finish or explicitly discard every issued job. Dropping a Python reference to an unfinished job is not an acknowledgement and can leave slots pinned indefinitely. This chapter supplies neither a durable work journal nor automatic recovery after a process crash.

34.7 Numerical equivalence is not bitwise identity

Let $\hat{h}_t$ be a backend's computed state and h_t the scalar reference for the same request and accepted token sequence. Suppose each live transition has componentwise implementation discrepancy at most ε, measured after applying the exact transition to the backend's prior state. For $\rho = \|a\|_\infty < 1$,

$$e_{t+1} \le \rho e_t + \varepsilon, \qquad e_t \le \rho^t e_0 + \varepsilon \frac{1-\rho^t}{1-\rho}.$$

This follows by subtracting the two updates: the input term is identical and only the retained state propagates earlier discrepancy. Inactive lanes add no error when implemented as exact copies. The bound is specific to this input-gated diagonal contract; state-feedback gates require another analysis.

Define $\|W\|_{1,\max} = \max_k \sum_j |W_{jk}|$. With a separately bounded output-reduction error η,

$$\|\hat{z} - z\|_\infty \le \|W\|_{1,\max} e_t + \eta.$$

If the best reference logit exceeds the second-best by more than twice this bound, greedy selection is stable. Near a tie, close logits can still produce different token choices. After a changed token is fed back, the hypothesis of an identical input sequence no longer holds. Reporting only a small maximum error while omitting token divergence is incomplete.

The tests use exact rational fixtures and float64 scalar/batch comparisons; they do not certify every low-precision, fused or parallel reduction backend.

34.8 Layouts, two axes and a state-native kernel boundary

For row-major $H[B, D]$ with element width b, the address of (r, j) relative to the buffer start is $(rD + j)b$. Threads spanning components within a row see contiguous storage; threads spanning requests at a fixed component see stride Db. A transposed representation reverses that trade-off.

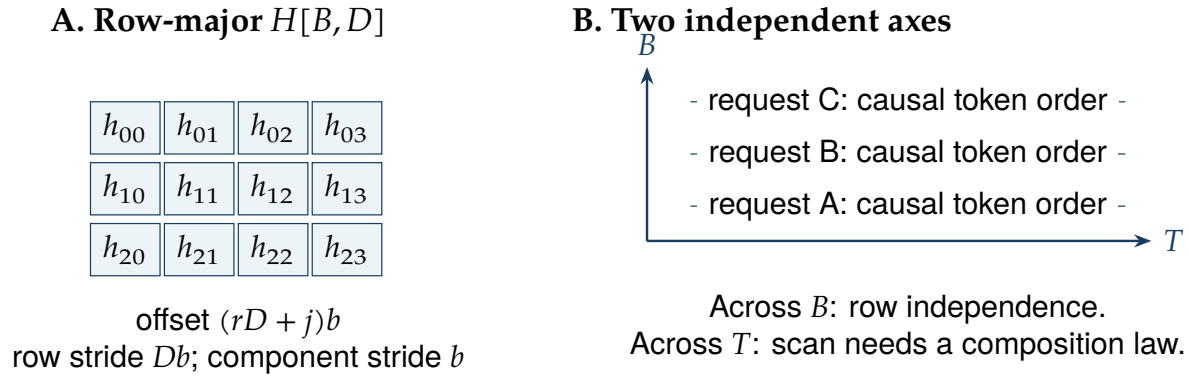


Figure 34.5: Memory stride is a concrete property. Request batching and token-axis scan solve different parallelism problems.

For a multi-field state, an array of structures groups fields per request, while a structure of arrays groups the same field across requests. Neither layout is universally optimal. The kernel's thread mapping, state-bank width, alignment and actual access pattern determine useful transaction locality. NVIDIA's guidance motivates coalescing and profiling, not choosing a layout without measuring it [82].

Request-axis batching uses row independence. Sequence-axis scan, by contrast, must preserve the ordered composition of a request's transitions. Chapter 26 owns that derivation. Combining the axes is legal only if both arguments remain true; a broadcast batch dimension does not make an arbitrary recurrent operator associative.

34.9 Count bytes before claiming a faster kernel

For this embedding-driven recurrence, count the logical reads of e and h, writes of h' and logits z, and omit shared parameter traffic:

$$M_{\text{base}} = B(3D + V)b.$$

This is a lower-scope payload model for a resident fused region. It is not the earlier dense-input-projection formula $B(I + 2D + O)b$: our actual model uses an embedding lookup and elementwise gate, not XW as a hidden-state projection.

A staged gather reads pool hidden values and writes a temporary H; staged scatter reads candidates and writes the pool. Those additional transfers contribute $4BDb$. Materializing and rereading one additional D-wide intermediate contributes another $2BDb$. A simple operation model counts approximately $2BDV + 4BD$ multiply/add operations, with sigmoid evaluation reported separately rather than silently equated to one FLOP.

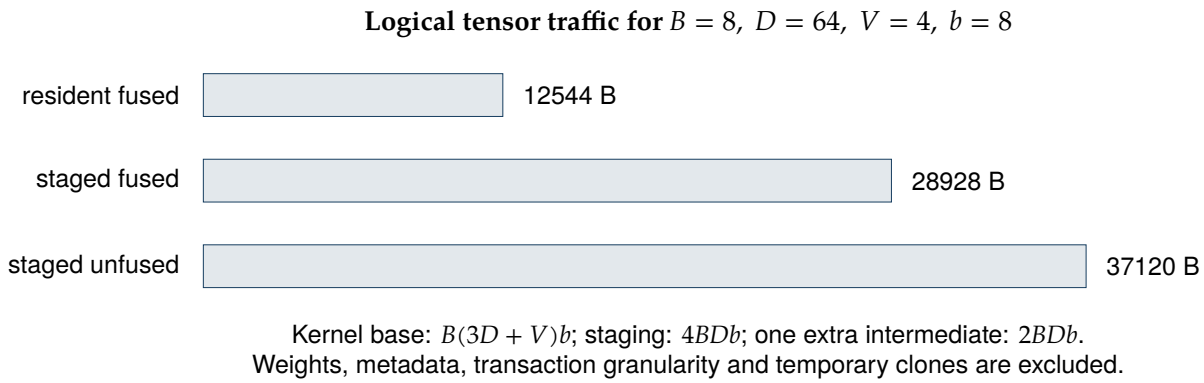


Figure 34.6: Separating kernel payload, staging and one intermediate prevents a misleading fusion-only comparison.

Policy	Base B	Stage B	Temp B	Total B
resident fused	12544	0	0	12544
staged fused	12544	16384	0	28928
staged unfused	12544	16384	8192	37120

The generated example has $B = 8, D = 64, V = 4, b = 8$ and 6,144 modeled multiply/add operations, excluding sigmoid. The corresponding idealized arithmetic intensities are about 0.490, 0.212 and 0.166 modeled operations per byte for the three policies. These ratios expose the staging penalty but are not a hardware roofline result. The actual Python reference uses indexed tensors, clones, tuple conversion and hashing, all of which add work not represented by this payload model.

Fusion may avoid a temporary while increasing registers, reducing occupancy or specializing too many shapes. A state-native kernel should consume the declared sufficient native state and preserve its semantics. The name alone does not imply optimal layout, fixed memory for all model variants, or a performance advantage over a Transformer.

34.9.1 What a subsequent backend must report

A credible experiment fixes artifact, dtype, geometry, vocabulary, active-lane fraction, batch-size distribution, prefix lengths and request traffic. It records hardware, driver/compiler/backend versions, warmup and synchronized timing scope. Measure useful accepted transitions, not allocated lane capacity; separate rejected/cancelled work, gather/scatter, kernel execution and output publication. Report latency distributions and throughput together. Compare resident and staged paths at equal semantics and include correctness checks across shapes, masks, permutations, overflow and cancellation.

This chapter runs no CUDA/Triton kernel and reports no device throughput. The source-linked byte table is arithmetic evidence, not an empirical speedup.

34.10 Reproduce, break and explain

Run the chapter builder to generate the literal function listings and tables, then the chapter tests. The tests cover actual scalar/batched parity across seeds and widths; all 81 length-four streams of the three-token tiny model; inactive canaries; duplicate aliases; cross-tenant lane

separation; stale and foreign handles; model changes; cancellation and deferred reuse; request permutations; and checkpoint-restored branches from Chapter 33.

The checkpoint integration is important: left and right branches start at $(5/8, 1/8)$, then consume tokens 0 and 2 in the same batch. They end at $(5/16, 1/32)$ and $(5/16, 17/32)$, while the immutable prefix remains replay-consistent. This is a tested continuation chain through three preceding interfaces, not merely two algebraic formulas that happen to resemble each other.

Key ideas

- Row independence must be established. Batch normalization, batch-wide top-k or shared random streams break it.
- Masking must happen before the embedding lookup. Feeding token zero to an inactive lane still decays its state and advances its clock.
- Each lane's acceptance is fenced separately: a cancelled lane rejects its result while a sibling commits. A batch is not a cross-request transaction.
- Staging adds $4BDb$ bytes and roughly halves arithmetic intensity in the worked example. These are modelled bytes, not a roofline measurement.

34.11 Twenty worked exercises

1. **Derive the output shape.** Why does $H'W$ produce one vocabulary vector per request? *Solution:* $[B, D][D, V] = [B, V]$ contracts only D. Contracting over B would mix independent owners and violate the row proof.

2. **Compute a second C transition.** Start at $(1/2, 0)$ and consume token 1. *Solution:* retention gives $(1/4, 0)$ and the write adds $(1/2, 0)$, hence $(3/4, 0)$.

3. **Find the mask counterexample.** Why does token zero not preserve $(1, 2)$? *Solution:* the write vanishes, but retention remains. The result is $(1/2, 1/2)$ and a live call also advances the clock.

4. **Ignore an invalid inactive ID safely.** Where must masking occur? *Solution:* before embedding lookup, by collecting active indices. A final selection after an invalid lookup is too late.

5. **Prove permutation legality.** Which reduction index matters? *Solution:* the output contraction is within a row over D; neither hidden updates nor outputs reduce over request rows.

6. **Break that proof.** Add a mean over batch hidden states to every update. *Solution:* each owner's answer now depends on membership. Scalar execution is no longer the correct oracle for the modified model.

7. **Distinguish clocks.** At tick 2, what is C's position? *Solution:* one after its first accepted transition, although the global scheduler has run twice.

8. **State a fairness assumption.** Does round robin guarantee latency? *Solution:* only a selection-call bound for a fixed continuously ready cohort and stable indexing; kernel duration and admission remain unspecified.

9. **Explain duplicate inactive aliases.** Why reject them? *Solution:* another lane could mutate the same physical state while the inactive lane promises identity. Conservative rejection avoids ambiguity.

10. **Identify preflight atomicity.** Does a bad final token pin earlier lanes? *Solution:* no; complete group validation and arithmetic precede pin acquisition in this serialized reference.

11. **Cancel one lane.** Can its sibling commit? *Solution:* yes; acceptance is independently fenced per request. Do not describe the batch as a cross-request atomic transaction.

12. **Reclaim cancelled storage.** When may its generation advance? *Solution:* after the final registered pin acknowledges. Terminal rejection does not itself prove that physical work can no longer access memory.

13. **Change a model parameter in flight.** What happens here? *Solution:* content identity changes; completion discards every issued pin and raises rather than returning stale-artifact logits.

14. **Derive an error bound.** Let $\rho = 1/2$, $e_0 = 0$ and per-step discrepancy ε. *Solution:* $e_t \le 2\varepsilon(1 - 2^{-t})$, below 2ε; this is not a universal DOGMA bound.

15. **Preserve greedy output.** A logit error bound is δ. What sufficient margin is required? *Solution:* more than 2δ, because the leader can decrease and its competitor increase by δ each.

16. **Count staging bytes.** Why $4BDb$ rather than $2BDb$? *Solution:* gather reads and writes one state tensor, and scatter reads and writes another. The kernel's own state read/write is already in the base.

17. **Count a temporary.** For the displayed example, what does one additional hidden intermediate cost? *Solution:* $2(8)(64)(8) = 8192$ logical bytes, for its write and reread.

18. **Choose layout responsibly.** Is a transposed hidden tensor always faster? *Solution:* no; it changes strides. Match thread/component mapping and measure transaction efficiency and end-to-end overhead.

19. **Connect checkpoints.** Why can two restored branches batch independently? *Solution:* restore gives distinct fresh allocations with copied payloads; each lane has its own owner and fence.

20. **Design the missing service test.** What is not proved by pool acceptance? *Solution:* streaming output publication and backpressure. Bind the native result and the runtime shell to the same accepted revision, then test terminal races and output conservation.

34.12 Handoff: a different representation of serving state

DOGMA's restricted recurrence carries a fixed-width vector. Hermon DNA is a Transformer: causal serving retains ordered keys and values at every layer. Chapter 35 will preserve the same causal model under prefill and decode, including its actual position convention and residual stack. Only after that oracle is established should Chapter 36 change the physical allocation into pages. Reusing an engine slogan is not a substitute for deriving each model's state.

Sources and further reading

Access checked 17 October 2026. The reference builds on the actual Chapter 31–33 native, pool and checkpoint contracts rather than on a detached recurrence or slot dictionary. Its tests, tables, code listings and illustrations are bound to those sources.

The USENIX Orca presentation-page abstract was inspected for iteration-level scheduling, not for a reproduced serving implementation. NVIDIA's live CUDA Best Practices Guide 13.4 was inspected on profiling, verification and coalesced accesses. No external benchmark is reproduced or transferred to DOGMA. Mathematical derivations and arithmetic models in this chapter are original and explicitly restricted to the declared recurrence.

The CPU reference is not a fused device kernel, concurrent service, durable work journal, authenticated ownership system or measured genomic model. Local technical and visual review does not replace independent review.

PART VIII

Hermon DNA Engine: attention-native inference

How is the actual Hermon DNA Transformer served exactly through caches, pages, schedulers and fast attention?

Part VIII keeps the real Chapter 23 multi-layer Transformer, with its learned absolute positions, and changes only how it is executed. Prefill and offset-masked decode preserve the full causal computation. Paged KV adds copy-on-write sharing and safe reclamation, and continuous batching adds explicit units for work, storage and latency. Speculative decoding preserves the target law exactly, and online softmax preserves the attention result exactly. Each step is checked against the original full model, layer by layer.

CHAPTERS IN THIS PART

35 Hermon DNA prefill, decode and KV state Preserve full Hermon semantics through prefill, chunks and decode.

36 Paged KV, prefix sharing and allocation Refine exact ordered KV into pages, sharing and safe allocation.

37 Continuous batching and precision Schedule mixed attention workloads and evaluate precision trade-offs.

38 Speculative decoding and attention kernels Test speculative verification and optimized attention kernels.

CHAPTER 35

Hermon DNA prefill, decode and KV state

In this chapter you will

- locate the state a causal Hermon DNA Transformer must retain: per-layer normalized keys and values
- derive prefill and offset-masked chunk continuation for the actual two-layer reference
- prove full/cached equivalence through the complete stack by induction over layers and chunks
- count resident KV payload separately from the quadratic cost of repeated copying

A Transformer can recompute the entire consumed prefix before predicting each next token. That is a useful correctness oracle and a costly serving strategy. A key/value cache retains intermediate results that future queries need, avoiding repeated old-token projections. But the cached numbers must come from the right model, layer, position convention and causal prefix. A tensor with the right shape can still be the wrong state.

This chapter implements caching for the *actual Chapter 23 Hermon DNA reference*: a pre-normalized Transformer with learned absolute positions, layer normalization, attention, residuals, GELU feed-forward sublayers, final normalization and an ACGT language-model head. We do not silently replace it with a single-head rotary-attention toy. The restricted executable fixture uses two layers, two heads, width eight, CPU float64 and deterministic evaluation. All parameters are assigned by a seeded initialization; no trained genomic capability is asserted.

We will first identify what persists, derive prefill and offset-masked chunk continuation, prove equivalence through the complete stack, and then count payload and copying costs. Chapter 36 may change allocation, but not these logical semantics.

35.1 Specify which Hermon computation is being served

Chapter 23 supports causal logits and offline full-record classification. Only its *causal* language-model path admits the ordinary left-to-right cache developed here. Offline attention or reverse-complement averaging can depend on unseen record suffixes. Their state is not interchangeable with a causal cache.

A request is a single unpadded canonical ACGT token record $u_0, \dots, u_{t-1}$; the mapping is $A = 0, C = 1, G = 2, T = 3$. Ambiguous bases, k-mer spans, multiple packed records and padded ragged batches are outside this adapter. Chapter 23's validity contract remains available in the full model, but the serving adapter does not pretend to implement all of it. A new record starts with an empty cache; carrying another record's history changes the conditional distribution.

The original model forms

$$x_i^0 = E_{u_i} + P_i,$$

where P_i is a learned position embedding. Position is the consumed-token ordinal in this unpadded contract. For variable-span genomic tokens it would not automatically equal a base

coordinate; that mapping must come from the data/coordinate contract, not a kernel's array index.

35.2 Trace one block and locate its retained state

Let $x_i^\ell \in \mathbb{R}^D$ be the input row of block ℓ. With H heads and $d_h = D/H$, the block computes

$$\begin{aligned} r_i^\ell &= \mathrm{LN}_1^\ell(x_i^\ell), \\ (q_i^\ell, k_i^\ell, v_i^\ell) &= \mathrm{QKV}^\ell(r_i^\ell), \\ y_i^\ell &= x_i^\ell + W_O^\ell \,\mathrm{MergeHeads}(\mathrm{Attention}_i^\ell) + b_O^\ell, \\ x_i^{\ell+1} &= y_i^\ell + \mathrm{FFN}^\ell(\mathrm{LN}_2^\ell(y_i^\ell)). \end{aligned}$$

The displayed output projection uses the mathematical column-vector convention. The reference uses PyTorch's linear layers and row tensors; it does not transpose or discard the learned biases.

Each key and value is computed from the *current layer's normalized activation*, not directly from the original token embedding. At layer two, that activation already includes layer one's causal attention and residual processing. Caching only first-layer token projections and reusing them at every layer is therefore wrong, even when dimensions happen to match.

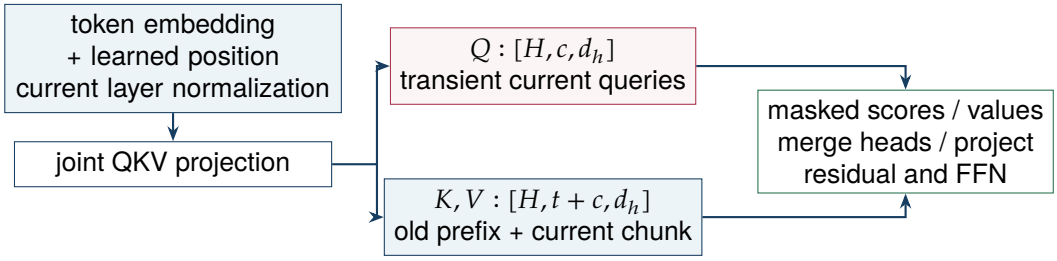


Figure 35.1: Query rows are temporary. Ordered K/V rows persist separately at each normalized attention projection.

For one head,

$$s_{ij} = \frac{\langle q_i, k_j \rangle}{\sqrt{d_h}}, \qquad a_{ij} = \frac{\exp s_{ij}}{\sum_{r \le i} \exp s_{ir}}, \qquad o_i = \sum_{j \le i} a_{ij} v_j.$$

Softmax is taken over allowed keys, not over heads, channels or requests. Use max subtraction for stable evaluation; the library softmax in the reference supplies that operation. Every query in this contract has at least its own current key, so it never needs an all-masked-row fallback.

The scaled dot-product and multi-head construction follows the Transformer formulation [6]. The particular pre-normalized architecture and learned-position choice belong to our existing reference, not to a claim that every Transformer has these exact conventions.

35.2.1 A small retrieval calculation

Suppose one query has three allowed scores $(0, \log 2, 0)$ and values $(1,0), (0,2), (2,1)$. The normalized weights are $(1/4, 1/2, 1/4)$, giving

$$o = (3/4, 5/4).$$

This calculation clarifies why caching old keys and values is enough: the new query needs the old *addresses and contents*, not the old queries. The chapter tests this assigned retrieval independently of the model fixture.

35.3 Define the cache before optimizing it

A **per-layer KV cache** at consumed length t contains

$$C_t^\ell = (K_t^\ell, V_t^\ell), \qquad K_t^\ell, V_t^\ell \in \mathbb{R}^{H \times t \times d_h}.$$

This is retained addressable history. It is neither Chapter 31's compressed recurrent vector nor the Transformer parameters. A cache's position is the number of consumed tokens, not the number of generated tokens since prefill.

The envelope binds owner, artifact, exact causal mode, ordered consumed-token metadata and the ordered collection of layer caches. Artifact identity includes the actual model's parameter tensors, position table, geometry, normalization epsilon settings and GELU approximation convention. The reference requires evaluation-mode finite CPU float64 parameters and bounded geometry before it accepts a cache.

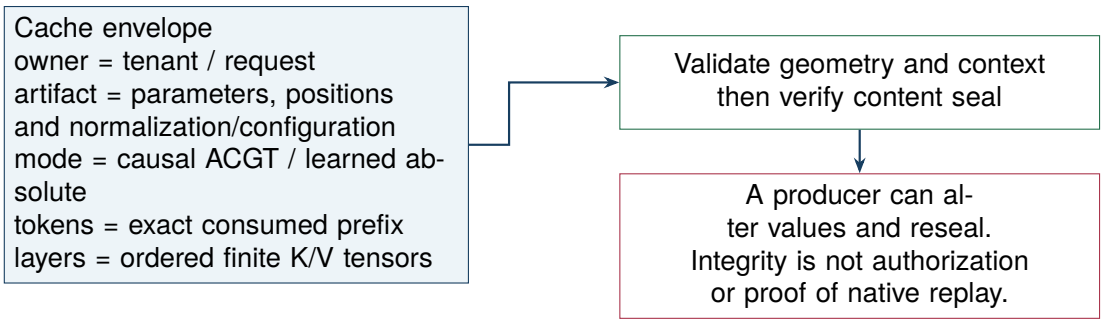


Figure 35.2: Cache correctness has context, geometry and integrity layers. None alone proves that stored values were produced by native replay.

The dataclasses are frozen, but tensors inside them are not intrinsically immutable. The adapter therefore creates fresh storage on continuation and checks a **cache content seal** over metadata and tensor values before use. This detects accidental mutation. A producer with access to both content and hashing can alter values and reseal them; the negative control demonstrates that such a structurally valid cache can change logits.

The seal is not authentication, a cryptographic authorization capability, a proof of correct prefix computation, hostile-process isolation or an untrusted-file loader. Nor does storing token IDs prove the tensors were derived from those IDs. Trusted producers and serialized callers are explicit assumptions.

```
def validate(model, cache, owner):
    owner_pair(owner)
    artifact = artifact_id(model)
    if not isinstance(cache, Cache) or cache.owner != owner:
        raise PermissionError("cache owner mismatch")
    if cache.artifact != artifact or cache.mode != MODE:
        raise ValueError("cache artifact/mode mismatch")
    if type(cache.tokens) is not tuple:
        raise ValueError("immutable consumed-token metadata")
    tokens_copy(cache.tokens, model.max_len)
    if type(cache.layers) is not tuple or len(cache.layers) != len(model.blocks):
        raise ValueError("exact per-layer cache")
    for layer, block in zip(cache.layers, model.blocks):
        if not isinstance(layer, LayerKV):
            raise ValueError("KV record required")
        for value in (layer.k, layer.v):
            if (
                not isinstance(value, torch.Tensor)
                or value.shape != (block.attn.h, len(cache.tokens), block.attn.dh)
                or value.device.type != "cpu"
                or value.dtype != torch.float64
                or value.requires_grad
                or not bool(torch.isfinite(value).all())
            ):
                raise ValueError("finite inference KV with declared geometry")
    if type(cache.seal) is not str or cache.seal != seal_for(cache):
        raise ValueError("cache content mutated; seal is not authentication")
```

Validation deliberately rejects dtype changes rather than silently converting the history. Casting a cache after construction changes the numerical contract; quantized cache designs require their own parity and quality experiments. Content hashing and tuple conversion are expensive teaching checks, not a proposed per-token production fast path.

35.4 Prefill constructs a causal history

Prefill consumes a prompt of T tokens starting from an empty cache. All prompt positions can be projected together, but query i can read only keys $j \leq i$. Parallel evaluation is not bidirectional information flow. Each layer stores its full prompt K/V, and the final normalized rows pass through the language-model head.

The last prompt row's logits predict the token *after* the prompt. When that selected token is later consumed by decode, its new logits predict the following token. Confusing predicted with consumed positions produces an off-by-one error that a shape check cannot detect. This chapter uses supplied tokens rather than a sampling loop to isolate execution parity.

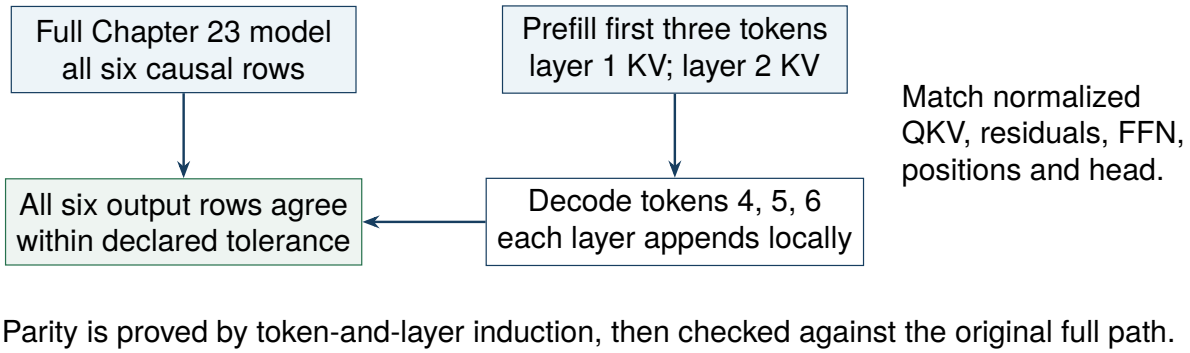


Figure 35.3: Different evaluation schedules must preserve all rows of the original full causal computation, not merely the final cache length.

The simplest independent oracle is the model's original `causal_logits` method. It performs complete full-prefix computation with its own square blocked-mask construction and never calls the cache append helper.

```
@torch.no_grad()
def full_oracle(model, tokens):
    artifact_id(model)
    tokens = tokens_copy(tokens, model.max_len)
    if not tokens:
        return torch.empty(0, 4, dtype=torch.float64)
    # Original full causal path: no cache, append helper or rectangular mask
     ↪ reuse.
    return model.causal_logits(torch.tensor([tokens], dtype=torch.long))[0]
```

Calling prefill twice through the same cache helper would not constitute an independent full-versus-cached oracle: the same offset or layer-order bug could appear on both sides.

35.5 Decode and causal chunks need the correct offset

Let t old tokens already be cached and let a continuation chunk contain c new tokens. New activations have shape $[c, D]$ and positions $t, \dots, t + c - 1$. At each layer,

$$Q_{\text{new}} \in \mathbb{R}^{H \times c \times d_h}, \qquad K_{\text{all}}, V_{\text{all}} \in \mathbb{R}^{H \times (t+c) \times d_h}.$$

Old rows and current rows concatenate in causal order. Local query index i is absolute position $t + i$, so its allowed-key condition is

$$A_{ij} = [j \le t + i], \quad 0 \le i < c, \quad 0 \le j < t + c.$$

The old prefix is entirely visible; only the future portion of the new chunk must be masked.

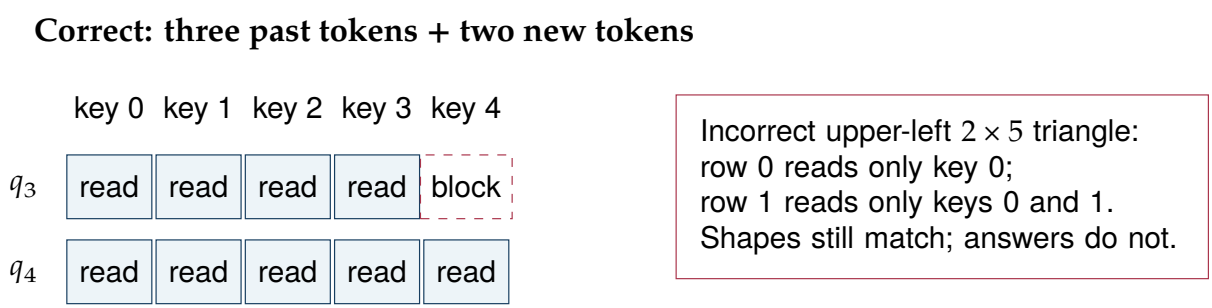


Figure 35.4: A rectangular continuation mask uses absolute query positions. An upper-left triangle can be shape-correct and semantically wrong.

```
def allowed_mask(past, chunk):
    if type(past) is not int or past < 0 or type(chunk) is not int or chunk < 1:
        raise ValueError("nonnegative prefix, positive chunk")
    return torch.arange(past + chunk)[None, :] <= (past + torch.arange(chunk))[:,
     ↪ None]
```

The reference's mask uses `True` to mean *allowed*. Chapter 23's `legality` helper uses `True` to mean *blocked*. The conversion in `consume` is explicit. PyTorch 2.10 SDPA accepts an allowed boolean mask; its default non-square causal alignment is upper-left, which is not this offset continuation mask [32]. Tests compare an explicit-mask SDPA oracle and deliberately show that the upper-left shortcut changes the result for a two-query, five-key continuation.

For **one-token decode**, $c = 1$ and all $t + 1$ stored/current keys are allowed. No triangular mask is needed then, *provided* the cache really contains only this causal prefix. The more general explicit-mask implementation covers both one-token decode and multi-token chunk continuation.

35.5.1 The complete append path

```
@torch.no_grad()
def consume(model, cache, owner, tokens):
    """Append a whole causal chunk; old cache remains untouched on every path."""
    validate(model, cache, owner)
    tokens = tokens_copy(tokens, model.max_len - len(cache.tokens))
    if not tokens:
        return torch.empty(0, 4, dtype=torch.float64), cache
    past, count = len(cache.tokens), len(tokens)
    ids = torch.tensor(tokens, dtype=torch.long)
    z = model.tok(ids) + model.pos(torch.arange(past, past + count))
    allowed = allowed_mask(past, count)
    layers = []
    for block, old in zip(model.blocks, cache.layers):
        a = block.attn
        raw = a.qkv(block.n1(z))
        q, k, v = raw.view(count, 3, a.h, a.dh).permute(1, 2, 0, 3)
        k = torch.cat((old.k, k), dim=1)
        v = torch.cat((old.v, v), dim=1)
        scores = (q @ k.transpose(-2, -1)) / math.sqrt(a.dh)
```

```
        weights = scores.masked_fill(~allowed, float("-inf")).softmax(-1)
        y = (weights @ v).transpose(0, 1).reshape(count, model.d)
        z = z + a.out(y)
        z = z + block.ff(block.n2(z))
        layers.append(LayerKV(k.clone(), v.clone()))
    logits = model.lm(model.norm(z))
    candidate = sealed(owner, cache.artifact, cache.tokens + tokens, layers)
    validate(model, candidate, owner)
    if not bool(torch.isfinite(logits).all()):
        raise ValueError("nonfinite model logits")
    return logits, candidate
```

A continuation call validates old state and the entire token chunk before arithmetic. It constructs candidates in new storage, checks finite logits and cache payloads, then returns a new cache. The caller decides whether to install it as the current request state. The old cache remains unchanged on success and on validation/numerical failure. This is functional candidate construction, not a thread-safe mutable cache transaction.

The special empty chunk returns zero rows and the exact same cache object. It is an identity, not an implicit BOS token. An empty prefill likewise produces no logits: a caller cannot select a next token from a nonexistent last row without defining a separate beginning-of-sequence convention.

35.6 Prove parity through tokens and layers

Proposition 35.1 (Full/cached causal equivalence)**.** *Fix model parameters and architecture, deterministic evaluation, token IDs, causal mode, position convention and record identity. Suppose a cache contains the exact per-layer K/V produced by full causal evaluation of the consumed prefix. Appending a valid continuation chunk computes the same new output rows and extended layer caches as full causal evaluation of the combined record, in real arithmetic.*

Proof. At layer zero, each new row receives the same token and absolute position embedding as full evaluation. Assume the old cached rows match full K/V and the new input activations match at the current layer. Normalization is row-local, so the new Q/K/V projections match. The offset mask selects exactly the same keys as the corresponding full causal row, including old prefix and allowed current-chunk rows. Softmax and value aggregation therefore match. The output projection, residual additions and feed-forward sublayer are row-local and use identical parameters, so the next layer's new inputs match. Induction over layers proves matching logits and extended K/V. Induction over chunks preserves the hypothesis for any continuation partition. □

Causality is essential to the old-cache hypothesis: appending future tokens cannot change old causal activations. A bidirectional layer would invalidate it. Deterministic evaluation is also essential to this argument as stated. The reference has no dropout modules; a future backend that introduces attention dropout must pass an explicitly zero inference dropout probability, not assume a surrounding evaluation flag overrides the functional API.

In floating-point arithmetic, full and chunk paths may use differently shaped matrix reductions. Tests use absolute and relative tolerances 2×10^{-12} in CPU float64. This is a checked fixture tolerance, not a universal backward-error theorem for every model, precision or device. Per-layer key/value checks narrow the location of a failure; final-logit parity alone can hide compensating mistakes.

Prefill	Max logit error	Final t	KV bytes
1	2.22045e-16	6	1536
2	3.05311e-16	6	1536
3	3.05311e-16	6	1536
4	1.94289e-16	6	1536
5	1.94289e-16	6	1536
6	0	6	1536

The table is generated from the two-layer six-token fixture. Every prefill split is followed by scalar decode through the actual stack. All end at six consumed tokens and 1,536 logical KV bytes. Separate tests cover every one of the $2^5 = 32$ nonempty chunk compositions of that record.

35.7 Positions cannot be repaired by tensor shape

Our reference's learned absolute embeddings enter *before* layer one. Restarting their ordinal at zero after prefill changes Q/K/V throughout the stack. Unlike standard rotary relative-score identities, learned position tables do not in general permit a uniform coordinate shift with invariant outputs. Importing such an identity from another architecture would be wrong.

A cache artifact therefore binds the position table and model semantics, and the next position is derived from consumed length. A negative control copies position-zero embedding into the next-position entry of a separately identified model: earlier prefix state stays comparable, but continuation logits change. Normal context validation would reject a mid-request artifact switch before computing those wrong logits.

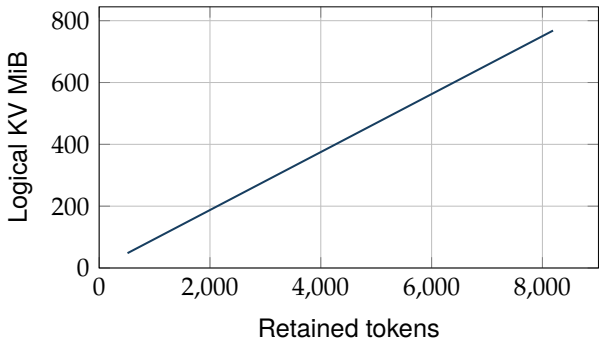
Changing strand orientation also changes the ordered token sequence and positions. A forward causal cache cannot simply be relabelled as a reverse-complement cache. An offline symmetry construction from earlier chapters is not a legal hidden transformation of a left-to-right serving state. A transformed-cache reuse rule would need a separate proof for the chosen model and readout.

35.8 Count resident payload separately from copying

For L layers, t retained tokens, H_{kv} key/value heads, head width d_h and element width b bytes,

$$M_{\mathrm{KV}}(t) = 2LtH_{kv}d_hb.$$

The factor two counts K and V. Query heads do not persist. With ordinary multi-head attention, $H_{kv} = H_q = H$; grouped-query and multi-query designs use fewer KV heads [84]. That changes the architecture and cache geometry. Our executable adapter implements the existing equal-head model, not a conversion to GQA, and asserts no quality-preserving head reduction.



$M(t) = 2LtH_{kv}d_hb$

Example: $L = 24$, $H_{kv} = 8$, $d_h = 128$, $b = 2$ bytes.

At 4,096 tokens: 384 MiB.

Repeated concatenation also copies old prefixes: quadratic total traffic over a long decode continuation.

Figure 35.5: Resident history grows linearly. Copying old contiguous prefixes on every append creates an additional, cumulative growth cost.

Tokens	KV bytes	MiB
512	50331648	48
1024	100663296	96
2048	201326592	192
4096	402653184	384
8192	805306368	768

The large arithmetic example uses $L = 24, H_{kv} = 8, d_h = 128, b = 2$: at $t = 4096$, $M = 402{,}653{,}184$ bytes or 384 MiB for one request. These dimensions describe a storage calculation, not the small executable fixture or a trained Hermon artifact. Parameters, activations, metadata, stored token IDs, allocator reserve and temporary copies are excluded. Actual device allocation must be measured separately.

35.8.1 The cost hidden by a simple concatenate

Define $\gamma = 2LH_{kv}d_hb$, the logical KV bytes per token. Starting with prompt length P, append G tokens one at a time. An implementation that copies the old prefix into a fresh destination performs old-prefix reads and equal-length writes totaling

$$C_{\text{old}} = 2\gamma \sum_{g=0}^{G-1} (P + g) = 2\gamma GP + \gamma G(G - 1).$$

This term is quadratic in generated length even though final resident payload $\gamma(P + G)$ is linear. New KV-row writes add γG; they are not included in C_{old}.

For the executable geometry, $\gamma = 256$ bytes. With $P = 3, G = 3$, the old-prefix read/write component is $2(256)(3 + 4 + 5) = 6144$ bytes. This is not the reference's measured total memory traffic: it additionally clones new layer tensors, creates temporary projections and hashes values. The formula isolates why repeated relocation becomes undesirable rather than pretending the Python code has achieved the lower bound.

35.9 Work, traffic and the limits of a serving comparison

A dense prefill of T tokens forms a $T \times T$ score array per head. Its mathematical causal dependency graph has $T(T + 1)/2$ allowed pairs, but the transparent implementation still

computes dense score and value products. A triangular dependency diagram is not proof that the code skips half the arithmetic.

For a new query at retained length t, score and value interactions require work proportional to td_h per query head; projection and feed-forward work remain. The amount of logically retained history that may need reading also grows with t. Whether decode is bandwidth-limited on particular hardware depends on batch size, cache reuse, dtype, weight traffic and kernel layout; it is a profile question, not an unconditional law.

FlashAttention demonstrates an IO-aware organization of exact attention that avoids materializing the full attention matrix in high-bandwidth memory [83]. This is a relevant algorithmic comparison. It does not mean this chapter's dense CPU function is FlashAttention or that finite-precision outputs from every backend are bitwise equal. Chapter 38 owns optimized attention-kernel experiments.

Comparing the fixed-width recurrence in Chapter 34 with this growing KV representation establishes a *state-geometry distinction*. It does not establish equal expressive capacity, genomic accuracy, latency, energy efficiency or a universal winner. Those require shared tasks, budgets, hardware and appropriate evidence from Chapters 25 and 29.

35.10 Verification and the allocator boundary

The local reference test suite checks every fixture prefill split across three model seeds, every chunk composition, original full-model QKV hooks at each layer, future-prefix invariance, fresh storage, empty identity, capacity rejection, model/position/configuration changes, invalid tokens, owner/mode mismatches, finite payload mutation, nonfinite payloads, wrong geometry/dtype/gradient state, and a resealed forgery.

A content seal can report a mutated tensor before use; it cannot explain whether a trusted producer supplied a false cache originally. Full native replay could check consistency but would itself cost prefix computation. The chapter neither authenticates arbitrary incoming cache files nor calls an executable object deserializer.

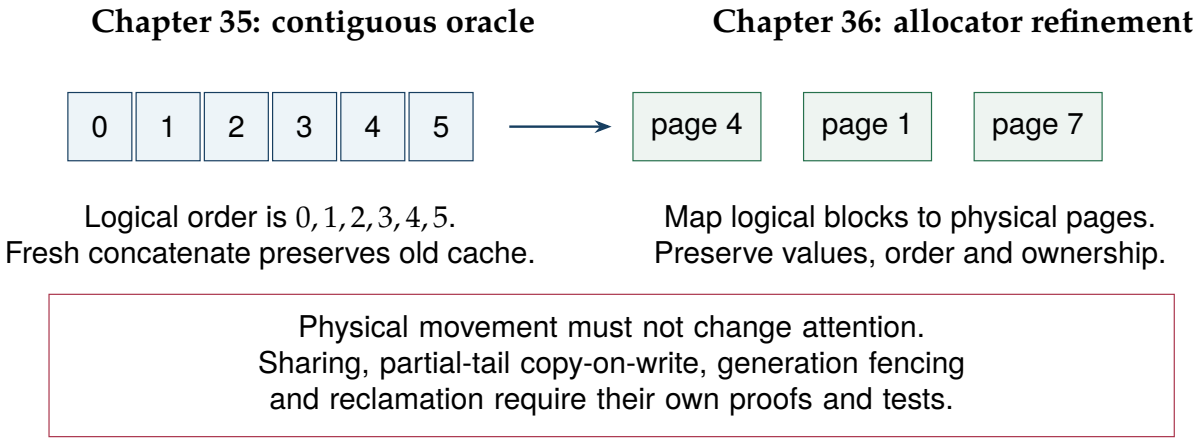


Figure 35.6: Chapter 36 may relocate K/V physically while preserving this chapter's exact logical history and ownership.

Preallocating a bounded contiguous buffer can avoid repeated relocation, but it creates capacity reservation and mutable-lifetime questions. Paging can address a different trade-off, allowing noncontiguous physical storage and prefix sharing. These are allocator refinements. They must preserve the ordered per-layer history visible to each query, artifact and coordinate

identity, and reference lifetime. They do not alter the attention formula to excuse a missing or duplicated key.

Key ideas

- Later-layer keys and values depend on earlier attention, residuals and feed-forward layers. Caching first-layer projections for every layer is wrong.
- A continuation query sits at absolute position $t + i$. PyTorch's upper-left causal shortcut is shape-correct and semantically wrong here.
- Learned absolute positions do not admit a rotary-style shift identity. Restarting the ordinal changes every layer.
- Resident KV grows linearly (384 MiB at $t = 4096$ in the large example), but copying the prefix on every append costs $2\gamma GP + \gamma G(G-1)$.

35.11 Twenty worked exercises

1. **Find the cache boundary.** Why cache normalized projected K/V, not token embeddings alone? *Solution:* later-layer K/V depend on earlier-layer attention, residuals and FFN. Embeddings alone would require recomputing that causal stack to recover them.

2. **Normalize three scores.** Evaluate $(0, \log 2, 0)$. *Solution:* exponentials are $(1, 2, 1)$; divide by four. The assigned values then aggregate to $(3/4, 5/4)$.

3. **Explain transient queries.** Why not retain old Q? *Solution:* future causal rows use new queries against old K/V. Old outputs are not recomputed in ordinary append-only decode.

4. **Derive offset legality.** For past length three and chunk two, which keys can the first new query read? *Solution:* absolute position three reads indices zero through three, not four.

5. **Expose the rectangular shortcut.** What does an upper-left 2×5 triangle do? *Solution:* its rows read only one and two keys. The correct offset mask reads four and five.

6. **Convert boolean conventions.** Chapter 23 returns blocked edges; SDPA accepts allowed edges. *Solution:* invert the boolean mask explicitly. Matching shapes do not reconcile opposite meanings.

7. **Locate the next token.** What do the last prompt logits predict? *Solution:* the token after the consumed prompt. Consuming that token then advances cache length and predicts another token.

8. **Handle empty input.** Does empty prefill supply a next-token distribution? *Solution:* no, it produces no output row. Define an explicit start-token policy if a generation API needs one.

9. **Prove layer induction.** What is the essential inductive hypothesis? *Solution:* old per-layer K/V and current new-layer inputs equal their full causal counterparts; row-local operations preserve equality.

10. **Count partitions.** Why 32 compositions for six tokens? *Solution:* each of the five internal gaps independently contains a chunk boundary or not.

11. **Test the whole model.** Is a single-head attention comparison enough? *Solution:* no; include layer-wise projections, residuals, normalizations, FFN, position embeddings and the final head.

12. **Shift absolute positions.** Does a uniform shift preserve this model? *Solution:* not generally; learned position rows are unrelated parameters. Do not import a rotary relative-position identity.

13. **Switch strands.** Can forward KV become reverse-complement KV by changing metadata? *Solution:* no. The sequence and causal positions change; a valid transformation would need a model-specific proof.

14. **Compute fixture payload.** Two layers, six tokens, two heads, head width four and eight-byte elements? *Solution:* $2(2)(6)(2)(4)(8) = 1536$ bytes, excluding all overhead.

15. **Compute the large example.** Convert 402,653,184 bytes to MiB. *Solution:* divide by 2^{20} to obtain 384 MiB, not 402.65 MiB.

16. **Separate head geometries.** Which head count controls KV? *Solution:* H_{kv}; query heads control a different part of attention work. Changing KV heads is not merely an allocator optimization.

17. **Derive copy growth.** Why quadratic cumulative traffic? *Solution:* each append recopies a prefix one token longer; summing $P, \ldots, P + G - 1$ creates $GP + G(G-1)/2$.

18. **Detect versus authenticate.** Can a producer reseal altered K/V? *Solution:* yes. Structural integrity accepts it, but a native replay/logit negative control exposes inconsistency.

19. **Describe a numerical guarantee honestly.** Does fixture float64 parity certify quantized GPU KV? *Solution:* no; dtype, kernel reductions, error propagation and downstream quality need new tests.

20. **State the paging invariant.** What must a page table preserve? *Solution:* exactly ordered layer/head/token K/V values, model and position identity, owner isolation and lifetime. Physical adjacency is not part of the attention semantics.

35.12 Handoff: retain the oracle while changing storage

Chapter 36 should compare paged allocation against this exact causal oracle, including partial final blocks, branch divergence, reference counting and reclamation. Eviction may remove a cache's ownership reference; it must not invalidate pages still used by a live request. Establishing those invariants will turn a storage diagram into an executable system argument without changing what the model computes.

Sources and further reading

Access checked 17 October 2026. The original Chapter 23 model was read and imported, including learned absolute positions, pre-normalized blocks, mask conventions and offline/causal separation. A rotary single-head attention toy would test a different architecture, so the chapter serves the actual model through a full-model adapter with explicit offset masks and layered cache validation. Chapter 36 refines its storage.

The Transformer HTML paper's attention and multi-head sections were inspected. PyTorch 2.10 SDPA documentation was read for boolean-mask polarity, non-square causal alignment, inference dropout and numerical backend limits; local tests use installed PyTorch 2.10 CPU float64. FlashAttention and GQA primary abstracts were inspected for the narrow comparisons cited, not reimplemented or independently benchmarked. The original parity

proof, cache adapter, copying derivation, figures and exercises do not reproduce those papers' empirical results.

This reference implements neither paging, GQA conversion, rotary positions, quantized KV, optimized attention kernels nor a trained genomic benchmark. Content seals are not authentication or replay proofs. Local source/test and render review remains distinct from independent specialist or reader review.

CHAPTER 36

Paged KV, prefix sharing and allocation

In this chapter you will

- translate logical token ordinals to physical pages through a block table
- conserve generational references and reference counts across fork, append and release
- copy-on-write a shared partial tail and preflight every page before publishing
- measure physical utilization and logical multiplicity separately

Two requests consume the same three-token prompt. One continues with A and the other with T. With two token positions per page, which bytes can they still share? The first full page is easy: neither continuation changes it. The second page is more subtle. It contains one valid prefix position and one vacant slot. Writing into that vacancy through a shared pointer would silently change the other request's history.

Chapter 35 made the ordered, per-layer Hermon cache explicit. This chapter changes physical placement while preserving that computation. Its allocator stores the actual two-layer Chapter 23 Transformer state through Chapter 35's adapter, including learned absolute positions. It is a serialized CPU reference with expensive validation and contiguous staging. It establishes allocation and continuation contracts; it is not a production paged kernel.

36.1 The logical object must survive relocation

For layer $\ell = 0, \dots, L-1$, head count H and head width d_h, a consumed prefix of T tokens has

$$K^\ell, V^\ell \in \mathbb{R}^{H \times T \times d_h}.$$

One token slice contains K and V across *all* layers and heads. The reference uses $L = 2, H = 2, d_h = 4$, eight-byte float64 elements: one such slice occupies $2LHd_h(8) = 256$ logical payload bytes. These are model tensors, not biological molecules. The DNA vocabulary supplies token identity; software allocation supplies the page mechanism.

Let C_T denote the tuple of layer tensors plus owner, model artifact, causal mode and ordered token metadata. The refinement obligation is

$$\text{flatten}(\text{paged}(C_T)) = C_T$$

for every layer and token ordinal, with fresh tensor storage returned by flatten. Physical page IDs may change. Logical order, learned-position ordinal and layer identity may not. Merely obtaining the right final length does not establish this equality.

Choose page capacity P token positions. Token ordinal t uses logical block $b = \lfloor t/P \rfloor$ and offset $o = t \bmod P$. A request's **block table** maps b to a physical reference. With $P = 2$, ordinal four is block two, offset zero, even when that block resides at physical page three. Attention must never sort page IDs to infer history.

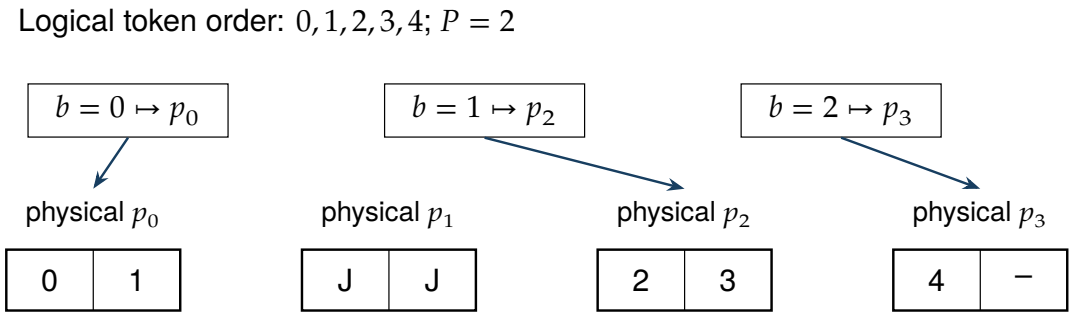


Figure 36.1: Logical ordinals pass through a block table to scattered storage. One occupied cell is the complete layer/head K/V slice at that ordinal.

The reference deliberately groups all layers into one allocation unit. A production engine may instead coordinate separate per-layer arenas, but then publication must make all layer tables consistent. Allocating one layer successfully and abandoning the others is not a valid half-cache.

36.2 References carry identity; counts carry lifetime

A **generational reference** is $\rho = (i, g)$: slot i and allocation generation g. Reclamation clears the page's logical payload and advances its generation. A stale reference to (i, g) fails after reuse as $(i, g + 1)$. This avoids an ABA mistake in which an old numeric slot ID appears valid again merely because a new object occupies the slot.

The request handle additionally contains an arena-instance identity and a monotonically issued serial. Released handles fail; a handle from another arena fails even when slot numbers happen to match. Owner checks bind a tenant/request pair. Refcounts alone do not supply that authorization.

Let $\mathcal{S}$ contain every registered live request, branch and cached prefix table. Our central conservation invariant is

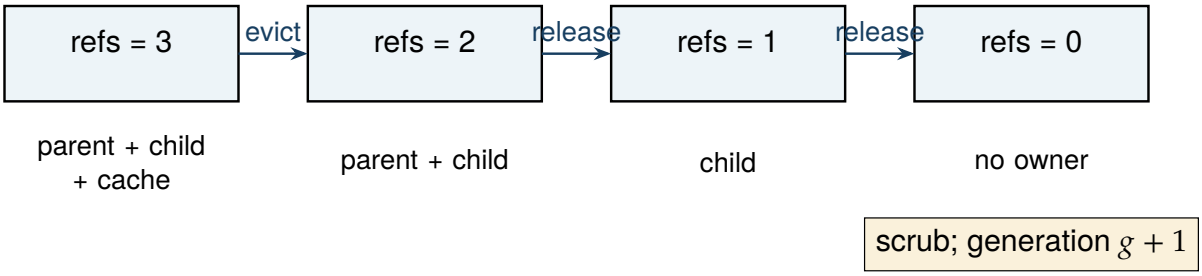
$$r_i = \sum_{s \in \mathcal{S}} \sum_{\rho \in s.\text{pages}} [\rho.\text{id} = i], \qquad r_i = 0 \iff i \text{ is free.} \tag{36.1}$$

No sequence repeats a physical reference within its own table. A free slot has no logical K/V tensors. A live page's generation agrees with every table reference, and its valid row count matches the logical span.

Proposition 36.1 (Reference conservation)**.** *Assume serialized operations, valid initial registries and no process failure during publication. Empty creation, same-tenant fresh-owner fork, append and release preserve Equation* (36.1).

Proof. An empty table adds no references. Fork adds one table entry and increments the corresponding count once for each entry; both sides increase equally. Release removes those entries and decrements the same counts. If a count becomes zero, no registered table refers to that page, so clearing it and advancing generation cannot invalidate a live table. Append either preserves an existing private entry, adds a new entry/count pair, or replaces one shared tail entry by a new private entry: decrement old, increment new. Unchanged prefix entries keep their counts. These cases exhaust the implemented transitions. Induction over operations proves the invariant. □

```
    def fork(self, handle, owner, new_owner):
        s = self.state(handle, owner)
        native.owner_pair(new_owner)
        if new_owner[0] != owner[0]:
            raise PermissionError("tenant-scoped sharing")
        if any(x.owner == new_owner for x in self._states.values()):
            raise ValueError("fresh owner required")
        for ref in s.pages:
            self.pages[ref.page_id].refs += 1
        return self._install(self._sealed(new_owner, s.tokens, s.pages))
```



Eviction removes one table's ownership, never a live branch's ownership.

Figure 36.2: Cache eviction drops only the cache's reference. A live child keeps the page alive until its own release. Generation changes only at zero.

The executable `audit` recomputes expected counts from the registry rather than trusting the counters it is meant to check. A negative test adds an unowned count and requires audit failure. A counter that is merely nonnegative would miss that leak.

```
    def audit(self):
        expected = Counter()
        for serial, state in self._states.items():
            self.state(Handle(self._arena, serial), state.owner)
            expected.update(r.page_id for r in state.pages)
        for i, p in enumerate(self.pages):
            if p.refs != expected[i] or (p.refs == 0 and (p.layers or p.seal)):
                raise AssertionError("reference conservation/scrub invariant")
        used = sum(p.refs > 0 for p in self.pages)
        rows = sum(p.layers[0].k.shape[1] for p in self.pages if p.refs)
        return {"used_pages": used, "free_pages": self.capacity-used,
                "references": sum(expected.values()), "physical_rows": rows,
                "tail_slots": used*self.page_size-rows}
```

36.3 Append requires a partial-tail rule

If $T \bmod P = 0$, all old pages are full. Append allocates a new page; it never writes the old full page. Otherwise there is a partial tail of o valid positions, where $1 \le o < P$. A private tail ($r = 1$) may receive new rows. A shared tail ($r > 1$) needs **copy-on-write**: copy its valid rows into private storage, redirect this branch, and append there.

For a three-token prefix and $P = 2$, parent and child initially hold the same two references. After child append, they still share the first page. The parent tail remains one row long; the

child's private tail is two rows long. A later append by the parent may use its now-private tail without a new allocation. Vacant capacity is not a consumed token and cannot be included in attention normalization.

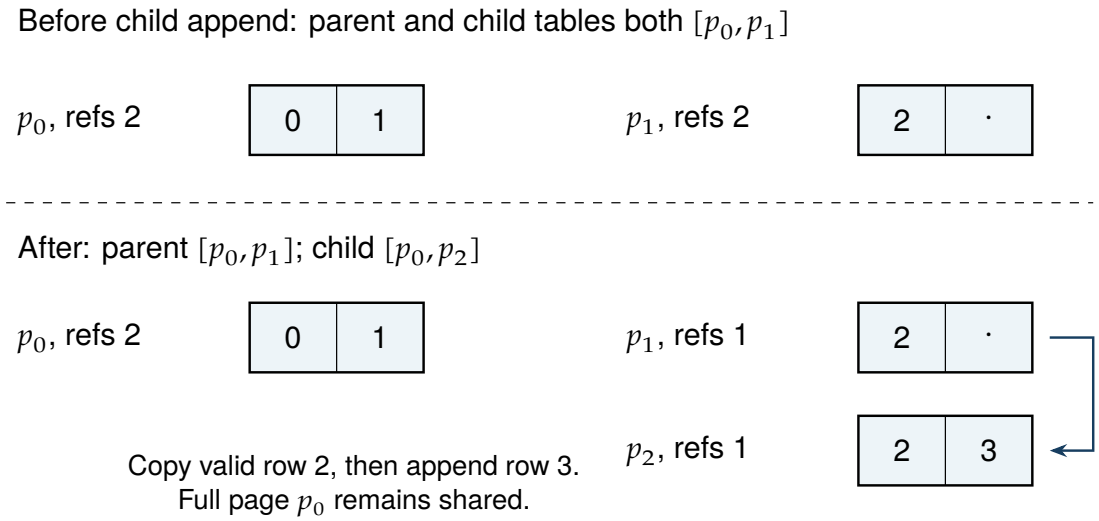


Figure 36.3: Appending to a shared partial tail redirects only the child. Full pages stay shared and the parent's logical history is unchanged.

Proposition 36.2 (Branch isolation and native parity)**.** *Fix the actual Hermon artifact, causal mode, learned-position convention and deterministic evaluation. If flattened registered caches equal their Chapter 35 counterparts, a successful append preserves that equality and does not change any other registered sequence's logical tensors or tokens.*

Proof. Chapter 35's complete-stack continuation computes the correct candidate layer tensors from the flattened old cache. Split those candidates at the same logical multiples of P. Old full blocks remain identical by causality. New pages contain exactly the candidate slices; a private tail has no other reader, and a shared tail is replaced only in the appending table. Thus concatenating its page slices returns the candidate in order, while all other tables still see their old slices. Learned positions are derived from consumed length, independent of page ID. This proves the step; induction over forks and continuation chunks proves the claim. □

The real-arithmetic statement inherits Chapter 35's assumptions. CPU float64 tests use 2×10^{-12} absolute/relative tolerance because different chunk shapes can change rounding. This is a checked fixture tolerance, not a numerical theorem for all Transformer artifacts.

36.4 Preflight the entire chunk before publishing it

Let c be append length, F the number of free pages, and $o = T \bmod P$. For $c > 0$, the required *additional* page count is

$$N_{\text{add}} = \begin{cases} \lceil c/P \rceil, & o = 0, \\ [r_{\text{tail}} > 1] + \lceil \max(0, c - (P - o))/P \rceil, & o > 0. \end{cases}$$

The second term subtracts the remaining capacity in the tail; the first term pays for copying a shared tail. For $c = 0$, the operation is identity and needs zero pages. Reject if $N_{\text{add}} > F$ before modifying any table, payload, generation or refcount.

```
    def required_pages(self, handle, owner, count):
        s = self.state(handle, owner)
        if type(count) is not int or not 0 <= count <=
         ↪  self.model.max_len-len(s.tokens):
            raise ValueError("bounded append count")
        if not count:
            return 0
        offset = len(s.tokens) % self.page_size
        if not offset:
            return math.ceil(count / self.page_size)
        cow = int(self.page(s.pages[-1]).refs > 1)
        return cow + math.ceil(max(0,
         ↪  count-(self.page_size-offset))/self.page_size)
```

With $T = 1, P = 2, c = 5$ and a shared tail, one new page holds the copied old row and first appended row; two further pages hold the other four rows. The requirement is three. A loop that allocates one page and then discovers exhaustion would need rollback; preflight avoids exposing that partial state.

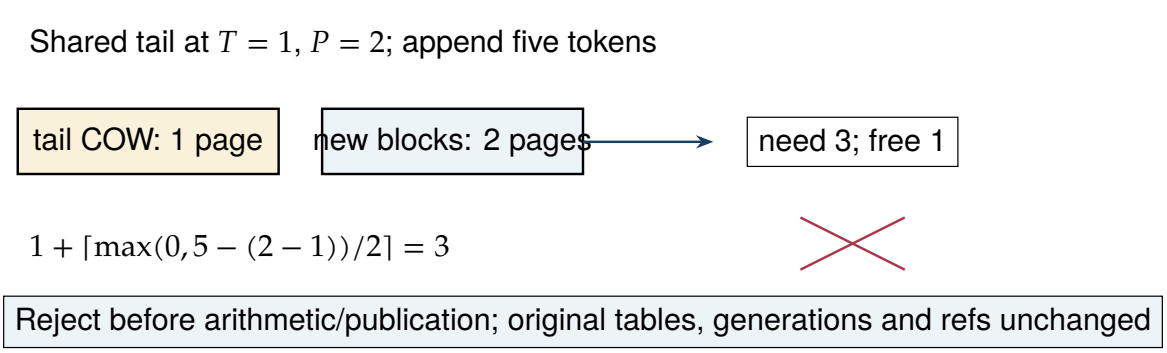


Figure 36.4: Preflight counts COW and every subsequent block. A failed multi-page append leaves the original snapshot unchanged.

```
    @torch.no_grad()
    def consume(self, handle, owner, tokens):
        """Preflight and stage before a serialized, non-yielding publication."""
        s = self.state(handle, owner)
        tokens = native.tokens_copy(tokens, self.model.max_len-len(s.tokens))
        needed = self.required_pages(handle, owner, len(tokens))
        free = [i for i, p in enumerate(self.pages) if p.refs == 0]
        if len(free) < needed:
            raise MemoryError("insufficient pages; original state unchanged")
        logits, candidate = native.consume(self.model, self.flatten(handle,
         ↪  owner),
                                           owner, tokens)
        if not tokens:
            return logits
        start = len(s.tokens) // self.page_size
        prepared = []
        for b in range(start, math.ceil(len(candidate.tokens)/self.page_size)):
            lo, hi = b*self.page_size, (b+1)*self.page_size
            layers = tuple(native.LayerKV(x.k[:, lo:hi].clone(),
                                          x.v[:, lo:hi].clone())
                           for x in candidate.layers)
            prepared.append((layers, page_seal(layers)))
```

```
        # All model arithmetic, validation, tensor allocation and hashes precede
        # publication. Not crash-atomic or a concurrent memory manager.
        refs = list(s.pages[:start])
        writes = []
        old_tail = s.pages[-1] if len(s.tokens) % self.page_size else None
        for i, (layers, seal) in enumerate(prepared):
            private = i == 0 and old_tail and self.pages[old_tail.page_id].refs ==
             ↪ 1
            slot = old_tail.page_id if private else free.pop(0)
            writes.append((slot, layers, seal))
            refs.append(PageRef(slot, self.pages[slot].generation))
        new_state = self._sealed(owner, candidate.tokens, refs)
        for slot, layers, seal in writes:
            page = self.pages[slot]
            page.layers, page.seal, page.refs = layers, seal, 1
        if old_tail and refs[start] != old_tail:
            self._drop(old_tail)
        self._states[handle.serial] = new_state
        return logits
```

All model arithmetic, tensor construction and content/table hashes occur before the short publication phase. The phase installs prepared page records and the new table without yielding. This establishes failure atomicity for the checked validation, arithmetic and capacity paths under the serialized model. It is not a durable transaction, a lock-free allocator or a proof against asynchronous process interruption. Device execution would also require pins/completion fences before reclamation, as earlier state-pool chapters explain.

36.5 Prefix caching owns pages too

Deterministic execution of an identical prefix under the same artifact produces the same per-layer K/V. The cache key therefore binds tenant, artifact digest, causal/position mode and the exact ordered token tuple. If adapters, attention masks or other model-affecting inputs are added, they must enter that identity. Token text alone is insufficient.

The bounded prefix cache retains a registered fork with its own owner. Acquiring a hit makes another fresh-owner fork; evicting releases only the cache-owned handle. LRU orders cached entries, not physical page IDs. The test retains a live hit, evicts its cache entry, releases its parent and continues reading the surviving branch. Each reference remains counted.

Sharing is tenant-scoped here. An exact key and a content hash do not prove permission to acquire another tenant's state. Cross-tenant public-prefix sharing would need a policy and side-channel analysis: even an immutable hit can reveal whether a prefix was present through timing. This local Python registry is not a hostile-process security boundary.

Block-table integrity matters separately from page contents. Reordering two valid page references changes attention history. The reference seals ordered table metadata and layer payloads to detect accidental mutation. It rejects a changed model artifact or stale generation. A producer able to rewrite both data and seals can still forge a false cache; these are consistency checks, not authentication or proof of native computation.

36.6 Count physical capacity and logical sharing separately

For a nonempty unshared sequence,

$$N_p = \lceil T/P \rceil, \qquad W = N_p P - T, \qquad 0 \le W < P.$$

Only its tail is partly unused. Fixed-size paging removes the need for a single contiguous region at this allocation level; it does not eliminate alignment, arena reserve or device-allocator fragmentation.

Page tokens	Pages for 13	Unused slots
1	13	0
2	7	1
4	4	3
8	2	3



Figure 36.5: The same thirteen rows use different table lengths and unused tails. Smaller pages reduce worst-case tail waste while increasing metadata and lookup operations.

With shared pages, sum physical occupancy once per distinct allocated page:

$$U_{\text{physical}} = \frac{\sum_{i:r_i>0} n_i}{NP}, \qquad M_{\text{payload capacity}} = NPs, \qquad s = 2LHd_h b.$$

Here n_i is the valid row count, N is the number of allocated pages, and b is bytes per element. When $N = 0$, utilization is undefined; report zero live payload separately. The sum $\sum_r T_r/(NP)$ instead measures logical multiplicity and can exceed one. It must not be called physical utilization.

In our prefix-three/child-four example there are three distinct pages of capacity two, storing five physical rows. Hence $U_{\text{physical}} = 5/6$. The two logical histories contain seven rows, giving multiplicity $7/6$. At $s = 256$, physical payload capacity is 1,536 bytes, occupied rows are 1,280 bytes, and equivalent independent contiguous payload is 1,792 bytes. Block tables, Python objects, hashes, token metadata and staging tensors are excluded. Cached owners add references without adding physical rows.

Request	Tokens	Page refs	Tail slots
parent	3	2	1
child	4	2	0

For B branches at a common prefix, full shared pages avoid $B - 1$ copies each. Partial-tail savings depend on which branches append and whether the original tail remains cached. Fanout alone is not a savings percentage. PagedAttention supplies the established serving-system comparison for block tables and COW [85]; its empirical speedups are not measurements of this reference.

36.7 What the implementation establishes

The tests compare every layer against the independent original full Hermon path across page sizes and split positions; force scattered placement; exercise full/partial tails, stale generations, foreign handles, cache eviction, all branch-release orders, invalid input and multi-page OOM; and deliberately corrupt tables, payloads and counters. These controls make incorrect order, leaks and premature reclamation observable.

Flattening and computing through Chapter 35 still copy contiguous tensors. Even a logically private tail is replaced by a prepared tensor object. Thus the reference demonstrates bounded physical-page placement and sharing, but does not claim reduced actual Python traffic, native in-place writes, GPU coalescing, concurrent correctness or speedup. Chapter 38 supplies a page-streaming attention reduction; Chapter 37 first treats pages as a finite scheduling resource.

36.8 Selective glossary

Logical block A consecutive span of token ordinals of capacity P.

Physical page An allocation unit holding all layer/head K/V at that span in this reference.

Generation Allocation identity advanced when a page is reclaimed; it is distinct from token position.

Reference count Number of registered owning table entries, including cache entries.

Copy-on-write Private replacement of a shared partial tail before an append changes its valid rows.

Physical utilization Distinct occupied page rows divided by distinct allocated page capacity.

Logical multiplicity Repeated request-visible rows per physical slot; sharing can make it exceed one.

Key ideas

- Flattening the pages must reproduce the Chapter 35 cache exactly. Physical page IDs may change; logical order may not.
- Reference counts carry lifetime and generations carry identity, but authorization still needs owner checks.
- Writing into a shared partial tail through a shared pointer would rewrite another request's history.
- Shared pages can push logical multiplicity above one, as in the 7/6 example. Physical utilization never exceeds one.

36.9 Twenty worked exercises

1. **Translate.** Ordinal 13, $P = 4$? *Solution:* block 3, offset 1; look up entry 3, never physical page 13.
2. **Size one cell.** $L = 2, H = 2, d_h = 4, b = 8$? *Solution:* $2(2)(2)(4)(8) = 256$ bytes, including both K and V.

3. **Count a page.** $P = 2$ for that geometry? *Solution:* 512 bytes of payload capacity, before metadata.
4. **Bound waste.** Show $W < P$. *Solution:* $N_p - 1 < T/P \leq N_p$ implies $0 \leq N_p P - T < P$.
5. **Empty state.** How many pages? *Solution:* zero; no all-masked pseudo-token is created.
6. **Trace COW.** Prefix three, $P = 2$, fork then child append? *Solution:* first page remains shared; child gets a private second page.
7. **Full tail.** Prefix four, $P = 2$, child append? *Solution:* retain both shared full pages and allocate a third.
8. **Private capacity.** One row in a private $P = 2$ tail with zero free pages can append one row? *Solution:* yes, no new page credit.
9. **Preflight.** Shared $T = 1, P = 2, c = 5$? *Solution:* $1 + \lceil (5 - 1)/2 \rceil = 3$ additional pages.
10. **Evict.** Count two becomes one. Free? *Solution:* no; the remaining table still owns the page.
11. **Invalidate reuse.** Why increment generation? *Solution:* an old reference cannot name a later allocation in that slot.
12. **Audit a leak.** Count positive with no registered owner? *Solution:* recomputed incidence is zero; conservation fails.
13. **Reject a reorder.** Do page hashes alone suffice? *Solution:* no; ordered table metadata must also be checked.
14. **Prefix identity.** Same tokens under a changed position table? *Solution:* different artifact, so reject reuse.
15. **Separate permission.** Does a matching digest authorize a hit? *Solution:* no; enforce tenant/trust-domain policy first.
16. **Interpret 7/6.** Is physical utilization above 100 percent? *Solution:* it is logical multiplicity, counting shared rows repeatedly.
17. **Explain parity.** What passes from Chapter 35 into the proof? *Solution:* exact ordered old K/V and identical learned positions produce correct candidate rows through every layer.
18. **Inject failure.** What snapshots should an OOM test compare? *Solution:* both branches' flattened contents, tables, counts and generations, including a failure requiring several pages.
19. **Design a page-size experiment.** *Rubric:* declare length/fanout distributions, capacity, metadata, high-water occupancy, copy volume and end-to-end latency; do not infer speed from waste alone.
20. **Design device reclamation.** *Rubric:* name in-flight pins, completion acknowledgement, cancelled writes and stale generations; prove reuse waits for the final physical reader, not only a logical eviction.

36.10 From storage to service

The allocator preserves each request's causal history but cannot choose which waiting request deserves the next page or token step. Chapter 37 adds iteration-level admission, work budgets and explicit precision policies. These mechanisms preserve Hermon's Transformer identity; they do not turn it into DOGMA's non-Transformer state representation.

Sources and further reading

Source access checked 17 October 2026. Kwon et al.'s primary HTML paper was inspected for Sections 4.1–4.4, including block tables, partial-tail COW and shared-prefix ownership. Its measurements were not reproduced. The actual Chapter 35 implementation and Chapter 23 model were read and imported; the relevant Chapter 32 generation/pin and Chapter 33 COW contracts were inspected. This chapter's proofs assume serialized trusted callers. Generation checks, hashes and refcounts do not establish authentication, hostile-process isolation or asynchronous device safety.

The allocator stores actual multi-layer KV tensors rather than abstract tuples, and it separates logical multiplicity from physical utilization.

CHAPTER 37

Continuous batching and precision

In this chapter you will

- separate consumed positions from emitted outputs
- reserve page credits at admission and prove the arena cannot overflow
- plan decode-first, chunked-prefill iterations under a token-work budget, and report TTFT, ITL and throughput in their own units
- attach precision to tensor roles and bound how KV error propagates into one attention head

A five-token prompt arrives while a shorter request is nearly ready to emit. Giving each request one turn sounds fair until a turn can mean either five prompt tokens or one decode token. Reserving one page per active request sounds safe until a long generation fills its last slot. Lowering KV precision sounds helpful until a small numerical perturbation changes a sampled future. Serving needs explicit units for work, storage, output time and error.

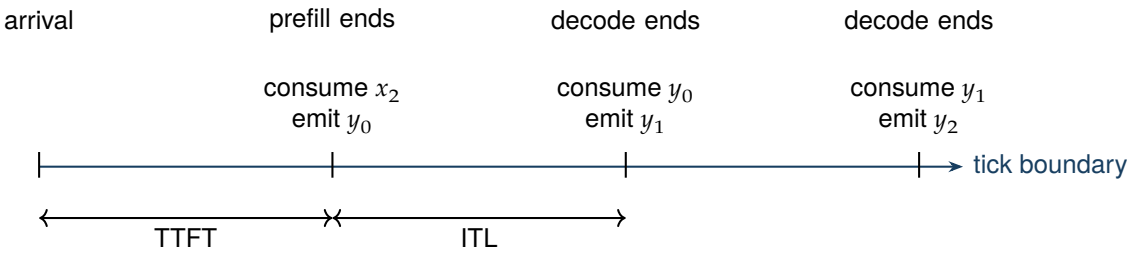
We study a decode-first, chunked-prefill policy and a set of precision experiments, and we execute the policy through the actual Chapter 36 allocator and Chapter 35 Hermon stack. The fixture is deterministic greedy ACGT generation, CPU float64, abstract iteration ticks and serialized execution. A list of scheduled lanes does not constitute a fused ragged device kernel or a measured service benchmark.

37.1 Define consumed work and emitted output

Let request r have nonempty prompt $x^{(r)}$ of length P_r and desired output count $O_r \geq 1$. Its last prompt logits predict output token y_0. Consuming y_0 then predicts y_1, and so on. Therefore completion with O_r outputs requires

$$T_{r,\max} = P_r + O_r - 1 \tag{37.1}$$

consumed positions, when the final emitted token is not subsequently consumed. A library may choose to consume it for future continuation, but then it needs an extra position and must declare that convention.



Three emitted outputs need only two output-token KV appends. Final emitted y_2 need not be consumed.

Figure 37.1: Prediction and cache advancement have different clocks. The last prompt row emits the first output; an unnecessary extra decode would change latency and page accounting.

At intermediate time, if p_r prompt tokens are consumed and g_r outputs emitted, the cache length is

$$T_r = p_r + \max(0, g_r - 1).$$

Before final prefill, $g_r = 0$; at its completion $g_r = 1$ without an additional consumed output. The scheduler checks this equation against the actual registered page table after every iteration. It rejects empty prompts rather than inventing an implicit BOS token absent from the model.

37.2 Admission reserves capacity; scheduling spends work

Take page capacity P and arena size C physical pages. The teaching admission policy conservatively reserves

$$R_r = \left\lceil \frac{P_r + O_r - 1}{P} \right\rceil$$

page credits for each admitted request. The declared output limit must keep consumed positions inside the model's learned-position table. An impossible single request is rejected immediately; others wait until their reservation fits. A reservation is an accounting promise, not preallocated K/V tensors.

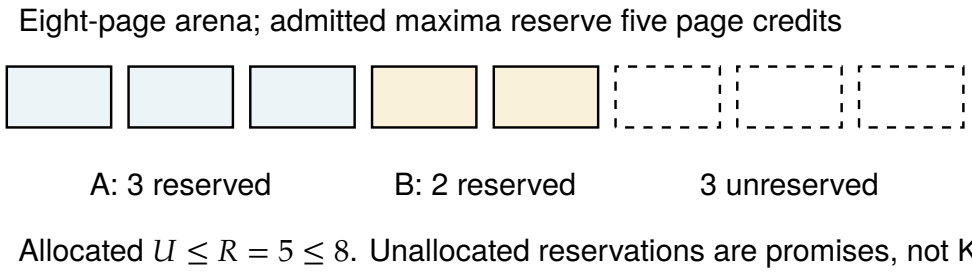


Figure 37.2: Allocated pages are bounded by admitted reservations. Waiting requests own neither pages nor credits in this policy.

This policy trades utilization for a simple progress guarantee. It supports no shared-prefix admission, preemption, spilling or speculative workspace. Because its pool is dedicated and its admitted requests have private tails, the final page count bounds every intermediate page count. A production policy reserving only immediate growth must instead handle later OOM; one that uses shared prefixes must account for COW obligations as well. Speculation needs temporary fork capacity beyond the live reservation.

Proposition 37.1 (Page-credit safety)**.** *Assume a dedicated arena, serialized native execution, validated length limits and no external forks. If admission maintains $\sum_{r \in A} R_r \le C$, then every scheduled native append fits the arena and physical allocation U satisfies*

$$U \le \sum_{r \in A} R_r \le C.$$

Proof. Every active request has $T_r \le T_{r,\max}$, so its private page count is at most R_r. No other state owns a page in the dedicated pool. Sum over active requests. During append, preparing Python tensors does not claim arena pages, old full pages remain referenced, and private-tail replacement needs no extra page credit. The number of newly allocated pages is at most the request's unused reservation. Completion/cancellation releases all its actual pages and removes its reservation. Admission adds only a reservation that fits the remaining credit. Induction over these operations proves the bound. □

This is arena-credit safety, not a bound on process RSS: temporary tensor construction, parameters, metadata and staging still consume memory.

37.3 Construct the next mixed iteration

Let B be the iteration token-work budget and C_p the maximum prompt chunk. First select at most one consumed token from each ready decoder, then use remaining budget for prompt chunks

$$n_r \leq \min(C_p, P_r - p_r, B_{\text{remaining}}).$$

The ready list rotates by tick to avoid always privileging its first decoder. The work plan is fixed before execution, so a prompt finishing in this iteration cannot also receive a decode in the same plan. Native operations preserve causal order within each request even while membership changes.

```
    def plan(self):
        # Rotating list avoids permanently privileging the first ready decoder.
        n = len(self.requests)
        ordered = self.requests[self.tick % n:]+self.requests[:self.tick % n] if n
         ↪  else []
        remaining, work = self.budget, []
        for phase in ("decode", "prefill"):
            for r in ordered:
                if r.phase != phase or remaining == 0:
                    continue
                count = 1 if phase == "decode" else min(
                    self.chunk, len(r.prompt)-r.prefilled, remaining)
                work.append((r, phase, count))
                remaining -= count
        return work
```

If the plan contains work units $n_1, \ldots, n_m$, repeated subtraction from nonnegative remaining budget proves $\sum_i n_i \leq B$. Each prefill count is at most C_p by construction, and every decode count is one. However, token counts are not equal execution costs. Attention to a long retained prefix can cost more than attention to a short prefix; projection, feed-forward and dispatch costs also matter. B bounds this policy's token numerator, not iteration milliseconds.

Orca is the primary precedent for iteration-level membership changes [81]. Sarathi-Serve studies chunked prefills mixed with decodes [86]. Those designs motivate this experiment; we do not claim this rotating serial scheduler reproduces their kernels or guarantees.

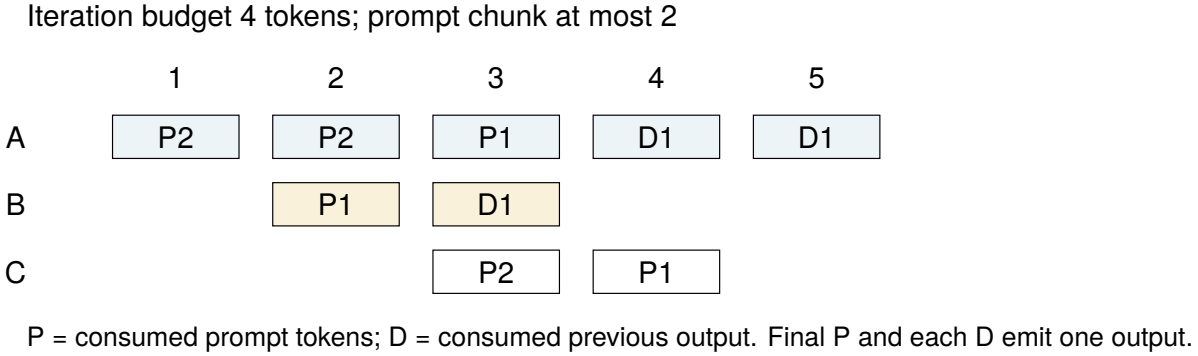


Figure 37.3: The actual small workload changes membership between iterations. P and D label consumed token work, not abstract requests of equal cost.

The example admits A at arrival 0 with prompt length five/output three, B at arrival 1 with prompt one/output two, and C at arrival 2 with prompt three/output one. Set $B = 4, C_p = 2, P = 2$ and arena capacity 12. The generated table counts emissions separately from consumed tokens and records page usage *after* releases at each completion boundary.

End tick	Consumed	Emitted	Pages used	Reserved
1	2	0	1	4
2	3	1	3	5
3	4	2	4	6
4	2	2	3	4
5	1	1	0	0

The first iteration uses only two of four token units because only A is eligible and one chunk per request is allowed. This policy need not fill every budget. At tick three's completion, B has finished and released its pages while A emits its first token and C has consumed two prompt tokens. No fabricated device timing is attached to these ticks.

```
    def step(self):
        self.admit()
        work = self.plan()
        record = []
        for r, phase, count in work:
            tokens = r.prompt[r.prefilled:r.prefilled+count] if phase == "prefill"
             ↪  else (r.generated[-1],)
            logits = self.pool.consume(r.handle, self.owner(r), tokens)
            if phase == "prefill":
                r.prefilled += count
            emit = phase == "decode" or r.prefilled == len(r.prompt)
            if emit:
                r.generated.append(int(logits[-1].argmax()))
                r.token_ticks.append(self.tick+1)  # completion boundary
                r.phase = "decode"
                if len(r.generated) == r.output:
                    self.pool.release(r.handle, self.owner(r))
                    r.handle, r.phase = None, "done"
            record.append((r.rid, phase, count, int(emit)))
        self.tick += 1
        stats = self.audit()
        self.trace.append({"tick": self.tick, "work": record, **stats})
        return record
```

Execution is per-lane serial publication. If an unexpected runtime error occurs after an earlier lane completed, the earlier lane remains completed; the batch is not claimed atomic. Expected page exhaustion is excluded by the conservative admission proof. Cancellation is immediate at a serialized boundary and releases that request's handle. A real asynchronous backend must separately fence work already using its pages.

37.4 Latency, throughput and fairness are different questions

For arrival a_r and completion-boundary output timestamps $t_{r,0}, \dots, t_{r,O_r-1}$,

$$\mathrm{TTFT}_r = t_{r,0} - a_r, \qquad \mathrm{ITL}_{r,i} = t_{r,i+1} - t_{r,i}, \qquad L_r = t_{r,O_r-1} - a_r.$$

A one-output request has no ITL sample; it does not have measured zero ITL. The table uses a dash for that undefined statistic.

Request	TTFT ticks	Max ITL	Completion
A	3	1	5
B	1	1	2
C	2	–	2

All quantities here are ticks. The example emits six outputs over five ticks, a useful-output rate of 6/5 outputs per abstract tick. It consumes 12 prompt/output tokens, giving 12/5 consumed tokens per tick. These numerators differ even before conversion to seconds. Queueing is included in TTFT; scheduling overhead and execution time would enter real timestamps.

Decode-first can protect existing token progress while delaying new prompts. Rotating decoders does not guarantee tenant fairness; clients can submit different numbers of requests, prompt costs vary, and sustained decode load can starve prefills. The finite-workload test establishes eventual completion only for its supplied requests. Fairness needs a declared unit, such as weighted client service, and a corresponding policy/proof; VTC provides one research comparison [87].

For an actual SLO experiment declare arrival process, offered rate, prompt/output distributions, cancellation mix, tenant weights and admission policy. Sweep load, recording generated tokens/s, requests/s, TTFT and ITL p50/p95/p99, completion latency, rejection rate and memory high-water mark. Compare configurations on the same workload and quality gate. A mean or a single low-load operating point cannot describe saturation behavior.

37.5 Precision belongs to a tensor role

A useful policy is

$$\Pi = (\Pi_W, \Pi_A, \Pi_{KV}, \Pi_{\text{accum}}, \Pi_{\text{out}}),$$

for weight, activation, retained KV, accumulation and output representations. An eight-bit storage label does not specify scale granularity, clipping, rounding, reduction precision or supported kernels. A model artifact and serving manifest must identify these choices alongside calibration data.

FP16 has five exponent and ten explicit fraction bits; BF16 has eight and seven. BF16 preserves a wider exponent range, while FP16 resolves smaller relative differences for normal values in its narrower range. Tests use PyTorch's actual CPU casts for representation study, including BF16 rounding, rather than calling a hand-written bit trick a native kernel. Large finite values that overflow FP16 trigger an explicit fallback error. FP8 is a family with multiple exponent/fraction and special-value contracts; we supply no FP8 conversion or hardware performance result.

For symmetric INT8 storage of finite nonempty tensor x, define

$$s = \max_i |x_i|/127, \quad q_i = \text{clip}(\text{round}(x_i/s), -127, 127), \quad \hat{x}_i = sq_i.$$

For the all-zero tensor choose $s = 1, q = 0$. With no saturation and nearest rounding, $|x_i - \hat{x}_i| \le s/2$: rounding changes x_i/s by at most one half, then multiplication restores units. This proof does not apply to values clipped outside a fixed calibration range.

```
def int8_symmetric(x, scale=None):
    x = finite_tensor(x)
    peak = x.abs().max().item()
    scale = (peak/127 if peak else 1.0) if scale is None else scale
    if not isinstance(scale, (float, int)) or not math.isfinite(scale) or scale <=
     ↪ 0:
        raise ValueError("finite positive scale")
    q = torch.clamp(torch.round(x/scale), -127, 127).to(torch.int8)
    return q, float(scale)
```

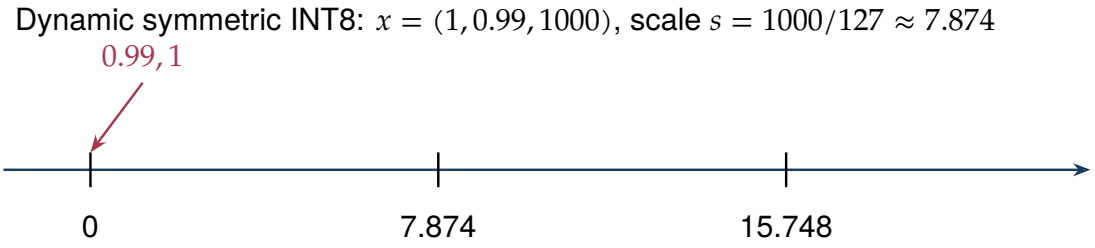


Figure 37.4: One outlier controls a per-tensor scale and erases small differences. A fixed scale can instead saturate previously unseen large values.

Dynamic per-tensor scaling avoids clipping finite current extrema but can change older reconstructed rows when a later outlier changes the scale. Per-channel/group scales trade metadata for finer resolution. With one four-byte scale for every g one-byte values, nominal storage is $1 + 4/g$ bytes/value before padding, not exactly one byte/value. SmoothQuant's equivalent rescaling is a method for moving quantization difficulty between activation and weight tensors; its W8A8 quality evidence does not validate a different KV policy [88].

37.6 Trace KV error into attention before judging output

For one fixed query $q \in \mathbb{R}^{d_h}$ and T keys/values, assume $\|\Delta k_j\|_2 \le \epsilon_K$ and $\|\Delta v_j\|_2 \le \epsilon_V$. Exact score error is bounded by

$$|\Delta s_j| = |q^\top \Delta k_j| / \sqrt{d_h} \le \delta := \|q\|_2 \epsilon_K / \sqrt{d_h}.$$

Let $p(s)$ be softmax. Along $s(\tau) = s + \tau \Delta s$,

$$\frac{dp_j}{d\tau} = p_j \left(\Delta s_j - \sum_k p_k \Delta s_k \right).$$

The absolute sum of derivatives is at most 2δ because both each $|\Delta s_j|$ and the probability-weighted mean magnitude are at most δ. Integrating over $\tau \in [0, 1]$ gives $\|p(s + \Delta s) - p(s)\|_1 \le 2\delta$.

If $\max_j \|v_j\|_2 \le V_{\max}$, decompose the output difference into changed values weighted by p' and changed probabilities weighted by v:

$$\|\hat{y} - y\|_2 \le \epsilon_V + 2V_{\max} \|q\|_2 \epsilon_K / \sqrt{d_h}. \tag{37.2}$$

The first term uses probabilities summing to one. This deliberately loose one-head bound assumes a fixed query and mask, exact arithmetic, and one attention operation. It does not bound multi-layer feedback, new queries computed from perturbed prior layers, softmax rounding or future sampled history. Those require additional stability assumptions or measurement.

37.7 Run the real model, then apply a declared gate

The executable experiment rounds stored K/V after every consumed token, then continues through Chapter 35's actual multi-layer model. All projections, attention and FFN arithmetic remain float64. INT8 reconstructs to float64 and is repeatedly requantized per current layer tensor; it is neither a compressed page arena nor a native INT8 matmul path. Its approximate state is labeled separately and never inserted as an exact Chapter 36 prefix hit.

```
def precision_rollout(model, tokens, mode):
    """Round stored K/V after each token; arithmetic remains float64.

    This intentionally approximate cache is re-sealed for the native reader.
    It is not labeled an exact Chapter 36 pool entry or compressed allocation.
    """
    owner = ("precision", mode)
    cache = native.empty(model, owner)
    outputs = []
    for token in native.tokens_copy(tokens, model.max_len):
        logits, cache = native.consume(model, cache, owner, (token,))
        outputs.append(logits)
        layers = tuple(native.LayerKV(represent(x.k, mode), represent(x.v, mode))
                       for x in cache.layers)
        cache = native.sealed(owner, cache.artifact, cache.tokens, layers)
    if not outputs:
        raise ValueError("nonempty precision experiment")
    return torch.cat(outputs)
```

KV format	Max logit error	Relative L2	Argmax fraction
fp64	2.22045e-16	1.86131e-16	1
fp16	4.65329e-05	1.9527e-05	1
bf16	0.000767289	0.000241877	1
int8	0.00138083	0.00067643	1

The generated eight-token fixture uses fixed untrained weights and teacher forcing, so all modes see identical token histories. This isolates numerical drift; it is not perplexity or a learned genomic task. Argmax agreement is computed per output row, not once over a flattened tensor. The report also contains mean target-to-candidate softmax KL, maximum logit error and relative L2 error with a declared denominator floor.

If each logit changes by at most ϵ and the reference winner's margin is $m = z_{(1)} - z_{(2)}$, then its perturbed advantage is at least $m - 2\epsilon$. Thus $m > 2\epsilon$ is sufficient for unchanged argmax. It is not necessary, and it does not imply unchanged sampling probabilities.

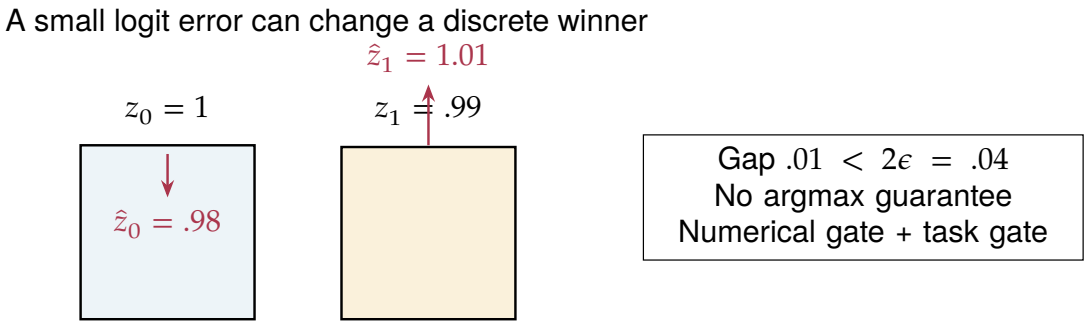


Figure 37.5: Maximum error and discrete output agreement answer different questions. An explicit near-tie negative control prevents a small-error report from certifying the wrong winner.

The gate rejects nonfinite reports, errors above declared limits, or insufficient per-row argmax agreement. Thresholds are experiment inputs, not universal quality standards. Representative calibration and held-out task evaluation must cover domains, context lengths, positions, outliers, structured-output validity and any stochastic sampling transformation. Record baseline artifact, candidate policy, data/split hashes and fallback. Do not choose thresholds retrospectively to rescue a failed configuration.

Lower KV storage can allow more resident requests, but a larger batch may increase iteration time or queueing. Precision therefore changes both numerical behavior and the scheduler's operating frontier. The float64 emulation here does not allocate fewer bytes; such a claim needs actual compressed storage accounting and supported-kernel measurements.

37.8 Selective glossary

Admission Permission to enter the active set under a storage-credit policy.

Reservation A bound on future arena allocation, distinct from resident payload.

Continuous batching Revising eligible membership between iterations while preserving each request's causal order.

TTFT and ITL Arrival-to-first-output delay and gaps between successive output completions.

Typed precision Representation choices attached to tensor roles, scales and reduction rules.

Calibration Data used to select quantization parameters; it is distinct from held-out task evaluation.

Logit margin Difference between the largest and second-largest reference logits; it controls a sufficient argmax-stability condition.

Key ideas

- O outputs need $P + O - 1$ consumed positions: the last prompt row already emits the first output.
- A token budget bounds counted tokens, not milliseconds. One prompt token and one decode token can cost very different amounts.
- A one-output request has no inter-token latency sample. That is undefined, not zero.
- A small maximum logit error preserves an argmax only when the margin exceeds twice the error. Near-ties need explicit negative controls.

37.9 Twenty worked exercises

1. **Consume versus emit.** Prompt five, output three? *Solution:* seven consumed positions; last prefill emits the first output.
2. **Reserve pages.** Those lengths with $P = 2$? *Solution:* $\lceil 7/2 \rceil = 4$ page credits.
3. **Distinguish reserve.** Four reserved, two allocated? *Solution:* two actual payload pages, two future credits; not four pages of current valid KV.
4. **Plan work.** Budget four, two ready decoders, prompt remains six and chunk two? *Solution:* one token per decoder then a two-token chunk.
5. **Predict a transition.** Can finishing prefill decode again in the same fixed plan? *Solution:* no; next iteration sees its new phase.
6. **Derive the budget.** Why no overspend? *Solution:* each selection is bounded by remaining budget and subtracts its exact count.
7. **Measure TTFT.** Arrival 2, first output at 4? *Solution:* two ticks, including waiting and prefill.
8. **Measure ITL.** One output only? *Solution:* no gap sample; report undefined, not measured zero.
9. **Name throughput.** Six emitted and twelve consumed over five ticks? *Solution:* 1.2 outputs/tick and 2.4 consumed tokens/tick.
10. **Challenge fairness.** Does decode-first prevent prompt starvation? *Solution:* no; sustained decode load can spend the whole budget.
11. **Choose precision fields.** What does "INT8 model" omit? *Solution:* tensor roles, scale granularity, accumulators and clipping.
12. **Compare sixteen bits.** FP16 versus BF16? *Solution:* FP16 has more fraction bits; BF16 more exponent bits.
13. **Prove INT8 rounding.** No saturation? *Solution:* nearest rounding error is at most half a scale unit.
14. **Find saturation.** $s = .01$, value ten? *Solution:* integer clips to 127; reconstructed value 1.27 violates the unsaturated half-step bound.
15. **Count scale overhead.** $g = 32$ and four-byte scales? *Solution:* 1.125 nominal bytes/value, before alignment.
16. **Propagate key error.** $\|q\|_2 = 2, d_h = 4, \epsilon_K = .01$? *Solution:* $\delta = .01$ and probability L1 change is bounded by .02.
17. **Protect a winner.** Margin .1 and $\epsilon = .02$? *Solution:* $.1 > .04$ suffices; margin .01 would not.

18. **Separate evidence.** All eight fixture argmaxes agree? *Solution:* only that fixture's discrete predictions agree; no trained quality or long-context guarantee follows.
19. **Design calibration.** *Rubric:* separate calibration and evaluation, declare contexts, domains and outliers, fix thresholds beforehand, record artifact/scale hashes and unsupported-case fallback.
20. **Design a load study.** *Rubric:* vary offered load, hold workload and quality fixed, report TTFT/ITL tails, useful throughput, rejections, page and temporary-memory high-water marks, and uncertainty.

37.10 The next optimization must preserve these contracts

Chapter 38 adds speculative verification and online exact attention. Speculation needs temporary state outside the live reservation; online reduction needs the same offset masks and precision policy. Neither can silently change token order, the target sampling law or Hermon's model identity to make a performance claim easier.

Sources and further reading

Source access checked 17 October 2026. Orca and VTC were inspected at their official USENIX abstract/proceedings pages; their full proofs, artifacts and experimental results were not independently audited. Sarathi-Serve's primary HTML Section 4, including Algorithm 3 and token-budget/chunk discussion, was inspected. SmoothQuant's primary HTML Sections 3–4 and calibration/implementation discussion were inspected. These sources motivate specific comparisons, not inherited speed or fairness claims.

Actual Chapters 35–36 implementations were read and executed. Native CPU FP16/BF16 casts and declared INT8 reconstruction are local representation experiments; no native FP8/INT8 engine or compressed KV pool is implemented. An attempted NVIDIA FP8 primer access failed, so no current device-support claim is based on it. Two tempting shortcuts are corrected in the text: an extra decode step in the timing model and an argmax agreement computed over a flattened tensor. Local author review is not independent specialist or task-quality acceptance.

CHAPTER 38

Speculative decoding and attention kernels

In this chapter you will

- derive the modified rejection sampler and prove it emits the target law
- extend it across blocks of conditional proposals with tentative KV ownership
- separate expected acceptance from speed
- derive the online-softmax merge and preserve offset causality across tiles and pages

Suppose the target next-token law is $(.55, .25, .15, .05)$ while a cheaper draft proposes from $(.45, .35, .15, .05)$. Accepting every proposal is fast but wrong: it overproduces token one and underproduces token zero. Rejecting some proposals and sampling from the original target distribution after rejection also gives the wrong mixture. The correction must supply precisely the target mass left uncovered by accepted proposals.

This chapter finishes two distinct engine mechanisms. Speculation changes which future computations are attempted before their need is known. Online attention changes how the exact normalization/value sum is accumulated. Both retain the actual Chapter 23/35 Hermon model and Chapter 36 page lifetime contracts. Our complete-stack implementations are CPU references; the title's kernel discussion supplies derivations and backend obligations, not an unmeasured GPU speed claim.

38.1 Construct a target-preserving one-token sampler

Fix a finite vocabulary and prefix h. Let $p(x \mid h)$ be the intended target distribution and $q(x \mid h)$ the actual draft proposal law, both nonnegative and normalized. Draw $X \sim q$ and independent $U \sim \mathrm{Uniform}[0,1)$. Accept X when

$$U < \alpha(X), \qquad \alpha(x) = \min(1, p_x/q_x) \quad (q_x > 0).$$

A $q_x = 0$ token is never proposed, so no division is needed there. Define $d_x = (p_x - q_x)_+$ and $R = \sum_x d_x$. If a proposal is rejected, draw a fresh token from $r_x = d_x/R$. When $R = 0$, $p = q$ and rejection has zero probability in ideal arithmetic; the reference returns p as an unused correction fallback.

Proposition 38.1 (One-position exact law)**.** *Under those distributions and independent exact random draws, the emitted token Y has distribution p.*

Proof. The joint probability of proposing and accepting token x is $q_x\alpha(x) = \min(p_x, q_x)$. Since p and q sum to one, their total positive and negative difference masses agree. Thus

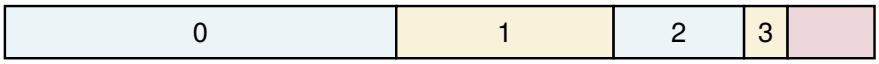
$$1 - \sum_x \min(p_x, q_x) = \sum_x (p_x - q_x)_+ = R.$$

The probability of rejecting any proposal and then emitting x is $Rr_x = d_x$. These disjoint paths sum to $\min(p_x, q_x) + (p_x - q_x)_+ = p_x$. The $R = 0$ case emits only accepted proposals, which already have law p. □

This is the modified rejection mechanism used in speculative decoding [89, 90]. The reference uses p for target and q for draft; some papers reverse those letters. Track roles, not letters.

Token	Target p	Draft q	Accepted mass	Missing mass
0	0.55	0.45	0.45	0.1
1	0.25	0.35	0.25	0
2	0.15	0.15	0.15	0
3	0.05	0.05	0.05	0

Accepted mass $\min(p, q)$ totals .90; residual target mass totals .10



Token 0: $.45 + .10 = .55$; token 1: $.25 + 0 = .25$

Tokens 2 and 3: .15 and .05. On rejection, correction emits token 0.

Figure 38.1: Accepted proposal mass totals .90. The missing .10 belongs entirely to token zero, so every rejection correction selects that token in this example.

Here $R = .10$ and $r = (1, 0, 0, 0)$. If we incorrectly used p itself after rejection, token zero would receive $.45 + .10(.55) = .505$, not .55. The negative test makes this incorrect fallback fail visibly. Support gaps are allowed: if $p = (.5, .5)$ and $q = (1, 0)$, acceptance supplies .5 at token zero and correction supplies .5 at token one.

The expected acceptance is

$$A = \sum_x \min(p_x, q_x) = 1 - \frac{1}{2} \sum_x |p_x - q_x| = 1 - \mathrm{TV}(p, q).$$

This measures distribution overlap at a specified prefix. It does not measure draft cost, cache allocation, communication or latency.

38.2 A block contains conditional proposals, not independent rows

The draft proposes $x_1, \ldots, x_k$ sequentially, retaining each actual law $q_i(\cdot \mid h, x_{<i})$. The target computes $p_i(\cdot \mid h, x_{<i})$ for the same candidate prefixes. Chapter 35's offset-masked causal chunk allows these target rows to be evaluated together. The verification draws use independent uniforms; at the first rejection i, emit accepted $x_{<i}$ followed by a correction y_i, and stop. Candidate x_{i+1} was conditioned on x_i, not y_i.

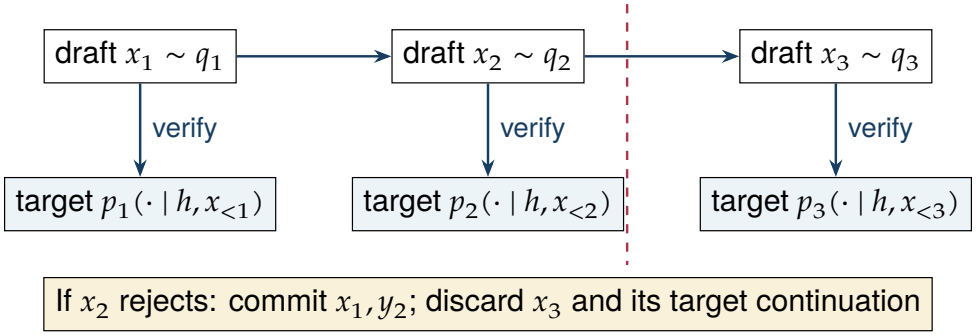


Figure 38.2: Only a contiguous accepted prefix is reusable. A rejection invalidates all descendants conditioned on the rejected proposal, even when their tensors are already computed.

```
def verify(proposals, ps, qs, rng):
    if not proposals or len(proposals) != len(ps) or len(ps) != len(qs):
        raise ValueError("nonempty aligned verification block")
    out = []
    for i, (x, p, q) in enumerate(zip(proposals, ps, qs)):
        p, q = pair(p, q)
        if type(x) is not int or not 0 <= x < len(q) or q[x] <= 0:
            raise ValueError("proposal must have positive draft mass")
        if rng.random() < min(1.0, p[x]/q[x]):
            out.append(x)
        else:
            r = correction_distribution(p, q)
            out.append(int(rng.choice(len(r), p=r)))
            return tuple(out), i
    return tuple(out), len(out)
```

Proposition 38.2 (Autoregressive law across blocks)**.** *Suppose target/draft callbacks depend only on the declared token prefix, both use the same token identities, and each proposal/verification draw is fresh with the prescribed conditional law. Repeated corrected blocks, truncated to a requested output length, have target joint distribution* $\prod_j p(y_j \mid h, y_{<j})$.

Proof. At the first uncommitted position, the one-position construction supplies the target conditional. If accepted, the next candidate was proposed at exactly the new committed prefix, so the same argument applies to that position. If rejected, the correction has completed the one-position mixture; later candidates are discarded and a fresh block starts at the corrected committed prefix. Thus every transition supplies the target law at its actual prefix. Multiplying conditional transition probabilities proves the joint law by induction over emitted positions. □

Merely passing a list of unconditional p_i, q_i arrays is not a general autoregressive implementation. A list-based example is useful only as the one-position algebraic lead. The reference extends it into explicit prefix callbacks and recorded proposal laws. Tests enumerate a two-token, two-symbol history-dependent case exactly, then check a seeded 20,000-run joint-frequency experiment with a predeclared .015 absolute tolerance. That finite statistical check supports the implementation; the proof, not one histogram, establishes the ideal law.

38.2.1 Identity, transforms and randomness

The target p must include the intended temperature, grammar mask, top-k or nucleus transformation and renormalization. The ratio must use the law from which the draft actually sampled. The draft may use a *different* temperature or support restriction: the proof does not require $p = q$ or identical transforms, only correctly recorded laws on compatible token IDs. A tokenizer mismatch or probabilities from an earlier prefix invalidate it.

Exact distribution equality does not mean token-for-token equality under the same seed as target-only sampling: the algorithms consume different random draws. Record RNG implementation, seed/stream policy, target/draft artifacts, transforms and numerical representation for reproducibility. Floating-point normalization and pseudorandom sampling approximate the ideal proof. The reference rejects nonfinite/negative vectors, mismatched vocabulary sizes and impossible zero-mass proposals; it rescales finite unnormalized inputs before summing to avoid overflow.

38.3 Verification state is tentative ownership

The native adapter starts from a nonempty consumed prefix and obtains its initial next-token logits. It forks the Chapter 36 live handle into a same-tenant verification owner and consumes the draft block there. If the prefix has length t, the initial logits supply p_1; the row computed after consuming x_i supplies p_{i+1}. Shifting these rows by one would verify each proposal against the wrong conditional.

After sampling the accepted/corrected output, it releases the entire tentative fork and recomputes only the committed tokens on the original handle. This conservative strategy avoids shortening shared physical pages. It is deliberately expensive, but a simple correctness target for a future transactional suffix adoption operation.

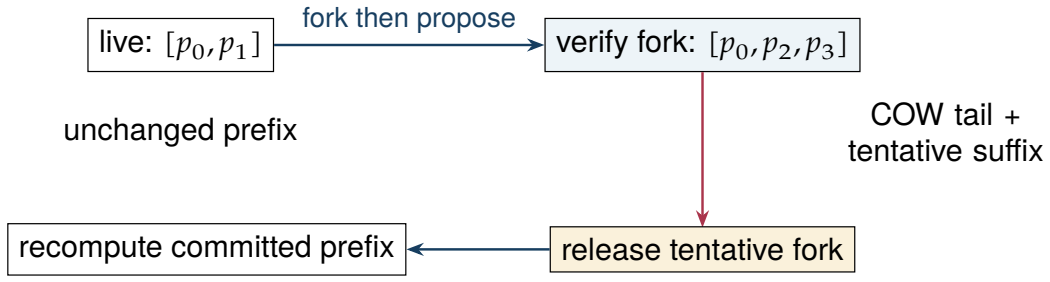


Figure 38.3: The live table survives verification unchanged. Releasing tentative ownership reclaims only unreferenced pages; committed recomputation then advances the live cache.

```
def native_speculative(pool, handle, owner, draft, k, rng):
    """Verify on a fork; discard tentative pages; recompute only committed rows.

    No bonus token in this variant. Every returned token is also consumed into
    KV. A caller must not consume it a second time. Prefix must be nonempty.
    """
    s = pool.state(handle, owner)
    if not s.tokens or k > pool.model.max_len-len(s.tokens):
        raise ValueError("nonempty prefix and bounded draft block")
    proposals, qs = propose(draft, s.tokens, k, rng)
    proposals = native.tokens_copy(proposals, k)
```

```
    branch_owner = (owner[0], "@verify-"+owner[1])
    branch = pool.fork(handle, owner, branch_owner)
    try:
        initial = native.full_oracle(pool.model, s.tokens)[-1]
        rows = pool.consume(branch, branch_owner, proposals)
        ps = torch.cat((initial[None], rows[:-1])).softmax(-1).numpy()
        committed, accepted = verify(proposals, ps, qs, rng)
    finally:
        pool.release(branch, branch_owner)
    pool.consume(handle, owner, committed)
    return committed, accepted
```

In this adapter, every returned token has also been consumed into KV. Callers must not consume it again. This differs from Chapter 37's final unconsumed-output convention and requires an explicit integration adapter. The no-bonus variant returns at most k tokens. Standard variants can emit a target bonus after all k proposals are accepted, provided their distribution comes from the full accepted prefix and its cache accounting is correct. Omitting a bonus changes throughput, not the corrected target law.

On capacity failure, the verification fork is released in a `finally` block and the live prefix remains unchanged. Tests force OOM on a shared partial tail and compare payload seals, refcounts and live-handle count. On successful commits, tests reconstruct the whole prefix with the original full model and compare every layer's K/V. Local serial cleanup does not certify concurrent cancellation or crash recovery.

38.4 Acceptance is not a speed measurement

Let N be emitted tokens per no-bonus iteration. It emits at least one; for $j \geq 2$, $N \geq j$ exactly when the first $j-1$ proposals are accepted. The tail-sum identity gives

$$\mathbb{E}[N] = \sum_{j=1}^{k} \Pr(N \geq j) = 1 + \sum_{i=1}^{k-1} \Pr(\text{first } i \text{ accepted}).$$

Under the additional approximation that acceptance events are independent with constant probability A, this becomes $\sum_{i=0}^{k-1} A^i = (1-A^k)/(1-A)$ for $A \neq 1$, and k for $A = 1$. History-dependent real acceptance generally violates that approximation.

For $k = 3, A = .9$, the no-bonus model emits $1 + .9 + .81 = 2.71$ tokens per iteration on average. A bonus variant adds $.9^3 = .729$, giving 3.439. With draft cost kc_d, target verification cost $c_v(k)$ and bookkeeping c_b, estimated time per emitted token is

$$\frac{kc_d + c_v(k) + c_b}{\mathbb{E}[N]}.$$

Compare this with measured target-only per-token time. Do not set $c_v(k)$ equal to a scalar decode cost without profiling. Our adapter also does prefix replay and committed recomputation; it has no speedup claim.

38.5 Attention can avoid a global score buffer

For one query $q \in \mathbb{R}^d$, keys $k_j \in \mathbb{R}^d$, values $v_j \in \mathbb{R}^{d_v}$ and allowed index set J,

$$s_j = q^\top k_j / \sqrt{d}, \qquad y = \frac{\sum_{j \in J} e^{s_j} v_j}{\sum_{j \in J} e^{s_j}}.$$

Direct exponentiation overflows for large scores. Subtracting a common maximum preserves the ratio. To traverse tiles, retain summary

$$m_J = \max_{j\in J} s_j, \quad \ell_J = \sum_{j\in J} e^{s_j - m_J}, \quad a_J = \sum_{j\in J} e^{s_j - m_J} v_j.$$

Here m and ℓ are scalars, $a \in \mathbb{R}^{d_v}$. For several heads/queries, maintain one such state per row. Values need not be positive; the normalizer is positive for every nonempty allowed set.

For disjoint tiles A and B, use $m = \max(m_A, m_B)$ and

$$\ell = e^{m_A - m}\ell_A + e^{m_B - m}\ell_B,$$
$$a = e^{m_A - m}a_A + e^{m_B - m}a_B.$$

This rescales the *old accumulator too*. Updating only the normalizer when a larger maximum arrives produces a wrong weighted value sum.

Proposition 38.3 (Online-softmax invariant)**.** *After any collection of disjoint allowed tiles, the merged summary equals (m_J, ℓ_J, a_J) for their union in real arithmetic. Consequently a_J/ℓ_J is the dense attention result. Merge is associative for nonempty sets under these exact summaries.*

Proof. Each old term obeys $e^{m_A - m}e^{s_j - m_A} = e^{s_j - m}$, and similarly for B. Summing the rescaled normalizers and vector accumulators therefore gives exactly the definitions for the union. The maximum also equals the union maximum. The union operation is associative, so any merge tree gives the same summary. Division cancels the common e^{-m} factor. Induction over tiles proves the online recurrence. □

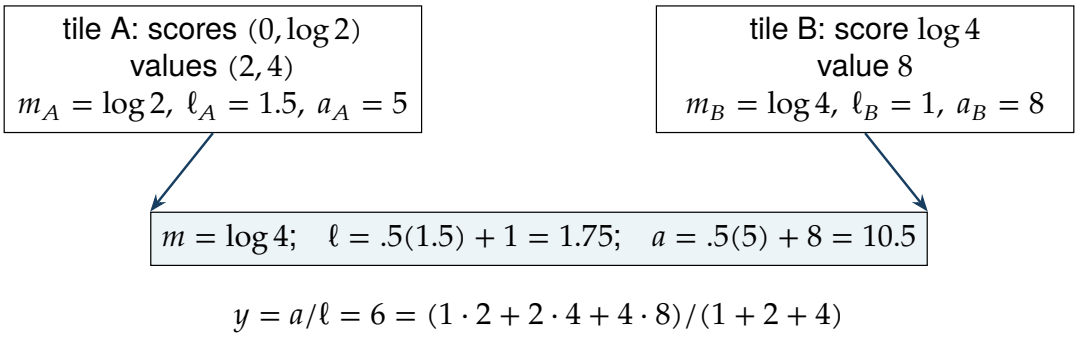


Figure 38.4: A larger second-tile maximum rescales the first tile by one half. The merged output is six, exactly the dense weighted mean.

The empty-set identity needs care: represent it by $m = -\infty, \ell = 0, a = 0$ but never evaluate $-\infty - (-\infty)$. The implementation substitutes a safe finite shift for an entirely masked row; all its exponential weights are zero. It returns zero attention when the full allowed set is empty, matching the earlier chapters' explicit policy. For a nonempty row at least one shifted weight is one, so the denominator cannot vanish ideally. Finite inputs that overflow the score dot product are rejected separately.

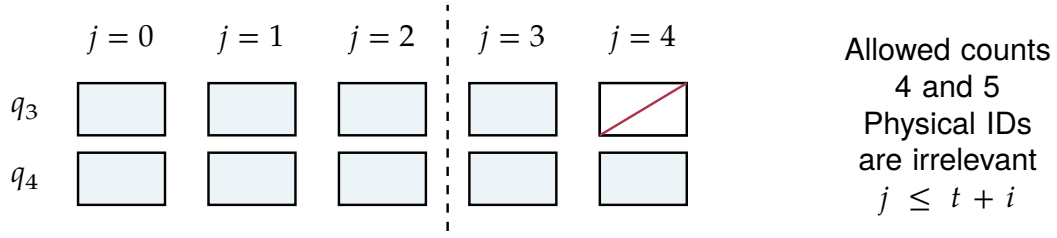
```
def online_attention(q, tiles):
    """Tiles are (K,V,allowed) in logical order, independent of physical IDs."""
    if q.ndim != 3 or min(q.shape) < 1:
        raise ValueError("nonempty H,N,d queries")
    maximum = torch.full(q.shape[:2], -torch.inf, dtype=torch.float64)
    normalizer = torch.zeros_like(maximum)
    accumulator, seen = None, False
```

```
    for k, v, allowed in tiles:
        attention_inputs(q,k,v,allowed)
        if accumulator is None:
            accumulator = torch.zeros(*q.shape[:2], v.shape[-1],
             ↪  dtype=torch.float64)
        elif accumulator.shape[-1] != v.shape[-1]:
            raise ValueError("value width changed")
        scores = q@k.transpose(-2,-1)/math.sqrt(q.shape[-1])
        if not bool(torch.isfinite(scores).all()):
            raise ValueError("score overflow")
        scores = scores.masked_fill(~allowed, -torch.inf)
        new_max = torch.maximum(maximum, scores.max(-1).values)
        safe_max = torch.where(torch.isfinite(new_max), new_max,
         ↪  torch.zeros_like(new_max))
        rescale = torch.exp(maximum-safe_max)
        weights = torch.exp(scores-safe_max[...,None])
        accumulator = rescale[...,None]*accumulator+weights@v
        normalizer = rescale*normalizer+weights.sum(-1)
        maximum, seen = new_max, True
    if not seen:
        raise ValueError("at least one nonempty tile")
    # Explicit all-masked policy: return zero, avoiding 0/0 and -inf-(-inf).
    return accumulator/normalizer.clamp_min(torch.finfo(torch.float64).tiny)[...,
     ↪  None]
```

Floating-point merge is not bitwise associative. Tile sizes and reduction order can change low-order bits. Tests compare tile sizes 1, 2, 3, 7 and oversized tiles against an independent dense implementation and PyTorch 2.10 SDPA, with explicit boolean masks and inference dropout zero [32]. They include extreme finite scores, initially masked tiles and all-masked rows. A normalization proof does not by itself validate every device backend.

38.6 Preserve offset causality and the complete model

With t past tokens and a c-token chunk, local query i is absolute position $t + i$, so key j is allowed exactly when $j \le t + i$. A physical page boundary does not reset that ordinal. For $t = 3, c = 2$, the allowed row counts are four and five, not one and two from an upper-left triangle.



Past $t = 3$, chunk $c = 2$. Dashed vertical line marks the past/new boundary.

Figure 38.5: The same causal mask applies across tiles and scattered pages. The blocked future cell belongs to the new chunk, not to an arbitrary physical page boundary.

`online_consume` replaces only the attention reduction in the actual native stack: learned absolute embeddings, normalized QKV, multi-head output projection, residuals, FFN, final normalization and LM head remain. It preserves all layer caches and the token/owner/artifact contract. Whole-function extraction keeps printed code bound to the tested implementation.

```
@torch.no_grad()
def online_consume(model, cache, owner, tokens, tile=2):
    """Actual complete Hermon stack with online attention and offset masks."""
    paging.positive(tile)
    native.validate(model, cache, owner)
    tokens = native.tokens_copy(tokens, model.max_len-len(cache.tokens))
    if not tokens:
        return torch.empty(0,4,dtype=torch.float64), cache
    past, count = len(cache.tokens), len(tokens)
    z = model.tok(torch.tensor(tokens))+model.pos(torch.arange(past,past+count))
    allowed = native.allowed_mask(past,count)
    layers = []
    for block, old in zip(model.blocks, cache.layers):
        a = block.attn
        q,k,v = a.qkv(block.n1(z)).view(count,3,a.h,a.dh).permute(1,2,0,3)
        k,v = torch.cat((old.k,k),1), torch.cat((old.v,v),1)
        tiles = ((k[:,i:i+tile],v[:,i:i+tile],allowed[:,i:i+tile])
                 for i in range(0,past+count,tile))
        y = online_attention(q, tiles).transpose(0,1).reshape(count,model.d)
        z = z+a.out(y)
        z = z+block.ff(block.n2(z))
        layers.append(native.LayerKV(k.clone(),v.clone()))
    result = native.sealed(owner,cache.artifact,cache.tokens+tokens,layers)
    native.validate(model,result,owner)
    logits = model.lm(model.norm(z))
    if not bool(torch.isfinite(logits).all()):
        raise ValueError("nonfinite logits")
    return logits, result
```

Tile tokens	Max full-model logit error
1	2.22045e-16
2	4.44089e-16
4	2.22045e-16
32	2.22045e-16

The table compares continuation after a three-token prefix across tile sizes against the original full causal model. Additional tests vary every split of the six-token record and compare every layer's K/V. The separate `paged_attention` function visits Chapter 36 pages in table order without flattening them for the attention reduction. Its queries and masks use logical ordinals; a scattered physical layout is a test fixture. That page visitor is not yet the complete model's in-place paged append kernel. The distinction is an explicit remaining backend integration task.

38.7 IO awareness, fusion and evidence

FlashAttention's tiled exact-attention algorithm and on-chip fusion provide the primary systems comparison [83]; FlashAttention-2 further changes parallelism/work partitioning [91]. These are backend designs, not a claim that any Python tiled loop has the same performance. Fusion can reduce global intermediate transfers while increasing register pressure, reducing occupancy or restricting shapes. Measure the combined page size, head geometry, dtype, masking and batch distribution.

For a deliberately narrow one-query traffic model, assume each Q/K/V input is read once and output written once, with element width b bytes. The base payload is

$$b(d + Td + Td_v + d_v).$$

An explicitly materialized score vector that is written once and read once adds $2Tb$ bytes. We exclude probability buffers, cache effects, repeated tile loads, metadata, weights and workspaces. This model cannot stand in for FlashAttention's multi-query IO complexity theorem or measured HBM traffic.

One-query model	Bytes
scores	2114048
online	2097664

For $T = 4096, d = d_v = 128, b = 2$, the isolated saving is 16,384 bytes, less than one percent of this one-query payload model. It would be misleading to turn that number into a universal speedup. Multi-query score matrices and their reuse require a different calculation.

Low-precision tiled attention must satisfy Chapter 37's numerical/task gates; exact algebra does not erase storage or reduction error. Speculative verification must use the intended target policy, including its actual precision. A quantized target's exact sampling law is not automatically the original float64 model's law.

A credible kernel study proceeds from semantics and adversarial parity to declared traffic/work models, then synchronized hardware profiles and finally service measurements. Record warmup, hardware/software versions, repetitions, shape distribution, offered load, TTFT/ITL tails, throughput, memory high-water and rejected/preempted work. Correctness, model arithmetic, CPU experiments, device measurements and end-to-end SLO results are distinct evidence levels.

38.8 Selective glossary

Proposal law Actual conditional distribution used by the draft to draw a candidate.

Residual correction Normalized positive target-minus-draft mass used after rejection.

Accepted prefix Contiguous candidates verified before the first rejection.

Tentative KV Computed state owned by verification, not yet part of committed history.

Online summary Maximum, shifted exponential sum and shifted value accumulator for processed allowed keys.

Exact attention The same mathematical softmax/value map, with finite-precision differences governed by a numerical contract.

IO model Declared transfer-count assumptions; it is distinct from observed hardware traffic or service latency.

Key ideas

- After rejection, sample from the normalized residual $(p-q)_+$. Resampling from p itself gives token zero probability .505 instead of .55.
- Expected acceptance is $1 - \mathrm{TV}(p, q)$, a statement about overlap. It says nothing about draft cost or latency.
- Only the accepted contiguous prefix is reusable. Descendants of a rejected proposal were conditioned on a history that never happened.
- When a larger maximum arrives, the online merge must rescale the old value accumulator as well as the normalizer.

38.9 Twenty worked exercises

1. **Equal laws.** $p = q$? *Solution:* acceptance one; correction branch has zero probability.
2. **Support gap.** $p = (.5, .5), q = (1, 0)$? *Solution:* token zero accepted with .5; rejection corrects to token one.
3. **Compute overlap.** Supplied four-token example? *Solution:* $.45 + .25 + .15 + .05 = .90 = 1 - \mathrm{TV}$.
4. **Reject the wrong fallback.** Sample p after rejection? *Solution:* token-zero probability becomes .505 instead of .55.
5. **Conditional rows.** Which prefix supplies p_3? *Solution:* original h followed by proposed x_1, x_2.
6. **First rejection.** Why discard x_3 when x_2 rejects? *Solution:* x_3 used a history that was not committed.
7. **Transforms.** Must draft temperature equal target temperature? *Solution:* no; ratio must use actual q and intended transformed p.
8. **Reproduce seeds.** Does same seed force target-only equality? *Solution:* no; draw counts/order differ even when laws agree.
9. **Shift logits.** Which row verifies first proposal? *Solution:* last old-prefix logits, before consuming that proposal.
10. **Commit state.** Can a rejected suffix stay in live KV? *Solution:* no; discard tentative ownership and install only committed history.
11. **Compare API clocks.** Returned native speculative tokens? *Solution:* already consumed; avoid consuming them a second time.
12. **Expected count.** No bonus, $k = 3, A = .9$ independent? *Solution:* $1 + .9 + .81 = 2.71$ emitted, not $3(.9)$.
13. **Add a bonus.** Same approximation? *Solution:* add .729 to get 3.439; this requires the extra target conditional.
14. **Merge summaries.** Plate's (m, ℓ, a)? *Solution:* $(\log 4, 1.75, 10.5)$; ratio six.
15. **Find an unstable update.** Rescale ℓ but not a? *Solution:* plate would use $a = 13$ instead of 10.5, giving a wrong mean.

16. **Empty masks.** How avoid $-\infty - (-\infty)$? *Solution:* safe finite shift for empty rows, zero weights and explicit zero output.
17. **Offset mask.** Past three, chunk two? *Solution:* allowed counts four and five across all page tiles.
18. **Traffic.** $2Tb$ for $T = 4096, b = 2$? *Solution:* 16,384 isolated score-buffer bytes, not measured speed.
19. **Design a distribution test.** *Rubric:* include zero support, $p = q$, a history-dependent exact enumeration, seeded statistics with fixed tolerances and a known wrong correction control.
20. **Design a backend release gate.** *Rubric:* whole-stack and per-layer parity, page-order/mask adversaries, precision/task gates, rollback/completion fences, synchronized device profiles and service load curves.

38.10 Closing the two engine directions

Chapters 30–34 supplied runtime/state-native references for a restricted DOGMA teaching recurrence; Chapters 35–38 now supply causal Hermon Transformer cache, allocation, scheduling and exact-attention references. Neither establishes equal learned capability or universal performance superiority. Evolutor's next layer must route typed work above those distinct semantics without exposing either engine's mutable internals as a universal state format. Chapter 39 owns that heterogeneous runtime boundary.

Sources and further reading

Source access checked 17 October 2026. Leviathan et al.'s primary HTML Section 2/Algorithm 1 and Appendix A.1 proof were inspected. Chen et al.'s Algorithm 2 and Section 4 conditional-scoring discussion were inspected; that paper reverses our target/draft letter convention. FlashAttention's primary HTML online-softmax derivation, Algorithm 1 and correctness/IO theorem statements were inspected; no CUDA implementation or empirical result was reproduced. FlashAttention-2 was inspected at abstract depth only. PyTorch 2.10 SDPA's boolean-mask, non-square causal, dropout and backend numerics documentation was inspected, and local tests use its CPU float64 path.

The speculative block uses actual history-conditioned proposals and native paged speculative state rather than a list of unconditional distributions. Complete Hermon online-attention continuation is tested separately from the paged attention visitor; neither is represented as a measured GPU kernel.

PART IX

Evolutor: heterogeneous execution

What does a compiler and runtime above both engines owe to the computations it coordinates?

Evolutor's own layer appears here: typed workflow plans and capability routing, opaque engine handles, provenance-aware hybrid memory, an effect-aware SSA compiler, a disciplined comparison with database engines and a governed lifecycle for structural adaptation. The common thread is that coordination may change where and when work runs, but never what it means, who owns it or who authorized it. Retrieved text is evidence; it never carries authority.

CHAPTERS IN THIS PART

39 **Evolutor runtime above both engines** Orchestrate heterogeneous engines under explicit common contracts.

40 **Hybrid compressed and addressable memory** Combine compressed and addressable memory with testable policies.

41 **Compiler, IR and execution planning** Lower typed IR through legal transformations and costed plans.

42 **Database engines as a systems comparison** Use database execution as a bounded systems comparison.

43 **Structural adaptation and lifecycle governance** Govern structural proposals, promotion, lineage and rollback.

CHAPTER 39

Evolutor runtime above both engines

In this chapter you will

- represent a heterogeneous workflow as a typed, single-assignment plan and validate it before any effect
- separate planning from routing, and route on semantic capability before cost
- address engine state through opaque, owner-bound handles
- separate planned, authorized and completed tool effects, and trace execution without claiming hidden reasoning

Consider a workflow with five operations: retrieve a document containing *memory*, consume tokens $(1, 2, 1)$ in a native recurrence, consume the same token IDs in a Transformer, add integers 7 and 8, and echo the sum through a permitted tool. Predict which outputs can be compared directly. The sum is 15; the retrieved document is evidence; the two engine calls produce different logit spaces. Neither engine's hidden state can be handed to the other.

This chapter builds Evolutor's coordination boundary above those distinct computations. DOGMA is the non-Transformer direction; Hermon DNA is the Transformer direction. The executable DOGMA-side fixture remains the *restricted Chapter 31 diagonal teaching recurrence*, not a complete trained DOGMA model. The Hermon side imports Chapter 35, which serves the actual Chapter 23 multi-layer Transformer with learned absolute positions. These are executable continuity contracts, not evidence that the workflow solves a learned genomic task.

39.1 Make the workflow a typed object

Evolutor owns the path from an objective to a plan, route, execution and observable trace. It does not own the internal algebra of every backend. The **planner** decides which operations are needed and their dependencies. The **router** chooses an eligible implementation of one already specified operation. A bad dependency graph and a missing backend are different errors.

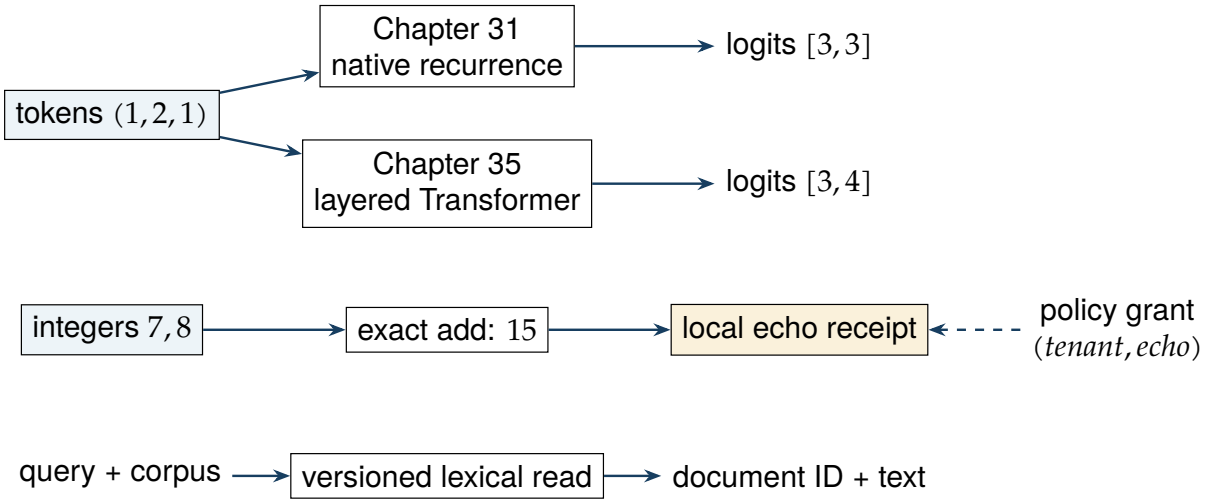


Figure 39.1: The executable fixture has typed values and separate control authority. Equal token IDs do not imply equal vocabulary semantics or engine outputs.

Represent a value by (τ, x, π): type τ, payload x, and provenance π. Our finite alphabet contains integers, token sequences, logits, documents and tool receipts. A token tuple alone is not a universal text representation. The Chapter 31 fixture has vocabulary size three; Hermon's causal ACGT head has size four. Deliberately using IDs in their intersection permits a systems test but does not establish equivalent inputs as language or biology. A real cross-model workflow needs an explicit tokenizer or record conversion operation, with its own loss and coordinate contract.

A step is

$$s = (id, op, I, o, \tau_o, A, C),$$

where I names previously defined inputs, o is its new output name, A contains operation-specific attributes, and C is a required engine capability when appropriate. The plan pins a tenant/request pair, immutable input values, an ordered step tuple and requested outputs. The tuple is a topological order of a straight-line graph; branches, loops and concurrent effects are outside this reference. Chapter 41 will make effects and lowering explicit in an IR.

Operation	Input type	Output type
engine	tokens	backend-specific logits
retrieve	version, query, immutable corpus attributes	documents
symbolic	integer, integer	integer
tool	integer, configured tool name	receipt

The source supplies `Step`, `Plan`, `Value` and `validate`. Validation checks the entire graph before opening an engine or calling a tool: unique IDs and SSA outputs, defined inputs, exact operation signatures, required attributes and defined final outputs. Integer validation excludes Python's Boolean subtype. A malformed later operation cannot be discovered only after an earlier effect has run.

This is type preflight, not omniscience. A well-typed token can still exceed one model's vocabulary or remaining context limit. The backend validates those value-dependent constraints immediately before arithmetic. Likewise, an earlier valid tool can have run before a later backend fails. We do not claim that a plan is one atomic transaction.

39.2 Route semantic requirements before optimizing cost

For step s, let C_s be required capabilities and Q_s hard constraints. Backend e is eligible only when

$$C_s \subseteq C_e, \qquad Q_s(e) = \text{true}.$$

Artifact, device, precision, state continuity, tokenizer and policy constraints belong in Q_s. Our minimal router requires one of `compressed_state` or `addressable_kv` and an exact artifact identity. It fails on zero matches or ambiguity. Multiple real eligible replicas could use an explicit tie-break and cost policy; list order should not silently choose semantics.

```
    def route(self, capability, artifact):
        matches = [e for e in self.engines if e.capability == capability
                   and e.artifact == artifact]
        if len(matches) != 1:
            raise RuntimeError("route must resolve uniquely with exact artifact")
        return matches[0]
```

The reference maps the former capability to the Chapter 31 adapter and the latter to Chapter 35. This does not mean that every task has a unique natural architecture. It means that an operation requiring continuation of a particular native state cannot be satisfied by an unrelated backend merely because both return logits. Fallback is legal only when the alternate implementation preserves the same observable contract. Otherwise the planner must create a new legal graph and record the semantic change.

After hard eligibility, a future policy may minimize

$$J(e) = w_L \hat{L}_e + w_M \widehat{M}_e + w_C \widehat{C}_e + w_R \widehat{R}_e.$$

Hats denote predictions: latency, memory, monetary or energy cost, and a defined quality-risk proxy. Weights convert their units to a common score. A large penalty is not a substitute for an authorization or compatibility constraint. No timings or learned quality estimates are measured here; Chapter 47 owns calibrated backend evidence.

39.3 Keep complete state private and identity explicit

An **opaque handle** contains adapter-instance identity, backend, artifact, tenant/request and serial number. The adapter maps it to its private native state. Equal shapes or equal numeric serials do not imply ownership.

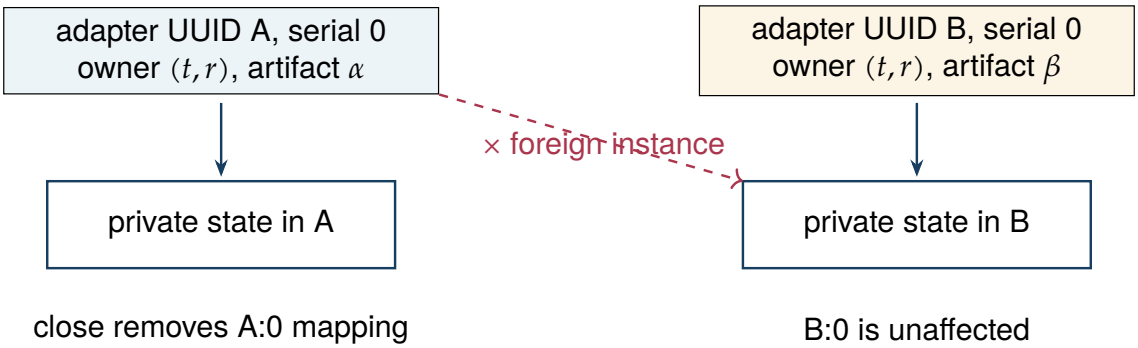


Figure 39.2: Handles address one live adapter mapping. Closing a handle invalidates that mapping; a different instance's serial zero remains distinct.

For Chapter 31, the sufficient state contains position t and hidden vector $h_t \in \mathbb{R}^D$, with fixed artifact parameters:

$$h_{t+1} = a \odot h_t + \sigma(w \odot E_{x_{t+1}}) \odot E_{x_{t+1}}, \qquad z_{t+1} = h_{t+1}^{\mathsf{T}} U.$$

Chapter 35 instead holds each layer's ordered $K^\ell, V^\ell \in \mathbb{R}^{H \times t \times d_h}$, consumed-token metadata, causal mode and artifact binding. It computes learned position P_t for the next consumed token. There is no lawful conversion between these objects implied by this shared handle envelope.

Shared method names such as `open`, `execute` and `close` promise lifecycle behavior, not state representation. A checkpoint operation could be added only with its native exact-continuation contract: Chapter 33 has a concrete DOGMA checkpoint; a Hermon checkpoint requires its complete KV and placement rules. This chapter does not invent a universal checkpoint encoder or claim Chapter 36 page integration.

```
    @torch.no_grad()
    def execute(self, handle, owner, tokens):
        state = self._check(handle, owner)
        if (type(tokens) is not tuple or not tokens or len(tokens) > 512 or
                any(type(x) is not int for x in tokens)):
            raise ValueError("nonempty bounded concrete tokens")
        if self.backend == "dogma":
            candidate, outputs = self.native.ingest(self.model, state, tokens,
             ↪ owner)
            logits = torch.stack(outputs)
        else:
            logits, candidate = self.native.consume(self.model, state, owner,
             ↪ tokens)
        # Both native routines preflight and return fresh state before this
         ↪ commit.
        self._states[handle.serial] = (handle, candidate)
        return Value(Ty.LOGITS, tuple(tuple(x) for x in logits.tolist()),
                     (self.backend, self.artifact))
```

Both imported routines validate before returning fresh candidate state. The adapter publishes that candidate only after successful execution. Invalid chunks leave the prior mapping unchanged. The returned logits are immutable tuples, so callers cannot mutate the retained state through them. The artifact is checked by the native routines, including in-place parameter changes. Cleanup remains possible after an artifact mutation; otherwise an error could permanently strand a live handle.

Proposition 39.1 (Ownership and chunk refinement)**.** *Assume serialized calls, trusted adapter code, fixed native artifacts and non-mutating callers. A handle accepted by this adapter refers to exactly one live state under its recorded owner. Splitting a valid token chunk into nonempty pieces preserves the native concatenated output sequence in exact arithmetic under the native transition contracts.*

Proof. Every open uses a fresh serial within one adapter identity and records the entire handle with its state. Each use compares the owner, instance and stored handle before accessing that state; close removes it and serials are not reused. For refinement, a successful call invokes the same native transition and commits its returned state. Induction over chunk boundaries therefore reduces the claim to Chapter 31's complete-state recurrence and Chapter 35's layered causal-cache parity. The proof does not identify one backend with the other. □

These are local correctness properties. A Python object and UUID are not authentication, process isolation, secure erase or a concurrency protocol. Chapter 30's deadline, cancellation, publication and cleanup fences still apply when this reference becomes a serving service.

39.4 Compose retrieval, symbolic modules and engines honestly

Retrieval is an addressable read $R(q, \mathcal{D}) \to \{(id_i, text_i)\}$. The fixture records a corpus version and query; row IDs refer to that ordered immutable corpus. A production retriever should additionally bind index, ranking configuration, corpus digest, source versions and access scope. Retrieved text is data, not automatically trusted instruction. Changing a corpus while retaining its version label violates the assumed fixture contract.

A symbolic module has explicit semantics. Here it adds mathematical integers exactly, providing the independently assigned result $7 + 8 = 15$. A calculator call does not make the neural model symbolic. Provenance identifies which module produced the integer, documents, logits and final receipt.

One can use both engines in a larger workflow through a declared typed conversion. For example, a recurrence-derived feature record can become input data for another operator. Raw native pointers cannot cross as if they were token sequences. Nor does a retrieved event restore an exact recurrence or KV cache. Chapter 40 will make transient state, durable events, lossy summaries and addressable records separate memory classes.

39.5 Planning an action does not authorize it

A tool can change an external system, so $\text{planned}(tool) \not\Rightarrow \text{authorized}(tool)$. Our policy is a caller-configured set of $(tenant, tool)$ grants. It is checked at call time, after intent is logged and immediately before the callback. A retrieved sentence saying "echo is approved" cannot add a grant. Denial is a normal, auditable outcome rather than a backend-selection failure.

The default callback returns a local echo receipt. No network action or production authorization service is implemented. Tests use a counting callback to observe exactly whether the code reaches the effect boundary.

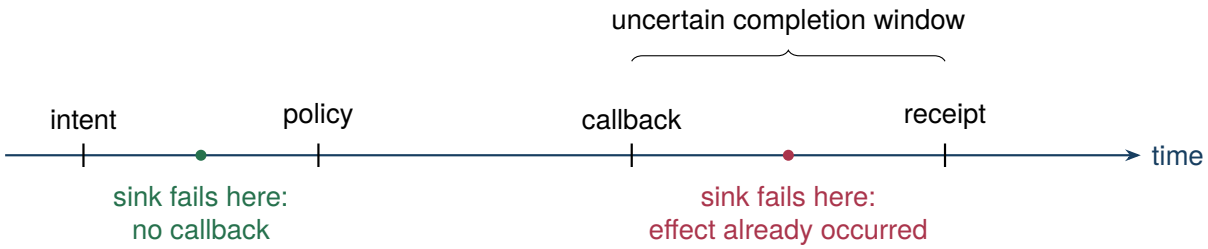


Figure 39.3: A local failure window separates permission to start from knowledge that an effect completed. An error after the callback does not imply rollback.

Three failure points differ. If the trace sink fails before intent, no callback runs. If policy denies the call, no callback runs. If the callback succeeds but writing its receipt fails, the runtime can no longer infer from the exception that nothing happened. Blindly retrying may repeat a non-idempotent action. Real services need effect identities, durable intent, reconciliation and tool-specific idempotency or compensation. Those mechanisms cannot be obtained by wrapping the entire plan in a Python `try` statement.

39.6 Trace observable execution and close resources on failure

The runtime emits plan, step, route and result facts. Each event has sequence, kind, metadata and the previous event digest; the record stores a canonical JSON body and its full SHA-256. Result digests identify payload bytes under this encoding. The hash chain detects accidental changes to a captured chain; an actor who can replace the chain can recompute it. This is neither an authenticated audit log nor a durable append-only storage system.

Trace is not hidden model reasoning. It says which step ran, which backend and artifact were selected, and which result or policy event occurred. Given deterministic modules, pinned inputs/artifacts and captured effect results, an execution can be replayed according to that declared observation contract. Changing retrieval corpora, clocks, external tools, random streams or numerical kernels requires additional capture and a tolerance policy.

```
    def execute(self, plan):
        validate(plan)  # reject the whole malformed graph before any effect
        routes = {s.sid: self.route(s.capability, s.payload)
                  for s in plan.steps if s.op == "engine"}
        env = dict(plan.inputs)
        self.trace.emit("plan_start", owner=plan.owner)
        for s in plan.steps:
            args = [env[x].data for x in s.inputs]
            self.trace.emit("step_start", sid=s.sid, op=s.op)
            if s.op == "engine":
                e = routes[s.sid]
                self.trace.emit("route", sid=s.sid, backend=e.backend,
                                artifact=e.artifact)
                h = e.open(plan.owner)
                try:
                    value = e.execute(h, plan.owner, args[0])
                    self.trace.emit("engine_result", sid=s.sid,
                                    result=digest(value.data))
                finally:
                    e.close(h, plan.owner)
            elif s.op == "retrieve":
                version, query, corpus = s.payload
                hits = tuple((i, doc) for i, doc in enumerate(corpus)
                             if query.casefold() in doc.casefold())
                value = Value(Ty.DOCS, hits, (version, query))
            elif s.op == "symbolic":
                value = Value(Ty.INT, args[0] + args[1], (s.sid, "exact-add"))
            else:
                # Intent must be recorded before policy/callback. A receipt
                ↪ failure
                # after the callback gives an uncertain effect, not a safe retry.
                self.trace.emit("tool_intent", sid=s.sid, name=s.payload)
                if not self.policy.authorize(plan.owner, s.payload):
                    self.trace.emit("tool_denied", sid=s.sid)
                    raise PermissionError(s.payload)
                receipt = self.tool(s.payload, args[0])
                value = Value(Ty.RECEIPT, receipt, (s.sid, "policy-checked"))
            env[s.output] = value
            self.trace.emit("step_result", sid=s.sid, result=digest(value.data))
        self.trace.emit("plan_complete", outputs=plan.outputs)
        return tuple(env[x] for x in plan.outputs)
```

The `finally` block closes an opened engine even if computation or result tracing fails. It does not reverse already executed earlier steps. Trace sink failures can also prevent the system from recording a later failure; the caller must retain the exception and reconcile service state. The short reference intentionally avoids claiming a complete durable failure reporting protocol.

39.7 Read the generated result and its negative controls

Step	Operation	Type	Rows/items
ret	retrieve	docs	1
d	engine	logits	3
h	engine	logits	3
sum	symbolic	int	1
act	tool	receipt	2

Rows/items count the concrete returned tuple in this fixture, not equal semantic content. Native state maps are empty at plan completion and the exact sum is 15. CPU float64 tests compare every output row across chunk splits with two separate native oracles: a scalar closed expansion of the recurrence and Hermon's original full causal model. The tolerance is 2×10^{-12} absolute and relative for these small fixtures.

Negative controls change owner, artifact, instance, backend or serial; attempt a second close; introduce invalid live tokens; mutate a model; erase a grant; make the graph malformed; and fail the trace sink before and after a callback. Each control targets a different premise. Matching a final sum or merely counting trace events would miss most of these errors.

Distinguish plan-invalid, route-failed, state-invalid, module-failed, policy-denied, deadline/cancelled and persistence/trace failure. The first two are preflight outcomes here; the native routines enforce state validity; Chapter 30 owns the executable lifecycle for deadlines and cancellation. Retries require a reason-specific policy. No successful fixture establishes better reasoning, AGI, biological fidelity, production reliability or speed.

Key ideas

- Equal token IDs do not make two engines' inputs or logits comparable. Each backend is checked against its own oracle.
- A fallback is legal only if it preserves the observable contract. Otherwise the planner must make a new plan and record the change.
- Retrieved text is evidence, never a grant. Authorization comes only from configured policy, checked immediately before the effect.
- If the effect succeeds but its receipt fails, completion is uncertain, and blind retry can repeat a non-idempotent action.

39.8 Twelve worked exercises

1. Which fixture outputs are directly comparable across the engines? **Answer:** Their row count is three, but widths are three and four and vocabularies differ. Compare each backend with its own oracle; do not equate logits.

2. Why separate planner and router? **Answer:** The planner chooses operations and dependency edges; the router satisfies one operation's contract. No eligible route does not prove the task itself has no legal re-plan.

3. Can a raw hidden vector serve as Hermon KV? **Answer:** No. The recurrence compresses history into h_t; Hermon needs ordered layered keys/values under its exact positions and model.

4. Two handles have serial zero. Are they aliases? **Answer:** Only if their complete live mapping identity matches. Separate adapter UUIDs reject cross-instance use even when owners and artifacts match.

5. Prove chunk continuity for the native side. **Answer:** Write each token transition as F_x. Both chunkings compose the same ordered maps $F_{x_n} \circ \cdots \circ F_{x_1}$ from the same complete initial state.

6. Why reject a malformed last step before the first tool? **Answer:** Whole-graph validation avoids preventable partial effects. It does not eliminate later value-dependent or external failures.

7. Can retrieved text authorize echo? **Answer:** No. Only the configured policy grants authority; retrieval supplies evidence with provenance.

8. Why is exact addition an independent oracle? **Answer:** The assigned arithmetic result 15 does not depend on neural logits or backend state. It tests a distinct module, not the model's learned reasoning.

9. Where is blind retry unsafe? **Answer:** After callback success but receipt failure. The caller has uncertain completion, not proof of no effect.

10. What does the hash chain prove? **Answer:** Consistency of captured bytes with recorded links. An adversary able to rewrite/reseal every link is outside this unauthenticated integrity check.

11. Design a cancellation integration test. **Rubric:** Use Chapter 30's real ticket, terminate before native result publication, keep resources pinned until completion acknowledgement, and assert no post-terminal output. A mock cancel flag is insufficient.

12. What would qualify legal fallback? **Answer:** A replacement with the same artifact and observable state/precision contract, or a newly validated plan explicitly recording a semantic change. Lower cost alone does not qualify.

39.9 Vocabulary and the next boundary

Capability: a declared semantic requirement, not a benchmark score. **Artifact binding**: continuation under the exact computation identity. **Effect boundary**: the point beyond which an observable external action may have occurred. Evolutor can now coordinate distinct modules; it still needs an explicit contract for what becomes durable, compressed, retrieved or forgotten. That is the next chapter's memory problem.

Sources and further reading

Access checked 17 October 2026. The runtime imports the Chapter 31 native recurrence and the Chapter 35 layered cache over the actual Chapter 23 Transformer, and their full source interfaces were inspected. Chapter 30's lifecycle, Chapter 32's ownership and Chapter 33's checkpoint contracts were read for the integration boundary; this serialized reference does not replace them.

The typed workflow, ownership proof, failure windows, figures, code and exercises are chapter-defined engineering contracts. No external novelty, biological-capability or performance claim depends on them. Physical allocation is connected through this chapter's adapter protocol by the companion `examples/integrated_spine/paged_runtime.py` (Chapter 41); serving-shell integration remains outside this chapter. Local author review is not independent specialist review or acceptance.

CHAPTER 40

Hybrid compressed and addressable memory

In this chapter you will

- distinguish engine-native state, durable events, lossy summaries and addressable records
- define availability so that expiry and deletion propagate through a provenance DAG
- witness exact information loss with a compression collision
- bound rehydration and working-context promotion, and keep memory outside the authority boundary

A source event says "alpha beta omitted-x." Another says "alpha beta omitted-y." A compressor retaining the first two distinct words gives both the same text. Can the agent later tell which event occurred? A larger context window does not answer this question. The system must preserve the source record, its identity and the link from summary to source—or explicitly accept that the distinction is gone.

Chapter 20 derived compressed numerical memory and an order-loss counterexample. Chapter 35 built token-addressable Transformer execution state. Chapter 39 coordinated those engines with retrieval and tools. Here we add a separate memory contract: durable events, lossy derived records, addressable reads, bounded working context and explicit unavailability. The reference is a serialized in-memory specification of those rules. Calling its objects durable names their intended lifecycle; it does not make this Python dictionary survive a process crash.

40.1 Four memory classes answer different questions

Engine-native transient state supports numerical continuation. For the restricted native recurrence it includes hidden vector and token position; for Hermon it includes every layer's ordered KV and model/position metadata. A checkpoint may persist those bytes, but that does not make them searchable semantic evidence.

An immutable **durable event** records an occurrence under a tenant and schema. A **summary** is a lossy derived record with explicit source edges. An **addressable record** has a stable identity that a lexical, vector, graph or temporal index can return. Addressability is an access property; events and summaries can both be addressable. These are not four disjoint physical storage devices.

A native checkpoint can be useful for exact continuation but poor for semantic search. A document can answer a retrieval query yet be insufficient to reconstruct the exact numerical state. A system may keep both, linked through provenance and artifact identities. It must not serialize a retrieved paragraph into a state tensor and claim exact continuation without a tested decoder relation.

40.2 Define identity, time and availability before ranking

Our record is

$$R = (id, tenant, kind, payload, sources, t_c, t_e, artifact).$$

t_c and optional t_e count *logical integer ticks*, not wall-clock seconds. A record expires at $t \geq t_e$, including equality. An event's artifact names its schema; a summary's artifact names the compressor and word limit. Production records would also pin data/prompt/policy versions, creation authority, quality evidence and content encodings.

The ID hashes the record fields and a fresh store serial using SHA-256. Including serial prevents a forgotten event from being resurrected by reinserting identical text under the same ID. An explicit later event with equal payload is a new occurrence; it does not restore old provenance edges. These hashes do not authenticate producers, protect a hostile object store or replace a transactional ID service.

Define $L_t(r)$ as the conjunction of existence, tenant match, absence of a tombstone and non-expiry. For an event, availability is $A_t(r) = L_t(r)$. For a derived record,

$$A_t(r) = L_t(r) \ \wedge \bigwedge_{s \in sources(r)} A_t(s). \tag{40.1}$$

This is a strict evidence policy: if any ancestor is unavailable, the summary is unavailable. Another system might return a stale summary with explicit warnings, but silently presenting it as current evidence is a different contract. Read-time validation dominates index contents.

```
    def _available(self, tenant, rid, seen, memo=None):
        memo = {} if memo is None else memo
        if rid in memo:
            return memo[rid]
        if rid in seen:
            return False  # defense if trusted internal storage is corrupted
        r = self.records.get(rid)
        if (r is None or r.tenant != tenant or rid in self.tombstones or
                (r.expires is not None and self.clock >= r.expires)):
            return False
        result = all(self._available(tenant, x, seen | {rid}, memo) for x in
         ↪  r.sources)
        memo[rid] = result
        return result
```

Summary creation accepts only existing available distinct source IDs. Edges therefore point to older registered objects, forming a DAG. The store bounds provenance depth to 32 and total records to 10,000; it also detects cycles if trusted internal storage is corrupted. The recursive checker memoizes shared ancestors within one availability call, bounding it by the reachable vertices and edges. Retrieval repeats that check for candidate records. A production DAG still needs incremental invalidation and deletion-transaction design; no scalable lookup claim follows from this small reference.

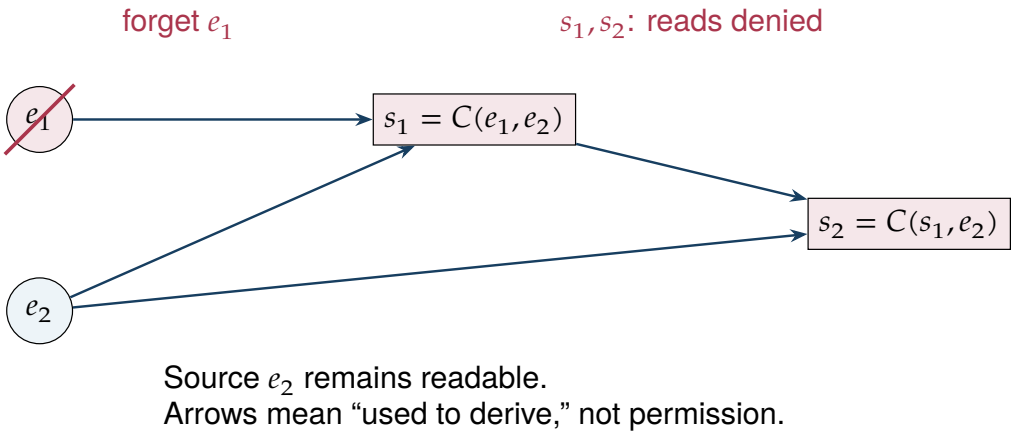


Figure 40.1: Deletion propagates through evidence dependencies. The surviving source remains readable, but both derived records fail the strict ancestor-availability rule.

Proposition 40.1 (Deletion dominance under fixed records)**.** *Assume a finite acyclic source graph, fixed record bytes, nondecreasing time, and tombstones that are never removed. Once a record is unavailable because of expiry or a tombstoned ancestor, no later read of that record becomes available merely by updating or retaining an index entry.*

Proof. For a source event, neither time advancing nor adding tombstones can turn L_t from false to true. Induct in source-before-derived order. Every derived availability is a conjunction of its own local predicate and already monotone ancestor predicates. That conjunction cannot turn true after one such cause makes it false. Ranking does not appear in Equation 40.1, so stale index entries cannot alter it. □

The fixed-record premise matters. A corrected event is a new record and requires new summaries. Editing bytes in place would invalidate both identity and this reasoning.

40.3 Compression preserves provenance, not all information

The reference compressor keeps the first distinct words encountered over ordered sources. We use that transparent rule because it makes loss inspectable, rather than pretending an untrained summarizer has learned semantic retention. Source IDs remain ordered and the compressor version includes the word limit.

```
    def summarize(self, tenant, source_ids, max_words=12, ttl=None):
        bound(max_words, 1, 1000)
        if (type(source_ids) is not tuple or not 1 <= len(source_ids) <= 64 or
                len(set(source_ids)) != len(source_ids)):
            raise ValueError("bounded distinct source tuple")
        sources = [self.get(tenant, rid) for rid in source_ids]
        words = list(dict.fromkeys(w for r in sources for w in r.payload.split()))
        payload = " ".join(words[:max_words])
        return self._put(tenant, "summary", payload, source_ids, ttl,
                         f"first-distinct-words-v1:{max_words}")
```

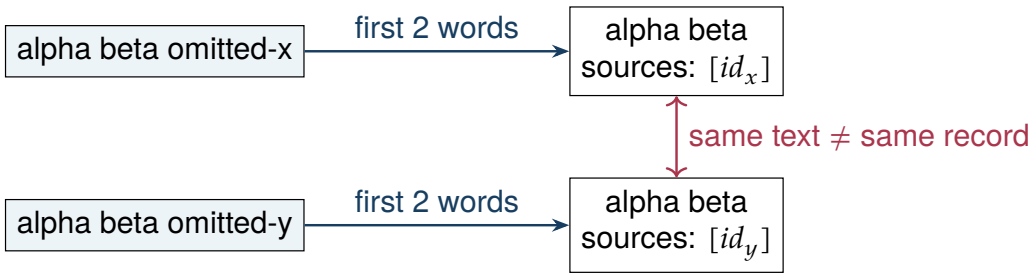


Figure 40.2: A payload collision is an exact information-loss witness. Provenance can distinguish the records even when their summary texts are identical.

For $C(X) = Z$, if $C(X_1) = C(X_2)$ for distinct source histories, no function of Z alone can always recover the history: the same input Z would require two outputs. This elementary non-injectivity argument is independent of floating-point precision. Source expansion is not a general inverse of C; it is an addressable read of separately retained evidence.

Measure compression and usefulness separately:

$$\rho_{payload} = \frac{\text{bytes}_{UTF8}(Z)}{\sum_i \text{bytes}_{UTF8}(X_i)}, \qquad U = \text{quality}(task \mid Z).$$

The generated fixture stores the exact numerator and denominator in `results.json`. This ratio excludes IDs, source edges, object overhead, index entries and retained original records. Keeping the originals *and* adding a summary increases total stored bytes even if its payload ratio is small. A summary useful for approximate topic recall can fail exact quotation, chronology or a rare identifier query. No utility measurement is implemented by counting words.

40.4 Rank records only after scope and provenance checks

The addressable query returns ordered record IDs with payloads after tenant, retention and ancestor checks. The reference lexical score is the number of distinct lowercase ASCII alphanumeric query terms also present in a record. Scores are nonnegative integers, not probabilities or semantic similarity measurements. Zero-overlap records are omitted. Ties use creation tick and full record ID, making the result deterministic within a fixed store image.

An optional candidate-ID list models a stale index. Before returning a hit, the object store rechecks current availability. A tombstoned ID still present in that list cannot bypass deletion; duplicates are removed. Cross-tenant candidate IDs produce no hits. Direct foreign-object reads raise a permission error in this teaching contract. Revealing existence through error differences can itself require a different policy in a hostile multi-tenant service.

Retrieval should be conditional on an evidence need, not blindly invoked before every token. MemGPT motivates explicit context tiers and movement between them [93]. Self-RAG explicitly makes retrieval adaptive [94]. Those narrow precedents do not establish that this lexical ranker reproduces either system, nor that a retrieved result is correct or safe. Their reported benchmark comparisons are not our evidence.

40.5 Bound source expansion and working-context promotion

For summary s, rehydration traverses source links breadth-first, visiting each ID at most once in its returned expansion. It accepts both a record-count ceiling B_n and a UTF-8 payload-byte ceiling B_b. The first whole record that would exceed the remaining byte budget stops traversal. This deterministic prefix policy favors provenance order rather than solving a knapsack problem.

```
    def rehydrate(self, tenant, summary_id, max_sources=4, max_bytes=1000):
        bound(max_sources, 0, 64); bound(max_bytes, 0, 100000)
        root = self.get(tenant, summary_id)
        if root.kind != "summary":
            raise ValueError("summary required")
        pending, visited, result, used = list(root.sources), set(), [], 0
        # Traverse unique nodes breadth-first. Existing-source DAG creation and
        # strict summary availability make every visited ancestor readable.
        while pending and len(result) < max_sources:
            rid = pending.pop(0)
            if rid in visited:
                continue
            visited.add(rid)
            r = self.get(tenant, rid)
            size = len(r.payload.encode("utf-8"))
            if used + size > max_bytes:
                break  # deterministic prefix policy, no partial records
            used += size
            result.append(r)
            pending.extend(r.sources)
        return Context(tuple(result), used)
```

If returned records are $r_1, \ldots, r_m$, the maintained invariant is

$$m \le B_n, \qquad used = \sum_{j=1}^{m} |payload(r_j)|_{UTF8} \le B_b.$$

Initially both sums are zero. A record is appended only if its full byte size fits; then both count and sum advance together. The loop guard enforces the count. This proves the payload bound by induction, not a bound on Python object memory or tokenizer context length. Repeated paths to a shared source do not duplicate that source in the result.

Promotion selects whole readable records into a **working set** under a declared byte ceiling. Unlike rehydration's prefix, the direct promotion operation is all-or-error: if the requested set is too large, it returns no partial context. Both interfaces are explicit so callers can choose rather than infer an accidental truncation rule.

```
    def promote(self, tenant, ids, max_bytes):
        bound(max_bytes, 0, 100000)
        if type(ids) is not tuple or len(ids) > 64 or len(ids) != len(set(ids)):
            raise ValueError("bounded unique record IDs")
        rows = tuple(self.get(tenant, rid) for rid in ids)
        size = sum(len(r.payload.encode("utf-8")) for r in rows)
        if size > max_bytes:
            raise MemoryError("whole working set exceeds UTF-8 payload budget")
        return Context(rows, size)  # never carries grants or tool permissions
```

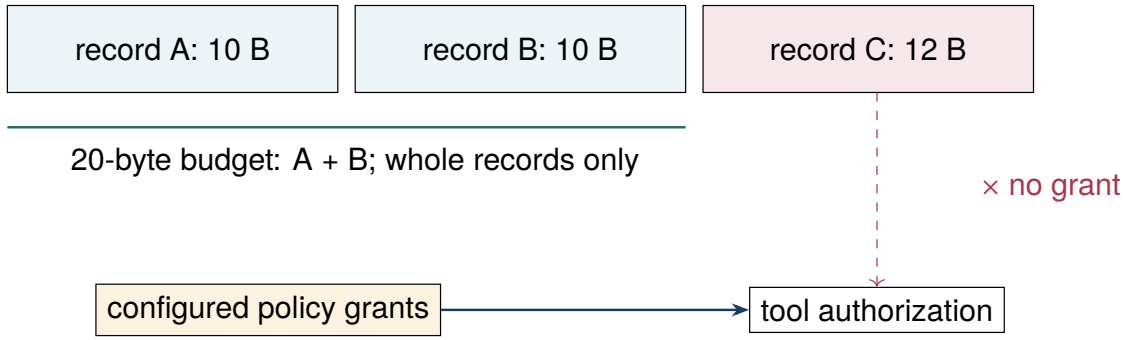


Figure 40.3: Payload budgets limit context movement. Authorization comes from the separate policy configuration, never from the selected record text.

UTF-8 bytes are not tokens. Two Chinese characters consume six UTF-8 bytes before spaces or other text; their tokenizer length depends on the exact tokenizer artifact. A real planner must reserve engine-context tokens after the declared conversion, as well as storage and transport bytes.

40.6 Retention, forgetting and the evidence firewall

Expiry expresses a declared availability horizon. Forgetting adds a tombstone even for an already expired record and is idempotent. It does not delete the dictionary payload. Retention policy may additionally require physical erasure, backup expiry, legal holds, index compaction or derived-data destruction; those actions are not implemented here.

An already returned immutable context still contains its payload after a source is forgotten. The store can reject future promotion, but it cannot recall bytes already issued to a client or a running model. Strong revocation therefore needs request-time fences, context leases or cancellation and an explicit policy for in-flight work. Claiming instantaneous distributed erasure from a local tombstone test would be false.

The **memory safety firewall** is a semantic authority boundary. Every `Context` has an empty authority tuple; memory operators cannot mint Chapter 39 policy grants. Retrieved text can contain stale facts, secrets or prompt injection. Provenance and quoting let a planner reason about it as evidence. Authorization remains an independent configured decision before an action. This small typed separation is not a production prompt-injection defense or proof of hostile-process isolation.

40.7 Consistency and failure controls

Phase	Ranked hits	Summary available
live	3	1
expired	1	0

At tick zero, both source events and the derived summary can contribute ranked hits. At tick two, one source expires; its summary becomes unavailable under the strict rule while the unrelated live event remains retrievable. The exact compression collision has equal summary text and unequal source IDs. These are deterministic logical examples, not retrieval-quality benchmarks.

Tests cover exact TTL equality, transitive invalidation, stale index entries, cross-tenant reads, duplicate sources, missing provenance, shared ancestors, UTF-8 budgets, corrupted cycles and

equal-payload reinsertion. A negative control deliberately retains an issued context after deletion to demonstrate the revocation limit. Testing only direct event lookup would miss the stale summary path.

The reference assumes serialized trusted callers and strongly consistent dictionary access. A real service must define atomicity among object writes, source edges, indexes and tombstones; conflict resolution; replication lag; deletion epochs; and failure recovery. A safe eventual index can lag while the authoritative read gate stays current. A stale authoritative tombstone replica needs a separate consistency protocol. Chapter 42's recovery model will provide vocabulary, not automatically implement that distributed store.

Key ideas

- A summary is available only if every ancestor is. A stale index entry cannot bring back a tombstoned source.
- If two histories compress to the same text, no function of that text can recover which one occurred; only retained sources can.
- UTF-8 bytes are not tokens, and a payload ratio is not total storage cost.
- A tombstone denies future reads. It cannot recall a context already handed to a client or a running model.

40.8 Twelve worked exercises

1. Is KV automatically durable memory? **Answer:** No. It supports native continuation. Persistent searchable evidence needs identity, retention and provenance beyond a transient cache.

2. Show a compression collision. **Answer:** The two strings ending in omitted-x and omitted-y both map to alpha beta under the two-word rule.

3. Why is rehydration not inversion? **Answer:** It reads retained source IDs. If those objects are unavailable, the summary cannot recreate them.

4. An event expires at tick 2. Is it readable at tick 2? **Answer:** No; the availability predicate requires $t < t_e$.

5. A summary depends on a forgotten source through another summary. What happens? **Answer:** Induction through the provenance DAG makes both derived records unavailable, even if indexes list them.

6. Sources have payload sizes 10, 10 and 12 bytes. What does prefix rehydration return under 20 bytes and three records? **Answer:** The first two, totaling 20 bytes; it does not cut the third record.

7. Does a 20-byte bound imply 20 tokenizer tokens? **Answer:** No. The byte encoding and tokenizer artifact define different units.

8. What happens when identical forgotten text is inserted again? **Answer:** It gets a fresh occurrence ID. Old summaries still point to the old tombstoned source and remain unavailable.

9. Can an already returned context disappear after forgetting? **Answer:** Not from this caller's memory. Future reads can be denied; revoking in-flight use needs a separate lifecycle protocol.

10. Why distinguish payload ratio from total storage? **Answer:** The latter includes original records, links, identities and index overhead. Summaries added to retained sources can increase storage.

11. How would you evaluate a learned compressor? **Rubric:** Pin source sets, compressor/prompt and budgets; compare raw records, truncation and summary-plus-retrieval on held-out exact and approximate tasks; include rare identifiers, order changes, deleted sources and adversarial instructions; report utility and full memory cost separately.

12. What must a distributed memory contract specify? **Answer:** Which store is authoritative, visibility/transaction boundaries, replication and conflict rules, tombstone propagation, recovery and in-flight revocation. Naming a memory tier supplies none of those guarantees.

40.9 Vocabulary and the compiler boundary

Provenance DAG: source relations among immutable evidence records. **Tombstone**: an explicit denial of future access, not proof of physical erasure. **Rehydration**: bounded source traversal rather than loss reversal. Chapter 41 now treats engine, memory, retrieval and tool operations as typed instructions whose effects must survive legal compilation.

Sources and further reading

Access checked 17 October 2026. The reference implements memory roles, lossy summaries with source links, retrieval, rehydration, retention, tombstones and an evidence firewall, and it adds transitive availability, non-resurrecting occurrence IDs, provenance-depth and store bounds, dual rehydration budgets, immutable context and explicit issued-context revocation limits. Chapter 20's compression arguments and the actual Chapter 35 cache contract were read for continuity.

MemGPT and Self-RAG were accessed at their primary arXiv abstract pages. Only the narrow memory-hierarchy and adaptive-retrieval claims are used; the full algorithms and experiments were not independently reproduced. No retrieval SOTA, security certification, physical erasure or learned utility claim follows from the deterministic lexical fixture. Local review remains distinct from acceptance.

CHAPTER 41

Compiler, IR and execution planning

In this chapter you will

- express a plan as a verified straight-line SSA IR with declared effects
- define the observation a compiler pass must preserve
- prove constant folding and dead-code elimination correct and test them against an independent oracle
- lower to eligible implementations and plan peak live storage at allocation time

The small program defines $a = 7$, $b = 8$, $n = a + b$, an unused integer 99, a retrieval whose documents are unused, and a tool consuming n. The requested answer is only n. Which operations can disappear? It is safe to replace the addition by integer 15 and remove the unused integer definitions. It is not safe to erase the tool because it returns no value. In our dialect, the retrieval is also retained: a corpus read can fail and is an observable effect.

Compilation should make this distinction explicit before optimizing a heterogeneous workflow. The path is plan, typed IR, verification, legal transformation, target lowering and executable schedule. Chapter 39's router does not decide which operations a plan needs; this compiler does not decide whether a model has learned the intended task.

41.1 An IR names semantics that a plan leaves implicit

Our **intermediate representation** is a straight-line SSA program $P = (\Gamma_{in}, O, outputs, version)$. Each operation has

$$o = (id, kind, inputs, output, type, attributes, effects).$$

Every input value has one defining operation or program-input declaration. Use-before-definition and redefinition are invalid. Operation IDs are unique and survive lowering and folding, giving a stable bridge from a runtime event to its source instruction.

Types include exact mathematical integers, text, token tuples, documents and backend-specific logits. An engine instruction consumes tokens and names an exact artifact and capability; it does not promise a universal vocabulary or logits dimension. Tools here consume an integer and are effect-only: their receipts belong to the observable event sequence, not an SSA output. Memory operations write text into or read text from Chapter 40's actual store. Handle construction and release remain inside the engine adapter.

This is a small *dialect*, not a replacement for Chapter 14's semantic-axis tensor types or Chapter 15's loop semantics. A future tensor-level lowering must retain shapes, axes and alias/lifetime obligations. Loops need block arguments, dominance and termination rules beyond this finite sequence.

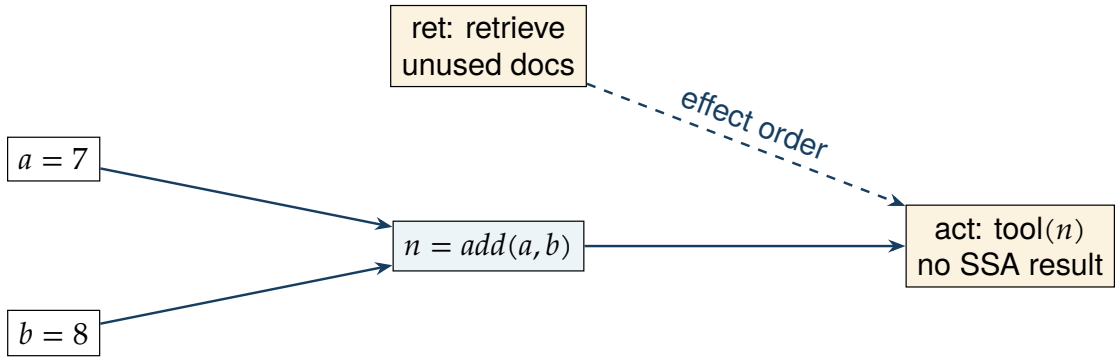


Figure 41.1: The graph carries value dependencies and an ordered effect chain. Lack of a value consumer does not make an operation semantically empty.

41.2 The verifier owns operation contracts

Verification checks schema version, bounded operation count, unique IDs, defined inputs, SSA outputs, operation signatures and exact required attributes. Only supported capability names are legal. The engine artifact field is required before lowering; missing/ambiguous actual routes still fail during lowering. Unknown schema versions fail closed rather than silently reinterpret new instructions.

Importantly, the producer cannot declare an engine or tool pure to obtain more optimization. The dialect computes an expected effect tuple per kind and rejects a different supplied declaration. Attribute pairs are immutable tuples, unique and exact; a stray field does not silently become policy. The verifier is in `drafts/ch41/reference.py`; the generated listings below extract whole tested functions rather than maintained line fragments.

Kind	Result	Dialect effects
const, add	integer	none
retrieve	documents	corpus read
engine	logits	native engine state
mem_get	text	memory read
mem_put	record ID text	memory write
tool	no SSA result	policy, external action

An IR can be well typed while its runtime input is invalid. The interpreter checks actual external integers/text/tokens and validates handler results. Native routines still check vocabulary, context length, finiteness and exact artifact identity. A type declaration is not a proof that an untrusted backend computed a correct result.

41.3 Define the observation before proving a pass

For each instruction, let η map defined SSA names to values and let σ denote external/module state. A pure arithmetic transition extends η without changing σ. An effectful transition invokes the declared handler, may change σ, and appends an event $(id, kind, args, result)$ to E.

Under fixed deterministic handlers and valid inputs, define observation

$$O(P, x) = (\eta(outputs), E, \sigma_{final}).$$

Failures additionally retain the effect prefix and failure class. We exclude wall-clock time, allocation exhaustion and profiling events from the pure arithmetic semantics. Otherwise removing an integer addition could trivially change a timing observation, and no useful optimization theorem would have the intended meaning. Real resource limits require a separate refinement contract rather than pretending an infinite-integer model is a bounded machine.

The only freely erasable instructions are total, deterministic `const` and `add` over mathematical integers. Retrieval is not freely erasable even for an immutable corpus fixture because the dialect preserves its read event and potential failure. A native engine call may allocate state, advance RNG or fail. A tool may mutate an external system. A memory read may depend on current deletion/expiry. Syntax alone does not make any of them pure.

MLIR's official language reference provides an extensible operation/value representation and multiple abstraction levels [95]. Its side-effect rationale separately discusses resource effects and speculation conditions [96]. Our conservative effect chain is much smaller: it neither implements MLIR nor inherits an optimizer correctness theorem.

41.4 Fold exact constants and preserve observable effects

For known integer definitions $a = \text{const}(u)$ and $b = \text{const}(v)$, replace $n = \text{add}(a, b)$ by $n = \text{const}(u + v)$, retaining the original operation ID. Folding can use a known constant across an effect barrier because SSA constants cannot be mutated, but it does not move or erase that barrier.

```
def constant_fold(p):
    verify(p)
    constants, ops = {}, []
    for op in p.ops:
        if op.kind == "const":
            constants[op.output] = dict(op.attrs)["value"]
        elif op.kind == "add" and all(x in constants for x in op.inputs):
            value = constants[op.inputs[0]] + constants[op.inputs[1]]
            constants[op.output] = value
            op = replace(op, kind="const", inputs=(), attrs=(("value", value),))
        ops.append(op)
    q = replace(p, ops=tuple(ops))
    verify(q)
    return q
```

Proposition 41.1 (Constant-fold observation preservation)**.** *For a verified program under the preceding exact-integer semantics, the folding pass preserves outputs, ordered effects and final handler state.*

Proof. Induct over instruction position, maintaining equality of values for all names defined so far and equality of handler states/event prefixes. An unchanged instruction receives equal arguments. A folded addition receives the two known integer constants and defines exactly their sum; it has no effects. Its replacement defines that same sum. Thus every later instruction receives the same values, and every effect executes at the same relative position with the same arguments. Requested outputs are equal at the end. □

Floating-point reassociation is not covered. For example, in a common binary floating-point format $(10^{16} + (-10^{16})) + 1$ and $10^{16} + ((-10^{16}) + 1)$ need not agree. Neither expression is a mathematical integer instruction in this dialect. A floating dialect must define rounding, exceptions, tolerances and permitted fast-math transformations explicitly.

Dead-code elimination walks backward from requested outputs and all effectful instructions. A retained instruction makes its input definitions live. An unneeded pure instruction can disappear; an unused retrieval or tool remains.

```
def dce(p):
    verify(p)
    live, kept = set(p.outputs), []
    for op in reversed(p.ops):
        if op.effects or op.output in live:
            kept.append(op)
            live.update(op.inputs)
    q = replace(p, ops=tuple(reversed(kept)))
    verify(q)
    return q
```

The backward invariant is that every already-examined retained instruction's input dependencies are in the live set. If a removed pure definition could affect an output or effect, there would be a first retained consumer whose backward processing marked it live, a contradiction. Since removed operations are total and state-free, their removal also preserves the effect/failure prefix. A pure-only program may reduce to zero operations while returning an input; that empty operation tuple remains a legal program.



Preserve answer 15, effect order and retained source IDs.

Figure 41.2: The fixture shrinks from six instructions to three. The retrieval and tool remain in order; answer 15 retains source ID sum.

41.5 Use an independent oracle, then break the premises

Comparing two executions of the same optimizer is weak evidence. Our second arithmetic oracle recursively expands the original definitions by dependency, rather than walking the sequential interpreter or calling the optimizer.

```
def expression_oracle(p, inputs):
    """Independent recursive arithmetic expansion, not sequential interpreter."""
    definitions = {o.output: o for o in p.ops if o.output is not None}
    def evaluate(name):
        if name in inputs:
            return inputs[name]
        op = definitions[name]
        if op.kind == "const":
            return dict(op.attrs)["value"]
        if op.kind == "add":
            return sum(evaluate(x) for x in op.inputs)
        raise ValueError("oracle accepts arithmetic output dependencies only")
    return tuple(evaluate(x) for x in p.outputs)
```

Tests generate deterministic random integer DAGs, including negative and very large exact integers and repeated operands. They compare original and optimized interpreter outputs with this recursive oracle, and check pass idempotence. A separate effect test runs both programs with actual Chapter 39 handlers and compares ordered events as well as outputs. Another test denies the tool before and after optimization; both must retain the same denial.

The verifier deliberately rejects erased tool effects, undefined values, wrong types, duplicate IDs, SSA redefinitions, unknown operations/capabilities, Boolean-as-integer constants, incomplete attributes and newer unsupported schema. Malformed handler results are rejected at runtime. These negative controls prevent a pass from appearing correct merely because every generated example is already legal and pure.

41.6 Lower legally before scheduling cheaply

Lowering maps abstract instructions to actual eligible implementations. Our `lower` function invokes Chapter 39's artifact-bound router for engine instructions and chooses named host/retriever/memory/tool handlers for the others. It creates a schedule only after complete verification.

```
def lower(p, engines):
    verify(p)
    rt = chapter(39).Runtime(engines, chapter(39).Policy())
    schedule, producer, last_effect = [], {}, None
    for op in p.ops:
        a = dict(op.attrs)
        if op.kind == "engine":
            cap = chapter(39).Capability(a["capability"])
            backend = rt.route(cap, a["artifact"]).backend
        else:
            backend = {"retrieve": "retriever", "tool": "toolhost",
                       "mem_get": "memory", "mem_put": "memory"}.get(op.kind,
                        ↪  "host")
        deps = {producer[x] for x in op.inputs if x in producer}
        if op.effects and last_effect is not None:
            deps.add(last_effect)
        schedule.append({"oid": op.oid, "backend": backend,
                         "dependencies": tuple(sorted(deps))})
        if op.effects:
            last_effect = op.oid
        if op.output is not None:
```

```
            producer[op.output] = op.oid
    return tuple(schedule)
```

Data dependencies come from producer IDs. Effectful instructions additionally depend on the previous effectful instruction, giving a conservative total effect order. Independent pure branches could run concurrently if resources permit; this implementation executes serially. Finer parallelism would require resource identities, conflict analysis and a proof that the reordered effects commute. Merely running independent-looking Python functions on two threads does not supply that proof.

The `Effects` class executes against the actual owned Chapter 39/40 modules and stable earlier engines. Tests lower both native and attention capabilities, execute their real adapter calls and close all handles. The native path still uses the restricted diagonal recurrence; the attention path still uses Chapter 23's learned-position stack through Chapter 35. No rotating position toy or mock KV allocator is substituted.

TVM is a useful precedent for separating tensor/graph optimization from target scheduling and code generation [97]. Our language includes retrieval, policy and durable-memory operations whose semantics need separate lowerings. We do not claim TVM's empirical performance or generate device code. Batch merge, prefix reuse, fusion and backend substitution remain candidates with specific obligations: matching artifacts/precision/owners, valid shared prefix identity, permitted numerical differences and preserved effects.

41.7 Plan resources with allocation-time liveness

For SSA value v, define $d(v)$ as definition time and $u(v)$ as last use. Without legal in-place reuse, storage must remain live through its final consumer's computation. Allocate an output before releasing dying inputs. For $c = a + b$, this means peak *three* live values, not one after post-operation cleanup.

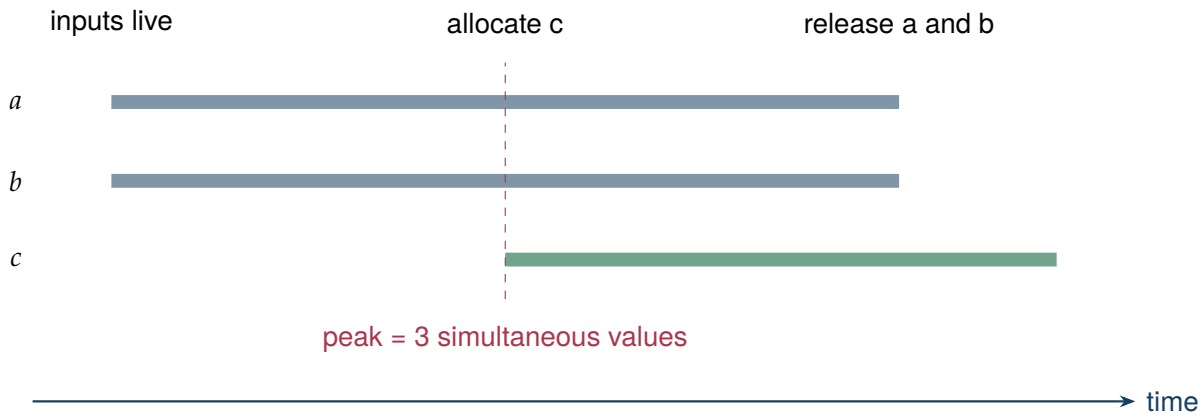


Figure 41.3: Peak pressure occurs during output allocation. Releasing operands after their final read cannot retroactively reduce that peak.

```
def peak_live_values(p):
    """Peak includes allocation before dying inputs are released; counts, not
     ↪ bytes."""
    verify(p)
    uses = {x: 0 for x, _ in p.inputs}
    for op in p.ops:
```

```
        for x in op.inputs:
            uses[x] = uses.get(x, 0) + 1
        if op.output is not None:
            uses.setdefault(op.output, 0)
    live = {x for x, _ in p.inputs if uses[x] or x in p.outputs}
    peak = len(live)
    for op in p.ops:
        if op.output is not None:
            live.add(op.output)
        peak = max(peak, len(live))
        for x in op.inputs:
            uses[x] -= 1
            if not uses[x] and x not in p.outputs:
                live.discard(x)
        if op.output is not None and not uses[op.output] and op.output not in
         ↪ p.outputs:
            live.discard(op.output)
    return peak
```

The count proxy is useful for debugging but not a byte model. If $b(v)$ is a declared payload size, a schedule's stronger model is

$$M_{peak} = \max_t \left[\sum_{v \in Live(t)} b(v) + W(t) + S(t) \right],$$

where W is operator workspace and S is external/native retained storage. Shared pages must be counted once by physical identity; checkpoint leases, prefix references and device pins may outlive an SSA handle's last use. A count of one logits value can conceal a large tensor. Python integers also use variable memory, so a live integer count is not a constant-byte bound.

Stage	Ops	Peak values	Effects	Answer
original	6	3	2	15
folded	6	2	2	15
DCE	3	2	2	15

These generated counts come from the exact fixture. Folding alone preserves instruction count; DCE removes three definitions while retaining two effects. The answer remains 15. No throughput conclusion follows from fewer instructions or lower value pressure without profiling the actual handlers.

41.8 Versioning, traceability and integration limits

The program pins schema `evod41-v1`. A production serialized IR needs a strict data-only encoding, compiler/dialect versions, exact artifacts, tenant/policy scope and deterministic content identity. This reference checks trusted Python objects; it is not an untrusted plan parser. Unknown required fields and effects must not be ignored during a future decoder.

Every lowered schedule item retains its operation ID. A serving trace should link runtime event to lowered operation to source IR and plan step. Constant folding preserves the ID of the operation whose value it replaces; richer fusion needs an explicit many-source provenance map. Such mapping explains external execution decisions, not hidden model reasoning.

The chapter-local compiler does not allocate Chapter 36 pages itself. The companion `examples/integrated_spine/paged_runtime.py` connects its actual effect handler and lowering to that allocator through Chapter 39's opaque adapter protocol. Cross-chapter tests compare full-model logits, shared-tail continuation, allocation failure, cleanup and retained effects. These serialized checks do not implement asynchronous cancellation or prove that device page/pin lifetimes are bounded by SSA liveness.

Key ideas

- A producer cannot declare a tool or engine pure to unlock optimization. The verifier computes the expected effects for each kind.
- Only total, deterministic pure operations may disappear. An unused tool or retrieval stays, because its effect is observable.
- Floating-point reassociation is outside these proofs: $(10^{16} - 10^{16}) + 1$ and $10^{16} + (-10^{16} + 1)$ can differ.
- Peak pressure occurs while an output is allocated, before its operands die, so $c = a + b$ needs three live values.

41.9 Twelve worked exercises

1. Which fixture operations survive optimization? **Answer:** sum as const 15, retrieval ret, and effect-only tool act. c1, c2 and dead vanish.
2. Why verify before optimizing? **Answer:** Undefined inputs and effect lies invalidate the premises of the backward liveness and folding proofs.
3. Can an unused tool be removed? **Answer:** No. Its output-free effect is observable and is a DCE root.
4. Is unused retrieval removable? **Answer:** Not in this dialect. Corpus reads are ordered observations and may fail; a different snapshot/purity contract would need its own proof.
5. Prove folding preserves a later tool argument. **Answer:** The prefix invariant equates every retained SSA value. The tool therefore receives the same integer; its relative effect order is unchanged.
6. Two model calls have equal text. Can they be commoned? **Answer:** Only with an explicit equivalence contract covering artifact, tokenizer, state, RNG, precision, policy, failure and observable-event semantics. Text equality alone is insufficient.
7. What is peak live count for $c = a + b$ from two inputs? **Answer:** Three while c is allocated and both operands remain live. An in-place form could change this only if alias and mutation rules permit it.
8. Can last SSA use free an engine page? **Answer:** Not while an external checkpoint, shared prefix or in-flight device operation still owns it.
9. Why preserve source operation IDs? **Answer:** They link a route, failure or resource estimate back to the original plan even after folding.
10. Propose a parallel schedule. **Rubric:** Respect all producer edges, preserve the declared effect order, reserve memory/workspace and justify any effect commutation. An unchanged final answer alone is inadequate.

11. Why is recursive expansion an independent check? **Answer:** It evaluates dependency expressions from the original IR without the sequential interpreter or optimizer, reducing shared-bug paths.

12. What does compiler correctness say about quality? **Answer:** Only that the declared program's observations are preserved. It does not prove that its chosen model, retrieval corpus or plan solves a task.

41.10 Vocabulary and the comparison boundary

SSA: one defining assignment per value name. **Effect**: observable behavior beyond a returned value. **Lowering**: selecting a legal implementation at a more concrete abstraction level. Database systems have long separated logical intent, physical choice and effects; the next chapter asks exactly which of those mechanisms transfer to Evolutor.

Sources and further reading

Access checked 17 October 2026. Treating engine and retrieval operations as freely pure would be unsound here: the dialect preserves those observations and checks declared effects against operation semantics. The independent recursive arithmetic oracle and actual Chapter 39/40 handlers test a more precise contract. Chapter 14 tensor typing, Chapter 15 transition semantics and Chapter 16 resource conventions were inspected; this finite IR does not implement every earlier language.

The official MLIR language reference's high-level operation/value model and side-effect rationale's categorization/modeling sections were inspected. The TVM primary arXiv abstract was inspected for compiler organization only. This reference implements neither framework, target code generation nor hardware experiments. Production serialization remains open; a companion example connects the compiler to Chapter 36's allocator.

CHAPTER 42

Database engines as a systems comparison

In this chapter you will

- separate a logical observation contract from its legal physical alternatives
- transfer Volcano and Cascades structure without importing relational equivalence
- attach object-specific validity to buffer management and transactions
- derive a committed-redo-only write-ahead recovery contract and test it at every crash cut

An artifact-promotion record has been appended to a log, but the process crashes before the serving metadata page is updated. Which version is active after restart? Now move the crash earlier, before the log reaches durable storage. The answer should change. Merely saying "use transactions" does not specify either case.

Database engines provide concrete precedents for separating intent, legal physical plans, execution, buffers, effects and recovery. We will borrow those mechanisms with explicit premises. A token generator is not a relational query, and a local dictionary simulating crash cuts is not a production database. The chapter works through a plan-selection comparison and a small write-ahead-log example, making semantic eligibility, record binding and the crash model precise.

42.1 Logical requirements delimit physical alternatives

Let G denote a required logical observation contract and $\mathcal{P}(G)$ the physical implementations that preserve it. A cost optimizer may choose

$$p^\star = \arg \min_{p \in \mathcal{P}(G),\, M_p \le M_{max},\, R_p \le R_{max}} J(p).$$

The legal set is not a penalty term. An implementation with a different tokenizer, artifact, numerical policy or state-continuation rule does not become equivalent because it is cheap. Chapter 41's verifier and lowerer determine legality before placement and scheduling optimize predicted cost.

Our fixture offers two abstract native-v1 alternatives and a cheaper attention-v1 alternative. These are stipulated contracts, not measurements of DOGMA versus Hermon quality. With latency in milliseconds, payload residency in MiB, risk a dimensionless policy proxy, and weights $w = (1, 0.1, 10)$, the illustrative score is

$$J = L_{ms} + 0.1 M_{MiB} + 10R.$$

Weights carry reciprocal units into one score. Risk remains subject to a hard ceiling in addition to its score term. It is not a calibrated failure probability in this example.

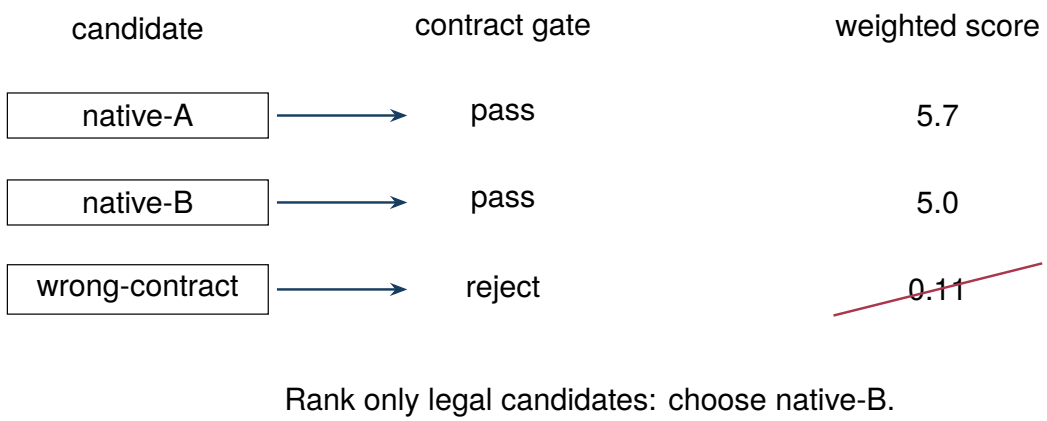


Figure 42.1: The cheapest candidate is ineligible. Contract and hard resource filters precede ranking, so the fixture chooses native-B.

Alternative	Weighted cost	Contract legal
native-A	5.7	1
native-B	5	1
wrong-contract	0.11	0

```
def choose(plans, contract, memory_limit, risk_limit, weights=(1., .1, 10.)):
    if (type(memory_limit) not in (int, float) or not math.isfinite(memory_limit)
            or memory_limit < 0 or type(risk_limit) not in (int, float) or
            not math.isfinite(risk_limit) or not 0 <= risk_limit <= 1):
        raise ValueError("finite resource/risk constraints")
    if len({p.name for p in plans}) != len(plans):
        raise ValueError("unique alternative names")
    for p in plans:
        cost(p, weights)
    eligible = [p for p in plans if p.contract == contract and
                p.memory_mib <= memory_limit and p.risk <= risk_limit]
    if not eligible:
        raise RuntimeError("no legal physical alternative")
    return min(eligible, key=lambda p: (cost(p, weights), p.name))
```

Selection validates finite nonnegative metrics and weights, unique names, resource ceilings and risk range. It breaks ties by name and fails explicitly when no legal alternative remains. A NaN must not silently win or evade a comparison. A model substitution that changes quality is a new task-level choice, not automatically an equivalent physical-plan rewrite.

42.2 Volcano and Cascades transfer structure, not equivalence

Volcano's operator interface separates execution mechanics from the semantics of individual item operations. Its exchange operator encapsulates parallel data movement [98]. The transferable lesson is to keep module boundaries stable while changing execution mechanism. A native state step, attention decode, retrieval read and symbolic operation may share lifecycle entry points without sharing an internal tensor representation.

Cascades organizes alternative expressions and physical requirements through memoized optimization tasks and rule-guided search [99]. Evolutor could similarly search legal

implementations under required properties, but needs an equivalence relation for each rule. A memo entry keyed only by prompt text could incorrectly merge different owners, model versions or random streams. We implement no Cascades memo or search algorithm here.

An autoregressive token step is prefix-stateful. Its selected output can become the next input, changing future state and work. It is not an iterator over independent rows. Nor is a database operator necessarily stateless: joins and aggregates retain state. The useful distinction concerns the declared algebra and legal transformations, not the simplistic claim that databases have no state while models do.

In a serving runtime, batching compatible requests can resemble vectorized operator execution, but each lane still needs its own local position, owner, artifact and completion fence. Chapter 34 proved such a lift for a restricted recurrence. The database analogy does not prove attention batching, and it does not replace Chapter 36's own allocation proofs.

42.3 Buffer management needs object-specific validity

A buffer manager tracks residency, ownership, pins and replacement. These mechanisms are relevant to weights, native recurrence state, attention KV, durable records and retrieved context. Their reconstruction rules differ. An immutable weight tensor can be reloaded from a verified artifact; a native state needs a complete checkpoint or exact replay; Hermon KV needs the exact ordered prefix, model and learned positions; a forgotten event may no longer be eligible for restoration at all.

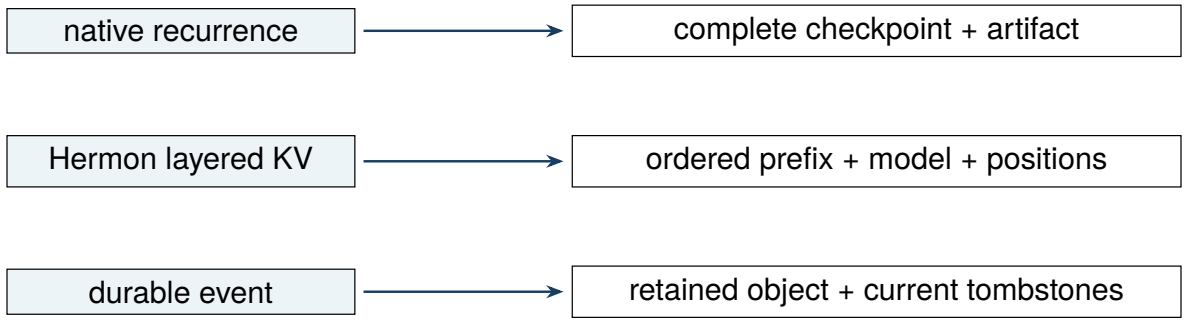


Figure 42.2: Representation and recovery semantics remain attached to each object type. Allocating equal-size pages does not make their contents interchangeable.

Pins protect physical lifetime, not logical validity. A pinned buffer with the wrong artifact is still wrong; a logically valid handle cannot be reclaimed while device work reads it. Chapter 32 implements generational native-pool ownership and delayed reclaim, while Chapter 35 specifies ordered layered KV. Chapter 36's page-sharing and eviction tests are where those lifetime constraints meet paged attention state. This chapter does not substitute a generic byte-page demo for those implementations.

Database index ideas also transfer with limits. Exact key/range access and approximate vector retrieval support different query contracts. An index is an access path, not the authoritative evidence object. Chapter 40 checks current tombstones and all summary ancestors before returning a stale index hit. Promoting the hit into context does not give it policy authority.

42.4 Transactions belong to particular durable mutations

Atomicity, consistency, isolation and durability can describe metadata writes, memory updates and an artifact-pointer switch when backed by a real transaction system. They do not by themselves describe sampling. A generated token may be computed privately, accepted into a request's output stream, or delivered to a client; those are separate events with different rollback possibilities.

Isolation should likewise name the shared resource. Concurrent durable updates to a tenant's memory might need serializable transactions. Separate request-local KV tensors need ownership and lifetime isolation. A phrase such as "serializable generation" does not specify either contract. External actions may support an idempotency key or compensation, but a database rollback cannot reverse an already sent message or completed physical action.

Materialization is a useful comparison for immutable prefixes, compiled artifacts and derived summaries. Reuse requires exact semantic identity, tenant/policy scope, source availability and the relevant artifact versions. An engine-native prefix cache and a semantic document cache are different materialized objects with different keys and invalidation rules.

42.5 Derive a small write-ahead recovery contract

ARIES is a primary precedent for WAL, log sequence numbers and recovery that repeats history with rollback machinery [100]. Our reference is *committed-redo-only*: every assignment becomes an independently committed unit when its log prefix is flushed. There are no concurrent transactions, uncommitted page writes, undo, compensation log records, checkpoints, torn pages, media loss or real `fsync`. It is not an ARIES implementation.

The crash model has a volatile append-only log, durable prefix boundary D, durable scalar pages $data[k]$, and page sequence numbers P_k. Appending an assignment creates immutable record (ℓ, k, v) with the next LSN ℓ. Flushing advances D only over existing contiguous records. A page write is legal only when

$$1 \le \ell \le D, \qquad log[\ell] = (\ell, k, v), \qquad P_k \le \ell. \tag{42.1}$$

The first condition is log-before-page ordering. The second binds the proposed page contents to the record, preventing a caller from using an unrelated durable LSN as permission for arbitrary data. The third prevents a stale page write from overwriting a newer assignment.

```
    def write_page(self, key, value, lsn):
        if type(key) is not str or type(value) not in (str, int):
            raise ValueError("page assignment must retain scalar type")
        if type(lsn) is not int or not 1 <= lsn <= self.durable_lsn:
            raise RuntimeError("WAL: assignment is not durable")
        if self.log[lsn - 1] != (lsn, key, value):
            raise ValueError("page content must match its log record")
        if lsn < self.page_lsn.get(key, 0):
            raise RuntimeError("stale page write")
        self.data[key], self.page_lsn[key] = value, lsn
```

A simpler check that tests only $\ell \le D$ would accept a different value or key under a valid record number. The negative controls reject both, reject flushes beyond log end and reject stale per-key writes. A hash or LSN is meaningful only when it binds the actual state transition being certified.

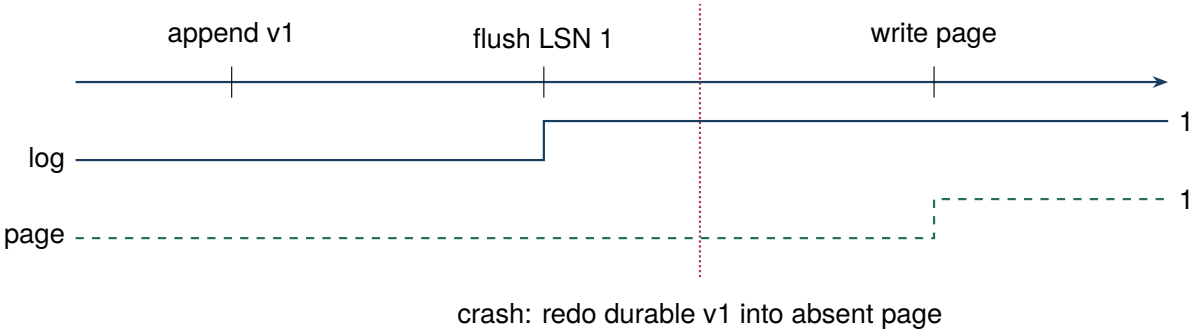


Figure 42.3: The durable log frontier precedes the durable page frontier. A crash in between requires redo; a crash before log flush loses the volatile assignment.

On crash, discard the volatile tail above D and keep durable pages. Recovery scans retained records in LSN order and writes a record only when its page LSN is older.

```
    def recover(self):
        for lsn, key, value in self.log[:self.durable_lsn]:
            if self.page_lsn.get(key, 0) < lsn:
                self.write_page(key, value, lsn)
```

Proposition 42.1 (Redo correctness and idempotence)**.** *Assume initially empty pages, only legal page writes under Equation 42.1, and a retained correct contiguous log prefix* $1, \dots, D$*. Recovery leaves each key equal to its latest assignment in that prefix, or absent if it has no assignment. Running recovery again changes neither data nor page LSNs.*

Proof. For key k, let ℓ_k be its maximum retained LSN. Legal page writes ensure any existing P_k names an actual retained record for k and $P_k \le \ell_k$. Scanning in increasing order either skips an already newer page or advances it to the record being scanned. By the time ℓ_k is visited, the page equals that assignment and its LSN is ℓ_k. A second scan finds every record's LSN at most the current page LSN and skips it. Keys absent from the retained log cannot have a legal page write. □

Corruption, missing log ranges or forged page LSNs violate the premises. Production storage needs checksums, validated recovery metadata and a real durability mechanism. The theorem does not turn an in-memory copy into a disk-crash guarantee.

42.6 Crash cuts need an independent expected state

The second recovery oracle simply applies last assignment per key over the durable prefix. It never invokes the page-LSN algorithm.

```
def replay_prefix(log, durable):
    """Independent last-assignment specification; does not use page LSN logic."""
    result = {}
    for _, key, value in log[:durable]:
        result[key] = value
    return result
```

Cut	Durable LSN	Page LSN	Recovered
0	0	0	absent
1	0	0	absent
2	1	0	v1
3	1	1	v1
4	1	1	v1
5	2	1	v2
6	2	2	v2

Cuts zero through six occur before work and after each append, flush and page write for v1 and v2. After append without flush, recovery cannot see the new assignment. After flush without page write, recovery supplies it. After page write, redo is a no-op for that record. The generated table and assigned expected answer vector test these distinctions rather than only asserting that recovery returns *something*.

Randomized schedules append assignments to three keys, flush varying prefixes, write some eligible pages and crash after every iteration. Recovered maps must equal the independent prefix specification, and second recovery must be idempotent. These tests exercise a stipulated fault model; they do not measure power-loss behavior, torn writes or an operating system's durability semantics.

42.7 Recovery, replay and stale statistics are different problems

Recovery restores a consistent durable metadata state. Replay reproduces declared execution observations under captured inputs, artifacts, RNG and numerical policy. A recovered serving pointer can be correct without regenerating every historical token. Conversely, numerically replaying a request does not prove its external tool effect was committed only once.

Cost statistics can be wrong while a plan remains legal. Relevant serving statistics include prompt/output lengths, native-state bytes, prefix reuse, speculative acceptance, batch occupancy, backend queues and retrieval quality. They need workload identity, collection time and uncertainty. Misestimation can cause a poor memory/latency choice; it must not expand semantic eligibility. The figures here contain assigned arithmetic, not measured workload statistics.

Four limits keep the comparison bounded: prefix-dependent token generation does not inherit relational rewrite laws; sampling is not ACID commit; native and KV objects have model-specific validity; and task quality/risk needs its own evidence beyond a physical cost estimate. Database engines are useful because they make such contracts explicit, not because using their vocabulary makes an inference runtime a database.

Key ideas

- Legality is a filter, not a cost penalty. The cheapest candidate here is ineligible.
- Pins protect physical lifetime, not logical validity. A pinned buffer with the wrong artifact is still wrong.
- A page write must cite a durable log record for the same key and value that does not regress the page's LSN. An LSN that certifies nothing specific certifies nothing.
- Recovery restores durable metadata; replay reproduces execution observations. Neither reverses an external effect.

42.8 Twelve worked exercises

1. Why reject a cost 0.11 candidate when another costs 5.0? **Answer:** The cheaper candidate has the wrong logical contract and is outside $\mathcal{P}(G)$.
2. Compute native-A's score for $(5\,ms, 4\,MiB, .03)$. **Answer:** $5 + .1(4) + 10(.03) = 5.7$ under the declared weights.
3. What transfers from Volcano? **Answer:** Stable operator boundaries and encapsulated execution/parallelism. A shared interface does not equate native recurrence state and layered attention KV.
4. Is every database operator stateless? **Answer:** No; joins and aggregates retain state. The legal algebra and observations determine rewrites.
5. Is an unpinned but logically valid page reclaimable? **Answer:** Only after all relevant owners, references and in-flight consumers release it under the actual allocator contract.
6. What three checks authorize the miniature page write? **Answer:** Its LSN exists and is durable, its key/value matches that record, and it does not regress the page's LSN.
7. Crash after append but before flush: what survives? **Answer:** The new assignment is outside the durable prefix and is lost. Earlier durable assignments remain recoverable.
8. Crash after flush but before page write: what happens? **Answer:** Redo installs the retained assignment if the page LSN is older.
9. Why is redo idempotent? **Answer:** After first recovery each page's LSN is its latest durable assignment; all older/equal records are skipped.
10. Does this reference implement transaction rollback? **Answer:** No. Every flushed assignment is committed; undo and multi-record atomicity are excluded. It cannot model an uncommitted transaction's dirty page.
11. Can model rollback reverse an external action? **Answer:** No. A tool needs its own idempotency, reconciliation or compensation protocol; an irreversible effect remains irreversible.
12. Design a real durability experiment. **Rubric:** Define storage and flush guarantees, inject crashes at named cuts, reopen actual files, validate log/page integrity and committed-prefix behavior, repeat recovery, and distinguish application-process failure from power loss. Dictionary copying alone does not qualify.

42.9 Vocabulary and the governance boundary

Physical alternative: an implementation satisfying a fixed logical contract. **WAL**: durable-log-before-durable-page ordering under a specified fault model. **Recovery**: restoring consistent durable state, distinct from replaying model execution. Chapter 43 applies these lessons to immutable candidate lineage, controlled promotion and validated rollback.

Sources and further reading

Access checked 17 October 2026. The chapter's write-ahead log adds exact log/page binding, bounded flushes, stale-write rejection, crash images and an independent durable-prefix oracle. It remains committed-redo-only, not an ARIES clone.

Volcano's primary-paper text was inspected for the standard interface, buffer-manager mechanism/policy separation and exchange description. Cascades' primary-paper sections on optimization tasks, memoization and on-demand exploration were inspected. ARIES access was narrower: publisher metadata/ abstract and an indexed primary-paper excerpt on log forcing/LSNs were read; two attempted full-PDF retrievals timed out. No full-paper reconstruction or ARIES implementation is claimed. The WAL proof is self-contained under this chapter's stated crash model. Source records distinguish these depths.

CHAPTER 43

Structural adaptation and lifecycle governance

In this chapter you will

- name exactly which object a proposed adaptation changes and record immutable lineage
- evaluate candidates with conjunctive gates bound to protocol, data and budget
- contrast a parametric stability penalty (EWC) with structural legality checks
- promote and roll back under role separation, revision guards and state compatibility

A candidate improves a quality score from .80 to .95 but lowers a safety invariant score from 1 to .5. Another reaches quality .85 while retaining safety 1. Which may become active? The first fails the required gate despite its larger quality gain. A system that can propose new structures is not thereby entitled to deploy them.

Chapter 13 distinguished transient state, learned weights, structural proposals and population selection. Chapters 39–41 supplied typed operations, memory and legal compilation. This chapter governs changes to those objects through immutable identity, evidence, bounded canaries, promotion and rollback. Its local registry is a serial specification, not a signed release service or a demonstration of autonomous scientific improvement.

43.1 Name the object that changes

Candidates can change weights, adapters, memory policies, retrieval indexes, compiler rules, prompts/policies or topology. These have different evaluation and state-compatibility requirements. A token-level router selecting an expert from a fixed graph is not automatically structural learning. Updating recurrent activation state is not parameter learning. "Self-improvement" is too coarse a type for an engineering change.

Let A be an immutable artifact and T a declared transformation with recipe, data and budget. A proposal creates $A' = T(A)$ with explicit parent identity; it never rewrites the active artifact's bytes. The executable registry stores payload, recipe, training-data identity and a native-state fingerprint in the content identity. Lifecycle stage and measured metrics live in separate records. Changing a stage therefore does not change artifact bytes or its identity.

```
    def create(self, payload, parent=None, recipe="identity",
     ↪  training_data="train-v1",
               state_fingerprint="state-v1", actor="search"):
        self._role(actor, "proposer")
        if parent is not None and parent not in self.artifacts:
            raise KeyError("registered parent required")
        values = (payload, recipe, training_data, state_fingerprint)
        if any(type(x) is not str or not 1 <= len(x) <= 4096 for x in values):
            raise ValueError("bounded nonempty provenance strings")
        body = (parent, *values)
        aid = hashlib.sha256(json.dumps(body, separators=(",",
         ↪  ":")).encode()).hexdigest()
```

```
        if aid in self.artifacts:
            return self.artifacts[aid]  # never reset an existing stage/evaluation
        a = Artifact(aid, parent, *values)
        self.artifacts[aid], self.stages[aid] = a, Stage.CANDIDATE
        self.log += (("create", aid, actor),)
        return a
```

Duplicate creation is idempotent: it returns the existing artifact without resetting a rejected or active stage. A parent must already exist. With a fresh child identity and immutable parent field, every new parent edge points to an older registered object, giving an acyclic lineage. Multiple children can share a parent, preserving rejected branches as evidence rather than rewriting the history around a winning candidate.

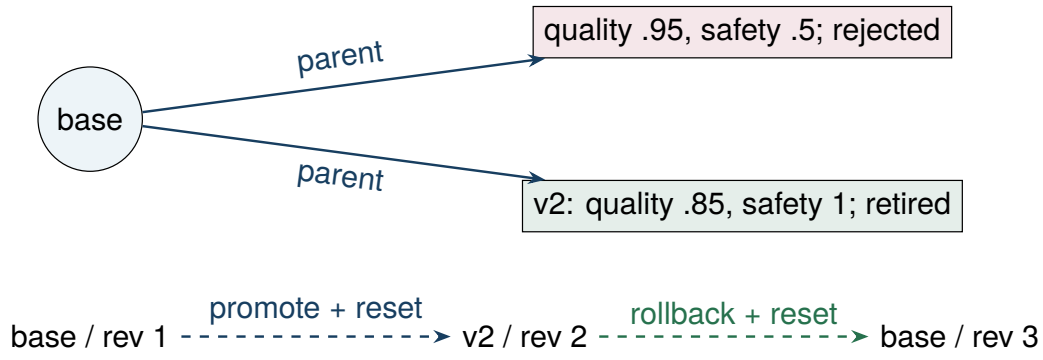


Figure 43.1: The unsafe branch remains in lineage but never serves. Promotion and rollback move serving identity; they do not reverse parent edges.

The hash binds these declared strings; it does not prove that named data were actually used, that the payload is a real trained model or that a producer is authorized. Production packaging must bind verified artifact bytes and complete configuration, and authenticated release mechanisms must enforce authority. Here role labels are trusted caller inputs and strings are explicit fixture provenance, not cryptographic attestations.

43.2 A gate is a conjunction of constraints

For a metric vector m, define each gate as a metric name, direction and threshold. Some metrics require lower bounds, others upper bounds:

$$Eligible(m) = \bigwedge_{i \in I_{min}} [m_i \geq g_i] \wedge \bigwedge_{j \in I_{max}} [m_j \leq g_j].$$

Our fixed registry policy requires quality at least .8, safety at least 1, and latency at most 10. Latency uses fixture milliseconds; quality and safety are stipulated dimensionless scores. They are not measured validation accuracy or a general safety probability. Missing metrics fail a gate; NaN, infinity, Boolean values and invalid gate directions are rejected before stage mutation. An empty gate set is not accepted through vacuous truth.

```
def gates_pass(metrics, gates):
    if not gates or len({g.metric for g in gates}) != len(gates):
        raise ValueError("nonempty unique gates")
    for g in gates:
        finite(g.threshold)
        if g.direction not in ("min", "max"):
            raise ValueError("known gate direction")
    for value in metrics.values():
        finite(value)
    return all(g.metric in metrics and
        (metrics[g.metric] >= g.threshold if g.direction == "min"
         else metrics[g.metric] <= g.threshold) for g in gates)
```

Gates belong to the registry's configured policy, not to the proposing candidate. The evaluator cannot pass a new empty or weaker gate set on each call. More realistic policies include confidence intervals, absolute invariants, regression budgets and workload slices. Hard constraints such as tenant isolation, authority-before-side-effect, bounded memory and complete lifecycle traces cannot be traded for a higher aggregate task score.

If all k deterministic gates pass, eligibility is true by conjunction. That logical statement says nothing about estimation error. With statistical metrics, each reported estimate needs its sample size and uncertainty. Testing many candidates against the same held-out set also adapts to that set; selection then needs a fresh final evaluation or an explicit sequential/multiple testing design. Calling the selection set held-out does not make it untouched.

43.3 Bind evaluation to candidate, protocol and budget

Evaluation is legal once from candidate stage. Its immutable report binds artifact ID, suite version, evaluation-data identity, evaluator, budget, metric tuple and pass decision. The registry requires its configured held-out data/suite labels and rejects data equal to the artifact's declared training data. A second evaluation of the same object is rejected; a new versioned candidate/evidence protocol is needed rather than overwriting its history.

These checks prevent certain bookkeeping errors, not data leakage by a dishonest producer. Dataset labels are not content hashes or proof of non-overlap. A real evaluation artifact must record exact examples/splits, task conventions, code, seeds, compute budgets, hardware, aggregation and uncertainty. Genomic splits may need sequence/homology/species controls; tool safety needs negative and adversarial workloads. This registry executes none of those experiments.

Population search should compare siblings under matched budgets or report their differences. Preserve failed candidates, negative controls and the selection rule. The learned or search-generated proposal can be wrong while the governance mechanism correctly rejects it. Conversely, a passed synthetic registry test does not establish that a structural change is useful.

43.4 Separate parametric stability from structural legality

EWC provides a concrete parametric stability/plasticity method. Its paper uses a diagonal Fisher-weighted quadratic around previous parameters,

$$L(\theta) = L_B(\theta) + \frac{\lambda}{2} \sum_i F_i(\theta_i - \theta^*_{A,i})^2,$$

motivated by a local approximate posterior [101]. This is an approximate method, not a universal guarantee against forgetting and not an authorization protocol. The empirical tasks in that paper are not reproduced by our registry.

To see the tradeoff without claiming trained behavior, stipulate one-coordinate task loss $L_B = \frac{c}{2}(\theta - b)^2$, previous optimum a, importance $F \geq 0$, $c > 0$ and $\lambda \geq 0$. Differentiate:

$$\frac{dL}{d\theta} = c(\theta - b) + \lambda F(\theta - a) = 0, \qquad \theta^* = \frac{cb + \lambda Fa}{c + \lambda F}.$$

The second derivative $c + \lambda F > 0$ proves a unique global minimizer for this stipulated quadratic. With $a = 0, b = 2, c = 1, F = 3$, it becomes $2/(1 + 3\lambda)$. Increasing the penalty keeps the coordinate nearer the old optimum while sacrificing the new task optimum.

Penalty strength	Stipulated optimum
0	2
1	0.5
3	0.2
10	0.0645161

Tests compare this closed form with an independent finite-difference derivative and endpoint objective values. The numbers are algebra, not a neural-network experiment or an estimated Fisher matrix.

Structural changes require different checks. Adding a memory operator, rerouting a tool or changing a compiler rule can alter dependency topology and effect order even if weights are unchanged. Chapter 41's original/optimized observations, verifier failures and native interfaces need regression coverage. A parameter regularizer cannot establish those compiler properties.

43.5 Make lifecycle transitions explicit

The stages are candidate, evaluated, canary, active, retired and rejected. Passing offline gates moves candidate to evaluated; failure moves it to rejected. A bounded successful canary is required before promotion. Rejected objects have no direct serving path. Changing a failed object and trying again creates a new immutable child with new evidence.

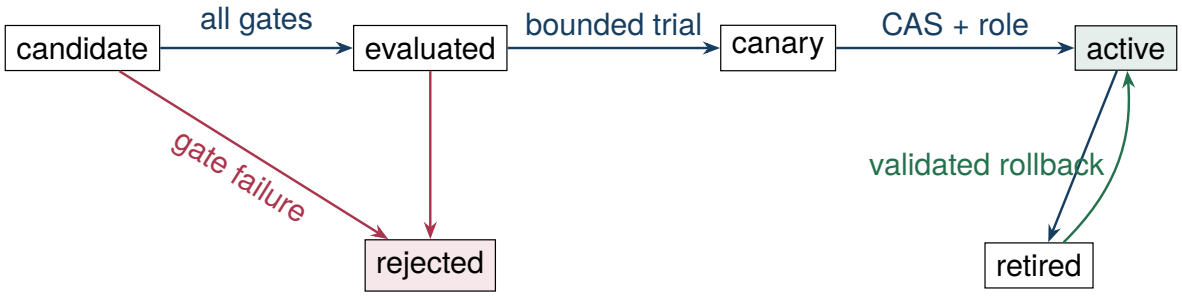


Figure 43.2: Guards constrain every serving path. Rollback has its own eligibility rule and cannot bypass rejection by naming an arbitrary object.

Shadow evaluation observes production-like inputs without controlling visible outputs. It must still isolate memory writes and tools; a shadow call that executes real effects is not harmless. Canary deployment serves a bounded fraction under predefined limits and rollback conditions. It follows offline correctness gates rather than using user exposure to discover basic type or authorization failures.

Our canary specification accepts an exposure fraction in $(0, .1]$, at least 20 observations, valid error counts and nonnegative p95 latency. Zero errors and p95 at most 10 move the candidate to canary; otherwise it is rejected. These are caller-supplied observations, not actual traffic routing or quantile measurement. A drain flag later is also a caller assertion, not implemented request draining.

Even honest zero errors among 20 independent Bernoulli trials gives weak statistical assurance. If the true error probability is p, the chance of zero errors is $(1-p)^{20}$. Solving $(1-p)^{20} = .05$ gives one-sided upper bound $p = 1 - .05^{1/20} \approx .139$. A small zero-error canary is therefore a transition fixture, not evidence of a tiny production failure rate. Dependence, nonstationarity and workload slices further limit that calculation.

43.6 Promote with authority and a current revision

Proposal, evaluation and promotion have separate caller roles in the reference: search, lab and release. Promotion checks passed offline/canary evidence and the current serving revision before changing any stage or pointer. Two release decisions based on revision r cannot both succeed against the serialized store: the first increments it, making the second stale.

```
    def _switch(self, target, expected_revision, drain, event):
        if type(expected_revision) is not int or expected_revision !=
         ↪  self.revision:
            raise RuntimeError("stale release revision")
        if type(drain) is not bool:
            raise ValueError("explicit boolean drain policy")
        current = self.artifacts.get(self.active)
        changed_state = (current is not None and current.state_fingerprint !=
                         self.artifacts[target].state_fingerprint)
        if changed_state and not drain:
            raise RuntimeError("state incompatible: drain/reset required")
        # All checks completed before serialized pointer/stage mutation.
        if self.active is not None:
            self.stages[self.active] = Stage.RETIRED
        self.active = target
        self.stages[target] = Stage.ACTIVE
        self.revision += 1
        self.history += (target,)
        self.log += ((event, target, self.revision, changed_state),)
```

The switch preflights all checks before mutating active identity, stages, history, revision and audit tuple. It retires the old active object, activates the target and appends a transition. This is one serialized method under trusted callers. It is not a database transaction: an interpreter crash, allocation failure or concurrent unsynchronized caller can break assumptions. A real implementation needs a durable transaction/CAS over the whole state and audit record, authenticated role separation and verified signatures/hashes. Chapter 42's redo-only dictionary does not supply that infrastructure.

Proposition 43.1 (Serving path and revision invariant)**.** *Starting from an empty registry, assume only successful public transitions, serialized calls and untampered internal dictionaries. Every active artifact has passed offline and canary evaluation, has appeared in serving history, and is the only artifact in active stage. Every successful switch increases revision by one.*

Proof. Creation and evaluation cannot create active stage. Promotion requires canary with passed evidence. A switch retires the previous active before activating its target, records that target and increments revision. Rollback requires a retired target already in serving history with passed evaluation, so by the induction hypothesis it originally passed canary through promotion. No other legal transition creates active state. Revision increments once in the common switch routine and nowhere else. □

This is a reachability invariant, not an empirical safety claim. Directly editing the store or forging evidence violates its premises.

43.7 Rollback is a governed switch, not time reversal

A naive registry would allow rollback to any registered artifact, including one that had never passed evaluation. We require a previously served, validated, currently retired target and the same promoter/revision checks.

```
    def rollback(self, target, expected_revision, drain=False, actor="release"):
        self._role(actor, "promoter")
        if (target not in self.history or self.stages[target] != Stage.RETIRED or
                not self.evaluations[target].passed):
            raise RuntimeError("rollback only to previously served validated
             ↪ artifact")
        self._switch(target, expected_revision, drain, "rollback")
```

A **state fingerprint** names the exact continuation compatibility class, including model parameters, tokenizer, native schema and numerical semantics. An interface version or tensor shape alone is insufficient. Incompatible promotion or rollback requires explicit drain/reset. Existing requests should remain pinned to their original artifact until a lifecycle fence proves their work and output are settled. New requests start with the target's fresh or independently validated restored state.

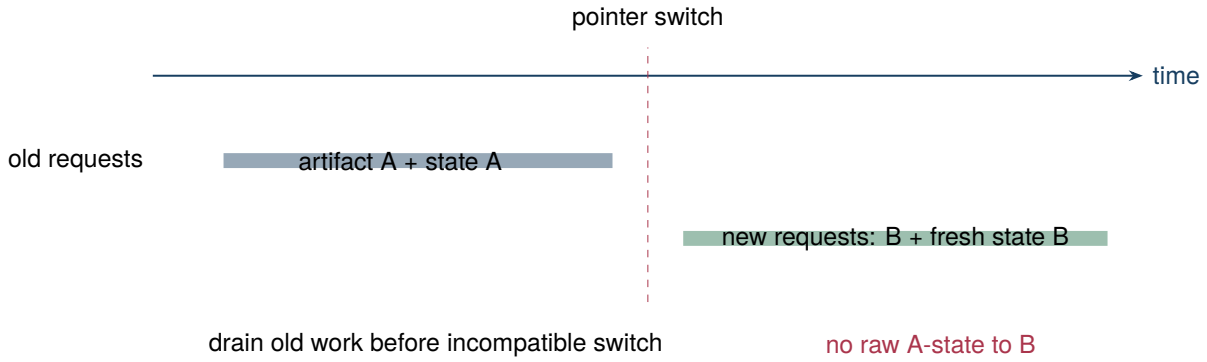


Figure 43.3: Serving identity and in-flight state have different lifetimes. The fixture's drain flag expresses a precondition that a real service must verify.

The registry stores a declared fingerprint string. It does not derive that fingerprint from Chapter 31/35 artifacts or implement a state migration. An integrated service must test old-request pinning, completion acknowledgement and new-request admission against the

real Chapter 30, 34 and 35 interfaces and the Chapter 36 allocator. Local transition tests do not close that boundary.

Rollback restores future serving choice; it cannot reverse messages, trades, deleted external objects or other irreversible actions already performed. Memory schema rollback can also require forward-compatible readers or an explicit migration. The active pointer, native execution state, durable memory state and external effects are distinct coordinates.

43.8 Read the lineage result and monitor after promotion

Candidate	Final stage	Ever served
base	active	1
unsafe	rejected	0
v2	retired	1

The generated trace serves baseline, rejects the high-quality unsafe child, serves compatible-by-reset v2, then returns to baseline with another reset. The final serving revision is three. Immutable parent links and all rejected evidence remain intact. These are deterministic fixture states, not deployment events or measured capability improvements.

Negative controls attempt empty/missing/NaN gates, training-data evaluation, unknown suites, wrong roles, insufficient canary observations, canary failures, promotion before canary, stale revisions, incompatible no-drain switches and rollback to never-served candidates. Tests verify failure leaves the serving state unchanged where the operation is rejected before mutation.

After real promotion, monitor workload distribution, regression/safety slices, latency/resource SLOs, memory changes and anomalous tool behavior. A drift signal can trigger investigation, bounded rollback authority or a halt. Unbounded automated rollback loops can themselves damage availability; define observation windows, thresholds, cooldowns and manual escalation. Monitoring adds evidence; it does not retrospectively justify a missing gate.

Key ideas

- A large quality gain cannot buy back a failed safety gate. Eligibility is a conjunction.
- Zero errors in 20 independent canary trials still allows a failure rate up to about 13.9% at one-sided 95%.
- Rollback is a governed switch to a previously served, validated artifact. It is not time reversal and cannot undo external effects.
- Equal tensor shapes do not make native state compatible. Carrying state across artifacts needs a verified conversion or a reset.

43.9 Twelve worked exercises

1. Why reject quality .95 and safety .5? **Answer:** Safety must reach 1 under the fixed conjunctive policy. Quality cannot compensate.

2. Does stage change alter artifact identity? **Answer:** No. Stage/evidence live separately from immutable content and lineage fields.

3. What prevents duplicate creation from reviving a rejected object? **Answer:** The content ID lookup returns the existing artifact without resetting its stage or evaluation.

4. Why are parent edges acyclic? **Answer:** A fresh child's parent must already exist; immutable edges always point to earlier registered objects.

5. Derive the stipulated regularized optimum. **Answer:** Set $c(\theta - b) + \lambda F(\theta - a) = 0$ and divide by the positive $c + \lambda F$, yielding the displayed weighted average.

6. Does EWC prove a compiler rewrite legal? **Answer:** No. It regularizes parameters; data/effect observations need separate compiler tests.

7. What does zero canary errors in 20 independent trials establish? **Answer:** Under the Bernoulli premise, a one-sided 95% upper bound is about 13.9%, far from a tiny failure-rate assurance.

8. Two promoters read revision 1. What follows? **Answer:** The first successful switch moves it to 2; the second's expected revision 1 fails. A real concurrent implementation needs an atomic durable check/update.

9. May rollback target a rejected child? **Answer:** No. It must be previously served, validated and currently retired.

10. Same tensor shape, different model bytes: may state carry over? **Answer:** Not without an independently verified conversion. Exact artifact-bound recurrence/KV semantics can change despite equal geometry.

11. Model rollback reverses external effects? **Answer:** No. External systems need their own reconciliation or compensation, if possible.

12. Design a structural-adaptation evaluation. **Rubric:** Specify transformation and inheritance, matched budgets, untouched final data, compiler and native-state regressions, hard policy invariants, bounded shadow/canary effects, authenticated promotion, state compatibility and rollback monitoring. Synthetic registry reachability alone is not evidence of learned improvement.

43.10 Vocabulary and industrial handoff

Lineage: immutable derivation relations among artifacts. **Promotion**: a governed change to serving identity after evidence gates. **Rollback**: a validated switch to a previously served compatible object, with state reset when needed. Chapter 44 turns these requirements into packaging, deployment and observable service contracts.

Sources and further reading

Access checked 17 October 2026. The reference separates content from stage, forbids unrestricted rollback, records immutable evidence, rejects nonfinite/ missing metrics and empty gates, requires a current release revision and asserted reset for incompatible state. Role labels, data labels and drain flags are trusted assertions, not cryptographic deployment evidence.

The primary EWC paper HTML was inspected in its method discussion and Equations 1–3, with experimental context. Only its approximate parametric regularization mechanism is summarized. The quadratic example is independently derived arithmetic, not trained EWC.

Chapter 13's state/weight/structure/population distinctions and Chapter 30/31/35 artifact-bound state conventions were read for continuity. Production signatures, durable CAS and request draining remain separate requirements. Local author review is not independent acceptance.

PART X

Industrial engineering

What must hold before a reference implementation can be operated as a service?

Industrial engineering is treated as another set of contracts with failure cases: release identity and byte verification, readiness and draining, tenant isolation and capability-based authority, distributed placement with fencing epochs, and measurement protocols that report tails, goodput and uncertainty honestly. None of these chapters runs a live service. They define what such a service would have to preserve and measure.

CHAPTERS IN THIS PART

44 **Packaging, deployment and observable services** Package reproducible services with observability and failure behavior.

45 **Multi-tenancy, isolation and security** Enforce tenant boundaries, quotas and explicit authority.

46 **Distributed serving and state placement** Place and migrate distributed state without breaking semantics.

47 **Profiling, performance and hardware backends** Profile real workloads and compare measured hardware backends.

CHAPTER 44

Packaging, deployment and observable services

In this chapter you will

- bind model, runtime, configuration, interpretation contracts and provenance into one release identity
- verify delivered bytes against an independently trusted manifest pin
- separate liveness, readiness and the ability to reserve capacity, and drain by ownership conservation
- observe without copying private content and turn observations into a falsifiable SLO with an error budget

A model artifact is not yet a service. Suppose revision r1 serves requests correctly and r2 changes a tokenizer option. The tensor dimensions still fit; the readiness endpoint returns success; a kernel test passes. Should r2 accept r1's checkpoints? Should it receive all traffic? Neither follows from those observations. We need to identify the complete computation, define compatible state and gather evidence about the service under a specified workload.

Our running example is a small release controller with two revisions, a bounded number of admitted requests and synthetic completion times. It never starts a service. It makes acceptance, promotion, draining and rollback executable contracts. A real deployment must implement those contracts across crashes, concurrency, authenticated artifact storage and device completion fences.

44.1 Bind the release before starting it

Define a bundle $B = (M, R, C, S, P)$: model identity M, runtime/compiler identity R, execution configuration C, interpretation contracts S and provenance P. Tokenizer identity belongs beside model identity even if it is stored in another file. S includes API, state/checkpoint, IR and policy schemas plus required backend features. The runtime is not free to reinterpret an old native vector or KV tensor merely because its shape fits.

The reference serializes a fixed record in sorted-key JSON and computes

$$d_B = \mathrm{SHA256}(\mathrm{encode}(B)).$$

This encoding has no floating-point or unordered-set ambiguity within its declared Python JSON profile; it is not a claim of cross-language RFC 8785 canonicalization. Production bundles should bind digests of actual files, a canonical manifest format and dependency closure; the fixture's strings are deliberately small substitutes. A mutable tag such as "latest" can locate a manifest, but the resolved digest must identify the revision used by each admitted request.

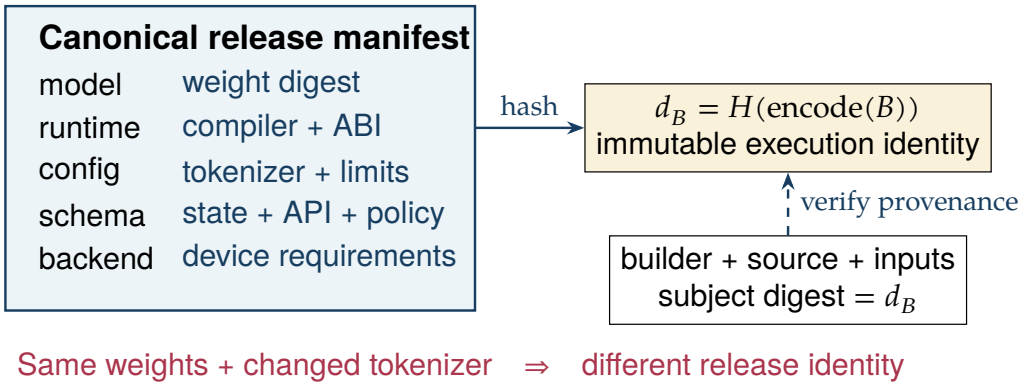


Figure 44.1: The manifest binds interpretation as well as payload. Changing a tokenizer or state schema creates a different identity even when weights are unchanged. Dashed arrows are verification inputs, not execution data.

```
    def digest(self):
        for k, v in asdict(self).items():
            if k in ("api", "state_schema", "policy_schema"):
                if type(v) is not int or v < 1:
                    raise ValueError("positive schema")
            else:
                text(v)
        return digest(asdict(self))
```

SLSA describes provenance as verifiable information about an artifact's production [102]. Record source revision, builder, dependencies, parameters and output digests. A nonempty provenance URI is not verification: an attacker can type one. Verification needs a trusted issuer, subject-digest binding and policy about acceptable builders. Our controller accepts a `trusted_provenance` boolean from such a hypothetical verifier; it implements no signature verification and claims no SLSA level.

44.1.1 A reproducibility test with two possible answers

Build the same manifest twice: its digest must match. Change only the tokenizer identifier: the digest must change. These are independent tests of serialization stability and semantic closure. Neither proves that recompiling source produces byte-identical binaries. Reproducible builds additionally control toolchains, dependency resolution, timestamps and environment inputs. Nor does artifact lineage prove model quality. Execution reproducibility, build reproducibility and task validity answer different questions.

44.1.2 Verify the delivered bytes against an independent identity

A release capsule must deliver the payloads named by its manifest. The local byte fixture has exactly four roles: model, runtime, tokenizer and configuration. Each manifest field stores the SHA-256 of its corresponding immutable byte string. The tokenizer payload is the four bytes `ACGT`; the other payloads are small assigned strings, not executable binaries or learned weights. The verifier rejects missing or extra roles, mutable payload containers, sizes above one million bytes per role and any content-digest mismatch.

```
def verify_capsule(bundle, payloads, trusted_digest):
    """Data-only bounded capsule; trusted identity is independently supplied."""
    if bundle.digest() != trusted_digest:
        raise ValueError("release identity differs from trusted pin")
    roles = ("model", "runtime", "tokenizer", "config")
    if type(payloads) is not dict or set(payloads) != set(roles):
        raise ValueError("exact payload role set")
    for role in roles:
        blob = payloads[role]
        if type(blob) is not bytes or len(blob) > 1_000_000:
            raise ValueError("bounded immutable payload")
        if hashlib.sha256(blob).hexdigest() != getattr(bundle, role):
            raise ValueError("payload content mismatch")
    return trusted_digest
```

First compare the manifest's identity with a trusted digest supplied through an independent channel. Then check each payload against that manifest. These checks answer different adversarial cases. Altering only weights fails the payload check. Altering weights and updating their manifest digest is internally consistent but changes the release identity, so the original trusted pin rejects it. If the attacker can also supply a replacement trusted pin, both checks pass: byte consistency cannot authenticate its own trust anchor. This negative control prevents "all hashes match" from being mistaken for provenance.

The contract establishes exact delivered byte identity under collision resistance and a trusted manifest pin. It does not parse model formats, reject NaN weights, prove reproducible compilation or establish a model's task quality. Those are separate gates. Chapter 28 already distinguishes tensor byte interpretation and finite numerical validity. Here the capsule extends that principle to the complete release envelope. The controller below still accepts metadata only: an integration must run payload verification and backend loading before marking the corresponding revision ready. No archive extraction or executable object deserialization occurs in this data-only fixture.

44.2 Compatibility is an acceptance relation

Write Accept(E, B) for an environment E accepting bundle B. It is a conjunction of required features and interpretation agreements, not a comparison of version strings. In the reference it requires exact API, state-schema, policy-schema and backend matches. This conservative rule rejects potentially convertible states. Supporting a converter requires a versioned transformation, old/new fixtures and a rollback policy.

For Chapter 31's restricted diagonal recurrence, bind the embedding, retention, write and readout parameters, dtype, vocabulary and consumed position. For Chapter 35's actual Chapter 23 Hermon stack, bind learned absolute positions, per-layer K/V geometry, normalization and causal mode. Replacing that stack with a rotary single-head demonstration would change the artifact. DOGMA remains the non-Transformer direction; Hermon DNA remains Transformer-based. This controller inspects release metadata. It does not verify either engine's numerical state, and it is not wired to the Chapter 39 runtime.

44.3 Roll out a revision, not a filename

A registered revision has passed metadata gates. It is not ready until its weights and dependencies are usable. A canary is eligible for a declared small traffic cohort; active means

selected for normal traffic. Superseded revisions stop receiving new work but may finish old requests. Draining closes admission before termination. A rolled-back candidate likewise retains responsibility for work already admitted.

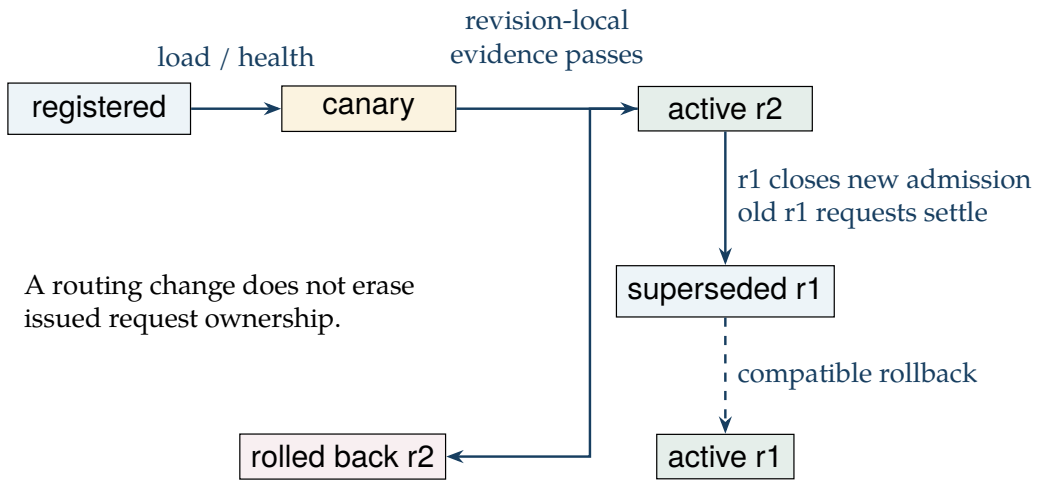


Figure 44.2: Promotion changes the routing choice. Existing request ownership remains attached to its admitted revision. Rollback closes the candidate's admission without erasing its unfinished requests.

Canary percentage, workload strata and observation window must be chosen before looking at results. A shadow candidate can run without publishing its output, but it must also be prevented from duplicating external effects. Shadow traffic that omits expensive tools or uses only short prompts cannot certify the corresponding full service workload.

```
    def promote(self, d):
        r = self.revisions[d]
        if r.state != "canary" or not r.ready or not r.live:
            raise RuntimeError("ready canary required")
        if not self.gate.passes(r.observations):
            raise RuntimeError("insufficient or failing revision-local evidence")
        if self.active is not None:
            self.previous = self.active
            old = self.revisions[self.active]
            old.state, old.ready = "superseded", False
        self.active, r.state = d, "active"
        self.emit("promoted", d)
```

A tempting shortcut promotes on global telemetry and lets an empty sample pass. The gate here uses only the candidate's own completed observations and requires a minimum count. This prevents r1's healthy history from certifying an unobserved r2. It is still a simple serial specification: a real canary needs windows, representative traffic and unresolved-request deadlines, since counting only completed requests can hide stuck ones.

44.4 Live, ready and able to reserve are separate predicates

Liveness asks whether a process can continue; readiness asks whether it should receive traffic. Startup may include loading or compilation. Kubernetes distinguishes these probe

purposes [104]; our service contract uses that distinction without implementing Kubernetes. A live process can be unready while warming, losing a dependency, exhausting memory or draining. Overload should not automatically trigger a liveness restart that discards useful work.

Even a ready revision can reject a request because capacity is exhausted. For request r, admission requires readiness, a compatible class, tenant authority, capacity and a plausible deadline. The simple controller reserves one global slot; the later isolation contract reserves resource vectors. Real estimates include prompt work, output reservation, native state or KV, tool budget and downstream decode capacity.

```
    def admit(self, d, request):
        text(request)
        r = self.revisions[d]
        if request in self.jobs or request in self.done:
            raise RuntimeError("request identity already used")
        if not r.ready or not r.live or r.state not in ("canary", "active"):
            raise RuntimeError("admission closed")
        if len(self.jobs) >= self.capacity:
            raise RuntimeError("global capacity")
        self.jobs[request] = d
        r.admitted += 1
        self.emit("admit", d, request)
```

```
    def complete(self, d, request, latency_ms, ok=True):
        nearest_rank([latency_ms], 1)
        if type(ok) is not bool:
            raise ValueError("boolean outcome")
        if self.jobs.get(request) != d:
            raise RuntimeError("unissued/foreign/already completed request")
        r = self.revisions[d]
        del self.jobs[request]
        self.done.add(request)
        r.completed += 1
        r.observations.append((latency_ms, ok))
        self.emit("complete", d, request)
```

Completion validates the observation before changing ownership, and binds an issued request ID to the revision that admitted it. Duplicate, foreign and late duplicate completions fail. This makes accounting testable instead of trusting callers to decrement a counter exactly once.

44.4.1 The conservation law behind graceful draining

Let A_d be cumulative admitted requests, C_d completed requests and I_d currently owned requests for revision d. All are integer counts, not rates. Initially they are zero. Admission increments A_d, I_d; completion increments C_d and decrements I_d. No other operation changes these quantities. Thus

$$A_d = C_d + I_d. \tag{44.1}$$

This is an induction over accepted events. Rejected operations preserve all three values. Closing admission prevents further increments of A_d; termination requires $I_d = 0$. A timeout is not an acknowledgement that a GPU kernel or tool no longer owns memory. A production

service must add explicit cancel/acknowledge and output-publication accounting from Chapter 30.

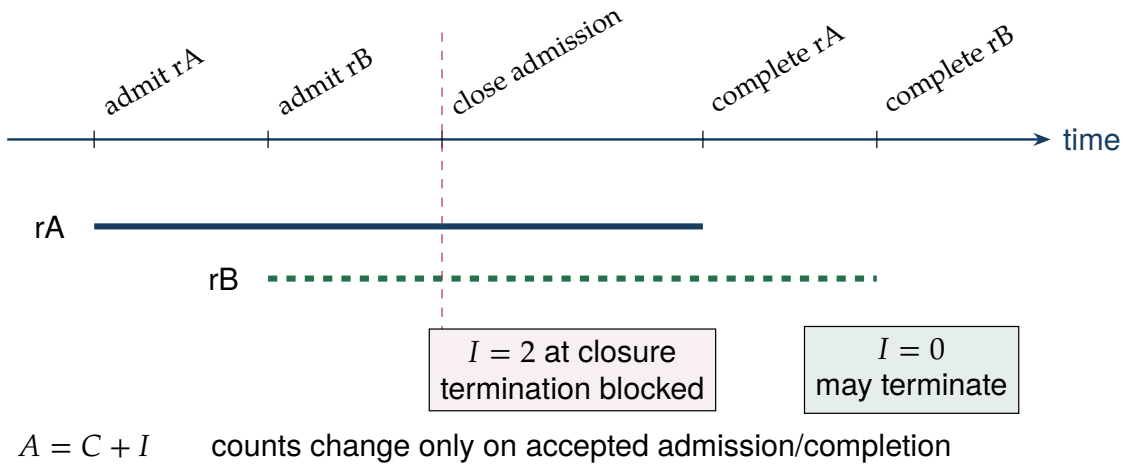


Figure 44.3: Drain is a protocol over ownership. At the middle instant admission is closed, but two issued requests still prevent termination. Late valid completions reduce ownership; duplicate completions cannot do so twice.

44.5 Observe causal operations without copying private content

A request may cross gateway, planner, compiler, router, engine, memory and tools. Parent/child spans represent causal operations; their durations can overlap, so summing them can exceed end-to-end latency. OpenTelemetry supplies common trace, metric and log concepts [103]. Domain attributes should bind artifact, logical engine, phase and outcome using versioned names. Trace context is correlation data, not authentication or tenant authority.

Track request/error rates, duration distributions, queue delay, TTFT, ITL and useful tokens/s alongside admission rejection. Record batch occupancy, KV/native-pool utilization, cache hit rates, retrieval/tool latency and rollback events so an SLO change can be related to resource pressure and plan changes. These are instrumentation requirements; the fixture emits only its small lifecycle schema and assigned completions, not these live metrics.

Use bounded-cardinality metric labels for engine, revision class and outcome. Putting request IDs, prompts or arbitrary resource paths into metric labels can exhaust the telemetry backend. Keep a restricted correlation ID in traces and diagnostic logs. Sampled traces cannot alone count every failed request; use unsampled counters for SLI denominators and record dropped telemetry. Redaction must happen before export, including exception messages and span attributes. Prompts, outputs, retrieved private documents and credentials are absent from our event schema.

The reference's sequential event numbers describe one process. They are not tamper-evident storage or distributed ordering. A production audit sink needs access control, retention, delivery/failure policy and privacy review.

44.6 Turn observations into a falsifiable service objective

An SLI is an observed indicator; an SLO is its target over an explicit population and window. For eligible offered requests in a one-hour class, an example is

$P(\text{success} \wedge TTFT \leq 500\,\text{ms}) \geq 0.99$. State how rejections, timeouts, cancellations and maintenance count. Accepted-only latency can improve merely by rejecting difficult requests.

The reference has a simpler completion-cohort gate: error fraction at most 0.01 and nearest-rank p95 latency at most 500 ms after at least 20 observations. For sorted values $x_{(1)} \leq \cdots \leq x_{(n)}$, the estimator is $x_{(\lceil qn \rceil)}$, $0 < q \leq 1$. Three durations $(10, 20, 900)$ have p95 900 ms. A lower interpolation/index shortcut could conceal that tail.

```
    def passes(self, observations):
        if (type(self.minimum) is not int or self.minimum < 1
                or not math.isfinite(self.max_error)
                or not 0 <= self.max_error <= 1
                or not math.isfinite(self.p95_ms) or self.p95_ms < 0):
            raise ValueError("invalid evidence gate")
        if len(observations) < self.minimum:
            return False
        return (sum(not ok for _, ok in observations) / len(observations)
                <= self.max_error and
                nearest_rank([ms for ms, _ in observations], .95)
                <= self.p95_ms)
```

Revision	n	p95 (ms)	Errors
r1	3	130	0
r2	3	130	0

The generated teaching trace deliberately overrides the minimum to three: both r1 and r2 observe $(40, 60, 130)$ ms, then r2 is rolled back. These are assigned numbers, not a reliability experiment or a recommended sample size.

For target A^*, allowed bad fraction is $E = 1 - A^*$. With N eligible requests, the budget is NE bad requests; burn rate is $b = (N_{bad}/N)/E$. At $A^* = 0.99$ and $N = 10{,}000$, 100 bad requests consume the budget; 200 imply $b = 2$. A short window estimates burn noisily. Even zero failures among n independent Bernoulli trials has one-sided 95% upper failure bound $1 - 0.05^{1/n}$: about 0.139 for $n = 20$. Independence and a stationary representative population are assumptions, often violated by correlated outages. Passing the deterministic gate is no confidence guarantee.

44.7 Configuration, secrets and a rollback that can actually run

Execution-critical configuration is immutable in the bundle. Dynamic policy knobs need version, validation and an effective-change event; admission must bind the applicable version. A secret reference can be configuration, but its resolved value must remain outside bundles, prompts, provenance and ordinary telemetry. Rotation needs scoped runtime resolution, not rebuilt model weights. This chapter resolves no secret and contacts no service.

Rollback changes executable selection. It does not reverse emails, payments, deletions or other effects already committed by tools. Retain old immutable artifacts and verify that the current durable-state schema is readable by the target. If r2 irreversibly migrated durable memory, reverting binaries alone can make recovery worse. A tested reversible migration, forward-compatible reader or separate restoration plan is required.

```
    def rollback(self, durable_schema):
        if self.active is None or self.previous is None:
            raise RuntimeError("no rollback pair")
        bad, target = self.active, self.previous
        r = self.revisions[target]
        if (r.state != "superseded" or not r.live
                or r.bundle.state_schema != durable_schema
                or not self.contract.accepts(r.bundle)):
            raise RuntimeError("rollback incompatible/unavailable")
        self.revisions[bad].state = "rolled_back"
        self.revisions[bad].ready = False
        r.state, r.ready = "active", True
        self.active, self.previous = target, None
        self.emit("rollback", target)
        return target
```

The reference refuses an incompatible or unavailable target, closes candidate admission and preserves its issued requests. It models one rollback pair, not a fleet consensus protocol. Crashes between routing and ownership updates, split controllers, lost journals and device completion races remain outside it.

44.8 Break the contract deliberately

Tests change semantic manifest fields, reject false provenance verification, attempt empty canary promotion, contaminate one revision with another's observations, exhaust global capacity, duplicate completions, drain with work outstanding and roll back into incompatible state. A bounded enumeration checks all $4^5 = 1024$ five-event words over admit/complete/drain/terminate against an independently maintained owner set. It catches accounting and state-machine errors, not arbitrary concurrent interleavings or process death.

Production experiments should additionally interrupt artifact download, kill the controller at each journal boundary, lose the telemetry sink, race shutdown with kernel acknowledgement and replay a routing update. Define expected recovery before injecting each fault. No such live experiments are run here.

Key ideas

- Changing only the tokenizer must change the release digest. Interpretation is part of identity.
- All hashes matching does not mean authenticated: an attacker who can replace the trust pin defeats byte consistency.
- Promotion evidence must come from the candidate's own observations. The old revision's healthy history certifies nothing about the new one.
- Accepted-only latency improves when you reject hard requests. Declare how rejections, timeouts and cancellations count.

44.9 Twelve worked exercises

1. **Bind configuration.** Weights are unchanged, but tokenization changes. May the digest remain equal? *Solution:* no; the computed token sequence and state interpretation can change. Include tokenizer/config identity.

2. **Separate provenance and quality.** A trusted builder produced poor weights. What passes? *Solution:* lineage verification may pass while task evaluation fails; require both gates rather than treating either as the other.

3. **Predict readiness.** Weights are loading in a healthy process. *Solution:* live, unready and closed to admission; liveness alone is insufficient.

4. **Drain arithmetic.** $A = 11, C = 8$. May shutdown finish? *Solution:* $I = 3$ by Equation 44.1; no. First close admission, then settle all three ownership obligations.

5. **Correlate logs.** Why include a safe trace ID but omit prompt text? *Solution:* the ID links causal diagnostics without copying protected payloads. It still needs access controls and is not an authorization credential.

6. **Calculate the tail.** Find nearest-rank p95 of $(10, 20, 900)$. *Solution:* rank $\lceil 2.85 \rceil = 3$, hence 900 ms; the 310 ms mean hides it.

7. **Test the SLO claim.** A kernel is twice as fast. Is the service SLO proved? *Solution:* no; queue, transfer, tools and offered-load outcomes remain unmeasured.

8. **Undo an effect.** r2 sent an email before rollback. *Solution:* selecting r1 does not retract it. Use an effect-specific compensation or idempotency protocol; never promise generic undo.

9. **Check schemas.** Old code reads schema 1; durable memory is schema 2. *Solution:* fail rollback unless a tested compatible reader/restoration exists.

10. **Place secrets.** Should a build manifest include an API token? *Solution:* no; include a scoped reference when needed and resolve after authorization.

11. **Quantify canary evidence.** Zero failures in 20 independent trials proves 99% reliability? *Solution:* no; the stated upper failure bound is about 13.9%. Representativeness and dependence are additional problems.

12. **Design a release experiment.** *Rubric:* freeze bundle and workload; declare compatibility, cohort, minimum evidence and timeout accounting; verify readiness/admission/drain; redact correlated telemetry; test rollback and durable schema; archive raw outcomes and rejected hypotheses. A checklist without outcomes earns no claim of a successful deployment.

44.10 Terms and the next boundary

Release identity binds the execution-critical manifest. **Readiness** permits routing but does not reserve capacity. **Error budget** is allowed bad service over a declared population/window. Chapter 45 adds mutually untrusted tenants. The same ownership and telemetry must now preserve identity, authority, resource quotas and privacy.

Evidence scope

Access checked 17 October 2026. Official SLSA v1.2 provenance definition, OpenTelemetry overview signal/context sections and Kubernetes probe semantics were inspected. These support the narrow terminology comparisons, not the correctness of a deployed Evolutor service. The bundle, evidence gate, conservation derivation, lifecycle oracle and figures are original teaching constructions. No signing, artifact build reproduction, telemetry exporter or

deployment experiment was performed. Local author review does not constitute independent specialist review.

CHAPTER 45

Multi-tenancy, isolation and security

In this chapter you will

- name the adversary and the trusted components before claiming isolation
- take identity from authentication and authorize deny-by-default against an epoch-versioned grant set
- issue capabilities with scope, session and expiry, and extend tenant namespaces through every stateful resource
- reserve every resource dimension before allocating, and keep untrusted text away from tool authority

A response can be numerically correct and still be a severe failure: it may contain another tenant's retrieved note, reuse a private prefix, execute an unauthorized tool or leak a credential through a trace. Consider Alice in tenant A and Bob in B. Both name an object `x`. A service that indexes only by `x` has already lost the intended separation before the model generates anything. Isolation is part of the execution contract.

The running fixture stores small text objects under tenant-qualified keys. It also issues local authority objects, reserves two resource dimensions and implements an in-memory echo tool. It authenticates nobody, resolves no credential and executes no external command. Its tests are local adversarial inputs to a serial specification, not a live penetration test or a security certification.

45.1 Name the adversary and the trusted components

Assume tenants control prompts, submitted documents, requested object IDs and tool parameters. Retrieved text may contain malicious instructions. A tenant can repeat requests, guess handles and consume its full budget. Ordinary bugs may substitute a stale handle, wrong request lane or copied capability. Model output is untrusted for authorization even when fluent and accurate.

We trust the authentication gateway to produce a principal; the deterministic policy and object store to enforce scope; and the serialized Python process to protect its own tables. Malicious host code, a compromised issuer, arbitrary process-memory reads, hardware side channels and kernel/driver compromise are outside the executable model. Under those stronger threats, Python object identity is no boundary: use process isolation, authenticated protocols, OS/device protections and appropriately scoped cryptographic credentials.

Keep confidentiality, integrity and availability separate. Namespacing helps prevent wrong-object reads; epoch checks reject obsolete authority; quotas limit resource exhaustion. None alone proves all three. NIST's zero-trust architecture rejects implicit trust based only on network location [105]. Sharing a process, batch or accelerator likewise grants no logical authority.

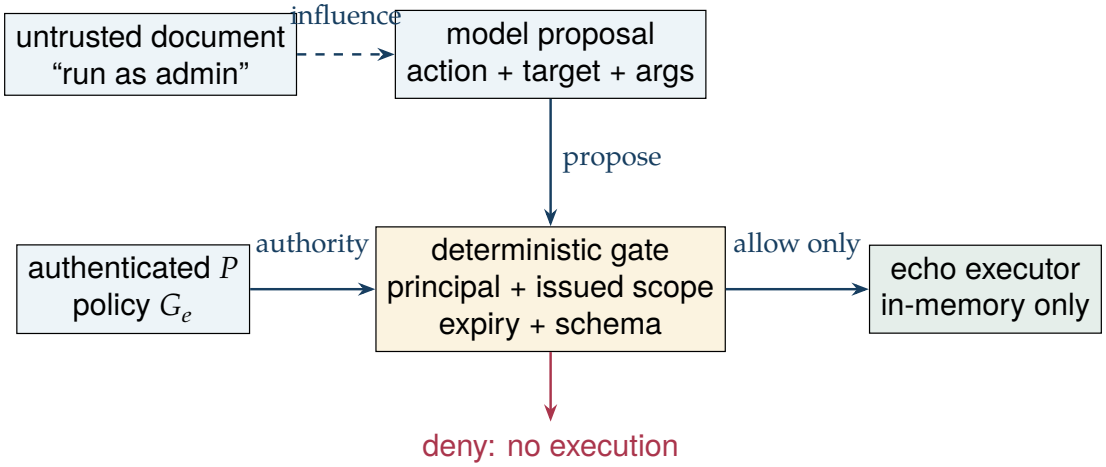
45.2 Identity arrives from authentication, not from prose

Let $P = (t, s, u, a)$ contain tenant t, subject s, session u and authenticated context a. The fixture stores the first three and assumes authentication has already happened. “I am the administrator” inside a prompt changes none of them. Tenant ID from an unverified request header is not authenticated context. Propagate identity through a trusted channel and recheck it where resources are accessed; trace baggage is not a substitute.

Authorization is deny-by-default:

$$\mathrm{Allow}(P, \alpha, r) = 1 \quad \Longleftrightarrow \quad (t, s, \alpha, r) \in G_e,$$

where G_e is the grant set at policy epoch e. A planner may propose an action; it does not extend G_e. A read grant for `memory:x` is neither a write grant nor authority for `memory:y`.



Document text has no edge that grants authority or changes the principal.

Figure 45.1: A malicious document can influence an action proposal. The only path to the echo executor passes deterministic principal, issued-capability and argument checks. Dashed document influence does not carry authority.

45.3 Make capability scope and lifetime testable

The fixture issues $c = (P, \alpha, r, e, \tau, n)$ with policy epoch e, exclusive expiry τ and serial n. Verification requires the exact issued object, matching principal/action/resource, current epoch, a still-present grant and $now < \tau$. Sessions are included so a stolen object from another session cannot pass through the normal interface. Replacing policy increments the epoch and invalidates all previously issued capabilities, including grants that happen to remain equal. This is conservative global revocation.

```
    def issue(self, p, action, resource, *, now=0, ttl=10):
        p.validate()
        name(action)
        name(resource)
        if type(now) is not int or now < 0 or type(ttl) is not int or ttl < 1:
            raise ValueError("integer time and positive TTL")
        if (p.tenant, p.subject, action, resource) not in self.grants:
            raise PermissionError("deny default")
        c = Capability(p, action, resource, self.epoch, now + ttl,
         ↪  self.next_serial)
```

```
        self.issued[c.serial] = c
        self.next_serial += 1
        return c
```

```
    def verify(self, p, c, action, resource, *, now=0):
        p.validate()
        if type(now) is not int or now < 0:
            raise ValueError("integer time")
        return (type(c) is Capability and self.issued.get(c.serial) is c
                and c.principal == p and c.action == action
                and c.resource == resource and c.epoch == self.epoch
                and now < c.expires
                and (p.tenant, p.subject, action, resource) in self.grants)
```

A deterministic nonce that is never checked against an issuance registry is not enough: constructing an equal-looking record would pass whenever a grant existed. The model here distinguishes an issued object from its copy. This is useful misuse protection for trusted callers; it is not a bearer-token format, MAC, signature or defense against code that can edit `issued`. A remote design needs issuer/audience binding, authenticated transport, anti-replay or idempotency appropriate to the action, key rotation and a revocation contract. Even here a capability can be used repeatedly until expiry; serial is not a one-shot nonce. The trusted integer clock must not roll backwards.

45.3.1 The time-of-check boundary

Our calls run serially, so revocation cannot occur between verification and store update. With concurrent workers, checking epoch then executing later can race policy replacement. Options include a transaction that binds decision and effect, or a short operation lease with explicitly bounded revocation delay. Specify whether revocation blocks new actions or also cancels already admitted ones. A prompt rule cannot resolve that race.

45.4 Namespaces extend through every stateful resource

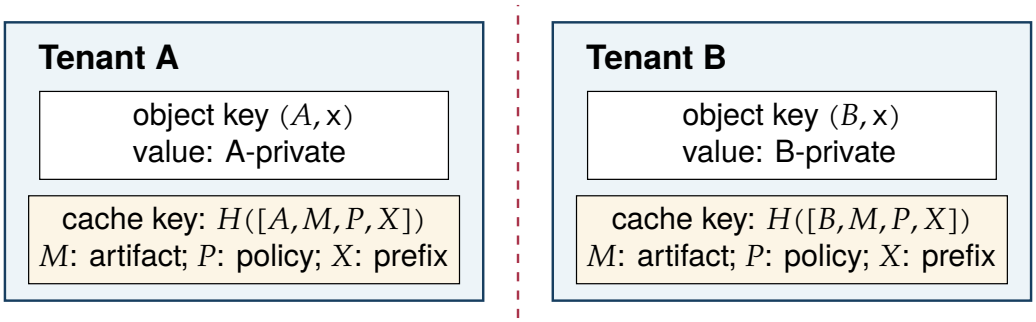
Objects are keyed by $(tenant, oid)$; authorization runs before lookup. Thus unauthorized guesses do not disclose whether another tenant's object exists through these two code paths. Real timing/error channels still require review. Our small store returns the caller's tenant object or a missing-object error; it never searches other tenants for a match.

A private cache key binds tenant, artifact, policy domain and prefix digest:

$$K = H(\text{encode}[t, d_M, d_{policy}, d_{prefix}]).$$

Length-delimited or structured encoding matters. Concatenating unescaped fields with a separator can alias `(A|B,C)` and `(A,B|C)`. A hash does not repair ambiguous input encoding. Nor does a tenant-scoped hash prove that the cached payload is authorized or consistent. Verify the stored envelope and compare complete identities on hit; a digest collision or poisoned entry must not become an authority shortcut. When subjects within a tenant have different rights, the policy domain must also distinguish their effective sharing permissions and revocation version. Tenant scope alone is then insufficient; a caller-supplied domain string cannot authorize sharing. The fixture tests key encoding, not such an ACL resolver.

```
def cache_key(p, artifact, policy_domain, prefix_digest):
    p.validate()
    fields = (p.tenant, artifact, policy_domain, prefix_digest)
    for v in fields:
        name(v)
    # JSON array is unambiguous even when names contain separators.
    return hashlib.sha256(json.dumps(fields, ensure_ascii=True,
                                     separators=(",", ":")).encode()).hexdigest()
```



Same object ID and prefix digest; distinct tenant-qualified keys.

No cross-tenant lookup, merge or shared authority is implied by equal values.

Figure 45.2: The same object ID and prefix digest occupy distinct logical tenant namespaces. Artifact and policy-domain fields also participate in reuse. Physical sharing is a separate, explicitly authorized refinement.

Apply the same scope to native state pools, checkpoints, KV pages, retrieval indexes, compiler plans, credentials and accounting. Chapter 32's generation protects reuse lifetime; tenant checks protect authority. A valid generation from a foreign tenant is still invalid. Pins must settle before storage is scrubbed and reassigned. Logical removal is not a demonstrated secure device erase; allocator, driver and hardware guarantees must be investigated.

45.4.1 A restricted noninterference claim

Let S_A, S_B be disjoint maps indexed by tenant. Suppose an operation by A checks A's authority, reads/writes only S_A, and its output is a function only of S_A, the request and fixed public configuration. Then replacing S_B while holding those inputs fixed cannot change A's output or S'_A. This follows directly by substitution: neither the read set nor transition contains S_B. Induction gives the same result for a serial sequence of A operations. The tests put different secrets at the same ID in A and B.

The premises exclude timing, shared capacity, audit scheduling and physical deduplication. With a global quota, B can make A's admission fail; strong availability noninterference is false. Likewise a private-prefix hit can change latency even if its content never leaks. State the observation model before claiming isolation; this proof concerns logical values only.

45.5 Reserve all resource dimensions before allocation

Let request r reserve vector $v_r = (s_r, b_r)$ of slots and bytes; both are nonnegative integers. For tenant limit L_t and global limit L_G, admit only if componentwise

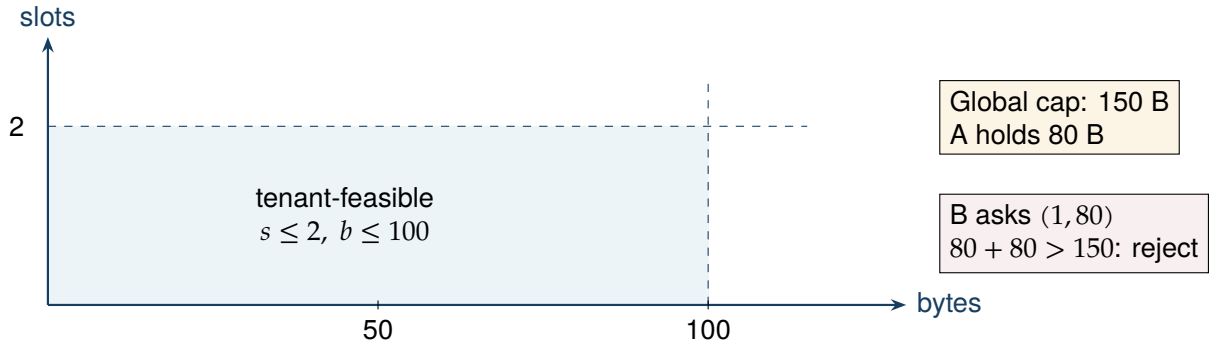
$$U_t + v_r \le L_t, \qquad U_G + v_r \le L_G, \quad U_t = \sum_{r \text{ held by } t} v_r.$$

Check the entire vector before acquiring anything. Reserving a slot first, then failing the byte check can leak slots unless rollback is exact. Release is bound to the issued reservation and can occur only once.

```
    def reserve(self, tenant, request, slots, nbytes):
        name(tenant)
        name(request)
        if any(type(v) is not int or v < 0 for v in (slots, nbytes)):
            raise ValueError("nonnegative integer resource vector")
        if tenant not in self.limits or (tenant, request) in self.held:
            raise RuntimeError("unknown tenant/duplicate reservation")
        want = (slots, nbytes)
        for used, limit in ((self.used(tenant), self.limits[tenant]),
                            (self.used(), self.global_limit)):
            if any(u + w > cap for u, w, cap in zip(used, want, limit)):
                raise RuntimeError("quota before allocation")
        r = Reservation(tenant, request, slots, nbytes)
        self.held[(tenant, request)] = r
        return r
```

Tenant	Held slots	Held bytes
A	1	80
B	1	80

With limits $(2, 100)$ for each tenant and $(4, 180)$ globally, two held requests reserve 160 bytes. A third 30-byte request may fit its own tenant but fails the global byte bound. Units are logical fixture bytes, not device allocation measurements. Real systems need KV/native-state capacity, compute, tool-call, bandwidth and durable-memory limits, plus conservative estimation and settlement against actual usage.



All dimensions checked before acquisition.
After rejection: A holds $(1, 80)$; B holds $(0, 0)$.

Figure 45.3: A two-dimensional feasible region rejects a request if either bound fails. The illustrated global byte failure leaves every reservation unchanged. Tenant quotas alone do not partition shared global capacity.

Quotas bound consumption; they do not guarantee fairness or deadlines. Rate limits, weighted scheduling, minimum reservations and backpressure need separate policies. Cancellation cannot release device memory before outstanding work acknowledges. Our fixture has no asynchronous device work and therefore models only logical reservation lifetime.

45.6 Untrusted text cannot mint tool authority

Prompt injection becomes a confused-authority problem when adversarial text influences a component with external privileges. OWASP recommends least privilege and controls outside the model [106, 107]. Segregate trusted instructions from retrieved data, but treat that labeling as one defense, not a proof that a model will never follow malicious text. System prompts guide behavior; they are not an access-control mechanism.

The executor validates principal, capability, target and a closed argument schema. In our only tool, the target must be `echo`, arguments must contain exactly one bounded text field, and output is an in-memory record. Untrusted text can be supplied but is not read by the authorization predicate. An authorized echo request containing "delete everything" remains echo; changing the target to root fails, and adding a shell argument fails schema validation. This tests authority separation, not a language-level detector.

```
    def tool(self, p, c, tool, args, untrusted_text="", now=0):
        # Only an in-memory echo operation exists. Never invokes a real tool.
        self.authorize(p, c, "execute", "tool:" + tool, now)
        if tool != "echo" or type(args) is not dict or set(args) != {"text"}:
            raise ValueError("closed tool/schema")
        if type(args["text"]) is not str or len(args["text"]) > 80:
            raise ValueError("bounded echo argument")
        return dict(executed="echo", tenant=p.tenant, text=args["text"])
```

For real tools, validate canonical resource targets, allowed parameter values, effect scope and output size. Resolve a credential only after authorization; never put it in a prompt. Bind any required approval to the concrete operation, resource, version and expiry so it cannot be replayed for another action. Retries need an effect-specific idempotency key; local model replay cannot guarantee exactly-once effects in an external service.

45.7 Isolation reaches memory, compiler and batch lanes

Retrieval must filter tenant scope before ranking and returning records. Deletion/tombstones and derived summaries preserve scope and provenance. Filtering only after global top-k both risks disclosure and changes recall: other tenants can displace a caller's valid records. This chapter does not import Chapter 40's memory API; these are the requirements an integration must meet.

An IR transformation may merge equal pure constants in a public domain, but equality of numerical tensors does not authorize private-state deduplication. Effect labels, tenant domain and policy version are legality premises for CSE, prefix reuse and materialization. Wiring these premises into Chapter 41's effect representation is a separate integration test.

Continuous batching may colocate tenants physically. Preserve separate masks, page tables, state handles, output channels, cancellation and accounting. A single extra cross-request attention edge violates the logical-value premise even if all arrays have valid shapes. Chapter 35 retains per-layer K/V of the actual learned-absolute-position Transformer; do not switch

position or model semantics to simplify an isolation demo. The Chapter 31 recurrence is a restricted teaching model, not full DOGMA security evidence.

Shared caches, accelerators and contention can expose timing and occupancy. Stronger adversaries may require partitioning, padding, dedicated workers or disabled sharing. No local authorization test eliminates those side channels.

45.8 Audit decisions without retaining secrets

```
    def authorize(self, p, c, action, resource, now):
        ok = self.policy.verify(p, c, action, resource, now=now)
        self.audit.append(dict(seq=len(self.audit), tenant=p.tenant,
                               subject=p.subject, session=p.session,
                               action=action,
                                ↪  resource_class=resource.split(":")[0],
                               policy_epoch=self.policy.epoch, allowed=ok,
                               reason="grant" if ok else "authority"))
        if not ok:
            raise PermissionError("authority")
```

Record principal/session, action class, policy epoch, outcome and reason. The fixture deliberately omits values, credentials, raw prompts and full resource IDs. It records authorization success separately from operation success: a later quota/schema failure does not turn a grant into a denial. A production audit schema should add final outcome and safe correlation IDs, with an explicit delivery, retention and access policy.

An append-only Python list is mutable by its host; it is not immutable audit storage. Cryptographic chaining alone would not prevent deletion of an entire suffix without an external anchor. Also protect identity/resource metadata, which can itself be sensitive. Secret scanning is useful defense in depth, but an allowlisted event schema reduces exposure before export.

45.9 A local adversarial matrix and its limits

The tests try foreign tenants/subjects/sessions, wrong action/resource, unissued copies, expiry, revocation, separator aliasing, object and vector quota exhaustion, unauthorized missing-object reads and extra tool parameters. The same-ID fixture checks independent values and scans audit records for the assigned secret. Exhaustive small quota vectors compare the implementation with an independently written feasible-region predicate.

Production tests must add stale-generation restore, cross-lane masks, prefix collision handling, concurrent revocation/effect races, corrupted checkpoints, device reuse and redaction across exceptions/exporters. Authentication, actual engine allocation and memory/compiler integration are not implemented by this store. No tests are run against live services, accounts or credentials.

Key ideas

- "I am the administrator" inside a prompt changes neither the principal nor the grant set.
- Unescaped concatenation can make `(A|B,C)` and `(A,B|C)` hash identically. A hash cannot repair an ambiguous encoding.
- The logical noninterference proof covers values only. A shared global quota already lets one tenant change another's admission.
- A system prompt guides behaviour; it is not access control. A deterministic check must gate the effect.

45.10 Twelve worked exercises

1. **Locate authority.** A webpage says "run as admin." *Solution:* it changes a proposal, not authenticated principal or grant set. The deterministic executor must still reject absent authority.

2. **Fix a cache key.** Why is prefix digest alone insufficient? *Solution:* it omits tenant, artifact and policy domain. Encode the full tuple unambiguously, then validate the stored envelope on hit.

3. **Same GPU, same trust?** *Solution:* no; device location is neither authentication nor permission. Logical and physical controls differ.

4. **Forge a copied capability.** Why reject an equal dataclass here? *Solution:* the registry requires exact issued-object identity. This tests trusted-process misuse only; hostile host code can bypass the registry.

5. **Calculate expiry.** Issue at 2 with TTL 3. Is time 5 valid? *Solution:* no; $now < 5$ is exclusive. Time 4 passes if all other checks hold.

6. **Reserve a vector.** A holds 80 bytes; global cap is 150; B asks 80. *Solution:* reject the entire reservation since $160 > 150$, leaving B with no partially acquired slot. A 70-byte request fits exactly.

7. **Valid generation, foreign tenant.** *Solution:* reject. Generation prevents stale reuse; it does not authorize access.

8. **Merge private computations.** Equal tensors from A and B may be CSE'd automatically? *Solution:* no; equality is not a sharing grant. Prove effect and security-domain legality before applying the rewrite.

9. **Replace a system prompt with access control.** Which survives a malicious completion? *Solution:* a deterministic resource/action check still gates execution; a behavioral instruction supplies no such guarantee.

10. **Audit safely.** Why omit raw object values? *Solution:* the audit may have broader retention/readership. Preserve decision evidence without duplicating protected content, and protect remaining metadata too.

11. **Break noninterference.** B fills a shared global quota. *Solution:* A's admission can change, so availability/timing noninterference does not follow from the logical-value proof.

12. **Design a release assessment.** *Rubric:* specify adversaries, trusted components and observations; test auth, issuance/revocation, scopes, quotas, state/cache/memory, tools and audit; declare side-channel and concurrency limits. Passing local tests earns no production-security certification.

45.11 Terms and the distribution boundary

Principal identifies an authenticated caller context. **Capability** is scoped issued authority under a declared trust model. **Noninterference** depends on which observations an adversary can make. Chapter 46 moves state across machines; placement must preserve these logical identities and authority even when location and ownership epoch change.

Evidence scope

Access checked 17 October 2026. NIST's publication overview was inspected for the no-implicit-network-trust principle; this is not a full implementation audit against every SP 800-207 requirement. OWASP's current LLM01:2025 and LLM07:2025 pages were inspected for mitigation and external authorization boundaries. Their recommendations are source leads, not authorization to test live systems. No penetration test was performed. The capability registry, namespace proof, quota oracle, local adversaries and mechanism figures are original teaching constructions. They assume trusted serialized host code and do not establish cryptographic or hardware isolation.

CHAPTER 46

Distributed serving and state placement

In this chapter you will

- separate the control plane from the data plane, and routing from placement
- represent state identity independently of location with an owner and a fencing epoch
- migrate from a stable snapshot through prepare, copy, verify and commit, and prove at most one owner in the serial model
- reserve paired prefill and decode capacity and estimate state-transfer cost

Where does a live request's state reside? A single process hides that question behind a pointer. Across hosts, moving the bytes and moving authority become different operations. Suppose worker a owns state at epoch 7 and worker b receives a copy. If both accept the next token, the request has forked even though the copied bytes were perfect. If neither accepts it after a crash, availability has failed even though confidentiality may remain intact.

Our central contract is that placement may change location but never logical identity or authority. A serialized cluster model will make preparation, reservation, verification, commit, cancellation and recovery explicit. It is a specification for a future distributed implementation, not a network transport, consensus service, RDMA system or live cluster experiment.

46.1 Separate control decisions from data movement

The control plane records worker membership, capabilities, capacity, artifacts, health and topology. The data plane carries token/tensor/state traffic. Advertisements are evidence with a freshness interval, not permanent facts: a worker may lose its accelerator after announcing free memory. Workers must revalidate reservations when accepting work. Authenticate control messages; being reachable on an internal network is not authority.

A useful advertisement includes device/backend, precision and kernel support, loaded artifact digests, available memory, zone/rack, schema and health epoch. Routing chooses a logical implementation; placement chooses a physical instance of that implementation. The scheduler cannot silently substitute a different model or precision merely because a worker is idle.

For request r, choose from hard-feasible set $\mathcal{E}(r)$ before ranking:

$$w^* = \arg\min_{w \in \mathcal{E}(r)} (\alpha Q_w + \beta T_{rw} + \gamma C_w + \delta R_w).$$

Q_w, T_{rw} are predicted queue/transfer durations in milliseconds; C_w, R_w are declared cost/risk scores with weights converting to the same objective units. They are estimates, not measured latency guarantees. Empty $\mathcal{E}(r)$ means reject or queue, not relax tenant or artifact constraints.

```
    def eligible(self, w, tenant, artifact, schema, backend):
        return (w.wid in self.alive and w.backend == backend
                and w.schema == schema and tenant in w.tenants
                and artifact in w.artifacts
                and self.used[w.wid] + self.reserved[w.wid] < w.capacity)
```

The fixture requires live membership, exact backend/schema, artifact residency, tenant permission and an available slot. It then adds a synthetic 10 ms nonpreferred-zone penalty to queue time and breaks ties by occupancy and ID. A locality preference is soft; authority and compatibility are hard.

46.2 Represent identity independently of location

The handle is $H = (sid, t, d_M, schema, backend, owner, e)$. Logical ID sid, tenant t, artifact d_M, state interpretation and backend contract survive migration. Owner and monotonic epoch e change together. Epoch is a fencing generation, not wall-clock time and not a model-token position. Value equality with the authoritative directory is required for a handle to operate in the fixture. This rejects accidental foreign/stale records; it is not authentication against a hostile process reading the directory.

Immutable weights, tokenizers and compiler outputs can be replicated after digest verification. Mutable request state instead needs a consistency protocol. We choose one authoritative owner, with temporary destination copies that cannot execute. A replicated durable-memory service can have another consistency model; do not infer it from engine-state ownership.

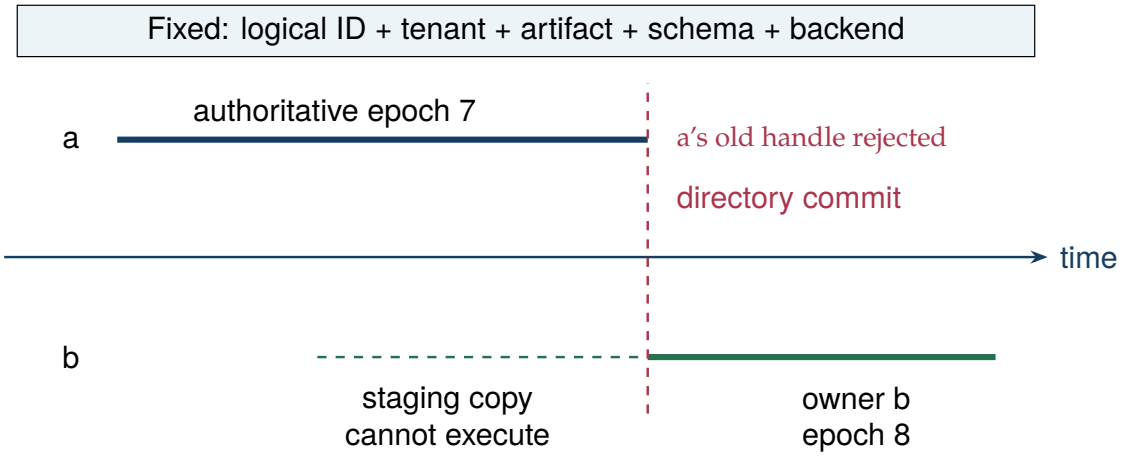


Figure 46.1: A copied snapshot at b is not an owner. Only the directory transition from epoch 7 to 8 authorizes b and fences a. The identity fields above the timeline remain fixed.

46.3 Derive migration from a stable snapshot

A prepare–copy–verify–commit sequence is the starting point, but preparing must also freeze updates or capture a versioned snapshot. Otherwise the source could advance while the old snapshot is copied and commit a stale continuation. We choose freezing to make the reference small and exact. Reads remain allowed; writes are rejected until commit or abort.

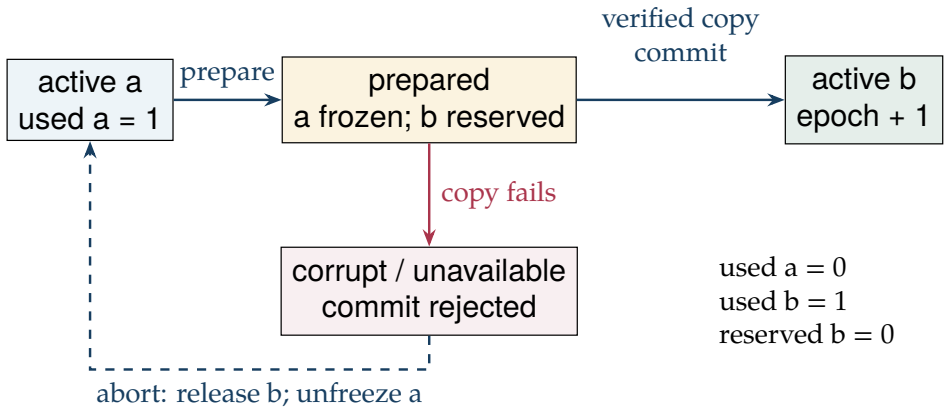
```
    def prepare(self, h, dst):
        s = self.current(h)
        if s.status != "active" or dst == h.owner or dst not in self.workers:
            raise RuntimeError("invalid prepare")
        w = self.workers[dst]
        if not self.eligible(w, h.tenant, h.artifact, h.schema, h.backend):
            raise RuntimeError("incompatible/full destination")
        t = Transfer(h, dst, s.payload, hashlib.sha256(s.payload).hexdigest())
        self.reserved[dst] += 1
```

```
        s.status = "frozen"
        self.pending[h.sid] = t
        self.emit("prepare", h, dst)
        return t
```

Preparation checks the complete destination contract before reserving one slot. It registers an exact transfer object containing the source handle, destination, immutable byte snapshot and digest. A production transfer needs an authenticated envelope binding these fields, size bounds and chunk numbers; a bare hash over bytes is not a MAC and does not establish who sent them.

Copy verification checks bytes, then commit changes owner and epoch in one serialized directory transition. Destination capacity was already reserved. The source reservation is released only at commit; during the transfer both source storage and destination staging consume capacity.

```
    def commit(self, t, received):
        if self.pending.get(t.handle.sid) is not t:
            raise RuntimeError("unissued/settled transfer")
        s = self.current(t.handle)
        if s.status != "frozen" or t.dst not in self.alive:
            raise RuntimeError("not commit-ready")
        if type(received) is not bytes or hashlib.sha256(received).hexdigest() !=
         ↪  t.sha256:
            raise RuntimeError("transfer integrity")
        # One serialized authority transition is the specification's linearization
         ↪  point.
        h = replace(t.handle, owner=t.dst, epoch=t.handle.epoch + 1)
        s.handle, s.payload, s.status = h, received, "active"
        del self.pending[h.sid]
        self.reserved[t.dst] -= 1
        self.used[t.handle.owner] -= 1
        self.used[t.dst] += 1
        self.emit("commit", h, t.dst)
        return h
```



Failure before commit preserves source authority; explicit abort settles staging.

Figure 46.2: Prepare reserves b and freezes a; a corrupted copy cannot cross the commit point. Abort restores a's writable state and releases b. Successful commit changes directory owner/epoch once and rejects old-epoch operations.

46.3.1 Safety proof and its distributed premise

Proposition 46.1 (Single authoritative owner in the serial model)**.** *Starting from one current handle per live state, any sequence of accepted create, prepare, abort, commit, cancel and recover calls leaves at most one handle accepted for mutation of each logical state.*

Proof. Create installs one handle. Prepare retains it but freezes writes; its copied payload grants no execution authority. Abort retains the handle and unfreezes it. Commit replaces it by an otherwise identical handle with a new owner and epoch. Exact directory comparison rejects the old handle. Cancel marks state terminal and advances its epoch; recovery replaces a failed owner's handle with a new epoch. No operation creates a second authoritative directory row. Induction over calls proves the property. □

This proof assumes one serial, durable-in-concept authority transition and that every side effect respects its fence. Two partitioned controllers each claiming authority violate the premise. A distributed implementation needs a linearizable metadata service, fenced storage operations or another explicitly proved protocol. Expiring a lease using local clocks does not by itself stop a paused old worker from resuming and writing. Fencing must be enforced at the resource that accepts the mutation. Our Python assignment supplies neither consensus nor crash durability.

46.3.2 Conserve both owners and staging reservations

For worker w, let O_w be current noncancelled owners, T_w incoming prepared transfers and D_w held prefill/decode reservations. With capacity C_w in fixture slots,

$$used_w = |O_w|, \quad reserved_w = |T_w| + |D_w|, \qquad used_w + reserved_w \le C_w.$$

Preparing adds T_w; commit exchanges one destination staging slot for an owner slot and removes the source owner slot. Abort removes only staging. Cancellation removes the owner and any pending transfer exactly once. Repeated migrations and fault tests check these equalities against directory and transfer tables rather than merely checking the final payload.

46.4 Failure windows, cancellation and recovery

Before prepare, failure leaves no destination obligation. During copy, destination failure permits abort if the source remains usable; source failure requires evidence-backed recovery or request failure. After commit, the old handle is stale even if the acknowledgement was lost. Retrying must query the authoritative directory by migration identity; blindly replaying commit can allocate twice. The reference rejects settled transfer objects rather than implementing a durable retry journal.

Cancellation targets the current owner epoch, aborts a prepared transfer and makes state terminal. A late old transfer cannot resurrect it. Real cancellation also requires propagation, device acknowledgement and delayed physical reclamation; changing a directory flag cannot stop a kernel already accessing memory. We model only serialized logical state and capacity.

```
    def recover(self, h, dst, replayed, expected_digest):
        s = self.current(h, live=False)
        if h.owner in self.alive or s.status != "active":
            raise RuntimeError("failed active owner required")
        if dst not in self.workers or not self.eligible(
                self.workers[dst], h.tenant, h.artifact, h.schema, h.backend):
            raise RuntimeError("incompatible recovery destination")
        if type(replayed) is not bytes or hashlib.sha256(replayed).hexdigest() !=
         ↪  expected_digest:
            raise RuntimeError("trusted replay evidence mismatch")
        nh = replace(h, owner=dst, epoch=h.epoch + 1)
        s.handle, s.payload = nh, replayed
        self.used[h.owner] -= 1
        self.used[dst] += 1
        self.emit("recover", nh)
        return nh
```

Recovery requires a failed active owner, a compatible destination and replay bytes matching a digest supplied by a trusted checkpoint/replay ledger. The expected digest must be independent evidence: hashing arbitrary replay and passing that same hash proves nothing. The fixture checks equality but implements neither that durable ledger nor deterministic replay itself. If no valid checkpoint or replay source exists, fail or restart the request; never guess missing state. An owner failing during a frozen transfer requires explicit abort before this recovery path.

Internal state replay does not guarantee exactly-once external effects. A tool may commit an action and crash before recording its acknowledgement. Retrying can duplicate it unless the external service supports an idempotency or transaction protocol. The logical model performs no external action.

46.5 DOGMA and Hermon move different representations

Chapter 31's restricted diagonal recurrence carries a hidden vector and token position; it is not full DOGMA. For a width D float-b vector, the hidden payload is Db bytes, excluding metadata, banks and parameters. A broader native model may carry much more. Compactness alone says nothing about equal task capacity or speed.

Chapter 35's causal Hermon adapter uses the actual Chapter 23 multi-layer Transformer with learned absolute positions. Its K/V payload is

$$M_{KV} = 2LtH_{kv}d_h b \quad \text{bytes},$$

for L layers, t consumed tokens, H_{kv} KV heads and width d_h. Migration must preserve ordered per-layer content, absolute consumed position, owner and artifact/mode metadata. A paged representation adds page tables, partial tails and references. Paging solves local allocation, not transport.

The local tests serialize actual Chapter 31 hidden values and actual Chapter 35 per-layer K/V into bounded data-only fixture envelopes, transfer them and reconstruct through those same model interfaces. One-token suffix logits match the unmoved state exactly. This is a trusted test-local adapter; it is not an untrusted checkpoint loader, page migration, pool ownership transfer or an integrated distributed engine. No detached toy replaces either model.

46.6 Place prefill and decode together only when justified

Transformer prefill and decode can interfere and favor different parallelism. DistServe studies their separation, phase-specific allocation and bandwidth-aware placement [108]. Its evaluated system is not this oracle, and its reported improvements cannot be transferred to Hermon or DOGMA without measurement.

Disaggregation introduces state transfer between phases. With payload B bytes, sustained bandwidth W bytes/s and fixed serialization/coordination cost ℓ ms, a simple estimate is

$$T_{move} = \ell + 1000B/W \quad \text{ms}.$$

It excludes contention, packet retries, overlap and reconstruction costs. Separate when predicted queue/interference savings exceed added critical-path transfer and coordination under the stated SLO. This is a plan-selection hypothesis, not a universal performance claim.

Payload	Bytes	Model ms
Native	8192	0.108192
KV	402653184	402.753

The assigned comparison uses 1 GB/s decimal bandwidth and 0.1 ms fixed cost. The 8,192-byte vector is a width-1,024 float64 payload example. The 384 MiB KV example uses $L = 24, t = 4096, H_{kv} = 8, d_h = 128, b = 2$ from Chapter 35's arithmetic example. These are different representations, not equal-capability models; neither row measures actual network or device performance.

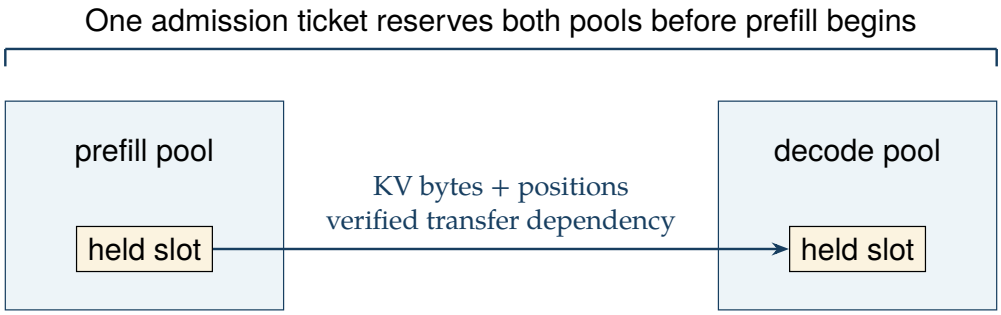


Figure 46.3: Admission reserves both pools before starting prefill. The KV transfer is an explicit causal dependency before decode; a full decode pool rejects the reservation without stranding prefill work.

```
def pd_reserve(cluster, tenant, artifact, zone=None):
    # No mutation until both constrained pools can reserve a slot.
    p = cluster.place(tenant, artifact, 1, "prefill", zone)
    d = cluster.place(tenant, artifact, 1, "decode", zone)
    cluster.reserved[p] += 1
    cluster.reserved[d] += 1
    pair = (p, d)
    ticket = object()
    cluster.pd[ticket] = pair
    return ticket, pair
```

Our paired reservation checks both pools before changing either counter, and returns a ticket released once. It does not start prefill or convert that ticket into a live engine request. A concurrent coordinator needs transactional or leased reservations and crash cleanup. Per-tenant quota also remains an upstream prerequisite, not an implemented distributed quota ledger.

46.7 Topology, backpressure and distributed observations

GPU-to-GPU, host, rack and zone links differ in latency, bandwidth and failure domain. A faster empty worker can lose to a nearby loaded one after transfer; a nearby incompatible worker remains ineligible. Measure serialization, network, destination reconstruction and commit separately, while recording overlap. Backpressure must expose downstream reservations and queue pressure upstream; unbounded queues merely move overload and grow retained state.

Each placement/migration record should bind request-safe correlation ID, tenant scope, artifact, logical backend, worker, owner epoch, reason and estimated/actual transfer bytes. Trace prefill, transfer and decode as distinct causal spans. Across hosts, clock offsets can make naive subtraction invalid; use local durations and explicit clock synchronization/error bounds where cross-host timestamps are compared. The reference records sequential event IDs only and does not measure distributed time.

46.8 Fault injection is a protocol test, not a slogan

Tests corrupt transfer bytes, copy issued transfer objects, kill source or destination during copy, cancel before/after preparation and commit, fill destination capacity, attempt incompatible placements, repeat migrations and reject unverified replay. Actual native/KV continuation fixtures test byte preservation at the model boundary. A destination reservation is not counted as a second owner, and failed preparation must not alter capacity.

Production validation should delay/duplicate messages, lose commit replies, partition the metadata service, restart processes with persistent journals, kill a worker at every protocol boundary and race real device completion. Write the expected owner, epoch, capacity and output disposition for each fault before running it. No network fault or remote experiment occurs here.

> **Key ideas**
>
> - Copying bytes does not move authority. Only the directory transition from epoch e to $e + 1$ does, and it fences the old owner.
> - Lease expiry on local clocks does not stop a paused old worker. Fencing must be enforced where the mutation is accepted.
> - Reserve both pools before starting prefill. Otherwise prefill can produce state with no admitted destination.
> - Replaying internal model state does not deduplicate external effects.

46.9 Twelve worked exercises

1. **Routing or placement?** Choosing Hermon versus a native engine is routing; choosing worker b for that engine is placement. *Solution:* preserve the semantic choice while optimizing its physical realization.

2. **Migrate epoch 7.** Which fields change? *Solution:* owner and epoch, now 8. Logical ID, tenant, artifact, schema and backend remain fixed.
3. **Corrupt the copy.** When can authority move? *Solution:* only after verification. Failure retains source authority and staging until abort.
4. **Use the old handle.** It still has correct bytes. *Solution:* reject owner/epoch mismatch; numerical validity is not authority.
5. **Replicate weights versus request state.** *Solution:* immutable digest-verified weights have no update race. Mutable replicas require an explicit consistency/ownership protocol.
6. **Compute movement.** Transfer 1 MB at 1 GB/s with 0.1 ms fixed cost. *Solution:* $1000(10^6/10^9) + 0.1 = 1.1$ ms, before contention or reconstruction.
7. **Replay an acknowledged tool?** *Solution:* model-state replay does not deduplicate an external effect; use the tool's own idempotency contract.
8. **Lose state without evidence.** *Solution:* fail/restart the request. A plausible invented state violates continuation correctness.
9. **Reserve downstream first.** Decode is full but prefill is empty. *Solution:* reject the pair before acquiring either reservation; otherwise prefill can produce state that has no admitted destination.
10. **Backpressure or larger queues?** *Solution:* bound queues and expose pressure. Larger unbounded queues increase memory and tail latency.
11. **Compact state guarantees cheaper service?** *Solution:* no; include actual representation, serialization, weights, computation, capability and topology. The payload table proves none of these end-to-end comparisons.
12. **Design distributed validation.** *Rubric:* define hard placement constraints; directory linearization/fencing; stable snapshots; reservation accounting; crash/cancel/retry outcomes; trusted replay evidence and tool effects; measure topology and trace provenance. A serial oracle cannot close those gates.

46.10 Terms and the measurement boundary

Fencing epoch rejects operations from an obsolete owner. **Staging reservation** accounts for a copy that cannot execute. **Linearization point** is the single authority transition assumed here. Chapter 47 asks which placement and kernel costs are actually measured, and which remain arithmetic estimates or synthetic traces.

Evidence scope

Access checked 17 October 2026. DistServe's primary OSDI paper PDF was accessed; abstract/introduction and selected architecture/phase-allocation passages were inspected, not every evaluation or appendix. The chapter cites the disaggregation and bandwidth-aware placement mechanism without adopting its performance numbers. The serial ownership proof, transfer/capacity oracle, fault fixtures and illustrations are original teaching constructions. Actual Chapter 31/35 and transitive Chapter 23 sources provide local model interfaces for test-only snapshot continuation. This is not a production distributed engine or a proof of consensus, transport authentication, durable recovery or exactly-once tools. Independent review remains open.

CHAPTER 47

Profiling, performance and hardware backends

In this chapter you will

- establish correctness parity before any timing comparison
- freeze a workload, an arrival process and a measurement population
- derive TTFT, ITL, goodput and nearest-rank percentiles from a single observation clock
- find the critical path, apply Amdahl's model and state roofline ceilings at a named memory boundary

An optimization changes a kernel from 100 to 80 ms. Its speedup is 1.25, while latency falls 20%. Now suppose that kernel occupied only one fifth of the request's critical path. Even an ideal twofold kernel improvement would reduce the request to 90% of its old duration. Optimizing the wrong region can make an impressive profiler screenshot irrelevant to the user.

This chapter follows a top-down measurement ladder over a synthetic request trace. It makes explicit the time populations, failure denominators, critical-path arithmetic, uncertainty and backend validation contracts. The executable analysis runs on assigned numbers. It reports no measured CPU, Metal, CUDA or Vulkan throughput and makes no hardware ranking.

47.1 Correctness is a precondition for comparison

Fix model/artifact, tokenizer, position convention, accepted input stream, state lifetime, output semantics and numerical tolerance before timing. For Chapter 31's restricted native recurrence, compare hidden state and logits with its scalar path; do not call it the full DOGMA architecture. For Chapter 35 use the actual Chapter 23 learned-absolute-position multi-layer Hermon Transformer and its original full causal oracle, including every layer's K/V. A detached rotary one-head fixture would test a different model.

Test multiple shapes, empty/partial inputs, masks, capacity boundaries, permutations and cancellation. Close logits do not guarantee the same selected token near a tie. If a lower-precision or approximate kernel changes sampling behavior, report numerical and task-quality effects under the new contract. Correctness tests are necessary; they do not establish learned capability.

47.2 Freeze a workload and a measurement population

A benchmark record includes commit/artifact and data hashes, hardware/firmware, OS, driver, compiler, backend libraries, precision/quantization, scheduler, memory budget, power/clocks, input/output lengths, arrival process, concurrency, cache policy, sampling, warmup, repetitions and raw outcomes. Preserve offered, accepted, rejected, cancelled, failed and completed counts separately. Report peak memory and energy only when actually measured. For energy per accepted token, declare the power-sampling boundary, idle

subtraction, integration interval and accepted-token denominator; device and whole-host energy differ.

An open-loop generator schedules arrivals independently of response time; a closed-loop generator waits for completion or a concurrency slot. Both are useful, but a slow closed-loop service reduces offered load automatically. It can hide the overload a production arrival stream would create. Fix and replay traces, include burstiness, and state whether the load generator itself became the bottleneck. Batching a preconstructed full tensor does not evaluate continuous serving under arrivals.

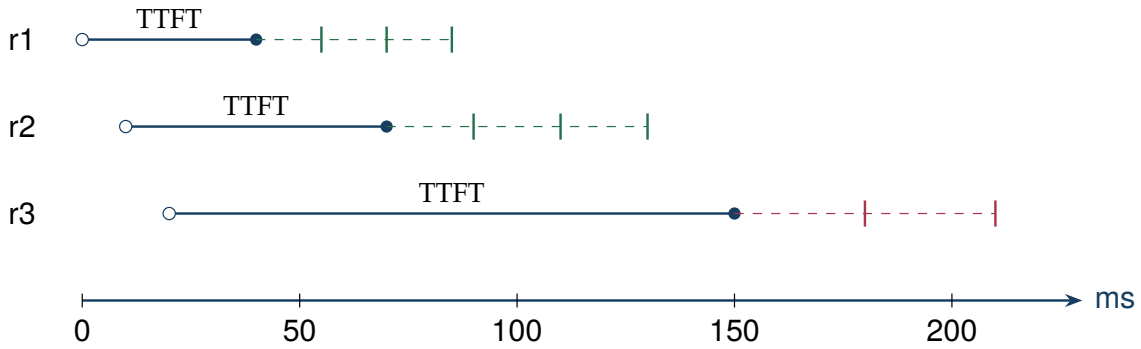
Separate cold artifact loading, compilation/graph capture, allocator startup and cache warming from steady state unless cold start is the target metric. Specify warmup by iterations or a declared stationarity rule, and archive discarded warmup samples. Do not choose the cutoff after inspecting the best performance region. Thermal/power changes can invalidate stationarity.

47.3 Trace timestamps into user-visible metrics

For request r, use a single observation clock: arrival a_r, first output token $t_{r,1}$ and subsequent token times $t_{r,2}, \dots, t_{r,n_r}$ in ms. The first token is not repeated in the subsequent-times tuple. Define

$$TTFT_r = t_{r,1} - a_r, \qquad ITL_{r,i} = t_{r,i} - t_{r,i-1} \quad (i \geq 2).$$

There is no ITL for a one-token response. A failed zero-output request has no TTFT sample, not a zero-millisecond success. It requires an explicit terminal timestamp; requests with outputs may also specify one after their last token. Keep failures in the outcome denominator and check terminal time against the window. Cross-host clocks need synchronization and error bounds before subtraction.



Open circle: arrival filled circle: first token tick: later token

Assigned synthetic timestamps; r3 gaps = 30 ms, above the 25 ms target.

Figure 47.1: The three supplied synthetic requests have different arrivals and token gaps. The first token counts once; TTFT includes waiting before it. The long third request violates both the 100 ms TTFT and 25 ms maximum-gap targets.

```
    def validate(self):
        finite(self.arrival_ms)
        if type(self.ok) is not bool or type(self.token_times_ms) is not tuple:
            raise ValueError("boolean outcome and immutable timestamps")
        if self.first_token_ms is None:
            if self.ok or self.token_times_ms:
                raise ValueError("only failed requests can have zero tokens")
```

```
            if self.terminal_ms is None or finite(self.terminal_ms) <
             ↪  self.arrival_ms:
                raise ValueError("tokenless failure requires terminal timestamp")
            return
        times = (self.arrival_ms, self.first_token_ms) + self.token_times_ms
        for t in times:
            finite(t)
        if any(b < a for a, b in zip(times, times[1:])):
            raise ValueError("monotonic same-clock timestamps")
        if self.terminal_ms is not None and finite(self.terminal_ms) < times[-1]:
            raise ValueError("terminal time before last token")
```

For a closed completed cohort observed over $[s, e]$, raw output-token rate is $\sum_r n_r/((e-s)/1000)$ in tokens/s. Our goodput counts successful requests meeting TTFT and *every* ITL bound, divided by the same seconds. This maximum-gap SLO is stricter than an average time-per-output-token (TPOT) target. A request with gaps $(1, 99)$ ms has mean 50 ms but fails a 60 ms maximum-gap objective. Do not compare systems using different SLOs under the same goodput label. DistServe uses a phase-sensitive TTFT/TPOT framework [108]; we do not reproduce its experiment.

```
def metrics(reqs, start_ms, end_ms, ttft_slo, itl_slo):
    finite(start_ms)
    finite(end_ms)
    finite(ttft_slo)
    finite(itl_slo)
    if not reqs or end_ms <= start_ms:
        raise ValueError("nonempty completed cohort and positive window")
    for r in reqs:
        r.validate()
        if (r.arrival_ms < start_ms or r.arrival_ms >= end_ms
                or r.completion_ms() > end_ms):
            raise ValueError("cohort outside declared observation window")
    tt = [r.ttft() for r in reqs if r.ttft() is not None]
    itl = [x for r in reqs for x in r.itls()]
    good = sum(r.ok and r.ttft() is not None and r.ttft() <= ttft_slo
               and all(x <= itl_slo for x in r.itls()) for r in reqs)
    tokens = sum((0 if r.first_token_ms is None else 1) + len(r.token_times_ms)
                 for r in reqs)
    seconds = (end_ms - start_ms) / 1000
    return dict(requests=len(reqs), output_tokens=tokens,
                p50_ttft=percentile(tt, .5) if tt else None,
                p95_ttft=percentile(tt, .95) if tt else None,
                p95_itl=percentile(itl, .95) if itl else None,
                output_tokens_per_s=tokens / seconds,
                goodput_req_per_s=good / seconds, slo_attainment=good / len(reqs))
```

The supplied trace has arrivals $(0, 10, 20)$, first tokens $(40, 70, 150)$ and subsequent tokens $(55, 70, 85)$, $(90, 110, 130)$, $(180, 210)$ ms. Over 250 ms, there are 11 tokens: 44 tokens/s. Two of three requests meet the 100/25 ms TTFT/ITL bounds: 8 good requests/s and attainment 2/3. TTFTs are $(40, 60, 130)$; nearest-rank p50/p95 are 60/130 ms and pooled p95 ITL is 30 ms. Every number is synthetic arithmetic, not a service measurement.

The analyzer requires all arrivals and emitted tokens to lie in its declared window. It is a closed-cohort analysis, not a streaming estimator with pending requests. A real offered-load SLO must also count requests unresolved at the deadline. Silently dropping them creates

survivor bias. Token-weighted pooled ITL and request-weighted worst-gap distributions are different populations; long outputs contribute more samples to the first.

47.4 Percentiles and uncertainty need declared estimators

For n sorted durations and $0 < q \le 1$, nearest rank selects $x_{(\lceil qn \rceil)}$. The reference rejects empty samples, nonfinite or negative times and invalid quantiles. Missing ITL is represented as no value, not zero. A four-sample p95 is the maximum and cannot characterize a rare production tail reliably.

```
def percentile(xs, q):
    if not xs or type(q) not in (int, float) or not math.isfinite(q) or not 0 < q
     ↪  <= 1:
        raise ValueError("nonempty finite samples and 0 < q <= 1")
    return sorted(finite(x) for x in xs)[math.ceil(q * len(xs)) - 1]
```

Repeat independent runs with matched workload traces and randomized execution order where feasible. Report sample sizes, run-level dispersion, raw samples and exclusions. Thousands of correlated token gaps from one run are not thousands of independent experiments. Block bootstrap or a hierarchical model can respect trace/run clustering, but assumptions and interval method must be explicit. The fixture computes only mean, sample standard deviation and range of matched-run speedup ratios; it supplies no confidence interval.

```
def paired_ratio(base, new):
    if len(base) != len(new) or len(base) < 2:
        raise ValueError("at least two matched runs")
    ratios = [speedup(a, b) for a, b in zip(base, new)]
    return dict(n=len(ratios), mean=statistics.mean(ratios),
                stdev=statistics.stdev(ratios), minimum=min(ratios),
                 ↪  maximum=max(ratios))
```

For paired durations $(100, 120, 80)$ and $(80, 100, 64)$ ms, ratios are $(1.25, 1.2, 1.25)$. Their mean is about 1.233 and sample deviation 0.0289. This is different from dividing arbitrary unpaired best runs. Repetitions that share a thermal drift or queue regime are not independent evidence.

47.5 Measure a critical path before summing durations

A latency decomposition such as queue + prefill + transfer + decode + tools + serialization is exact only for nonoverlapping serial phases covering the whole request. If retrieval overlaps prefill, summing both overcounts. Represent a dependency DAG with span duration d_i and predecessor set $pred(i)$; assuming sufficient resources and earliest-start scheduling,

$$E_i = d_i + \max_{j \in pred(i)} E_j, \qquad L = \max_i E_i,$$

where the maximum of an empty predecessor set is zero. Resource contention must be represented by additional edges or measured waiting spans.

```
def critical_path(spans):
    """DAG durations in ms; span=(name,duration,predecessor_names)."""
    end = {}
    for label, duration, predecessors in spans:
        if label in end or any(p not in end for p in predecessors):
            raise ValueError("unique topologically ordered DAG")
        end[label] = finite(duration) + max((end[p] for p in predecessors),
         ↪  default=0)
    if not end:
        raise ValueError("nonempty DAG")
    return max(end.values())
```

The exact fixture has queue 10 ms, then prefill 40 and retrieval 30 ms in parallel, followed by decode 20 ms waiting for both. The path is 70 ms, whereas summing all spans gives 100. Halving retrieval to 15 changes no critical-path latency. Halving prefill to 20 gives $10 + 30 + 20 = 60$ ms; retrieval becomes the limiting branch.

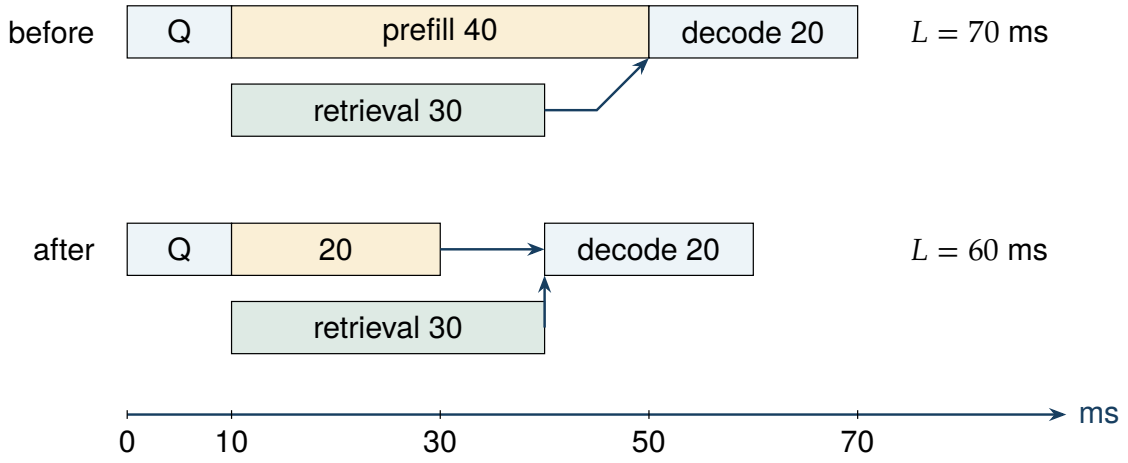


Figure 47.2: Parallel spans are not additive latency components. After shortening prefill, the retrieval branch becomes critical. Solid connectors denote dependencies; horizontal extent denotes assigned duration.

For a serial fraction f sped up by s, Amdahl's model gives

$$S_{total} = \frac{1}{(1-f) + f/s}.$$

Derive it by dividing original total time T into $(1-f)T$ unchanged and fT/s improved. With $f = 0.2, s = 2$, total speedup is $1/0.9 \approx 1.111$, even though the region doubled. It assumes fixed work and no new overhead, queue feedback or overlap changes; service optimization often violates those assumptions. Measure the resulting system, not only the local ratio.

47.6 Count arithmetic and traffic at a named boundary

For F operations and B bytes moved at a specified memory level, arithmetic intensity $I = F/B$ has units FLOP/byte. With compute ceiling P FLOP/s and bandwidth W bytes/s, a throughput ceiling is

$$P_{attainable} \le \min(P, WI), \qquad I^* = P/W.$$

Equivalently, ideal execution time is at least $\max(F/P, B/W)$ under these resource assumptions. Adding synchronization/launch overhead can only increase time. The Roofline

model provides useful ceilings [109]; peaks, measured sustainable bandwidth and actual traffic must not be interchanged.

```
def roofline_bound(flops, bytes_moved, peak_flops, bandwidth):
    return min(finite(peak_flops, True),
               finite(bandwidth, True) * arithmetic_intensity(flops, bytes_moved))
```

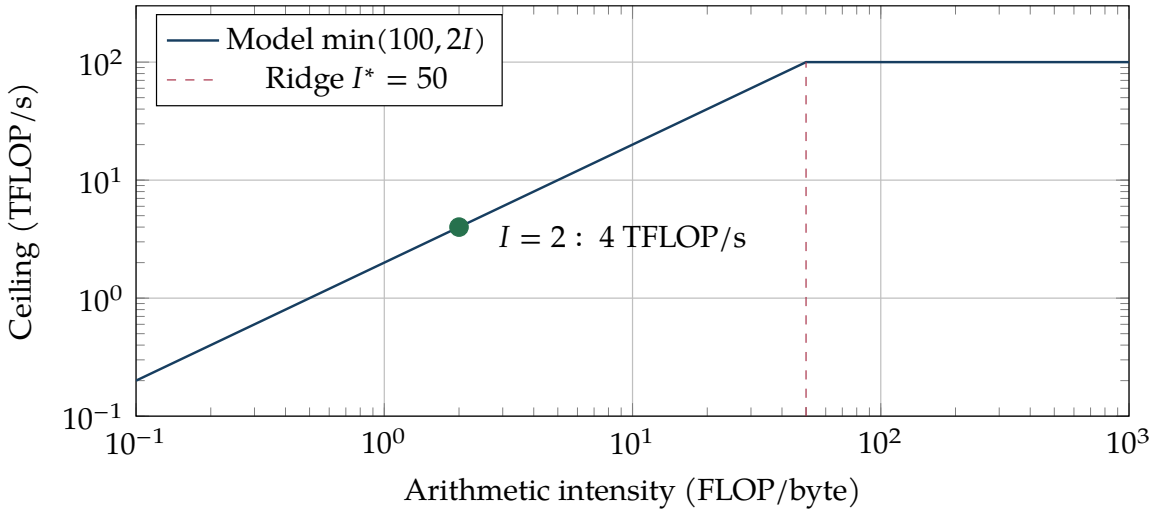


Figure 47.3: Assigned ceilings of 100 TFLOP/s and 2 TB/s decimal meet at 50 FLOP/byte. The point at intensity 2 has a 4 TFLOP/s ceiling. These curves are a model, not a device profile.

The same operation can have different intensity at register, shared/on-chip, last-level-cache and device-memory boundaries. Tensor element counts alone do not reveal all device traffic. Include output writes, staging, spills, alignment, metadata and parameter reads where the scope requires them. Comparing useful operations to total physical bytes can differ sharply from comparing nominal dense operations to ideal tensor payloads.

47.6.1 Attention and native examples without a false speed ranking

For one decode query per layer/head, QK and AV interactions cost approximately $4LHtd_h$ multiply/add FLOPs, excluding softmax, projections and other layers. Reading K/V once costs $2LHtd_hb$ logical bytes. Their ratio is $2/b$ FLOP/byte. At two-byte elements, it is one, independent of retained length; total work and bytes still grow with t. Batch reuse, GQA, cache layout and kernel fusion change the traffic. FlashAttention's IO-aware tiling is a relevant mechanism comparison [83]; it does not make this CPU oracle an optimized kernel.

Chapter 34's actual restricted recurrence counts $2BDV + 4BD$ multiply/add operations, sigmoid evaluations separately, and resident payload $B(3D + V)b$. Gather/scatter adds $4BDb$. With batch $B = 8$, width $D = 64$, vocabulary $V = 4$ and element width $b = 8$, the generated values are:

Region	Modeled ops	Logical bytes	Ops/byte
Decode KV	402653184	402653184	1
Native resident	6144	12544	0.489796
Native staged	6144	28928	0.212389

The attention row uses Chapter 35's large storage geometry, not the small two-layer executed fixture. The native operation count excludes sigmoid and shared parameter traffic; it is not a complete FLOP count. These rows cannot rank capabilities, latency or energy. They expose declared accounting scopes.

47.7 Profile top down with tools appropriate to the backend

First measure offered-load SLOs and request phases. Then inspect system timeline gaps, transfers, allocator stalls and synchronization. Only after locating a critical region inspect operators and kernel counters. PyTorch Profiler can collect operator activity, shapes/stacks and device activity where supported [110]. Detailed collection itself adds overhead; confirm final performance with an appropriately uninstrumented protocol.

Nsight Systems supports CPU/GPU system tracing and focused regions; Nsight Compute analyzes CUDA kernels [111, 112]. Use phase annotations to connect prefill, decode, native steps, memory and transfer to a request trace. Counter collection/replay can perturb cache state and timing. Architecture-specific occupancy is evidence to interpret, not a universal objective to maximize.

Backend	Record explicitly	Investigate on that platform
CPU	ISA, core/NUMA topology, affinity, threads	Vectorization, caches, memory bandwidth, dispatch
Metal	SoC, OS/framework, precision, memory pressure	Native GPU capture/counters, scheduling, bandwidth
CUDA	GPU, driver/toolkit, clocks/power, sharing	System timeline, kernel metrics, interconnect
Vulkan	Device/driver, shader compiler, subgroup features	Barriers, timestamp support, layout, vendor counters

These are experiment fields, not measurements. Apple's Metal tooling supports platform GPU performance investigation [113]; unified memory does not remove bandwidth/capacity limits or synchronization. Vulkan exposes explicit synchronization responsibilities [114]; portability of an API does not imply portable performance. Do not mechanically translate CUDA counters into Metal or Vulkan terms. Unsupported operators, precision and fallbacks must be disclosed before comparing backends.

47.8 Time asynchronous work at an intentional boundary

Host timing around an enqueue can measure submission rather than completion. Device events or a synchronized wall-clock boundary are needed for completed work, with exact scope documented. Synchronizing every operation can destroy the overlap the service normally uses. Compare equivalent timing boundaries and include required output materialization. CPU/GPU clock domains and profiler timestamps are not automatically interchangeable.

Batching may improve device occupancy but increase queue delay and token gaps. Sweep concurrency and arrival rate, report occupancy distributions, and distinguish useful accepted tokens from padded/cancelled work. For distributed placement, measure queue, serialization, network, reconstruction and commit with overlap and clock error stated. Chapter 46's bandwidth model is a hypothesis to calibrate against real transfer traces, not a measured input.

47.9 An optimization loop that can reject its own hypothesis

Freeze baseline and workload; reproduce correctness; locate the critical region; state one mechanism and predicted signature; change it; recheck correctness; remeasure paired runs. For example, fusion predicts fewer intermediate bytes and launches. It may fail because register pressure or specialization overhead grows. Archive that failure, raw traces and all configuration changes. A speedup without the predicted signature invites a different causal explanation such as changed workload or warmup.

The reference tests exact synthetic outcomes, percentile permutation invariance, invalid clocks, missing-token outcomes, maximum-gap versus mean SLOs, a throughput/goodput reversal, DAG overlap, roofline units, Amdahl and the earlier chapters' declared traffic arithmetic. It launches no hardware backend. Backend quality/performance experiments against completed Chapters 34/38/46 remain integration work; synthetic success is never their substitute.

47.10 Terms and the application boundary

Goodput counts successful work satisfying a declared service objective. **Critical path** is the longest causal duration under stated resource assumptions. **Arithmetic intensity** names both work and a memory boundary. The industrial contracts now define what a service would have to preserve and measure. They do not establish production readiness. Chapter 48 returns to genomic applications, where task-specific biological evidence remains a separate requirement from a correct or fast runtime.

Key ideas

- A failed request with no output has no TTFT sample. Dropping it from the denominator creates survivor bias.
- A maximum-gap SLO is stricter than mean TPOT: gaps $(1, 99)$ average 50 but fail a 60 ms bound.
- Parallel spans are not additive. Halving the off-path retrieval changes nothing; halving prefill makes retrieval critical.
- One-query decode attention has intensity about $2/b$ FLOP/byte whatever the context length.

47.11 Twelve worked exercises

1. **Fastest kernel, fastest service?** *Solution:* only if its critical contribution and system effects support that conclusion. The parallel retrieval example changes no latency when shortened alone.
2. **Compute TTFT and ITL.** Arrival 10, first 70, then 90 and 110 ms. *Solution:* TTFT 60 ms; gaps 20,20 ms. First token is counted once.
3. **Throughput versus goodput.** Why retain failed requests? *Solution:* omitting them inflates attainment. Goodput counts successful SLO-qualified requests per common observation time; raw tokens can include partial failed responses if explicitly reported.
4. **Calculate a ceiling.** $I = 2$ FLOP/B and $W = 2$ TB/s. *Solution:* bandwidth ceiling 4 TFLOP/s decimal; take the minimum with compute peak. It is not achieved throughput.
5. **Name the byte boundary.** Is tensor size equal to DRAM traffic? *Solution:* no; caches, rereads, writes, spills, staging and metadata alter it.

6. **Warmup honestly.** Remove the slowest first ten samples after seeing results? *Solution:* not as a predeclared steady-state protocol. State and archive the rule before collecting comparison evidence.
7. **Trace before counters.** What might dominate besides kernels? *Solution:* queueing, copies, launch gaps, allocator waits and synchronization. Use the critical path to choose detailed profiling scope.
8. **Quote CUDA peak.** Does it establish application performance? *Solution:* no; collect workload timings and relevant achieved counters, with exact precision, sharing mode, versions and clocks.
9. **Unified memory.** Does Metal avoid all movement cost? *Solution:* no; bandwidth, capacity, locality and synchronization still matter. Use native evidence rather than a CUDA analogy.
10. **Speedup arithmetic.** 20 to 10 ms is 2×; 100 to 80 is 1.25× and 20% reduction. *Solution:* divide old time by new for speedup, divide saved time by old time for fractional reduction.
11. **Synthetic versus measured.** May the 44 tokens/s trace be labeled a GPU benchmark? *Solution:* no; all timestamps were assigned. A hardware claim needs archived real samples and the complete measurement contract.
12. **Design a backend comparison.** *Rubric:* fix model/tolerance, workload/arrivals and versions/hardware; separate warmup; time completed work; archive repeated raw outcomes, tails/goodput/memory, explanatory profiles and uncertainty. Disclose unsupported paths and failed changes.

Evidence scope

Access checked 17 October 2026. An earlier Roofline URL was inaccessible; the authors' Berkeley 2008 technical-report version was accessed instead, especially Section 3's memory-boundary definition and ceiling formula. DistServe's abstract/introduction and selected phase/placement passages were inspected. FlashAttention access was abstract-only. Official profiler guides were inspected at overview/API and relevant overhead/synchronization passages; no profiler was run. Apple's tools overview and Khronos's synchronization overview support narrow platform distinctions, not backend performance rankings. All chapter traces, ceilings and operation/byte rows are assigned or analytic. No real-backend result is reported. Original analysis/tests and figures are not copied empirical results or independent validation.

PART XI

Applications and capability research

What evidence would show that these systems are useful for genomics, persistent agents, continual learning or general capability?

The application chapters turn capability language into experimental designs. Genomic tasks need leakage-resistant splits, strand-correct labels and calibrated metrics, and they need biological validation when mechanisms are claimed. Persistent agents need event logs, intent-before-dispatch recovery and reconciliation. Continual learning needs a full acquisition–retention matrix. Claims about general intelligence need held-out task families, budget axes and an honest statement of what finite tests cannot prove.

CHAPTERS IN THIS PART

48 **Genomic sequence applications** Evaluate genomic tasks with leakage-resistant biological controls.

49 **Streaming, language, code and persistent agents** Test persistent streaming/agent tasks without capability inflation.

50 **Continual learning and population adaptation** Measure transfer, forgetting and population adaptation.

51 **AGI capability hypotheses and limits** Form falsifiable capability hypotheses and document their limits.

CHAPTER 48

Genomic sequence applications

In this chapter you will

- turn a biological question into an observation contract: inputs, target, sampling unit and metric
- close leakage relations (overlap and declared homology) transitively before splitting
- transform sequences, coordinates and labels consistently under reverse complementation
- report discrimination and probability metrics with declared tie and undefined-value policies, and keep variant scores separate from mechanisms

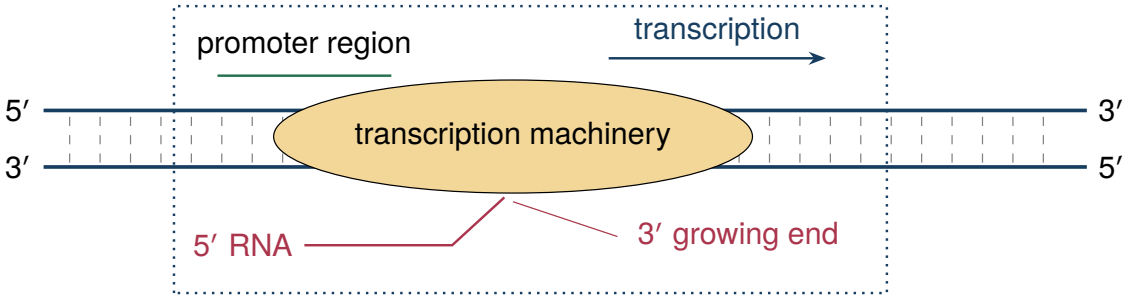
Two windows from the same locus can look like two examples while carrying almost the same evidence. Suppose window A covers bases $[0, 100)$ and B covers $[80, 180)$. A random split can train on A and test on B, sharing twenty bases and possibly the very motif that defines the label. A larger model cannot repair the resulting scientific question. We begin with the unit of generalization, then make sequence orientation, metrics and interpretation equally explicit.

DNA is biological material; a recorded DNA sequence is symbolic data; a predictor operating on that sequence is a computational hypothesis. These three statements do not collapse into one another. The executable work here checks a genomic evaluation harness. It trains no genomic model and reports no measured molecular effect.

48.1 From a biological question to an observation contract

Write the task as (X, Y, U, M): permitted sequence and metadata X, target Y, sampling unit U, and metric M. A promoter classifier predicts a window label; a splice-site segmenter predicts positions; a chromatin-profile model predicts assay tracks; a variant scorer compares two alleles in a declared context. They have different losses, resolutions and information boundaries. An offline chromatin predictor may use both flanks. A causal next-base predictor may not inspect its target through an overlapping token.

For a promoter application, specify the organism, assembly, cell condition, assay, positive-label rule and negative-sampling process. Promoters are DNA regions at which transcription machinery associates to initiate transcription [115]. Sequence is only one part of that physical process. The diagram distinguishes base pairing, protein association and transcription; none of its arrows is an edge in a neural network.



Recorded window X + assay-derived target Y: observation, not molecular simulation

Figure 48.1: A schematic transcription setting supplies the biological question. Solid paired rails denote DNA backbones, dashed rungs complementary bases, and the oval associated transcription machinery. The RNA product grows 5′ to 3′. Sequence-based classification observes a window and an assay-derived label; it does not simulate the machinery or establish its binding mechanism.

Each observation retains assembly, contig, zero-based half-open interval, orientation, assay/source identifier, preprocessing version and label provenance. The reference's `Example` carries the coordinate and grouping subset; an actual data manifest must supply the remaining provenance. Its interval length must equal its stored sequence length. Coordinates from two assemblies cannot be compared by merely matching their integers. A liftover is a separately versioned mapping with ambiguous and failed mappings reported.

48.2 Leakage is a relation, then a transitive closure

The generalization question decides which relationships must remain inside a split: overlap, locus, donor, homolog family, experiment or species. No split is universally correct. An unseen-chromosome test asks a different question from an unseen-person test on familiar loci. Both may still contain homologs of training sequence. Pretraining exposure must also be audited when a foundation model has already seen the reference genome.

Our small harness links two records when their same-assembly, same-contig intervals overlap, or when their nonempty declared homology IDs agree. Let $G = (V, E)$ be this undirected graph. Legal groups are its connected components. For half-open intervals, overlap means

$$\max(a_i, a_j) < \min(b_i, b_j).$$

Adjacent $[0, 4)$ and $[4, 8)$ windows do not overlap. If A overlaps B and B is homolog-linked to C, all three stay together even if A and C have no direct link. A pairwise check on only the initially selected test IDs misses this bridge.

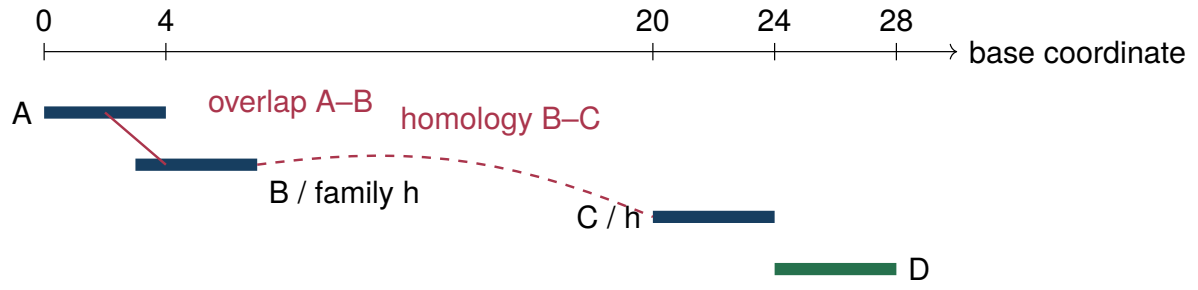


Figure 48.2: Edges encode prohibited split crossings. Holding out A also holds out B and C through transitive closure. D begins at C's end and has no homology link, so adjacency alone does not join it. The grouping policy is an input assumption, not a discovery of all possible biological dependence.

```
def group_split(examples, test_ids):
    """Selecting one ID holds out its entire connected component."""
    ex = tuple(examples)
    ids = set(test_ids)
    if ids - {x.eid for x in ex}:
        raise ValueError("unknown holdout id")
    held = {i for g in components(ex) if ids.intersection(g) for i in g}
    return tuple(x for x in ex if x.eid not in held), tuple(x for x in ex if x.eid
     ↪  in held)
```

Proposition 48.1 (No declared edge crosses the split)**.** *If each connected component is assigned wholly to one split, no edge in E has endpoints in different splits.*

Proof. The endpoints of an edge are connected by a path of length one and hence belong to the same component. Whole-component assignment gives them the same split. Transitive paths make the same argument apply to indirectly related records. □

This theorem covers the declared relation only. Undetected paralogs, misidentified donors or target-derived preprocessing can still leak. The reference uses an $O(n^2)$ pair scan to expose semantics; production-scale interval sweeps and homology clustering need their own parity tests.

48.3 Reverse complement acts on data and labels

For $\Sigma = \{A, C, G, T, N\}$, complement exchanges A/T and C/G and fixes N. For length L, $R(x)_i = c(x_{L-1-i})$. Applying reversal and complement twice restores every index and base, so $R^2 = I$. This is a representation symmetry, not an assertion that all assays are orientation invariant.

For a scalar strand-invariant score, a shared two-pass construction is

$$f_{\mathrm{sym}}(x) = \tfrac{1}{2}[g(x) + g(Rx)].$$

Then $f_{\mathrm{sym}}(Rx) = \frac{1}{2}[g(Rx) + g(R^2x)] = f_{\mathrm{sym}}(x)$. The equality assumes exactly the same deterministic g and preprocessing in both passes. A strand-specific label instead requires its

declared label action, potentially reversing positions and swapping strand channels. Pooling it to one invariant scalar can erase the required distinction.

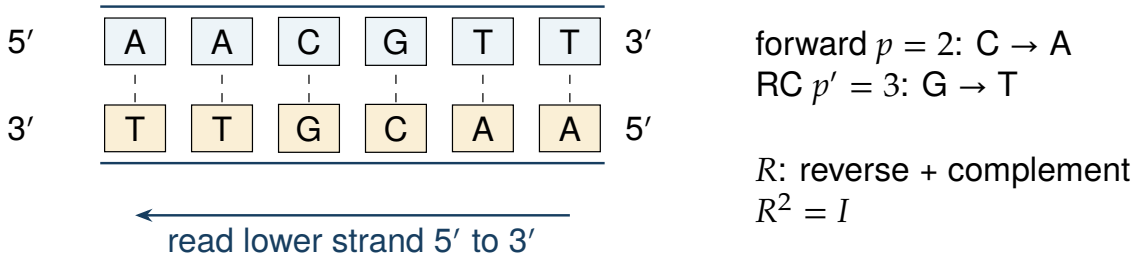


Figure 48.3: Both strings describe the same antiparallel duplex, each read 5′ to 3′. A substitution at forward index p moves to $L - 1 - p$ with complemented alleles. Reversing positions without complementing bases produces the wrong variant. Unknown N is an observation symbol, not a fifth chemical base.

CG is a tempting test motif, but it is its own reverse complement, so an invariance test built on it can pass without exercising two distinct motif orientations. The motif-count example therefore uses ACG and its reverse complement CGT. The assigned score for AACGTT is one, and the C-to-A change at index two removes both contributions, giving $\Delta = -1$. This is a detector calculation, not a learned affinity or expression change.

The reference rejects characters outside ACGTN, treats N as an all-zero one-hot row with a separate validity flag, and returns an undefined GC fraction for an all-N record. Silently treating N as A would fabricate a base. Chapter 23 accepts canonical ACGT; the local integration test uses that exact interface and its offline shared-RC readout, with two untrained layers, learned absolute positions and CPU float64. It proves neither learned invariance nor accuracy.

48.4 Resolution, context and the baseline ladder

Base tokens preserve nucleotide resolution. Fixed k-mers change the sequence length and perturbation footprint: one substituted base can affect several overlapping tokens. Chapter 24's span availability condition remains necessary even under triangular token attention. A context measured in tokens must also be reported in bases, with stride, padding and uncovered tails declared.

Nucleotide Transformer evaluates pretrained Transformer representations on genomic tasks; its original tokenization uses six-base units [54]. HyenaDNA studies single-nucleotide modeling with contexts up to one million tokens [15]. Caduceus constructs bidirectional RC-equivariant MambaDNA components [48]. These are distinct prior approaches, not interchangeable instances of DOGMA. The 2025 DNA-model benchmark evaluates frozen foundation representations with downstream classifiers and finds task-dependent comparisons with specialized models [116]. Frozen representations versus task-trained competitors is an adaptation condition that must accompany the conclusion; it is not a universal ranking of all fine-tuning procedures.

Start with composition/Markov or k-mer counts, linear and MLP readouts, then local CNNs, GRUs, appropriate SSM/long-convolution models and Transformers. Choose meaningful controls rather than requiring every architecture on every task. Freeze splits, legal evidence, tuning trials, training tokens and primary metric. Report parameter, state, context and measured serving budgets separately. The actual Chapter 31 recurrence is a restricted teaching model, not full DOGMA. Hermon DNA is Transformer-based; DOGMA is the non-Transformer direction.

A useful long-context ablation retains the central window while masking or shuffling distal context under a declared nuisance-matching rule. Compare several context lengths at matched training and tuning budgets. If only same-locus random splits benefit, leakage is a competing explanation. If a long context has no task information, success at storing it is not evidence of regulatory understanding.

48.5 Derive discrimination and probability checks

For binary predictions, retain TP, TN, FP, FN before computing any scalar. An all-negative classifier at 1% prevalence has 99% accuracy and zero recall. The Matthews correlation coefficient is

$$\mathrm{MCC} = \frac{TP\,TN - FP\,FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}.$$

A zero denominator makes this expression undefined. Our harness reports `None`; a benchmark choosing zero must disclose that convention. The Brier score for probabilities p_i is $n^{-1}\sum_i (p_i - y_i)^2$. It measures squared probability error; a reliability curve by score bins adds diagnostic information that one scalar cannot show.

Average precision (AP) is one specified summary of a precision–recall curve, not an unspecified trapezoidal AUPRC. At distinct score thresholds k, let r_k be recall and q_k precision after all tied examples enter. Define

$$\mathrm{AP} = \sum_k (r_k - r_{k-1}) q_k. \tag{48.1}$$

For scores $(.9, .8, .7)$ and labels $(1, 0, 1)$ the positive increments are $1/2$ at precision one and $1/2$ at precision $2/3$: AP is $5/6$. An all-tied predictor has AP equal to prevalence. Reordering tied rows cannot improve it. With no positives AP is undefined under this contract.

```
def average_precision(y, score):
    """Threshold-group AP: all equal scores enter together; no tie-order bonus."""
    y, score = paired(y, score, True)
    positives = sum(y)
    if not positives:
        return None
    tp = seen = 0
    ap = 0.0
    for threshold in sorted(set(score), reverse=True):
        indices = [i for i, s in enumerate(score) if s == threshold]
        new = sum(y[i] for i in indices)
        tp += new
        seen += len(indices)
        ap += (new/positives)*(tp/seen)
    return ap
```

Synthetic statistic	Value
Accuracy	0.5
MCC	0
Brier	0.1925
AP	0.833333

The generated table uses labels $(1,1,0,0)$, predictions $(1,0,1,0)$ and probabilities $(.8,.4,.6,.1)$. Accuracy is $1/2$, MCC zero, Brier .1925, and AP $5/6$. These values validate arithmetic; they are not dataset results.

48.6 Uncertainty follows the independent unit

Holding trained predictors fixed, a cluster bootstrap resamples biological groups with replacement and includes all windows of every sampled group. If groups are independent draws from the target population, this approximates sampling variation at that level. It does not include training-seed variation, unseen assays or uncertainty about the grouping relation.

```
def cluster_interval(y, p, groups, draws=1000, seed=7):
    """Percentile interval for fixed-predictor row accuracy under cluster
     ↪  resampling."""
    y, p = paired(y, p)
    groups = tuple(groups)
    if len(groups) != len(y) or type(draws) is not int or draws < 20:
        raise ValueError("aligned groups and at least 20 draws")
    units = sorted(set(groups))
    if len(units) < 2:
        raise ValueError("at least two independent units required")
    rows = {g: [i for i, h in enumerate(groups) if h == g] for g in units}
    rng = random.Random(seed)
    values = []
    for _ in range(draws):
        idx = [i for _ in units for i in rows[rng.choice(units)]]
        values.append(sum(y[i] == p[i] for i in idx)/len(idx))
    values.sort()
    return values[int(.025*(draws-1))], values[int(.975*(draws-1))]
```

The reported statistic is row-weighted accuracy on each resampled collection; large groups receive more weight. Equal-group performance would first average within groups and then across groups, a different estimand. Percentile endpoints use the explicitly coded order-statistic indices. Two clusters are enough to exercise the implementation and far too few for a convincing scientific interval. Compare model differences on paired resamples, preserve seed-level results, and stratify by distance, GC, ambiguous-base rate and label source.

48.7 Variant scores and mechanistic claims

For a normalized variant and fixed context, define $\Delta = s(x_{\mathrm{alt}}) - s(x_{\mathrm{ref}})$. The sign convention, score scale, reference allele, assembly and context must be fixed before interpretation. The local oracle implements substitutions only; indels need an explicit alignment and output-coordinate policy.

```
def snv(seq, pos, ref, alt):
    seq, ref, alt = dna(seq), dna(ref), dna(alt)
    if (type(pos) is not int or not 0 <= pos < len(seq) or len(ref) != 1
            or len(alt) != 1 or ref == alt or "N" in ref+alt or seq[pos] != ref):
        raise ValueError("normalized matching single nucleotide substitution")
    return seq[:pos]+alt+seq[pos+1:]
```

Saliency, attention and in-silico mutation measure sensitivity of a predictor. Controls include label shuffling, matched GC negatives, motif insertion/deletion, RC consistency and perturbations outside the tested footprint. A motif effect in a synthetic detector is a useful positive control for the harness. It does not show that a protein binds that sequence in a cell.

A biological follow-up protocol states the hypothesized mechanism, intervention, assay, cell condition, replication unit and alternative explanations. Reference and alternate constructs need matched backgrounds, assay controls and an analysis plan. No wet-lab experiment is conducted here. Computational prioritization is a handoff to such evidence, not a replacement for it.

48.8 Selective glossary

Sampling unit. The biological entity treated as independent for the generalization and uncertainty claim; it need not equal one sequence window.

Split closure. Connected-component grouping under all declared relations that must not cross train/evaluation boundaries.

RC action. Reversal plus base complement, accompanied by the appropriate coordinate and label transformation.

Average precision. Recall-increment-weighted precision with a declared tie policy; different from an unspecified trapezoidal PR area.

Variant score. A model-score contrast between normalized reference and alternate sequence, with no automatic causal or clinical interpretation.

Key ideas

- Windows $[0, 100)$ and $[80, 180)$ share evidence. A random window split tests a partly seen locus, not an unseen one.
- CG is its own reverse complement, so it cannot test two orientations. Use ACG and CGT.
- At 1% prevalence, an all-negative classifier has 99% accuracy, zero recall and undefined MCC.
- Saliency and in-silico mutation measure a predictor's sensitivity. Showing that a protein binds a sequence in a cell needs a designed experiment, which this book does not perform.

48.9 Exercises and worked solutions

1. Why does the opening random split change the question? **Answer:** it tests a partly observed locus rather than an independently unseen locus.

2. Do adjacent half-open intervals overlap? **Answer:** no; their intersection has zero length and the strict inequality fails.

3. A overlaps B; B and C share a family. Hold out A. **Answer:** hold out A, B and C by connected-component closure.

4. Does the closure theorem prove all homology is removed? **Answer:** only links supplied by the audited relation; undetected homology remains possible.

5. Compute $R(\text{AACGTN})$. **Answer:** NACGTT.

6. Prove $R^2 = I$. **Answer:** each index reverses twice and each base is complemented twice.
7. Why is CG a weak two-orientation test motif? **Answer:** its RC is CG, so missing the second orientation can be hidden.
8. Transform a substitution at $p = 2$ in a six-base record. **Answer:** index three, with both reference and alternate complemented.
9. Why keep a validity mask for N? **Answer:** zero feature mass must not be confused with observed adenine or an actual chemical fifth base.
10. Must a splice track be invariant? **Answer:** no; positional and strand channels can require an equivariant label transformation.
11. A 1% positive task, all-negative predictor? **Answer:** 99% accuracy, zero recall, undefined MCC under our denominator convention.
12. Compute AP for $(1, 0, 1)$ sorted by descending score. **Answer:** $\frac{1}{2} + \frac{1}{2}(2/3) = 5/6$.
13. What is all-tied AP at 20% prevalence? **Answer:** .2; breaking ties by labels would leak evaluation information.
14. Brier for labels $(1, 0)$ and probabilities $(.8, .2)$? **Answer:** $(.04 + .04)/2 = .04$.
15. Does resampling windows estimate donor uncertainty? **Answer:** not when windows from one donor are dependent; resample the donor groups.
16. Why distinguish frozen embeddings and fine-tuning? **Answer:** they permit different adaptation and compute, changing the comparison.
17. Is the tested Hermon path a newly trained genomics model? **Answer:** no; it is the actual untrained Chapter 23 interface checked for algebraic symmetry.
18. Design a distal-context ablation. **Rubric:** central evidence fixed, declared distal perturbation, nuisance matching, legal split, matched opportunity, paired metrics, seed/cluster uncertainty and retained negative results.
19. What could falsify the motif explanation? **Answer:** gain survives motif deletion but vanishes with a GC-matched nuisance intervention, motivating a competing shortcut explanation.
20. Propose a biological validation handoff. **Rubric:** explicit allele and cell context, causal intervention, assay controls, independent replication, predeclared analysis and bounded interpretation; no invented measurements.

48.10 Bridge

Chapter 49 changes domains but preserves the central discipline: record what was observed, retain the identity of carried state, and declare which evidence may influence the next decision. A persistent service needs those contracts even when its inputs are language and code rather than genomic bases.

CHAPTER 49

Streaming, language, code and persistent agents

In this chapter you will

- order events within a session and bound queues with visible rejection
- keep native state, active context, durable memory and external state distinct
- bind replay to identity and contract with a prefix digest
- run a recovery state machine that records intent before dispatch and reconciles ambiguous crashes

A code agent submits a tool action, the process crashes, and no acknowledgement was recorded. Should restart submit the action again? The answer depends on what the external service did, not on how confident the model sounds. This small ambiguity connects streaming, memory, authority and persistence more directly than a large prompt does.

We build an event/session example in which tool calls are replayable intents and results rather than an in-memory cache. The result is a serialized recovery oracle: it emits instructions to dispatch or reconcile, but executes no tools, stores no durable log and implements no production concurrency. Language and code are application workloads requiring their own trained models and evaluators. Neither the ACGT Transformer nor the restricted native recurrence becomes a language agent by being placed behind this shell.

49.1 Events, ordering and bounded pressure

An event is (i, k, p): a sequence number i, a kind k and a typed payload p. Within one session the log orders accepted observations and decisions. It does not recover the true wall-clock order of all outside events. If two producers race, a sequencer or transactional append rule must decide the order. The reference has one serialized caller, so i is exactly the previous length.

User input, proposals and emitted output are observations. Tool intents and results change a recovery state machine. Closure is a terminal event. Large artifacts belong behind immutable handles with content identity and access checks; copying binaries into every event multiplies retention and privacy cost. Our small payloads are canonical JSON strings. Freezing only a dataclass while retaining a mutable dictionary would still permit old history to change through an alias.

Let a_t be arrivals, d_t departures and q_t queue occupancy after service at tick t. For a capacity K, accepted arrivals satisfy

$$q_{t+1} = q_t + a_t - d_t, \qquad 0 \le q_{t+1} \le K.$$

If an arrival would violate K, the protocol must block, reject, persist to another bounded tier or shed under a declared policy. Changing K cannot solve a sustained arrival rate above service rate. Dropping a tool completion is also different from dropping a best-effort telemetry sample.

```
class Stream:
    def __init__(self, capacity):
        if type(capacity) is not int or capacity < 1:
            raise ValueError("positive queue capacity")
        self.capacity, self.q = capacity, []

    def push(self, item):
        if len(self.q) == self.capacity:
            raise BufferError("backpressure")
        self.q.append(item)

    def pop(self):
        return self.q.pop(0) if self.q else None
```

The generated FIFO test pushes two items, rejects the third, then returns the first two unchanged. Its conservation check concerns accepted items only; the rejected producer remains responsible for the item. Capacity here counts items, not bytes; large variable payloads require a byte budget too. The reference retains its event list in memory without compaction. It establishes neither bounded total-history storage nor durable retention.

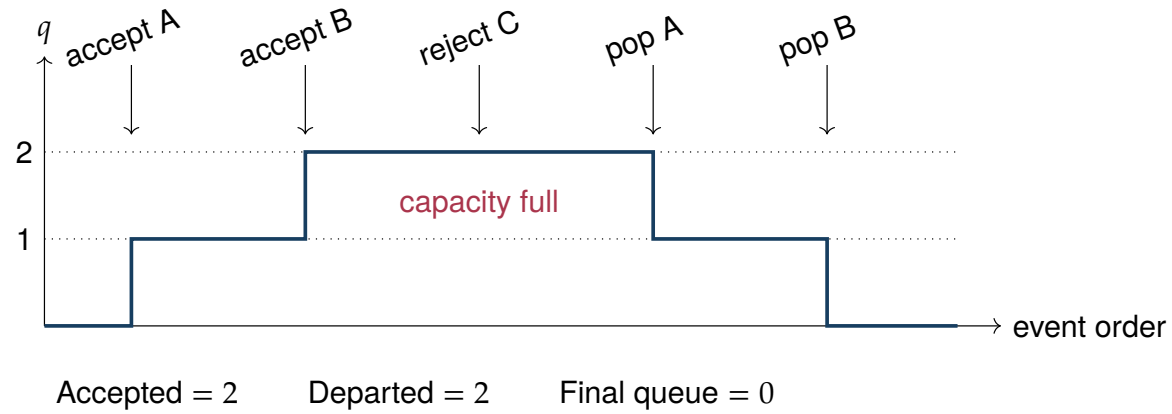


Figure 49.1: The upper trace shows bounded queue occupancy; the third arrival is rejected while two credits are occupied. The lower trace shows the two accepted items leaving in FIFO order. Rejection transfers no ownership and must be visible to the producer.

49.2 Four states that cannot substitute for one another

Keep engine-native state, active model context, durable addressable memory and authoritative external state separate. DOGMA-style native state is a model-specific compressed representation. Hermon's per-layer K/V retains addressable attention history. Active context is the input selected for a model call. Durable memory is a versioned record store. Files, databases and tool-side objects are authoritative external state.

MemGPT provides prior work on managing a bounded context through memory tiers and function calls [93]. The applicable systems lesson is explicit movement between tiers. A summary remains a lossy view of source events. It cannot silently become an instruction with greater authority than its source, and a deletion/retention policy must address summaries and indexes as well as original records.

The event log is not automatically an engine checkpoint. To resume Chapter 31, the native carry needs its artifact, owner and consumed-token position. For Chapter 35, the ordered K/V

at every layer needs its model identity and learned absolute positions. Their shapes and meanings differ. The local tests call the real stable interfaces and check whole-versus-chunk continuation on their actual vocabularies. No conversion between the two state types is invented.

49.3 Identity binds replay to one contract

A session identity contains tenant, session, artifact and policy version. The checkpoint also binds context limit, action budget and lease expiration. These latter fields are contract configuration, not text that the model may rewrite. Migration to a new model or policy requires an explicit transition and an appropriate state reset or proven migration.

For a log prefix of n events define

$$D_n = \mathrm{SHA256}(\mathrm{canonical}(I, C, E_{0:n})), \tag{49.1}$$

where I is identity and C the session contract. We use an event count n, so an empty prefix has $n = 0$. This avoids interpreting a negative array index as a checkpoint. Later append-only events leave D_n unchanged. Altering a prefix payload, artifact, policy or budget invalidates the checkpoint.

A digest detects inconsistency against a trusted recorded digest. An adversary able to replace both log and checkpoint can recompute both. Authentication, authorization, encryption, retention and crash-atomic storage remain separate requirements. A valid digest also does not prove that a recorded tool result is true; it only binds the bytes of that observation.

49.4 An action state machine for ambiguous crashes

For each operation key k, record a fingerprinted command and one of three states: absent, intent-only, or completed. The command includes operation name and canonical arguments. Reusing k with a different command is an error. The key's scope includes session/tenant identity; global raw keys would allow cross-session conflicts.

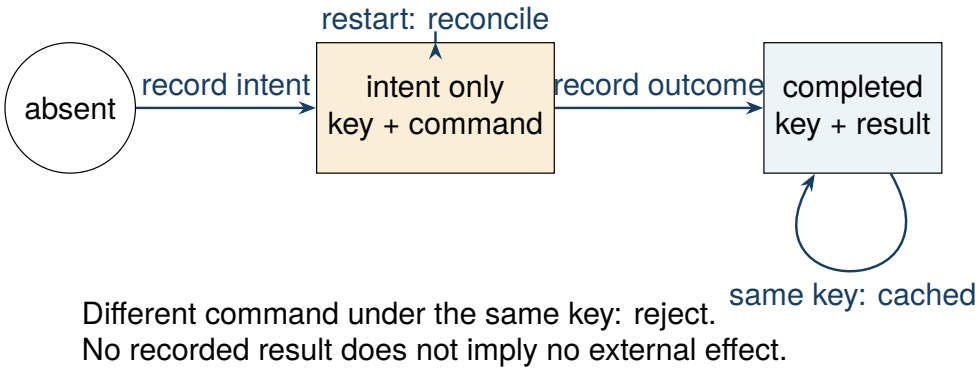


Figure 49.2: The same operation key can denote only one command. A new admission records intent before dispatch. A crash before a result leaves an unresolved intent, which requires reconciliation. Only a recorded completion allows returning a cached result. No transition assumes that a timeout means the outside effect did not occur.

```
    def issue(self, key, name, args, authorized, tick):
        """Reserve before dispatch; existing unresolved intent requests
         ↪  reconciliation."""
        if self.closed or authorized is not True:
            raise PermissionError("closed or unauthorized")
        if type(tick) is not int or not 0 <= tick < self.expires:
            raise PermissionError("lease expired")
        command = json.loads(canonical({"name": name, "args": args}))
        if key in self._intents:
            if self._intents[key] != command:
                raise ValueError("idempotency key reused for different command")
            if key in self._results:
                return "cached", json.loads(canonical(self._results[key]))
            return "reconcile", None
        self._record("intent", {"key": key, "command": command, "tick": tick})
        return "dispatch", None
```

The policy decision is supplied by trusted application code, never extracted from a proposal string. The reference requires the literal Boolean true, rejects an expired lease, then reserves one action budget unit before dispatch. Repeated lookups do not reserve another unit. Admission counts distinct intents, including failed or unresolved actions; refund policies would be a separate contract. An action admitted before expiration may complete afterward, so its observation must still be recorded. Expiration blocks new dispatch, not truth. Ticks and authorization decisions are trusted caller inputs in this oracle; a host must supply a trusted clock and policy service. The code does not read an external clock or protect itself against a privileged caller forging history.

```
    def complete(self, key, value):
        # Observation of a previously admitted action may arrive after lease
         ↪  expiry.
        value = json.loads(canonical(value))
        if key in self._results:
            if self._results[key] != value:
                raise ValueError("conflicting duplicate completion")
            return
        self._record("result", {"key": key, "value": value})
```

Duplicate identical completion is harmless; a different value for the same key is a conflict. Closure requires every intent to have a result in this narrow reference. A real service might instead close with explicit unresolved/cancelled states and a separate reconciliation owner. It cannot silently forget them.

Proposition 49.1 (At most one local dispatch instruction per key)**.** *In one serialized valid session history, the reference returns* `dispatch` *for a key at most once.*

Proof. The first dispatch records an intent for that key before returning. Every later issue either rejects changed arguments, returns the recorded result, or returns reconciliation. None of those paths dispatches. Resume reconstructs the same intent set from the log before allowing further issue calls. □

The theorem is about local instructions. It is not exactly-once external execution. A dispatcher can fail between accepting the instruction and sending it, or an external tool can execute before

the result is logged. A tool protocol with durable idempotency may permit a same-key retry; without it, query the authoritative state or require reconciliation by an operator. No prompt can supply missing external transactional semantics.

State	Next instruction
First admission	dispatch
Resume unresolved	reconcile
Observed completion	cached

This generated trace has one admitted read, restart with no result, then a recorded result. It is a constructed event sequence, not a failure-injected distributed-service measurement.

49.5 Replay is reduction, not re-execution

```
def resume(identity, events, cp, max_context=4, budget=3, expires=100):
    session = Session(identity, max_context, budget, expires)
    for event in tuple(events):
        session._apply(event)
    if not session.verify(cp):
        raise ValueError("checkpoint identity/prefix/contract mismatch")
    return session
```

Replay applies validated event transitions to fresh derived state and verifies the checkpoint prefix. The full provided suffix is also validated. It does not call a model, rerun a shell command or resend an action. Its event schemas reject out-of-order sequence numbers, result-before-intent, duplicate intents, and events after closure. The log is trusted input for authorization history: replay validation establishes internal consistency, not that its original producer had permission.

Use induction to justify reconstruction. The empty log yields empty intent and result maps. A valid observation leaves them unchanged. A valid intent adds exactly the recorded key/command; a valid result adds its unique value. Thus after every prefix the replay maps equal the maps obtained by original serialized application of that prefix. This proves replay consistency under deterministic reduction, not disk durability or linearizability.

Model replay is a different issue. Reproducing sampled tokens additionally requires the artifact, sampler, RNG state, tokenizer and numerical environment. For audit, recording emitted tokens and tool observations is usually stronger than assuming a future call will regenerate them exactly. Once output has been emitted, a rollback cannot make a recipient unsee it.

49.6 Language and code workloads need task evidence

For language streams, measure turn completion, contextual consistency, retrieval quality, latency and cancellation under bounded context. Retain authoritative user constraints outside an evictable summary. Test whether an old correction survives summarization, whether a deleted source is still retrieved, and whether ambiguous references produce calibrated deferral. These are evaluation proposals, not capabilities demonstrated by our event log.

For a coding task bind repository revision, working tree identity, task, dependency lockfiles, environment and allowed operations. A patch candidate is meaningful only relative to that

base. Inspection, editing, static checks, targeted tests and diff review provide different evidence. A passed unit test does not certify security, compatibility or every hidden requirement.

SWE-agent studies how an agent-computer interface changes software-engineering agent performance [117]. Our proposed interface uses typed operation names, structured arguments, immutable resource handles and explicit errors. It exposes useful observations but never grants authority merely because a tool name exists. Sandbox filesystem, network, process, credential and resource permissions are supplied by the host. Generated code and retrieved repository text are untrusted inputs.

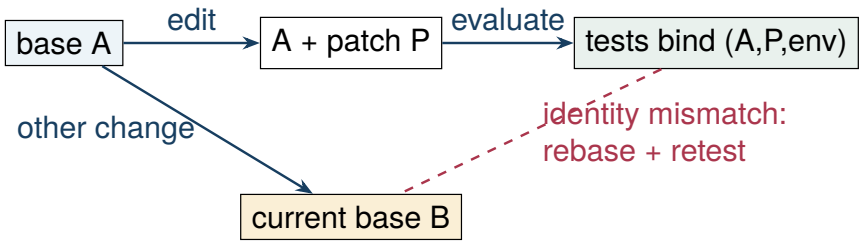


Figure 49.3: A patch is evaluated against a specific base revision. If the base changes during evaluation, the tested candidate and promotion target no longer agree. The conflict path requires rebase/revalidation rather than reusing the earlier test result. This is a version contract, not an implementation of repository transactions or a sandbox.

49.7 Plans, leases, humans and observability

A long-horizon plan is a versioned hypothesis with preconditions and completion evidence. Re-read mutable external state before acting on stale assumptions. A lease specifies lifetime; budgets specify tokens, compute, storage and tools in their own units. An integer tool-count bound in our code is not monetary accounting. For concurrent work, reserve resources atomically before launch and reconcile completion charges rather than checking only after consumption.

If approval is required by the application policy, bind it to the concrete action, resource version, scope and expiry. A changed action invalidates the approval. A summary saying "approved" is not an approval record. Human intervention rate, success rate and violations should be reported separately; silently accepting more risk can increase apparent autonomy.

Trace session, plan, engine and tool spans with durations, queue pressure, retries, unresolved actions and budget use. Avoid secrets and unnecessary private content. Concurrent branches may share immutable sources but need a conflict rule for writes. Production adapters for the Chapter 39, 40 and 45 contracts are not implemented. This chapter does not assume those contracts compose automatically, and it claims no passing multi-tenant persistent-agent deployment.

49.8 Selective glossary

Intent. A recorded admission for one fingerprinted operation key, before its dispatch outcome is known.

Reconciliation. Determining an unresolved external outcome using its tool protocol or authoritative state rather than blindly repeating the effect.

Idempotency key. A scoped identity whose repeated use denotes the same command; safety depends on the external service honoring the protocol.

Replay. Deterministic reduction of recorded events into derived state, separate from re-execution of effects.

Lease. A bounded lifetime for authority, distinct from a count or cost budget; late observations may still require recording.

Key ideas

- A larger queue cannot fix a sustained arrival rate above the service rate; it only delays exhaustion.
- The event log is not an engine checkpoint. Native carry and per-layer K/V each need their own artifact-bound state.
- At most one local dispatch per key is not exactly-once external execution. A crash after intent requires reconciliation, not blind retry.
- Replay reduces recorded observations into derived state. It never re-runs tools.

49.9 Exercises and worked solutions

1. A producer sends three items into an empty capacity-two queue without service. **Answer:** two are accepted; the third is rejected and retained by the producer under this contract.
2. Derive queue conservation. **Answer:** add accepted arrivals and subtract departures; neither blocked arrivals nor retries count as new stored items.
3. Can a larger queue fix indefinite overload? **Answer:** no; it only delays exhaustion when arrival rate exceeds service rate.
4. Is K/V durable memory? **Answer:** it is artifact-specific native attention history, not a general authoritative record store.
5. What does an empty checkpoint count? **Answer:** zero events, with a digest over empty history and the bound identity/contract.
6. Append after a checkpoint: must it fail? **Answer:** no; the bound prefix remains unchanged and suffix transitions are validated.
7. Can replacing log and digest evade detection? **Answer:** yes; unauthenticated hashes do not establish origin or authority.
8. Why canonical strings rather than frozen dict fields? **Answer:** nested mutable dictionaries would preserve aliases into history.
9. Same key, changed path? **Answer:** reject the fingerprint conflict.
10. Crash after intent, before acknowledgement? **Answer:** reconcile the external outcome; do not assume absent acknowledgement means absent effect.
11. Does the dispatch theorem prove exactly-once effects? **Answer:** no; dispatch and external execution are different boundaries.

12. Does a cached lookup bypass current policy? **Answer:** not here; issue checks current authorization and lease before returning a result.
13. Why reserve before dispatch? **Answer:** post hoc accounting can overspend when multiple actions are in flight.
14. May an admitted result arrive after expiry? **Answer:** yes; recording its outcome is needed for reconciliation even when new dispatch is denied.
15. Replay calls the tool again? **Answer:** no; it reduces observations into derived maps and leaves unresolved intents explicit.
16. What does a source-less summary lose? **Answer:** provenance, fine detail and potentially valid deletion/authority interpretation.
17. A patch passed tests on revision A; deploy to changed B? **Answer:** reconcile the base and revalidate; the old evidence concerns a different object.
18. Design a crash test. **Rubric:** enumerate before-intent, after-intent, after-effect and after-result crashes; specify durable records, tool idempotency, reconciliation and duplicate-result expectations.
19. Design a language-memory test. **Rubric:** held-out sessions, provenance/deletion controls, fixed retrieval budget, paired summarization baseline, success/latency/deferral metrics and audited human intervention.
20. Why are the real-engine continuation tests limited? **Answer:** they preserve actual native semantics on existing vocabularies; they do not train language models, implement cross-engine conversion or validate a service.

49.10 Bridge

Persistent state changes what evidence is available at the next decision. Chapter 50 asks which changes are learning, how they interfere with old skills, and what tests an update must face before becoming an active artifact.

CHAPTER 50

Continual learning and population adaptation

In this chapter you will

- separate context, memory, parameter, structural and population updates
- derive acquisition, forgetting and an anchored compromise in a transparent quadratic
- read the full experience matrix with a declared forgetting convention and backward transfer
- count population-search opportunities and apply feasibility before Pareto selection

Train a two-coordinate learner toward $(1, 0)$, then toward $(0, 1)$. Should it finish at the new target, the midpoint, or somewhere else? Each answer optimizes a different objective. A memory store that preserves old examples, a regularizer that resists movement and a population search that chooses among learners introduce different costs and causal explanations. We derive those differences before using the word adaptation as evidence.

A quadratic learner and an exploit/mutate example form the numerical spine. We add an independent closed-form oracle, explicit retention conventions, replay gradients, feasibility constraints and counted search opportunities. All reported values are synthetic arithmetic. No DOGMA or Hermon training improvement follows from them.

50.1 Five adaptation layers and their owners

Context updates change current inputs. Durable memory updates change later retrieval opportunities. Parameter or adapter updates change the function. Structural changes alter its graph, routing or memory topology. Population updates change which candidates exist and are selected. A saved conversation changes neither weights nor architecture merely by being stored.

Let h_t be inference state, M_v a memory version, θ_k weights, G_s a structure and $\mathcal{P}_g$ a population. The indices name update events, not a universal ordering of time scales. A deployment should identify which versions a request sees. Changing θ while retaining an incompatible native carry or K/V state can silently change the request's meaning.

Chapter 13's search inherited masks with fresh weights; its alternative inheritance map was separately defined. The same discipline applies here. Copying weights alone differs from copying weights, optimizer moments, normalization statistics, data cursor and RNG. A structural mutation needs a semantic coordinate map, not merely tensors of matching length. The local reference is not connected to Chapter 43's governance registry; it neither self-modifies a runtime nor promotes an artifact.

50.2 A transparent acquisition–retention conflict

Let $\theta \in \mathbb{R}^2$ and task target $u \in \mathbb{R}^2$ be dimensionless. Define a task loss and a bounded teaching score

$$L_u(\theta) = \tfrac{1}{2}\|\theta - u\|_2^2, \qquad Q_u(\theta) = \max\{0, 1 - \|\theta - u\|_2^2/4\}.$$

Q is not classification accuracy; its scale is chosen to make retention visible. The reference function is still named `accuracy_matrix`; it returns this score, not accuracy. Clipping can hide differences between very poor candidates, so an actual study should retain the underlying loss as well.

Gradient descent with rate η gives $\theta_{k+1} = (1-\eta)\theta_k + \eta u$. Subtracting u and iterating yields

$$\theta_n - u = (1-\eta)^n(\theta_0 - u).$$

For $0 < \eta < 2$ the error contracts. Starting at zero, four steps at $\eta = 1/2$ toward $u_1 = (1, 0)$ give $\theta_A = (15/16, 0)$. Four more toward $u_2 = (0, 1)$ give

$$\theta_B = (15/256, 15/16).$$

The new task is almost fitted; the first coordinate has mostly been forgotten. This is the objective's behavior, not a software bug.

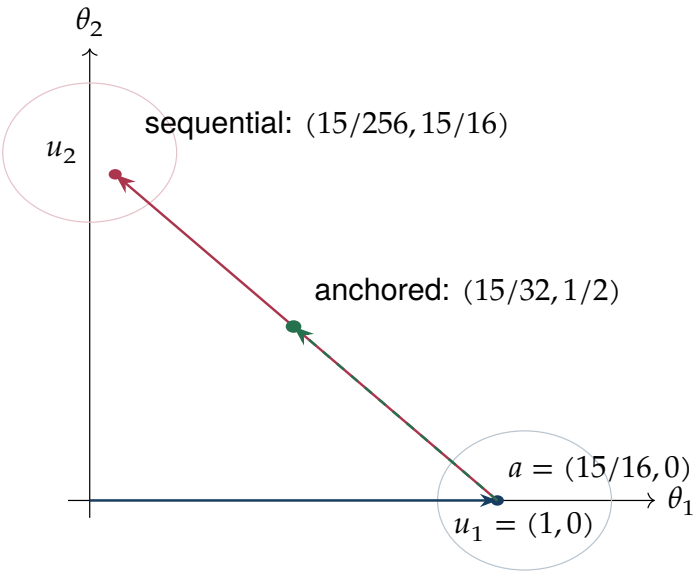


Figure 50.1: Parameter space contains two task optima. Four half-rate updates approach task one, then task two. An anchor changes the second objective and converges to a compromise. The path is a deterministic numerical fixture, not a trajectory measured in a trained neural network.

50.3 Derive an anchored objective and its stability bound

EWC uses a quadratic penalty weighted by parameter importance estimates [118]. In the usual approximation, a diagonal Fisher-information estimate represents local importance around a previous optimum. The original paper discusses approximations and uncertainty limits; the method does not guarantee retention on arbitrary new tasks. Our teaching fixture sets $F_i = 1$ explicitly. It does not estimate a Fisher matrix and is therefore an EWC-like anchored quadratic, not a reproduction of the complete method.

With anchor a and nonnegative weights F_i, let

$$L(\theta) = \tfrac{1}{2}\|\theta - u\|^2 + \tfrac{\lambda}{2}\sum_i F_i(\theta_i - a_i)^2, \qquad \lambda \geq 0.$$

Coordinate differentiation gives

$$g_i = (\theta_i - u_i) + \lambda F_i(\theta_i - a_i), \quad \theta_i^* = \frac{u_i + \lambda F_i a_i}{1 + \lambda F_i}.$$

Set $c_i = 1 + \lambda F_i$. Subtracting the optimum from one gradient step gives

$$\theta_{k+1,i} - \theta_i^* = (1 - \eta c_i)(\theta_{k,i} - \theta_i^*), \quad \theta_{n,i} = \theta_i^* + (1 - \eta c_i)^n(\theta_{0,i} - \theta_i^*). \tag{50.1}$$

Thus all coordinates contract when $0 < \eta < 2/\max_i c_i$. Stronger anchoring increases curvature and can require a smaller learning rate. An implementation can be arithmetically correct while its optimization diverges. The negative control takes $\eta = 3$ on the unanchored scalar loss and observes increasing error.

```
def step(theta, target, lr=.5, anchor=None, importance=None, lam=0.0):
    theta, target = aligned(theta, target)
    if not math.isfinite(lr) or lr <= 0 or not math.isfinite(lam) or lam < 0:
        raise ValueError("positive learning rate and nonnegative penalty")
    if anchor is None:
        if importance is not None or lam != 0:
            raise ValueError("penalty needs an anchor")
        anchor, importance = theta, (0.0,)*len(theta)
    else:
        anchor, importance = aligned(anchor, importance)
        if len(anchor) != len(theta) or any(f < 0 for f in importance):
            raise ValueError("aligned nonnegative importance")
    return tuple(x-lr*((x-t)+lam*f*(x-a))
                 for x, t, a, f in zip(theta, target, anchor, importance))
```

```
def closed_form(theta, target, steps, lr=.5, anchor=None, importance=None,
 ↪  lam=0.0):
    """Solve diagonal affine dynamics independently of repeated step calls."""
    theta, target = aligned(theta, target)
    if type(steps) is not int or steps < 0:
        raise ValueError("nonnegative step count")
    step(theta, target, lr, anchor, importance, lam)  # input contract only
    if anchor is None:
        anchor, importance = theta, (0.0,)*len(theta)
    out = []
    for x, t, a, f in zip(theta, target, anchor, importance):
        curvature = 1+lam*f
        optimum = (t+lam*f*a)/curvature
        out.append(optimum+(1-lr*curvature)**steps*(x-optimum))
    return tuple(out)
```

With $a = \theta_A$, $F = (1, 1)$, $\lambda = 1$ and $\eta = 1/2$, the contraction factor is zero. One second-task step reaches $(15/32, 1/2)$ exactly, and the remaining steps leave it there. Old-task retention improves relative to the naive path, while new-task acquisition worsens. This tradeoff is the central result of the fixture; reporting only reduced forgetting would hide its cost.

Update	Old Q	New Q	Forgetting	BWT
Sequential	0.558712	0.998165	0.440311	-0.440311
Anchored	0.866943	0.882568	0.13208	-0.13208

The generated values use four updates per experience and the stated synthetic score. The tests compare loops with the affine formula, exact rational first-task values and a finite-difference gradient. These checks have different failure modes: a wrong update count, derivative or normalization can be detected without reusing the same iterative implementation as its own oracle.

50.4 Read the full experience matrix

For experiences $E_1, \ldots, E_T$, let $A_{t,j}$ be the evaluation score on task j after training through task t. Row t uses one fixed artifact; column j uses one fixed evaluation distribution. The diagonal measures immediate acquisition, and entries below it measure later retention.

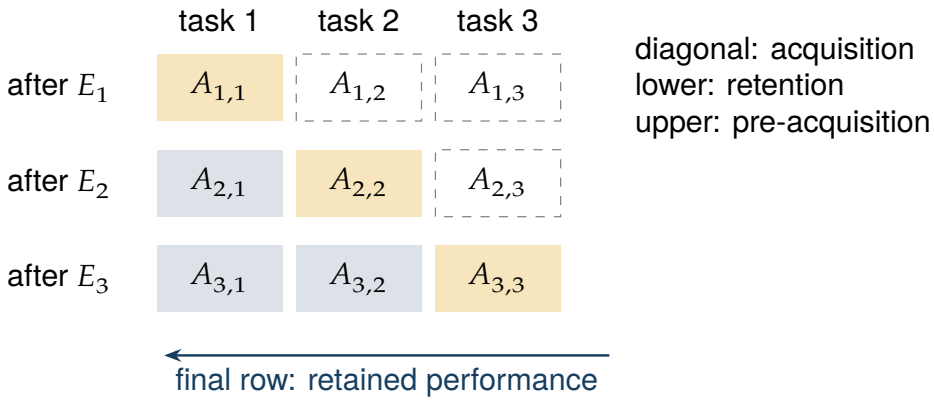


Figure 50.2: A matrix entry binds one model time and one evaluation task. Diagonal cells mark acquisition; lower cells are retention; upper cells concern pre-acquisition behavior and are not training data. Future evaluations must stay hidden from the learner and selection process.

Average final score is $T^{-1} \sum_j A_{T,j}$. We define nonnegative peak forgetting for each task as

$$F_j = \max_{j \le t \le T} A_{t,j} - A_{T,j}.$$

Including the final row makes this quantity nonnegative. Taking the maximum over all earlier rows instead would include pre-acquisition values; the post-acquisition convention used here avoids that. Last-task forgetting is zero by definition, not evidence of long-term retention.

```
def forgetting(A):
    A = matrix(A)
    # Best observed since acquisition, including final row: nonnegative
     ↪ convention.
    return [max(A[t][j] for t in range(j, len(A)))-A[-1][j] for j in
     ↪ range(len(A))]
```

Signed backward transfer is

$$\mathrm{BWT} = \frac{1}{T-1} \sum_{j<T} (A_{T,j} - A_{j,j}), \qquad T > 1.$$

It can be positive even when peak forgetting is zero. For $A_{1,1} = .6$ and $A_{2,1} = .8$, backward transfer is .2, while peak forgetting is zero. Forward transfer needs a target-only or pretraining baseline before acquisition; an off-diagonal number alone does not supply that counterfactual.

The two-task oracle allows the analyst to calculate both task scores at every row, since both quadratics are known. The learner's update uses only the current target and its permitted anchor. In a real online protocol, future task labels cannot guide updates or selection. Declare whether task IDs and boundaries are given, how often evaluations occur, and which evaluations may be inspected. Avalanche is prior work on a shared continual-learning experimental framework [119]; using a common framework does not itself make two data-access protocols equivalent.

50.5 Replay and modularity change different terms

If a replay batch mixes old and new losses with old-data fraction ρ,

$$L_{\text{mix}} = (1-\rho)L_{u_2} + \rho L_{u_1}, \qquad \nabla L_{\text{mix}} = \theta - [(1-\rho)u_2 + \rho u_1].$$

The fixture therefore uses the mixture target directly. At $\rho = 1/2$, starting from zero with half-rate learning produces $(.25, .25)$ in one step. Replay with a finite biased buffer does not equal the exact old-data objective. Report buffer bytes, selection policy, old/new exposures and privacy restrictions.

```
def replay_step(theta, new, old, rho=.5, lr=.5):
    theta, new = aligned(theta, new)
    _, old = aligned(theta, old)
    if not 0 <= rho <= 1 or not math.isfinite(rho):
        raise ValueError("old-data fraction in [0,1]")
    target = tuple((1-rho)*n+rho*o for n, o in zip(new, old))
    return step(theta, target, lr)
```

Adapters can isolate new parameters but still share routing, normalization or base representations. A router regression can prevent access to an intact old adapter. A no-interference claim therefore needs end-to-end task tests, not merely a check that old weight bytes were unchanged. A joint/offline reference has access to all allowed data and a different training opportunity; call it an upper reference only under the actual task/optimization assumptions, not a guaranteed mathematical upper bound on every algorithm.

50.6 Population search counts all its opportunities

Population-based training interleaves model training with exploitation and hyperparameter exploration [120]. Our preserved fixture is much narrower: evaluate each two-coordinate candidate on each target, retain the best mean quality, then create two deterministic coordinate mutations. It has no asynchronous workers or learned hyperparameter schedule. Ties retain the first candidate in declared input order, so completion speed cannot decide them.

For N candidates and J tasks the selection uses NJ candidate-task evaluations. A budget below NJ rejects before selection. This count excludes candidate training and runtime overhead, which actual studies must also charge. Offspring are proposals and require their own future evaluation. Their parent score cannot certify the changed coordinates.

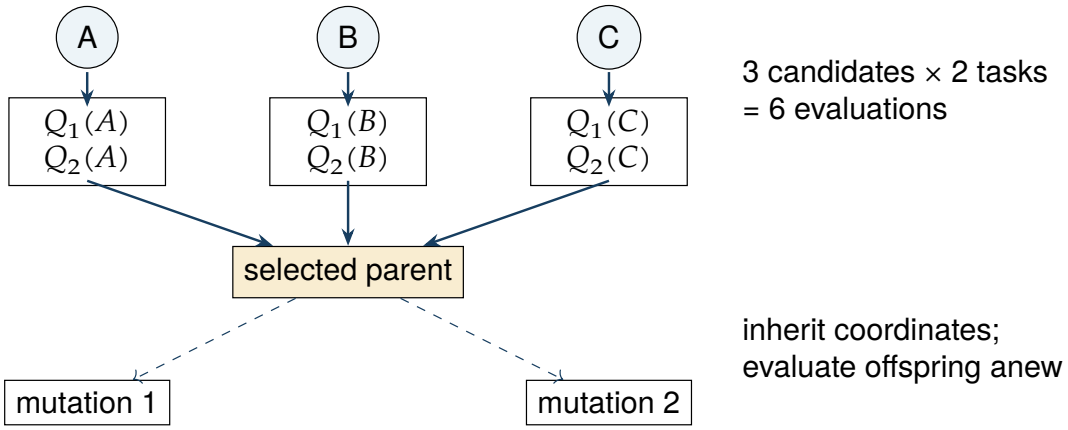


Figure 50.3: Three candidates receive two task evaluations each, consuming six opportunities. The selected parent supplies inherited coordinates to two new proposals. Dashed descent indicates inheritance; it does not carry the parent's evaluation evidence to the mutations.

Selection only by current mean can erase complementary skills. Diversity can be measured in behavior, architecture or failures; parameter distance need not imply behavioral diversity. A useful evaluation freezes population size, mutation rate, inheritance map, training budget and selection data. Compare equal total work as well as per-candidate work, and reserve a final evaluation that search never queries. Repeated search on a public test is development.

50.7 Feasibility, Pareto comparisons and poisoning

A scalar fitness $q - \alpha c - \beta r - \gamma f$ combines unlike units only after explicit scaling and policy weights. A risk constraint is often better expressed as a hard admission condition. Our reference first removes candidates above a stated risk ceiling, then computes the Pareto set for quality (maximize), cost and forgetting (minimize). Candidate A dominates B if it is no worse on every axis and strictly better on at least one. Incomparable candidates remain; the frontier is not an automatic deployment choice.

```
def pareto(candidates, max_risk):
    """Hard risk feasibility, then maximize quality and minimize cost/forgetting."""
    candidates = tuple(candidates)
    if len({c.cid for c in candidates}) != len(candidates) or not math.isfinite(max_risk) or max_risk < 0:
        raise ValueError("unique candidates and finite nonnegative risk ceiling")
    for c in candidates:
        if (not c.cid or any(not math.isfinite(v) for v in (c.quality, c.cost, c.risk, c.forgetting))
                or not 0 <= c.quality <= 1 or min(c.cost, c.risk, c.forgetting) < 0):
            raise ValueError("valid candidate measurements")
    feasible = [c for c in candidates if c.risk <= max_risk]

    def dominates(a, b):
        va, vb = (-a.quality, a.cost, a.forgetting), (-b.quality, b.cost, b.forgetting)
        return all(x <= y for x, y in zip(va, vb)) and any(x < y for x, y in zip(va, vb))

    return tuple(sorted(c.cid for c in feasible if not any(dominates(d, c) for d in feasible)))
```

The synthetic candidates `a` and `b` survive, a dominated candidate is removed, and a high-quality unsafe candidate fails feasibility. Risk here is a supplied number, not a measured probability or security certification. Tests enforce the mathematical selection rule only.

For a real update, preserve parent lineage, data admission decisions, optimizer, seeds, artifact identity, acquisition/retention matrix, calibration, budgets and failure cases. Quarantine untrusted experience before training: poisoning can persist in memory and weights. An anomaly filter has false negatives; no one filter guarantees clean training. Ablate replay, importance penalty, mutation and selection separately, including matched random-change controls.

Promotion remains an external authority decision. A canary needs bounded scope, duration, observable failure thresholds and a rollback artifact. Rollback restores future behavior under an old version; it cannot erase messages already emitted or outside actions already performed. Correct data deletion may require removing derived memory and retraining artifacts, not merely changing a version pointer.

50.8 Selective glossary

Acquisition. Performance on a task after its learning experience under the declared update budget.

Peak forgetting. Best post-acquisition score minus final score, including the final row in the maximum under this chapter's convention.

Backward transfer. Signed final-minus-acquisition score on earlier tasks; positive values indicate improvement rather than loss.

Replay mixture. A training objective combining permitted old and new data, with memory and exposure costs explicitly counted.

Pareto dominance. No worse on every chosen objective and strictly better on at least one, after hard feasibility constraints are applied.

Key ideas

- Sequential training on $(1, 0)$ then $(0, 1)$ ends at $(15/256, 15/16)$: the objective forgets by construction.
- Anchoring trades acquisition for retention and increases curvature, which tightens the stable learning-rate bound.
- Last-task forgetting is zero by definition. That is not evidence of long-term retention.
- A parent's score does not validate its mutations. Offspring are new candidates and need their own evaluation.

50.9 Exercises and worked solutions

1. Saving a dialogue changes which layer? **Answer:** durable evidence or context, unless a separate parameter update actually runs.
2. Derive unanchored θ_n. **Answer:** subtract u from the update and iterate the factor $(1 - \eta)$.
3. Four half-rate steps from zero toward $(1, 0)$? **Answer:** $(15/16, 0)$.
4. Four further steps toward $(0, 1)$? **Answer:** $(15/256, 15/16)$.

5. Is Q classification accuracy? **Answer:** no; it is an explicitly clipped quadratic score and can hide differences below its floor.
6. Derive anchored optimum. **Answer:** set the coordinate gradient to zero, yielding $(u_i + \lambda F_i a_i)/(1 + \lambda F_i)$.
7. Why may stronger anchoring destabilize an old rate? **Answer:** curvature becomes $1 + \lambda F_i$, tightening the contraction bound.
8. Does $F_i = 1$ reproduce Fisher estimation? **Answer:** no; it is a supplied identity importance matrix for a quadratic fixture.
9. What is the one-step optimum for the anchored second task? **Answer:** $(15/32, 1/2)$ under the stated half-rate and unit penalty.
10. Why report both task scores? **Answer:** retention can improve by refusing new acquisition; reduced forgetting alone hides this cost.
11. Last-task peak forgetting? **Answer:** zero by definition, since only its acquisition/final row has been observed.
12. Old score improves .6 → .8. **Answer:** BWT .2, nonnegative peak-forgetting convention zero.
13. Does an upper matrix cell prove forward transfer? **Answer:** no; compare the permitted pre-acquisition behavior with a target-only baseline.
14. Derive the replay target. **Answer:** combine gradients linearly, giving $(1 - \rho)u_2 + \rho u_1$.
15. Three candidates, two tasks, budget five? **Answer:** reject; six candidate-task evaluations are needed before any new offspring evaluations.
16. Does a parent's score validate its mutation? **Answer:** no; changed coordinates define a new candidate.
17. Why can two Pareto survivors remain? **Answer:** one can have higher quality and higher cost, so neither dominates the other.
18. Design an adaptation ablation. **Rubric:** frozen/sequential/replay/ anchor controls, declared task access, equal total data and search budget, seed distributions, full retention matrix, calibration and untouched final tests.
19. Design a poisoning test. **Rubric:** bounded injected experience, source provenance, quarantine controls, targeted and clean-task evaluations, persistence/deletion checks and no claim of security certification.
20. Can rollback undo outside side effects? **Answer:** no; restore artifact behavior separately from reconciling external state.

50.10 Bridge

Adaptation mechanisms can pass every numerical contract and still fail to acquire useful skills. Chapter 51 turns that gap into operational capability hypotheses with explicit evidence and falsifiers.

CHAPTER 51

AGI capability hypotheses and limits

In this chapter you will

- separate evidence labels, from speculative to replicated, from performance numbers
- treat breadth, depth and autonomy as separate dimensions of a capability vector
- design unknown-task evaluation behind an information firewall, and report acquisition along a budget axis
- prove that finite tests cannot establish unrestricted generality, and quantify what they can bound

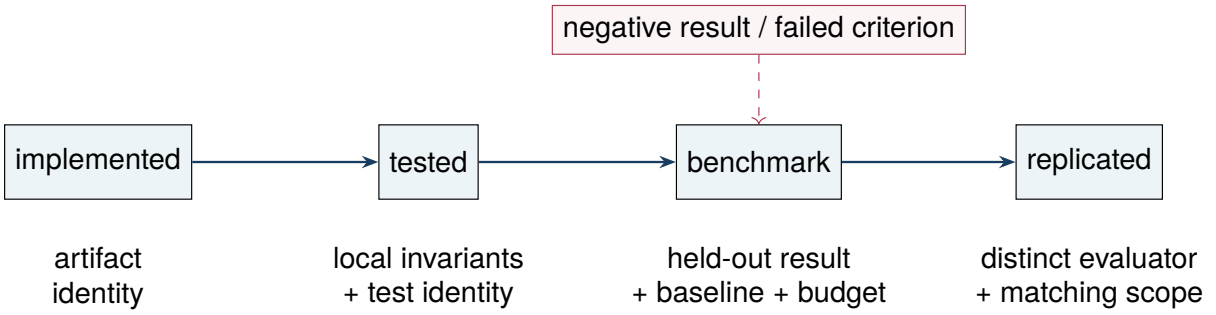
A system has memory, can call tools and can edit its own code. Which new task can it now solve, under what budget, and how often does it fail? Without those questions, a component inventory is easy to mistake for a capability argument. Evolutor, DOGMA and Hermon DNA remain research directions; the teaching implementations in this book do not establish artificial general intelligence.

We turn a capability taxonomy and a claim ledger into operational contracts. Local tests, held-out demonstrations and independent replication require different records. An executable metadata gate can reject an incomplete claim, but it cannot determine whether an experimenter told the truth or whether the chosen tests capture general intelligence.

51.1 An evidence label is not a performance number

Use six evidence labels: speculative, hypothesized, implemented, tested, benchmark-demonstrated and replicated. Speculative names a direction; hypothesized adds a testable mechanism and falsifier. Implemented binds an artifact. Tested adds local test evidence. Benchmark-demonstrated adds a bounded empirical comparison. Replicated adds a separate reproduction of a specified claim. These are editorial evidence categories, not a universally accepted scientific maturity scale.

A serial promotion path is useful for preventing skipped checks, but passing one edge cannot itself justify the next. A tested memory allocator remains tested software even if its name contains “intelligence.” A hypothesized capability can be falsified while the mechanism remains implemented. A new scope or stronger wording requires a new claim rather than silently inheriting old evidence.



Every record names one scope and falsifier; a broader claim needs new evidence.

Figure 51.1: Artifacts, local tests, bounded benchmark results and independent reproduction support different edges. Each evidence node binds the same claimed scope. A failure remains attached to that scope; adding a box to the architecture does not supply an empirical edge.

The reference requires source and test digests for tested claims, then a supporting evaluation with a baseline, data/artifact identity, declared budget, sampling unit, held-out declaration, successful outcome and lower confidence bound above a frozen threshold. Replication needs a distinct evidence ID, a different evaluator and a declaration of independence under matching suite/artifact/data/baseline/budget/unit/threshold. Here matching identities define exact reproduction; robustness under changed data or hardware should be a related, separately scoped claim.

```
def supports(e, capability):
    return (isinstance(e, Evidence) and e.capability == capability
            and all(isinstance(v, str) and bool(v) for v in
                    (e.eid, e.suite, e.baseline, e.budget, e.unit, e.evaluator))
            and sha(e.artifact_digest) and sha(e.data_digest)
            and e.held_out is True and type(e.independent) is bool
            and e.outcome == "success"
            and math.isfinite(e.lower_bound) and math.isfinite(e.threshold)
            and e.lower_bound > e.threshold)
```

```
def promote(c, new_level, records=(), **changes):
    if not valid(c, records) or new_level not in LEVELS:
        raise ValueError("valid source claim and target level required")
    if LEVELS.index(new_level) != LEVELS.index(c.level)+1:
        raise ValueError("one evidence step at a time")
    if set(changes)-{"implementation_digest", "tests_digest", "evidence"}:
        raise ValueError("promotion cannot silently change scope or falsifier")
    candidate = replace(c, level=new_level, **changes)
    if not valid(candidate, records):
        raise ValueError("missing distinct supporting evidence")
    return candidate
```

These fields are necessary metadata only. A fake digest can have valid syntax, an asserted held-out split can be contaminated, and two named teams can share the same hidden development decisions. The code does not authenticate artifacts, recompute statistics or inspect institutional independence. It deliberately does not turn the sample ledger into evidence of an actual benchmark win.

51.2 Breadth, depth and autonomy form a vector

The Levels of AGI framework separates performance and generality and examines autonomy as an interaction/deployment choice [121]. We use that distinction without assigning Evolutor a level. A broad assistant with human intervention and a narrow unattended controller answer different questions. Autonomy is not a multiplier that converts narrow skill into breadth.

Define a capability vector over declared task families, accompanied by budgets and failure strata. A single mean can conceal a missing capability or a catastrophic subgroup. The taxonomy below supplies operational tests, not a claim that every term has a unique uncontested definition.

51.2.1 Generalization, transfer and reasoning

Generalization asks about held-out distributions, task rules or environments. Name what is novel: examples, compositions, dynamics, goals or task families. An unseen row generated by the same public template may test interpolation rather than new-rule acquisition. Audit pretraining exposure, prompt examples, retrieval corpora and human tuning against the final evaluation.

Transfer compares a source-trained or adapted system with a target-only control under matched total opportunity. Report whether the source experience is charged to the budget. A target score alone cannot show positive transfer. Reasoning should be decomposed into compositional, symbolic, quantitative, spatial, causal or interactive tasks with checkable outputs. Plausible intermediate prose is not proof of a valid latent mechanism. Use answer verifiers, minimal counterexamples and interventions on task structure rather than scoring only the fluency of explanations.

51.2.2 Planning, world modeling and tool use

Planning requires goal achievement through actions under cost and uncertainty. Measure success, actions, violations, recovery, wall time and human assistance. A static plan-shaped answer has not yet interacted with a changing environment. World modeling asks whether predicted transitions or latent dynamics improve held-out decisions. Compare against reactive, retrieval-only and model-free controls while fixing action and compute budgets.

Tool use requires correct selection, typed arguments, error recovery and task completion under deterministic authorization. A connector proves availability, not competence. Remove a needed tool, introduce a controlled timeout or return a version conflict. The outcome should distinguish justified deferral from invalid execution, not reward any completed action regardless of policy.

51.2.3 Memory, continual learning and adaptation

Memory evaluation names the storage type: transient carry, context, addressable records or learned parameters. Measure retention horizon, retrieval precision and recall, interference, provenance and deletion. A record surviving on disk does not imply the model can retrieve or use it. Chapter 50's acquisition and retention matrix is the corresponding contract for continual learning; saving events alone is not weight learning.

Adaptation declares which of context, memory, weights, structure or population changed. Freeze data access, number of trials and update budget. A task-ID oracle changes the problem

relative to a task-free stream. Performance after hundreds of target-specific trials is not evidence of efficient acquisition from a few observations.

51.2.4 Metacognition, self-improvement and open-endedness

Operational metacognition includes calibrated uncertainty, detection of errors, strategy switching and deferral when evidence is insufficient. Evaluate selective risk at several coverage levels: a system that refuses everything has low observed error but no useful coverage. Confidence text without probability/error calibration is weak evidence.

Self-editing produces a candidate change; self-improvement requires repeated, causally attributable held-out gains with declared total cost and regression limits. Compare against equally budgeted random edits, human-designed updates or fixed search. Preserve negative generations. Deployment authority stays outside the proposal mechanism. Open-ended learning additionally requires sustained useful novelty beyond a fixed suite, with task/niche definitions, novelty measures, transfer and resource accounting. New strings or new filenames do not establish new skills.

51.3 Unknown-task evaluation needs an information firewall

Before a final run, freeze artifact, prompts, sampler, retrieval corpus, adaptation algorithm, baseline and allowed tools. Separate development tasks, adaptation observations and final evaluation. The evaluator may reveal the observations a new environment legitimately supplies without exposing withheld answers. Task selection and human interventions are part of the protocol.

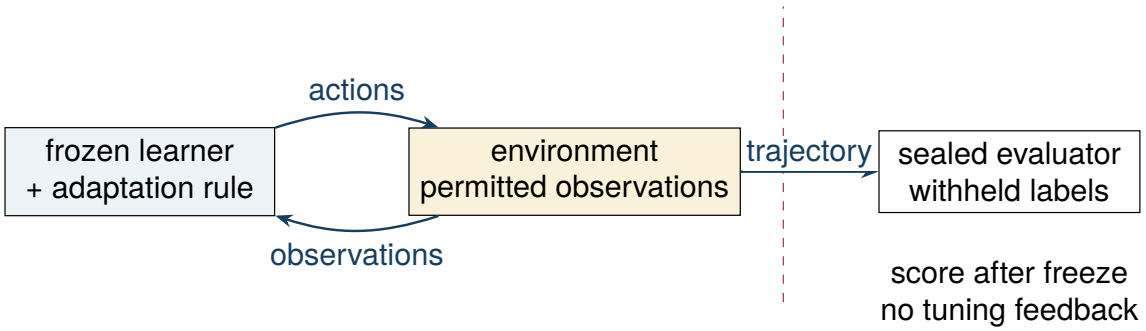


Figure 51.2: A learner sees permitted interaction observations but no final labels. The evaluator scores sealed trajectories after artifact and adaptation rules are frozen. Repeated development feedback from the final suite breaks this boundary even if the filenames still say test.

ARC-AGI-2 and ARC-AGI-3 provide specific benchmark designs, not universal certificates. The official ARC-AGI-2 page emphasizes novel reasoning and efficiency reporting [122]. The ARC-AGI-3 page describes interactive environments, exploration and action efficiency [123]. Their versions and access dates belong in a result capsule because benchmark policies evolve. We import no benchmark scores and make no claim that Evolutor ran either suite.

Interactive evaluation must distinguish environmental reset, episode context, cross-episode memory and parameter learning. If task instances share a secret rule, carrying memory across instances changes the independence assumptions. Charge environment actions, tokens, model

calls and any human help separately. Report failed runs and budget exhaustion as outcomes rather than silently dropping them from the denominator.

51.4 Skill acquisition has a budget axis, not a magical ratio

A conceptual ratio such as $\Delta\text{capability}/(\text{data} + \text{compute} + \text{actions} + \text{risk})$ identifies an important concern but adds unlike units. Do not print it as an empirical efficiency metric without a declared conversion policy. Use separate curves or a vector of constraints instead.

For one chosen action axis $0 = b_0 < \cdots < b_K = B$, let $q_k \in [0,1]$ be quality after budget b_k. Under linear interpolation, a normalized area is

$$\bar{q}_B = \frac{1}{B} \sum_{k=1}^{K} (b_k - b_{k-1}) \frac{q_k + q_{k-1}}{2}. \tag{51.1}$$

It lies in $[0,1]$ because it averages bounded interpolated quality over the budget interval. It changes if one chooses a different interpolation or budget axis. It cannot compare dollars with actions without an explicit conversion.

```
def acquisition_area(actions, quality):
    """Normalized trapezoidal area over one declared action-budget axis."""
    x, y = tuple(actions), tuple(quality)
    if (len(x) != len(y) or len(x) < 2 or x[0] != 0
            or any(not math.isfinite(v) for v in x+y)
            or any(b <= a for a, b in zip(x, x[1:]))
            or any(not 0 <= v <= 1 for v in y)):
        raise ValueError("increasing nonnegative budget and aligned quality")
    return sum((b-a)*(u+v)/2 for a, b, u, v in zip(x, x[1:], y, y[1:]))/x[-1]
```

Illustrative curve	Normalized area
Early acquisition	0.65
Late acquisition	0.35

Both illustrative curves reach one at budget two. The early curve $(0, .8, 1)$ has area .65; the late curve $(0, .2, 1)$ has area .35. Final score alone loses this distinction. Multiplying all budget coordinates by a positive constant preserves the normalized area, a tested unit-rescaling property.

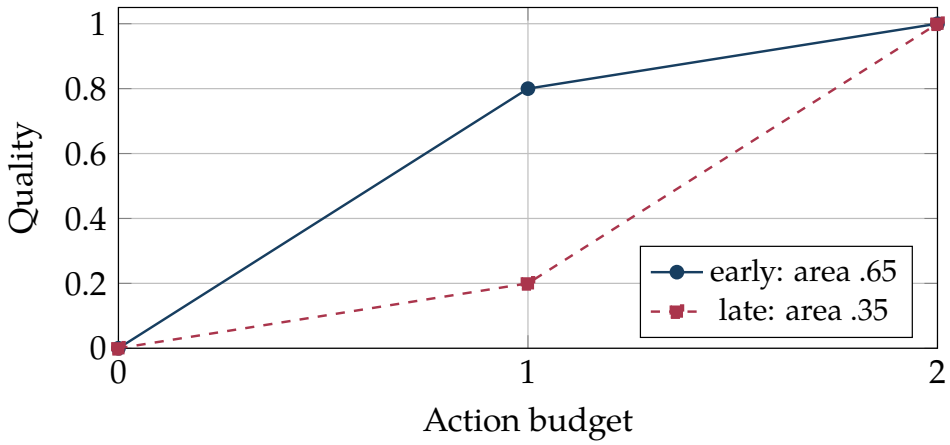


Figure 51.3: The same final score conceals different acquisition histories. Areas assume linear interpolation between the three stated points. The curves are assigned numbers, not results from an agent benchmark.

51.5 Finite tests cannot prove unrestricted generality

Proposition 51.1 (Finite observational underdetermination)**.** *Suppose an evaluator observes a finite set of task histories and the system class permits arbitrary behavior on an unobserved history. Two systems can agree on every observed output while differing on that unobserved history.*

Proof. Take one system f. Construct g to return f on every observed history and a different output on an unobserved history h^*. By assumption the class permits that construction. The finite transcript is identical for f and g, so it cannot establish their agreement at h^*. □

The conclusion is limited: evidence supports a distributional or structural claim under assumptions, not universal competence. It does not make benchmarks useless. Hidden tasks, broad coverage, strong priors and mechanistic arguments can strengthen the inference while leaving its scope explicit.

For independent bounded task-family success values in $[0,1]$, Hoeffding's inequality gives $\Pr(|\hat{q} - q| \geq \epsilon) \leq 2e^{-2n\epsilon^2}$. For m frozen claims, a union bound gives total error at most $2me^{-2n\epsilon^2}$. Thus $n \geq \log(2m/\delta)/(2\epsilon^2)$ is a sufficient sample count under those assumptions. Correlated puzzles, adaptive selection and repeated inspection invalidate the simple independent-sample interpretation. The units must be task families if that is the claimed population, not thousands of near-duplicate instances treated as independent breadth.

Zero observed violations is also weaker than zero risk. For n independent Bernoulli trials with constant failure probability p, zero failures has probability $(1-p)^n$. Solving $(1-p)^n = \alpha$ gives a one-sided upper bound

$$p_{\text{upper}} = 1 - \alpha^{1/n}.$$

```
def zero_failure_upper(n, alpha=.05):
    """One-sided exact upper bound after zero independent Bernoulli failures."""
    if type(n) is not int or n < 1 or not 0 < alpha < 1:
        raise ValueError("positive sample size, alpha in (0,1)")
    return -math.expm1(math.log(alpha)/n)
```

Independent trials	95 percent upper failure bound
10	0.258866
100	0.029513
1000	0.00299125

These generated bounds are a mathematical illustration, not a certification of an agent's safety. Dependence, adversarial distribution shift and missing failure detection can make the premise inappropriate.

51.6 Three research directions, three falsifiable hypotheses

DOGMA's non-Transformer state organization motivates quality–memory–latency comparisons with strong recurrent/SSM and other appropriate sequence baselines. State boundedness alone says nothing about retained task information. If matched long-context tasks show no advantage within the predeclared effect margin, reject that bounded efficiency claim rather than changing the task list afterward. Chapter 31's diagonal recurrence is narrower than full DOGMA and cannot certify that comparison.

Hermon DNA preserves Transformer semantics while testing genomic structure and serving machinery. A task-trained genomic improvement can establish domain skill; it does not establish broad reasoning. Compare a DNA-aware mechanism with an otherwise matched Transformer, and separate serving improvements from learned quality. Chapter 35's learned-absolute-position cache is a semantic implementation, not a new pretrained foundation model.

Evolutor's compiler, memory and governance direction motivates persistent workflow tests: can a frozen system recover, use provenance-aware memory and complete held-out tasks within policy and budget? Cross-engine integration is an engineering property until those evaluations run. The book's runtime, memory and authority references are separately bounded contracts; their composition into a durable service and its task evaluations are not assumed by this chapter's ledger.

Capability and deployment safety remain separate axes. A capable model can violate policy; a correctly fenced tool host can serve a limited model. Measure policy violations, isolation, revocation and recovery alongside success. More human intervention may improve safety and success while reducing autonomy; report all three rather than hiding assistance in the runtime.

51.7 Selective glossary

Capability scope. The task population, information access and budget to which a behavioral claim applies.

Held-out evaluation. Evidence withheld from development and selection, with permitted online adaptation observations separately specified.

Replication. A distinct evaluation of the same bounded claim by a separate evaluator; metadata declarations alone do not verify independence.

Acquisition area. A budget-axis-dependent integral of quality under a declared interpolation; final score alone does not determine it.

Autonomy. The degree of independent task execution and recovery relative to human assistance, distinct from breadth or performance.

Key ideas

- A component inventory is not a capability argument. Memory, tools and self-editing name mechanisms, not demonstrated skills.
- Autonomy is not a multiplier that converts narrow skill into breadth.
- Two systems can agree on every observed history and differ on an unobserved one. Evidence supports scoped claims under assumptions, never universal competence.
- This book demonstrates executable evidence contracts and falsifiable hypotheses. It demonstrates no AGI capability.

51.8 Exercises and worked solutions

1. A planner object exists. What evidence level follows? **Answer:** implemented if artifact-bound; successful planning still needs task evidence.
2. Memory survives restart. Does continual learning follow? **Answer:** no; acquisition, retention and the update layer remain untested.
3. Why require a falsifier? **Answer:** it names a result that would weaken or reject the claim instead of allowing every outcome to count as success.
4. Can one evidence record establish replication? **Answer:** no; the ledger requires a distinct evaluator/record under a matching contract.
5. Is declared independence proof of independence? **Answer:** no; it is metadata requiring external verification.
6. Why cannot promotion broaden scope? **Answer:** evidence for one task population does not automatically support a larger population.
7. A tool connector is available. What remains? **Answer:** selection, arguments, recovery, authorized completion and held-out success under budget.
8. Compare autonomy with breadth. **Answer:** intervention independence and task coverage are different dimensions.
9. Does fluent reasoning text prove causal reasoning? **Answer:** no; use checkable outcomes and task-structure interventions.
10. Why is final-suite feedback dangerous? **Answer:** repeated tuning turns the supposedly held-out suite into development information.
11. Compute the early curve area. **Answer:** $(.4 + .9)/2 = .65$.
12. Compute the late curve area. **Answer:** $(.1 + .6)/2 = .35$.
13. Rescale actions by ten. **Answer:** both numerator and denominator scale by ten, preserving normalized area.
14. Why not sum actions and risk in a denominator? **Answer:** different units need explicit policy conversions; a vector is often clearer.
15. Construct the finite-test counterexample. **Answer:** copy all tested outputs and change one unobserved history; the transcript stays identical.
16. Derive zero-failure upper bound. **Answer:** invert $(1 - p)^n = \alpha$, giving $1 - \alpha^{1/n}$.

17. Does zero failure among correlated tasks use the same bound? **Answer:** not under the stated independent Bernoulli premise.
18. Design self-improvement evaluation. **Rubric:** external frozen evaluator, matched search control, held-out gains across iterations, charged total budget, regressions, lineage and separately authorized promotion.
19. Design an interactive breadth test. **Rubric:** distinct held-out task families, observation/action rules, assistance accounting, contamination audit, separate success, cost, calibration and failure metrics, and task-level uncertainty.
20. Strongest conclusion here? **Answer:** executable evidence-contract checks and falsifiable research hypotheses; no demonstrated AGI capability.

51.9 Bridge

Chapter 52 binds claims to experiments, failures and artifact closures. A final synthesis must make unresolved work easier to find, including evidence that contradicts the preferred architecture.

PART XII

Research synthesis

What has actually been established, what failed, and what should be tested next?

The synthesis separates model semantics, systems engineering and capability science, and records which layer each result belongs to. It keeps failures and counterexamples as first-class artifacts, binds results to dependency-closed capsules and states open problems as experiments that could reject the preferred architecture. The Epilogue that follows places the book among three meanings of genomic computation and looks at frontiers beyond it.

CHAPTERS IN THIS PART

52 **Failures, open problems and reproducible synthesis** Synthesize failures, unresolved questions and reproducible evidence.

CHAPTER 52

Failures, open problems and reproducible synthesis

In this chapter you will

- state what is implemented and what remains a research claim for DOGMA, Hermon DNA and Evolutor
- preserve the closest competitor in every evidence matrix
- turn a symptom into a durable failure artifact and keep it after resolution
- bind results to a dependency-closed capsule and state open problems as disconfirming experiments

A final ledger contains one hypothesis, one unsuccessful comparison and one preserved counterexample. Has the project failed, or has it learned something? The useful answer names what the counterexample rules out, which implementation still behaves correctly, and what experiment comes next. A research book should end with fewer vague claims than it began with.

This chapter joins the failure taxonomy, claim/experiment/theorem ledgers and post-book agenda to the repository's actual evidence. All 52 chapters are now assembled as a complete manuscript candidate with executable references. That is not publication acceptance, a deployed production Evolutor runtime, a trained genomic result or a demonstration of AGI.

52.1 What is actually implemented, and what is still staged?

Separate three research layers. Model semantics define functions and state transitions. Systems engineering preserves those functions while changing storage, scheduling and execution. Capability science tests behavior on held-out tasks. A correct kernel does not prove a learning hypothesis; a predictive gain does not validate a biological analogy.

Direction	Concrete local evidence	Remaining claim boundary
DOGMA	Earlier model references and the actual Chapter 31 restricted input-gated diagonal recurrence; tested native state operations	Restricted engine teaching recurrence is not full DOGMA or a trained genomic benchmark
Hermon DNA	Chapter 23 multi-layer Transformer and Chapter 35 exact causal prefill/chunk/decode contract with learned absolute positions	Cache parity is not genomic skill, RC usefulness or device performance
Evolutor	Typed research/runtime contracts; split, recovery, adaptation and evidence oracles	Reference composition is not an integrated, durable, concurrent production service

These statements concern this book's reference code, not any separate Evolutor software product. The continuous candidate includes Chapters 1–52. Its separate paged-runtime companion connects the actual Chapter 36 allocator to the Chapter 39 runtime and Chapter 41

compiler, with full-model parity, ownership, failure-cleanup and effect-preservation checks. A complete manuscript and this bounded integration do not supply durable agent adapters, distributed fencing, device kernels or task-trained empirical evidence.

The cross-chapter checks in Chapters 48–52 use the actual model interfaces: Chapter 48 exercises Chapter 23's offline shared-RC path; Chapter 49 checks chunk continuation in Chapters 31 and 35; Chapter 52 checks the original Hermon causal path under a suffix perturbation. Local trusted serialized checks establish the stated behavior on these fixtures. They do not establish production concurrency, security isolation or learned language-agent ability.

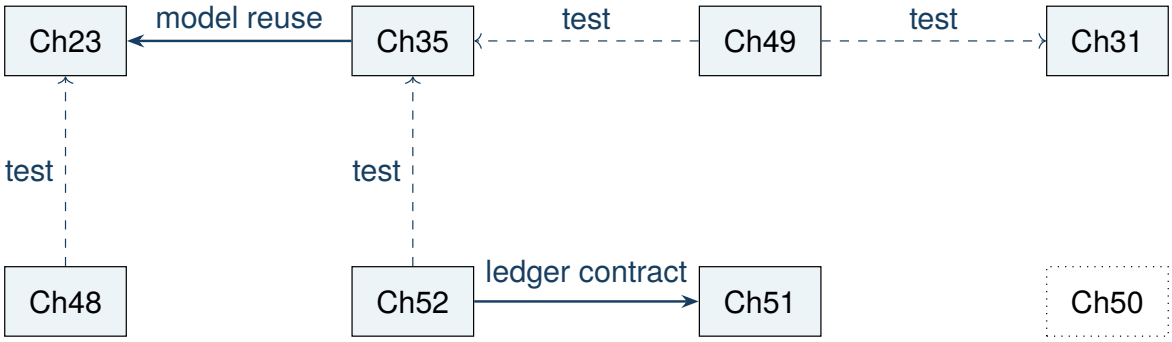


Figure 52.1: Solid arrows denote source-level reuse of the actual earlier Hermon model and the owned capability contract. Dashed arrows denote cross-chapter test probes. A manifest can name all eight reference files without claiming that every node imports every other node. External runtime/library versions remain an additional closure dimension.

52.2 Evidence matrices should preserve the closest competitor

For DOGMA, the central question is quality–memory–latency tradeoff against strong recurrent/SSM and other appropriate baselines. Markov/MLP/CNN controls matter when local statistics or motifs can solve the task. Transformer and long-context controls matter when addressable history is the claimed advantage. Non-Transformer identity is an architectural fact, not evidence of superiority.

For Hermon DNA, hold the Transformer backbone, training opportunity and task contract fixed while ablating the proposed genomic mechanism. Compare strand-aware transformations against augmentation and shared two-pass pooling; compare coordinate policies at the same legal information boundary. A faster attention kernel can improve serving cost while leaving learned quality unchanged. Charge extra data, context and tuning rather than crediting those gains to the biological inductive bias.

For Evolutor, evaluate persistent workflow completion, state recovery, provenance, authorization, governance and resource accounting. Measure human intervention as part of the workload. A policy gate can correctly deny an action that would have increased raw completion rate. Task success and policy success need distinct denominators and a declared joint admissibility rule.

Every comparison records parameters, training examples/tokens, optimizer and tuning budget, state/context budget, precision, hardware time, seeds and service constraints, under the earlier shared evidence contracts [125]. Modeled operation counts, synthetic scores, CPU

timings and actual accelerator measurements are different evidence. Absence of a measured hardware column should appear as unmeasured, never as zero cost.

52.3 Turn a symptom into a durable failure artifact

A practical failure taxonomy includes semantic correctness, numerical instability, causality/target leakage, baseline and optimization regressions, scaling, contamination, reproducibility, security/policy, service SLOs, biological interpretation and capability overclaim. These categories may overlap. A noncausal feature can cause both an implausible loss and a false capability claim. Store a primary category plus the causal explanation instead of forcing every failure into exactly one box.

Reduce a failing run to the smallest input that preserves the discrepancy. Keep the original run too: minimization can remove conditions relevant to deployment frequency. Record expected behavior, observed behavior, artifact, environment, command, data identity and exact comparison rule. A later fix adds a resolution record and a regression test; it does not delete the original counterexample.

```
    def add_failure(self, f):
        if (not isinstance(f, Failure) or not f.fid or f.fid in self.failures
                or f.claim_id not in self.claims or f.experiment_id not in
                 ↪  self.experiments
                or self.experiments[f.experiment_id].claim_id != f.claim_id
                or not all((f.category, f.symptom, f.counterexample)) or
                 ↪  f.resolved_by):
            raise ValueError("failure must link the same claim and experiment")
        self.failures[f.fid] = f
```

The failure check binds claim and experiment to the same identity. Checking only that both exist would allow a failure about one claim to name another claim's experiment. This relational error could make the wrong research direction appear disproved or resolved. The local tests include that mismatch as a negative control.

Known audit targets include overlapping-window and homology leakage, target bases embedded in tokens, incompatible RC labels, cross-record state carry, stale generations, silent fallback, invalid perplexity denominators, external tool replay and tuned-on final benchmarks. These are failure hypotheses to test continually, not a claim that every current system exhibits them.

52.4 Random-data sanity is an expectation with assumptions

Let the target Y be uniformly distributed over V classes and independent of the predictor's permitted context. Conditional on that context, the predictor supplies probabilities $q_1, \dots, q_V > 0$ summing to one. The expected next-target cross entropy in nats is

$$\mathbb{E}[-\log q_Y] = -\frac{1}{V}\sum_{i=1}^{V} \log q_i = \log V + D_{\mathrm{KL}}(U\|q) \geq \log V. \tag{52.1}$$

Equivalently, Jensen's inequality for convex $-\log$ gives $-V^{-1}\sum_i \log q_i \geq -\log(V^{-1}\sum_i q_i) = \log V$. Equality holds for uniform q. Zero probability for a possible target gives infinite expected loss. This is a population statement under independence.

For $V = 4$, uniform prediction gives $\log 4 \approx 1.38629$ nats. For assigned $q = (.7, .1, .1, .1)$ the expected loss is larger, but one sampled A has loss $-\log .7 \approx .356675$, below $\log 4$. Thus a finite empirical average below the expectation is not logically impossible. Report variation, sample size, class distribution and dependence before declaring leakage. Sustained implausible gains warrant audits of target alignment, overlapping tokens, padding, masks and loss denominators. A shuffled-label control must also preserve the intended sampling structure; shuffling within leaked groups can leave an unintended shortcut.

52.5 Causality tests need a changing negative control

For a causal predictor, changing suffix $x_{p:}$ while retaining prefix $x_{:p}$ must not change logits strictly before p. Freeze model mode and numerical conditions. A test comparing an output tensor with itself proves nothing; the changed suffix must actually be different, and at least one permitted later output should change in a suitable positive control.

The local test calls Chapter 35's full oracle, which calls the original Chapter 23 causal path. It compares ACGT and ACAA: the first two logits are equal and later logits differ on the assigned model. This checks a real implemented interface without substituting a detached attention toy. It does not prove every possible model setting by finite sampling. The architectural causal proof still depends on legal feature construction and layer masks. An offline reverse-complement average is useful for an invariant classification task but cannot silently feed future sequence into a next-base predictor.

52.6 A ledger distinguishes plans, observations and conclusions

Each claim has exact wording, scope, evidence stage and falsifier. Each experiment has a hypothesis link, baseline, budget, metric, threshold, data/artifact identity, seeds, sampling unit and an outcome. A planned run has no observed confidence bound. A completed failed run is evidence against the criterion, not an implementation milestone that unlocks a benchmark claim.

```
    def promote(self, cid, new_level, evidence_ids=()):
        c = self.claims[cid]
        if new_level in ("benchmark-demonstrated", "replicated") and any(
                f.claim_id == cid and not f.resolved_by for f in
                 ↪ self.failures.values()):
            raise ValueError("unresolved counterexample blocks empirical
             ↪ promotion")
        updated = capability_module().promote(self._capability(c), new_level,
                    self.evidence_records(), evidence=evidence_ids or c.evidence)
        self.claims[cid] = replace(c, level=updated.level,
         ↪ evidence=updated.evidence)
```

The reference imports the owned Chapter 51 evidence contract. It rejects empirical promotion while a linked counterexample remains unresolved. A resolution must cite a distinct supporting experiment; the failure remains in the capsule. This policy deliberately favors explicit treatment of negative evidence. It does not adjudicate scientific disputes: an experiment labeled successful may still be invalid, and an apparent resolution requires human mechanistic/statistical review.

Synthetic ledger record	Count
Claims	1
Experiments	1
Preserved failures	1

The one-experiment fixture is explicitly synthetic. Its assigned candidate quality .7 loses to baseline .8 in one case. The research claim stays hypothesized. The placeholder artifact digests have valid syntax but are not represented as real trained model artifacts. A real empirical claim must resolve its digests to accessible evidence, which this fixture does not do.

The theorem ledger is different. It records hypotheses, domain/codomain, proof and dependencies. In Chapters 48–52, split closure has a graph premise, at-most-one local dispatch has a serialized-history premise, anchored convergence has a curvature/rate premise, and finite-test underdetermination has a permissive system-class premise. Numerical tests check implementations and counterexamples. They do not silently extend those proofs to production concurrency, full DOGMA, biological causation or universal capability.

52.7 Reproducibility is a dependency closure, not one checksum

A result capsule needs source revision and dirty-tree identity; data, tokenizer/model identities and licenses; complete configuration and seeds; compiler, runtime, library, driver and hardware versions; precision; exact commands; raw measurements/checkpoints/logs; analysis scripts; tolerances; known failures and exclusions. A seed alone fixes none of the missing inputs. Private or licensed data may need controlled access rather than publication, but the access limitation must be stated.

```
    def capsule(self):
        obj = {"schema": "evod52-ledger-v1",
               "claims": [asdict(x) for x in sorted(self.claims.values(),
                ↪  key=lambda x: x.cid)],
               "experiments": [asdict(x) for x in
                ↪  sorted(self.experiments.values(), key=lambda x: x.eid)],
               "failures": [asdict(x) for x in sorted(self.failures.values(),
                ↪  key=lambda x: x.fid)]}
        return {"ledger": obj, "sha256": digest(obj)}
```

The ledger capsule canonically sorts records, serializes their exact content and hashes it. Record insertion order cannot change its identity. Changing a failure or result changes the digest. Byte integrity and scientific validity remain distinct: a reproducible error is still an error, but is easier to investigate than an undocumented one.

The separate file manifest hashes eight actual reference sources: Chapters 23, 31, 35 and 48–52 [124]. This is a deliberately bounded code closure used by the checks of Chapters 48–52. It is not a whole-book capsule or an environment lock. The local review records additionally bind each chapter's authored LaTeX, figures, tests and resulting PDF. Their claims are local author review with `accepted=false` and `independent_review=false`.

```
def verify_files(root, manifest):
    if (set(manifest) != {"files", "sha256"} or not isinstance(manifest["files"],
     ↪  dict)
            or digest(manifest["files"]) != manifest["sha256"]):
        raise ValueError("manifest envelope mismatch")
```

```
    missing, changed = [], []
    for name, expected in manifest["files"].items():
        path = safe_path(root, name)
        if not capability_module().sha(expected):
            raise ValueError("file digest syntax")
        if not path.is_file():
            missing.append(name)
        elif hashlib.sha256(path.read_bytes()).hexdigest() != expected:
            changed.append(name)
    return {"missing": sorted(missing), "changed": sorted(changed)}
```

The manifest verifier reports missing and changed files. A separate envelope hash detects changes to its listed mapping relative to the trusted envelope. Canonical relative paths and exclusion of symlinks narrow its input boundary; this trusted local reader is not a hardened hostile-filesystem service. A concurrent file replacement can invalidate assumptions between checks and use.

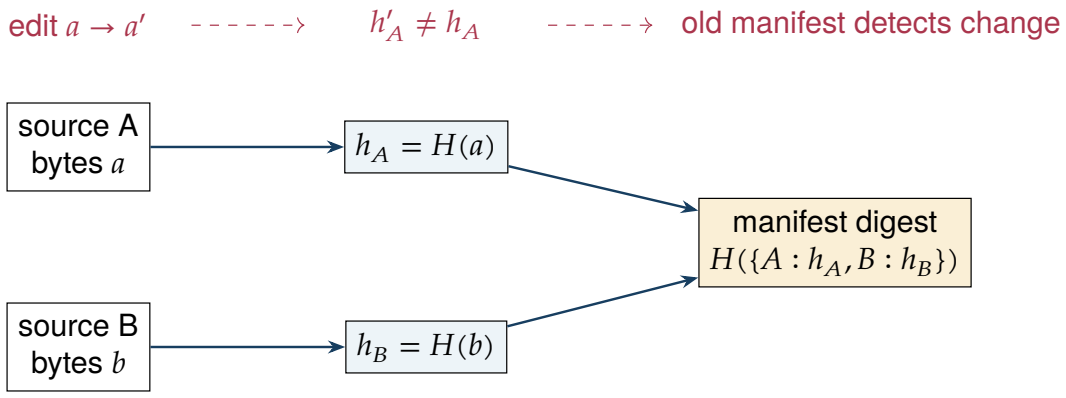


Figure 52.2: One source mutation changes its file digest and therefore the manifest digest. The old trusted manifest reports a changed file; creating a new manifest changes identity rather than certifying the old result. The dependency order additionally checks declared references, while undeclared dependencies remain a separate audit responsibility.

```
def dependency_order(files, edges):
    """Validate a declared acyclic dependency closure; infer no hidden imports."""
    files = set(files)
    if set(edges) != files or any(set(v)-files for v in edges.values()):
        raise ValueError("every artifact and dependency must be in the closure")
    done, active, order = set(), set(), []

    def visit(node):
        if node in active:
            raise ValueError("cyclic declared dependency")
        if node in done:
            return
        active.add(node)
        for dep in sorted(edges[node]):
            visit(dep)
        active.remove(node); done.add(node); order.append(node)

    for node in sorted(files):
```

```
        visit(node)
    return tuple(order)
```

The declared dependency graph must name every referenced artifact and contain no cycle. Depth-first validation marks active ancestors; re-entering one detects a cycle. After all dependencies of a node are completed, append that node. By induction, every dependency precedes its consumer in the returned order. This is a proof about the supplied graph. The routine does not infer imports, external datasets or dynamic dependencies, so completeness remains an explicit closure audit.

52.8 Open problems require disconfirming experiments

52.8.1 State, regulation and structural learning

Which information should remain exactly addressable, and which can be compressed without damaging the task? Compare retention/retrieval distortion against state bytes and measured latency, including adversarial collisions and delayed-use tasks. A fixed-size vector is bounded storage, not a guarantee of sufficient memory.

Does a proposed regulation mechanism improve quality or efficiency beyond ordinary gating, routing or conditional computation? Name the nearest existing mechanism, freeze opportunity, and ablate the claimed difference. If gains disappear under a matched gate control, revise the explanation. For structural learning, count all proposals, invalid candidates and tuning; test bounded complexity, inheritance compatibility and regression. Search over structures is distinct from gradient learning and from deployment authorization.

52.8.2 Memory, continual learning and genomic inductive bias

A rate–distortion–retrievability study can vary exact records, lossy summaries, native state and parameters under fixed task information. Measure errors caused by compression separately from retrieval failure and misuse. Include provenance and deletion constraints, which an unconstrained accuracy objective can ignore.

Continual learning should report acquisition, peak forgetting, signed transfer, calibration, training cost and replay storage under declared data access. Test novel skills and poisoning resistance without assuming that a growing memory store is learning. Genomic inductive-bias tests need coordinate-correct variant pairs, strand-label actions, homology-aware splits, matched GC controls, task trained baselines and independent biological follow-up where mechanistic claims are made. Better synthetic motif arithmetic is not a wet-lab result.

52.8.3 World models, agents, self-improvement and open-endedness

Persistent agents should be tested under controlled stale observations, ambiguous effects, tool failures and long-horizon changes. Ablate native state, retrieval and external simulators separately to identify their contribution. Record human intervention and reconciliation costs. A service that merely continues running has not thereby solved a longer task.

For self-improvement, preregister held-out criteria and matched search controls across repeated generations; charge the evaluator and failed proposals too. For open-endedness, define useful niches and sustained novelty before scoring new skills. For AGI, retain the unresolved breadth/depth/acquisition questions from Chapter 51. No finite book artifact

closes them. Safety, authority and human control require their own evidence rather than being assumed from capability scores.

52.9 A staged program and a separate release decision

Phase A freezes tasks, baselines and datasets. Phase B tests DOGMA/Hermon scaling and mechanism ablations. Phase C studies memory and continual adaptation. Phase D tests persistent interactive agents. Phase E seeks independent replication and adversarial evaluation. Each phase has a stopping rule: failure to meet a predeclared effect, budget or policy constraint can end the hypothesis. Phases are research dependencies, not promised successes or deadlines.

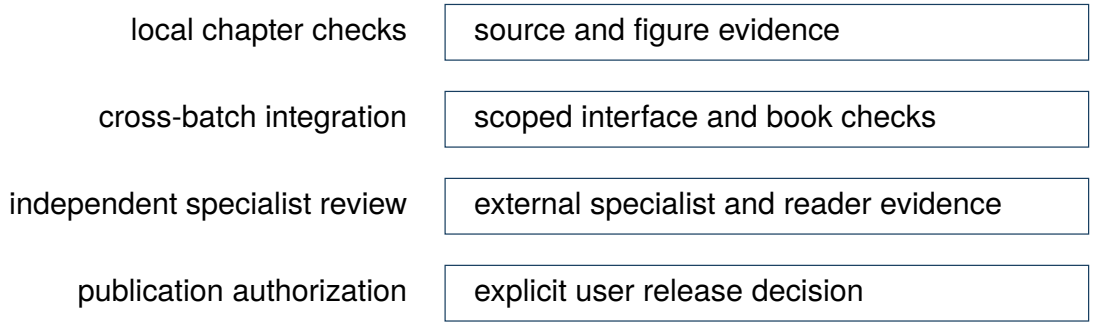


Figure 52.3: A chapter can pass local code and render checks while whole-book integration and independent review remain open. Publication is a distinct authorization gate. These evidence dimensions must not be collapsed into one percentage-complete indicator.

Whole-book integration reconciles source adoption, stable cross-chapter interfaces, bibliography, labels, notation, glossary and index; reruns examples against their real dependencies; builds the continuous candidate; runs whole-repo regressions without concurrent author builds; and inspects the cumulative pages. Material revisions require those checks again. Independent specialist/reader review and rights/access auditing remain distinct release gates. A completed chapter sequence is not final-book acceptance. No stage in this chapter authorizes publishing.

52.10 Selective glossary

Counterexample. A concrete input and expected/observed discrepancy that contradicts a stated property under its assumptions.

Resolution record. New evidence addressing a failure while preserving its original identity and symptom.

Capsule. Bound artifacts, environment, commands, raw observations and analysis needed to reconstruct a scoped result.

Dependency closure. The complete declared set of inputs and their relationships; validating a declared graph does not infer hidden dependencies.

Acceptance gate. A review or authorization decision with its own scope, separate from passing local tests or producing a PDF.

Key ideas

- A correct kernel does not prove a learning hypothesis, and a predictive gain does not validate a biological analogy.
- A finite empirical loss can fall below $\log V$ by chance. The bound concerns an expectation under independence.
- A resolution adds evidence; it never deletes the original counterexample.
- Implementation, integration, independent review and publication are separate gates and must not be collapsed into one percentage.

52.11 Final exercises and worked solutions

1. A chapter test passes. Which layer is evidenced? **Answer:** its stated local semantics, not every capability motivated by the chapter.
2. Is Chapter 31 full DOGMA? **Answer:** no; it is the actual restricted input-gated diagonal teaching recurrence used by these engine checks.
3. Does Hermon cache parity establish genomic usefulness? **Answer:** no; it preserves a function whose task quality requires separate evaluation.
4. Candidate beats a smaller baseline. Architecture gain? **Answer:** not without capacity/opportunity controls appropriate to the claim.
5. Name three failure categories for suspicious loss. **Answer:** target leakage, invalid loss denominator and distribution mismatch; finite sample variation is another explanation to quantify.
6. Derive the expected uniform-target bound. **Answer:** apply Jensen to $-\log$ over the normalized probabilities, obtaining $\log V$.
7. Can one random target beat that value empirically? **Answer:** yes; a chance A under probability .7 has loss $.356675 < \log 4$.
8. Why change the suffix in a causality test? **Answer:** identical inputs or accidentally ignored suffixes can make a test pass vacuously.
9. What does the later-output control add? **Answer:** evidence that the assigned perturbation actually changes a permitted part of the computation.
10. Is a planned experiment sufficient for empirical promotion? **Answer:** no; it contains no observed successful result or justified confidence bound.
11. Can a failure cite another claim's experiment? **Answer:** no; the relational check rejects that inconsistent link.
12. Does resolution delete the failure? **Answer:** no; it adds a distinct supporting reference while retaining the counterexample.
13. A digest changed after editing source. **Answer:** the artifact identity changed; the old review cannot certify the new bytes.
14. Does a manifest prove dependency completeness? **Answer:** no; it verifies named files, while hidden/dynamic dependencies require audit.

15. Prove dependency-before-consumer order. **Answer:** each node is appended only after recursively completing all named dependencies; active-stack re-entry rejects cycles that would violate this ordering.

16. Does a seed form a complete capsule? **Answer:** no; data, code, configuration, environment, analysis and failures remain necessary.

17. Design an independent reproduction. **Rubric:** verified inputs, matching scoped contract, separate evaluator, raw results, declared numerical tolerances, failure preservation and externally checked independence.

18. Design a state-memory study. **Rubric:** fixed information/budget, exact and compressed controls, delayed retrieval, collision/shift tests, quality–latency–memory curves, provenance and deletion costs.

19. What remains after manuscript integration? **Answer:** independent scientific and reader review, full editorial and accessibility checks, and explicitly unimplemented production interfaces and empirical studies. A passing reference adapter does not close those distinct gates.

20. What is the defensible final synthesis? **Answer:** a collection of precise implementations, premise-bound proofs and falsifiable research contracts whose broader empirical claims still require evidence.

52.12 Final synthesis

DOGMA asks whether DNA-native computational organization can provide useful non-Transformer alternatives. Hermon DNA asks how genomic structure and Transformer semantics can be combined and served. Evolutor asks how heterogeneous models, memory, tools, learning and governance can form a persistent research system. Their strongest future claims must be earned by experiments that can also reject them. Preserve failures, bind conclusions to exact artifacts, and keep implementation, integration, independent review and publication separate.

Epilogue: genomic computation in three senses

The preface separated three meanings of genomic computation. One is computation *about* DNA, where recorded sequence is data. Another is computation *with* DNA, where molecules are the machine. The third is computation *inspired by* DNA, where genomic organization is a design hypothesis for digital systems. This book has worked almost entirely in the first and third senses. This epilogue steps back. It asks what the frontier in each sense has actually demonstrated, how the book's evidence discipline applies to each, and where the three might meet. It treats these meeting points as research hypotheses, not as forecasts.

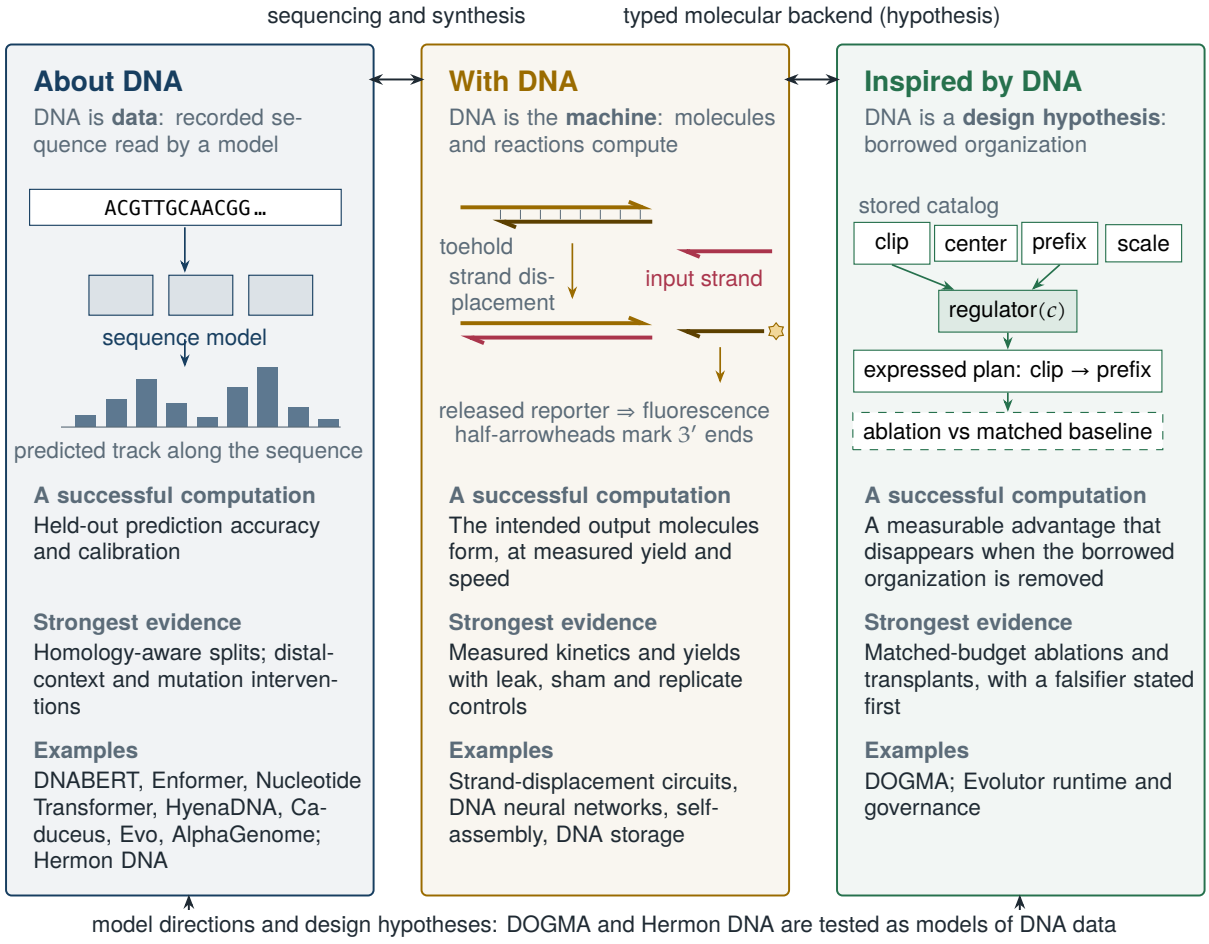


Figure E.1: Three senses of genomic computation. Each has its own objects, its own notion of a successful computation and its own strongest form of evidence. The labelled interfaces between columns (sequencing and synthesis, design hypotheses, typed backends) are where the senses could meet, and each needs its own contract and test before it carries a claim.

Computation about DNA: what the frontier models teach

Sequence models of genomes have changed quickly. DNABERT adapted masked pretraining to overlapping k-mer tokens [53], and DNABERT-2 made tokenization efficiency itself an

architectural question [16]. Enformer predicted gene-expression and chromatin tracks from sequence by integrating information over distances of up to roughly a hundred kilobases [142]. Nucleotide Transformer showed the value of scale, data diversity and a systematic downstream evaluation protocol [54]. HyenaDNA asked whether a non-attention operator could reach long single-nucleotide contexts [15]. Caduceus built reverse-complement equivariance and bidirectionality into the architecture [48]. Evo and Evo 2 pushed single-nucleotide modelling to genome scale and across all domains of life [143, 49]. AlphaGenome predicts thousands of functional-genomic tracks, up to single-base resolution, from a one-megabase input window, and in its authors' evaluation it matched or exceeded the strongest external models on most of their variant-effect benchmarks [144].

Several of this book's chapters exist because these models made their questions unavoidable. Tokenization changes what a prediction means (Chapters 5 and 24). Strand symmetry is an architectural choice with a testable transformation law, not a data-augmentation habit (Chapters 18 and 21). Long context is first an access graph and only then a speed claim (Chapter 24). The hardest lessons concern evidence. Benchmark comparisons between DNA foundation models turn out to be task-dependent, and frozen representations and fine-tuned competitors answer different questions [116]. Homologous and overlapping sequences can leak across a random split (Chapters 25 and 48). A model's sensitivity to a mutation is not the mechanism a cell uses (Chapters 17 and 48).

Against this frontier, DOGMA and Hermon DNA are small and honest. They are reference implementations with exact contracts, not trained foundation models, and their pilots stayed at or near chance on a synthetic motif task under the declared budgets. The scientific value of the two directions is that their claims are now stated precisely enough to test. The three experiments that would most change the assessment are these:

1. a matched-budget comparison of DOGMA's addressable loci with a capacity-matched recurrent or state-space baseline on tasks with sparse, delayed dependencies;
2. a Hermon DNA ablation that holds the Transformer backbone fixed and varies only a genomic mechanism (span-aware positions, tied strand features or local-plus-global access) on homology-aware splits;
3. a study of the Evolutor runtime on persistent workflows that measures recovery, provenance and policy compliance together with task success.

Each comes with a falsifier written before the run, as Chapters 22, 29 and 51 require.

> **A boundary this book keeps**
>
> Some frontier genomic models are also used to *generate* biological sequences. This book does not cover designing sequences intended to function in living systems. Its examples are synthetic, its evaluation designs concern prediction, and its biological follow-up protocols stop at stating hypotheses and controls. Generative biological design raises biosafety and governance questions that need specialist oversight. Those questions fall outside an implementation textbook.

Computation with DNA: what molecules can now compute

In 1994 Adleman solved a small instance of a directed Hamiltonian path problem with DNA strands, ligation and selection [128]. The field that followed now has a programmable primitive in toehold-mediated strand displacement, whose kinetics can be tuned over many orders of magnitude [129]. It also has a compiler precedent: any formal chemical reaction

network can, in principle, be approximated by DNA strand-displacement reactions [130]. Strand-displacement cascades compute digital circuits [131] and implement neural networks [132]. Molecular winner-take-all networks recognize patterns of many bits [133]. Algorithmic self-assembly runs reprogrammable molecular algorithms from one tile set [137], and the nucleation kinetics of self-assembly can itself classify patterns [138]. DNA also stores digital data at high density, with error-correcting codes designed for its synthesis and sequencing channel [139, 140, 141].

Two recent results changed what "DNA computing" can mean. In 2025, Cherry and Qian reported supervised learning carried out by DNA itself. Training examples were written into molecular concentration memories that then served as the weights for classifying test patterns, all in vitro [134]. In the same year, Song and Qian showed heat-rechargeable circuits and neural networks that compute repeatedly with new inputs [135]. Learning and reuse, long the weakest points of molecular computation, now have concrete demonstrations.

The resource semantics of Chapter 16 apply directly, and they expose how different the substrate is. A molecular computation runs on very many molecules at once, but each bimolecular step at nanomolar concentration takes minutes to hours. Its "memory" is a set of concentrations with leak and decay, and its output is a fluorescence or sequencing measurement with noise, not a register read. A fair comparison with electronic computation must state units, workload and error model, exactly as Chapters 29 and 47 demand of silicon. On those terms molecular computers are not general replacements for electronic ones. Their distinctive strengths are operating where electronics cannot (inside a test tube or a sample), interfacing directly with biomolecules, and storing information densely and durably.

Where the senses could meet

Evolutor's typed contracts suggest several concrete interfaces between the three senses. Each is stated as a hypothesis with the evidence it would need.

DNA as an archival memory class. Chapter 40 separated native state, durable events, lossy summaries and addressable records. DNA storage could be a further class: very dense and long-lived, but slow to write and read and subject to a characteristic error channel. The hypothesis is that some evidence capsules and lineage records (Chapters 43 and 52) are written once and read rarely, so a durable molecular tier could hold them. Evidence would need a typed contract (capacity, write and read latency, retrieval granularity, error and erasure semantics) and a cost comparison with conventional archives under the same retention requirement. Deletion is especially hard, which matters for the forgetting semantics of Chapter 40.

A molecular backend behind the runtime. Chapters 39 and 41 route typed operations to backends whose contracts are declared. A strand-displacement or self-assembly program could in principle be such a backend for operations that must happen in a sample. Its contract would include reaction semantics, concentration units, time-to-result, leak and error rates, single-use versus rechargeable resources [135] and the readout's measurement model. The compiler precedent already exists, from formal reaction networks to DNA reactions [130]. The open questions are verification and reliability at scale, together with an honest account of when a wet backend is worth its latency.

Models that read molecular programs. Sequence models could help *design and check* molecular computers: predicting hybridization, secondary structure and leak for strand-displacement components before synthesis. This keeps the model in the first

sense (computation about DNA) and the machine in the second. It requires validation against measured kinetics, not against the model's own predictions.

Molecular learning as a comparison point. Supervised learning in DNA neural networks [134] gives Part VI's methodology an unusual test case. The same estimands, splits, controls and uncertainty apply, and so do the same warnings about selecting on the test set, whether the learner is silicon or chemistry.

Inspiration that must earn its keep. The third sense, the book's own DOGMA and Evolutor directions, borrows organization rather than chemistry. The relevant evidence is never that a mechanism resembles biology. It is a matched-budget comparison in which removing the borrowed organization costs something measurable.

What would count as evidence of general or superhuman capability

Some readers will hope that systems built from these ideas lead towards artificial general intelligence, or towards capabilities beyond human performance across broad domains. Chapter 51 explained why no finite test establishes unrestricted generality, and why breadth, depth and autonomy form a vector rather than a level [121]. The same discipline gives five questions to ask of any claim of general or superhuman capability, including claims made about this book's directions:

1. **Which task families, and how were they held out?** Name the population of tasks, show that development never saw the final suite, and audit pretraining exposure.
2. **Against which human or system baseline, under what budget?** "Superhuman" needs a defined comparison group, matched information and time, and reported uncertainty.
3. **How efficiently is new skill acquired?** Report quality as a function of data, compute, actions and assistance, not only final scores (Chapter 51).
4. **What happens out of distribution and under adversarial pressure?** Include the failure cases, deferral behaviour and calibration, not only the successes.
5. **Is it reproducible and safe to operate?** Independent replication, policy compliance, isolation, revocation and human control need their own evidence. They do not follow from capability scores (Chapters 43–45).

None of this book's results answers these questions. That is its scope, not a verdict on the questions. A research program that wants to answer them will need trained models at scale, sealed evaluations and independent replication. It will also need the habits this book has tried to make automatic: name the mechanism, derive the consequence, and test the claim in a way that could have failed.

Open problems

The problems below collect the most consequential unresolved questions raised across the book. Each is phrased so that a result could settle it in either direction.

1. **Addressable versus compressed memory.** For a fixed task-information budget, when do explicit loci or retained KV beat a compressed state, measured as quality per state byte and per unit latency?
2. **Regulation beyond gating.** Does any regulation-inspired mechanism improve quality or efficiency beyond ordinary input-dependent gating and routing after capacity matching?

3. **Strand symmetry in practice.** On which genomic tasks does exact reverse-complement equivariance improve sample efficiency, and on which does it erase orientation-specific signal?
4. **Long-context genomics.** Which distal dependencies do learned models actually use? Answer with distal-context ablations on homology-aware splits, not with attention maps.
5. **Exact engines at scale.** Can the exact-refinement proofs of Parts VII and VIII (state pools, paged KV, speculative decoding) be carried into device kernels with tested completion fences and measured benefit?
6. **Effects in compilers.** What effect system lets an AI-workflow compiler reorder and cache tool, memory and model calls safely, and how is that verified?
7. **Continual adaptation without regression.** Can structural and parametric updates be promoted under the governance of Chapter 43 with measured, bounded regressions?
8. **Persistent agents.** Do provenance-aware memory and intent-before-dispatch recovery reduce human intervention on long-horizon tasks without increasing policy violations?
9. **Molecular tiers.** Under what retention and access patterns does a DNA-based archival tier beat conventional archives on total cost and durability?
10. **Learning across substrates.** Can molecular learners and electronic learners be compared on a shared task with shared estimands and controls, and what does the comparison reveal about each substrate's limits?

The book ends where research begins: with precise objects, honest negative results and experiments that have not yet been run.

APPENDIX A

Molecular biology for computing readers

This appendix collects the biology that the genomic chapters assume. It explains what the symbols in a sequence file refer to, why orientation and coordinates are part of the data contract, and which experimental measurements supply the labels that models predict. It is a reader's guide for computing specialists, not a laboratory manual. It contains no procedures, and it makes no clinical interpretation of variants. The standard textbook treatment is Alberts and colleagues [2].

A.1 The molecule behind the string

DNA is a polymer of nucleotides. Each nucleotide joins a deoxyribose sugar, a phosphate group and one of four bases: adenine (A), cytosine (C), guanine (G) or thymine (T). Phosphodiester linkages join the 3′ carbon of one sugar, through a phosphate group, to the 5′ carbon of the next. A single strand therefore has a chemical direction, and a sequence is written by convention from its 5′ end to its 3′ end.

In the double helix two strands pair *antiparallel*: one runs 5′ → 3′ left to right, the other 3′ → 5′. A pairs with T and G pairs with C. The second strand is therefore determined by the first, and reading it in its own 5′ → 3′ direction gives the *reverse complement* $R(x)$. Chapters 5, 18, 21 and 24 build their strand symmetries on exactly this fact. Complementing without reversing describes the partner strand read in the wrong direction. It is a common bug, not a biological operation.

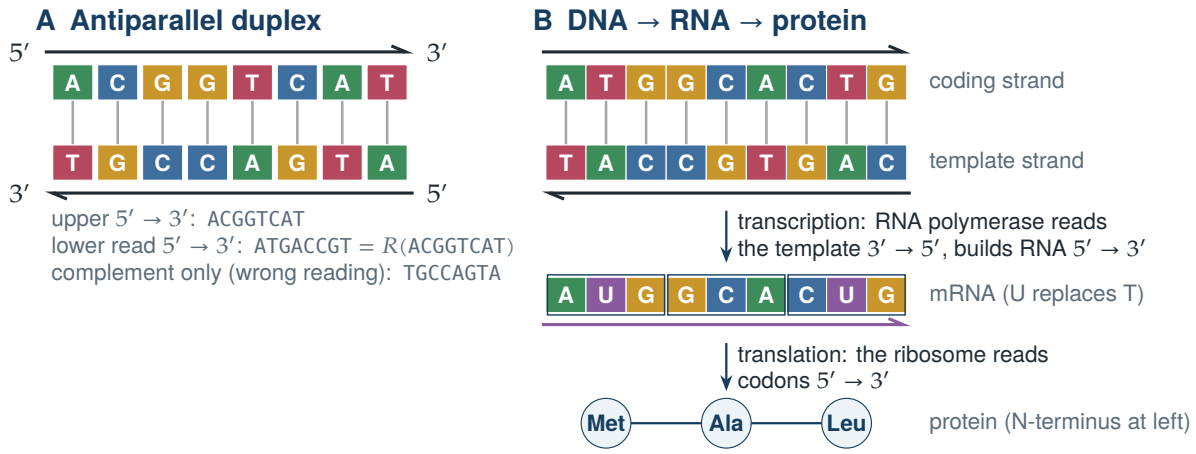


Figure A.1: Panel A: a short duplex. Each strand carries a half-arrowhead at its 3′ end; grey rungs are base pairs. Read 5′ → 3′, the lower strand is the reverse complement of the upper one. Panel B: the flow from DNA to RNA to protein. RNA polymerase reads the template strand 3′ → 5′ and synthesizes RNA 5′ → 3′, so the RNA has the sequence of the coding strand with U in place of T. The ribosome reads codons 5′ → 3′. Regulatory factors (Chapter 11) act at several of these steps. Shapes are schematic.

A.2 Genomes, assemblies and coordinates

A *genome* is the complete DNA of an organism, packaged into chromosomes. A *reference assembly* is a versioned consensus sequence used as a coordinate system, and positions are meaningful only relative to a named assembly. The same variant can have different integer coordinates in two assemblies. Converting between assemblies is a separately versioned mapping that can fail or be ambiguous (Chapter 48). A *locus* is a region in that coordinate system. It may contain a gene, a regulatory element or neither.

File formats disagree about how positions are counted, which is a classic source of off-by-one errors:

Format	Position convention	Example
BED	zero-based start, end excluded [145]	the first 100 bases are `0 100`
VCF	one-based `POS` [146]	the first base has `POS` 1
GFF3	one-based start and end, start ≤ end [147]	the first base starts at 1
This book	zero-based, half-open $[a, b)$	the first 100 bases are $[0, 100)$

Annotations also carry a strand (`+` or `-`). A feature on the minus strand is read from the reverse complement. Its coordinates map as $[a, b) \mapsto [L - b, L - a)$ inside a length-L window (Chapters 21 and 24). The symbol `N` marks an unknown base. It is an observation symbol, not a fifth nucleotide, and the book's reference code rejects or masks it explicitly rather than mapping it to A.

A.3 From gene to function

Transcription copies a stretch of DNA into RNA. RNA polymerase reads the template strand and synthesizes RNA $5' \to 3'$. In eukaryotes the initial transcript is processed: introns are removed by splicing, and the mature messenger RNA (mRNA) leaves the nucleus. Translation reads mRNA in three-base codons and assembles a protein. Many RNAs are never translated and act directly. "Expression" can therefore mean RNA abundance, protein abundance or a downstream activity. Chapter 11 shows why these must be named separately, and why protein data alone may be unable to identify the rates that produced them.

Expression is regulated at many steps [2]. *Promoters* are regions where the transcription machinery assembles. *Enhancers* and other regulatory elements, sometimes far from the gene, bind *transcription factors* that recognize short sequence motifs. Whether a region is accessible depends on *chromatin*, the packaging of DNA with histone proteins and its chemical modifications. In three dimensions, distant elements can come into contact. These facts motivate the book's interest in long context, distal access and stateful regulation. Remember throughout that a model's sensitivity to a motif is not proof that a protein binds it in a cell (Chapters 17 and 48).

A.4 Where the labels come from

Genomic models are trained and judged against measurements, and each kind of measurement defines a different prediction task. In outline:

RNA sequencing counts transcripts and supplies expression levels.

Chromatin accessibility assays mark regions open to cutting enzymes.

Immunoprecipitation-based assays report where a chosen protein or histone mark associates with DNA.

Chromosome conformation assays report which regions are near each other in three dimensions.

Massively parallel reporter assays measure the regulatory activity of many designed sequences in one experiment.

Every assay has a cell type, condition, resolution, noise model and processing pipeline, and a label is only as meaningful as that provenance (Chapters 2, 25 and 48). A model that predicts one assay in one cell type has not thereby learned regulation in general.

A.5 Variation and relatedness

Genomes differ. A single-nucleotide variant (SNV) replaces one base. An insertion or deletion (indel) changes length, and structural variants rearrange larger segments. A variant is described by assembly, position, reference allele and alternate allele. Under reverse complementation both alleles are complemented and the position is mirrored (Chapter 48). Sequences also share ancestry. Homologous regions within and between species can be nearly identical, so a random split of genomic windows can put near-copies on both sides of a train/test boundary. This is why the book closes leakage relations transitively before splitting (Chapters 2, 25 and 48).

A.6 What this appendix does not cover

Laboratory methods, organism engineering and clinical genetics are outside the book. Where a chapter proposes biological validation, it names the hypothesis, intervention, controls and replication unit that specialists would need. It does not describe how to perform the experiment.

APPENDIX B

Notation and conventions

Chapters define their symbols where they first use them. This appendix lists the conventions that hold across chapters, so that a symbol borrowed from an earlier chapter keeps its meaning.

B.1 Sequences, positions and strands

Symbol	Meaning
Σ, V	Token alphabet and its size; $\{A, C, G, T\}$ for canonical DNA
$x = (x_0, \dots, x_{L-1})$	A record of length L, indexed from zero
$[a, b)$	Half-open base interval: $a \le p < b$
$c(\cdot)$, $R(x)$	Base complement and reverse complement, $R(x)_i = c(x_{L-1-i})$
u_t, y_t	Model input and next-token target at step t ($y_t = x_{t+1}$ in causal mode)
m_t, v_t	Validity or target masks; a mask is a weight or a legality flag, never a label
$\mathcal{F}(p)$	Mutation footprint: the tokens whose spans contain base p

B.2 Tensors, models and state

Symbol	Meaning
$[B, T, D]$	Batch, time (positions) and feature width; H heads of width $d_h = D/H$
θ, $\nabla_\theta \mathcal{L}$	Learned parameters and the gradient of a scalar loss
h_t, M_t	Recurrent fast state and memory loci after step t
K^ℓ, V^ℓ	Layer-ℓ key and value cache, shape $[H, t, d_h]$
(a, b), $\star$ / $\otimes$	Affine pair $h \mapsto a \odot h + b$ and its chronological composition
$\odot$	Elementwise product
$\sigma(\cdot)$	Logistic sigmoid $1/(1 + e^{-z})$
softmax	Row-wise $\exp(z_j) / \sum_k \exp(z_k)$ over *allowed* entries
LN	Layer normalization over the feature axis of one token

In mathematics vectors are columns. When the text follows PyTorch storage, batches are rows and a linear layer applies the transpose of its stored weight matrix. Chapters say which convention a formula uses.

B.3 Probability, evaluation and units

Symbol	Meaning
$\mathcal{L}$, nats	Mean negative log-likelihood with natural logarithms
BPB	Bits per raw base: summed loss divided by $B \ln 2$ for B represented bases
$D_{\mathrm{KL}}(p\|\|q)$	Kullback–Leibler divergence $\sum_x p(x) \log(p(x)/q(x))$
$\mathrm{TV}(p, q)$	Total variation distance $\frac{1}{2} \sum_x \|p_x - q_x\|$
$\widehat{\Delta}$	An estimated paired difference between two predictors
$p_{\mathrm{upper}} = 1 - \alpha^{1/n}$	One-sided upper bound on a failure rate after n independent zero-failure trials
MAC, FLOP	Multiply–accumulate; floating-point operation (a MAC is often counted as two FLOPs)
KiB, MiB / KB, MB	Powers of 1024 / powers of 1000 bytes
tick	One logical step of a reference program, not a time measurement

B.4 Typographic conventions

`Monospace` marks code, file names and literal sequences. A **bold term** is defined where it appears and is listed in the index. Listings are extracted from the tested reference programs at build time and are never retyped. "Generated" tables come from running the chapter's reference code; "assigned" numbers are chosen by hand to make arithmetic visible and are not measurements.

APPENDIX C

Evidence classes and the claim ledger

Every result in this book belongs to one of the evidence classes below. The classes are not a ranking of worth. A clean proof and a careful measurement answer different questions, and the most common error is to let evidence of one class stand in for another.

C.1 Eight classes of evidence

Class	What it can establish	What it cannot	Example in the book
Proof under premises	A property for every input satisfying stated assumptions	That the assumptions hold for a real system	Scan associativity (Ch. 7, 26)
Finite reference test	That an implementation agrees with an oracle on declared cases	Behaviour on untested inputs or other implementations	KV-cache parity (Ch. 35)
Negative control	That a test can detect a specific failure	That no other failure exists	Future-copy predictor (Ch. 2)
Synthetic trace	Exact behaviour of a mechanism on constructed input	Usefulness on natural data	Selective-memory traces (Ch. 19)
Numerical emulation	The effect of a numerical policy on a fixed model	Hardware speed or storage savings	KV precision rollout (Ch. 37)
Trained held-out result	Predictive performance under a split and budget	Biological mechanism; transfer beyond the split	Motif pilots (Ch. 22–23)
Hardware measurement	Time, memory or energy on stated hardware and workload	Performance on other workloads or devices	None reported in this edition
Biological observation	What happened in a declared experiment	Generality beyond its conditions	Cited primary studies only

A ninth category is the *hypothesis*: a falsifiable statement with no supporting evidence yet. Hypotheses are valuable, and the book states many of them, but they are never reported as results.

C.2 A claim template

Chapters 29, 51 and 52 use a ledger that binds a claim to the evidence that would support or refute it. A minimal entry has these fields:

Field	Content
Claim	Exact wording, with no adjectives that the evidence does not carry
Scope	Task population, information access, budget and units
Evidence class	One of the classes above, and the artifacts that carry it
Baseline	The closest competitor, given the same opportunity
Falsifier	A result, fixed before the run, that would weaken or reject the claim
Uncertainty	Sampling unit, number of independent units, interval method
Artifacts	Content identities of code, data, model, environment and raw results
Status	Hypothesized, implemented, tested, benchmark-demonstrated or replicated
Failures	Linked counterexamples, kept after any resolution

C.3 Ten questions to ask of any result

1. Is the target aligned with the information the model may legally use?
2. Could overlap, homology, reverse complements or shared sources leak across the split?
3. Does a trivial baseline (composition, length, majority class) explain the score?
4. Were all runs, seeds and failed configurations reported, or only the best?
5. Is the uncertainty computed over the actual independent units?
6. Do the compared systems have matched data, compute, tuning and state budgets?
7. Does an ablation or sham intervention isolate the claimed mechanism?
8. Is a measured number labelled with its hardware, workload and timing boundary?
9. Could the result be reproduced from the recorded artifacts by someone else?
10. Is the conclusion worded no more strongly than its evidence class allows?

A result that survives these questions is not thereby important. It has become something that can be built on.

Glossary

These entries prevent distinct model and runtime objects from being conflated. They describe the manuscript's interfaces, not demonstrated biological capabilities.

DOGMA Proposed non-Transformer DNA-native model/runtime research direction.

Hermon DNA Transformer-based DNA/genomic model/runtime direction.

Evolutor Broader orchestration, research, compiler/runtime, memory, adaptation and governance layer; not the external model named Evo 2.

Parameter Learned value shared across uses of a model operation.

Activation state Values produced for a particular input or continuing record with fixed parameters; not automatically learned persistent knowledge.

Optimizer state Learning-procedure memory such as moments or momentum, distinct from a model's record-level carry.

Carried state All fields needed to resume the same computation under the same configuration, including boundary metadata.

Checkpoint Version-bound stored snapshot of the declared model or learning/runtime state; a parameter file alone may not restore the full run.

Padding identity Storage padding leaves the logical state unchanged. An uncertain biological base is not automatically padding.

Record reset Explicit replacement with initial state at an independent record boundary; neither zero retention nor a chunk boundary implies reset.

Update clock Events, elapsed time or genomic-distance increments that advance an explicitly defined state update.

Retention Attenuation of old state in a specified recurrence; not proof of task-useful long-range memory.

Memory read Access to retained information under a declared addressing and information-availability policy.

Equivariance Input transformation induces a specified output or feature transformation; this differs from leaving the result unchanged.

Invariance A result remains unchanged under a declared transformation. Strand-sensitive labels may require a nontrivial label action instead.

Offline/full-record access Available information can include suffix content. The Chapter 21 dual/RC representation belongs to this contract.

Causal feature Depends only on the permitted prefix and declared external knowledge. Target alignment and token spans must also be legal.

Gradient parity Agreement of derivatives as well as output values; detaching chunk state can preserve one and break the other.

Functional ablation Intervention on one mechanism with a declared fixed parameter set; retraining a controlled alternative answers a different question.

Non-identifiability Distinct parameter explanations implement the same observed function under the declared measurement interface.

Token span Half-open interval in nucleotide coordinates carried by a token. A token ordinal is not its physical base coordinate or width.

Joint strand action Sequence reverse complement together with the specified orientation/feature/label transformation.

Rotary position Pairwise feature rotation indexed by declared position units; relative-score algebra does not prove unseen-context generalization.

Legal attention edge A query-key access allowed by all mask, validity, information and objective constraints.

Global relay A designated within-record attention position aggregating information; another layer reads its updated representation.

Retrieval Access to a specified external corpus under a legal query and knowledge policy, not merely a global attention token.

Capability evidence Task- and protocol-bound observations of performance; implemented memory or agent persistence alone is not evidence of general intelligence.

Estimand Precisely defined quantity an experiment intends to estimate, including predictors or training procedure, target population and sampling unit.

Dependency closure Connected components of all declared relationships that prohibit cross-partition placement; curation adequacy remains separate.

Paired comparison Evaluate both predictors on the same record and retain their joint outcomes when estimating differences and uncertainty.

Cluster resampling Draw independent clusters with replacement while retaining their dependent records and paired predictions within each draw.

Conditional test uncertainty Evaluation-sampling variation for fixed predictors, not automatically variation across training procedures or seeds.

Resource inventory Static auditable counts and units, distinct from instrumented workload measurements such as peak memory, latency or energy.

Affine prefix summary Chronological pair (A, B) mapping incoming state to $A \odot h + B$; each prefix is required, not merely a terminal reduction.

Scan eligibility Coefficients must be available without computing the recurrent states to be scanned; causality alone does not establish eligibility.

Reset-before-update Replace incoming state by a declared reset state, then apply the current valid transition; distinct from identity padding.

Boundary detach Preserve a carried value while intentionally severing its earlier derivative path; not a transparent gradient-preserving optimization.

Parallel dependency depth Idealized dependent stage count, distinct from total work, memory traffic and measured wall-clock latency.

Global weighted reduction Sum all weighted valid-element loss numerators and divide by their global positive mass, not an unweighted mean of local means.

Gradient accumulation Add microbatch derivative contributions at fixed parameters before a single complete optimizer update.

Activation checkpointing Recompute omitted forward intermediates during backward under the same declared input, parameter and stochastic-state contract.

Stochastic replay Reproduce the required random state during recomputation; matching first-forward loss alone does not prove gradient equivalence.

Rank mass Sum of fixed valid-element weights on a rank; not necessarily its record count.

Tensor partition Disjoint operand/output coordinate ownership with a derived sum or concatenation contract.

Persistent shard Owned stored array range; does not exclude transient full materialization.

Artifact integrity Byte/schema agreement relative to a declared identity; not independently authenticated authorship.

Trusted pin Content identity obtained independently of the artifact being checked.

Inference reload Reconstruct and evaluate a declared model from artifact bytes without reusing its live training object.

Paired percentile interval Interval under the stated empirical resampling law for shared test cases; it does not include all training or selection uncertainty automatically.

Crossed bootstrap Independent resampling of seed and group axes, with the same drawn groups shared across seed rows for a declared equal-axis estimand.

Experimental unit Independently assigned or sampled unit under the design; duplicated windows do not create new units.

Benchmark envelope Shared records, target, legal access, metric and opportunity, while preserving architecture-native state and cost.

Linearization point Declared serialized instant at which an operation takes effect relative to other operations.

ABA reuse A familiar identifier returns after release; increasing generations distinguish old leases from the new owner.

Terminal reason Immutable lifecycle outcome, distinct from output draining and physical resource cleanup.

Quiescence No pending physical work may access the allocation; logical cancellation alone does not establish it.

Backpressure Refusal to launch additional work when the declared output reservation cannot fit; not silent output dropping.

Artifact-bound state Carry tied to model content, schema, owner and valid position; matching tensor shape alone does not make states interchangeable.

Allocation generation Temporal identity of a reusable slot. Reclamation advances it; live updates within that allocation use a separate version.

State pin Captured input snapshot whose completion or explicit discard must be acknowledged before its allocation can be reclaimed.

Retiring slot Released logical ownership with outstanding physical work; it remains occupied and cannot be ordinarily accessed or reallocated.

Inference snapshot Copied values that can be reconstructed independently; serialization to scalars does not preserve a differentiable training graph.

Logical scrubbing Reset of API-visible payload and metadata before reuse; not proof of physical secure erasure across all copies and platform memory.

Checkpoint envelope Bounded immutable data binding a format, complete interpretation context, exact consumed prefix and state values to an integrity digest. The digest does not authenticate its producer.

Continuation equivalence Original and restored complete state produce equal successive states and readouts on a common legal suffix under unchanged deterministic semantics and numerical policy.

Prefix key Content index for context and exact consumed tokens. Lookup must still validate the returned envelope; an indexing collision is not acceptance.

Copy-on-write refinement Physical sharing and private-on-write copying whose observations agree with independently owned logical copies.

Replay audit Recompute a prefix under a trusted model to check state consistency. This costs computation and does not authorize access.

Wire-byte budget Bound on retained encoded checkpoint bytes; excludes Python heap, allocator metadata, live branch storage and device buffers.

Lane map Mapping from a temporary batched row to its original owner, pool handle, generation, pin and output order; row number is not allocation identity.

Fenced scatter Per-lane pool installation through the registered snapshot pin and version; independent lane acceptance is not cross-request atomicity.

Request-local token clock Number of accepted consumed-token transitions for one request; differs from scheduler ticks and inactive-lane capacity.

Per-layer KV cache Ordered keys and values retained at each attention layer's normalized projection boundary for a causal prefix.

Offset causal mask Continuation query i after t cached tokens may read key j exactly when $j \leq t + i$; a rectangular upper-left triangle differs.

Cache content seal Integrity check over cache context and values; resealing by a trusted producer is not authentication or native replay proof.

Logical block and physical page A block names an ordered token interval; a page is its current storage allocation. A block table preserves the logical order even when physical addresses are scattered.

Prefix sharing Multiple live owners reference immutable prefix pages; an append to a shared partial tail must obtain a private copy before mutation.

Page credit Conservative capacity reserved for a request's declared future prefix growth; not a measurement of device memory or a fairness theorem.

Speculative verification A target model checks draft proposals under a declared acceptance/correction law. Only committed tokens may advance the authoritative continuation state.

Online softmax summary Running maximum, scaled normalizer and weighted value sum that permit exact attention reduction in real arithmetic without storing the entire score matrix; floating-point order still matters.

Capability routing Choose an implementation satisfying the requested semantics, artifact and resource constraints before comparing predicted costs. Here capability means an interface property, not general intelligence.

Effect trace Ordered observable reads, writes, state operations and tool actions; output-value equality alone does not establish equivalent execution.

Provenance DAG A directed acyclic record of source dependencies used to check derived-memory visibility. Revoking future reads does not erase copies already issued or prove neural unlearning.

Write-ahead log Recovery records made durable before their dependent data writes under a stated storage protocol; a dictionary crash model does not by itself supply durable transactions.

Serving revision Immutable artifact/configuration identity attached to admitted requests, distinct from the mutable pointer selecting future requests.

Ownership epoch Monotonically advancing authority version checked at a resource boundary; a metadata counter alone cannot fence a remote stale writer.

Goodput Rate of eligible useful completions satisfying declared quality and service constraints. The workload, time interval and denominator must be named.

Time to first token Elapsed time from the declared request origin to the first visible output; prefill's final prediction can produce that output without first consuming it as a new input token.

Reconciliation Resolve an already admitted action whose external result is uncertain before deciding whether another dispatch is safe. A local idempotency record is not an external exactly-once guarantee.

Acquisition–retention tradeoff Improvement on a new task can come at a cost to earlier tasks, while anchoring earlier parameters can restrict acquisition. The manuscript's assigned quadratic illustrates this, not a trained benchmark.

Evidence level Editorial classification distinguishing hypotheses, formal or implemented contracts, evaluated tasks and replication. A valid label or digest cannot establish the truth of its supporting observations.

Reproducibility capsule A declared manifest of source, configuration, data and environment inputs. Its completeness must be audited; a correct hash over an incomplete subset does not close the dependency boundary.

Bibliography

[1] National Human Genome Research Institute. Gene Expression. Source link.

[2] Bruce Alberts et al. An Overview of Gene Control. Molecular Biology of the Cell, 4th edition (2002). Source link.

[3] Scott F. Gilbert. Principles of Development: Genes and Development. Developmental Biology, 6th edition (2000). Source link.

[4] Scott F. Gilbert. Mechanisms for the environmental regulation of gene expression. Birth Defects Research C 72 (2004), 291–299. Source link.

[5] Noam Shazeer et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer (2017). Source link.

[6] Ashish Vaswani et al. *Attention Is All You Need*. 2017. Sections 3.2.1–3.2.3 inspected in the author's paper HTML. `https://arxiv.org/abs/1706.03762`.

[7] Barret Zoph and Quoc V. Le. Neural Architecture Search with Reinforcement Learning (2017). Source link.

[8] Sumit Gulwani, Alex Polozov, and Rishabh Singh. Program Synthesis (2017). Source link.

[9] John R. Koza. Research overview: genetic programming. Source link.

[10] A. Gu and T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. Primary preprint arXiv:2312.00752v2 (31 May 2024). `https://arxiv.org/abs/2312.00752v2`.

[11] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models (2023). Source link.

[12] Ian Walsh et al. DOME: recommendations for supervised machine learning validation in biology. Nature Methods 18 (2021), 1122–1127. Primary article.

[13] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. JMLR 11 (2010), 2079–2107. Primary article.

[14] Alfred Ferrer Florensa et al. SpanSeq: similarity-based sequence data splitting method for improved development and assessment of deep learning projects. NAR Genomics and Bioinformatics 6 (2024), lqae106. Primary preprint.

[15] E. Nguyen et al. *HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution*. 2023, v2. `https://arxiv.org/abs/2306.15794`.

[16] Z. Zhou et al. *DNABERT-2: Efficient Foundation Model and Benchmark For Multi-Species Genome*. arXiv:2306.15006, 2023; version 2, 18 March 2024. `https://arxiv.org/abs/2306.15006`.

[17] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht and Oriol Vinyals. Understanding deep learning requires rethinking generalization. ICLR (2017). Primary preprint.

[18] Xavier Bouthillier et al. Accounting for Variance in Machine Learning Benchmarks. MLSys (2021). Primary article.

[19] Chuan Guo, Geoff Pleiss, Yu Sun and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. PMLR 70 (2017), 1321–1330. Primary article.

[20] PyTorch. *CrossEntropyLoss*, version 2.10 documentation. Class-index targets, ignored indices, and reduction semantics. Accessed 13 September 2026. `https://docs.pytorch.org/docs/2.10/generated/torch.nn.CrossEntropyLoss.html`.

[21] PyTorch. *Reproducibility*, version 2.10 documentation. Randomness controls and the limits of cross-version and cross-platform reproducibility. Accessed 13 September 2026. `https://docs.pytorch.org/docs/2.10/notes/randomness.html`.

[22] PyTorch. *Saving and Loading Models*. Tutorial, accessed 13 September 2026. `https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html`. This is a moving tutorial, not a version-pinned 2.10 page; the subset used here was checked by executable tests on PyTorch 2.10.0.

[23] PyTorch 2.10 documentation. `torch.nn.Embedding`. `https://docs.pytorch.org/docs/2.10/generated/torch.nn.Embedding.html`. Relevant API description, shapes, padding behavior, initialization notes, and examples inspected. The local numerical demonstration uses PyTorch 2.10.0 and an explicitly assigned float64 table. A padding row's lookup gradient behavior is not a guarantee about all optimizer or downstream behavior.

[24] Sepp Hochreiter and Jürgen Schmidhuber (1997). Long Short-Term Memory. *Neural Computation* 9(8), 1735–1780. `https://www.bioinf.jku.at/publications/older/2604.pdf`. Introduction and early gradient-transport discussion inspected on 19 September 2026. Cited for motivation, not as the source of every feature in a modern forget-gate cell. No reported experimental performance is reproduced.

[25] PyTorch 2.10 documentation. `LSTMCell`. `https://docs.pytorch.org/docs/2.10/generated/torch.nn.LSTMCell.html`. Equations, input/state shapes, and parameter layout inspected. Our implementation follows the documented modern unprojected, no-peephole cell.

[26] PyTorch 2.10 documentation. `GRUCell`. `https://docs.pytorch.org/docs/2.10/generated/torch.nn.GRUCell.html`. Gate equations, state convention, and packed weight/bias shapes inspected. Local parity checks use PyTorch 2.10.0, CPU float64, and assigned weights.

[27] PyTorch 2.10 documentation. `GRU`, implementation note. `https://docs.pytorch.org/docs/2.10/generated/torch.nn.GRU.html`. The reset-before versus reset-after distinction was inspected explicitly. Our matrix counterexample and diagrams are original.

[28] G. E. Blelloch. Prefix Sums and Their Applications. In *Synthesis of Parallel Algorithms*, J. H. Reif (ed.), Morgan Kaufmann (author publication listing: 1993). `https://www.cs.cmu.edu/~guyb/papers/Ble93.pdf`.

[29] Albert Gu, Karan Goel, and Christopher Ré. Efficiently Modeling Long Sequences with Structured State Spaces. `https://arxiv.org/abs/2111.00396`. Abstract and bibliographic record inspected on 20 September 2026. Used only for the high-level S4 identity and structured-parameterization distinction, not a full technical reconstruction.

[30] J. T. H. Smith, A. Warrington and S. W. Linderman. Simplified State Space Layers for Sequence Modeling. Primary preprint arXiv:2208.04933v3 (3 March 2023). https://arxiv.org/abs/2208.04933v3.

[31] B. Peng et al. *RWKV: Reinventing RNNs for the Transformer Era*. arXiv:2305.13048 (2023). https://arxiv.org/abs/2305.13048.

[32] PyTorch contributors. *scaled_dot_product_attention*. Version 2.10 documentation. https://docs.pytorch.org/docs/2.10/generated/torch.nn.functional.scaled_dot_product_attention.html.

[33] R. Xiong et al. *On Layer Normalization in the Transformer Architecture*. PMLR 119, 10524–10533 (2020). https://proceedings.mlr.press/v119/xiong20b.html.

[34] PyTorch contributors. *LayerNorm*, version 2.10: feature statistics and affine parameters. https://docs.pytorch.org/docs/2.10/generated/torch.nn.LayerNorm.html.

[35] PyTorch contributors. *GELU*, version 2.10: defining function and approximation modes. https://docs.pytorch.org/docs/2.10/generated/torch.nn.GELU.html.

[36] PyTorch contributors. *TransformerEncoderLayer*, version 2.10: norm-first, activation, shape, and bias configuration. https://docs.pytorch.org/docs/2.10/generated/torch.nn.TransformerEncoderLayer.html.

[37] PyTorch contributors. *torch.nn.functional.dropout*, version 2.10: explicit training flag. https://docs.pytorch.org/docs/2.10/generated/torch.nn.functional.dropout.html.

[38] W. Fedus, B. Zoph, and N. Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. Journal of Machine Learning Research 23(120), 1–39 (2022), sections 2.1–2.2. https://www.jmlr.org/papers/v23/21-0998.html.

[39] PyTorch contributors. *torch.argmax*, version 2.10, first-maximal-index semantics. https://docs.pytorch.org/docs/2.10/generated/torch.argmax.html.

[40] H. G. Garcia and R. Phillips. *Quantitative dissection of the simple repression input–output function*. PNAS 108(29), 12173–12178 (2011). https://doi.org/10.1073/pnas.1015616108. Author-hosted article: https://garcialab.berkeley.edu/courses/papers/Garcia2011c.pdf.

[41] P. Prusinkiewicz and A. Lindenmayer. *The Algorithmic Beauty of Plants*. Springer (1990), electronic edition, Chapter 1, sections 1.1–1.2, pp. 1–5. https://www.algorithmicbotany.org/papers/abop/abop-ch1.pdf.

[42] A. Novikov and collaborators. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, version 1, 16 June 2025. https://arxiv.org/html/2506.13131v1.

[43] OpenXLA. *StableHLO Specification*. Execution and token-ordering discussion, current web specification. https://openxla.org/stablehlo/spec.

[44] LLVM Project. *SCF Dialect*. Introduction, condition, while and yield semantics. https://mlir.llvm.org/docs/Dialects/SCFDialect/.

[45] H. Grodin. *Amortized Analysis as a Cost-Aware Abstraction Function*. CMU CSD research exposition, 14 May 2025. https://www.cs.cmu.edu/~csd-phd-blog/2025/amortized-abstraction/.

[46] PyTorch contributors. *torch.profiler*, PyTorch 2.14 documentation. API descriptions for activities, profile_memory and with_flops. https://docs.pytorch.org/docs/2.14/profiler.html.

[47] Transformer Circuits research collaboration. *Circuit Tracing: Revealing Computational Graphs in Language Models*. Methods report, 2025. https://transformer-circuits.pub/2025/attribution-graphs/methods.html.

[48] Y. Schiff et al. *Caduceus: Bi-Directional Equivariant Long-Range DNA Sequence Modeling*. 2024, v1. https://arxiv.org/abs/2403.03234.

[49] G. Brixi et al. *Genome modelling and design across all domains of life with Evo 2*. Nature 652 (2026), 1349–1361; published 4 March 2026. https://doi.org/10.1038/s41586-026-10176-5.

[50] A. Gu, T. Dao, S. Ermon, A. Rudra and C. Ré. *HiPPO: Recurrent Memory with Optimal Polynomial Projections*. arXiv:2008.07669v2 (2020). https://arxiv.org/abs/2008.07669v2.

[51] A. Behrouz, P. Zhong and V. Mirrokni. *Titans: Learning to Memorize at Test Time*. arXiv:2501.00663v1 (submitted 31 December 2024). https://arxiv.org/html/2501.00663v1.

[52] S. Hochreiter and J. Schmidhuber. *Long Short-Term Memory*. Neural Computation 9 (1997), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735.

[53] Y. Ji, Z. Zhou, H. Liu and R. V. Davuluri. *DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome*. Bioinformatics 37 (2021), 2112–2120. https://doi.org/10.1093/bioinformatics/btab083. Author implementation: https://github.com/jerryji1993/DNABERT.

[54] H. Dalla-Torre et al. *Nucleotide Transformer: building and evaluating robust foundation models for human genomics*. Nature Methods 22 (2025). https://doi.org/10.1038/s41592-024-02523-z.

[55] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen and Y. Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864, 2021; version 5, 8 November 2023. https://arxiv.org/html/2104.09864v5.

[56] I. Beltagy, M. E. Peters and A. Cohan. *Longformer: The Long-Document Transformer*. arXiv:2004.05150, 2020; version 2, 2 December 2020. https://arxiv.org/abs/2004.05150.

[57] J. Dodge, S. Gururangan, D. Card, R. Schwartz and N. A. Smith. Show Your Work: Improved Reporting of Experimental Results. *EMNLP-IJCNLP*, 2185–2194 (2019). https://doi.org/10.18653/v1/D19-1224. ACL Anthology metadata/abstract inspected for validation/development reporting.

[58] X. Bouthillier et al. Accounting for Variance in Machine Learning Benchmarks. Primary preprint arXiv:2103.03098v1 (2021); abstract and author metadata. https://arxiv.org/abs/2103.03098v1.

[59] M. Dehghani, Y. Tay, A. A. Gritsenko, Z. Zhao, N. Houlsby, F. Diaz, D. Metzler and O. Vinyals. The Benchmark Lottery. arXiv:2107.07002v1 (14 July 2021). https://arxiv.org/abs/2107.07002v1. Primary abstract/version inspected for ranking sensitivity to task selection.

[60] T. Chen, B. Xu, C. Zhang and C. Guestrin. Training Deep Nets with Sublinear Memory Cost. Primary preprint arXiv:1604.06174v2 (22 April 2016). https://arxiv.org/abs/1604.06174v2.

[61] P. Micikevicius et al. Mixed Precision Training. Primary preprint arXiv:1710.03740v3 (15 February 2018). https://arxiv.org/abs/1710.03740v3.

[62] PyTorch contributors. torch.utils.checkpoint. Version 2.10 documentation. https://docs.pytorch.org/docs/2.10/checkpoint.html.

[63] S. Li et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. arXiv:2006.15704v1 (28 June 2020). https://arxiv.org/abs/2006.15704v1. Primary abstract and version metadata inspected.

[64] S. Rajbhandari, J. Rasley, O. Ruwase, Y. He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. arXiv:1910.02054v3 (13 May 2020). https://arxiv.org/html/1910.02054v3. Primary abstract and selected introduction/partition-stage overview inspected. No reported cluster result is reproduced.

[65] Y. Zhao et al. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. arXiv:2304.11277v2 (12 September 2023). https://arxiv.org/abs/2304.11277v2. Primary abstract and version metadata inspected.

[66] Safetensors project. Official README, Format section. https://github.com/safetensors/safetensors. Selected format specification inspected 6 October 2026: header, offsets, layout, scalar/empty tensors, and numerical-value boundary. This is a mutable documentation reference, not an installed-library validation. Our named raw-byte demo is not a safetensors implementation.

[67] X. Bouthillier et al. Accounting for Variance in Machine Learning Benchmarks. *Proceedings of Machine Learning and Systems* 3 (2021). https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html. Primary abstract inspected for variation from sampling, initialization and hyperparameter choices; no study replication is claimed.

[68] M. Bishop (ed.). *HTTP/3*, RFC 9114 (June 2022), Section 4.1.1, Request Cancellation and Rejection. https://www.rfc-editor.org/rfc/rfc9114.html#section-4.1.1. Primary standards section inspected. Protocol processing/retry distinctions provide context, not the definition of the original lifecycle oracle.

[69] R. Fielding, M. Nottingham and J. Reschke. *HTTP Semantics*, RFC 9110 (June 2022), Section 9.2.2, Idempotent Methods. https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2. Primary standards section inspected. Does not imply generation or external side effects are automatically idempotent.

[70] PyTorch contributors. `torch.clone`, version 2.10 API documentation. https://docs.pytorch.org/docs/2.10/generated/torch.clone.html. The complete API entry was inspected; cloning copies storage and preserves autograd relationships.

[71] PyTorch contributors. `Tensor.detach`, version 2.10 API documentation. https://docs.pytorch.org/docs/2.10/generated/torch.Tensor.detach.html.

The complete API entry was inspected; detachment removes graph relationships but shares storage.

[72] Evolutor book reference implementations: Chapter 26 restricted affine composition; Chapter 28 artifact and ownership contracts; Chapter 30 serialized request lifecycle, specifically `drafts/ch30/reference.py`. Chapter 31 imports that local runtime rather than duplicating its implementation. Original recurrence derivations, rational fixtures and vector plates are teaching artifacts; internal reference checks are not independent model validation.

[73] M. M. Michael and M. L. Scott. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. *PODC* (1996). Author-maintained corrected pseudocode: https://www.cs.rochester.edu/research/synchronization/pseudocode/queues.html. Access: the complete author page and allocator/reclamation assumptions were inspected. Pointer tagging and reclamation supply context; this chapter does not implement that queue or claim lock-free execution.

[74] Evolutor book, Chapters 30–31. Serialized request lifecycle and artifact-bound input-gated recurrence. The pool bridge imports the actual Chapter 31 reference, copies inference values and intentionally does not preserve the training graph. Original pool code, canary traces and vector plates are teaching artifacts, not production security, hardware-performance or durability certification.

[75] Python Software Foundation. *json: JSON encoder and decoder*, Python 3.13 documentation. Standard compliance, repeated object names, nonfinite numbers, hooks and implementation limits inspected. https://docs.python.org/3.13/library/json.html.

[76] PyTorch contributors. *Serialization semantics*, restricted-loading security section. PyTorch 2.14 documentation reached by stable redirect; limits inspected, not an end-to-end loader security audit. https://docs.pytorch.org/docs/2.14/notes/serialization.html.

[77] vLLM contributors. *Automatic Prefix Caching*, design documentation. Hash components, exact tokens, additional identities, cache salting, reference tracking and eviction sections inspected. https://docs.vllm.ai/en/stable/design/prefix_caching/.

[78] vLLM contributors. *Automatic Prefix Caching*, user documentation. Introduction, hybrid Mamba boundary/checkpoint conditions and limits inspected. https://docs.vllm.ai/en/stable/features/automatic_prefix_caching/.

[79] vLLM contributors. *Engine Arguments*. Recurrent cache mode, prefix match granularity, retention interval and shared-prefix checkpoint entries inspected. https://docs.vllm.ai/en/stable/configuration/engine_args/.

[80] NIST. *FIPS 180-4: Secure Hash Standard* (2015). Official publication metadata and abstract inspected for digest/integrity context; no independent cryptographic implementation audit claimed. https://doi.org/10.6028/NIST.FIPS.180-4.

[81] Gyeong-In Yu et al. *Orca: A Distributed Serving System for Transformer-Based Generative Models*. OSDI, 2022. Official abstract inspected. https://www.usenix.org/conference/osdi22/presentation/yu.

[82] NVIDIA. *CUDA C++ Best Practices Guide*, version 13.4, sections on profiling, reference comparison, numerical accuracy and coalesced access to global memory. Live

documentation inspected; no hardware performance validation performed. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html.

[83] Tri Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. 2022. Primary abstract only. https://arxiv.org/abs/2205.14135.

[84] Joshua Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. Primary abstract inspected. https://arxiv.org/abs/2305.13245.

[85] Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023. Primary HTML Sections 4.1–4.4 inspected. https://arxiv.org/html/2309.06180.

[86] Amey Agrawal et al. *Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve*. 2024. Primary HTML Section 4 inspected. https://arxiv.org/html/2403.02310v3.

[87] Ying Sheng et al. *Fairness in Serving Large Language Models*. OSDI, 2024. Official abstract inspected; VTC guarantees are not attributed to our scheduler. https://www.usenix.org/conference/osdi24/presentation/sheng.

[88] Guangxuan Xiao et al. *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models*. 2022/2023. Primary method and calibration text inspected. https://arxiv.org/html/2211.10438.

[89] Yaniv Leviathan, Matan Kalman and Yossi Matias. *Fast Inference from Transformers via Speculative Decoding*. 2023. Section 2 and Appendix A.1 inspected. https://arxiv.org/html/2211.17192v2.

[90] Charlie Chen et al. *Accelerating Large Language Model Decoding with Speculative Sampling*. 2023. Algorithm 2 and Section 4 inspected. https://arxiv.org/html/2302.01318v1.

[91] Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. 2023. Primary abstract inspected. https://arxiv.org/abs/2307.08691.

[92] Evolutor companion sources. Chapters 14–16, 20, 23 and 30–35. Typed semantics, memory distinctions, lifecycle and actual native/attention reference implementations. Repository-local primary sources, inspected at the hashes recorded in the chapter-local review.

[93] C. Packer et al. *MemGPT: Towards LLMs as Operating Systems*. 2023, v2. https://arxiv.org/abs/2310.08560.

[94] Akari Asai et al. *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. 2023. Primary abstract inspected for on-demand retrieval. https://arxiv.org/abs/2310.11511.

[95] MLIR contributors. *MLIR Language Reference*. Official documentation, high-level structure and operations/values inspected 17 October 2026. https://mlir.llvm.org/docs/LangRef/.

[96] MLIR contributors. *Side Effects & Speculation*. Official rationale, overview, categorization and modeling sections inspected 17 October 2026. https://mlir.llvm.org/docs/Rationale/SideEffectsAndSpeculation/.

[97] Tianqi Chen et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. 2018. Primary abstract inspected; no benchmark reproduction. https://arxiv.org/abs/1802.04799.

[98] Goetz Graefe. *Volcano—An Extensible and Parallel Query Evaluation System*. IEEE TKDE 6(1), 120–135, 1994. Primary interface/buffer/exchange passages. https://people.eecs.berkeley.edu/~prabal/teaching/resources/eecs582/graefe94volcano.pdf.

[99] Goetz Graefe. *The Cascades Framework for Query Optimization*. IEEE Data Engineering Bulletin 18(3), 19–29, 1995. Primary paper, optimization tasks and exploration sections. https://15721.courses.cs.cmu.edu/spring2019/papers/22-optimizer1/graefe-ieee1995.pdf.

[100] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh and Peter Schwarz. *ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging*. ACM TODS 17(1), 94–162, 1992. Publisher record and indexed primary excerpt; full-PDF access incomplete. https://doi.org/10.1145/128765.128770.

[101] James Kirkpatrick et al. *Overcoming catastrophic forgetting in neural networks*. PNAS 114(13), 3521–3526, 2017. Primary method HTML, approximate posterior and Fisher-weighted penalty, Equations 1–3. https://arxiv.org/html/1612.00796v2. https://doi.org/10.1073/pnas.1611835114.

[102] SLSA contributors. *SLSA v1.2: Provenance*. Official approved definition inspected. https://slsa.dev/spec/v1.2/provenance.

[103] OpenTelemetry contributors. *Specification: Overview*. Signal, span/context, metric and log overview inspected; semantic-convention terminology only, no exhaustive convention audit. https://opentelemetry.io/docs/specs/otel/overview/.

[104] Kubernetes contributors. *Liveness, Readiness, and Startup Probes*. Official conceptual documentation, probe purposes inspected. https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/.

[105] Scott Rose, Oliver Borchert, Stu Mitchell and Sean Connelly. *Zero Trust Architecture*. NIST SP 800-207, 2020. Official publication overview inspected for resource authentication/authorization and network-location trust boundary. https://csrc.nist.gov/pubs/sp/800/207/final.

[106] OWASP Gen AI Security Project. *LLM01:2025 Prompt Injection*. Types and mitigation sections inspected. https://genai.owasp.org/llmrisk/llm01-prompt-injection/.

[107] OWASP Gen AI Security Project. *LLM07:2025 System Prompt Leakage*. Mitigation/privilege-boundary guidance inspected. https://genai.owasp.org/llmrisk/llm072025-system-prompt-leakage/.

[108] Yinmin Zhong et al. *DistServe*. OSDI 2024. Selected mechanism/metric passages inspected, not reproduced. https://www.usenix.org/system/files/osdi24-zhong-yinmin.pdf.

[109] Samuel Williams, Andrew Waterman and David Patterson. *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. UCB/EECS-2008-134, 2008. Primary author technical report, Section 3 inspected; predecessor of the 2009 CACM publication. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.pdf.

[110] PyTorch contributors. *torch.profiler*, version 2.10. API, scheduling and shape/stack overhead guidance inspected. https://docs.pytorch.org/docs/2.10/profiler.html.

[111] NVIDIA. *Nsight Systems User Guide*. Timeline, focused collection/NVTX and overhead guidance inspected. https://docs.nvidia.com/nsight-systems/UserGuide/index.html.

[112] NVIDIA. *Nsight Compute Profiling Guide*. Kernel-profiling and counter/replay overview inspected. https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html.

[113] Apple. *Metal Developer Tools*. Tools, dependencies and memory-inspection overview inspected; not an API benchmark. https://developer.apple.com/metal/tools/.

[114] Khronos. *Vulkan Guide: Synchronization*. Synchronization, validation and pipeline-barrier overview inspected. https://docs.vulkan.org/guide/latest/synchronization.html.

[115] National Human Genome Research Institute. *Promoter*, Talking Glossary. https://www.genome.gov/genetics-glossary/Promoter.

[116] H. Feng et al. *Benchmarking DNA foundation models for genomic and genetic tasks*. Nature Communications (2025). https://doi.org/10.1038/s41467-025-65823-8.

[117] J. Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024; arXiv v3. https://arxiv.org/abs/2405.15793.

[118] J. Kirkpatrick et al. *Overcoming catastrophic forgetting in neural networks*. PNAS 114 (2017); arXiv v2. https://arxiv.org/abs/1612.00796.

[119] V. Lomonaco et al. *Avalanche: an End-to-End Library for Continual Learning*. 2021. https://arxiv.org/abs/2104.00405.

[120] M. Jaderberg et al. *Population Based Training of Neural Networks*. 2017, v2. https://arxiv.org/abs/1711.09846.

[121] M. R. Morris et al. *Levels of AGI: Operationalizing Progress on the Path to AGI*. 2023; arXiv v2. https://arxiv.org/abs/2311.02462.

[122] ARC Prize Foundation. *ARC-AGI-2*. Official benchmark description, accessed 17 October 2026. https://arcprize.org/arc-agi/2.

[123] ARC Prize Foundation. *ARC-AGI-3*. Official benchmark description, accessed 17 October 2026. https://arcprize.org/arc-agi/3.

[124] *Evolutor*, chapter reference sources and local test suites: Chapters 23, 31, 35 and 48–52. Source identities are recorded in the generated Chapter 52 reference manifest and chapter-local review records.

[125] *Evolutor*, Chapter 25 shared experimental contract, Chapter 29 comparative evidence, and Chapters 48–51 task and evidence protocols. Earlier primary references remain with those chapters; this synthesis introduces no external empirical performance result.

[126] *DNA Computing: Molecular mechanisms, algorithms and evidence*. Companion volume, prepublication review edition (October 2026). https://github.com/hermonai/dna-computing-book.

[127] S. Raschka. *Build a Large Language Model (From Scratch)*: author companion overview and chapter map. https://www.sebastianraschka.com/llms-from-scratch/.

[128] L. M. Adleman. Molecular computation of solutions to combinatorial problems. *Science* 266 (1994), 1021–1024. https://doi.org/10.1126/science.7973651.

[129] D. Y. Zhang and E. Winfree. Control of DNA strand displacement kinetics using toehold exchange. *Journal of the American Chemical Society* 131 (2009), 17303–17314. https://doi.org/10.1021/ja906987s.

[130] D. Soloveichik, G. Seelig and E. Winfree. DNA as a universal substrate for chemical kinetics. *Proceedings of the National Academy of Sciences USA* 107 (2010), 5393–5398. https://doi.org/10.1073/pnas.0909380107.

[131] L. Qian and E. Winfree. Scaling up digital circuit computation with DNA strand displacement cascades. *Science* 332 (2011), 1196–1201. https://doi.org/10.1126/science.1200520.

[132] L. Qian, E. Winfree and J. Bruck. Neural network computation with DNA strand displacement cascades. *Nature* 475 (2011), 368–372. https://doi.org/10.1038/nature10262.

[133] K. M. Cherry and L. Qian. Scaling up molecular pattern recognition with DNA-based winner-take-all neural networks. *Nature* 559 (2018), 370–376. https://doi.org/10.1038/s41586-018-0289-6.

[134] K. M. Cherry and L. Qian. Supervised learning in DNA neural networks. *Nature* 645 (2025), 639–647. https://doi.org/10.1038/s41586-025-09479-w.

[135] T. Song and L. Qian. Heat-rechargeable computation in DNA logic circuits and neural networks. *Nature* 646 (2025), 315–322. https://doi.org/10.1038/s41586-025-09570-2.

[136] P. W. K. Rothemund. Folding DNA to create nanoscale shapes and patterns. *Nature* 440 (2006), 297–302. https://doi.org/10.1038/nature04586.

[137] D. Woods, D. Doty, C. Myhrvold, J. Hui, F. Zhou, P. Yin and E. Winfree. Diverse and robust molecular algorithms using reprogrammable DNA self-assembly. *Nature* 567 (2019), 366–372. https://doi.org/10.1038/s41586-019-1014-9.

[138] C. G. Evans, J. O'Brien, E. Winfree and A. Murugan. Pattern recognition in the nucleation kinetics of non-equilibrium self-assembly. *Nature* 625 (2024), 500–507. https://doi.org/10.1038/s41586-023-06890-z.

[139] G. M. Church, Y. Gao and S. Kosuri. Next-generation digital information storage in DNA. *Science* 337 (2012), 1628. https://doi.org/10.1126/science.1226355.

[140] N. Goldman, P. Bertone, S. Chen, C. Dessimoz, E. M. LeProust, B. Sipos and E. Birney. Towards practical, high-capacity, low-maintenance information storage in synthesized DNA. *Nature* 494 (2013), 77–80. https://doi.org/10.1038/nature11875.

[141] Y. Erlich and D. Zielinski. DNA Fountain enables a robust and efficient storage architecture. *Science* 355 (2017), 950–954. https://doi.org/10.1126/science.aaj2038.

[142] Ž. Avsec et al. Effective gene expression prediction from sequence by integrating long-range interactions. *Nature Methods* 18 (2021), 1196–1203. https://doi.org/10.1038/s41592-021-01252-x.

[143] E. Nguyen, M. Poli et al. Sequence modeling and design from molecular to genome scale with Evo. *Science* 386 (2024), eado9336. https://doi.org/10.1126/science.ado9336.

[144] Ž. Avsec, N. Latysheva, J. Cheng et al. Advancing regulatory variant effect prediction with AlphaGenome. *Nature* 649 (2026), 1206–1218. https://doi.org/10.1038/s41586-025-10014-0.

[145] UCSC Genome Browser. *Data file formats* (BED definition: first base numbered 0; `chromEnd` not included). Accessed 18 October 2026. https://genome.ucsc.edu/FAQ/FAQformat.html.

[146] SAMtools/hts-specs. *The Variant Call Format specification, VCFv4.3* (POS: the 1st base has position 1). Accessed 18 October 2026. https://samtools.github.io/hts-specs/VCFv4.3.pdf.

[147] Sequence Ontology. *Generic Feature Format version 3* (*GFF3*) (one-based start/end, start ≤ end). Accessed 18 October 2026. https://github.com/The-Sequence-Ontology/Specifications/blob/master/gff3.md.

Index

ABA problem, 429
ABA reuse, 358
ablation, 27, 303
Acceptance probability, 452
acquisition, 557
acquisition area, 569
Activation checkpointing, 324
active parameters, 138
adaptation
  structural versus parametric, 494
Adapted structure, 4
adapter
  semantic parity, 301
adapters, 561
addressable record, 469
admission, 507
Admission control, 439
affine prefix summary, 309
agent-computer interface, 553
aliasing, 176, 235
Allocation
  preflight, 431
Amdahl model, 534
amortized analysis, 194
amortized cost, 198
Anti-homomorphism, 250
arithmetic
  exact integers, 479
Arithmetic intensity, 535, 538
arithmetic intensity, 196, 198
artifact
  immutable lineage, 494
Artifact binding, 462, 467
artifact capsule, 579
artifact identity, 565
Artifact integrity, 338
artifact integrity, 505
atomic state transition, 177
Attention
  paged traversal, 454
  perturbation bound, 443
  tiled reduction, 452
attention, 105
attention complexity, 113
attention gradient, 109
audit, 520
authentication, 515
authority
  memory firewall, 474
  release roles, 498
authorization
  tool calls, 464
Autograd
  checkpoint boundary, 394
automatic differentiation, 156
autonomy, 567
autoregression, 19
Autoregressive distribution, 449
auxiliary loss, 136
average precision, 545

backpressure, 529, 549
backward transfer, 560
Baldwinian, 168
baseline controls, 544, 576
batch isolation, 519
benchmark contamination, 568
benchmark record, 531
BF16, 442
Bidirectionality, 217, 250
biological analogy, 147
biological validation, 546
bits per base, 28
block table, 428
bold term, 594
Bonus token, 452
bootstrap
  cluster, 298
Brier score, 545
budget accounting, 576
budget axis, 569
budget parity, 561
budget reservation, 551
buffer manager, 488

cache content seal, 417
cache equivalence, 122
cache isolation, 516
cache parity, 112
Calibration, 444
canary, 506
  statistical limits, 497
cancellation, 526
canonical encoding, 504
canonicalization, 79
Capability, 467, 515, 522
capability
  evidence, 565

capability science, 575
capability vector, 567
capacity, 134
Carried state, 238
Cascades, 487
catastrophic forgetting, 557
causal attention, 107
causality, 23
  suffix intervention, 578
checkpoint, 63, 189
  prefix digest, 551
Checkpoint approximation error, 390
Checkpoint envelope, 390
Checkpoint spacing, 397
chunk equivalence, 134
Chunked prefill, 440
claim ledger, 578
clock synchronization, 529
closed-form oracle, 558
closed-loop load, 531
cluster bootstrap, 546
cohort, 134
compatibility, 506
compiler, 153
  intermediate representation, 477
  legality, 178
  lowering, 481
  verification, 478
compiler legality, 519
compression
  non-injectivity, 471
computational gene, 174
Conditional proposal, 449
configuration, 181, 189
conflict component, 22
consistency
  memory store, 474
constant folding, 479
Consumed token, 438
content digest, 579
content-bound artifact, 366
continual learning
  adaptation layers, 557
continuation
  chunk parity, 462
Continuation equivalence, 389
Continuous batching, 440
control plane, 523
copy-on-write, 430
Copy-on-write refinement, 396
correctness parity, 531
cost model, 190, 198
  hard constraints, 486
  routing, 462
counterexample, 577
Counterfactual execution, 206
CPU backend, 537
crash recovery, 553
crash-cut testing, 490
Critical path, 534, 538
cross entropy, 28
  uniform target, 577
crossed bootstrap, 347
CUDA backend, 537

data leakage, 22
  dependency closure, 293
  genomic overlap, 542
Data lineage, 69
data plane, 523
dead-code elimination, 479
Decode-first scheduling, 440
dense oracle, 132
deny by default, 515
dependency closure, 579
Development, 8
developmental encoding, 151
device timing, 537
discrete decisions, 133
dispatch, 131
DNA-addressed state, 217
DOGMA, 12
  hypothesis, 571
  state movement, 527
draining, 507
drift monitoring, 500
durable event, 469

e-fold time, 229
Effect, 485
  declaration, 478
effect
  uncertain completion, 464
Effect boundary, 467
effects, 183
elastic weight consolidation, 496, 558
elitist search, 165
Emitted token, 438
entropy control, 23
equivariance, 213, 241
Error budget, 509, 512
estimand, 292
evaluation
  conjunctive gates, 495
  held-out data, 496

event ordering, 549
Eviction, 433
evidence level, 565
Evolutor, 11
exact propagation, 144
Exact replay, 69
exactly-once effects, 526
execution systems, 575
execution trace, 465
experiment ledger, 578
experimental unit, 347
expert parameters, 131
expiry, 469, 515
Expressed computation, 4
expression, 142

Failure atomicity, 431
failure resolution, 578
failure taxonomy, 577
Fairness, 441
fallback, 462
Falsification criterion, 262
feed-forward sublayer, 120
fenced scatter, 406
fencing, 524
Fencing epoch, 530
finite intervention, 23
finite observational underdetermination, 570
FlashAttention, 455
forgetting
  issued contexts, 474
  peak convention, 560
forward transfer, 560
FP16, 442
Fragmentation
  internal, 434
Fresh-owner restore, 393
fuel, 185
Functional ablation, 227

gated recurrent unit, 85
gather and scatter, 131
GELU, 120
Gene expression, 5
generality, 567
generational handle, 377
generational reference, 429
genome, 174
genomics
  task contract, 541
genotype, 163
genotype and phenotype, 151
golden case, 60
Goodput, 532, 538
gradient accumulation, 320
gradient descent, 163
Gradient parity, 227

held-out evaluation, 568
Hermon DNA, 12, 265
  hypothesis, 571
  KV movement, 527
Hermon DNA Engine, 271
Hoeffding bound, 570
homology, 542
human assistance, 554

idempotency
  recovery, 551
idempotency key, 551
identifiability, 147
identity bypass, 173
immutable payloads, 549
independent review, 582
index
  authority, 488
  stale entries, 472
inference snapshot, 383
Information capacity, 238
inheritance, 557
  parameter map, 167
Input-conditioned coefficient, 227
input-gated diagonal recurrence, 366
integration gate, 582
Inter-token latency, 441
interactive evaluation, 568
intermediate representation, 477
intervention, 147
IR schema version, 483
ITL, 532

Jensen inequality, 577

k-mer, 71
Kernel fusion, 455
KV cache, 110, 550
  paged layout, 428
  precision, 444
  tentative fork, 451

Lamarckian, 168
lane map, 405
layer ownership, 122
LayerNorm, 117
learning rate
  curvature bound, 558

lease, 554
Lineage, 501
linear time-invariant, 97
Linearization point, 530
linearization point, 357
liveness, 193, 198, 507
  allocation overlap, 482
load balancing, 136
Locality, 229
log sequence number, 489
logical and physical plans, 486
Logical multiplicity, 434
Logical ordinal, 428
Logit margin, 444
long context, 544
long short-term memory, 84
Longest-prefix lookup, 394
loop invariant, 184
Lowering, 485
LRU eviction, 395

Masked attention
  empty row, 452
masked softmax, 108
masked state transition, 402
materialization, 489
Matthews correlation coefficient, 545
Mechanism intervention, 262
mechanism transplant, 303
Mechanistic faithfulness, 207
memory, 146
  native versus durable, 469
  payload accounting, 471
  tiers, 550
  workspace, 482
memory safety firewall, 474
Memory traffic
  model, 455
memory traffic, 535
Metal backend, 537
migration, 524
mixture of experts, 129, 191
model semantics, 575
Model-owned parameter, 262
Modified rejection sampling, 448
Momentum buffer, 69
multi-head attention, 112
multiple testing, 168
multiply-accumulate, 191
multiply-accumulate count, 138

namespace, 516
native state, 550
Negative control, 69
negative control, 578
negative evidence, 577
next-token objective, 124
non-identifiability, 225
nonfinite metrics, 495
Noninterference, 516, 522

observability, 509, 554
observation
  compiler equivalence, 478
Observational equivalence, 189
Offset causal mask, 454
offset causal mask, 111
one-token decode, 420
Online softmax, 452
opaque handle, 462
open-loop load, 531
operational semantics, 181
optimization legality, 186
optimizer
  statistics, 491
oracle
  recursive expressions, 480
Output action, 250
ownership epoch, 524

padding, 198
Padding identity, 217
padding mask, 124
page LSN, 489
Page reservation, 439
Page table, 428
paired comparison, 297
paired percentile interval, 344
paired runs, 534
parallel rewriting, 152
parameter learning, 557
parameter sharing, 156
Parameter tying, 250
Pareto comparison, 348
Pareto dominance, 562
patch validation, 553
per-layer KV cache, 417
percentile, 509, 534
perplexity, 28
persistent agents
  event log, 549
Physical alternative, 492
Physical utilization, 434
pin
  physical lifetime, 488
placement, 523

planner, 460
poisoning, 562
population search, 561
  budgets, 496
population variance, 117
positional encoding, 119
potential function, 194
pre-normalization, 116, 137
Precision policy, 442
Prefill, 418
prefill/decode disaggregation, 528
Prefix cache, 433
Prefix key, 394
prefix scan, 99
prefix-dependent generation, 487
Principal, 522
profiling, 537
promoter, 142
Promotion, 501
  revision guard, 498
prompt injection, 519
provenance, 504
  ancestor availability, 469
Provenance DAG, 476
publication authorization, 582

Quality gate, 444
Quantization
  INT8, 442
query, key, and value, 105
Quiescent snapshot, 392
quota, 518

random seed
  reporting, 299
rank mass, 331
ranking function, 184, 189
read-write dependency, 186
Readiness, 507, 512
reconciliation, 551
record boundary, 107, 124
record identity, 469
Record reset, 217
Recovery, 492
recovery versus replay, 491
recurrent neural network, 82
recursive module, 153
redaction, 520
redo
  idempotence, 490
Redundancy, 206
Reference conservation, 429
Reference count, 429
regulation, 142, 174
Rehydration, 476
  budgets, 473
release bundle, 504
Release identity, 512
Repeated-prefix metadata, 397
replay, 526, 553
  mixture objective, 561
  orchestration, 465
Replay audit, 391
replication, 565
reproducibility
  capsule, 579
request-local token clock, 404
Request-owned state, 262
research synthesis
  evidence boundaries, 575
reservation, 518, 524
reset-before-update, 311
residual connection, 153
Residual correction, 448
residual stream, 116, 137
resource accounting, 157
resource budget, 300
resource semantics, 190
Retention, 238
retrieval
  lexical ranking, 472
  provenance, 464
Reverse complement, 250
reverse complement, 22, 212, 543
revocation, 515
Rollback, 501
  eligibility, 499
rollback, 506, 562
roofline, 196, 535
router, 129, 460
routing, 523
  capability, 462
routing gradient, 133
row-parallel linear, 334

Sampling
  random streams, 449
sampling unit, 546
sandbox, 553
Saturation, 442
scheduler tick, 404
scheduling, 193
  effect dependencies, 481
Scored target, 69
search

budget, 165
selection bias, 296
self-improvement, 571
semantic axis, 172
Semantic boundary, 388
session binding, 515
session identity, 551
shadow evaluation, 497
shadow execution, 506
Shapley allocation, 202
side channel, 519
single assignment, 153
single static assignment, 175, 460, 477
skill acquisition, 569
SLO, 509
small step, 189
small-step relation, 183
snapshot, 524
snapshot semantics, 176
Softmax
sensitivity, 443
softmax scaling, 106
Speculative decoding, 448
rollback, 451
split closure, 542
spurious feature, 163
SSA, 485
stability and plasticity, 496
staged decode, 373
Staging reservation, 530
state compatibility, 499
state fingerprint, 499
State ownership, 316
state ownership, 147, 175, 462
state pin, 380
state schema, 506
State semantic version, 390
state update, 162
State-path Jacobian, 227
statistical weight, 143
step response, 146
step workspace, 383
Stored structure, 4
strand equivariance, 543
structural adaptation, 557
structural intervention, 200
structural proposal, 162
summary, 469
survivor bias, 532
symbolic operator, 464
synchronization, 537
Synergy, 207
synthetic training, 125

Tail-sum identity, 452
target alignment, 19
telemetry cardinality, 509
Tenant namespace, 395, 433
tensor shapes, 106
termination, 185
theorem ledger, 578
threat model, 514
Throughput, 441
Time to first token, 441
Token mean, 69
token steps, 87
tokenization
base resolution, 544
Tombstone, 476
tool authorization, 519
top-1 selection, 129
topology, 529
Total variation, 448
TPOT, 532
trace
replay, 177
source identity, 483
trace context, 509
trace replay, 187
transactions
external effects, 489
transcription, 143
transfer model, 528
translation, 143
transplant test, 27
truncated backpropagation, 90
trusted pin, 338, 505
TTFT, 532
turnover, 143
type checking, 154, 460
type preservation, 175

uncertainty, 534
unknown bases, 543
Update clock, 238
UTF-8 payload bytes, 473

validation
adaptive reuse, 168
variant effect score, 546
vector-Jacobian product, 118
Volcano, 487
Vulkan backend, 537

WAL, 492

warmup, 531
weighted reduction, 320
working set, 473
write-ahead logging, 489

zero trust, 514
zero-failure bound, 570
zero-order hold, 95